

从 C++ 到机器执行

第一册：底层原理、汇编接口与可信性能测量

CPU Performance Study

2026 年 6 月 16 日

序言

这本书是整个系列的第一册。它的目标不是让读者“看懂一些汇编”，而是建立一套能继续学习系统、编译器、体系结构和性能工程的基础模型：

C/C++ 源码语义

- > 编译、汇编、链接
- > x86-64 汇编
- > 指令、寄存器和内存
- > ABI、调用约定和二进制接口
- > 工具观察和可信性能测量

本书面向刚学完 C 语言和 C++ 基础、希望进入现代计算机体系结构、操作系统、编译器和高性能计算的读者。它会从“程序如何运行”“CPU 如何执行指令”“内存中的字节如何解释”这些基础问题讲起，再进入 x86-64 汇编、System V AMD64 ABI、C++ 与汇编互操作，以及性能实验的基本方法。

第二册再展开现代 CPU 微架构、cache/TLB、SIMD、HPC kernel、GEMM 和 AI CPU 算子优化。第一册先把地基打稳：读者必须能从一段 C/C++ 代码追到汇编和运行时证据，能解释寄存器、内存、栈、调用约定和测量误差，才能进入后续的优化主题。

本书写作遵循一个原则：任何性能结论都必须能被解释、观察和验证。只看源码不够，只看汇编不够，只看运行时间也不够。真正可靠的性能工程需要把抽象、实现、工具和实验连接起来。

本书不是速查手册

本书不是指令表，不是编译参数列表，也不是算法模板合集。每个核心机制都会按以下问题展开：

- 它为什么存在？
- 如果没有它，会遇到什么问题？
- 它给程序员提供了什么抽象？
- 底层大致如何实现？
- C/C++ 代码如何映射到这个机制？
- 如何用工具观察它？
- 它对性能有什么影响？
- 初学者最容易误解什么？

学习要求

读者需要已经写过基本 C/C++ 程序，理解变量、函数、指针、数组、结构体、类、模板的基础语法。除此之外，本书会补齐学习汇编、CPU 执行模型、二进制接口和性能测量所需的底层知识。

每一章都包含实验和作业。只读正文不算完成。你需要运行命令、观察输出、写代码、看汇编、做 benchmark，并写报告。

背景知识与学习路线

本册的读者不需要已经学过完整的计算机体系结构、操作系统、编译原理或高性能计算。但你必须愿意把一个程序拆开来：源码是一层，编译器是一层，汇编和 ABI 是一层，CPU 执行是一层，测量方法又是一层。本册要做的事，就是把这些层按经典教材的方式连成一条能推导、能观察、能复现实验的路径。

已经需要具备的基础

开始学习本册前，最好已经具备下面这些 C/C++ 基础。它们不需要达到专家水平，但不能完全陌生：

- 能写、编译、运行一个多文件 C/C++ 程序。
- 理解变量、表达式、函数、作用域、头文件和源文件。
- 理解数组、指针、引用、结构体、类和基本对象生命周期。
- 知道整数类型、浮点类型、`sizeof`、`alignof` 的基本用法。
- 能使用命令行运行程序、查看文件、重定向输出。
- 能读懂简单循环、条件分支、函数调用和基础标准库容器。

如果这些基础还不稳定，本册仍然可以读，但要放慢速度。每遇到一个 C/C++ 语法点，都要先写最小程序验证，而不是直接跳到汇编或性能结论。

本册要补齐的五类背景

一、底层硬件背景

硬件背景不是要求你设计 CPU，而是要求你知道程序最终运行在真实机器上。第一册会建立下面这些最小硬件模型：

- bit、byte、word、地址、寄存器、内存和指令。
- CPU 是一个会读取指令、更新状态的机器，不是直接理解 C++ 的解释器。
- ISA 是软件和硬件之间的合同，微架构是实现这个合同的具体方式。
- load/store、分支、函数调用、栈和返回地址都是可观察的机器行为。
- cache、TLB、分支预测、流水线和乱序执行先作为性能直觉出现，深入模型留到第二册。

本册对硬件知识的要求是“能解释现象”，不是“能背参数”。例如，看到一个数组求和函数，读者应能说明它为什么需要地址计算、load、add、循环控制和返回值寄存器；至于某代 CPU 的端口分配和具体延迟，属于后续进阶内容。

二、软件系统背景

软件背景的核心是：程序不是从源码直接运行。它会经过工具链和操作系统提供的多层机制：

- 预处理、编译、优化、汇编、链接、加载和运行时启动。
- 翻译单元、目标文件、符号、重定位、可执行文件和共享库。
- 进程、虚拟地址空间、栈、堆、全局区、只读区和动态映射。
- `main` 不是进程第一条指令，运行时会在调用 `main` 前完成初始化工作。
- 编译器优化可能删除、移动、内联或改写代码，只要可观察行为符合语言规则。

这类知识对性能学习非常关键。很多错误 benchmark 不是算法慢，而是测到了初始化、动态链接、I/O、随机数生成、内存分配、页错误或被编译器删除后的空循环。

三、算法和数据背景

本册不会讲复杂算法设计，但会训练你用底层视角重新看基础算法和数据：

- 一个循环不仅是语法结构，也是控制流、归纳变量、地址计算和内存访问模式。

- 一个数组不仅是容器，也是连续对象序列和一组可推导的字节地址。
- 一个结构体不仅是字段集合，也是 offset、padding、alignment 和 ABI 约束。
- 一个函数不仅是代码块，也是调用约定、参数传递、返回值位置和寄存器保存规则。
- 一个性能数字不仅是结果，也是输入规模、分布、热路径、时间源、统计口径和环境共同产生的证据。

第一册的算法训练目标很朴素：能把 `sum`、`copy`、`dot`、条件计数、简单查表、结构体数组访问这类基础 kernel 解释清楚。后续卷中的 SIMD、GEMM、attention 和算子融合，都会建立在这些基础 kernel 的可推导模型上。

四、汇编和 ABI 背景

汇编不是单独背指令表。它是连接 C/C++、机器码、寄存器、内存和 ABI 的中间层。本册学习汇编时只抓住几个稳定问题：

- 参数从哪里进来，返回值从哪里出去。
- 每条指令读哪些寄存器或内存，写哪些寄存器或内存。
- 地址表达式如何由 base、index、scale、displacement 组成。
- 条件跳转依赖哪些 flags，`cmp` 和 `test` 为什么不保存普通结果。
- `call` 和 `ret` 如何使用栈保存和恢复控制流。
- caller-saved、callee-saved、stack alignment 和 red zone 这些 ABI 规则为什么存在。

学完第一册后，读者不需要能手写大型汇编库，但必须能读懂普通优化汇编，能从 C++ 函数签名推导 ABI 位置，能写小型手写汇编函数，并能用 `nm`、`readelf`、`objdump`、GDB 证明它真的按预期工作。

五、实验和测量背景

性能学习必须从一开始就建立实验纪律。本册把 benchmark 当成受控实验，而不是随手计时：

- 先证明正确性，再讨论速度。
- 明确 workload、kernel、timed region 和 validation 的边界。
- 保存原始样本，不只保存一个汇总数字。
- 报告编译器、编译参数、CPU、操作系统、输入规模和运行命令。
- 用反汇编确认测到的代码确实存在。
- 说明结论适用范围，不能从一个输入规模推断所有场景。

本册不要求每个实验都达到论文级严谨，但要求每个结论都有证据链。一个合格结论的形态应当是：

在这台机器、这个编译器、这些参数、这个输入规模下，
这个 kernel 的热路径生成了这些核心指令，
重复测量得到这些样本，
因此可以在这些限制内说它表现为某种趋势。

第一册的知识链

第一册的学习顺序不是随意安排的。每一部分都为下一部分提供必要背景：

第 1-5 章：

建立源码、工具链、机器执行、数据表示和调试工具的地基。

第 6-10 章：

用 x86-64 汇编和 System V AMD64 ABI 解释真实二进制接口。

第 11-12 章：

把前面的源码、汇编、机器行为变成可复现的性能实验。

不要跳过第一部分直接读汇编。没有源码到进程、CPU 状态、数据表示和工具链背景，汇编很

容易变成指令速查。也不要跳过汇编直接做 benchmark。没有二进制审计能力，你无法判断自己测到的是源码里想象的代码，还是编译器改写后的另一段代码。

每个阶段的最低产出

学完第一部分后

你应该能做到：

- 画出从 `.cpp` 到正在运行进程的粗粒度流程。
- 解释 CPU 执行的是机器码字节，不是 C++ 源码。
- 解释寄存器、内存、`rip`、`rsp` 和 `flags` 的作用。
- 从 `a[i]` 推导出字节地址公式。
- 用 `gdb`、`objdump`、`readelf`、`nm` 做基础观察。

学完第二部分后

你应该能做到：

- 读懂普通 x86-64 Intel 语法反汇编。
- 从 C++ 函数签名推导前几个参数和返回值的位置。
- 解释整数运算、地址计算、条件跳转、循环、函数调用和 `switch` 的常见汇编形态。
- 判断一个手写汇编函数是否破坏 ABI。
- 写小型 C++ 与汇编互操作实验，并用工具证明符号、重定位和调用路径。

学完第三部分后

你应该能做到：

- 把临时代码升级成有输入配置、重复测量、CSV 输出和环境记录的 benchmark。
- 区分 `setup`、`warmup`、`timed region`、`validation` 和 `reporting`。
- 用 `reference` 或不变量证明结果正确。
- 用反汇编和必要的 `perfstat` 尝试支撑性能解释。
- 写出别人能复现实验的报告。

经典教材式学习方法

本册每个概念都应按同一种方法学习：

1. 先问它为什么存在。没有这个机制会有什么问题。
2. 再看它给程序员提供了什么抽象。
3. 然后看底层大致如何实现或如何表现出来。
4. 接着写最小代码，用工具观察证据。
5. 最后说明它对正确性、性能和调试有什么影响。

例如学习 ABI 时，不应只背“前六个整数参数放在 `rdi`、`rsi`、`rdx`、`rcx`、`r8`、`r9`”。还要知道为什么需要统一调用约定，调用者和被调用者如何分工，栈为什么要对齐，返回大结构体为什么可能出现隐藏指针，手写汇编破坏规则后会产生什么错误，以及如何用反汇编和 GDB 证明。

不追求一次理解所有细节

第一册会反复预告 `cache`、`TLB`、分支预测、编译器优化、SIMD、PMU 和 AI 算子，但不会在这里彻底展开。预告的目的不是提前堆高级内容，而是告诉读者：今天学的指针、对象布局、ABI 和 benchmark，以后都会在更复杂的性能问题里重复出现。

学习时应接受一个事实：第一遍读懂主线，第二遍跑通实验，第三遍写出解释，第四遍回头修正理解。底层知识不是靠记忆一次完成的，而是靠“代码、工具、证据、报告”反复闭环建立起来的。

如何使用本书

本书按经典教材方式组织，但它同时也是一本实验书。每章都应按下面顺序学习：

1. 阅读本章目标和起始问题。
2. 读正文，手工推导关键例子。
3. 运行章节中的命令。
4. 修改例子，观察输出如何变化。
5. 查看汇编、IR、二进制或性能数据。
6. 完成实验。
7. 完成作业并写报告。
8. 对照验收标准检查自己是否真正掌握。

每章的正确读法

本册每章都可以按“问题、模型、证据、报告”四步读。

1. **问题**：先明确本章回答什么问题。例如“CPU 如何执行指令”不是问 CPU 有多快，而是问指令如何改变寄存器、内存和控制流。
2. **模型**：建立足够简单但不错误的模型。例如先把内存看成按字节寻址的数组，再逐步叠加对象、对齐、cache line 和虚拟地址。
3. **证据**：每个模型都要找到工具证据。源码对应编译命令，汇编对应 `objdump`，运行时状态对应 GDB，性能结论对应原始样本和环境记录。
4. **报告**：把观察写下来。没有报告，就很难发现自己把语言层、ABI 层、硬件层和测量层混在了一起。

读正文时不要只划重点。底层知识的重点不是一句定义，而是你能否把定义用于一个真实例子。例如“指针是地址”这句话不够；你要能说明指针值、指针类型、对象生命周期、地址计算、load 指令和虚拟地址之间分别是什么关系。

三条主线

本书有三条主线。

第一条：抽象到实现

从 C/C++ 语言抽象出发，追到编译器、汇编、机器码、CPU 执行和操作系统机制。

第二条：模型到证据

先建立模型，再用工具观察。比如先理解寄存器、栈和调用约定，再用 `gdb`、`objdump`、`readelf` 和 `benchmark` 数据验证。

第三条：基础到工程

先掌握最小函数、简单循环、数组访问、函数调用、ABI 和测量误差，再进入后续卷的现代 CPU 性能、SIMD、HPC 和 AI 算子。

学习节奏建议

第一遍学习时，建议按下面节奏推进：

- 第 1-5 章不要急着优化代码，重点是跑通工具链、看懂地址、寄存器、对象布局和调试输出。
- 第 6-10 章每天都要读反汇编。只读汇编语法不够，必须把每个例子从 C++ 函数签名追到参数寄存器、栈和返回值。
- 第 11-12 章不要追求漂亮数字，重点是建立可复现实验结构：输入、热路径、校验、重复样本、CSV、环境记录和报告。

每章至少做一个实验、一个反例和一份短报告。实验验证模型，反例暴露误解，报告训练表达。对底层教材来说，这三者缺一项，理解都会不稳。

怎样使用 Linux 源码案例

本书附录提供 Linux 源码阅读案例。它们不是要求你一开始就读完整内核，而是给每个系统现象一个真实源码入口。阅读这些案例时，先用用户态小程序制造现象，再用 `strace`、`readelf`、`/proc`、GDB 或 `perf` 取得工具证据，最后进入内核源码寻找对应路径。

源码阅读必须记录版本。你读到的路径属于某个 Linux tag、commit 或发行版源码包；如果没有运行同一份内核，就不能把源码阅读直接写成当前机器行为的证明。正确写法是：当前工具输出证明了用户态现象，某版本 Linux 源码解释了这种机制通常如何实现，二者共同支持一个有边界的结论。

建议在读完第 4 章后开始做附录中的 `write`、ELF 加载和 `/proc/self/maps` 案例；读完第 11 章后再做缺页、调度噪声和 `perf_event_open` 案例。不要急着扩大范围，先把一条窄路径读清楚。

遇到难点时的处理方式

如果某一章读不动，通常不是因为某个术语太难，而是前面某一层没有闭合。可以按下面顺序排查：

1. 看不懂汇编，先回到 C++ 函数签名和 ABI 参数位置。
2. 看不懂地址表达式，先回到 `sizeof`、数组连续性和指针加法。
3. 看不懂栈变化，先画 `rsp`、返回地址、参数和局部对象。
4. 看不懂 benchmark 结果，先确认热路径有没有被编译器删除或内联改写。
5. 看不懂性能差异，先确认输入规模、访问模式、cache 状态和测量重复次数。

不要用背术语替代理解。如果一个概念说不清，就写一个 20 行以内的最小程序，用编译命令、反汇编和 GDB 把它拆开。

报告要求

每个性能实验报告至少包含：

- 实验目标。
- 机器和环境。
- 编译器和编译参数。
- correctness reference。
- 输入规模。
- benchmark 方法。
- 汇编、IR 或 profiling 证据。
- 结果表格。
- 失败实验。
- 结论和下一步验证。

报告的最低质量线

一份合格报告必须能被别人复现。它至少要回答：

- 代码从哪里来，提交或文件路径是什么。
- 用什么命令编译，优化级别和 ISA 参数是什么。
- 在什么机器和系统上运行。
- 输入如何生成，规模和分布是什么。
- 测的是哪一段代码，哪些工作不在计时区域内。
- 结果是否正确，正确性 reference 是什么。
- 原始样本在哪里，汇总口径是什么。
- 反汇编或工具输出是否支持你的解释。
- 结论有哪些限制，下一步需要验证什么。

写报告时宁可保守，不要把一个实验夸大成普遍规律。高质量教材训练的不是“说得像专家”，而是“证据能支撑到哪里，就说到哪里”。

全书架构设计

本册是系列教材的基础卷，正文分为三个部分、十二章，并保留若干附录作为工具和阅读材料。它不再把尚未写成正文的主题放进目录占位；没有正文的内容进入后续卷规划。

写作目标

第一册的写作目标是覆盖：

- C/C++ 程序如何变成机器执行。
- CPU 执行指令所需要的寄存器、内存、控制流和硬件状态模型。
- 数据表示、对象布局、指针、栈和最基本的软件运行时概念。
- x86-64 汇编语法、整数指令、控制流和 System V AMD64 ABI。
- C++ 与汇编互操作的边界、风险和验证方法。
- 可信性能测量、benchmark 输入设计、热路径和报告规范。
- 通过 Linux 源码阅读案例，把进程、ELF、虚拟内存、系统调用、调度噪声和 perf 证据接到真实内核实现。

现代 CPU 微架构、cache/TLB、编译器优化、SIMD、GEMM、生产库和 AI CPU 算子属于第二册。它们需要第一册建立的机器执行模型、汇编读写能力和实验方法作为前提。

三部分结构

1. 底层模型：从程序到机器执行。回答源码如何成为进程、CPU 如何执行指令、数据如何落在内存里、如何用 Linux 工具观察证据。
2. 汇编原理：x86-64、ABI 与二进制接口。回答 C/C++ 控制流、整数运算、函数调用和对象传递如何映射到汇编与调用约定。
3. 实验原理：可信性能测量。回答为什么很多 benchmark 不可信，以及如何设计输入、隔离热路径、记录环境并写出可复现实验报告。

章节深度标准

每章按三层写：

1. 必须掌握：刚学完 C/C++ 的读者能跟上。
2. 工程理解：能用于真实代码分析和优化。
3. 深入方向：连接体系结构、OS、编译器或生产库更深内容。

规模目标和统计口径

第一册最终正文规模目标不少于 36 万单位。这里的“单位”只按可阅读教材内容统计：

- 中文按汉字字符计数。
- 英文按单词计数。
- 代码、命令、反汇编、终端输出、LaTeX 命令和工程配置不计入正文规模。
- 附录可以计入书稿总规模，但不能替代 12 个正文章节的讲解深度。

工程中使用 `scripts/count_text_units.py` 统计该口径。扩写时必须同时满足字数目标和质量目标；不能通过堆代码块、重复命令、无解释清单或空泛段落凑数。

反凑数质量红线

36 万单位是教材深度目标，不是机械长度目标。扩写时宁可放慢进度，也不能牺牲解释质量。新增正文必须至少承担下面一种教学功能：

- 给出一个读者原本缺失的背景知识，并说明它为什么影响后续学习。

- 把一个机制从动机、约束、实现边界到工具观察讲成闭环。
- 纠正一个常见误区，并说明错误推理发生在哪一层。
- 解释一个实验应该观察什么、怎样解释结果、结论边界在哪里。
- 把当前章节与后续汇编、ABI、内存、测量或第二册性能主题建立必要接口。

以下内容不得作为扩写手段：

- 为增加长度而重复前文已经讲清楚的定义。
- 没有解释目的的代码块、命令块、反汇编块或工具输出。
- 只罗列名词、不说明关系和边界的清单。
- 只说“很重要”“需要理解”但不给机制和证据的空泛段落。
- 为了追求覆盖面而把第二册主题提前讲成浅层摘要。

每次扩写后应做一次结构审查：章节顺序是否自然，是否存在同一概念重复讲解，新增段落是否服务本章目标，实验是否配套解释，统计增长是否来自可阅读正文而不是代码或标记。

部分	建议规模	质量要求
底层模型	12 万到 15 万单位	补齐硬件、软件、数据、工具和实验背景。
汇编原理	13 万到 15 万单位	每条主线都要从 C++、汇编、ABI 和工具证据闭环。
实验原理	7 万到 9 万单位	形成可复现 benchmark 工程和报告体系。
前置与附录	2 万到 4 万单位	提供学习路线、术语、速查和评分标准。

经典教材写法

本册不按“名词解释合集”来写。每章都应从一个具体问题进入，然后逐层展开：

1. 起始问题：读者在真实编程或实验中会遇到什么困惑。
2. 最小例子：用一段短 C/C++ 程序或短汇编建立观察对象。
3. 背景知识：补齐理解本章所需的硬件、软件、算法或工具背景。
4. 底层机制：解释这个机制为什么存在，没有它会出什么问题。
5. 工具证据：给出编译命令、反汇编、GDB、readelf、nm 或 benchmark 数据。
6. 反例和误区：指出初学者最容易得出的错误结论。
7. 实验和作业：要求读者修改输入、改变编译参数、观察差异并写报告。
8. 验收标准：明确读者学完后应该能独立完成什么任务。

每章都必须讲清楚“语言层、二进制层、硬件层、实验层”的边界。比如讲数组访问，不能只讲 `a[i]` 是第 `i` 个元素；还要讲指针类型决定缩放、对象连续存储、地址表达式、`load` 指令、`cache line` 预告和 benchmark 可能如何受访问模式影响。

案例、实验和源码阅读要求

后续扩写时，每章都要尽量形成“一个主案例、多组小实验、一个工程报告”的训练结构。主案例负责把本章概念串起来；小实验负责验证局部机制；工程报告负责训练读者把事实、解释、限制和下一步分开写。

案例不应只停留在玩具代码。第一册可以从 20 行以内的 C/C++ 程序开始，但必须逐步接到真实工具和真实系统。例如讲进程时要接 `readelf`、`/proc` 和 ELF loader；讲内存时要接 `/proc/self/maps`、`mmap` 和 `page fault`；讲测量时要接 `raw samples`、反汇编和必要的 `perfstat`。Linux 源码阅读附录提供的案例，应作为每章后续扩写时的真实系统素材库。

每个新增案例至少应说明：

- 用户态现象是什么。
- 它对应本章哪个概念。
- 可以用哪些命令观察。
- 如果进入 Linux 源码，应从哪些文件或函数名开始搜索。

- 读者最后要交付什么报告。

实验数量可以多，但不能变成命令堆砌。每个实验都必须说明预期观察、可能失败原因、结论边界和报告要求。Linux 源码案例尤其要记录版本，避免把某个 commit 的实现细节写成永久事实。

第一册和第二册的边界

第一册负责建立可复用基础。以下内容可以预告，但不在第一册深挖：

- 现代乱序核心的完整性能模型。
- cache/TLB 的定量实验和 Roofline 模型。
- LLVM IR、SSA、自动向量化和复杂编译器优化。
- AVX2/AVX-512/FMA/VNNI intrinsics 和 SIMD kernel。
- GEMM blocking、packing、microkernel 和 MiniBLAS。
- softmax、layernorm、attention、operator fusion 和 AI CPU 算子库。
- 多线程、NUMA、ISA dispatch 和生产库源码阅读。

这些主题放到第二册，是为了避免第一册失焦。第一册真正要打牢的是：读者能从源码追到汇编，能解释机器状态，能遵守 ABI，能设计可信测量。没有这些能力，高级优化只会变成经验口号。

交付物标准

每章最终应包含：

- 不少于一个贯穿例子。
- 不少于三个实验。
- 不少于三组习题。
- 一个报告型大作业。
- 一组常见误区。
- 一组验收标准。
- 能连接真实系统的案例或源码阅读入口；若本章不适合直接读 Linux 源码，也要说明它怎样服务后续源码阅读。

目录

序言	ii
背景知识与学习路线	iii
如何使用本书	vi
全书架构设计	ix
第一部分 底层模型：从程序到机器执行	
第 1 章 学习地图：把程序、机器、系统和性能连成一条主线	2
第 2 章 程序如何从源码变成正在运行的进程	32
第 3 章 CPU 如何执行指令	64
第 4 章 数据表示、内存、指针和对象布局	103
第 5 章 Linux、工具链、调试器和学习工作流	153
第二部分 汇编原理：x86-64、ABI 与二进制接口	
第 6 章 x86-64 汇编语法总览	200
第 7 章 整数指令、位运算和地址计算	241
第 8 章 控制流：跳转、循环、调用与 switch	283
第 9 章 System V AMD64 ABI	336
第 10 章 C++ 与汇编互操作	383
第三部分 实验原理：可信性能测量	
第 11 章 可信性能测量基础	434
第 12 章 Benchmark 设计：输入、热路径和报告	478
附录 A x86-64 速查表	532
附录 B Linux 与二进制工具命令速查	533
附录 C LaTeX 写作规范	534
附录 D 参考资料和延伸阅读	535
附录 E 实验和作业评分标准	536
附录 F Linux 源码阅读案例	537
术语表	543

第零部分

底层模型：从程序到机器执行

第 1 章

学习地图：把程序、机器、系统和性能连成一条主线

本章不是普通导读

如果一本系统性能书只在开头列出“C++、汇编、CPU、操作系统和性能工具”，它并没有真正降低学习难度。初学者最缺的通常不是名词清单，而是一张可以反复返回的结构图：当你在本册看到 `mov`、栈、ABI 和 benchmark，在第二册看到 cache line、PMU event、SIMD 或 softmax 时，必须知道这些内容如何共同描述同一个问题：程序怎样在真实机器上运行，并怎样被可信地测量。

本章的目标就是建立这张地图。

刚学完 C/C++ 的读者通常已经知道变量、函数、数组、指针、结构体、类和简单的标准库调用。但从性能工程角度看，这些只是最上层的表达。程序真正运行时，下面还存在许多层：

```
C/C++ source
-> language semantics
-> compiler frontend
-> IR and optimization
-> target-specific backend
-> assembly
-> machine-code bytes
-> object file and executable
-> loader and process
-> virtual memory
-> ISA architectural state
-> microarchitectural execution
-> cache / TLB / branch predictor / memory system
-> measured time and performance counters
```

性能优化的难点在于：每一层抽象都在隐藏下一层的复杂性，但这种隐藏从来不是绝对的。日常业务开发依赖抽象来提高效率；系统分析、性能测量和高性能 kernel 开发则经常需要穿过抽象，检查它在什么条件下可靠、在什么边界上泄漏，以及编译器、CPU 或操作系统是否以另一种形式重新解释了你的代码。

核心思想

本书的主线不是先堆概念再谈优化，而是从一个 C/C++ 表达式出发，追踪它如何变成机器码、如何改变寄存器和内存、如何穿过 CPU 流水线和缓存层级、如何受到操作系统影响，最后如何形成可信或不可信的性能数据。

一个贯穿全书的例子

先看一个最小但足够典型的函数：

```
int sum(int const* a, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        s += a[i];
    }
    return s;
}
```

从 C++ 层看，它的语义直接：遍历数组，把元素累加起来。但要解释它在真实机器上的性能，至少需要回答下面这些问题：

- `int const*a` 是什么？它只是一个地址，还是还携带了类型和别名语义？
- `a[i]` 如何变成地址计算？为什么是 `base+i*4`？
- `s+=a[i]` 在汇编中会使用哪些寄存器？会不会每次都访问内存？
- 编译器能否把循环展开？能否向量化？为什么有时候不能？
- 如果 `n` 很小，循环开销和函数调用开销是否比内存访问还重要？
- 如果 `n` 很大，性能瓶颈是整数加法，还是内存带宽？
- 如果 `a` 没有按 cache line 对齐，是否一定慢？
- 如果多个线程同时处理数组，是否会发生 false sharing？
- benchmark 的结果为什么每次都不一样？操作系统调度、CPU 频率、cache 状态分别可能贡献什么噪声？

这些问题都不能只靠 C++ 语法解决。它们分别落在语言语义、编译器、汇编、ISA、微架构、操作系统和测量方法上；任何一层被忽略，性能解释都会失去关键依据。

心智模型

以后看到任何性能代码，都先把它拆成四个视角：

1. 源码视角：程序员写了什么语义。
2. 编译器视角：哪些语义限制了优化，哪些语义允许优化。
3. 机器视角：实际执行了哪些指令，访问了哪些地址。
4. 测量视角：数据是否真的证明了你的判断。

算法复杂度不是机器成本

很多读者已经学过算法复杂度，会用 $O(n)$ 、 $O(n \log n)$ 、 $O(1)$ 描述算法规模变化。这当然重要，但它不是性能工程的全部。复杂度告诉我们输入规模变大时增长趋势如何，却不直接告诉我们某个真实程序在某台机器上花多少时间，也不告诉我们瓶颈在哪里。

例如两个函数都是线性扫描数组：

```
for (std::size_t i = 0; i < n; ++i) {
    s += a[i];
}

for (std::size_t i = 0; i < n; ++i) {
    s += a[index[i]];
}
```

它们从复杂度看都是 $O(n)$ ，但机器成本可能完全不同。第一个循环连续读取，容易利用 cache line 和硬件预取；第二个循环间接访问，地址由另一个数组决定，可能造成大量随机 load。两者的比较不能只停在算法层，必须进入地址模式、cache、TLB、乱序执行和带宽模型。

再看两个 $O(1)$ 操作：整数加法和一次系统调用都可以说是常数时间，但它们的常数相差巨大。一个普通整数加法可能只占用很少的核心资源，而系统调用需要从用户态进入内核态，可能涉及权限切换、内核路径、调度和设备状态。复杂度把它们都压缩成常数，却不能替代机器成本分析。

因此，本书使用两套问题：

算法问题	机器问题
复杂度是什么	每次迭代执行哪些指令
输入规模如何增长	访问的是连续内存还是随机地址
数据结构抽象是否合适	对象布局、padding 和 cache line 如何影响访问
操作次数大概多少	指令依赖、分支预测和 load/store 是否限制吞吐
理论上是否最优	编译器是否生成预期机器码，测量是否可信

这不是说复杂度不重要，而是说复杂度只是第一层筛选。性能工程要在复杂度可接受的方案里

继续比较常数、访存模式、编译器可优化性、并行性和测量稳定性。很多工业级优化并不是把 $O(n)$ 改成 $O(\log n)$ ，而是让同样 $O(n)$ 的循环更贴近硬件。

核心理想

算法复杂度回答“规模增长会怎样”，机器成本回答“这台机器实际怎样执行”。高质量性能分析必须同时包含两者。

抽象层错位：许多误判从这里开始

学习底层时，最常见的错误不是完全不知道某个术语，而是在一个层次提出问题，却用另一个层次的直觉回答。

例如，问“这个 C++ 变量在哪里”是源码和调试信息层的问题。优化后，这个变量可能在寄存器里，可能被拆成多个值，可能被常量传播消失，甚至根本没有一个稳定位置。如果你用 -O0 的栈上局部变量经验去解释 -O3 热路径，就会误判。

再比如，问“这条 load 为什么慢”是微架构和数据状态问题。只回答“因为它是一条内存访问指令”不够。它可能 L1 命中，也可能 L3 miss，也可能被前面的地址依赖卡住，也可能被 TLB miss 或 page fault 影响。指令助记符相同，机器状态不同，成本就不同。

再比如，问“这个 benchmark 为什么快”是测量问题。只回答“因为算法更好”也不够。它可能真的更好，也可能是编译器删除了循环，可能是输入太小全部在 cache 中，可能是只测了 warm cache 场景，可能是没有校验结果导致错误输出。

本书要求你在每次分析时先标注问题层次：

- 语言层：程序是否有定义，表达式语义是什么。
- 编译器层：优化器能否重写，是否内联、展开、向量化。
- ABI 层：参数、返回值、栈和寄存器责任如何约定。
- ISA 层：指令如何改变架构状态。
- 微架构层：依赖、cache、预测和资源如何影响速度。
- 操作系统层：进程、虚拟内存、调度和权限如何参与。
- 测量层：数据是否能支撑结论。

只要层次标清楚，很多争论会自动变简单。比如“这个函数有没有调用”要看最终二进制，不看源码感觉；“这个地址是否合法”要看进程虚拟地址空间和对象生命周期；“这个优化是否正确”要先看 C++ 语义，而不是只看当前机器上似乎能跑。

计算机系统为什么要分层

现代计算机太复杂，不可能让程序员直接控制每个晶体管、每条内部总线、每个缓存替换决策和每个乱序执行细节。因此系统必须分层。分层的作用是让上层只依赖下层的一部分承诺。

比如 C++ 不要求你知道整数加法电路如何实现，但它要求机器能表示某些整数值。汇编不要求你知道 CPU 内部有多少执行端口，但它要求 CPU 能按照 ISA 的语义执行 add。操作系统不要求应用知道物理内存条的真实地址，但它提供虚拟地址空间，让进程以为自己拥有连续内存。

分层带来的好处是可编程性，代价是性能信息被藏起来。性能工程正是在这些隐藏处工作。

物理层：信号、晶体管和时钟

最底层是物理世界。电路通过电压高低表示 0 和 1。晶体管可以被看作受控制的开关。大量晶体管组合成逻辑门、触发器、寄存器、加法器、选择器、缓存 SRAM 单元和控制逻辑。

本书不会把你训练成芯片设计工程师，但你必须知道两个事实。

第一，硬件不是“瞬间完成”的。信号传播需要时间，门电路切换需要时间，存储单元读写需要时间。因此 CPU 有时钟周期，有关键路径，有流水线。性能中的 latency 和 throughput，最终都与“工作需要经过多少硬件阶段”和“硬件能否并行处理多个工作”有关。

第二，硬件资源是有限的。一个核心不可能无限发射指令，不可能无限读取内存，不可能无限预测分支，不可能无限保存乱序执行中的临时状态。因此现代 CPU 性能常常不是由某一条高级语句决定，而是由执行端口、load/store 单元、ROB 容量、物理寄存器、L1/L2/L3 cache、TLB、内存带宽等资源共同决定。

数字逻辑层：组合逻辑和时序逻辑

组合逻辑的输出只由当前输入决定，比如加法器、比较器、多路选择器。时序逻辑会记住状态，比如寄存器、触发器、计数器和缓存 tag。CPU 可以被看作一个巨大的状态机：每个时钟周期，它根据当前状态和输入，计算下一状态。

这个模型非常重要，因为 ISA 也可以被理解为状态机规则。以 `add rax, rbx` 为例，抽象语义是：

```
RAX_next = RAX_old + RBX_old
RFLAGS_next = flags produced by the addition
```

真实 CPU 内部可能把这条指令拆成微操作，可能乱序执行，可能重命名寄存器，但最终对软件可见的结果必须像上面的状态转移一样。这就是后面“架构状态”和“微架构实现”的区别。

ISA：软件和硬件之间的合同

ISA（Instruction Set Architecture）是指令集体系结构。它定义软件能看到的机器模型：有哪些寄存器、有哪些指令、指令如何编码、内存寻址如何表达、异常如何发生、哪些状态对程序可见。

x86-64 ISA 给程序员承诺了很多东西，例如：

- 有一组通用寄存器，如 `rax`、`rbx`、`rcx`、`rdx`、`rdi`、`rsi`、`rsp`、`rbp`、`r8` 到 `r15`。
- 指令可以读取寄存器、写寄存器、读取内存、写内存、改变控制流。
- 内存可以按字节寻址。
- 常见寻址模式可以写成 `base+index*scale+displacement`。
- `rip` 表示当前执行位置，`rsp` 表示栈顶。

但 ISA 不告诉你 CPU 内部有多少流水线阶段，也不告诉你某条指令占用哪个执行端口，不告诉你 L1 cache 多大，不告诉你分支预测器怎么猜。这些属于微架构。

常见误区

初学者经常把 ISA 和 CPU 实现混在一起。说“x86-64 有 `rax`”是 ISA 层；说“某 Intel 核心上整数加法吞吐约为每周期若干条”是微架构层。优化时必须区分这两层，否则会把一台机器上的性能经验误当成所有 x86-64 机器的规律。

微架构：同一份 ISA 可以有不同的执行机器

同样执行 x86-64 指令，不同 CPU 的内部结构可以非常不同。一个简单核心可以顺序执行；一个高性能核心会取指、解码、分派、重命名、乱序执行、访存、提交。CPU 还会预测分支、预取数据、维护多级缓存、处理 TLB、解决内存一致性问题。

微架构的存在解释了一个看似奇怪的事实：两段汇编指令数量相同，性能可能差很多；两段 C++ 源码看起来复杂度相同，真实性能也可能差很多。

例如：

```
add eax, dword ptr [rdi + rcx*4]
```

这条指令表面上是“从数组加载一个 int 并加到 `eax`”。微架构层至少可能发生：

- 地址生成单元计算 `rdi+rcx*4`。
- load 单元向 L1 data cache 发起读取。
- 如果虚拟地址还没有 TLB 命中，需要页表相关机制参与。
- 如果 L1 miss，访问 L2；L2 miss，访问 L3；再 miss，访问内存。
- 加法依赖 load 的结果，因此 load latency 会影响后续加法。
- 如果循环中有多个独立 load，乱序执行可以重叠等待。

同一条汇编，在 cache 命中和 cache miss 时的代价完全不同。这就是为什么本书必须把“指令”和“数据所在位置”一起讲。

操作系统：程序不是独占机器

C++ 程序不是直接裸奔在 CPU 上。普通 Linux 程序运行在操作系统提供的进程环境中。操作系统负责创建进程、建立虚拟地址空间、加载可执行文件、处理系统调用、调度线程、管理页面、处理中断和异常。

从性能角度看，操作系统至少带来五类重要影响：

1. 虚拟内存让程序看到的是虚拟地址，不是物理地址。
2. 页面和 TLB 会影响大数组、随机访问和多线程程序。
3. 调度会让线程在不同核心之间迁移，影响 cache 热度和测量稳定性。
4. 系统调用和 page fault 会造成远高于普通指令的开销。
5. CPU 频率、电源管理、后台任务和中断会污染 benchmark。

因此，真正的性能工程不是“写一段快代码”这么简单。你还要控制运行环境，知道什么时候该固定 CPU 频率，什么时候该 pin 线程，什么时候该丢弃第一次运行，什么时候要看 page fault、context switch 和 PMU 事件。

编译器：它不是翻译员，而是证明器和重写器

很多初学者把编译器理解成“把 C++ 翻译成汇编的工具”。这个理解只对了一半。现代优化编译器更像一个在语言规则约束下工作的证明器和重写器。

它会问：

- 这个变量的值是否一定不被使用？
- 这个循环是否可以展开？
- 两个指针是否可能指向同一块对象？
- 这个函数是否可以内联？
- 这个条件是否永远成立或永远不成立？
- 这个浮点表达式能不能重排？
- 这个内存访问是否连续，能不能向量化？

只要 C++ 标准允许，编译器就可能把源码改写成完全不同的机器执行形式。比如下面的代码：

```
int f(int x)
{
    return x + 1 > x;
}
```

在没有有符号整数溢出的前提下，编译器可以认为 $x+1 > x$ 恒为真，从而返回 1。可是如果你按机器补码回绕去想， x 等于最大正整数时似乎会溢出。关键在于：C++ 有符号整数溢出是未定义行为，编译器可以假设它不会发生。

深入理解

性能优化必须尊重语言规则。很多“我看机器码应该这样”的推理，如果违反 C++ 对对象生命周期、别名、对齐、未定义行为的规定，就不是可靠优化，而是在给编译器提供错误前提。后续讲 alias、restrict、std::bit_cast、fast-math 和自动向量化时，会反复回到这一点。

ABI：函数之间、模块之间必须说同一种话

ABI（Application Binary Interface）是二进制接口。它规定函数调用时参数如何传递、返回值放在哪里、哪些寄存器由调用者保存、哪些由被调用者保存、栈如何对齐、结构体如何返回、异常和动态链接如何约定。

源码层面两个函数看起来只是：

```
int g(int x, int y);
int z = g(1, 2);
```

机器层面却必须回答：

- 1 放进哪个寄存器？
- 2 放进哪个寄存器？
- 调用前 `rsp` 要满足什么对齐要求？
- `g` 能随便改哪些寄存器？
- 返回值从哪里取？

System V AMD64 ABI 是 Linux x86-64 上的核心规则之一。后面学习汇编时，如果不懂 ABI，就看不懂函数序言、函数尾声、栈帧、寄存器保存和 C++ 与汇编互操作。

内存：性能工程的第二个主角

第一个主角是指令，第二个主角是数据。现代 CPU 的算术单元非常快，但内存层级相对慢得多。一个 AI 算子是否快，常常不取决于“乘法会不会写”，而取决于数据如何布局、是否连续、是否复用、是否落在 cache、是否适合 SIMD load/store。

看下面两个结构：

```
struct ParticleAoS {
    float x;
    float y;
    float z;
    float mass;
};

struct ParticleSoA {
    float* x;
    float* y;
    float* z;
    float* mass;
};
```

AoS 和 SoA 的差异不是语法审美问题，而是内存访问模式问题。如果一个 kernel 只需要所有粒子的 `x`，AoS 会把不需要的 `y`、`z`、`mass` 一起拖进 cache line；SoA 则能连续加载 `x`。后续讲 SIMD、cache line、GEMM packing 和 AI 算子 fusion 时，这个思想会不断出现。

性能不是运行一次得到的数字

性能工程最危险的习惯之一，是把一次运行时间当成真相。真实机器上，一次运行结果可能受到很多因素影响：

- 编译器是否把代码优化掉。
- 输入数据是否还在 cache 中。
- CPU 当前频率是多少。
- 线程是否被调度到另一个核心。
- 后台进程和中断是否打断了运行。
- 分支预测器和 TLB 是否已经被预热。
- 第一次执行是否触发动态链接、page fault 或内存分配。

所以本书要求你形成“证据链”意识。一个高质量性能结论通常至少包含：

1. correctness reference：优化前后结果是否一致。
2. 编译参数：是否启用合理优化，是否说明目标 ISA。
3. 汇编或 IR：机器实际执行的形态是什么。
4. benchmark 方法：如何预热、重复、统计、避免被优化删除。
5. profiling 或 PMU：瓶颈证据支持哪个解释。
6. 反例检查：有没有输入规模或机器配置让结论失效。

常见误区

“我的版本快了 20%”不是结论，只是观察。真正的结论必须说明：在哪台机器、什么编译器、什么参数、什么输入规模、什么统计方法下快了 20%；为什么快；有什么证据排除了测量噪声和错误优化；在哪些条件下不保证快。

正确性是性能的前提

性能优化有一个简单但经常被忽略的底线：错误程序没有性能意义。一个函数快，但结果错了；一个 benchmark 快，但热路径被编译器删除了；一个手写汇编快，但破坏了调用约定，这些都不能算优化成功。

正确性在本书里至少有四层。

第一层是语言正确性。C++ 程序不能依赖未定义行为。越界访问、use-after-free、未初始化读取、错误对齐、违反严格别名、数据竞争、有符号整数溢出等问题，都可能让编译器做出看似奇怪但合法的优化。性能代码越接近底层，越容易触碰这些边界。

第二层是功能正确性。优化前后结果要一致。对整数函数，通常可以逐值比较；对浮点函数，需要定义误差标准；对随机算法，需要固定种子或比较统计性质；对近似算法，需要说明允许误差来自哪里。

第三层是二进制接口正确性。函数调用必须遵守 ABI。参数寄存器、返回值、栈对齐、callee-saved 寄存器、异常边界和符号名都不能随意处理。手写汇编或跨语言互操作时，功能测试通过还不够，因为某些 ABI 破坏可能只在特定调用者、优化级别或异常路径下暴露。

第四层是测量正确性。计时区域必须包含想测的工作，不包含不该测的工作；结果必须被使用，防止死代码消除；输入必须代表目标场景；重复次数和统计口径必须清楚。错误测量会把优化方向带偏。

正确性层次	常见失败	主要证据
语言正确性	未定义行为、对象生命周期错误	警告、sanitizer、代码审查
功能正确性	输出错误、误差超界	reference、单元测试、随机测试
ABI 正确性	寄存器破坏、栈不对齐、符号不匹配	ABI 推导、GDB、反汇编
测量正确性	空循环、测错区域、样本污染	harness 审计、原始数据、报告

这四层并不是最后才检查。正确流程是先有 reference，再写 baseline，再做优化；每次修改都重新验证正确性和二进制形态。很多高水平工程师并不是因为知道更多指令才稳定，而是因为他们不会让错误结果混进性能比较。

从观察到结论需要中间推理

本书会反复要求你区分观察、解释和结论。

观察是工具或实验直接给出的东西。比如反汇编里出现 `cmov`，GDB 显示 `rsp` 某个值，CSV 中某组样本中位数更低。

解释是你用模型把观察连起来。比如因为 `cmov` 消除了数据相关分支，所以在随机输入下可能减少 branch miss；因为 `rsp` 没有按 ABI 对齐，所以某些调用可能崩溃或变慢；因为样本分布有长尾，所以均值比中位数更容易受噪声影响。

结论是解释在证据约束下能站住的说法。它应该有边界。比如“在这个输入分布和这台机器上，条件移动版本比分支版本更稳定；尚未证明在有强偏置分支时仍然更快”。这比“`cmov` 一定比 branch 快”更专业。

一个完整推理链可以写成：

1. 源码层：两个版本功能等价，并通过 reference 校验。
2. 编译层：二者使用相同编译器和优化级别。
3. 二进制层：版本 A 使用条件跳转，版本 B 使用条件移动。
4. 输入层：输入分布接近随机，分支方向难预测。

5. 测量层：多次样本显示版本 B 中位数更低，方差更小。
6. 解释层：较少分支错误可能是主要原因。
7. 限制层：还需要 `perfstat` 的 `branch-misses` 支持，并测试偏置输入。

这就是教材中的“讲清楚”。它不是堆更多词，而是把每一步证据和假设放在正确位置。

本书的四条长期能力线

为了在 3 到 4 个月内尽量逼近专家水平，本书不按“轻松每周一点点”的节奏写，而是按高强度训练设计。你要同时推进四条能力线。

能力线一：机器阅读能力

机器阅读能力是指你能从源码一路读到汇编、机器码、寄存器和内存变化。最低要求是能读懂简单函数的汇编；更高要求是能看出编译器为什么生成这种形式，性能瓶颈可能在哪里。

这条线的训练材料包括：

- `objdump` 和 `llvm-objdump`。
- `gdb` 单步执行。
- Compiler Explorer。
- `clang-S`、`clang-emit-llvm`。
- 手工追踪寄存器和栈。

能力线二：性能模型能力

性能模型能力是指你能先不用跑程序，就对瓶颈做出合理预测。比如一个循环每次加载 64 字节、做 16 次浮点运算，你应该能估算它更可能受内存带宽限制还是计算吞吐限制。

这条线会反复训练：

- latency 和 throughput 的区别。
- dependency chain 和 instruction-level parallelism。
- cache line、working set、reuse distance。
- memory bandwidth 和 arithmetic intensity。
- SIMD lane、向量宽度、tail 处理。
- Roofline 模型。

能力线三：测量和诊断能力

测量能力是指你知道怎样设计 benchmark，怎样避免错误测量，怎样用工具定位问题。性能工程中，错误测量比不会优化更危险，因为它会把你带到错误方向。

这条线的工具包括：

- `perfstat` 和 `perfrecord`。
- `time`、`taskset`、`numactl`。
- `llvm-mca`。
- Intel VTune 或 AMD uProf。
- 自己写 benchmark harness。

能力线四：kernel 设计能力

kernel 设计能力是指你能把机器模型用到真实计算核心里。最终目标不是会背 CPU 概念，而是能写出、解释和验证高性能 kernel，例如 reduction、transpose、stencil、GEMM microkernel、softmax、layernorm、GELU、attention 中的小核心。

这条线要求你能同时处理：

- 数据布局。
- 分块和复用。
- SIMD intrinsic。
- 数值稳定性。

- 多线程和 NUMA。
- ISA dispatch。
- benchmark 和 profiling。

读一段底层代码的十个问题

从本章开始，每当你看到一段可能与性能或机器行为有关的 C/C++ 代码，都可以用下面十个问题检查自己是否真的看懂。

1. 这段代码的输入、输出和副作用是什么。
2. 这段代码依赖哪些 C++ 语义前提，例如对象生命周期、别名、对齐和溢出规则。
3. 热路径在哪里，哪些代码只是 setup、validation 或 reporting。
4. 核心循环的归纳变量是什么，循环退出条件是什么。
5. 每次迭代访问哪些内存地址，地址是否连续、跨步或间接。
6. 关键值会放在寄存器、栈还是内存里，是否需要看反汇编确认。
7. 是否存在循环携带依赖，是否限制并行执行。
8. 是否存在数据相关分支，输入分布会怎样影响预测。
9. 编译器可以做哪些优化，哪些语义阻止优化。
10. 如果要证明它快或慢，需要哪些测量和工具证据。

这十个问题覆盖了本册的主线。前几章你可能只能回答一半；学到汇编和 ABI 后，能回答寄存器、栈和控制流；学到测量章节后，能回答 benchmark 证据。第二册再继续把 cache、SIMD、PMU 和多线程加入这张问题表。

例如对 `sum_i32`，你至少应该能说：输入是数组地址和长度，输出是累加和；每次迭代连续读取一个 32 位整数；累加变量形成依赖；循环控制会产生分支；编译器可能展开或向量化；如果结果没有被使用，整个计算可能被删除；要证明性能，需要看优化汇编和原始样本。

学习失败的几种模式

本书对质量要求高，不只是要求内容多，也要求学习方式正确。下面几种失败模式要尽早避免。

只背术语，不做状态追踪

知道 `rip` 是指令指针、`rsp` 是栈指针，并不等于理解函数调用。你必须能在 `call` 前后写出 `rsp` 如何变化，返回地址写到哪里，`ret` 又如何恢复 `rip`。底层知识如果不能落到状态变化，就只是名词记忆。

只看源码，不看二进制

性能工程不能停在源码。编译器可能内联、展开、删除、向量化、合并分支、替换指令。你以为测的是一个函数，实际二进制里可能没有它；你以为循环每次都调用某个函数，实际已经内联。凡是性能结论，必须至少看一次优化后的二进制证据。

只看一次时间，不看分布

一次运行时间很容易受噪声影响。即使多次运行，也不能只挑最漂亮的数字。至少要保存原始样本，观察最小值、中位数、最大值和异常点。第三部分会系统讲统计口径，但从第一章开始就要建立这个习惯。

只优化速度，不保留 reference

没有 reference 的优化会慢慢失控。你需要一个清晰、简单、可信的版本作为正确性基准。优化版本可以复杂，但它必须不断和 reference 对比。浮点和近似计算还要定义误差标准。

只追新技术，不补基础链路

SIMD、GEMM、attention、operator fusion 都很重要，但它们建立在基础链路上：对象布局、地址计算、ABI、汇编阅读、cache 模型、benchmark 方法。如果这些基础不稳，直接看高级主题只会变成复制代码。第一册的任务就是把基础链路补稳。

一条完整的优化闭环

本书要求你以后按下面流程做优化：

1. 写出清晰正确的 baseline。
2. 准备 correctness reference 和误差标准。
3. 建立粗略性能模型：计算量、访存量、预期瓶颈。
4. 设计 benchmark，避免死代码消除和输入偏差。
5. 编译并查看汇编或 IR。
6. 运行测量，记录环境和统计结果。
7. 用 profiling 或 PMU 验证瓶颈。
8. 修改一个因素：布局、循环、分块、SIMD、预取或并行。
9. 再次验证正确性和性能。
10. 写报告，说明哪些假设被证实，哪些被推翻。

这就是专业性能工程和“凭感觉改代码”的区别。

第一册的边界：先把基础链路打通

这本书现在压缩成第一册的 12 章，不是把重要内容删掉，而是重新安排学习顺序。底层性能知识很容易膨胀：一旦讲到 cache，就会牵出预取、TLB、NUMA 和一致性；一旦讲到 SIMD，就会牵出指令编码、寄存器宽度、尾部处理、对齐、混洗和降频；一旦讲到 AI 算子，就会牵出 GEMM、softmax、attention、KV cache、量化和算子融合。所有这些都重要，但如果读者还没有建立源码、汇编、ABI、进程、内存和测量之间的基本链路，直接进入这些主题，往往只能背结论、抄代码，无法独立判断。

因此第一册的目标非常明确：让读者能把一个普通 C++ 程序放到真实 Linux x86-64 环境里，解释它怎样成为进程、怎样成为机器码、怎样调用函数、怎样使用栈和寄存器、怎样访问内存、怎样被调试和反汇编、怎样被测量。第一册不追求把所有现代 CPU 细节一次讲完，而是先建立后续一切高级主题都必须依赖的基础坐标系。

第一册重点	读完后应具备的能力
底层原理	能解释源码、对象、地址、进程、指令和数据之间的关系。
硬件原理	能用状态机、寄存器、内存和指令生命周期理解 CPU 执行。
软件原理	能理解编译、链接、加载、进程、虚拟地址空间和调试工具。
算法原理	能把复杂度、数据结构、访问模式和机器成本同时纳入分析。
汇编原理	能读写基本 x86-64 汇编，理解控制流、整数指令和 ABI。
测量原理	能设计可信 benchmark，知道数字是否有资格成为证据。

这也解释了为什么第一册没有把 AI 算子作为主线。AI 算子非常适合做第二册，因为它们需要把第一册知识组合起来：softmax 需要数值稳定性、内存访问和向量化；layernorm 需要 reduction、误差和带宽模型；attention 需要数据布局、cache 行为和多级循环；GEMM 需要 blocking、packing、寄存器分块和微内核。没有第一册的基础，这些内容很容易变成“看起来高级，实际无法验证”的代码展示。

核心思想

第一册不是低配版，而是基础版。基础版的质量标准不是“讲得简单”，而是把每个后续高级主题都会用到的底层合同讲清楚：语言合同、编译合同、二进制合同、调用合同、进程合同、指令合同和测量合同。

第一册不急着解决的问题

为了保持学习路径清楚，第一册会有意识地延后一些问题。

第一，第一册不会系统展开现代 cache 层级、硬件预取、PMU 事件分类和顶层微架构分析。它会在必要处解释 cache、TLB、分支预测和乱序执行的基本意义，但不会把它们发展成完整性能模型。读者在第一册只需要知道：访存不是同一种成本，分支预测会影响控制流，依赖会限制并行，测量必须控制机器状态。

第二，第一册不会系统讲 SIMD intrinsic 和向量化 kernel。它会解释为什么连续数据、对齐、循环结构和别名信息影响编译器优化，但不会要求读者立即掌握 AVX2 或 AVX-512 的全部指令。第二册再把这些基础转化为实际向量代码。

第三，第一册不会系统讲多线程、原子操作、锁、NUMA 和 false sharing。它会提醒操作系统调度和线程迁移会污染测量，但不会把并发性能作为主线。多线程性能需要更复杂的内存模型和系统实验，放在后续更合适。

第四，第一册不会以 AI 算子为核心案例。它会在例子中偶尔提到 reduction、dot product 和数组扫描，因为这些是底层性能训练的最小单元。真正的算子级优化需要更完整的数据布局、数值误差、并行调度和硬件特性知识。

这种边界不是降低要求，而是提高要求。因为每一册都要有清晰的主线，不能把所有名词堆在一起。第一册的任务是把“程序如何在机器上成立”讲扎实；第二册才能把“如何利用机器把算子做快”讲扎实。

七个合同：理解系统分层的核心工具

前面说过计算机系统要分层。更精确地说，层与层之间依靠合同连接。合同规定了上层可以依赖什么，下层可以自由实现什么。学习底层时，最重要的能力之一就是知道自己正在使用哪个合同。

语言合同

C++ 源码首先受语言合同约定。语言合同规定表达式如何求值、对象什么时候存在、指针能指向哪里、引用是否必须绑定有效对象、整数溢出有什么规则、浮点运算是否允许重排、线程之间怎样形成数据竞争。编译器优化的出发点就是语言合同，而不是某台机器上的直觉。

例如越界访问在某次运行中可能“碰巧读到一个值”，但语言合同不保证这种行为。编译器可以利用“程序不会越界”这个前提重排或删除代码。再例如一个对象生命周期结束后，旧指针的数值可能还像地址，但语言层已经不允许通过它访问对象。很多性能 bug 的根源不是硬件太复杂，而是程序先违反了语言合同，然后编译器在合法范围内做了读者没有预期的优化。

编译合同

编译器接收源程序，输出等价的目标程序。这里的等价不是“每行源码对应几条汇编”，而是“对语言定义的可观察行为等价”。只要可观察行为不变，编译器可以内联、删除、合并、常量传播、循环展开、向量化、重排，甚至让你在汇编里找不到原来的局部变量。

这就是为什么调试构建和发布构建表现差异巨大。调试构建通常保留更多局部变量和栈帧形态，方便观察；优化构建更关心机器执行。读者必须学会在两种构建之间切换：用调试构建理解源码意图，用优化构建分析真实性能。

目标文件和链接合同

一个翻译单元编译后通常先变成目标文件。目标文件包含机器码、符号、重定位信息、节区和调试信息。链接器再把多个目标文件、静态库和动态库组合成可执行文件或共享库。这里的合同决定函数名如何被解析、外部符号在哪里、全局对象如何初始化、代码和数据放进哪些段。

如果只看单个源文件，就会错过很多二进制事实。函数可能在另一个目标文件里；库函数可能通过动态链接进入；全局符号可能因为可见性和重定位产生额外间接层；链接时优化可能跨文件改写调用关系。后面第 1 章和第 4 章会把这个过程具体化。

ABI 合同

ABI 合同让不同编译单元、不同语言、不同汇编文件可以互相调用。它规定参数寄存器、返回值寄存器、栈对齐、寄存器保存责任、结构体传参、异常边界和符号命名。ABI 的重要性在手写汇编时尤其明显：只要你破坏 callee-saved 寄存器或栈对齐，程序可能在当前测试里没崩，但在另一个调用点突然出错。

因此，本书讲汇编不是为了让读者炫耀几条指令，而是为了让读者知道二进制接口的责任边界。

你必须能回答：调用者保证什么，被调用者保证什么，返回后哪些状态仍然有效，哪些状态可以被破坏。

操作系统合同

普通程序运行在操作系统提供的进程合同里。进程拥有虚拟地址空间、文件描述符表、当前工作目录、环境变量、权限、信号处理方式和线程集合。操作系统负责把可执行文件加载进内存，给用户态程序提供系统调用入口，在异常和中断发生时接管控制。

这个合同解释了为什么同一个二进制在不同环境下表现不同。当前目录改变，打开文件可能失败；环境变量改变，动态库搜索路径可能改变；地址空间随机化改变，某些地址数值会变；内存压力改变，page fault 可能增多；调度策略改变，benchmark 波动可能变大。

ISA 合同

ISA 合同是软件可见机器状态的规则。它规定 `rax`、`rsp`、`rip` 和标志位这样的架构状态如何被指令读取和修改。汇编阅读的核心就是在 ISA 合同下做状态追踪。看到 `cmp`，要知道它主要写标志位；看到 `jle`，要知道它读取标志位决定控制流；看到 `call`，要知道它写返回地址并改变 `rip`。

ISA 合同不承诺内部执行成本。它告诉你指令“应该产生什么结果”，不告诉你“用了多少周期”。后者属于微架构和具体机器状态。

测量合同

测量合同是你自己给实验制定的规则。它规定什么算输入、什么算输出、什么代码在计时区域内、重复多少次、如何统计、如何保存原始数据、如何证明 correctness。很多性能争论没有结果，就是因为双方没有共享同一个测量合同：一个人在测端到端，一个人在测热循环；一个人在测 cold cache，一个人在测 warm cache；一个人报告最小值，一个人报告均值。

高质量教材必须训练这种合同意识。以后读任何性能文章、benchmark 图表或优化报告，都先问：它的合同是什么？它没有说清的部分，就是结论最脆弱的部分。

背景知识补课清单

读者不需要在开始本书前已经是系统专家，但必须愿意补齐一些基础。下面的清单不是考试大纲，而是后续学习的支撑点。遇到不熟的地方，不要停下来翻完所有资料再继续；更好的方式是在本书实验中反复验证。

C++ 基础：从语法走向语义

最低要求不是会写很多语法，而是知道语法背后的语义问题。变量不是“盒子”这么简单，它有类型、对象身份、生命周期、存储期和可能的地址。指针不是“整数地址”这么简单，它还受对象边界、对齐、别名和生命周期限制。引用不是可空指针，数组到指针转换不是数组复制，函数调用不是文本替换，未定义行为不是“随机返回一个值”。

后续最常用的 C++ 背景包括：

- 基本整数类型、符号位、转换和溢出规则。
- 指针、引用、数组、结构体和对象生命周期。
- `const` 的含义，以及它和内存只读之间的区别。
- 函数调用、重载、内联、模板实例化的基本概念。
- 编译单元、头文件、声明和定义。
- RAII、构造析构和资源所有权的基本思想。

这里最容易出错的是把 C++ 当作“高级汇编”。C++ 当然能贴近机器，但它首先是一门有抽象语义的语言。想要写可靠高性能代码，必须同时尊重语言语义和机器成本。

二进制和整数：别把表示当成数值本身

机器用位表示数据。一个 32 位模式可以解释成无符号整数、有符号整数、浮点数、指令编码、地址的一部分，也可以只是字节序列。表示和解释必须分开。

例如位模式 `0xffffffff` 可以表示无符号整数最大值，也可以在常见补码机器上表示有符号 `-1`。但是 C++ 的有符号溢出规则、无符号回绕规则和机器补码表示不是同一个层次。后续学习汇编时，你会看到同一条 `add` 可以用于有符号和无符号加法；区别常常体现在你如何解释标志位和后续跳转，

而不是加法器本身换了一套。

需要掌握的基础包括：

- 位、字节、字、十六进制表示。
- 补码的基本思想。
- 左移、右移、掩码、按位与或异或。
- 大端和小端的概念。
- 整数宽度和截断。
- 地址计算中的 scale 和位移。

这些知识看似低级，但会直接影响你是否看得懂反汇编。看到 `[rdi+rsi*4+8]`，你要立刻想到“基址、索引、元素大小、字段偏移”，而不是把它当成一串难读符号。

Linux 命令行：工具链是学习的一部分

本书默认在 Linux 或接近 Linux 的环境中学习。原因不是其他系统不重要，而是 Linux 工具链更容易直接观察编译、链接、进程、内存映射、系统调用和性能计数。你不需要成为运维专家，但必须能在 shell 中完成基本实验。

至少要熟悉：

- 文件路径、当前目录、相对路径和绝对路径。
- `mkdir`、`cp`、`mv`、`rm`、`find`、`grep` 或 `rg`。
- 重定向、管道、退出码和环境变量。
- `gcc`、`clang`、`make` 的基本使用。
- `objdump`、`readelf`、`nm`、`ldd` 的基本用途。
- `gdb`、`strace`、`perf` 的基本概念。

工具不是附属品。底层学习的很多知识必须通过工具验证。你不能只说“编译器可能内联”，你要用反汇编证明；不能只说“程序访问了非法地址”，你要用 GDB、sanitizer 或 core dump 证明；不能只说“测量有噪声”，你要保存样本并展示分布。

数学和算法：够用但要准确

本书不要求读者提前学完高等数学，但一些基础必须准确。算法复杂度要能用于规模分析；对数和指数要能理解数量级；均值、中位数、分位数、方差要能用于读 benchmark；简单线性模型要能用于估算吞吐和带宽。

更重要的是把数学语言和机器现象连接起来。复杂度里的 $O(n)$ 只说明增长阶，机器成本还要乘上每个元素的字节数、每次迭代的指令数、依赖长度、cache miss 概率和并行度。均值是样本摘要，不是全部事实；中位数能代表典型情况，但可能隐藏长尾；最小值常被用来近似低噪声状态，但不能代表用户体验。

学习性能时，数学不是为了形式化炫技，而是为了防止语言含糊。说“快很多”不如说“中位数降低 18%，最小值降低 15%，p95 没有改善”。说“内存访问很多”不如说“每个元素读取 8 字节、写 8 字节，理论最少 16n 字节流量”。这种表达会迫使你面对事实。

调试心态：把不确定性写下来

底层学习里最坏的习惯是过早确定答案。程序崩溃时，先不要说“肯定是栈坏了”；benchmark 波动时，先不要说“肯定是系统调度”；汇编变短时，先不要说“肯定更快”。正确做法是列出假设、设计观察、逐步排除。

例如一个函数变慢，可以先列出：

1. 编译器生成了不同机器码。
2. 输入规模改变导致 cache 状态不同。
3. 分支预测命中率下降。
4. 内存分配或初始化混进计时区域。
5. CPU 频率或调度状态改变。
6. correctness 校验路径不一致。

然后逐项找证据。看反汇编验证第一个假设；固定输入和规模验证第二个；用 `perfstat` 或输入分布验证第三个；审计 harness 验证第四个；记录环境和 raw samples 验证第五个；统一 reference 验证第六个。这种过程比“经验猜测”慢一点，但它能积累可靠能力。

把一个表达式拆到机器层

为了把抽象链路落地，来看一个更小的表达式：

```
std::uint64_t y = a[i] + 7;
```

源码层说：从数组中取第 `i` 个元素，把它和常量 7 相加，得到 `y`。但底层分析要继续拆。

语言层首先问：`a` 是否指向至少 `i+1` 个有效元素？`i` 是否在范围内？`a[i]` 的类型是什么？读取是否违反对象生命周期？如果这些不成立，程序语义已经不可靠。

类型和布局层问：元素宽度是多少？如果 `a` 指向 `std::uint64_t`，每个元素 8 字节；如果指向 `std::uint32_t`，每个元素 4 字节。数组下标不是神秘语法，本质上会引出地址计算。

编译层问：`i` 是否已知？`a` 是否可能和 `y` 所在对象别名？表达式是否在循环中？编译器能否把地址计算和 load 移动、合并或删除？如果 `y` 后续没有可观察使用，整句可能被删掉。

ISA 层问：基址在哪个寄存器？索引在哪个寄存器？使用哪种寻址模式？加法写哪个寄存器？标志位是否重要？可能的形态是：

```
mov rax, qword ptr [rdi + rsi*8]
add rax, 7
```

微架构层问：这个 load 是否命中 L1？地址生成是否依赖前面的计算？后面的指令是否依赖 `rax`？如果在循环中，是否有多个独立 load 可以重叠？如果每次 `i` 来自另一个数组，硬件预取是否还能工作？

测量层问：如果你想比较两种写法，计时区域是否只包含表达式所在 kernel？输入是否防止编译器常量折叠？结果是否被消费？样本是否重复？是否看过汇编确认两种写法真的生成不同代码？

心智模型

底层学习的核心动作是“拆开”。把一行源码拆成语义前提、类型布局、地址计算、指令状态、微架构成本和测量证据。只有拆到这些层次，优化讨论才不会停留在表面。

学习本书时如何做笔记

建议读者从第一章开始建立三类笔记，而不是只摘抄概念。

概念笔记：记录定义和边界

概念笔记用来回答“这个词在本书中是什么意思”。例如 ISA、ABI、virtual memory、cache line、undefined behavior、latency、throughput、benchmark。每个概念至少写三句话：它解决什么问题；它不解决什么问题；一个最小例子是什么。

差的笔记是：“ABI 是应用二进制接口。”这只是翻译。好的笔记是：“ABI 规定二进制级函数调用合同，例如参数寄存器、返回值、栈对齐和寄存器保存责任。它不规定 CPU 内部如何执行指令，也不保证函数语义正确。手写汇编调用 C++ 函数时，必须遵守 ABI，否则调用者和被调用者对机器状态的预期会冲突。”

证据笔记：记录命令、输出和结论

证据笔记用来回答“我怎么知道”。例如你写下：

```
clang++ -O3 -S -masm=intel sum.cpp -o sum.s
```

然后摘出关键汇编，说明参数寄存器、循环结构、地址计算和返回值。不要只保存命令，也不要只保存结论。命令、关键输出和解释必须在一起。这样几周后回看，才知道自己当时凭什么相信某个判断。

错误笔记：记录被推翻的假设

底层学习中，被推翻的假设非常宝贵。比如你以为 `-O3` 一定更快，后来发现小输入里函数调用和分支结构让结果相反；你以为结构体字段顺序不重要，后来发现 `padding` 改变了对象大小；你以为 `benchmark` 测的是计算，后来发现测到了随机数生成。每次记录错误时，都写清：

- 1. 原始假设是什么。
- 2. 哪个实验推翻了它。
- 3. 正确解释是什么。
- 4. 下次遇到类似问题先检查什么。

这种笔记会显著提高学习质量。因为专家并不是从不犯错，而是会把错误变成更可靠的判断规则。

十二章之间的依赖关系

第一册虽然只有 12 章，但不是 12 篇孤立文章。它们之间有清楚的依赖。

第 1 章解释程序如何从源码进入进程世界。没有这章，就容易把源码、目标文件、可执行文件和运行中进程混成一件事。第 2 章解释 CPU 如何执行指令，帮助读者建立架构状态、依赖、分支和访存的基本模型。第 3 章解释数据、内存、指针和布局，把 C++ 对象和机器地址连接起来。第 4 章讲 Linux 工具和调试方法，让前面三章的概念能被真实观察。

第 10 章开始进入 x86-64 汇编语法。它不是突然跳到另一门语言，而是把前面“机器状态”具体化。第 11 章讲整数指令，让读者能追踪算术、位运算和标志位。第 12 章讲控制流，让读者理解分支、循环、比较和函数跳转。第 13 章讲 System V ABI，把函数调用变成可验证的二进制合同。第 14 章讲 C++ 与汇编互操作，要求读者真正把 ABI、类型布局、栈对齐和测试结合起来。

第 15 章和第 16 章把前面的观察能力转成测量能力。第 15 章问“性能数字为什么可能不可信”，第 16 章问“怎样设计一个长期可维护的 benchmark”。到这里，第一册形成闭环：你能读源码，能看二进制，能理解执行状态，能证明接口正确，能设计测量，并能写出有边界的结论。

阶段	核心问题	产出能力
第 0 到 4 章	程序如何成为进程和机器状态。	能观察源码、进程、内存和指令。
第 10 到 12 章	汇编如何表达计算和控制流。	能追踪寄存器、标志位、跳转和循环。
第 13 到 14 章	函数边界如何在二进制层成立。	能写和审计 C++ 与汇编互操作。
第 15 到 16 章	性能数字如何成为可信证据。	能设计、运行和解释 benchmark。

学习时不要跳过前面的观察训练。很多人急着看汇编，却不知道可执行文件如何生成；急着看 benchmark，却不知道优化器实际删除了什么；急着写手工汇编，却不知道 ABI 保存责任。第一册的章节顺序就是为了避免这些断裂。

如何判断自己真的学会了

读完一章不等于学会一章。底层知识的验收标准应该是能执行、能解释、能迁移。

能执行，是指你可以在自己的机器上复现实验。比如生成汇编、运行 GDB、查看 `/proc`、构造 benchmark、保存 CSV。不能执行的知识，很容易停留在想象。

能解释，是指你可以把观察结果连接到模型。比如不仅知道 `rsp` 变了，还能说出 `call` 写入返回地址；不仅知道某个版本快，还能说出输入、汇编、样本和噪声如何支持这个判断。

能迁移，是指你可以把同一个模型用到新例子。比如学会数组求和后，能分析 dot product；学会函数调用后，能分析参数更多的函数；学会计时区域后，能审计另一个 benchmark；学会 ABI 后，能检查手写汇编是否破坏寄存器。

每章结束时，建议用下面的自测问题检查：

- 1. 我能不能不用看书，画出本章核心对象之间的关系图？
- 2. 我能不能写一个最小程序触发本章现象？
- 3. 我能不能用至少一个工具观察它？

4. 我能不能说明观察结果有什么限制？
5. 我能不能把同样方法迁移到另一个小例子？

如果答案是否定的，说明还需要回到实验，而不是继续背下一章。

从第一册走向第二册

第一册结束时，读者应当具备进入第二册的条件。第二册可以把主线扩展到更深的性能主题：cache 层级、带宽模型、SIMD、循环优化、GEMM、AI 算子、PMU、线程、NUMA 和生产级性能诊断。那时每个高级主题都可以接到第一册的基础链路上。

例如讲 cache line 时，可以回到第 3 章的对象布局 and 地址；讲 SIMD 时，可以回到第 10 到 12 章的寄存器和指令；讲手写 microkernel 时，可以回到第 13 到 14 章的 ABI；讲 PMU 时，可以回到第 15 到 16 章的测量合同；讲 AI 算子时，可以回到算法复杂度、访存字节数、数值正确性和 benchmark 报告。

这样安排还有一个好处：第一册可以独立成为一本扎实的系统基础教材。即使读者暂时不做 AI 或 HPC，只要从事 C++、系统软件、编译器、数据库、存储、网络、游戏引擎、图形、嵌入式或性能敏感业务代码，这些基础仍然有价值。它们不是某个热点方向的附属品，而是理解程序如何在机器上运行的基本功。

不同读者的学习路线

读者背景不同，学习时的重点也不同。第一册的章节顺序建议保持不变，但每类读者可以在实验和复习上采用不同策略。

刚学完 C/C++ 的读者

这类读者最大的风险是把底层知识理解成一堆新名词：寄存器、栈、ABI、ELF、cache、benchmark。你不应该急着背完所有术语，而应该每章都抓住一个可执行动作。

第 1 到 4 章重点是“观察”。你要学会编译一个小程序，找到可执行文件，运行它，在 `/proc` 中观察进程，用 GDB 看寄存器和栈，用 `objdump` 看机器码。此阶段不要求你解释所有性能现象，但要求你不再把程序当作黑盒。

第 10 到 14 章重点是“追踪”。你要能拿一段简单汇编，写出每条指令前后寄存器、标志位和栈的变化。遇到函数调用，要能画出参数从哪里来、返回值到哪里去、哪些寄存器必须保存。不要跳过手算状态变化，因为汇编理解的核心不是会读助记符，而是能追踪状态。

第 15 到 16 章重点是“怀疑”。你要学会不轻信一次时间数字。任何 benchmark 都先问：结果正确吗？代码真的执行了吗？输入是什么？计时区域在哪里？重复次数够吗？汇编看过吗？这个习惯比背某个工具命令更重要。

已经会写项目但底层不稳的读者

这类读者常常有工程经验，能写 C++ 项目，知道构建系统和调试工具，但对编译器、ABI、机器码和测量证据缺少系统理解。你的重点不是“学会写代码”，而是把已有经验下沉到二进制层。

读第 1 章时，不要只看编译命令，要追踪翻译单元、目标文件、符号、链接和加载。读第 3 章时，不要只说“结构体有 padding”，要能用 `sizeof`、`alignof`、`offsetof` 和内存 dump 证明布局。读第 13 章时，不要只背参数寄存器顺序，要能审计一个 C++ 调用和一个手写汇编函数是否遵守 ABI。

你的实验报告应该比初学者更强调证据链。比如同样做 `sum_i32`，初学者可以回答参数寄存器和循环形态；你还应该说明优化级别如何改变机器码，为什么某些变量消失，是否发生内联或向量化，benchmark 结果是否与反汇编一致。

做编译器、系统软件或底层库的读者

这类读者需要把本书当作接口合同手册来读。你关心的不只是某条指令怎么写，而是语言语义、IR、ABI、目标文件、运行时和操作系统之间怎样衔接。

对编译器读者来说，第 0 章的“七个合同”尤其重要。前端不能把未定义行为和目标机器补码直觉混淆；中端优化不能破坏语言可观察行为；后端必须遵守 ABI；调试信息和优化代码之间存在不可避免的张力；benchmark 必须证明优化没有改变语义。后续如果你设计自己的编译器或语言，这些问题都会以工程形式出现。

对系统库读者来说，第 13 到 16 章尤其重要。库函数是别人依赖的二进制边界，不能只在当前

调用点正确；性能报告要面对多机器、多编译器、多输入规模；手写汇编必须和 C++ wrapper、测试、调度和 benchmark 一起设计。一个快但边界不清的函数，迟早会成为维护风险。

目标是 AI/HPC 优化的读者

如果你的目标是 AI 算子、高性能计算或 SIMD kernel，第一册看起来可能“离目标有点远”。实际上它是必要前置。AI/HPC 优化不是从 intrinsic 开始，而是从证据链开始：数据在哪里，地址怎样变化，循环依赖是什么，编译器生成什么，ABI 如何调用，benchmark 是否可信。

学习第一册时，你可以把每个小例子都映射到第二册主题。数组求和是 reduction 的基础；dot product 是向量乘加和误差分析的基础；结构体布局是 tensor layout 的基础；ABI 是 microkernel 调用的基础；benchmark 报告是算子性能对比的基础。这样读，第一册不是等待进入高级内容，而是在搭建高级内容的承重结构。

学习诊断：先知道自己缺哪一层

开始正式学习前，建议做一次自我诊断。下面的问题不需要全部答对，但答案会告诉你应该在哪些章节多花时间。

源码和语言层

1. `int*p` 中，`p` 的数值、指向对象、对象生命周期和指针类型是什么关系？
2. 为什么 `std::int32_t` 有符号溢出和 `std::uint32_t` 无符号回绕不是同一种语义？
3. 一个局部变量在 `-O3` 下为什么可能没有固定内存地址？
4. `const` 指针参数是否保证底层内存物理只读？
5. 两个指针是否可能别名，为什么会影响优化？

如果这些问题不稳，第 1 章和第 3 章要慢读，实验中多看对象布局和优化后汇编。不要急着进入 SIMD，因为很多 SIMD bug 本质上是对象、别名、对齐和生命周期问题。

机器和汇编层

1. `call` 对 `rsp` 和内存做了什么？
2. `cmp` 的结果存在哪里，后续跳转如何使用它？
3. `[rdi+rsi*4+8]` 这个寻址表达式能对应到什么 C++ 数据访问？
4. caller-saved 和 callee-saved 寄存器的区别是什么？
5. 栈对齐为什么会影响跨语言调用？

如果这些问题不稳，第 10 到 14 章要配合 GDB 和反汇编手算。不要只看解释文字，一定要写出寄存器变化表。汇编学习中，“我大概知道意思”常常不够，必须能精确到每一步状态。

系统和工具层

1. 可执行文件和运行中进程有什么区别？
2. `ldd`、`readelf`、`nm`、`objdump` 分别适合观察什么？
3. `/proc/<pid>/maps` 能告诉你什么？
4. `core dump`、`sanitizer`、GDB 分别适合定位哪类错误？
5. 动态链接失败和运行时崩溃如何区分？

如果这些问题不稳，第 4 章要多做实验。系统工具的学习不能靠记命令列表，要靠“为了证明某个事实，我选择哪个工具”的判断。

测量和性能层

1. 为什么一次运行时间不能证明优化成功？
2. benchmark 中 `setup`、`warmup`、`timed region`、`validation` 应如何分开？
3. 如何防止编译器删除被测代码？
4. `median`、`min`、`p95` 分别表达什么？
5. 什么情况下 PMU counter 不能支持你的解释？

如果这些问题不稳，第 15 到 16 章要反复练。性能工程中的错误结论往往不是因为工具不高级，

而是因为最基本的实验协议不清楚。

核心理想

自我诊断不是为了给读者分类，而是为了避免盲目推进。底层知识是一条链，哪一环不稳，后面的高级主题都会变成记忆负担。

每章交付物：把学习变成工程产物

本书不建议只读不做。每章都应该留下可检查的产物。产物不是形式主义，它能证明你真的把概念落到了机器上。

源码产物

每章实验都应保留源码。源码要尽量小，文件名清楚，能独立编译。例如：

```
labs/ch03_layout/struct_padding.cpp
labs/ch10_asm/read_add_flags.s
labs/ch13_abi/call_registers.cpp
labs/ch15_measurement/bench_sum.cpp
```

小源码比大项目更适合学习底层，因为变量少、证据清楚、反汇编短。每个小程序最好只验证一个问题：结构体 padding、参数寄存器、栈变化、DCE、分支跳转。不要把太多现象混在一个实验里。

命令产物

每个实验都应保存关键命令：

```
clang++ -std=c++20 -O3 -S -masm=intel sum.cpp -o sum.s
objdump -drwC -Mintel ./bench_sum > bench_sum.objdump.txt
gdb -q ./a.out
perf stat ./bench_sum
```

命令是复现实验的入口。只写“我编译了程序”没有价值。要写完整命令，包括标准版本、优化级别、目标架构、输出文件、运行参数。性能报告尤其要保存完整命令，因为一个少掉的 `-march=native` 就可能改变结论。

观察产物

观察产物包括反汇编片段、GDB 输出、`/proc` 内容、CSV 样本、`perfstat` 输出和错误日志。观察产物不需要完整贴进正文，但要保存在结果目录中。报告正文只摘关键部分，并说明它证明什么。

例如反汇编片段不应只是贴代码，而要标注：

```
Observation:
    rdi holds pointer a
    rsi holds n
    loop uses [rdi + rcx*4]

Interpretation:
    scale 4 corresponds to sizeof(int32_t)
    rcx is the induction variable
```

这种标注能训练你从工具输出走向解释，而不是把工具输出当成答案。

报告产物

每章至少写一份短报告。报告不追求长，但必须包含：

- 问题：这次实验验证什么。
- 方法：用什么源码、命令、工具。
- 观察：关键输出是什么。
- 解释：观察如何支持或推翻假设。
- 限制：还有什么没有证明。

如果报告里没有限制，通常说明你还没有想清楚证据边界。比如“我证明了 `-O3` 一定更快”就

是过度结论；更好的写法是“在这个输入、这个编译器和这台机器上，-O3 生成的版本更快；尚未测试小输入和不同编译器”。

错误产物

建议专门保存失败实验。比如编译错误、链接错误、段错误、sanitizer 报告、benchmark 被优化器删除、perf 权限不足。失败产物非常适合学习，因为它们暴露边界。

记录失败时不要只写“失败了”。要写：

1. 触发命令。
2. 错误输出。
3. 初始猜测。
4. 证据排查过程。
5. 最终原因。
6. 下次如何预防。

底层工程师的能力很大一部分来自处理失败路径。只保存成功路径，会让学习看起来顺利，但真实项目不会这么配合。

本书中的“讲清楚”到底是什么意思

用户常说要“讲清楚”。在底层教材里，讲清楚不是把每个词解释得很长，也不是把所有历史细节都写进去。讲清楚至少包含五层。

定义清楚

术语要有边界。例如“寄存器”在 ISA 层是软件可见状态，在微架构层可能对应物理寄存器和重命名机制；“内存”在 C++ 层是对象存储，在 OS 层是虚拟地址空间，在硬件层涉及 cache、TLB 和物理内存。定义清楚就是不让同一个词在不同层次之间偷换。

动机清楚

每个概念都要说明为什么存在。ABI 不是为了增加复杂度，而是为了让独立编译的二进制能互相调用；虚拟内存不是为了让地址更神秘，而是为了隔离、权限、装载和内存管理；benchmark 协议不是为了麻烦，而是为了防止错误数字变成错误决策。

机制清楚

机制要落到状态变化。函数调用要讲 rsp、返回地址、参数寄存器和保存责任；数组访问要讲基址、索引、scale 和 load；条件分支要讲 flags 和跳转；benchmark 要讲计时区域、样本、统计和防优化。没有状态变化的解释，很容易停留在比喻。

证据清楚

底层结论要能被工具验证。说“参数在寄存器里”，要能用反汇编或 GDB 证明；说“编译器删除了循环”，要能展示优化前后机器码；说“测量不可信”，要能指出 raw samples、计时区域或 correctness 的问题。证据清楚能防止教材变成纯口头权威。

边界清楚

每个说法都要知道适用范围。System V ABI 不等于 Windows x64 ABI；某台 CPU 的吞吐不等于所有 x86-64；-O3 的结果不等于 -O0；warm-cache benchmark 不等于 cold-cache；微基准不等于端到端。边界清楚是专业写作的核心。

心智模型

判断一段解释是否清楚，可以问五个问题：定义是什么，为什么需要它，状态如何变化，用什么证据验证，适用边界在哪里。

高强度学习节奏

如果目标是在较短时间内建立坚实基础，建议按“读、做、查、写、复盘”的节奏推进。

读：先建立主线，不追求一次记住

第一次读每章时，先抓主线。不要在每个术语上停太久，也不要因为某个工具不熟就完全中断。底层知识需要循环学习，很多概念第一次只能获得轮廓，做完实验后再回来会清楚很多。

做：每章至少运行一个实验

只读不做会产生错觉。你以为自己懂 ABI，直到手写汇编忘记保存 `rbx`；你以为自己懂 benchmark，直到发现循环被删除；你以为自己懂对象布局，直到 `sizeof` 和预期不同。实验是让错觉破裂的工具。

查：遇到边界再查资料

不要在还没有问题时查一整天资料。更有效的方式是先做实验，遇到具体边界再查。比如看到 `lea`，再查它和地址计算、标志位的关系；看到 `perf_event Paranoid` 权限错误，再查 Linux perf 权限；看到栈对齐崩溃，再查 System V ABI 对 `rsp` 的要求。

写：用报告逼自己表达证据

写报告会暴露理解漏洞。你可能以为自己知道为什么快，但写到“证据”一栏时发现没有看汇编；你可能以为知道输入规模，但写到 bytes 时发现没有计算工作集。报告不是作业负担，而是把模糊直觉变成清晰推理的工具。

复盘：把错误变成规则

每周至少复盘一次。列出本周最重要的三个错误判断：哪次以为是硬件问题，实际是 benchmark 写错；哪次以为是 ABI 问题，实际是对象生命周期；哪次以为是编译器 bug，实际是未定义行为。把错误写成规则，下次优先检查。

核心思想

底层能力不是靠一次读懂建立的，而是靠反复把源码、工具输出、机器状态和报告连接起来。读懂只是第一步，能复现、能解释、能复盘才算掌握。

把底层知识连成因果链

学习底层系统时，最重要的不是记住更多术语，而是能把术语连成因果链。一个事实如果不能进入因果链，就很难用于解释真实问题。例如“CPU 有 cache”只是孤立知识；“数组连续访问通常比随机访问更容易利用 cache line，因此在同样算法复杂度下可能有更高吞吐”才是可以用于分析的因果链。再进一步，“这个 benchmark 的工作集从 32 KiB 增长到 64 MiB 后吞吐下降，可能是因为工作集越过 cache 容量；需要用输入规模扫描和 PMU 事件验证”就是工程化因果链。

本书要求你在每个概念后面追问五个问题：

1. 它属于哪一层：语言、编译器、ABI、ISA、操作系统、微架构还是测量。
2. 它约束了谁：程序员、编译器、链接器、加载器、CPU 还是测量者。
3. 它能解释什么现象：崩溃、错误结果、汇编形态、性能差异还是工具输出。
4. 它需要什么证据：源码、命令、反汇编、调试器、计数器、样本数据还是规范文档。
5. 它有什么边界：在哪些平台、优化级别、输入范围或实验协议下成立。

这五个问题能把“知道一个词”变成“会使用一个概念”。比如“ABI”这个词，如果只解释成“调用约定”，还不够。你要知道它属于函数和模块边界，约束调用者和被调用者，能解释参数寄存器、返回值、栈对齐、callee-saved 保存、名字修饰和动态库兼容，需要通过函数声明、反汇编、符号表和 ABI 文档验证，边界则包括平台、编译器、语言类型和链接模式。

因果链要允许被推翻

工程分析中的因果链不是作文，而是可被证据推翻的假设。你可以先假设“这个循环慢是因为随机访问导致 cache miss”，但随后要设计证据：改变访问模式、扫描工作集大小、查看反汇编、收集 cache miss 相关事件、比较顺序访问 baseline。若证据不支持，就要修改假设。不能因为某个解释听起来高级，就把它固定成结论。

高质量学习笔记应保留这种修正过程。例如：


```

Initial hypothesis:
    random access is slower because of cache misses

Evidence:
    sequential and random versions have similar objdump shape
    random version slows down only when data exceeds L2-sized range
    perf stat shows more cache misses for large sizes

Revised conclusion:
    cache behavior explains the large-size gap, but small-size gap is likely
    dominated by loop overhead and branch/data dependency differences

```

这样的笔记比直接写“随机访问 cache miss 多”更有价值，因为它显示了证据如何约束结论。

第一册的知识地图

第一册虽然只有十二章，但覆盖了从源码到可信测量的完整基础链路。可以把它看成六个连续区域。

第一区域是系统总图。第 0 章建立抽象层和学习方法，第 1 章把程序从源码带到进程，第 2 章解释 CPU 如何把指令作为状态变化执行。这一区域回答“程序如何成为正在运行的东西”。

第二区域是数据和工具。第 3 章解释对象、内存、指针、布局、对齐和别名，第 4 章解释 Linux 工具、构建、调试和报告。这一区域回答“程序状态如何被表示、观察和复现”。

第三区域是汇编阅读。第 10 章讲汇编语法、符号、section、寻址和机器码，第 11 章讲整数指令、flags、扩展、位运算和地址计算，第 12 章讲控制流、循环、调用和 switch。这一区域回答“如何从机器码恢复数据流和控制流”。

第四区域是二进制接口。第 13 章讲 System V ABI，第 14 章讲 C++ 与汇编互操作。这一区域回答“函数、模块、语言边界如何共享同一个二进制合同”。

第五区域是测量。第 15 章讲可信测量的基本原则，第 16 章讲 benchmark suite 设计。这一区域回答“如何把运行时间变成可审查证据，而不是一次性数字”。

第六区域是后续第二册的入口。AI/HPC/SIMD/微架构优化不在第一册展开，但第一册为它们准备了必要前提：你已经能读汇编、理解 ABI、解释内存布局、设计 benchmark、区分事实和推测。没有这些基础，第二册的高级优化会变成记技巧。

为什么章节编号保留跳跃

本项目中第一册保留了第 10 章到第 16 章这样的编号，不代表中间缺失正文。它反映了教材从更大规划中收缩到第一册基础卷的过程：旧规划里许多 AI、HPC、SIMD、微架构深挖章节被推迟到第二册，第一册只保留最基础、最先需要打通的链路。这样做比把所有主题浅浅讲一遍更合理。教材质量来自边界清楚，而不是目录看起来覆盖一切。

读者不需要担心“是不是少学了中间部分”。第一册的目标不是完整覆盖现代性能工程所有话题，而是让你具备进入那些话题的能力。只要能完成第一册的实验和报告，你就已经拥有阅读第二册高级内容的基本工具。

经典教材式学习：定义、模型、例子、反例、实验

一本严肃技术教材不能只给技巧。每个重要知识点都应该经历五步。

第一步是定义。定义要说清一个词是什么，不是什么，属于哪一层。例如“进程是操作系统管理的运行实例，不等于可执行文件，也不等于源码”。定义不清，后面所有解释都会漂。

第二步是模型。模型把复杂对象简化成可以推理的结构。例如把 CPU 先看成状态机，把内存先看成按字节寻址的大数组，把 benchmark 看成实验系统。模型不是现实全部，但能让初学者开始推导。

第三步是例子。例子要足够小，能完整分析。例如用 `add2` 观察参数寄存器，用 `sum_u64` 观察循环，用小结构体观察 padding。例子太大，读者只能相信结论；例子足够小，读者能亲手复查。

第四步是反例。反例用于打破过度简化。例如 `lea` 不一定取地址，栈槽不一定等于局部变量，运行一次更快不等于优化成立，硬件能执行不代表 C++ 定义良好。没有反例，读者容易把入门模型当成绝对真理。

第五步是实验。实验把知识变成证据。你要运行命令、保存输出、解释差异、写报告。只读不做，底层知识会停留在词语层；做了不写，经验又难以复用。第一册每章都安排实验，就是为了形成这个闭环。

怎样判断一个解释是否“教材级”

教材级解释至少满足四个条件。第一，可定位：读者知道这个解释属于哪一层。第二，可推导：读者能跟随步骤从前提到结论。第三，可验证：读者知道用什么命令或实验检查。第四，可限界：读者知道它在哪些情况下不成立。

例如“-03 比 -00 快”不是教材级解释。更好的解释是：-00 主要服务调试可见性，常保留栈槽和直接源语句形态；-03 主要服务优化后的可观察行为，可以删除无用计算、内联、展开、向量化和重排；是否更快需要对具体程序、编译命令、输入和二进制进行测量。这个解释有层次、有机制、有验证方式，也有边界。

学习中的最低数学和硬件直觉

第一册不要求读者先学完数字电路、操作系统、编译原理和概率统计，但需要一些最低直觉。

数学方面，你要熟悉二进制、十六进制、幂、对数、比例、平均数、中位数、百分位数、简单集合和函数映射。算法复杂度要理解 $O(n)$ 、 $O(n \log n)$ 、 $O(n^2)$ 的增长差异，但也要知道复杂度不等于机器时间。一个 $O(n)$ 算法可能因为随机访问、分支 miss、内存带宽或常数巨大而慢；一个小规模 $O(n^2)$ 算法可能因为数据都在 cache 中而足够快。

硬件方面，你要知道 CPU 不是一次只做一条指令的简单解释器。它会取指、解码、执行、访存、写回和提交；它有寄存器、flags、cache、TLB、分支预测和流水线；它可能乱序和推测执行，但最终要维持架构语义。第一册不会要求你背微架构表，但会反复提醒：机器成本来自依赖、内存、分支、前端、执行资源和系统噪声共同作用。

统计方面，你要知道一次测量不是结论。样本有分布，分布有中心和尾部；噪声可能来自随机因素，也可能来自实验设计偏差；两个版本差异小于波动时，不应强行宣布胜负。性能工程不需要一开始使用复杂统计，但必须尊重基本不确定性。

学习输出的质量等级

为了避免“看过”和“学会”混淆，可以把学习输出分成四级。

第一级是复述。你能说出概念定义，例如“rdi 是 System V AMD64 中第一个整数参数寄存器”。这只是入门。

第二级是解释。你能结合例子说明机制，例如“这个函数第一个参数是指针，所以入口 rdi 是 base；mov eax, [rdi] 从该地址读取 4 字节返回”。这说明你能读简单代码。

第三级是诊断。你能面对异常现象提出假设和证据，例如“返回值正确但调用者随后崩溃，怀疑手写汇编破坏了 callee-saved 寄存器；需要在调用前后记录 rbx 等寄存器”。这说明你能处理真实问题。

第四级是设计。你能主动设计接口、实验和报告，例如“这个汇编 kernel 用 C++ wrapper 检查参数，内部只接受指针和长度；benchmark 保存 raw CSV、summary、objdump 和环境，并把结论限制在 warm-cache 单线程场景”。这说明你能把知识变成工程资产。

第一册的目标是让读者至少达到第三级，并开始具备第四级能力。后续第二册的高级优化，只有在这个基础上才有意义。

综合案例：一次性能问题应该怎样被拆开

假设你接到一个问题：“新版本比旧版本慢了 20%。”这句话本身不是一个可处理的问题，因为它没有说明哪一个程序、哪一个输入、哪一个构建、哪一个指标、哪一台机器、哪一段路径。第一册训练的正是把这种模糊说法拆成可验证问题。

第一步，确认对象。新版本和旧版本分别对应哪个 Git 提交？工作区是否有未提交修改？运行的是哪个可执行文件？动态库是否来自同一个目录？编译器和参数是否一致？如果这些问题没有回答，所谓“新旧版本”只是口头标签，不是工程对象。

第二步，确认正确性。两个版本是否对同一输入产生同一结果？是否使用同一个 reference？是否覆盖边界输入？如果新版本少处理了尾部、跳过了错误检查、使用了未定义行为，它可能更快或更慢，但这种性能比较没有意义。正确性是性能的前提，不是性能之后才补的手续。

第三步，确认测量协议。计时区域包含什么？是否把输入生成、内存分配、打印、校验放进 timed region？重复次数和 warmup 是多少？raw samples 是否保存？如果只有一次运行时间，就只能说观察到一次结果，不能说形成性能结论。

第四步，确认二进制。热点函数是否还存在？是否被内联、展开、向量化、删除、替换成库调用？当前机器是否走了某个 ISA dispatch 路径？如果没有反汇编或符号证据，就不知道测到的是源码意图、优化后热路径，还是完全不同的实现。

第五步，建立假设。若新版本多了随机访问，可能是 cache 或 TLB；若多了间接调用，可能是内联失败或分支目标预测；若多了小对象分配，可能是 allocator 和 page fault；若汇编变成标量，可能是别名信息阻止向量化。每个假设都必须能对应下一步证据。

第六步，设计对照。改变输入规模可以区分固定开销和带宽瓶颈；改变数据分布可以区分分支预测和算术成本；固定 CPU 核心可以减少调度噪声；比较 objdump 可以确认机器码变化；使用 perfstat 可以为分支、cache 或指令数假设提供硬件证据。

最后，写出有边界的结论。合格结论不是“新版本慢 20%”，而是类似：“在这台机器、这个编译器、这个输入规模和 warm-cache 协议下，新版本 median ns/element 增加 20%。正确性检查通过。反汇编显示新版本热循环多了一次间接 load，规模扫描显示大工作集退化更明显。当前假设是内存访问局部性变差；尚未在另一台机器复测。”

核心思想

底层分析的基本动作是把一句模糊现象拆成对象、正确性、协议、二进制、假设、对照和结论。这个动作贯穿第一册所有章节。

自测清单：进入下一章前应能回答什么

进入第 1 章之前，读者应该能回答下面问题。如果答不上来，不需要停在本章很久，但要在后续学习中持续补齐。

1. 源码、目标文件、可执行文件、进程和 benchmark 报告分别是什么。
2. C++ 语言规则、ABI、ISA、微架构和操作系统分别约束什么。
3. 为什么同一段源码在 -O0 和 -O3 下可能完全不同。
4. 为什么“能运行”不等于“定义良好”，也不等于“可以谈性能”。
5. 为什么一次运行时间不能直接成为性能结论。
6. 为什么反汇编能证明机器码形态，但不能单独证明热路径频率。
7. 为什么 AI/HPC/SIMD 优化应放在第二册，而不是在基础未稳时提前展开。
8. 一份底层实验报告至少需要哪些证据。

这些问题不是考试题，而是学习导航。后面每章都会反复回到它们。你学会的不是某个 isolated trick，而是一套把程序从语言层带到机器层、再带回工程结论的方法。

背景补课：把缺口变成路线，而不是焦虑

很多读者进入底层学习时，会发现自己同时缺 C++、操作系统、汇编、编译器、硬件和统计。最容易出现反应是焦虑：觉得必须先把所有前置课程补完，才能开始读本书。这个想法会拖慢学习。更有效的方法是把缺口变成路线：先建立主线，遇到具体问题再精确补课。

如果 C++ 语义不稳，优先补对象生命周期、值类别、类型转换、未定义行为、别名、存储期、异常和 RAII。不要一开始陷入模板元编程或标准库实现细节。第一册最常用的是“对象何时存在”“某次访问是否合法”“编译器能假设什么”。这些语义直接影响汇编形态和优化边界。

如果操作系统不稳，优先补进程、虚拟地址空间、文件描述符、动态链接、权限、信号、调度和 /proc。不需要先写内核。第一册只要求你知道普通程序不是独占机器，地址是进程内虚拟地址，动态库和环境会改变运行路径，调度和缺页会影响测量。

如果硬件不稳，优先补位、字节、补码、寄存器、load/store、cache line、TLB、分支预测和流水线直觉。不需要先背每个 CPU 的端口表。第一册只要求你能从机器码恢复状态变化，并知道性能不是由助记符单独决定。

如果编译器不稳，优先补翻译单元、前端语义检查、IR、优化、后端、汇编器、链接器和 ABI。不要一开始就试图理解所有优化 pass。第一册最重要的是知道编译器不是逐行翻译器，而是在语言合同内重写程序；你看到的汇编是优化后机器程序，不是源码的简单影子。

如果统计不稳，优先补样本、分布、中位数、百分位、噪声、偏差、置信范围和实验变量。第一册不会要求复杂统计证明，但要求你知道一次测量不是结论，平均值不是万能，异常值不能偷偷删

除，实验协议必须写清。

这种补课方式有一个原则：补到能解释当前现象为止。比如你看到 `movsxd`，就补符号扩展和整数类型；看到 `undefinedreference`，就补声明、定义和链接；看到 `benchmark` 波动，就补样本和噪声。不要把补课变成逃避实验的借口。

学习复盘：每周应该留下什么

如果把本书当成长期训练，每周至少留下四类产物。

第一类是概念卡片。每张卡片只解释一个概念，例如 ABI、red zone、RIP-relative、dead-code elimination、cache line、branch miss。卡片必须包含定义、作用、证据和边界。只写一句中文解释不够。

第二类是命令记录。把本周最重要的构建、反汇编、调试、测量命令保存下来。以后遇到同类问题，可以直接复用。命令记录要包含工作目录和输出文件，不要只写裸命令。

第三类是错误复盘。每周挑一个错误判断，写清“我原来以为是什么”“证据如何推翻”“正确规则是什么”。底层能力增长最快的地方，往往是错误被精确命名的地方。

第四类是小报告。报告不需要长，但要完整：问题、方法、观察、解释、限制、下一步。只要坚持写，几周后你会发现你不再用模糊词说话，而是自然地说“这个结论缺少二进制证据”“这个差异小于样本波动”“这个调用点跨了 ABI 边界”。

这套复盘方式是第一册质量要求的一部分。教材写得再细，如果读者不把知识变成产物，底层能力仍然会停留在阅读体验上。真正的掌握来自可复现、可解释、可修正。

课堂讲解：从“会写代码”到“会解释代码”

很多学生已经会写 C++ 项目，却仍然读不懂底层现象。原因是写代码和解释代码需要不同能力。写代码时，你主要在源码层思考：函数、类型、对象、算法、模块。解释代码时，你必须跨层：源码如何被编译，目标文件如何被链接，进程如何加载，指令如何执行，测量如何采样。两种能力相关，但不是同一种。

例如你写：

```
return a[i] + b[i];
```

会写代码的人知道这是两个数组元素相加。会解释代码的人还会问：`i` 是否在范围内？`a` 和 `b` 是否可能别名？元素类型多宽？地址公式是什么？`load` 是否连续？编译器是否能向量化？结果是否会溢出？这段表达式在 `-O3` 下是否仍以两个 `load` 和一个 `add` 存在？`benchmark` 是否真的执行了它？

这不是把简单问题复杂化，而是把隐含前提显式化。大多数底层 bug 和性能误判，都来自隐含前提不成立。程序员以为数组不重叠，编译器不知道；程序员以为下标非负，输入不是；程序员以为测了循环，优化器删了；程序员以为调用了新库，动态链接器加载旧库。解释能力就是发现这些前提。

第一册每一章都在训练这种能力。第 1 章让你解释程序身份；第 2 章让你解释指令状态；第 3 章让你解释地址和对象；第 4 章让你解释工具证据；第 10 到 14 章让你解释汇编和 ABI；第 15 到 16 章让你解释时间数字。所有章节共同目标是：以后看到任何底层现象，都能问出正确的下一步问题。

课堂讲解：为什么本书不先讲最炫的优化

AI、SIMD、GEMM、attention、cache blocking、AVX-512、VNNI 这些主题很吸引人，也确实重要。但如果在基础未稳时直接讲，读者容易把高级优化学成口诀。例如“用 SIMD 会快”“blocking 提高 cache locality”“branchless 更好”“AVX-512 更强”。这些话在某些条件下成立，在另一些条件下不成立。

没有第一册基础，读者无法判断条件。SIMD 需要知道数据布局、对齐、尾部、ABI、编译器和 benchmark；blocking 需要知道 cache line、工作集、内存带宽和循环结构；branchless 需要知道控制依赖、数据依赖和合法提前执行；AVX-512 需要知道 ISA dispatch、频率影响和目标机器。高级优化不是孤立技巧，而是基础链路上的组合决策。

所以第一册先讲基础，不是降低目标，而是提高上限。基础越稳，第二册讲 AI/HPC 时越能深

入。到那时，读者看到一个矩阵乘法 kernel，不只会看循环，还能看 ABI、汇编、寄存器、内存布局、cache、测量协议和二进制证据。这样的学习才不会停留在“照抄优化模板”。

教材路线：把困难拆成可检验的小层

经典系统教材不会把复杂系统直接说成“很复杂”，而是把复杂性拆成少数稳定问题：状态是什么，状态如何改变，谁有权改变，观察点在哪里，错误会以什么形态暴露。本书也按这个方式写。第一册不要求读者立刻掌握所有现代处理器细节，而要求读者先把每个问题都落到可检验的小层上。

例如“这个循环慢”不是一个可以直接处理的问题。第一层要问算法做了多少逻辑工作，输入规模是否相同，结果是否正确。第二层要问编译器是否生成了预期代码，热循环是否仍在，是否被内联、展开、向量化或删除。第三层要问指令中有多少 load、store、分支和依赖链。第四层才讨论 cache、分支预测、执行端口、前端带宽和操作系统扰动。若前面几层没有确定，后面越高级的解释越容易变成故事。

同样，“这段汇编是什么意思”也不能只查助记符。要先确定函数边界、入口约定、参数来源、寄存器保存、栈帧形态、基本块和跳转边。然后才读每条指令如何改变寄存器、内存和 flags。最后再把机器状态映射回 C++ 层的对象、表达式、分支和循环。这个顺序看似慢，其实是最快的路径，因为它避免了在错误层次上反复猜。

本书的正文、实验和作业都围绕这种拆解方式组织。正文负责给出模型，实验负责把模型落到真实工具，作业负责让读者独立复现和表达。若某一章读完后只记住了名词，却不能写出一份带命令、输出和解释的小报告，就说明学习还停在表面。

从任务反推知识，而不是从术语堆开始

很多底层学习会失败，是因为起点是一长串术语：ELF、ABI、TLB、BTB、ROB、uop、PMU、SIMD。术语当然重要，但术语本身不是能力。能力来自任务。你要定位一次崩溃，就会自然需要寄存器、栈、地址映射和反汇编；你要解释一次链接失败，就会自然需要符号、重定位、库搜索路径和动态加载；你要判断一次优化是否成立，就会自然需要正确性基准、二进制证据、样本分布和环境记录。

因此，学习时可以把每个术语都放回一个任务。问三个问题：它解决哪个问题，它提供什么证据，它的边界是什么。比如 ABI 不是“函数调用规则”这个短句就够了。ABI 解决的是独立编译、链接和调用的问题；它提供参数位置、返回值位置、寄存器保存、栈对齐、类型传递、符号可见性等证据；它的边界是同一平台和调用约定内成立，跨平台、跨语言、跨编译器选项时必须重新核对。

再比如 cache 不是“内存加速器”。cache 解决的是处理器和主存速度差距；它提供局部性、cache line、工作集、容量、冲突、预取等解释维度；它的边界是无法单独决定性能，因为控制流、依赖、TLB、带宽、写分配和并发干扰都会参与。把术语这样放回任务，读者就不会把名词当成答案。

规格书、工具手册和实验之间的关系

底层知识有三类来源。第一类是规格书和标准，它们定义合同。C++ 标准定义语言层允许什么，ABI 文档定义二进制接口怎样传值，ISA 手册定义机器指令的架构语义。第二类是工具手册，它们告诉你如何观察。编译器、链接器、调试器、反汇编器、性能工具都有选项和输出格式。第三类是实验，它把前两类知识放到你的机器、你的编译器和你的代码上验证。

三者不能互相替代。只看规格书，容易不知道现实工具如何呈现证据；只看工具输出，容易把实现细节误认为通用规则；只做实验，容易把偶然结果当成原理。合格的学习方式是：用规格书确定应然边界，用工具手册确定观察方法，用实验确认当前环境中的具体事实。

例如 System V AMD64 ABI 规定前几个整数参数放入哪些寄存器，但你在反汇编中可能看不到这些寄存器直接使用，因为优化器已经内联、常量传播或删除了函数。此时不能说 ABI 失效，也不能说工具错了。正确处理是构造一个能保留调用边界的实验，关闭内联或使用外部目标文件，然后再观察寄存器和栈。原则、工具和实验必须相互校准。

如何把第一册学成长期能力

第一册的目标不是让读者完成一轮阅读，而是建立一套以后可以反复使用的分析习惯。这个习惯可以概括为五句话：先定义问题，再确认身份；先找正确性，再谈性能；先看二进制，再解释优化；先保存原始证据，再写结论；先说明边界，再推广经验。

先定义问题，是为了防止把多个问题混在一起。编译失败、链接失败、启动失败、运行崩溃、结果错误、性能波动，是不同类别的问题。每类问题需要的工具不同。确认身份，是为了知道你分析的

到底是哪份源码、哪个目标文件、哪个可执行文件、哪个进程和哪次运行。没有身份，证据无法归属。

先找正确性，是因为错误程序没有性能结论。一个 benchmark 如果结果被优化器删除，或者 candidate 算错，或者输入和 baseline 不一致，那么速度数字越漂亮越危险。先看二进制，是因为性能发生在机器码上。源码能表达意图，二进制才表达这次构建实际执行什么。

保存原始证据，是为了让结论可复核。命令、环境、反汇编、原始样本、工具输出、报告应该在同一个结果目录中。说明边界，是为了防止经验滥用。你可以说“在这台机器、这个编译器、这个输入分布、这个版本上，A 比 B 快”，但不能随手写成“A 总是更快”。

学习时应主动制造反例

真正稳固的理解往往来自反例。学习分支预测时，不要只测随机输入，也要测全真、全假、排序和偏斜分布。学习别名时，不要只写不重叠数组，也要故意传入重叠区间，观察为什么某些优化不能合法发生。学习 ABI 时，不要只写两个整数参数，也要写结构体、浮点、栈参数、变参和大返回值。学习 benchmark 时，不要只保留成功结果，也要制造会被优化删除、会算错、会被计时边界污染的失败例子。

反例的价值在于逼迫边界出现。一个概念如果只在最顺利的例子里成立，读者很容易把它学成口号。边界一出现，定义才会变清楚：什么是未定义行为，什么是实现定义，什么是 ABI 规定，什么是编译器选择，什么是微架构现象，什么是测量噪声。高质量教材必须把这些边界说出来，学习者也必须实验中亲手碰到它们。

把笔记写成可迁移模板

学习笔记不要只写“今天学了什么”。更好的写法是模板化。比如阅读汇编的模板可以固定为：函数身份、构建命令、优化级别、入口参数、关键基本块、寄存器角色、内存访问、控制流边、源码对应、无法确认的部分。调试崩溃的模板可以固定为：复现命令、信号、崩溃地址、当前指令、寄存器、栈、地址映射、最近一次写入、最小修复。benchmark 模板可以固定为：问题陈述、baseline、candidate、输入矩阵、计时协议、正确性、样本、汇编、硬件事件、结论限制。

模板的好处不是让表达机械化，而是降低遗漏概率。底层问题常常不是因为不知道某个高级技巧而失败，而是因为漏了一个普通条件：忘了记录编译参数，忘了保存反汇编，忘了确认符号来自哪个库，忘了检查结果正确性，忘了交错运行。模板让这些基础动作变成习惯。

第一册的学习产物应该能被别人复查

本书不把“读懂了”当作最终标准。读懂是主观体验，可复查产物才是工程能力。每学完一章，读者都应该留下至少一种产物：一份小报告、一个实验目录、一段带注释的反汇编、一张控制流图、一份 benchmark raw data、一组构建命令或一个失败复盘。这些产物让学习从记忆变成证据。

可复查产物有三个要求。第一，别人知道你做了什么。命令、输入、构建和环境要保存。第二，别人知道你看到了什么。原始输出、反汇编、样本数据和调试记录要保留。第三，别人知道你为什么这样解释。报告要区分事实、推测和限制。只有这三点同时满足，产物才不只是个人笔记。

这套要求会贯穿全书。第 1 章的产物是程序身份和地址空间报告；第 2 章是指令状态追踪；第 3 章是布局和内存访问报告；第 4 章是工具证据链；第 10 到 14 章是汇编、整数、控制流、ABI 和互操作审计；第 15 到 16 章是可信测量和 suite 报告。每章产物合起来，就是读者自己的底层工程档案。

复查比炫技更重要

底层学习很容易陷入炫技：写一段奇怪汇编、背很多微架构术语、跑一个看起来很快的 benchmark。炫技不一定没价值，但它不能替代复查。一个能复查的普通实验，比一个不能解释的惊人数字更有教学价值。工程项目也一样：可维护的保守实现，常常比无法审计的短期技巧更可靠。

复查要求会迫使读者诚实面对不确定性。你可能发现某个结论缺少二进制证据，某个 benchmark 没有 raw data，某个 ABI 推导没有调用点验证，某个性能解释没有输入分布。这不是坏事。发现缺口，就是学习在发生。

学习中的三种边界意识

第一种是语义边界。C++ 语言、ABI、ISA、操作系统和硬件实现各有合同。语言允许优化器假设未定义行为不发生；ABI 规定函数边界状态；ISA 规定指令对架构状态的效果；操作系统规定进程资源和权限；硬件实现决定成本。把一层的事实拿到另一层当规则，是许多错误的来源。

第二种是证据边界。工具能证明什么，不能证明什么，要说清楚。反汇编能证明机器码形态，不能单独证明源码意图；GDB 能显示一次运行的现场，不能自动证明根因；benchmark 能显示某条件下的样本，不能自动推广到所有机器；PMU 能支持硬件假设，不能代替代码语义分析。

第三种是适用边界。一个优化在某输入规模有效，不代表所有规模有效；在某 CPU 上有效，不代表所有 CPU 有效；在 warm-cache 下有效，不代表 cold-cache 有效；在内部 kernel 上有效，不代表 public API 端到端有效。高质量结论必须写适用边界。边界写得清楚，经验才可以迁移。

从第一册进入第二册应具备什么

第二册会讨论更深入的现代 CPU、SIMD、HPC 和 AI 算子优化。进入那些主题前，读者至少应具备六项基础能力。第一，能从源码构建出可审计的二进制，并确认运行身份。第二，能读懂基本汇编，恢复寄存器、内存和控制流。第三，能解释对象布局、地址公式、别名和生命周期对优化的影响。第四，能从函数声明推导 ABI，并写出 C++ 与汇编互操作边界。

第五，能设计可信 benchmark：正确性、输入矩阵、计时区域、样本、二进制证据和环境记录都要完整。第六，能把性能结论写成有限、可复查、可推翻的工程报告。没有这些能力，第二册的高级优化会变成模板堆砌；有了这些能力，读者才能真正判断 SIMD、blocking、prefetch、dispatch 和 AI kernel 什么时候成立。

因此，第一册不是“基础但简单”的书。它是在为更难的问题建立审查能力。越高级的优化，越需要基础合同清楚。一个 attention kernel 慢了，可能是算法、布局、cache、SIMD、ABI、动态分派、测量协议或输入分布任何一层的问题。第一册训练的，就是把这些层拆开并逐层验证的能力。

教师视角：本书应如何被讲授

如果把本书用于课程或训练营，建议不要按“讲完概念再统一实验”的方式组织。更好的节奏是每个主题都包含短讲、现场观察、独立复现和报告复盘。底层知识抽象层多，学生只有在工具中看到证据，概念才会稳定。

课堂上可以反复使用同一套路：先给一个错误直觉，例如“这个 if 一定慢”“这个指针就是整数地址”“Release 崩溃一定是优化器 bug”；再给一个小实验；然后用反汇编、GDB、readelf 或 benchmark 推翻或修正直觉；最后让学生写出结论边界。这样教学比单纯罗列术语更接近真实工程。

评分也应重视证据质量。代码能跑只是最低要求；报告是否保存命令，是否说明环境，是否区分事实和推测，是否写出限制，是否能复现，才是底层能力的核心。第一册的作业设计，应让学生习惯用证据说话。

读者自学时的节奏控制

自学时不要一次性追求读完所有章节。更有效的方法是按小循环推进：读一节，做一个最小实验，写一段解释，再读下一节。遇到不懂的术语，不必立刻跳到最深资料；先问它在当前任务中解决什么问题。等实验需要时，再查手册或规范。

每周可以选一个主题做深一点。例如本周只把“函数调用”做清楚：写 C++ 函数、看 call/ret、看栈、看 ABI、写一个手写汇编函数、测一次调用开销。下一周再做“内存布局”。这种纵向小闭环，比横向快速扫过所有名词更有效。

学习进度不应只用页数衡量，而应看产物。若一周只读了少量内容，但写出了一份可复查报告，这比读很多页却没有证据更有价值。

贯穿训练：同一个问题在十二章中如何升级

为了防止知识变成孤立章节，读者可以选一个贯穿问题反复升级。例如“一个数组求和函数为什么慢”。第 1 章先确认你运行的是哪份源码、哪份二进制、哪个进程；第 2 章追踪 CPU 执行了哪些 load、add、branch；第 3 章推导数组元素宽度、地址公式、对齐和工作集；第 4 章保存构建、反汇编、调试和环境证据；第 10 章读汇编语法；第 11 章检查长度、下标和累加溢出；第 12 章画循环

CFG；第 13 章确认参数和返回值 ABI；第 14 章把 C++ wrapper 和汇编 kernel 分层；第 15 章设计可信计时；第 16 章把它放进 suite。

这个训练的重点不是求和本身，而是同一个现象在不同层次上有不同问题。源码层问算法和语义，二进制层问实际机器码，硬件层问依赖和资源，系统层问运行环境，测量层问信度和限制。若读者能把一个小问题沿十二章完整走一遍，就具备了处理更大问题的框架。

反过来，如果某一层缺失，结论就会变弱。没有程序身份，反汇编可能来自旧二进制；没有内存布局，cache 解释可能错误；没有 ABI，汇编参数可能读错；没有 correctness，速度数字无效；没有 raw data，报告不可复查。第一册的价值，就是让这些缺口变得可见。

最终自评：能否独立完成一份底层分析

读完第一册后，读者应能独立完成一份底层分析报告。报告题目可以很小，例如“比较两个整数求和实现”“解释一次链接失败”“分析一个短路保护函数的汇编”“搭建一个 C++ 调汇编的安全 wrapper”。题目小没有关系，关键是报告完整。

完整报告应包含问题、环境、源码、构建、二进制、工具输出、推理、实验、限制和下一步。它不一定要很长，但每个结论都应有来源。若写到“可能是 cache”，就要说明访问模式和样本；若写到“参数在 rdi”，就要说明调用约定；若写到“优化器删除了代码”，就要给反汇编或符号证据；若写到“版本 B 更快”，就要给 correctness、raw data 和统计口径。

这份自评比选择题更能检验学习效果。底层工程没有单一答案，只有证据链是否充分。能把证据链写清楚，说明读者已经从“看懂教材”走向“能处理问题”。

第一册的质量底线

第一册的质量底线不是篇幅，而是每个重要判断都能回到证据。讲 C++，要能回到语言语义和构建产物；讲 CPU，要能回到机器码和状态变化；讲内存，要能回到对象、字节和地址公式；讲 ABI，要能回到调用者和被调用者的共同合同；讲性能，要能回到正确性、输入、样本和环境。若某段内容只给结论不给路径，它就不符合本书的写法。

这条底线也适用于读者自己的学习。不要满足于“我知道这个词”。要能回答：它解决什么问题，它在哪一层成立，它需要哪些证据，它有哪些反例。比如“branchless”不是高级同义词，它只是在某些条件下把控制选择转成数据选择；如果候选计算不安全，或者原分支很好预测，branchless 可能错误或更慢。这样的边界意识，才是底层能力。

第一册最终要训练的是一种表达方式：少用“应该”“可能快”“大概是”，多用“在这些条件下，证据支持这个解释；这些条件外还没有证明”。这种表达看起来保守，却能让工程协作更高效。因为别人知道哪些部分可以信，哪些部分需要继续验证。

为什么这本书要求反复写报告

报告不是形式主义。底层知识如果只停留在脑中，很容易在真正排查时散掉。报告迫使你把问题、对象、证据、解释和限制排成顺序。排不出来，说明理解还不稳定。很多时候，写报告的过程本身就会发现漏洞：忘了记录编译参数，忘了确认动态库路径，忘了保存 raw data，忘了说明 signed 还是 unsigned。

报告也是团队沟通的单位。一个人说“我测过了”，别人无法复查；一份报告给出命令、输入、反汇编、样本和限制，别人可以继续推进。底层工程依赖这种可交接性。越接近机器边界，越不能只靠口头经验。

因此，第一册反复要求写小报告。它不是为了增加负担，而是训练一种表达肌肉：把复杂现象压缩成可审查证据链。到第二册进入更复杂的 SIMD、cache blocking、AI kernel 时，这种能力会比记住某个技巧更重要。

第一册读完后的第二遍阅读

第一遍阅读的目标是建立主线，第二遍阅读的目标是发现缺口。读者可以不再按页码顺序读，而是按问题回读。比如你想解释一次崩溃，就回读程序身份、内存生命周期、工具证据和控制流保护；你想写一个汇编 kernel，就回读整数范围、ABI、互操作和测量；你想做 benchmark，就回读正确性、输入矩阵、二进制证据和报告模板。第二遍阅读应带着自己的问题进入章节，而不是把章节当成孤立知识。

第二遍还要检查自己的旧报告。把第一遍写的实验报告拿出来，问它是否保存了完整命令，是否记录了编译器和参数，是否能重新生成同一段反汇编，是否有 raw data，是否写明限制。很多早期报告会显得粗糙，这正说明能力在增长。不要删除这些粗糙报告，可以在旁边写复盘：当时缺了哪些证据，现在会怎样改写。

第一册真正完成的标志，不是把所有术语背下来，而是遇到新问题时能自然地拆层。先确认对象，再证明正确性，再看二进制，再提出性能假设，再设计测量，再写限制。只要这条路径稳定，后面进入更复杂的主题时，读者就不会被术语和工具淹没。

本章实验：建立你的机器档案

本实验不是为了追求速度，而是为了建立后续所有实验都要引用的机器档案。请在 `results/machine-profile/` 下保存报告。

任务 1：记录 CPU 和系统信息

运行：

```
lscpu
cat /proc/cpuinfo | sed -n '1,80p'
cat /proc/meminfo | sed -n '1,40p'
uname -a
gcc --version
clang --version
ldd --version
```

报告中至少记录：

- CPU 型号、核心数、线程数。
- L1d/L1i/L2/L3 cache 信息。
- 支持的 ISA 扩展，例如 SSE、AVX、AVX2、AVX-512、FMA、VNNI。
- 内存总量。
- Linux kernel 版本。
- GCC/Clang 版本。

任务 2：建立第一个源码到汇编证据链

创建 `labs/ch00_machine_map/sum.cpp`：

```
#include <stddef>
#include <stdint>

extern "C" std::int64_t sum_i32(std::int32_t const* a, std::size_t n)
{
    std::int64_t s = 0;
    for (std::size_t i = 0; i < n; ++i) {
        s += a[i];
    }
    return s;
}
```

分别生成未优化和优化后的汇编：

```
clang++ -std=c++20 -O0 -S -masm=intel sum.cpp -o sum_00.s
clang++ -std=c++20 -O3 -S -masm=intel sum.cpp -o sum_03.s
```

报告必须回答：

1. `-O0` 和 `-O3` 的循环形态有什么差异？
2. 参数 `a` 和 `n` 分别从哪些寄存器进入函数？
3. 汇编中哪里体现了 `sizeof(std::int32_t)==4`？
4. 编译器有没有展开或向量化？证据是什么？

任务 3：第一次测量但不急着下结论

写一个小 benchmark，测量 $n=1024$ 、 $n=1048576$ 、 $n=67108864$ 三种规模。每个规模运行多次，记录最小值、中位数和最大值。报告中必须写明为什么三种规模可能对应不同 cache 行为。

本章作业：写出你的第一份性能工程备忘录

习题与作业

提交一份不少于 1500 字的中文报告，题目为“从 C++ 到机器执行的第一张地图”。报告必须包含以下部分：

1. 画出从 C++ 源码到 CPU 执行的链路图，并用自己的话解释每一层的职责。
2. 用 `sum_i32` 例子解释源码、汇编、寄存器、内存访问之间的关系。
3. 说明 ISA 和微架构的区别，并举一个“同一条汇编在不同数据状态下性能不同”的例子。
4. 说明操作系统为什么会影响 benchmark。
5. 列出你当前最不熟悉的 10 个概念，并为每个概念写一句你准备如何验证它。

评分重点不是语言华丽，而是证据完整、层次清楚、没有把不同抽象层混在一起。

验收标准

完成本章后，你应该能做到：

- 画出源码到机器执行的完整粗粒度链路。
- 区分语言语义、编译器优化、ISA、微架构、操作系统和测量方法。
- 对一个简单循环提出至少三种可能性能瓶颈。
- 用汇编证据回答“编译器实际生成了什么”。
- 解释为什么一次运行时间不能直接当成性能结论。

后续章节会把这张地图逐层放大：先看源码如何变成进程，再看 CPU 如何执行指令、数据如何落在内存里，随后进入 x86-64 汇编、ABI、C++ 与汇编互操作，最后用可信 benchmark 方法把观察结果写成证据。内存层级、SIMD、GEMM 和 AI 算子属于第二册。

第 2 章

程序如何从源码变成正在运行的进程

问题与目标

刚学完 C/C++ 时，通常已经掌握三件基本操作：

1. 写一个 .cpp 文件。
2. 用编译器生成可执行文件。
3. 在终端运行它。

但这些操作背后的系统机制往往仍然模糊：

- 头文件到底是不是被“编译进来”。
- `g++main.cpp-oapp` 背后调用了哪些工具。
- `main` 为什么通常不是程序真正执行的第一条指令。
- 可执行文件里除了机器码还有什么。
- 操作系统如何把文件变成一个进程。
- 指针地址为什么每次运行可能不同。
- 这些机制为什么会影响性能测量。

核心思想

程序运行并不是把源码直接交给 CPU。源码要经过预处理、编译、优化、汇编、链接、加载和运行时启动，最后才变成 CPU 可以执行的机器指令。性能工程中的许多误判，都来自没有看见这条链路。

学习本章的正确方式

本章不是要求你背诵一串工具名称。预处理器、编译器、汇编器、链接器、加载器和运行时库当然各有细节，但第一阶段更重要的是掌握每一层的边界。所谓边界，就是这一层接收什么输入，产生什么输出，对下一层作出什么承诺，又有意隐藏了哪些信息。只记住命令而不理解边界，遇到真实问题时仍然很难定位原因。

学习这条路径时，可以把一个程序看成不断改变形态的对象。刚开始它是人写的源文本，带有注释、宏、头文件、类型名、函数名、模板和命名空间。经过预处理后，它变成一个更大的源文本，很多你没有亲手写的声明已经被展开进来。经过前端后，它不再只是字符，而是带有语法结构和类型含义的内部表示。经过优化后，很多源代码层面的形状会被打散、合并、删除或移动。经过后端后，它变成目标机器能表达的汇编动作。经过汇编器后，它变成带符号表和重定位信息的目标文件。经过链接器后，它变成操作系统能加载的可执行文件或共享库。最后，操作系统和运行时库把它放入一个进程环境中，准备栈、环境变量、动态库、线程局部存储和初始化逻辑，然后才把控制权交到用户代码附近。

这条路径上最容易出错的地方，不是术语翻译，而是把不同层的规则混为一谈。例如，C++ 说对象有类型和生命周期，但 CPU 执行的是读写寄存器和内存的指令；源代码里有一个函数名，优化后的机器码里这个函数却可能已经不存在；反汇编里显示一个地址，用户态程序看到的通常也不是物理地址；benchmark 源码里写了循环，优化器仍可能证明结果无用并把它删除。把这些层次分清，是后续学习汇编、ABI、缓存、SIMD 和性能计数器的基础。

心智模型

读本章时始终问四个问题：这一层的输入是什么，输出是什么，谁负责完成转换，转换后哪些信息被保留、哪些信息被丢失。只要这四个问题清楚，后面再看工具输出就不会把现象误认为本质。

源码、可执行文件和进程不是同一个东西

日常交流中常说“运行这个程序”。这句话方便，但不精确。至少有三个概念需要严格区分：源码、可执行文件和进程。

源码是程序员写下的文本。它描述的是语言层面的意图：声明变量，定义函数，创建对象，调用库，表达控制流。源码不是 CPU 的输入，甚至也不一定是编译器最终直接理解的形态，因为预处理会先展开头文件和宏。

可执行文件是存放在磁盘上的二进制文件。它包含机器码，也包含操作系统和加载器需要的元数据。它是静态对象：你不运行它时，它只是文件系统中的一段内容。它可以被复制，可以被校验，可以被反汇编，也可以被多个进程同时映射。

进程是一次运行中的实例。一个可执行文件可以被运行多次，每次运行都得到一个不同的进程。每个进程有自己的虚拟地址空间、栈、堆、文件描述符、环境变量、线程状态和内核维护的资源。两个进程可能来自同一个可执行文件，但它们的栈地址、堆对象、随机种子、文件描述符状态和调度历史都可以不同。

这个区分直接影响调试和性能分析。调试源码中的函数写法是一类问题；检查二进制里是否存在某个符号是另一类问题；解释某次运行为什么变慢，又是第三类问题。性能工程常常需要在三者之间切换：先在源码层提出假设，再在可执行文件层检查编译结果，最后在进程运行层观察时间、地址、缺页、调度和硬件事件。

常见误区

不要说“这个源码运行很慢”就停止分析。严格说，慢的是某次构建得到的二进制，在某个操作系统、某组动态库、某种输入、某个 CPU 频率和调度环境下的一次或多次运行。把上下文说清楚，不是形式主义，而是避免性能结论失真的基本要求。

阶段边界：每一步都在改变问题

从源码到进程不是一条单纯的“翻译”链路。每一步都在改变问题的表达方式。预处理阶段关心文本替换和条件选择；前端关心语法和语义；优化器关心等价变换；后端关心目标机器；汇编器关心指令编码和重定位记录；链接器关心符号解析和地址安排；加载器关心进程地址空间；运行时关心语言环境；CPU 关心指令语义和机器状态。把这几件事都叫“编译”会掩盖很多关键细节。

预处理器不理解完整的 C++ 语义。宏替换只是文本层面的机制，它不知道一个标识符是不是局部变量，也不知道一个表达式是否会造成未定义行为。宏因此很强，也很危险。一个宏可以让源码看起来像函数调用，却在展开后重复求值实参；一个条件编译分支可以让同一份源码在不同机器上变成不同程序。

编译器前端的任务不是提高性能，而是先确定这段文本在语言规则下是什么意思。名字查找、重载解析、模板实例化、类型检查、访问控制、常量表达式判断、对象生命周期和异常规范，都是语义问题。只有当语义确定后，优化器才知道哪些变换是允许的。性能学习不能绕过语言语义，因为优化器的自由度正是从语言规则里来的。

优化器不是“尽量保留源码形状”的工具。它的目标是在保持可观察行为的前提下生成更好的程序。可观察行为在 C++ 中有专门含义，涉及 volatile 访问、输入输出、原子操作、函数调用副作用、异常、对象构造析构等。普通局部变量的中间值，如果最终不影响可观察行为，就可能被完全删除。因此，用没有结果消费的循环做性能测试，本质上是在测试优化器是否能看穿你的代码，而不是测试算法有多快。

后端把中间表示映射到具体目标。不同目标机器有不同寄存器数量、指令格式、寻址模式、调用约定和指令代价。即使源码和中间表示相同，面向 x86-64、AArch64 或 RISC-V 的后端也会做出不同选择。以后讨论性能时，必须把“语言层算法复杂度”和“目标机器上的实现代价”分开。

汇编器和链接器处理的是二进制组织问题。汇编文本中的标签、外部符号、局部跳转和数据段引用，很多在汇编阶段还没有最终地址。目标文件里会留下重定位记录，告诉链接器哪些位置以后需要修正。链接器把多个目标文件和库合成更大的二进制，解析符号，安排段和节，生成动态链接信息。你在反汇编里看到的调用目标、地址位移和符号名字，许多都是这个阶段安排后的结果。

加载器和运行时处理的是“把文件变成运行环境”的问题。加载不是把整个可执行文件逐字节复制到内存这么简单。现代操作系统通常把可执行文件和共享库的若干段映射到进程虚拟地址空间中，按权限设置页面，准备初始栈，传入参数和环境变量。动态链接器还可能在运行前或首次调用

时解析共享库符号。C++ 运行时会处理全局对象构造、异常机制、线程局部存储和退出时析构。所有这些都可能影响程序启动和第一次调用。

核心理念

阶段越往后，源码里的名字和结构就越不可靠；阶段越往前，机器上的代价和布局就越不可见。成熟的分析方式是在合适的层问合适的问题，而不是试图用一个层的直觉解释所有现象。

为什么经典教材要从这条链路讲起

经典系统教材通常不会一上来就讲某个炫目的优化技巧，而会从程序表示、机器级程序、链接、异常控制流、虚拟内存和系统调用讲起。原因很现实：没有这些基础，后面的任何性能结论都没有解释力。

假设你写了两个函数，源码层一个看起来多做了几次判断，另一个看起来更简洁。你测量后发现前者更快。没有编译链知识时，你可能会猜测“分支不一定慢”或者“编译器很聪明”。这两个说法都太粗。你应该先看优化级别、内联情况、生成的指令、数据是否在寄存器里、循环是否被展开、分支是否被消除、函数是否被常量传播、调用是否经过 PLT、第一次运行是否触发动态链接、输入是否改变 cache 状态。也就是说，真正的解释路径必须穿过本章建立的层次。

再假设一个学生说：“我的指针地址每次运行不同，所以内存分配不稳定，性能不可控。”这个说法也混合了多个层。地址不同可能只是 ASLR 改变了虚拟地址布局，不等于物理内存位置一定以同样方式变化，也不等于 cache 行对齐必然改变，更不等于性能差异来自堆分配器。要分析它，需要区分虚拟地址、页、物理页、cache index、堆分配策略和测量噪声。这些都建立在“进程拥有虚拟地址空间”这个基础上。

再看第三个例子：一个 benchmark 第一次运行特别慢，后面多次运行稳定。没有本章的基础时，容易把第一次慢归因于 CPU “热身”。这可能对，也可能不对。第一次运行可能包括动态链接的延迟绑定，可能包括页面首次触碰产生的缺页，可能包括文件系统缓存，可能包括全局构造，可能包括 JIT 或插件初始化，也可能只是 CPU 频率从低功耗状态升上来。只有知道程序从文件到进程的启动过程，才能把这些可能性拆开。

因此，本章不是背景闲话，而是本册的地基。后面任何一次“看汇编”“解释地址”“写实验报告”“判断优化是否有效”，都会回到这里。

从一个最小程序开始

先看一个最小但真实的 C++ 程序：

```
#include <iostream>

int main()
{
    int x = 1;
    int y = 2;
    std::cout << x + y << '\n';
    return 0;
}
```

从人的角度看，它做了三件事：

创建两个整数
计算 $x + y$
打印结果

从系统角度看，它至少涉及：

```

source file
-> preprocessor
-> compiler frontend
-> optimizer
-> compiler backend
-> assembly
-> assembler
-> object file
-> linker
-> executable ELF
-> loader
-> process address space
-> runtime startup
-> main
-> CPU executes instructions

```

本章的任务就是把这条路径铺开。

为什么这条路径必须学

如果最终要进入高性能计算、现代 CPU 优化或 AI CPU 算子，为什么第一个基础问题不是 AVX2，而是“程序怎么运行”？

因为很多性能现象发生在源码之外：

- 编译器可能删除你以为正在测量的循环。
- 函数可能被 inline，二进制里不再有独立函数体。
- 动态链接和全局构造可能影响第一次运行时间。
- 不同编译参数可能生成完全不同的指令。
- 同一个地址表达式可能映射到栈、堆、全局区或 mmap 区域。
- 共享库调用、PLT/GOT、lazy binding 可能影响调用路径。
- 程序启动成本不是 kernel 成本。

如果你不知道这些层，就很容易把系统行为误认为算法行为。

翻译单元：编译器真正看到的输入

源文件和头文件是编辑器里的概念，翻译单元才是 C/C++ 编译的基本边界。一个翻译单元可以粗略理解为“一个源文件经过预处理后的结果”。预处理器会处理头文件包含、宏展开、条件编译和注释删除。一个很短的源文件，只要包含了标准库头文件，预处理后就可能变成非常大的文本；编译器前端真正分析的是这个展开后的文本。

这个模型解释了一个常见误区：头文件通常不是被单独编译后再与源文件合并。头文件的内容会被包含进当前源文件，成为当前翻译单元的一部分。因此，头文件中的声明、内联函数、模板定义、宏和条件分支，都会影响当前源文件的编译结果。

声明和定义的区别在这里非常关键。声明告诉编译器“有这样一个名字，它的类型是什么，可以怎样使用”；定义则真正提供实体本身。函数声明可以在多个翻译单元里重复出现，只要一致即可；普通外部函数定义通常只能在整个程序里有一个；类定义可以通过头文件进入多个翻译单元，但必须保持一致；模板定义需要在实例化点可见，否则编译器无法为具体类型生成代码。很多链接错误，本质上不是链接器神秘，而是声明、定义和翻译单元边界没有处理好。

翻译单元还会影响优化视野。编译器在编译一个源文件时，通常只能直接看到当前翻译单元里的函数体和类型信息。假如一个小函数的定义在头文件里，编译器可以在调用点看到它的实现，于是更容易内联、常量传播和消除分支。假如函数只在另一个目标文件里定义，当前翻译单元只看见声明，那么普通编译流程下编译器不能随意假设这个函数的内部行为。链接时优化可以扩大这个视野，但那是额外机制，不是默认一定发生。

深入理解

翻译单元是理解 C++ 工程性能的第一道门。你看到的“函数调用开销”，可能来自函数体不可见；你看到的“模板编译慢”，可能来自模板定义在许多翻译单元中重复实例化；你看到的“改一个头文件全项目重编译”，可能来自这个头文件被许多翻译单元包含。编译速度、二进制体积和运行性能，经常同时受翻译单元组织方式影响。

头文件保护和一次定义规则也属于这个主题。头文件保护避免同一个头文件在一个翻译单元内被重复展开导致重复声明或重复定义。一次定义规则约束整个程序中的实体定义必须一致。初学者常把“头文件只包含一次”和“一次定义规则”混为一谈。前者解决的是一个翻译单元内部的重复包含问题；后者解决的是整个程序范围内实体定义是否一致的问题。它们相关，但不是同一件事。

当你阅读大型项目时，不要只看一个源文件。你要追问：这个函数的定义在哪里，调用点是否能看到函数体，宏是否改变了源代码形态，某个配置宏是否选择了不同实现，同一个头文件进入了多少翻译单元。后续学习高性能库时，你会看到许多实现被放入头文件，不只是为了方便发布，更是为了让编译器在调用点看到常量、类型、维度、对齐和目标指令集条件，从而生成专门化代码。

声明、定义和链接属性：名字如何跨文件成立

程序从一个源文件走向多个目标文件时，最核心的问题是名字如何被不同阶段理解。源码层的名字给程序员阅读，编译器前端用它做语义分析；目标文件层的符号给链接器解析；运行时的地址给 CPU 执行。声明、定义和链接属性就是把这些层连接起来的规则。

声明告诉编译器如何使用一个名字

声明的作用是让当前翻译单元知道某个名字的类型和使用方式。例如：

```
int add_i32(int a, int b);
```

这行声明告诉编译器：存在一个名为 `add_i32` 的函数，它接收两个 `int`，返回一个 `int`。有了这个信息，当前翻译单元就能检查调用是否类型正确，也能生成一次调用所需的目标文件引用。但声明本身通常不提供函数体。它不能让链接器最终找到代码。

这解释了一个常见现象：只写声明，编译可以通过，链接却失败。编译阶段只需要知道如何生成引用；链接阶段需要找到引用对应的定义。如果整个程序里没有提供函数体，链接器就会报告未定义引用。

定义提供实体本身

定义真正创建函数体或对象。例如：

```
int add_i32(int a, int b)
{
    return a + b;
}
```

函数定义会让编译器生成对应代码，并在目标文件中产生一个可以被链接器解析的符号。全局变量定义会产生存储空间。类定义、模板定义、内联函数定义又有特殊规则，因为它们经常需要进入多个翻译单元，但仍必须满足一致性要求。

下面这个例子经常导致重复定义：

```
// bad.hpp
int global_counter = 0;
```

如果 `bad.hpp` 被两个 `.cpp` 包含，每个翻译单元都会生成一个 `global_counter` 定义。链接器看到多个强定义，就可能报重复定义。正确写法通常是头文件放声明，某个源文件放定义：

```
// good.hpp
extern int global_counter;

// good.cpp
int global_counter = 0;
```


C++17 以后也可以在合适场景使用 `inline` 变量，但那是另一条明确规则，不是把定义随便放头文件就安全。

外部链接和内部链接

链接属性决定一个名字是否能被其他翻译单元引用。外部链接的名字可以跨翻译单元解析；内部链接的名字只在当前翻译单元内部可见。

例如：

```
int public_function();

namespace {
int private_helper()
{
    return 1;
}

static int another_private_helper()
{
    return 2;
}
```

`public_function` 通常具有外部链接。匿名命名空间中的 `private_helper` 和 `static` 函数通常具有内部链接。内部链接的好处是减少名字冲突，也给优化器更多自由：既然外部目标文件不能直接引用这个符号，编译器更容易删除、内联或改写它。

性能工程中，链接属性不是纯语法细节。一个函数如果对外可见，编译器和链接器可能要保留某些 ABI 边界；如果它只在当前翻译单元内部使用，优化器可以更大胆。库设计中也常通过符号可见性控制公共 ABI 面积，避免内部函数被外部用户依赖。

C++ 名字和二进制符号名

C++ 支持函数重载和命名空间，因此源码里的名字不能直接作为唯一二进制符号。两个函数都叫 `f`，只要参数类型不同，在 C++ 中可以共存。链接器却主要处理符号名。编译器需要把 C++ 名字、参数类型、命名空间、类作用域等信息编码进符号，这就是名字改编。

例如：

```
int f(int);
double f(double);
namespace math {
int f(int);
}
```

源码层看起来都有 `f`，目标文件符号却会不同。用 `nm-C` 可以反改编成人能读的名字：

```
nm -C build/app | rg " f\\("
```

如果要让 C++ 函数用 C 风格符号名暴露给汇编或 C 代码，可以使用：

```
extern "C" int add_i32(int a, int b);
```

注意，`extern"C`”主要改变语言链接和符号命名，不会自动改变参数类型的 ABI 责任，也不会让 C++ 对象、异常、引用生命周期 magically 变得适合跨语言边界。后面互操作章节会专门强调这一点。

核心思想

声明解决“当前翻译单元能否理解这个名字”，定义解决“程序中是否真的提供这个实体”，链接属性解决“这个名字能否跨翻译单元被找到”，符号名解决“链接器看到的二进制名字是什么”。

存储期和初始化：对象什么时候进入进程

源码里一个对象看起来只是一个变量，但从进程角度看，它还涉及存储期、初始化时机、所在映射区域和生命周期。理解这些概念，可以避免把所有内存都简单叫作“变量所在的地方”。

自动、静态和动态存储期

自动存储期对象通常是函数内的普通局部变量。它们随着作用域进入和退出而创建和销毁，常见实现会把它们放在栈帧中，或者在优化后直接放进寄存器，甚至被完全消除。

静态存储期对象在程序整个运行期间存在，包括全局变量、命名空间作用域变量、函数内 `static` 局部变量等。它们通常与可执行文件的数据段、BSS 段、只读数据段或运行时初始化表有关。它们的初始化可能发生在进入 `main` 前，也可能在第一次执行到某个局部静态变量时发生。

动态存储期对象通过分配器创建，例如 `new`、`malloc` 或容器内部的堆分配。它们通常位于堆或匿名映射区域，由运行时分配器管理。动态分配对象的生命周期不由作用域自动完全决定，而由所有权和释放动作决定。

存储期	常见例子	常见观察位置
自动存储期	函数局部变量。	栈、寄存器、或优化后消失。
静态存储期	全局变量、静态局部变量。	data、BSS、rodata、初始化表。
动态存储期	<code>new</code> 、 <code>malloc</code> 、容器缓冲区。	heap、匿名 mmap、分配器管理区域。

这张表是观察线索，不是语言保证。C++ 语言不要求普通局部变量一定有栈地址，也不要求优化构建保留每个变量的独立存储。调试时如果一个变量显示为 `optimized out`，不一定是调试器坏了，而是优化器不再维护一个源码层变量对应的稳定位置。

零初始化、常量初始化和动态初始化

静态存储期对象的初始化有多个阶段。零初始化会把对象先置为零值；常量初始化可以在编译或加载时完成；动态初始化需要运行代码，可能在 `main` 前发生。

例如：

```
int global_zero;
int global_value = 42;

int make_value()
{
    return 7;
}

int global_dynamic = make_value();
```

`global_zero` 可能进入 BSS 相关区域，不需要在文件里保存一份全零数据。`global_value` 可以放入已初始化数据。`global_dynamic` 需要运行函数得到初始值，启动时就可能执行额外代码。

这对性能测量很重要。如果一个 benchmark 程序有复杂全局对象，第一次运行可能包含全局初始化成本。若你用整个进程时间衡量一个小 kernel，就会把这些初始化成本混进去。更稳的做法是在 `main` 中显式构造 workload，把 `setup` 和 `timed region` 分开。

局部静态变量的第一次初始化

函数内静态局部变量常用于懒初始化：

```
std::vector<int>& table()
{
    static std::vector<int> values = make_values();
    return values;
}
```

第一次调用 `table` 时，`values` 需要初始化；后续调用通常只返回已有对象。C++ 还要求局部静态初始化具有线程安全语义，这可能引入 `guard` 检查。对普通业务代码，这很方便；对微基准，这可能让第一次样本明显更慢。

如果你测量某个函数并发现第一次调用慢很多，要检查是否有局部静态初始化、全局对象、动态链接首次解析、内存分配器初始化或首次触页。不要简单把第一次慢解释成“CPU 还没热”。

常见误区

对象什么时候初始化，是程序启动和第一次调用成本的一部分。性能报告如果没有区分初始化成本和稳态 kernel 成本，就很容易把运行时机制误判为算法成本。

构建产物身份：你测的是哪一个程序

性能实验和调试实验都依赖一个朴素事实：你必须知道自己运行的是哪一个二进制。大型项目中，经常同时存在多个构建目录、多个配置、多个编译器、多个同名可执行文件和多个动态库版本。只写“运行 app”远远不够。

二进制身份由多个因素共同决定

一个可执行文件的身份至少包括：

- 源码版本和未提交修改。
- 编译器名称和版本。
- 编译参数和宏定义。
- 目标架构和 ISA 选项。
- 链接命令、库版本和链接方式。
- 是否启用 LTO、PIE、调试信息、strip。
- 构建时间、路径和输出文件名。

这些因素任何一个变化，都可能改变机器码或运行时行为。比如 `-DNDEBUG` 会移除 `assert`；`-march=native` 可能启用更宽的向量指令；LTO 可能跨文件内联；不同标准库版本可能改变容器实现；PIE 和 ASLR 可能改变地址布局。

报告中的二进制身份证

建议每次正式实验都保存一个二进制身份证：

```
source:
  git commit
  dirty status
  diff stat if dirty

compiler:
  clang++ --version
  g++ --version if used

build:
  full compile and link command
  build directory
  output binary path

binary:
  file output
  size output
  sha256sum if important
  ldd output for dynamic dependencies
```

对教学实验来说，这看起来很严肃；对真实性能工程来说，这是基本卫生。没有二进制身份，实验结果很难复现，也很难判断一次变化来自源码、编译器、链接器、库还是环境。

旧二进制问题

最常见的低级错误是：源码改了，构建失败了，旧二进制还在，脚本继续运行旧二进制。另一个常见错误是：构建成功了，但运行命令指向另一个目录下的同名程序。第三个常见错误是：可执行文件是新的，但动态库仍然加载旧版本。

排查时可以先做三件事：

```
ls -l build-release/app
```

```
file build-release/app
ldd build-release/app
```

再看构建日志和反汇编。若你刚改了目标函数，但反汇编没有变化，要么改动没有进入构建，要么优化器把差异消除了，要么你看的不是实际运行的二进制。三种情况都需要证据区分。

核心思想

运行路径、二进制路径、动态库路径和源码版本必须能互相对上。否则，性能数字再精确，也可能只是旧程序的数字。

预处理器：文本层面的改写

预处理器（preprocessor）处理 include directive、宏、条件编译和注释删除。

例子：

```
#define SCALE 4

int f(int x)
{
    return x * SCALE;
}
```

预处理后大致变成：

```
int f(int x)
{
    return x * 4;
}
```

命令：

```
mkdir -p scratch/ch01
cat > scratch/ch01/preprocess.cpp <<'EOF'
#include <iostream>

#define SCALE 4

int f(int x)
{
    return x * SCALE;
}

int main()
{
    std::cout << f(3) << '\n';
}
EOF

g++ -E scratch/ch01/preprocess.cpp -o scratch/ch01/preprocess.i
wc -l scratch/ch01/preprocess.cpp scratch/ch01/preprocess.i
```

你通常会看到 `preprocess.i` 远比原文件长，因为标准库头文件展开了大量声明和模板。

常见误区

不要以为编译器看到的就是你手写的 `.cpp`。宏、include 和条件编译会先改写输入。性能分析遇到不符合直觉的行为时，必要时要看预处理结果。

编译器前端：从文本到语义

预处理结果仍然是 C++ 文本。编译器前端要把它转成内部结构。

简化流程：


```

characters
  -> tokens
  -> parser
  -> AST
  -> semantic analysis
  -> IR generation

```

Token

代码：

```
int x = a + 3;
```

会被词法分析成类似：

```
int
x
=
a
+
3
;
```

AST

抽象语法树（Abstract Syntax Tree）描述语法结构。上面的声明可理解成：

```

declaration x
  type: int
  initializer:
    binary +
      left: a
      right: 3

```

语义分析

语义分析检查：

- 名字是否声明。
- 类型是否匹配。
- 函数调用参数是否正确。
- 重载解析选择哪个函数。
- 模板实例化是否合法。
- `const`、引用和生命周期规则是否满足。

这一步决定了程序的 C++ 语义。后续优化必须在这个语义边界内进行。

IR 和优化器：在语义边界内重写程序

编译器通常不会直接从 C++ 生成最终机器码，而是先生成 **中间表示**（intermediate representation），简称 IR。

IR 的作用是提供一个比 C++ 简单、比机器码抽象的优化平台。

例如：

```
extern "C" int add_i32(int a, int b)
{
  return a + b;
}
```

LLVM IR 可能类似：

```
define i32 @add_i32(i32 %a, i32 %b) {
entry:
    %sum = add i32 %a, %b
    ret i32 %sum
}
```

优化器会在 IR 上做：

- 删除无用代码。
- 常量折叠。
- 函数内联。
- 循环优化。
- 自动向量化。
- 别名分析。

一个重要例子：无用计算会消失

如果你写：

```
int sum(int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        s += i;
    }
    return s;
}

int main()
{
    sum(1000000);
}
```

如果返回值没有可观察用途，优化器可能删除整个调用。这就是 benchmark 必须防止 dead-code elimination 的原因。

后端、汇编和编译器驱动程序

编译器后端把优化后的 IR 降低到目标机器。

后端要解决：

- 指令选择：使用哪些 x86-64 指令。
- 寄存器分配：值放在哪些寄存器。
- 指令调度：指令如何排序。
- 调用约定：参数和返回值放哪里。
- 汇编输出：生成 .s 文件。

生成汇编：

```
g++ -O3 -S -masm=intel scratch/ch01/add.cpp -o scratch/ch01/add.s
```

汇编是机器码的人类可读表示。CPU 最终执行的不是汇编文本，而是汇编器编码后的字节。

你调用的命令通常是驱动程序

很多教材和教程会写“用编译器编译”。这句话在入门时够用，但如果要理解构建问题和性能实验，就要更精确。你在终端里调用的 `g++` 或 `clang++` 通常是驱动程序。驱动程序读取命令行参数，决定调用哪些真正的阶段工具：预处理器、编译器前端、优化器、后端、汇编器、链接器，以及目标平台需要的启动文件和运行时库。

同一个命令根据参数不同，会停在不同阶段。只预处理时输出预处理结果；生成汇编时停在汇编文本；只编译时生成目标文件；不加停止选项时继续链接生成可执行文件。理解这些停止点的意义，比记住某一个命令更重要。因为你之后定位问题时，往往要把完整构建拆开：先看预处理后宏

是否按预期展开，再看汇编是否生成预期指令，再看目标文件是否有符号，再看最终可执行文件是否链接到正确库。

驱动程序还会隐式加入很多内容。你写命令时可能只给了一个源文件，但最终链接时还会加入启动代码、运行时库、标准库、动态链接器路径和默认系统库。C++ 程序尤其如此。一个看似最小的程序，只要使用标准库输入输出，就会牵涉大量库代码、初始化逻辑和动态链接信息。初学者如果以为“我的源文件只有十行，所以最终程序也只有十行机器码”，就会在第一次看反汇编时非常困惑。

优化级别、调试信息、目标 CPU、标准版本、异常、运行时类型信息、链接方式、可见性、位置无关代码，这些参数都会改变生成结果。性能实验必须记录它们，因为它们不是注释，而是程序的一部分。一个用调试构建测出的慢，不能直接代表发布构建；一个默认目标 CPU 下的汇编，不能代表启用特定指令集后的汇编；一个静态链接程序的启动和加载行为，不能直接代表动态链接程序。

常见误区

不要把构建命令当作实验报告里的附属信息。对性能实验来说，构建命令是实验条件。没有构建命令，别人无法判断你测的是哪一个程序，更无法复现实验。

汇编器和目标文件

汇编器（assembler）把汇编文本变成目标文件。

命令：

```
g++ -c -O3 scratch/ch01/add.cpp -o scratch/ch01/add.o
```

目标文件已经包含机器指令，但通常还不是完整程序。可以把目标文件看成一个带注释的二进制片段：里面有一些已经编码好的指令和数据，也有一些还需要链接器以后填上的位置。这个“还需要以后填上”的事实，就是重定位存在的原因。

目标文件通常按节组织内容。代码放在文本相关的节中，只读常量放在只读数据节中，已初始化全局变量放在数据节中，未初始化或零初始化全局变量放在 BSS 相关节中，调试信息放在调试节中，符号表记录名字与位置的关系，重定位节记录哪些机器码或数据位置还依赖外部符号或最终地址。节主要服务于链接器和调试工具；后面加载进进程时，加载器更关心程序头描述的可加载段。

符号表是目标文件之间互相认识的方式。一个目标文件可以定义符号，也可以引用外部符号。定义符号意味着“我这里提供这个名字对应的代码或数据”。未定义符号意味着“我需要别人提供这个名字”。链接器的基本工作之一，就是把所有未定义引用匹配到某个定义上。如果找不到，就产生未定义引用错误；如果普通强符号出现多个不兼容定义，就可能产生重复定义错误。

C++ 的符号比 C 更复杂，因为 C++ 支持函数重载、命名空间、类成员函数、模板和不同调用形式。为了让链接器区分这些实体，编译器会把源代码层面的名字编码成更详细的二进制符号名，这通常叫名字改编。你在工具输出中看到一串看起来很难读的符号，可能只是一个带命名空间和参数类型的 C++ 函数。工具的反改编选项可以把它还原成人更容易读的形式。

重定位是目标文件的另一个核心。编译一个源文件时，编译器和汇编器常常不知道某个函数或全局变量最终会被放到哪里。于是目标文件里先留下占位信息，并记录“这里以后要填入某个符号的地址或偏移”。链接器合并目标文件、安排布局、确定符号位置后，再根据重定位记录修正这些位置。位置无关代码和动态链接会让这个过程更复杂，但基本思想仍然是：某些地址相关信息不能在单个目标文件阶段完全确定。

核心思想

目标文件不是失败的可执行文件，而是专门为链接设计的中间产物。它的价值在于保留了机器码、符号、节和重定位记录，让多个翻译单元可以独立编译，最后再组合成程序。

从性能角度看，目标文件阶段提供了一个很好的观察窗口。你可以在还没有链接完整程序前，查看某个函数是否生成了预期指令，是否出现了不必要的栈访问，是否保留了符号，是否产生了对外部函数的调用。这种观察比只看最终可执行文件更局部，适合定位“是这个翻译单元生成得不好，还是链接后环境改变了调用路径”。

链接器：把独立编译变成完整程序

一个真实程序通常由多个目标文件和库组成。

例子：

```
// add.cpp
extern "C" int add_i32(int a, int b)
{
    return a + b;
}
```

```
// main.cpp
extern "C" int add_i32(int, int);

int main()
{
    return add_i32(1, 2);
}
```

构建：

```
g++ -c add.cpp -o add.o
g++ -c main.cpp -o main.o
g++ add.o main.o -o app
```

链接器（linker）负责把 `main.o` 里对 `add_i32` 的引用解析到 `add.o` 中的定义。

它还会处理：

- 标准库。
- 启动代码。
- 静态库。
- 共享库引用。
- relocation。
- 符号可见性。

链接的核心不是“把文件粘起来”，而是二进制层面的名字解析、布局安排、地址修正和库选择。链接器会把多个目标文件中的节合并或安排到最终输出中，并根据重定位记录修正机器码或数据里的地址相关字段。如果是静态链接，许多引用可以在链接时完全确定；如果是动态链接，有些引用要留给动态链接器在程序加载时或首次调用时处理。

静态库和共享库体现了两种不同组合方式。静态库本质上是一组目标文件的归档，链接时需要的部分会被取出来合入最终输出。共享库则在运行时作为独立映射进入进程，多个进程可以共享只读代码页。静态链接可能让部署更简单，但二进制更大，更新库需要重新链接；动态链接可以共享库和独立更新，但启动、符号解析和版本兼容问题更复杂。

链接还引入了一个对性能很重要的边界：编译器在单个翻译单元内做优化，链接器负责跨目标文件组合。如果没有链接时优化，普通编译器很难跨越目标文件边界内联函数或做全程序分析。启用链接时优化后，编译器可以把更多中间表示保留到链接阶段，再进行跨翻译单元优化。这可能显著改变函数边界、符号可见性、内联决策和最终性能。因此，报告性能结果时必须说明是否启用了链接时优化。

深入理解

许多看似运行时的问题，根源其实在链接阶段：链接到了错误版本的库，库顺序导致符号没解析，调试构建混入发布构建，动态库搜索路径指向旧文件，或者链接时优化改变了函数边界。遇到这类问题，应该检查目标文件符号表和链接命令，而不是只盯着调用点源码。

ELF 可执行文件：机器码之外的二进制合同

Linux x86-64 上的可执行文件通常是 **ELF**（Executable and Linkable Format）。

ELF 不是“一段纯机器码”。它是一个带元数据的二进制容器，包含：

- ELF header。
- program headers。
- sections。
- symbol table。
- relocation information。
- dynamic linking information。
- debug information。

查看：

```
file app
readelf -h app
readelf -l app
readelf -S app
nm -C app
objdump -drwC -Mintel app
```

现在不要求你完全读懂每一项。本册只要求你建立“可执行文件是操作系统加载器能理解的一种结构化文件”这个模型；ELF、重定位、动态链接和加载器细节会在后续系统专题中继续展开。

ELF 头描述文件的基本身份，例如目标架构、文件类型、入口地址、程序头位置和节头位置。程序头描述加载器关心的段，它告诉内核或动态链接器哪些文件范围要映射成进程中的可读、可写或可执行区域。节头主要服务于链接和分析工具，描述代码节、数据节、符号表、字符串表、重定位节、调试节等。一个已经去除节头的可执行文件仍然可能运行，因为加载器主要依赖程序头；但没有足够符号和调试信息时，人类分析会困难很多。

这就解释了为什么同一个二进制可以有“运行视角”和“分析视角”。运行视角关心可加载段和入口点；分析视角关心节、符号、源码行号和调试信息。你用工具查看节表，看到的是链接器和调试器组织信息的方式；你查看程序头，看到的是加载器如何把文件映射进进程；你查看反汇编，看到的是某些可执行字节被解释成指令；你查看符号表，看到的是名字和地址之间的关系。每个视角都是真的，但它们回答的问题不同。

可执行文件也有权限结构。代码段通常可读可执行但不可写；只读数据通常可读不可写；可写数据段可读可写但不可执行；栈和堆通常不可执行。这些权限不是装饰，而是安全和调试的重要基础。把数据页设为不可执行可以阻止一些攻击；把代码页设为不可写可以防止程序随意修改自身；违反权限的访问会触发异常并由操作系统处理。后续学习段错误时，要把它理解成进程访问了当前页表和权限不允许的地址，而不是简单地说“指针坏了”。

ELF 还保存动态链接信息。动态段会记录需要的共享库、符号查找表、重定位表、初始化和终止函数等。动态链接器根据这些信息把共享库映射进进程，并处理符号绑定。第一次调用某个外部函数时，如果使用延迟绑定，调用路径可能会先进入解析逻辑，再跳到真实函数。这就是为什么一些函数第一次调用比后续调用慢，也解释了为什么严谨 benchmark 常常要预热或把第一次观测单独处理。

常见误区

不要把 ELF 简化成“里面装着机器码”。如果只看机器码，你会漏掉入口点、段权限、动态库依赖、重定位、符号、调试信息和异常展开信息。系统性能问题往往就藏在这些元数据和运行时解释过程里。

操作系统加载程序

当你运行：

```
./app
```

shell 会请求内核创建新进程。内核和加载器大致做：


```

read ELF header
create process address space
map executable segments
map shared libraries if needed
prepare stack
prepare argc / argv / envp
start dynamic linker if dynamically linked
transfer control to entry point

```

如果是动态链接程序，动态链接器还会解析共享库和符号，然后运行 C/C++ runtime 初始化。

核心思想

`main` 是 C/C++ 程序员约定的入口函数，但进程真正开始执行的位置通常是 ELF header 中的 entry point，例如 `_start`。

映射是理解加载的关键词。文件中的某些范围可以被映射到进程虚拟地址空间。映射并不一定意味着所有内容立刻从磁盘读入物理内存。现代操作系统通常使用按需分页：页面第一次被访问时，如果还没有物理页或内容尚未载入，就触发缺页，由内核把所需内容准备好。这样可以减少启动时不必要的读入，也让多个进程共享同一份只读代码页。

按需分页对性能解释非常重要。一个程序第一次访问大数组时可能变慢，不一定是算法突然变差，而可能是页面首次触碰导致缺页、零页分配或物理页建立。第一次执行某段很少用的库代码也可能触发指令页载入。后续运行同一路径时，页面可能已经在内存中，文件系统缓存也可能已经热起来。若不区分这些现象，就会把加载和内存管理成本误记到算法本身。

动态链接器的工作也属于加载路径的一部分。它要找到共享库文件，检查依赖，映射库段，处理符号和重定位，运行初始化函数。库搜索路径、运行时路径、系统缓存和环境变量都可能影响它找到哪个库。一个程序在开发机能运行，在另一台机器上启动失败，常常不是源码问题，而是动态库版本、路径或 ABI 不匹配。

进程和虚拟地址空间

进程（process）可以理解为：

```

running program instance
+ virtual address space
+ OS-managed resources

```

进程拥有：

- 虚拟地址空间。
- 文件描述符。
- 当前工作目录。
- 信号处理状态。
- 一个或多个线程。
- 权限和资源限制。

简化的虚拟地址空间：

```

high address
stack
...
shared libraries / mmap
...
heap
bss
data
rodata
text
low address

```

这只是模型。真实地址受 ASLR、动态链接、内核配置和运行环境影响。

入口点不是 main

查看入口点：

```

readelf -h app | rg "Entry point"
objdump -d -Mintel app | sed -n '/<_start>:/,+40p'
nm -C app | rg "main$_start"

```

你会看到 `_start` 一类符号。它最终会调用运行时库，再调用你的 `main`。这对性能测量很重要：

- 程序启动成本不是函数成本。
- 动态链接和首次解析可能影响第一次运行。
- 全局对象构造可能在 `main` 前发生。
- benchmark 应该把 setup 和 timed region 分开。

运行时启动：main 之前已经发生了很多事

C++ 程序员习惯把 `main` 看成入口，这是语言层面的入口，不是进程第一条用户态指令。从操作系统转入用户态后，通常先执行运行时启动代码。启动代码从初始栈中取出参数、环境变量和辅助向量，准备运行时环境，调用全局构造相关逻辑，设置退出处理，然后调用 `main`。当 `main` 返回后，运行时还要处理返回值、执行退出回调和全局析构，最后通过系统调用结束进程。

全局对象构造是 C++ 特有的重要成本来源之一。带构造函数的全局对象、静态存储期对象、某些库内部对象，都可能在进入 `main` 前初始化。它们可能分配内存、打开文件、注册回调、初始化互斥量或触发其他库逻辑。如果 benchmark 把整个程序运行时间当作某个小函数的成本，就会把这些启动动作全部混入结果。

线程局部存储也可能参与启动和访问成本。线程局部变量给每个线程提供独立实例，这对并发程序很有用，但它需要运行时和 ABI 共同支持。不同类型的线程局部访问模型有不同代价。初学阶段不需要掌握全部细节，但要知道：看似普通的全局或线程局部变量访问，在机器层可能不是一条简单内存读写。

异常机制和栈展开信息也在二进制和运行时中占有位置。即使正常路径没有抛出异常，支持异常的 C++ 程序也可能带有展开表和处理逻辑。编译选项是否启用异常，会影响代码生成、二进制体积和调用边界。性能分析时，如果一个项目关闭异常或运行时类型信息，它生成的二进制和另一个默认项目可能不具有可比性。

标准库初始化同样不可忽略。输入输出流、区域设置、内存分配器、同步设置等，都可能在启动或首次使用时带来成本。初学者常用标准输出打印调试信息，但打印本身涉及库、缓冲、系统调用和终端设备，代价远高于普通整数运算。做微基准测试时，计时区域内应避免输出；需要验证结果时，可以在计时外消费结果。

心智模型

`main` 是你代码的入口，不是进程世界的起点。严谨分析要把“程序启动”“运行时初始化”“测试数据准备”“被测核心区域”“结果校验”“进程退出”分开。只有被测核心区域才应该代表算法或 `kernel` 的性能。

进程资源：运行的不只是指令

进程不只是指令流。一个进程是操作系统管理的资源集合。它有虚拟地址空间，有一个或多个线程，有文件描述符表，有信号处理状态，有当前工作目录，有权限和资源限制，有调度属性，有环境变量。一个程序运行得快或慢，可能与这些资源状态有关。

文件描述符是进程与外部世界交互的重要接口。标准输入、标准输出、标准错误通常就是前三个文件描述符。打开文件、socket、管道、事件对象，都可以表现为文件描述符。输出到终端、输出到文件和输出到管道，成本和阻塞行为都可能不同。因此，把打印放进计时区域，会让测量受外部设备和内核缓冲状态影响。

环境变量会影响程序行为。动态库搜索、线程库设置、内存分配器行为、OpenMP 线程数、某些高性能库的 ISA 分发策略，都可能通过环境变量控制。两个运行命令看起来一样，如果环境变量不同，实际执行路径可能不同。高质量实验报告应记录关键环境变量，至少说明没有依赖特殊环境，或者明确列出影响性能的设置。

当前工作目录和文件路径也会影响运行。相对路径从当前工作目录解析；程序可能读取配置文件、模型文件、输入数据或动态插件。如果你在不同目录运行同一个可执行文件，加载的数据可能不同，甚至链接到的插件也可能不同。性能实验应把输入数据来源和工作目录说清楚。

调度状态也属于进程运行环境。操作系统会把线程安排到 CPU 核心上运行，也可能中断、迁移或抢占它。线程迁移会改变缓存热度；抢占会让墙钟时间变长；后台任务会争用内存带宽、共享缓存和执行资源；电源管理会改变频率。后续测量章节会系统讨论这些噪声，但本章先建立一个原则：进程不是独占机器，性能数据总是运行环境和程序共同作用的结果。

虚拟地址空间：把地址理解成进程内名字

用户态程序中的指针值通常是虚拟地址。虚拟地址可以先粗略理解为进程地址空间中的名字。CPU 执行 `load` 或 `store` 时，虚拟地址要经过地址翻译，找到对应物理位置或触发异常。操作系统和硬件共同维护这种映射。这样做带来隔离、保护、按需分页、共享映射、文件映射和内存超量承诺等能力。

虚拟地址让每个进程看起来拥有自己的连续地址空间。两个进程可以使用相同的虚拟地址而互不干扰，因为它们的页表映射到不同物理页。共享库的只读代码页又可以被多个进程映射到各自地址空间中，从而共享物理内存。写时复制让父子进程在创建后先共享页面，直到某一方写入时才复制。所有这些机制都说明：指针数值只是入口，不能直接等同于物理存储位置。

地址空间通常按用途分区。文本区域保存程序代码，只读数据保存字符串字面量和常量，可写数据保存已初始化全局变量，BSS 保存零初始化全局变量，堆用于动态分配，栈用于函数调用和局部自动对象，映射区域用于共享库、文件映射和匿名映射。这个模型是简化的，但足以帮助你解释许多现象：为什么局部变量地址靠近栈，为什么动态分配对象在堆附近，为什么字符串字面量通常不应该修改，为什么访问空指针会失败。

栈和堆的区别尤其容易被误解。栈不是“快内存”，堆也不是“慢内存”。它们都是虚拟地址空间中的区域，最终访问仍然经过缓存和内存系统。栈分配通常只是移动栈指针，管理成本低，局部性也常常好；堆分配需要分配器维护元数据，可能加锁，可能触发系统调用或页分配，管理成本更高。但一旦对象已经分配好，访问栈对象和堆对象的速度主要取决于地址是否命中缓存、访问模式是否连续、是否存在依赖和争用，而不是“栈”这个名字本身。

页面是虚拟内存管理的重要单位。常见页面大小是若干 KiB，也有大页机制。首次访问新分配的大块内存时，可能触发页面建立和清零；随机访问大数组时，可能受到 TLB 容量影响；跨页访问和缺页会显著改变延迟。第二册讲缓存、TLB 和内存层级时会深入这些问题。本章先要求你知道：地址不是孤立数字，它属于页，页属于映射，映射有权限和来源。

常见误区

不要把“地址小”“地址大”“两次地址接近”直接解释成物理距离或性能关系。用户态看到的是虚拟地址。要讨论性能，还要考虑页面映射、对齐、cache 行、TLB、访问模式和运行时分配器。

可观察性：每个工具只能回答一类问题

学习本章时会接触许多工具。工具越多，越需要知道它们分别回答什么问题。预处理输出回答“编译器前端看到的文本是什么”。汇编输出回答“编译器后端打算让汇编器编码哪些指令”。目标文件反汇编回答“这个目标文件中已经生成了哪些机器码和重定位”。符号工具回答“哪些符号存在，哪些未定义，哪些可见”。ELF 查看工具回答“文件格式、段、节、入口和动态链接信息是什么”。运行时映射文件回答“进程当前虚拟地址空间如何映射”。调试器回答“某次运行中寄存器、栈、内存和控制流状态如何变化”。

没有哪个工具能单独告诉你全部真相。只看源码，你看不到优化后的机器码；只看汇编，你看不到操作系统调度；只看运行时间，你看不到是否测到了错误区域；只看符号表，你看不到函数是否在热路径中执行；只看平均值，你看不到第一次运行、缺页和迁移造成的异常值。高质量分析不是迷信某个工具，而是让多个工具回答各自擅长的问题，然后把证据拼起来。

工具输出还要结合构建上下文解释。同一个源文件，用不同优化级别、不同目标 CPU、不同编译器版本、不同链接方式，会得到不同输出。不同 Linux 发行版、不同动态库版本、不同内核配置，也可能改变加载和运行行为。因此，工具输出不能脱离命令、版本和环境。你应该养成在实验目录保存命令、源码、构建日志、工具输出和测量结果的习惯。

性能误判：从源码直觉跳到时间数字最危险

本章建立的路径可以用来识别常见性能误判。

第一类误判是把未被执行的代码当作性能对象。源码里存在一段循环，不代表二进制里存在这段循环；二进制里存在指令，不代表你的输入会执行到；执行到一次，不代表它是热点。优化器删除无用代码、分支条件选择、错误的测试输入、提前返回，都可能让你测到的不是想测的路径。

第二类误判是把启动成本当作核心算法成本。一个程序运行一次需要创建进程、加载映射、初始化运行时、解析动态库、构造全局对象、准备输入数据。若被测函数只运行很短时间，这些固定成本会淹没真实差异。正确做法是把计时区域缩小到核心代码，同时保留必要的预热和结果校验。

第三类误判是把调试构建当作真实性能。调试构建为了可调试性会保留更多栈变量、减少优化、增加检查或调试信息。它适合定位功能错误，不适合代表最终性能。发布构建则可能内联、展开、向量化、删除边界检查或重排代码。两者的汇编差异可能大到像两个程序。

第四类误判是把一次测量当作结论。进程调度、频率变化、缺页、缓存状态、分支预测状态、输入数据布局、动态链接首次解析，都可能让一次结果偏离稳态。可信测量需要重复、预热、隔离、记录环境、报告分布，并解释异常值。

第五类误判是把工具名当作证据。说“我用反汇编看过”还不够，你要指出哪一个函数、哪一个构建、哪一段指令、它对应哪个源码假设。说“我用计时器测过”也不够，你要说明计时范围、重复次数、输入规模、优化级别、是否防止死代码消除、是否排除输出和初始化。证据必须能被别人复查。

核心思想

性能结论的最低标准是：源码假设、构建产物、运行路径和测量数据能够互相印证。任何只停留在源码直觉或单次时间数字上的判断，都不够可靠。

把本章路径用于真实工程阅读

当你拿到一个陌生 C++ 工程时，可以按本章路径建立第一轮认识。先找到构建系统，确认编译器、标准版本、优化级别、目标架构和链接方式。再找到主要可执行文件或库的入口，区分哪些是业务源码，哪些是第三方库，哪些是生成文件。然后选择一个小函数，从源码追到汇编和目标文件，确认工具链真的按你理解的方式工作。最后运行程序，观察进程映射、动态库依赖和基本资源使用。

这套流程听起来慢，但它能避免很多低级错误。你不会在调试构建上浪费一周优化；不会把旧

动态库当作新代码测；不会因为函数被内联就以为符号丢失是编译失败；不会把标准输出成本计入内核性能；不会把第一次运行的加载成本当作算法波动。越是复杂项目，越要先建立这些底层事实。

阅读高性能库时，还要特别关注条件编译和 ISA 分发。许多库会根据编译参数、运行时 CPU 检测或模板参数选择不同实现。源码中你看到的那段循环，未必是当前机器真正执行的路径。可能有标量后备实现、SSE 实现、AVX2 实现、AVX-512 实现，也可能通过函数指针或间接调用在运行时选择。要判断性能，必须确认当前构建和当前 CPU 走的是哪条路径。

阅读系统库时，要关注 ABI 和二进制边界。函数参数如何传递，结构体如何返回，异常能否跨库边界传播，符号是否可见，库版本是否兼容，这些都不是纯源码问题。后面 ABI 章节会详细讲调用约定，但本章先给出位置：ABI 位于源码语言和二进制执行之间，是不同目标文件、库和语言运行时能够协作的约定。

怎样解释一个构建失败

为了把本章知识真正用起来，可以从构建失败开始练习。构建失败并不可怕，可怕的是不知道错误属于哪一层。只要能定位层次，问题通常就会变得具体。

如果错误信息说缺少分号、括号不匹配、类型不匹配、名字没有声明，问题通常在当前翻译单元的前端阶段。此时你应该检查源文件、头文件包含、命名空间、声明顺序、模板参数和宏展开。不要一上来怀疑链接器，因为链接器根本还没有机会看到一个语义正确的目标文件。

如果错误信息说某个函数或变量未定义，源文件本身又能编译成目标文件，那么问题通常进入了链接阶段。常见原因包括：只写了声明没有写定义；定义所在源文件没有加入构建；库没有出现在链接命令中；静态库顺序不正确；C 和 C++ 的符号名约定不一致；函数签名在声明和定义之间不一致；模板定义没有在实例化点可见。处理这类错误时，应该检查目标文件符号表和链接命令，而不是只盯着调用点源码。

如果程序已经链接成功，但运行时提示找不到共享库，问题通常在加载阶段。可执行文件记录了它需要哪些共享库，动态链接器要在运行环境中找到这些库。库路径、运行时搜索路径、系统缓存、容器环境和安装位置都会影响结果。源码正确、链接成功，不代表部署环境一定能加载。

如果程序能启动，但一进入某个函数就崩溃，问题可能已经到运行阶段。此时要考虑空指针、越界访问、悬垂引用、栈破坏、ABI 不匹配、库版本不兼容、未定义行为、线程竞争等。反汇编和调试器可以帮助定位崩溃指令，但崩溃根因可能在更早的源码层或链接层。

心智模型

遇到错误时先问：它发生在预处理、前端、后端、汇编、链接、加载，还是运行阶段？错误所在阶段决定第一批证据应该从哪里收集。跳过阶段判断，容易把简单问题查成复杂问题。

举一个典型例子：你在头文件里写了一个普通函数定义，这个头文件被两个源文件包含，最后链接时报重复定义。初学者可能以为“头文件保护失效”。但头文件保护只防止同一个翻译单元里重复包含同一个头文件，不能防止这个函数定义出现在两个不同翻译单元中。链接器看到两个目标文件都定义了同一个外部符号，于是报错。正确修法可能是把函数定义移到一个源文件中，在头文件只保留声明；或者把它改成符合规则的内联函数；或者如果是内部实现细节，给它内部链接属性。这个例子说明，同一个头文件问题，可能要用翻译单元和链接规则解释。

再举一个例子：你在 C++ 源文件里调用一个 C 库函数，头文件声明看起来正确，链接却找不到符号。原因可能是 C++ 编译器对函数名做了名字改编，而 C 库导出的符号没有这种改编。解决办法通常是在声明处使用 C 链接约定。这个问题不是函数不存在，而是二进制符号名不一致。它处在语言边界和链接边界之间，正好体现了 ABI 的重要性。

怎样解释一个运行成功但结果异常的程序

程序运行成功不代表它按你的意图运行。性能工程中更常见的问题不是程序崩溃，而是程序“看起来没问题”，但结果不可信。

第一种情况是输出结果正确，但测量对象错误。假设你想测一个小函数的成本，却把进程启动、输入构造、标准输出、内存分配和结果校验都放进计时区。程序当然可以运行成功，输出也可以正确，但时间数字混合了太多不相关成本。此时要把计时范围缩小，先准备输入和内存，再开始计时，

计时结束后再验证结果。

第二种情况是二进制不是你以为的二进制。构建目录里可能同时存在旧文件和新文件，脚本可能运行了旧路径，动态库可能加载了系统版本而不是本地版本，容器里可能缺少某个编译参数。程序可以运行成功，但运行的不是你刚刚修改的实现。遇到这种情况，要检查构建时间戳、符号、反汇编、动态库依赖和运行命令。

第三种情况是优化器改变了你以为的执行路径。源码里写了一个循环，结果没有被使用，优化后循环消失；源码里写了一个函数调用，优化后被内联；源码里写了一个分支，常量传播后分支被删除。这些都可能让程序输出仍然正确，却让测量不再代表原始源码形状。性能实验必须用二进制证据确认热路径确实存在。

第四种情况是第一次运行和稳态运行混在一起。第一次调用可能触发动态绑定、页面首次触碰、内存分配器初始化、分支预测和缓存状态变化。程序长期运行时的稳态成本与第一次成本可能相差很大。报告中应明确区分冷启动、预热后、固定输入和多次重复的结果。

第五种情况是输入规模不合适。太小的输入会让函数调用、计时器开销、循环控制和启动成本占比过高；太大的输入可能引入内存带宽、缺页、换页或缓存容量效应。一个算法在小输入上的快慢，不一定代表大输入；一个 kernel 在热缓存数据上的表现，也不一定代表真实工作负载。输入规模是实验条件，不是附属说明。

常见误区

“能运行”只是最低门槛，不是证据链的终点。高质量实验至少要证明：运行的是预期二进制，执行的是预期路径，测量的是预期区域，输入规模匹配问题，结果没有被优化器删除，环境噪声已经被记录或控制。

一个完整解释样例：从函数调用到二进制证据

假设你写了一个函数，用来计算两个整数之和。源码层面它只有一次加法。一个初学者可能直接说：“这个函数成本就是一次整数加法。”这句话通常不完整。

首先要问函数是否真的被调用。如果调用点传入常量，优化器可能直接把结果算出来。如果返回值没有被使用，整个调用可能被删除。如果函数定义在头文件里，调用点能看到函数体，编译器很可能内联它。如果函数定义在另一个目标文件中，又没有启用链接时优化，调用可能保留下来。源码层面的“一个函数”在二进制层可能对应函数体、内联片段、常量结果，甚至什么都没有。

其次要问参数如何传递。按照常见 x86-64 System V ABI，前几个整数参数会放在寄存器中，返回值也放在寄存器中。此时一次简单加法函数可能只需要移动或相加寄存器，再返回。若函数没有内联，还要付出调用和返回的控制流成本。若调用跨共享库边界，还可能经过过程链接表。若第一次调用触发延迟绑定，第一次成本和后续成本又不同。

再次要问构建模式。调试构建可能保留帧指针、把局部变量放到栈上、减少寄存器优化，让汇编比发布构建复杂很多。发布构建可能只剩一两条指令，甚至完全没有独立函数。比较两个版本性能时，如果一个是调试构建，一个是发布构建，结论没有意义。

最后要问测量方式。如果把这个函数调用一次，然后用进程总时间判断成本，测到的主要是启动和计时器噪声。如果把函数调用放进大循环，又不消费结果，优化器可能删除循环。如果每次循环都打印结果，测到的主要是输出成本。正确测量需要让结果以编译器不能删除的方式被使用，同时把输出放在计时区域外，并重复足够多次。

这个例子说明：一行 C++ 表达式的真实性能，不只由表达式本身决定，还由编译可见性、优化级别、ABI、链接方式、加载行为和测量方法共同决定。本章的价值就在于让你知道每个问题应该落在哪一层。

动态库边界：为什么同一个调用可能有不同路径

现代 Linux 程序大量使用共享库。共享库让多个程序共享同一份库代码，减少磁盘和内存占用，也方便系统更新。但共享库同时引入了二进制边界。调用一个本地静态函数、调用同一目标文件里的函数、调用另一个目标文件里的函数、调用共享库导出的函数，在机器路径和优化机会方面都不完全一样。

如果函数在当前翻译单元可见，编译器可能内联它，也可能进行常量传播和删除无用分支。如果函数只在另一个目标文件里可见，普通编译阶段通常看不到它的实现，除非启用跨翻译单元优化。

如果函数位于共享库中，编译器还要遵守动态链接和符号可替换性的规则。在一些平台和选项下，默认可见符号可能被运行时的其他定义替换，这会限制编译器对调用目标的假设。

过程链接表和全局偏移表是动态链接中常见的机制。初学阶段不需要背它们的每个字段，但要知道它们解决的问题：共享库可以被映射到不同虚拟地址，外部函数和全局对象的位置可能需要运行时确定。为了让代码在不同加载地址仍然工作，二进制会使用间接表和相对寻址。于是一次外部函数调用可能不是直接跳到最终函数，而是经过一层跳转或解析入口。

动态库边界还影响部署和复现。你编译时链接的库版本，运行时未必就是实际加载的库版本。系统路径、本地路径、运行时路径、环境变量和容器镜像都会影响最终选择。如果性能突然变化，除了看源码改动，还要检查动态库是否变了。高质量报告应记录关键库版本，尤其是标准库、数学库、线程库和高性能计算库。

共享库也影响符号可见性。库作者可以选择导出哪些符号，隐藏哪些内部实现。隐藏内部符号有助于减少符号冲突，改善优化机会，也让 ABI 表面更稳定。一个面向长期维护的库，不能随意把所有内部函数暴露为公共 ABI，因为一旦外部用户依赖这些符号，后续修改就会受限。

深入理解

动态库是工程能力和性能复杂性的交换。它让程序复用、更新和部署更灵活，同时引入加载、符号解析、版本兼容、可见性和间接调用问题。以后分析库函数性能时，要先确认调用路径是否跨过动态库边界。

程序身份审计：确认你运行的是谁

真实工程里，“我已经重新编译了”并不等于“我运行的是新程序”。程序身份审计就是把源码、构建命令、目标文件、可执行文件、动态库和运行时进程对应起来。没有这一步，调试和性能测量都可能建立在错误对象上。

一个程序身份至少包含六个层面。

第一，源码身份。它包括 Git commit、工作区 diff、生成文件版本、配置文件和编译选项来源。若工作区有未提交修改，只记录 commit hash 不够。若构建系统自动生成头文件，也要记录生成输入。

第二，构建身份。它包括编译器路径和版本、标准库、目标架构、优化级别、宏定义、include 路径、链接库和链接顺序。两个二进制可能来自同一源码，却因为 `-O0` 与 `-O3`、`-march=native`、`-fPIC`、`LTO` 或 `sanitizer` 不同而完全不同。

第三，文件身份。可执行文件有路径、mtime、大小、build-id、符号表、section 和动态依赖。运行脚本若指向旧路径，或者 `PATH` 中先找到另一个同名程序，你可能在测旧二进制。

第四，库身份。动态库在链接时和运行时都参与。链接命令里的 `-lfoo` 不保证运行时加载的就是你以为的 `libfoo.so`。运行时搜索路径、`LD_LIBRARY_PATH`、`RPATH`、`RUNPATH`、系统缓存和容器镜像都会影响选择。

第五，进程身份。程序启动后成为进程，有 PID、当前工作目录、环境变量、打开文件、内存映射、线程和资源限制。两个进程运行同一个可执行文件，也可能因为环境变量和工作目录不同而行为不同。

第六，实验身份。benchmark 还要记录输入、seed、重复次数、计时区域、raw samples 和报告版本。程序身份正确，只说明运行对象对；实验身份正确，才说明测量对象对。

身份审计的最小命令组

可以用下面命令建立最小审计：

```
git rev-parse HEAD
git status --short
cmake --build build-release --verbose
realpath build-release/app
ls -l build-release/app
file build-release/app
readelf -n build-release/app | sed -n '1,80p'
ldd build-release/app
readelf -d build-release/app | rg 'NEEDED|RPATH|RUNPATH'
```

运行时再记录：

```
pwd
env | sort > results/env.txt
./build-release/app --version
cat /proc/$(pidof app)/maps > results/maps.txt
```

这些命令不是每次都要完整手敲，但严肃报告应保存等价信息。尤其是性能数据异常时，身份审计能快速排除“测错程序”“加载错库”“运行旧文件”“工作目录不对”这些低级但常见的问题。

build-id 和符号证据

许多 ELF 文件包含 build-id。它是识别二进制的一种常用方式。调试器、core dump 和符号服务器可以用 build-id 把崩溃现场对应到正确的二进制和调试信息。若你分析 core dump，却使用了另一个构建的可执行文件，反汇编地址和源码行可能完全错位。

符号表也能帮助确认身份。若你期望某个函数独立存在，但 nm 或 objdump 中找不到，可能是被内联、被删除、隐藏为局部符号、链接到另一个库，或者构建根本没有包含该文件。符号不存在不一定是错误，但它必须能解释。反过来，符号存在也不代表热路径一定调用它；调用可能被绕过，或者当前输入不走那条路径。

启动时间线：main 之前和之后

理解程序生命周期，必须知道 main 不是进程的第一个动作。一个普通动态链接 C++ 程序启动时，大致经历下面时间线：

1. shell 或父进程调用 `execve`。
2. 内核读取 ELF header，建立初始虚拟地址空间。
3. 内核映射可执行文件和动态链接器，准备初始栈。
4. 动态链接器加载共享库、处理重定位和符号解析。
5. C/C++ 运行时建立环境，处理初始化数组。
6. 全局和静态对象的动态初始化执行。
7. 控制流进入 `main`。
8. `main` 返回或调用退出函数。
9. 静态对象析构、`atexit` handler 和运行时清理执行。
10. 进程退出，内核回收资源。

不同平台、链接方式和运行时会有差异，但这条线足够帮助你避免两个误解。第一，进程启动成本不是你的核心算法成本。第二，全局对象初始化、动态库加载和首次符号解析可能在 `main` 前后引入额外行为。若 benchmark 只运行一个很短程序并测总 wall time，测到的很可能主要是启动和初始化。

全局初始化为什么影响实验

C++ 允许具有静态存储期的对象在 `main` 前动态初始化。例如某个全局 `std::vector`、全局 logger、全局 registry 或全局 benchmark 对象，可能在程序真正开始之前分配内存、打开文件、注册函数、读取环境变量。这些行为会影响启动时间，也可能影响 allocator 和 cache 状态。

性能实验应尽量避免让复杂全局初始化混入被测对象。更好的做法是显式 setup：在 `main` 或 runner 中创建输入、注册 kernel、分配输出，并把这些步骤放在计时区域外。若必须使用全局 registry，也要在报告中说明它属于 harness，而不是 kernel。

退出路径也可能有成本

程序结束时，静态对象析构、缓冲刷新、日志写入、临时文件清理和 runtime handler 都可能执行。如果用外部 `time./app` 测一个短程序，退出成本也包含在总时间里。对于端到端工具，这可能是合理指标；对于微基准，这不是核心 kernel 成本。计时区域应该在进程内部明确开始和结束，并把输出和清理放在外面。

从翻译单元到链接图

大型 C++ 工程不是“很多文件一起编译”，而是许多翻译单元分别编译成目标文件，再由链接器组合。理解这一点有助于解释为什么某些优化能发生，某些不能发生。

一个翻译单元由源文件经过预处理后形成。编译器前端和优化器在普通编译阶段主要看到当前翻译单元。若函数定义在另一个翻译单元，当前编译器通常只知道声明，不知道函数体。于是它不能普通内联，也不能基于函数体做常量传播。链接器后续能解析符号，但传统链接器不做完整高级优化。启用 LTO 后，更多 IR 被带到链接阶段，跨翻译单元优化才成为可能。

这解释了几个现象。把函数定义放进头文件，调用点可见，可能更容易内联；把函数放进单独 .cpp，普通构建可能保留调用；开启 LTO 后，原本跨文件的调用可能被内联；动态库边界可能限制跨边界优化；符号可见性影响编译器是否能假设某个函数不会被替换。

链接图不是调用图

链接图描述目标文件和库之间的符号依赖；调用图描述运行时函数可能调用谁。两者相关但不同。一个目标文件依赖某个库符号，不代表每次运行都会调用该符号；一个间接调用可能在链接图中看不出具体目标；一个函数被内联后，运行时没有普通调用边，但链接图中可能仍有其他符号依赖。

性能分析中要避免把链接依赖当作热路径。ldd 显示程序依赖某个库，不代表热点在该库；nm 显示某个函数存在，不代表当前输入执行它；反汇编显示某个调用存在，也不代表它在样本中占比高。热路径需要运行时 profiling 或至少输入路径分析支持。

程序生命周期中的证据层级

本章的所有材料可以整理成证据层级：

层级	典型证据	能回答的问题
源码层	.cpp、头文件、宏、Git diff	程序员写了什么语义意图
构建层	编译命令、构建日志、compile database	编译器按什么条件工作
目标层	.o、符号表、重定位	每个翻译单元产出了什么
链接层	可执行文件、动态段、build-id	最终二进制依赖什么
加载层	ldd、/proc/pid/maps	运行时映射了哪些文件
执行层	GDB、trace、profiling、samples	程序实际走了哪些路径
测量层	raw CSV、summary、环境记录	运行成本如何分布

做报告时，先判断你的结论需要哪一层证据。若结论是“源码使用了某个算法”，源码层足够；若结论是“这个函数没有被内联”，需要二进制层；若结论是“运行时调用了共享库版本”，需要加载或执行层；若结论是“该实现更快”，需要测量层，同时还要有正确性和二进制证据。

核心思想

程序从源码到进程不是一条黑箱通道，而是一串可观察边界。高质量工程分析会选择与结论匹配的证据层级，而不是把源码直觉、反汇编片段和时间数字混在一起。

综合案例：从一份崩溃报告回到生命周期

假设你收到一份崩溃报告：程序在用户机器上启动后立即 SIGSEGV，本地无法复现。一个没有生命周期模型的排查方式，是直接在源码里搜索可能空指针的位置。更稳的方式是按本章链路倒推。

第一步，确认运行的是哪个二进制。崩溃报告是否包含 build-id？用户机器上的可执行文件是否与你本地调试的文件一致？是否 stripped？调试符号是否匹配？如果 build-id 不一致，后续源码行和反汇编地址都可能错。

第二步，确认加载了哪些共享库。用户机器是否加载了旧版本 libfoo.so？RPATH、RUNPATH、LD_LIBRARY_PATH 是否改变？崩溃发生在 main 前，还是进入业务代码后？如果崩溃在动态初始化或共享库构造函数中，业务函数断点可能根本不会触发。

第三步，确认入口和初始化。崩溃是否发生在全局对象初始化、线程局部对象初始化、配置读取、日志系统启动、插件扫描、CPU dispatch 初始化？这些都可能发生在 main 之前或刚进入 main 时

发生。报告中“启动后立即崩溃”不等于“第一行业务代码崩溃”。

第四步，确认环境。用户当前工作目录、配置文件路径、权限、环境变量、CPU ISA、内核版本、容器限制是否与本地不同？如果程序启动时加载配置或选择 ISA kernel，环境差异可能直接决定路径。

第五步，映射地址到模块。core dump 中的 rip 是运行时虚拟地址，需要通过进程映射找到它属于可执行文件、共享库、JIT 区域还是匿名映射。只有映射到正确模块，才能用对应二进制反汇编。不要把地址直接拿去本地二进制里查，ASLR 和不同构建会让地址失效。

第六步，回到源码和构建。确认崩溃指令后，再看它来自哪个源码路径、哪个编译选项、哪个目标文件。若崩溃在手写汇编或 ABI 边界，要检查参数寄存器、栈对齐和 callee-saved；若崩溃在内存 load，要检查对象生命周期和地址来源；若崩溃在间接调用，要检查函数指针、虚表和动态库版本。

这个案例说明，运行时问题经常跨越多个阶段。源码只是证据之一。真正的排查路径是：二进制身份、加载身份、进程环境、崩溃地址、反汇编、源码语义。跳过前面阶段，可能会在错误源码上调试正确问题。

综合案例：性能比较中的“旧库”问题

再看一个性能案例。团队说新算法比旧算法慢，但反汇编显示新算法热循环更短。最后发现运行时加载的是系统安装的旧共享库，而不是刚编译的本地库。

这个问题在生命周期链路中很典型。编译阶段生成了新的 libfast.so；链接阶段可执行文件记录了需要 libfast.so；加载阶段动态链接器按照搜索路径找到了另一个目录下的旧库；执行阶段自然没有运行新代码；测量阶段却把结果归因到新算法。

排查证据包括：

- ldd./app 显示实际库路径。
- readelf-d./app 显示 RPATH/RUNPATH。
- /proc/<pid>/maps 显示运行时映射。
- nm-D 或 build-id 显示库版本。
- 反汇编旧库和新库，确认热循环差异。

修复可能是设置正确的运行时路径、调整安装目录、使用绝对路径测试、清理系统旧库，或在程序启动时打印实现版本。无论哪种修复，报告都应说明“之前测到的是旧库”，并废弃原性能结论。

常见误区

性能比较必须证明运行时加载的是预期二进制。只证明源码改了、构建过了、链接过了，还不够。

深入讲解：为什么“程序”这个词容易误导

日常语言里，我们会说“运行这个程序”“这个程序有 bug”“这个程序很慢”。但在工程分析里，“程序”这个词太粗。它可能指源码项目、某个目标文件、某个可执行文件、某个正在运行的进程、某个动态库版本、某次 benchmark 配置，甚至某个运行路径。不同含义混在一起，就会产生大量误判。

例如“这个程序很慢”可能有至少六种解释。第一，源码算法复杂度高。第二，编译器没有生成预期优化。第三，链接到了慢版本库。第四，程序启动和初始化成本高。第五，当前输入触发了冷路径。第六，benchmark 把 I/O 或分配放进计时区域。每一种解释对应不同证据和不同修复。如果不拆开“程序”，就只能凭经验猜。

再如“这个程序崩溃”也不够精确。崩溃可能发生在动态链接器加载共享库时，可能发生在全局对象初始化时，可能发生在 main 开始前，可能发生在某个线程里，可能发生在退出析构阶段。崩溃地址属于哪个模块、哪个映射、哪个 build-id，决定后续怎么找源码和反汇编。

因此，本章建议在报告中避免单独使用“程序”作为分析对象。应该写：

- 源码项目：哪个 commit，是否 dirty。
- 构建产物：哪个路径，哪个 build-id，哪个编译参数。
- 运行实例：哪个 PID，哪个环境，哪些动态库映射。
- 执行路径：哪个函数、哪个输入、哪个分支、哪个实现版本。

- 测量对象：哪个 timed region，哪些样本，哪个 summary。

这不是文字洁癖，而是工程精度。对象命名越准确，证据越容易匹配；对象命名越模糊，调试越容易绕圈。

深入讲解：构建系统也是程序语义的一部分

初学者常把构建系统当成“把代码编译出来的工具”，而不是程序行为的一部分。实际上，构建系统决定哪些源文件进入翻译单元，哪些宏被定义，哪些 include 路径优先，哪些库被链接，哪些编译选项启用，哪些生成文件参与构建。这些都会改变最终程序。

一个宏可能改变结构体布局；一个 include 路径可能让你使用了另一个版本头文件；一个链接库顺序可能改变解析到的符号；一个 `-DDEBUG` 可能删除断言；一个 `-ffast-math` 可能改变浮点语义；一个 `-fno-exceptions` 可能改变异常边界；一个 `-march=native` 可能生成不能在其他机器运行的指令。这些都不是构建系统外部的“小配置”，而是程序身份的一部分。

因此，工程报告必须保存构建命令。CMake、Make、Ninja、Bazel 等工具隐藏了很多命令细节。你要能导出 verbose build 或 compile database。只写“用 Release 构建”不够，因为不同项目的 Release flags 可能不同；同一个项目不同机器上的 toolchain file 也可能不同。

构建系统还影响增量构建正确性。某个头文件变了但目标文件没重新编译，某个生成文件没更新，某个旧目标文件残留在 build 目录，都可能造成“源码看起来对，二进制不对”。遇到诡异问题时，clean build 是一种验证，但不要把 clean build 当作长期解决方案；长期应修复依赖关系和构建规则。

深入讲解：动态链接是运行时选择

动态链接的核心复杂性在于，最终绑定不只发生在编译时。可执行文件记录需要某些共享库和符号，动态链接器在运行环境中找到库、映射库、处理重定位，并可能在第一次调用时解析函数地址。运行时环境参与了程序身份。

这对性能和调试都有影响。第一次调用某个外部函数可能触发 lazy binding，导致第一次比后续慢；不同库版本可能有不同实现；符号可见性和 interposition 可能限制编译器优化；容器和系统环境可能加载不同库。高质量 benchmark 应尽量预热或固定动态链接状态，并记录库路径。

动态链接还带来 ABI 稳定问题。共享库的公共符号一旦被外部使用，修改签名或结构体布局就可能破坏旧程序。库作者需要控制导出符号，隐藏内部实现，使用版本脚本或命名策略。调用者则需要记录库版本，避免把系统升级造成的变化误判为源码性能变化。

第一册不要求你成为动态链接专家，但要求你养成两个习惯：看到外部调用时想到 PLT/GOT 和库版本；做实验时记录实际加载的共享库。这样后续分析 ABI 和互操作时，才不会把运行时选择误认为编译时事实。

地址空间观察报告应该怎样写

本章有一个观察地址空间的实验。这个实验的目的不是让你背某次运行中打印出的具体地址，而是训练解释方式。一份合格报告至少应包含四类信息。

第一类是构建和运行条件。报告要说明编译器、优化级别、是否带调试信息、系统类型、是否启用地址随机化、运行次数和运行命令。地址空间布局和符号位置可能受这些条件影响。没有条件说明，地址比较没有意义。

第二类是对象分类。你应该把函数地址、已初始化全局变量、零初始化全局变量、字符串字面量、栈上局部变量、堆分配对象、容器内部缓冲区分别归类。不要只写“地址不同”。要说明这些对象分别属于代码、可写数据、BSS、只读数据、栈、堆或堆内部二次分配。容器对象本身如果在栈上，它的控制字段在栈上；容器管理的元素缓冲区通常在堆上。这种区分非常重要。

第三类是跨运行比较。你可以比较同一对象类别在两次运行中的地址是否变化，也可以比较同一次运行中不同类别地址的大致关系。若地址变化，应解释这可能来自地址空间布局随机化、动态库映射、堆分配状态或栈位置变化。若某些相对关系稳定，也要谨慎说明它只是当前环境下的观察，不是 C++ 语言保证。

第四类是与映射文件对应。只打印地址还不够，最好结合进程映射信息，把地址落到某个映射区间中。一个地址如果落在可执行文件的可执行映射内，可以解释为代码；落在可写映射内，可能

是全局可写数据；落在栈映射内，通常是栈对象；落在匿名可写映射内，可能与堆或内存分配器有关。映射区间的权限也很重要，可读、可写、可执行权限能帮助解释为什么某些写入会崩溃。

核心理想

地址实验的目标不是记住地址，而是把“对象类别、虚拟地址、映射区间、权限、运行差异”连起来。能建立这种连接，后续学习栈帧、堆分配、页面、TLB 和缓存时才不会断层。

从本章到后续章节的知识接口

本章结束后，后续章节会不断复用这里的概念。第二章讲 CPU 如何执行指令时，会把可执行文件中的机器码看成指令字节，再追踪取指、译码、执行和提交。第三章讲数据、内存和指针时，会把虚拟地址空间进一步细化到对象布局、对齐、数组寻址和生命周期。工具章节会把这里提到的预处理、反汇编、符号和调试命令组织成日常工作流。

进入汇编部分时，本章的链接和 ABI 背景会变得更重要。你看到寄存器传参、栈对齐、返回地址、调用者保存和被调用者保存规则时，要记得它们不是某个编译器的随意选择，而是二进制模块互相调用所需的约定。没有这些约定，不同源文件、不同库甚至不同语言写出的函数就无法在机器码层可靠协作。

进入测量部分时，本章的加载、运行时启动和进程资源背景会变得更重要。你需要把计时区域与程序启动分离，把动态链接和首次缺页与稳态执行分离，把输出、分配、输入准备和结果验证与核心 kernel 分离。否则，测量章节再讲统计方法也救不了错误的实验边界。

进入第二册的缓存、SIMD 和 AI 算子时，本章依然有用。高性能 kernel 往往以库函数、模板实例化、ISA 分发和动态链接的形式出现。你必须确认当前机器执行的是哪一个实现，确认编译器是否生成了目标指令，确认数据是否按预期布局，确认测量没有混入加载和调度噪声。也就是说，越往高性能方向走，越不能忘记最基础的“源码如何变成进程”。

程序身份审计：先确认你在分析什么

很多排查失败不是因为缺少高级知识，而是因为一开始就分析错了对象。源码窗口里打开的是新代码，终端里运行的是旧可执行文件；当前目录下有一个库，动态链接器实际加载的是系统库；命令行参数写在笔记里，真实运行脚本又加了另一组参数；编译器输出了目标文件，最终可执行文件却链接了缓存中的旧对象。这些情况都属于程序身份不清。

程序身份审计要回答五个问题。第一，源码身份是什么，包括 Git 提交、工作区是否干净、关键生成文件是否来自当前源码。第二，构建身份是什么，包括编译器、编译参数、宏定义、include 路径、链接库路径和构建类型。第三，二进制身份是什么，包括可执行文件绝对路径、文件时间、校验和、架构、调试信息和 build id。第四，运行身份是什么，包括命令行、工作目录、环境变量、输入文件、配置文件、权限和容器环境。第五，加载身份是什么，包括共享库实际路径、RPATH/RUNPATH、LD_LIBRARY_PATH、符号解析和插件路径。

只有这五类身份一致，后面的调试和测量才有基础。若某个身份不清，结论应降级。例如你可以写“当前反汇编来自 `build/app`，但尚未确认线上进程是否使用同一 build id”，而不要写“线上执行了这段指令”。工程报告中承认身份缺口，比假装确认更可靠。

最小身份记录

一次可复现实验至少应保存下面信息：

```
source:
  git commit
  git status --short

build:
  compiler version
  full compile and link commands
  build type and important macros

binary:
  absolute path
  sha256 or build-id
```



```

file output
ldd output if dynamically linked

run:
working directory
command line
environment differences
input and config paths

```

这些信息看似普通，却能排除大量伪问题。比如 benchmark 结果突然变化，先比对二进制校验和和动态库路径，可能立刻发现新旧版本混用。崩溃只在线上发生，先比对 build id 和符号文件，才能保证 core dump 对应正确源码。链接错误只在某台机器出现，先比对 include 路径和库路径，往往比猜 C++ 语义更快。

启动路径排查：从 `execve` 到 `main`

普通 C++ 程序不是从 `main` 的第一行凭空开始。操作系统创建进程映像，动态链接器映射共享库，运行时建立栈和初始化环境，C/C++ runtime 调用全局构造，最后才进入 `main`。若问题发生在启动阶段，盯着 `main` 里的代码可能没有用。

启动失败可以按时间线排查。第一阶段是执行文件选择：shell 是否找到了你以为的路径，脚本是否调用了另一个可执行文件，文件是否有执行权限，架构是否匹配。第二阶段是加载：ELF 解释器是否存在，共享库是否找到，版本符号是否满足，重定位是否成功。第三阶段是运行时初始化：全局对象构造是否崩溃，线程局部对象是否初始化，环境变量是否改变库行为。第四阶段才是 `main` 中的用户逻辑。

`strace` 对启动路径非常有用，因为它能看到 `execve`、`openat`、`mmap`、`access` 等系统调用。若程序报告找不到配置文件，`strace` 能告诉你它实际尝试打开哪些路径。若程序报告找不到共享库，动态链接器错误、`ldd`、`readelf-d` 和环境变量要一起看。若程序在 `main` 前崩溃，GDB 的 `backtrace`、全局构造和共享库初始化顺序就进入排查范围。

为什么启动成本不能混进 kernel 成本

性能测量必须区分启动成本和稳态 kernel 成本。启动阶段可能包含动态链接、缺页、全局构造、内存分配、配置解析、线程池创建、JIT 或首次符号解析。这些成本可能很大，但它们不是某个热循环每次调用的成本。若把整个程序运行时间当作 kernel 时间，就会把启动噪声误判为算法差异。

这并不意味着启动成本不重要。命令行工具、短生命周期服务、插件加载和在线推理冷启动都可能关心启动时间。关键是要清楚：你测的是端到端启动，还是核心循环吞吐。前者应保留加载和初始化；后者应把它们移出 `timed region`。两类实验都合理，但不能混用同一结论。

从构建图理解链接错误

链接错误常被初学者当成“编译器报错”，但它其实发生在更后面的阶段。编译每个翻译单元时，编译器只需要知道声明是否可用；链接时，链接器才需要把未定义符号解析到某个目标文件或库中的定义。若声明存在但定义缺失，编译可以通过，链接会失败。

排查链接错误要画构建图。节点包括源文件、目标文件、静态库、共享库和最终可执行文件。边表示编译或链接依赖。看到 `undefined reference` 时，要问：哪个目标文件引用了符号，哪个库应该提供符号，该库是否出现在链接命令中，顺序是否正确，符号名是否因为 C++ name mangling 不匹配，函数签名是否一致，目标架构是否一致，符号可见性是否隐藏。

这套方法也适用于“能链接但运行错”的情况。链接成功只表示某个符号被解析，不表示解析到的是你想要的实现。弱符号、符号 `interposition`、动态库搜索路径、插件系统和版本脚本都可能改变最终绑定。性能报告中若声称使用了某个优化库，应保存符号和动态加载证据，而不是只看 CMake 配置。

综合案例：同一源码为什么跑出两个程序

设想你修改了 `fast_sum.cpp`，本地运行 benchmark 发现没有任何变化。第一反应不应该是“优化无效”，而应该做身份审计。检查编译日志发现目标文件没有重新生成，因为构建系统没有把某个生成头文件列为依赖。清理构建目录后重新编译，二进制校验和改变，反汇编中出现新循环，benchmark 才显示变化。

另一个常见版本是动态库混用。你编译了 `build/libfast.so`，可执行文件启动时却加载 `/usr/lib/libfast.so`。源码和本地库都正确，进程执行的却是系统库。此时 `ldd`、`readelf-d`、`LD_DEBUG=libs` 和 `/proc/<pid>/maps` 能提供证据。若只盯着源码，会一直误判。

这个案例说明，程序不是一个抽象名词，而是一组具体产物和运行条件。底层学习从第一章就要求读者建立这种意识：任何现象都要先绑定到明确对象。后面读汇编、查 ABI、测性能，都要继承这个习惯。

编译选项也是语义环境

编译选项不只是速度开关。优化级别、目标架构、异常、RTTI、断言、浮点模型、栈保护、可见性、链接方式、LTO、调试信息，都会改变最终程序。两个可执行文件即使来自同一源码，只要编译选项不同，就应视为不同实验对象。

`-O0` 适合观察接近源码结构的机器码，但它不是性能对象。`-O3` 更接近优化构建，但会内联、删除、重排和向量化。`-march=native` 可能生成当前机器支持而其他机器不支持的指令。`-DNDEBUG` 会删除 `assert`，可能改变错误路径。`-ffast-math` 会放宽浮点语义，不能和严格浮点版本直接当作同一程序比较。

因此，报告不能只写“用 clang 编译”。要写完整命令，至少写关键 flags。若使用 CMake，也要保存 `verbose build` 或 `compile commands`。构建环境越复杂，越需要把隐含选项显式化。

源文件边界和翻译单元思维

C++ 不是把整个项目一次性解释执行。每个源文件经过预处理后形成翻译单元，编译器分别编译，再由链接器组合。这个边界影响声明可见性、内联、模板实例化、静态对象、匿名命名空间、内部链接和外部链接。

很多问题必须用翻译单元思维解释。头文件里只写声明，没有定义，编译可能通过但链接失败；在一个源文件中定义了 `static` 函数，另一个源文件不能引用；模板定义若不在头文件中可见，实例化可能失败；两个源文件各自有同名内部链接对象，它们不是同一个对象。链接器看到的是符号和目标文件，不是完整 C++ 语义。

优化也受边界影响。没有 LTO 时，编译器通常看不到其他翻译单元的函数体，只能按声明和 ABI 生成调用。有 LTO 时，边界可能被打通，函数被内联，符号消失，性能和反汇编都变化。看到函数“没有了”时，要先问是否内联、LTO、死代码删除或符号可见性改变，而不是立刻判断源码没有执行。

运行时资源也是程序的一部分

进程运行不仅依赖二进制，还依赖资源：配置文件、输入数据、环境变量、当前目录、文件描述符、网络、权限、CPU 亲和性、动态库、插件、临时目录。对于系统软件和性能实验，这些资源会直接改变行为。一个程序在 IDE 中能运行，在终端失败，常常只是工作目录或环境不同。

记录运行时资源，是为了让问题可复现。若程序打开相对路径，报告应写当前工作目录；若读取环境变量，报告应记录关键变量；若使用动态库和插件，报告应记录实际路径；若 benchmark 绑定 CPU 或设置线程数，报告应记录这些设置。程序身份不是二进制文件单独决定的，而是二进制加运行资源共同决定的。

构建可复现性的工程底线

可复现构建不一定要每个字节在所有机器上完全相同，但至少要让别人知道如何得到同类产物。第一层是命令可复现：源码、编译器、参数、依赖和构建脚本明确。第二层是环境可复现：系统、容器、工具链、库版本和路径记录清楚。第三层是产物可识别：生成的目标文件和可执行文件有校验和、build id 或版本信息。第四层是运行可复现：输入、配置、环境变量和工作目录可重建。

缺少这些底线，后续章节的所有分析都会变弱。你不能稳定生成同一类二进制，就无法比较汇编；不能确认加载库，就无法讨论 ABI；不能复现输入，就无法比较性能。可复现性不是发布工程才需要的事，从第一个实验开始就应训练。

实际项目可以逐步提高要求。个人实验先保存命令和环境；团队项目加入 `compile database`、锁定依赖和 CI 构建；发布项目再考虑容器、构建元数据、符号服务器和制品归档。要求逐步提高，但核心思路不变：程序不是“我刚才运行的那个”，而是可命名、可追溯、可复查的产物。

把程序启动写成一张时间线

学习本章后，建议把任意一个程序的启动写成时间线。时间线从 shell 解析命令开始，到 `execve`，到内核加载 ELF，到账户权限和文件描述符继承，到动态链接器映射共享库，到运行时初始化，到全局构造，到进入 `main`，再到用户代码读取输入。每一步都写出可能失败的原因和可用工具。

这张时间线能帮助读者处理很多现实问题。程序启动时报共享库缺失，定位在动态链接阶段；全局对象构造崩溃，定位在 `main` 之前；相对路径找不到文件，定位在工作目录和资源阶段；第一次调用慢，可能涉及 lazy binding、缺页或初始化；benchmark 把启动时间算进去，说明计时边界错了。

时间线思维也能防止把所有问题都归咎于源码逻辑。进程是源码、二进制、运行时和操作系统共同建立的执行实例。能把启动拆开，才真正理解“程序到进程”的含义。

项目里的“程序”往往不止一个

一个仓库中常常不只有一个程序。它可能包含命令行工具、测试程序、benchmark、动态库、插件、生成器、脚本和示例。每个产物都有自己的入口、链接关系、运行资源和使用场景。读者在报告中写“程序”时，最好直接写具体产物名和路径，而不是用一个模糊名词。

这种区分对性能尤其重要。测试程序为了覆盖边界，可能使用 Debug 或 sanitizer；benchmark 为了测热路径，可能使用 Release 和固定输入；命令行工具关心端到端启动；动态库关心 ABI 和符号可见性。把这些产物混在一起，会造成错误比较。比如用测试程序的时间评价库性能，或用 benchmark 的输入假设评价用户场景，都不严谨。

构建系统也要反映这种区分。不同 target 应有清楚的编译参数、依赖和输出目录。报告中应说明分析的是哪个 target，是否带 LTO，是否静态链接，是否启用 sanitizer，是否包含测试辅助代码。只有产物身份清楚，后面的汇编、ABI 和测量才有对象。

本章的最后提醒：不要跳过身份确认

以后每次遇到底层问题，都先问“我正在分析哪个产物”。这句话看似普通，却能避免大量错误。运行路径、构建目录、动态库、输入文件、配置、环境变量、符号文件，只要有一个不匹配，后面的推理都可能偏离。

身份确认不是慢动作，而是加速器。它让你少走弯路，也让报告更可信。第一章打下的这个习惯，会一直延伸到最后一章：可信性能数据同样需要可信程序身份。

一个最小身份清单

以后做任何实验，至少写下五行：源码提交，构建命令，二进制路径，运行命令，输入路径。若涉及共享库，再加实际加载路径；若涉及性能，再加机器和编译器版本。这个清单很短，却能让多数实验从“我记得”变成“可复查”。

不要等项目变复杂才开始记录。小实验越早养成习惯，大项目越不容易混乱。

这五行还会帮助你未来的自己沟通。几个月后重新打开实验目录时，你不需要猜当时运行了什么，也不需要凭记忆恢复环境。底层学习中，很多“遗忘”其实不是知识遗忘，而是当初没有给证据留下入口。

身份记录也会降低协作成本。别人接手你的实验时，最怕看到一堆输出文件却不知道它们来自哪次构建、哪个命令和哪个输入。清单越短，越要稳定；稳定的小清单比临时写的大段回忆更可靠。以后分析崩溃、汇编、ABI 或性能时，这个清单就是所有证据的根目录。

把身份写清楚，也是在训练工程耐心。底层问题往往不是没有答案，而是证据对象混乱导致答案无法归属。先把对象钉住，后面的推理才有落点。

这一步越早做，后面越省时间，也越少产生误判。

当实验结果需要交给别人复查时，身份清单就是最小交接协议。它让源码、构建、二进制、输入和运行环境之间的关系保持可追踪，也让后续章节中的每一条证据都有明确归属。

没有归属，证据就会失去工程价值，也难以复查。

本章小结：从文本到执行实例

现在可以把整条链路重新压缩成一句话：程序员写的是源码，编译系统把它变成带元数据的二进制文件，操作系统和运行时把二进制文件变成进程，CPU 在进程上下文中执行指令，而性能测量记录

的是这个执行过程在某个环境中的表现。

这句话里的每个名词都不能省略。没有源码，就没有语言语义；没有编译系统，就没有优化和目标机器选择；没有二进制元数据，加载器不知道如何建立进程；没有操作系统和运行时，普通 C++ 程序无法获得参数、栈、库和资源；没有 CPU 执行，机器码只是文件；没有测量上下文，时间数字没有解释力。

完成本章后，你不需要立刻掌握 ELF 的所有字段，也不需要背下动态链接的全部重定位类型。但你必须能在脑中画出这条路径，并在遇到现象时判断应该去哪一层找原因。这是后面学习汇编、内存、ABI 和 benchmark 的共同基础。

贯穿实验：拆开一个程序

创建文件：

```
mkdir -p scratch/ch01
cat > scratch/ch01/add.cpp <<'EOF'
extern "C" int add_i32(int a, int b)
{
    return a + b;
}
EOF
```

逐层生成中间产物：

```
g++ -E scratch/ch01/add.cpp -o scratch/ch01/add.i
g++ -S -O0 -masm=intel scratch/ch01/add.cpp -o scratch/ch01/add_00.s
g++ -S -O3 -masm=intel scratch/ch01/add.cpp -o scratch/ch01/add_03.s
g++ -c -O3 scratch/ch01/add.cpp -o scratch/ch01/add.o
objdump -drwC -Mintel scratch/ch01/add.o
```

观察任务：

1. `.i`、`.s`、`.o` 分别是什么。
2. `-O0` 和 `-O3` 汇编差异。
3. `objdump` 输出中的机器码字节。
4. 函数符号名是否保留为 `add_i32`。

实验：观察地址空间

```
cat > scratch/ch01/address.cpp <<'EOF'
#include <iostream>
#include <vector>

int global_initialized = 42;
int global_zero;
char const* message = "hello";

int main()
{
    int stack_value = 1;
    auto* heap_value = new int(2);
    std::vector<int> vec(16, 3);

    std::cout << "main" << reinterpret_cast<void*>(&main) << '\n';
    std::cout << "global_initialized" << &global_initialized << '\n';
    std::cout << "global_zero" << &global_zero << '\n';
    std::cout << "message variable" << &message << '\n';
    std::cout << "message literal" << static_cast<void const*>(message) << '\n';
    std::cout << "stack_value" << &stack_value << '\n';
    std::cout << "heap_value" << &heap_value << '\n';
    std::cout << "vector data" << &vec.data() << '\n';

    delete heap_value;
}
EOF
```

```
g++ -O0 -g scratch/ch01/address.cpp -o scratch/ch01/address
./scratch/ch01/address
./scratch/ch01/address
```

报告任务：

1. 比较两次运行地址是否一致。
2. 推测哪些属于 text、data、BSS、rodata、stack、heap。
3. 解释为什么地址可能每次变化。
4. 查看 `/proc/self/maps` 的方式，并解释它能提供什么信息。

反例：看似合理但错误的理解

常见误区

误区一：C++ 程序从 main 的第一行开始。

不准确。进程入口通常是 runtime 启动代码，main 是被运行时调用的用户入口。

误区二：头文件会被单独编译。

通常不对。头文件被包含进源文件，形成翻译单元。

误区三：目标文件就是可执行文件。

不对。目标文件还需要链接。

误区四：源码里有函数，机器码里一定有这个函数。

不对。优化后函数可能被 inline、删除、合并或改变可见性。

误区五：指针就是物理地址。

用户态程序看到的是虚拟地址。

作业

习题与作业

作业 1：画出完整流程。

画出从 `hello.cpp` 到正在运行进程的完整路径。每一层写明：

- 输入是什么。
- 输出是什么。
- 哪个工具或系统组件负责。
- 性能学习为什么要关心。

作业 2：函数消失实验。

写三个函数：

- 一个允许 inline。
- 一个使用 `noinline` 阻止 inline。
- 一个从不调用。

分别用 `-O0` 和 `-O3` 编译，用 `nm` 和 `objdump` 观察哪些符号存在，哪些函数消失。

作业 3：启动成本和 kernel 成本。

写一个程序，让 `main` 调用某个小函数一次，再调用同一函数一亿次。分别测整个程序时间和循环区域时间。解释为什么启动成本不能代表 kernel 成本。

验收标准

完成本章后，你应该能：

- 解释预处理、编译、汇编、链接、加载的区别。
- 生成并解释 `.i`、`.s`、`.o` 和可执行文件。
- 用 `readelf`、`nm`、`objdump` 做基础观察。

- 解释为什么 `main` 不是真正的进程入口。
- 画出简化虚拟地址空间。
- 说明为什么性能测量要排除启动、初始化、动态链接和全局构造等因素。

第 3 章

CPU 如何执行指令

问题与目标

上一章已经把一份 C/C++ 源码拆成了预处理、编译、汇编、链接、加载和进程执行。那条路径告诉我们：源代码最后会变成可执行文件中的机器码字节，操作系统把这些字节映射到进程的虚拟地址空间里，然后 CPU 从某个入口地址开始执行。

本章继续向下推进一层，回答一个基础但关键的问题：

CPU 到底怎样把一条指令变成一次真实的机器行为？

初学阶段，这个问题经常被几种简化说法遮住：

- “CPU 执行 C++ 代码”。
- “汇编就是 CPU 直接看到的东西”。
- “一条汇编就是一个硬件动作”。
- “CPU 一条一条顺序执行，所以源代码顺序就是真实执行顺序”。
- “内存就是一个大数组，访问一个变量就是直接去内存取它”。

这些说法都有合理成分，但都不足以解释真实执行。为了理解程序行为、性能测量和后续 CPU kernel 开发，必须建立更细的模型。本章不要求你立即掌握现代乱序执行核心的全部细节，但要先形成一条稳定链路：

C/C++ 表达式 → 汇编文本 → 机器码字节 → 指令语义 → 架构状态变化 → 微架构执行过程 → 性能现象

后续优化都会反复使用这条链路。分析一个循环为什么慢，最终仍要回到下面几个问题：

- 它生成了哪些指令？
- 这些指令读写哪些寄存器和内存？
- 哪些指令依赖前面的结果？
- 哪些访问会打到 cache，哪些会去更远的内存层级？
- 哪些分支会打断前端供给？
- CPU 内部能否把多条指令重叠执行？

核心思想

本章不是把 CPU 讲成神秘黑盒，而是把 CPU 拆成若干可观察、可推理、可实验验证的层次。性能工程的第一原则是：先确认机器真正执行了什么，再讨论优化。

从一个最小函数开始

考虑这个 C++ 函数：

```
extern "C" int add_i32(int x, int y)
{
    return x + y;
}
```

在 x86-64 System V ABI 下，前两个整数参数通常通过 `edi` 和 `esi` 传入，32 位整数返回值通常放在 `eax`。一个容易理解的汇编版本可以写成：

```
mov eax, edi
add eax, esi
ret
```

这三行已经包含本章的核心问题：

- `mov` 为什么能把一个参数复制到返回寄存器？
- `add` 读了什么，写了什么？
- `ret` 为什么能回到调用者？
- CPU 怎样知道下一条该执行哪条指令？
- 这些汇编文本进入可执行文件后到底是什么？

先强调一个事实：CPU 不知道变量名 `x` 和 `y`。变量名、函数名、类型名属于源代码和调试信息层；CPU 执行时看到的是指令字节，操作的是寄存器、内存地址和状态位。

如果把这个函数放进目标文件，用 `objdump-d-Mintel` 观察，机器码字节可能类似：

```
89 f8      mov eax, edi
01 f0      add eax, esi
c3         ret
```

也可能在优化后看到：

```
8d 04 37   lea eax, [rdi+rsi]
c3         ret
```

编译器选择哪一种，取决于优化级别、目标 CPU、上下文和它是否需要保留 flags。这里先不急着手判断哪种更好。本章先建立一个原则：

心智模型

汇编是人类可读的指令表示，机器码是 CPU 取指单元读取的字节序列。C++ 源码不是 CPU 的输入，汇编文本通常也不是 CPU 的最终输入；真正被执行的是内存中的机器码字节。

CPU 是什么：从状态机开始理解

很多教材会直接说 CPU 由控制器、运算器和寄存器组成。这个说法没错，但对性能学习还不够。我们需要更底层一点的心智模型：CPU 是一个巨大而高速的状态机。

所谓状态机，就是它有一组状态，每个时钟周期根据当前状态和输入计算出下一个状态。对程序员来说，最重要的状态包括：

- 当前要执行的位置，也就是 instruction pointer，在 x86-64 中叫 `rip`。
- 通用寄存器，例如 `rax`、`rbx`、`rcx`、`rdx`、`rdi`、`rsi`。
- 栈指针 `rsp`。
- 条件标志位 RFLAGS，例如 zero flag、carry flag、sign flag、overflow flag。
- 内存中可见的数据。

从硬件实现角度看，CPU 内部有大量组合逻辑和时序逻辑。

组合逻辑 (combinational logic) 的输出只由当前输入决定。加法器、比较器、多路选择器、地址生成单元都可以从这个角度理解。给加法器两个输入位模式，它经过门电路传播后给出结果和进位。

时序逻辑 (sequential logic) 会保存状态。寄存器、流水线寄存器、重排序缓冲区中的条目、cache tag 阵列中的状态都属于这一类。时钟边沿到来时，一部分计算结果被锁存成新的状态。

你不需要在第一册就掌握电路设计，但必须理解一点：机器执行不是抽象概念，它最终落实为状态的读、组合逻辑的计算和状态的写。

```

old state + instruction bytes
      |
      v
decode instruction meaning
      |
      v
read operands -> compute -> write results
      |
      v
new state

```

这就是最朴素的 CPU 执行图景。现代 CPU 会把这个过程拆开、重叠、预测、乱序执行、再按程序顺序提交结果，但它必须最终制造出和 ISA 规定一致的可见状态。

ISA：软件和硬件之间的契约

ISA（instruction set architecture，指令集体系结构）是软件和处理器之间的契约。对 x86-64 来说，ISA 规定了：

- 有哪些寄存器。
- 每条指令的编码格式。
- 每条指令读写什么操作数。
- 指令如何改变寄存器、内存、flags 和 `rip`。
- 异常、中断、权限、页表等系统级行为如何表现。

ISA 不规定 CPU 内部必须如何实现。例如同样是执行 `add eax, esi`：

- 一个简单教学 CPU 可以顺序取指、解码、读寄存器、加法、写回。
- 一个现代桌面 CPU 可能把它解码为 micro-op，进入调度队列，等待操作数 ready，在某个整数执行端口上执行，最后在 retire 阶段提交。
- 一个仿真器可以用软件解释这条指令。

只要对程序可见的结果一致，它们都符合 ISA。

架构状态（architectural state）是 ISA 规定的软件可见状态，例如通用寄存器、`rip`、`RFLAGS` 和内存。**微架构**（microarchitecture）是某个具体 CPU 实现 ISA 的内部方式，例如 pipeline、decode queue、uop cache、register renaming、reservation station、load/store queue、L1 cache、TLB 和 branch predictor。

核心思想

ISA 决定正确性语义，微架构决定性能表现。同一份 x86-64 程序在不同 CPU 上结果应该一致，但速度可能差很多，因为内部实现不同。

三台机器：抽象机器、架构机器和实现机器

学习 CPU 执行时，最容易混乱的地方不是某个寄存器名字，而是把不同层次的“机器”混成一个概念。为了后面不反复犯错，本书把机器分成三层。

第一层是语言抽象机器。C++ 标准描述的是 C++ 程序的语义，它关心对象生命周期、类型、表达式求值、序列关系、未定义行为、实现定义行为和可观察副作用。C++ 抽象机器并不承诺每个变量都有一个真实内存位置，也不承诺每条语句都会生成一段对应汇编。只要最终可观察行为等价，编译器可以内联、删除、重排、常量传播、循环展开、向量化，甚至把整段计算折叠成一个常量。

第二层是 ISA 架构机器。x86-64 ISA 描述了软件能看到的寄存器、flags、地址空间、异常和指令语义。汇编学习主要在这一层工作。你读到 `add eax, esi`，就应该能说出它读取 `eax` 和 `esi`，写回 `eax`，并更新若干 flags。ISA 层不关心这条加法在真实 CPU 内部占用哪个端口，也不关心它是否先进入调度队列。

第三层是微架构实现机器。真实 CPU 会取指、解码、预测、重命名、调度、执行、访存、提交。它为了速度会把很多指令重叠执行，也会推测执行一些以后可能被丢弃的工作。微架构层解释的是

“为什么同样语义会有不同速度”，不是“这段程序是否合法”。例如两个 CPU 都正确执行 `mov eax, dword ptr [rdi]`，但一个可能因为更大的乱序窗口或更强预取器而在某个循环上更快。

层次	主要问题	典型证据
C++ 抽象机器	源码含义是否成立，优化器能假设什么	标准语义、编译警告、sanitizer、测试
ISA 架构机器	指令如何改变寄存器、内存、flags 和控制流	汇编、反汇编、ABI 文档、GDB 寄存器
微架构实现机器	指令如何被真实硬件重叠、等待、预测和提交	计时、PMU、厂商手册、微基准实验

这三层之间不是互相替代的关系，而是约束关系。C++ 编译器必须生成符合 C++ 语义的机器码；机器码必须符合 ISA；真实 CPU 必须让提交后的架构状态符合 ISA。性能分析要穿过三层，但结论必须说清楚自己站在哪一层。例如“这个函数没有独立符号”是二进制层观察；“这个循环可以向量化”是编译器优化层判断；“这个随机访问慢是因为 cache miss 多”是微架构假设，需要测量证据支持。

常见误区

不要用微架构经验去修改语言语义，也不要源码直觉否定二进制证据。比如“机器整数会回绕”不能让 C++ 有符号整数溢出变得有定义；“源码里有函数调用”也不能证明优化后二进制里一定有 `call`。

指令语义的阅读模板

读汇编时，不要只把每一行翻译成一句中文。那样很容易停留在表面。每条关键指令都应该按一个固定模板阅读：

- 1. 这条指令的显式操作数是什么。
- 2. 它是否还有隐含操作数。
- 3. 它读取哪些架构状态。
- 4. 它写入哪些架构状态。
- 5. 它是否读取或写入内存。
- 6. 它如何改变 flags。
- 7. 它如何改变 `rip`。
- 8. 它可能触发哪些异常。

以 `add eax, esi` 为例，显式操作数是 `eax` 和 `esi`。它读取旧的 `eax`、读取 `esi`、写回新的 `eax`，并更新若干 flags。它通常不访问内存，顺序执行时让 `rip` 前进到下一条指令。若只看整数加法语义，它很简单；但若后面紧跟条件跳转，flags 就成了这条指令和后续控制流之间的隐含数据通道。

以 `ret` 为例，表面上没有显式操作数，但隐含读取 `rsp` 和栈顶内存，隐含写 `rsp` 和 `rip`。它不只是“结束函数”，而是一次通过内存恢复控制流的动作。栈顶保存的返回地址如果被破坏，`ret` 就会把 CPU 带到错误位置。

以内存操作数为例：

```
mov eax, dword ptr [rdi + rsi*4 + 8]
```

这条指令读取 `rdi` 和 `rsi` 来计算地址，从这个地址读取 4 字节，写入 `eax`，并因为 32 位写入规则清零 `rax` 高 32 位。它通常不改 flags。它还可能因为地址不可访问、页面权限错误或地址未规范化而触发异常。

这个模板看起来比“翻译指令名”慢，但它能避免初学者最常见的错误：漏掉隐含状态。很多汇编 bug、ABI bug 和 benchmark 误判，本质都是漏看了 flags、栈指针、返回地址、内存别名、对齐、异常或调用约定。

心智模型

一条指令的完整含义不是助记符，而是状态转移。你真正要读的是：旧状态中的哪些部分被读取，经过什么语义，变成新状态中的哪些部分。

x86-64 程序员模型

学习汇编时，先不要把 CPU 想成无限复杂的乱序机器。先从程序员模型开始。所谓程序员模型，就是写汇编、读反汇编、理解 ABI 时必须看到的那部分 CPU 状态。

通用寄存器

x86-64 有 16 个常用通用整数寄存器：

64 位	32 位低半部分	常见用途
<code>rax</code>	<code>eax</code>	返回值、通用计算
<code>rbx</code>	<code>ebx</code>	callee-saved 通用寄存器
<code>rcx</code>	<code>ecx</code>	第 4 个整数参数、计数、通用计算
<code>rdx</code>	<code>edx</code>	第 3 个整数参数、乘除辅助、通用计算
<code>rsi</code>	<code>esi</code>	第 2 个整数/指针参数
<code>rdi</code>	<code>edi</code>	第 1 个整数/指针参数
<code>rbp</code>	<code>ebp</code>	栈帧指针或通用 callee-saved 寄存器
<code>rsp</code>	<code>esp</code>	栈指针
<code>r8</code>	<code>r8d</code>	第 5 个整数/指针参数
<code>r9</code>	<code>r9d</code>	第 6 个整数/指针参数
<code>r10</code>	<code>r10d</code>	caller-saved 临时寄存器
<code>r11</code>	<code>r11d</code>	caller-saved 临时寄存器
<code>r12</code>	<code>r12d</code>	callee-saved 通用寄存器
<code>r13</code>	<code>r13d</code>	callee-saved 通用寄存器
<code>r14</code>	<code>r14d</code>	callee-saved 通用寄存器
<code>r15</code>	<code>r15d</code>	callee-saved 通用寄存器

寄存器不是变量。寄存器是 CPU 内部的高速状态存储单元。编译器会把源代码里的变量、临时值、地址、中间计算结果分配到寄存器或栈位置。一个变量可能根本没有固定的寄存器；一个寄存器在不同指令之间也可能承载不同含义。

32 位写入会清零高 32 位

x86-64 有一个非常重要的规则：写入 32 位寄存器通常会把对应 64 位寄存器的高 32 位清零。例如：

```
mov eax, edi
```

这条指令写 `eax`，同时会让 `rax` 的高 32 位变成 0。这个规则经常帮助 CPU 避免旧高位造成的依赖，也让 32 位整数计算自然产生零扩展结果。

常见误区

不要把 `eax` 理解成完全独立于 `rax` 的寄存器。它是 `rax` 的低 32 位视图。类似地，`ax`、`al` 也是同一个物理架构寄存器的更小视图。部分寄存器写入在某些老架构上还会带来额外依赖问题，后面讲性能时会再次出现。

RIP：CPU 下一步从哪里取指

`rip` 是 instruction pointer。它保存下一条要执行的指令地址。普通顺序执行时，CPU 取出当前指令后，会把 `rip` 更新到下一条指令地址。

x86 指令长度不是固定的。刚才的例子中：

```
89 f8    mov eax, edi
01 f0    add eax, esi
```

```
c3      ret
```

`mov eax, edi` 长 2 字节, `add eax, esi` 长 2 字节, `ret` 长 1 字节。所以如果第一条指令地址是 `0x1000`, 顺序取指时大致会这样:

执行前 rip	指令字节	顺序下一地址
<code>0x1000</code>	<code>89f8</code>	<code>0x1002</code>
<code>0x1002</code>	<code>01f0</code>	<code>0x1004</code>
<code>0x1004</code>	<code>c3</code>	由栈中的返回地址决定

跳转、调用、返回和异常都会改变 `rip`。控制流的本质就是改变 CPU 接下来从哪里取指。

RFLAGS: 条件判断的隐含状态

很多整数指令会设置 RFLAGS。例如 `add` 不只写目标寄存器, 还会更新 `zero flag`、`sign flag`、`carry flag`、`overflow flag` 等状态。条件跳转指令读取这些 `flags` 来决定是否跳转。

例如:

```
cmp edi, esi
jg greater
```

`cmp edi, esi` 的语义类似计算 `edi-esi` 并更新 `flags`, 但不保存结果。`jg` 读取 `flags`, 判断有符号意义下 `edi` 是否大于 `esi`。

这解释了为什么 `lea eax, [rdi+rsi]` 有时会替代 `add: lea` 做地址形式的整数计算, 但通常不改 `flags`。如果后续代码还需要旧 `flags`, 使用 `lea` 可以避免破坏它们; 如果后续不需要 `flags`, 编译器也可能因为编码或调度原因选择其中一种。

flags 是一条隐含的数据流

很多初学者读汇编时只盯着通用寄存器, 于是会漏掉 `flags`。这个错误很隐蔽, 因为 `flags` 不像 `rax` 那样经常作为显式操作数出现, 却会直接决定后续控制流或条件结果。

看下面两段代码:

```
cmp edi, esi
jg greater
```

```
cmp edi, esi
setg al
```

第一段中, `cmp` 写 `flags`, `jg` 读 `flags`。第二段中, `cmp` 仍然写 `flags`, 但读取者变成 `setg`, 结果被写入 `al`。从数据流角度看, `flags` 就像一组没有名字的临时寄存器。它们承接比较、测试、加减运算产生的信息, 再被条件跳转、条件移动、条件置位等指令消费。

`flags` 数据流有三个工程后果。

第一, 它会约束指令重排。如果一条条件跳转必须读取某次 `cmp` 产生的 `flags`, 那么在这两条指令之间插入另一条会改 `flags` 的指令, 就会改变程序语义。编译器和手写汇编都必须维护这种隐含依赖。

第二, 它会影响指令选择。若后续还要使用旧 `flags`, 编译器可能选择不改 `flags` 的 `lea` 完成整数加法或地址形式计算; 若后续马上要根据加法结果判断溢出或零值, 编译器反而会利用 `add` 或 `sub` 顺手产生 `flags`。

第三, 它会影响读反汇编的切分方式。你不能把 `cmp` 看成一条孤立的“比较语句”, 必须向后找消费 `flags` 的指令; 也不能把 `jcc` 看成孤立跳转, 必须向前找最近的、在数据流上有效的 `flags` 生产者。

指令类别	对 flags 的关系	阅读重点
cmp、test	写 flags，不保存普通结果	后面谁读取这些 flags
add、sub	写目标，也写 flags	flags 是否被后续使用
lea、普通 mov	通常不改 flags	是否用于保留旧 flags
jcc、cmovcc、setcc	读取 flags	判断条件来自哪一次比较或运算

心智模型

flags 不是“附带信息”，而是控制流和条件数据流的隐含寄存器。读汇编时要把 flags 生产者和消费者连成一条边。

机器码字节和汇编文本

汇编文本是给人看的。机器码字节才是 CPU 取指单元读取的内容。汇编器负责把：

```
mov eax, edi

变成：

89 f8
```

这两个字节不是随意来的。x86 指令编码中包含 opcode、寄存器编号、寻址模式、立即数、位移、前缀等部分。完整 x86 编码非常复杂，本章只建立直觉。

以 `mov eax, edi` 为例：

- 89 是一类 `mov` 指令的 opcode。
- f8 是 ModRM 字节，描述源寄存器和目标寄存器。

以 `ret` 为例：

- c3 就是一字节 `near return` 指令。

x86-64 是变长指令集。一条指令可以很短，也可以很长。变长编码带来代码密度优势，也让前端解码变复杂。现代 x86 CPU 为了高速执行，必须在前端投入大量硬件：取指、边界识别、解码、缓存已经解码过的 uops 等。后面讲前端瓶颈时，这一点会非常重要。

心智模型

一条汇编指令不是一个字符串，而是一段字节的解释方式。反汇编器把字节翻译回人类可读文本；汇编器把文本翻译成字节。性能分析时，二者都要看，但最终以实际二进制中的字节和指令为准。

指令生命周期：从字节到状态变化

现在把一条指令的生命过程串起来。假设内存中某个地址开始存着下面的字节：

```
01 f0

在某种解码上下文中，它可以被反汇编为：

add eax, esi
```

对 CPU 来说，这不是先有字符串再执行。真实过程更接近下面这条链：


```

instruction bytes
|
v
instruction boundary and opcode decoding
|
v
architectural semantics
|
v
internal uops and scheduling
|
v
execution result
|
v
architectural commit

```

这条链里每一层都可能给分析带来不同问题。

字节层关心的是可执行文件或进程内存里到底是什么。若反汇编位置错了，或者从中间字节开始解释，就会得到完全不同的指令文本。x86 变长编码让这种问题尤其明显。调试跳转到错误地址、函数指针损坏、返回地址破坏时，第一步不是猜 C++ 源码，而是确认 `rip` 附近的字节是否真的是有效指令边界。

语义层关心的是 ISA 规定的状态变化。看到 `add eax, esi`，你要知道它读 `eax` 和 `esi`，写 `eax`，写 `flags`，让顺序 `rip` 前进。这个层次回答“程序结果应该是什么”。

内部执行层关心的是具体 CPU 如何实现这条语义。它可能把指令变成一个或多个 uops，分配到某些执行单元，等待源操作数 ready，再写入物理寄存器。这个层次回答“为什么快或慢”。

提交层关心的是结果什么时候正式成为架构可见状态。对普通整数指令来说，这个差异不容易被源代码直接看到；对异常、调试、分支预测错误和乱序执行来说，它非常关键。CPU 可以提前执行，但不能让错误路径或异常之后的结果正式污染架构状态。

核心思想

同一条指令同时有字节身份、ISA 语义身份和微架构执行身份。正确性分析以 ISA 语义为准，性能分析还必须追踪微架构执行条件。

最朴素的 Fetch-Decode-Execute 模型

先构造一个极简 CPU 模型。它每次只执行一条指令，每条指令完整执行完才开始下一条：

```

while running:
    bytes = fetch memory[RIP]
    inst = decode(bytes)
    next_RIP = RIP + length(inst)
    operands = read_registers_or_memory(inst)
    result = execute(inst, operands)
    write_registers_or_memory(result)
    RIP = choose_next_RIP(inst, next_RIP)

```

这个模型非常简化，但它的每个词都值得解释。

Fetch：取指

取指就是根据 `rip` 到内存层级中取机器码字节。现代 CPU 通常不是每次直接去主内存取字节，而是从 L1 instruction cache、uop cache 或其他前端结构中拿。第一次学习时可以先想象：

`rip` → instruction memory → instruction bytes

如果指令字节不在前端 cache 里，取指就可能变慢。代码太大、分支太乱、热循环跨越不友好的边界，都可能影响前端供给。

Decode: 解码

解码器把机器码字节解释成指令语义。它要知道：

- 这是什么指令。
- 指令有多长。
- 操作数在哪里。
- 是否有立即数或内存位移。
- 这条指令应该读写哪些架构状态。

现代 x86 CPU 通常会把复杂 x86 指令解码成更接近内部执行格式的 micro-ops，简称 uops。不要把 uops 当成另一套必须立刻精通的汇编语言；现在只需要知道：软件看到 x86 指令，硬件内部可能执行一个或多个更小的操作。

Read operands: 读取操作数

指令需要输入。输入可能来自：

- 寄存器，例如 `edi`、`esi`。
- 立即数，例如 `addeax, 5` 里的 5。
- 内存，例如 `moveax, dwordptr[rdi]`。
- 隐含状态，例如 `ret` 隐含读取 `rsp` 指向的栈内存。

读寄存器通常很快。读内存则复杂得多，因为地址可能还要计算，虚拟地址可能要经过 TLB 翻译，数据可能在 L1/L2/L3 cache 或主内存。

Execute: 执行

执行是根据指令语义完成计算或动作：

- 整数加法由整数执行单元完成。
- 地址计算由地址生成单元完成。
- `load` 从内存层级读取数据。
- `store` 把数据写向内存系统。
- `branch` 计算下一条指令地址。
- `floating-point` 和 `SIMD` 指令使用向量/浮点执行单元。

Write back / Commit: 写回和提交

计算结果最终要改变状态。简单 CPU 可以执行完就写架构寄存器。现代乱序 CPU 会先把结果写到内部物理寄存器或缓冲结构中，再在 `retire` 阶段按程序顺序提交。这样可以保证异常和中断时仍然呈现精确的架构状态。

本章先记住一句话：

核心思想

CPU 可以在内部提前做很多事，但最终必须让软件看到像是按照程序顺序执行每条指令一样的结果。这是理解乱序执行和精确异常的基础。

取指不只是读取下一个字节

在最简模型里，取指似乎只是按照 `rip` 从内存中拿字节。但真实 x86-64 前端要解决的问题更多。

首先，x86 指令是变长的。CPU 不能只按固定 4 字节或 8 字节切分指令。它必须识别指令边界：前缀在哪里，opcode 在哪里，ModRM 和 SIB 字节是否存在，位移和立即数有多长。这就是为什么 x86 前端复杂。反过来，RISC 风格 ISA 常常使用更规则的指令编码，解码难度较低，但代码密度和历史兼容性取舍不同。本书以 x86-64 为主，是因为 Linux/C++ 性能工程和桌面服务器环境中它非常常见。

其次，取指看到的是虚拟地址。用户程序里的 `rip` 是虚拟地址，指令字节来自进程地址空间中的可执行映射。CPU 前端会依赖 instruction TLB 和 instruction cache。如果代码页没有执行权限，或者地址没有映射，取指本身就on能触发异常。换句话说，异常不只来自数据 load/store；执行一段不存在或不可执行的内存同样会失败。

再次，现代 CPU 不会等每条分支真正执行完才继续取指。前端会根据分支预测器猜测接下来取哪里。预测正确时，取指和解码持续供给后端；预测错误时，错误路径上的工作会被丢弃，前端重新从正确地址取指。对程序可见的结果仍然像顺序执行，但性能已经被预测质量影响。

最后，热代码可能进入专门的前端缓存结构。某些 x86 CPU 会缓存已经解码过的 uops，让循环再次执行时减少重复解码成本。第一册不要求你背每代 CPU 的前端结构，但要知道：代码大小、分支布局、对齐和循环形态会影响前端供给。后面如果 benchmark 显示一段小循环很快，不能只看后端算术指令，也要问前端是否能稳定喂饱后端。

常见误区

取指阶段出问题时，症状不一定写着“取指失败”。非法指令、执行不可执行页面、跳到错误地址、函数指针损坏、返回地址损坏，都可能表现为崩溃。调试这类问题时，`rip`、当前反汇编、内存权限和调用栈是第一组证据。

解码后的指令仍然要等操作数

解码告诉 CPU “这条指令是什么”，但还没有保证它能立刻执行。执行单元需要操作数 ready。

寄存器操作数看起来最直接，但在乱序 CPU 内部，架构寄存器名会先经过重命名。后端真正等待的是某个物理寄存器或某个尚未完成的 uop 结果。若一条指令依赖前一条 load 的结果，即使它已经解码，也必须等 load 返回数据。

内存操作数更复杂。一条带内存源操作数的整数指令，经常可以拆成地址生成、load 和计算几个内部动作。地址生成要等 base、index、scale、displacement 所需的寄存器值 ready；TLB 查询要确认虚拟地址到物理地址的翻译；cache 访问要看数据所在层级；最后计算单元才能使用取回的数据。源码里一个简单的 `a[i]`，在硬件里是一串依赖链。

store 也不是一句“写内存”就结束。store 通常包含地址和值两个输入：地址可能来自指针计算，值可能来自前面计算。CPU 会把 store 放入 store buffer，并在满足顺序、权限和一致性要求后写入 cache 层级。对单线程程序，store 似乎按程序顺序生效；对多线程程序，还要受内存模型、cache coherence 和 fence 约束。第一册只要求形成边界意识：普通整数寄存器计算、load 和 store 的硬件路径不同，不能把所有指令都想成同样成本。

深入理解

很多性能初学者看见一条指令里同时有内存操作数和算术操作，就把它当成“一条指令所以成本低”。这不可靠。ISA 文本是一条指令，微架构内部可能分解成多个 uops；真正成本取决于依赖、cache 命中、端口资源、load/store 队列和前后指令。读汇编时要把“指令条数”和“执行成本”分开。

提交、精确异常和程序顺序

现代 CPU 可以乱序执行，但异常和调试要求程序有清晰边界。假设下面的指令序列：

```
mov eax, dword ptr [rdi] ; 可能触发 page fault
add r8d, r9d            ; 与 eax 无关
add r10d, r11d          ; 与 eax 无关
```

如果第一条 load 很慢，后两条独立加法可能已经在内部执行完成。但如果第一条最终触发 page fault，操作系统和调试器应该看到异常发生在第一条指令上，而不是看到后两条已经“正式改变了程序状态”。这就是精确异常的要求：当异常报告给软件时，架构状态要像异常指令之前的指令都已经完成、异常指令之后的指令都还没有完成。

为了做到这一点，乱序 CPU 通常把执行完成和提交分开。执行完成表示某个内部操作算出了结果；提交表示这个结果按程序顺序成为架构可见状态。重排序缓冲区一类结构会记录尚未提交的操作。只有当更早的指令都没有未处理异常，当前指令的结果才能按顺序提交。

这对学习有两个重要影响。

第一，GDB、异常地址和反汇编位置通常站在架构状态层，而不是微架构内部时间线。你看到程序停在某条指令，并不表示 CPU 物理上从未执行过后面的任何推测工作；它表示后面的工作没有成为架构可见结果。

第二，性能分析不能把“架构顺序”和“执行时间线”混为一谈。某条 load 在文本上很早，但它的结果可能很晚才 ready；某条独立加法在文本上较晚，却可能提前执行。后面讲依赖图、乱序窗口和 benchmark 解释时，会反复依赖这一区分。

核心思想

架构状态给软件一个顺序、精确、可调试的世界；微架构为了性能在内部制造并行、推测和乱序。两者通过提交机制连接起来。

逐条执行 add_i32

现在手工执行这个版本：

```
mov eax, edi
add eax, esi
ret
```

假设函数入口处：

- `edi` = 10。
- `esi` = 32。
- `rsp` 指向栈顶。
- 栈顶 8 字节保存调用者的返回地址。

第一条：

```
mov eax, edi
```

语义是把 `edi` 的低 32 位复制到 `eax`。执行后：

- `eax` = 10。
- `rax` 高 32 位清零。
- 通常不改变 `flags`。
- `rip` 指向下一条指令。

第二条：

```
add eax, esi
```

语义是：

$$\text{eax} \leftarrow \text{eax} + \text{esi}$$

执行后：

- `eax` = 42。
- `RFLAGS` 根据结果更新。
- `rax` 高 32 位仍然是 0。
- `rip` 指向下一条指令。

第三条：

```
ret
```

近似语义是：

```
target = memory[rsp]
rsp = rsp + 8
rip = target
```


也就是说，`ret` 从栈顶弹出返回地址，更新 `rsp`，然后把 `rip` 改成返回地址。函数返回值已经在 `eax` 中，调用者按照 ABI 从那里读取返回值。

调用和返回：栈为什么参与控制流

函数调用不是魔法。假设调用者执行：

```
call add_i32
```

`call` 的关键语义可以近似理解为：

```
rsp = rsp - 8
memory[rsp] = address_of_next_instruction
rip = address_of_add_i32
```

所以 `call` 做了两件事：

- 保存返回地址。
- 跳转到被调用函数入口。

`ret` 则做反向操作：

- 从栈顶取回返回地址。
- 跳回调用者。

这解释了为什么破坏 `rsp` 会让程序崩溃，也解释了为什么栈溢出、返回地址覆盖和控制流劫持是安全问题。对性能学习来说，理解 `call/ret` 也很重要，因为函数调用涉及 ABI、寄存器保存、栈对齐、返回地址预测和 `inline` 决策。

常见误区

`call` 和 `ret` 改变的是控制流，不是普通数据流。你必须同时追踪 `rip`、`rsp` 和栈内存，才能正确理解函数调用。

内存访问：一条 `load` 背后的路径

再看一个稍微复杂的函数：

```
extern "C" int add_load(const int* p, int y)
{
    return p[0] + y;
}
```

可能出现这样的汇编：

```
mov eax, dword ptr [rdi]
add eax, esi
ret
```

这里 `rdi` 保存指针 `p`，`dword ptr [rdi]` 表示从地址 `rdi` 处读取 4 字节整数。从 ISA 语义看，这条 `load` 很简单：

$$\text{eax} \leftarrow \text{memory}[\text{rdi} \dots \text{rdi}+3]$$

从微架构看，它可能经过很多步骤：

1. 地址生成单元计算有效地址。
2. 如果使用虚拟内存，CPU 用 TLB 查询虚拟地址到物理地址的翻译。
3. L1 data cache 根据地址查找 cache line。
4. 如果 L1 命中，数据很快返回。
5. 如果 L1 未命中，继续查 L2、L3，甚至访问主内存。
6. 取回 4 字节并送到目标寄存器。

这就是性能工程中经常说的：寄存器计算和内存访问不是一个数量级的问题。一个整数加法可能吞吐很高，而一次真正去主内存的 `load` 会慢很多。

load 的三个问题：地址、权限和数据

初学者常把 load 理解成“从内存拿一个值”，但 CPU 真正要回答三个问题。

第一个问题是地址是多少。有效地址由寄存器、scale、位移等组成。地址计算本身依赖前面指令产生的寄存器值。若指针来自上一条 load，当前 load 就形成 pointer chasing；这种链式访问很难并行，因为下一个地址必须等上一个数据回来才知道。

第二个问题是这个地址能不能访问。用户态程序使用虚拟地址。CPU 需要确认地址属于当前进程映射，页表权限允许读取，访问宽度没有跨到非法区域，地址形式符合架构要求。权限检查失败会触发异常。很多段错误不是“内存坏了”，而是当前指令访问的虚拟地址没有合法映射或权限不允许。

第三个问题是数据在哪里。即使地址合法，数据也可能在 L1、L2、L3、主内存，或者因为 page fault 需要操作系统把页面调入。性能上最常见的差异来自这一层。同一条 `mov eax, dword ptr [rdi]`，当 `rdi` 指向热数据和冷数据时，指令文本完全相同，执行时间却可能相差巨大。

因此，分析内存指令不能只问“它访问了什么类型”，还要问：

- 地址是否能提前计算出来。
- 访问是否连续。
- 是否跨 cache line 或页面。
- 工作集是否适合 cache 容量。
- 多次访问之间是否有可利用的局部性。
- 是否存在指针追逐导致的串行依赖。

store 为什么更难观察

load 的结果通常会进入寄存器，很容易在 GDB 里看到。store 的效果则写向内存，而且可能先进入 store buffer。对单线程调试来说，你通常可以把 store 看成按程序顺序更新内存；但做性能分析时要知道，store 也有自己的资源和延迟。

例如：

```
mov dword ptr [rdi], eax
```

这条 store 至少需要两个输入：目标地址和要写入的值。地址依赖 `rdi`，数据依赖 `eax`。CPU 还要处理写权限、cache line 所有权、store buffer 空间以及与前后 load/store 的顺序关系。如果 store 很密集，瓶颈可能不是整数计算，而是 store 带宽、写分配、cache line 状态转换或内存系统。

store 还有一个常见误区：写了内存不代表数据立刻到达主内存。现代缓存层级会让写入先在 cache 中可见，之后再逐级写回。对同一线程的后续读取，CPU 必须维持程序语义；对其他核心看到写入的时机，则涉及 cache coherence 和内存模型。第一册暂不展开并发内存模型，但你要避免把“内存”想成一个单一、同步、瞬时更新的大数组。

常见误区

benchmark 中如果只写输出 buffer 而从不使用它，优化器可能删除整个写入循环；如果用过重的 checksum 又会污染测量。正确做法是明确可观察使用、正确性校验和计时区域边界。这个问题会在第三部分系统处理。

store-to-load forwarding：刚写完又读取

程序里经常出现先写一个位置、马上又从这个位置读取的情况：

```
mov dword ptr [rsp + 12], eax
mov ecx, dword ptr [rsp + 12]
```

从 ISA 语义看，第二条 load 应该读到第一条 store 写入的值。真实 CPU 不一定要等 store 最终写入 cache 后再读。它可以从 store buffer 里把数据转发给后续 load，这通常叫 store-to-load forwarding。

这个机制说明了两件事。

第一，内存系统不是只有 cache 和主内存。store buffer 是执行路径中的真实结构，它让 store 可以先被记录下来，后续再按规则写入缓存层级。对同一核心的后续 load，如果地址和大小匹配，CPU

可以直接从 store buffer 得到数据。

第二，store/load 的宽度、对齐和地址关系会影响成本。如果先写 4 字节，后面读 8 字节；或者访问跨越边界；或者地址计算太晚，CPU 无法及时判断是否同一位置，就可能产生额外等待。第一册不要求你背具体处理器的所有 forwarding 规则，但要知道“刚写又读”不是普通 load，也不是免费的抽象变量访问。

在 C++ 层，编译器常常会把局部变量留在寄存器里，避免这种内存往返。只有在优化级别低、寄存器压力高、地址被取出、ABI 需要、调试需要或对象生命周期需要时，才更容易看到栈上 store/load。用 -O0 观察栈变量有助于学习，但不能把它当成优化后机器执行的常态。

TLB：地址翻译也可能成为成本

虚拟内存让每个进程像拥有独立地址空间。程序里的指针值是虚拟地址，cache 和内存硬件最终要定位物理地址。这个翻译过程依赖页表；为了不每次访问都走完整页表，CPU 使用 TLB 缓存最近的地址翻译。

因此，一次 load 的关键路径可以粗略想成：

```

base/index registers ready
    |
    v
effective virtual address
    |
    v
TLB translation
    |
    v
cache lookup
    |
    v
data returned to dependent uop
  
```

如果访问集中在少量页面，TLB 命中率通常较好；如果工作集跨越大量页面，随机访问又缺少局部性，TLB miss 可能和 cache miss 一起拖慢程序。很多“随机访问慢”的真实原因不是单一的 cache miss，而是 cache line 利用率低、硬件预取困难、TLB 压力大、内存并行度不足共同作用。

这也解释了为什么后面做实验必须记录数据规模和访问步长。一个数组是否超过 L1、L2、L3，是否跨很多页面，是否连续访问，都会改变同一段汇编的表现。没有规模信息的性能结论通常不可复现。

深入理解

TLB 不是 C++ 语言概念，也不是 ISA 普通算术语义的一部分；它属于虚拟内存和微架构交界处。它不改变合法程序的计算结果，但会强烈影响访存延迟和吞吐。学习底层性能时，很多现象都来自这种“语义无关、性能相关”的结构。

地址计算：数组访问并不神秘

考虑：

```

extern "C" int get_i(const int* a, long i)
{
    return a[i];
}
  
```

核心地址计算是：

$$\text{address} = a + i * \text{sizeof}(\text{int})$$

x86-64 寻址模式能直接表达常见的 $\text{base} + \text{index} * \text{scale} + \text{displacement}$ ：

```
mov eax, dword ptr [rdi + rsi*4]
ret
```

这里：

- `rdi` 是 base，也就是数组起始地址。
- `rsi` 是 index，也就是下标。
- `4` 是 scale，因为 `int` 通常 4 字节。
- 没有额外 displacement。

这条指令同时包含地址计算和内存读取。现代 CPU 内部通常会把地址生成、TLB 查询、cache 访问等步骤分到 load pipeline 中。后面学习 cache、TLB 和 prefetch 时，这个例子会继续使用。

内存顺序的第一印象

单线程程序给人的感觉是：第一条语句先执行，第二条语句后执行，写入以后马上读取就能看到。这个直觉在学习基础汇编时有用，但不完整。

编译器可以在不改变单线程可观察行为的前提下重排代码。CPU 也可以在内部乱序执行 load、store 和计算。它们之所以没有把普通程序搞乱，是因为语言规则、ISA 规则和硬件顺序规则共同限制了哪些重排可以被观察到。

在 x86-64 上，常规内存顺序相对强，但这并不意味着“所有东西都绝对按文本顺序发生”。store 可能先进入 store buffer；后续 load 可能从 store buffer 转发；独立 load 可以并行发起；推测执行可能提前读取未来路径上的数据。对单线程结果来说，这些内部动作要伪装成合法顺序；对性能和多线程来说，它们会变得重要。

第一册当前只建立三个边界：

1. 普通单线程汇编阅读可以先按 ISA 顺序理解可见结果。
2. 性能分析要允许 CPU 内部重叠和乱序执行独立内存操作。
3. 多线程共享内存必须回到 C++ 内存模型、原子操作、fence 和平台内存顺序，不能只靠“我觉得这条先写那条后读”。

例如一个线程写普通变量，另一个线程不使用同步就读这个变量，在 C++ 层可能已经是数据竞争；这时讨论某条 x86 指令是否“看起来会先执行”没有意义。语言层未定义或无保证的程序，不能靠硬件直觉补救。

心智模型

先用顺序模型理解正确性，再用乱序模型解释性能；遇到多线程共享数据，先回到语言内存模型，不要直接跳到汇编猜测。

控制流：顺序、跳转和分支

没有跳转时，`rip` 顺序前进。有跳转时，下一条指令地址由跳转目标决定。

例如：

```
extern "C" int abs_i32(int x)
{
    if (x < 0) {
        return -x;
    }
    return x;
}
```

一个未必最优但容易理解的汇编形态是：

```
mov eax, edi
test edi, edi
jge done
neg eax
done:
ret
```


`test edi, edi` 会计算 `edi` 和自身的按位与，只更新 `flags`，不保存结果。它常用于判断是否为 0 或正负。`jge done` 根据 `flags` 决定是否跳到 `done`。

控制流引入了性能问题：CPU 前端必须知道接下来取哪里。如果等分支真正执行完再取下一条，流水线会经常停住。现代 CPU 因此引入 `branch predictor`，提前猜测分支方向和目标地址。

条件分支的两段式语义

条件分支通常可以拆成两个动作：先产生条件，再消费条件。以刚才的绝对值函数为例：

```
test edi, edi
jge done
```

`test` 根据 `edi` 的位模式设置 `flags`；`jge` 根据 `flags` 决定 `rip` 是落到下一条，还是跳到 `done`。所以条件分支不是“if 语句直接变成跳转”，而是一组指令协作完成。

编译器也可以选择不同形态。某些情况下，它可能生成条件移动：

```
mov eax, edi
neg eax
cmovge eax, edi
ret
```

这个形态仍然依赖 `flags`，但不使用条件跳转改变控制流。它可能减少分支预测错误，却会让两条路径的一部分计算都发生，并引入条件移动的数据依赖。是否更快取决于输入分布、分支可预测性、指令成本和关键路径长度。

因此，不要把“无分支”当成绝对优化。高可预测分支可能很便宜；为了消除分支而增加长依赖链，反而可能变慢。第一册希望你先掌握判断框架：分支是否高频，方向是否稳定，条件值何时 `ready`，错误预测代价是否超过替代方案成本。

循环分支和退出条件

循环通常至少有一个控制分支。求和循环里的：

```
cmp rcx, rsi
jl loop
```

会在大多数迭代中跳回循环头，最后一次不跳。这样的分支往往容易预测，因为模式稳定。相比之下，如果循环内部根据数据内容分支，例如“元素大于阈值才累加”，而数据分布接近随机，预测器就更难稳定判断。

循环还有一个常见细节：退出条件本身可能在依赖链上。若下标递增、地址计算、比较和跳转形成固定顺序，循环控制开销就会占用前端和后端资源。编译器可能通过展开循环减少分支频率，也可能通过向量化一次处理多个元素。第一册这里只建立概念，后面 `benchmark` 章节再要求用实验验证。

为什么需要流水线

假设一条指令要经历五步：

1. 取指。
2. 解码。
3. 读操作数。
4. 执行。
5. 写回。

如果一条指令完成所有步骤后才开始下一条，硬件资源大部分时间都在空闲。流水线的想法是把不同指令的不同阶段重叠起来：

周期	阶段 1	阶段 2	阶段 3	阶段 4	阶段 5
1	I1 取指				
2	I2 取指	I1 解码			
3	I3 取指	I2 解码	I1 读数		
4	I4 取指	I3 解码	I2 读数	I1 执行	
5	I5 取指	I4 解码	I3 读数	I2 执行	I1 写回

流水线提高的是吞吐，不一定降低单条指令延迟。这里要提前区分两个性能概念：

延迟（latency）是一个操作从开始到结果可用需要多久。**吞吐**（throughput）是单位时间内可以完成多少操作。

一条加法可能延迟 1 个周期，但如果 CPU 每周期能发射多条独立加法，那么吞吐可以高于每周期 1 条。后面讲 latency、throughput、ILP 时会严格量化这些概念。

依赖：为什么有些指令不能并行

考虑：

```
add eax, esi
add eax, edx
add eax, ecx
```

每条指令都读写 `eax`。第二条必须等待第一条产生新的 `eax`，第三条必须等待第二条。这叫数据依赖。

再考虑：

```
add eax, esi
add r8d, r9d
add r10d, r11d
```

这三条指令互相独立。现代 CPU 更容易把它们重叠执行。高性能代码经常要制造足够多的独立工作，让 CPU 的多个执行单元都有活干。

核心思想

性能优化不是简单减少源代码行数，而是控制机器指令的依赖图、内存访问模式和前端供给。短代码不一定快，能让硬件持续高效工作的代码才快。

依赖的三种基本形态

为了把“能不能并行”说清楚，需要把依赖分成几类。第一册不要求你画完整的硬件调度图，但至少要在一段热点汇编里标出关键依赖边。

真依赖：后者需要前者的结果

真依赖也叫 read-after-write。后一条指令要读取前一条指令写出的值，所以必须等待。

```
imul eax, esi
add eax, edx
```

`add` 读取 `eax`，而这个 `eax` 正是前一条 `imul` 的结果。即使 CPU 有很多执行单元，`add` 也不能在乘法结果 ready 之前得到正确输入。循环累加、链式哈希、链表遍历、递归依赖都经常形成真依赖链。

真依赖决定了很多代码的最短时间。若每次迭代都必须等待上一轮结果，吞吐再高也不能把这一条链完全并行化。后面看到 reduction、前缀和、指针追逐时，都要先找真依赖。

反依赖和输出依赖：名字造成的约束

反依赖也叫 write-after-read。后一条指令要写某个寄存器或位置，而前一条更早的指令还需要读取旧值。输出依赖也叫 write-after-write。两条指令都写同一个名字，后写必须成为最终可见结果。

在架构寄存器数量有限的机器上，这两类依赖看起来会限制并行：

```
mov ecx, eax
add eax, esi
```

第一条要读旧 `eax`，第二条写新 `eax`。如果第二条过早覆盖旧值，第一条就错了。再看：

```
mov eax, edi
mov eax, esi
```

两条都写 `eax`，最终可见值必须来自第二条。

现代乱序 CPU 用寄存器重命名处理很多这种由“名字复用”造成的假依赖。它会把同一个架构寄存器的不同生命期映射到不同物理寄存器。这样，很多表面上写同一个寄存器的独立计算可以并行推进。

内存依赖：地址必须先弄清楚

内存依赖比寄存器依赖难，因为 CPU 先要判断两个内存操作是否访问同一个地址。

```
mov dword ptr [rdi], eax
mov ecx, dword ptr [rsi]
```

如果 `rdi` 和 `rsi` 指向同一地址，第二条 load 应该看到前一条 store 的值；如果地址不同，它们可以更自由地重叠。问题在于地址本身可能还没计算出来，或者依赖前面的 load。CPU 的 load/store queue、store buffer、内存消歧机制都在处理这类问题。

对 C++ 程序员来说，这和别名分析直接相连。若编译器无法证明两个指针不重叠，它就必须保守地维持某些顺序；若程序通过类型、限定、局部性或接口设计让别名关系更清楚，优化器就可能生成更容易并行的机器码。第三章会从对象和指针角度继续讲这个问题。

依赖类型	典型形式	分析意义
真依赖	先写后读同一值	决定关键路径，不能随意消除
反依赖	先读旧值后写同名位置	可能由重命名缓解
输出依赖	多次写同名位置	最终可见顺序必须正确
内存依赖	load/store 地址可能重叠	需要地址、别名和顺序证据

心智模型

性能分析里最重要的图不是源码缩进图，而是依赖图。先找哪些结果必须等待，再讨论 CPU 能把多少独立工作重叠起来。

为什么需要乱序执行

顺序流水线遇到长延迟操作会卡住。例如：

```
mov eax, dword ptr [rdi] ; 可能 cache miss
add r8d, r9d             ; 与 eax 无关
add r10d, r11d           ; 与 eax 无关
add eax, esi             ; 依赖 load
```

如果第一条 load 很慢，而后两条加法与它无关，理想情况下 CPU 不应该傻等。乱序执行的目标就是：在不改变架构可见结果的前提下，先执行已经 ready 的指令。

现代乱序 CPU 大致会：

- 按程序顺序取指和解码。
- 通过 register renaming 消除一部分假依赖。
- 把 uops 放入调度结构。
- 哪些操作数 ready，哪些执行单元可用，就先执行哪些。
- 最后按程序顺序 retire，保证异常和结果可见性正确。

这解释了一个重要现象：你在汇编里看到的是程序顺序，但 CPU 内部的实际执行顺序可能不同。性能分析不能只看文本顺序，还要看依赖和资源。

乱序执行的边界：能提前执行什么

乱序执行的目标不是把程序顺序随意打乱，而是在保持架构语义的前提下寻找可提前完成的工作。判断一条操作能否提前，至少要看四个条件。

第一，源操作数是否 ready。若一条加法需要某个 load 的结果，而这个 load 还没有返回数据，加法就不能执行。若另一条加法只依赖已经 ready 的寄存器，它就可能先执行。

第二，执行资源是否可用。即使操作数 ready，如果对应执行单元、load 端口、store 端口、调度队列或缓冲结构满了，也要等待。性能瓶颈有时不是数据依赖，而是资源竞争。

第三，内存顺序是否允许。load 和 store 之间如果可能访问同一地址，CPU 必须保证结果符合 ISA 的内存语义。它可以预测、消歧和转发，但不能让后续 load 错过前面必须可见的 store。

第四，提交顺序是否满足。内部提前算出的结果要等到更早指令都安全之后，才能按程序顺序提交。若更早的指令触发异常，后面的结果即使已经算出来，也不能成为架构可见结果。

用这四个条件看下面的片段：

```
mov eax, dword ptr [rdi]
add r8d, r9d
add r10d, r11d
add eax, esi
```

第二、第三条加法不依赖第一条 load，可以在 load 等待内存时执行；第四条加法依赖 `eax`，必须等 load 返回；所有结果最终还要按程序顺序提交。这样看，乱序执行并不神秘，它只是围绕依赖、资源、内存顺序和提交顺序做调度。

常见误区

不要把乱序执行理解成源码语句可以任意互换。C++ 优化器的重排受语言语义约束，CPU 内部调度受 ISA 和微架构约束。两者层次不同，但都必须保留合法程序的可观察行为。

寄存器重命名：为什么架构寄存器不等于物理寄存器

ISA 只暴露 16 个通用寄存器，但现代 CPU 内部通常有更多物理寄存器。寄存器重命名会把架构寄存器名映射到物理寄存器。

看这个例子：

```
mov eax, edi
add eax, esi
mov eax, r8d
add eax, r9d
```

从文本看，四条指令都写 `eax`。但第二组计算并不需要第一组计算的结果。没有重命名时，对同一个架构寄存器的写会制造不必要的顺序限制。重命名可以让两个不同生命期的 `eax` 映射到不同物理寄存器，从而消除假依赖。

这对 C++ 优化有一个启发：不要用源代码变量名想当然地判断寄存器压力和依赖。编译器和 CPU 都会重命名、合并、消除和重排中间值。真正可靠的方式是看编译后的汇编，再结合微架构模型分析。

重命名不能消除真依赖

寄存器重命名很强，但它不是魔法。它能解决很多由于架构寄存器名字复用造成的假依赖，却不能消除真正的数据依赖。

```
add eax, esi
add eax, edx
add eax, ecx
```

这里每条加法都需要上一条产生的新 `eax`。即使用不同物理寄存器保存每个版本，第二条仍然要等第一条结果，第三条仍然要等第二条结果。重命名能把“哪个版本的 `eax`”说清楚，却不能让尚未计算出来的值提前存在。

再看独立计算：


```

mov eax, edi
add eax, esi
mov r8d, r9d
add r8d, r10d

```

两组计算使用不同结果，不存在真依赖。CPU 更容易把它们重叠执行。高性能循环常常通过多路累加、展开或把独立元素分批处理来增加这种独立工作。第一册只讲直觉，第二册再系统讨论如何利用它。

重命名还有一个重要副作用：它让“寄存器名少”不等于“CPU 内部只能保存这么多中间值”。ISA 暴露 16 个通用寄存器，但真实核心内部可能有更多物理寄存器和调度条目。尽管如此，物理寄存器、ROB、队列和端口仍然有限。代码太复杂、活跃值太多、load 太多或分支太乱，仍然会让窗口装不下足够有用工作。

核心思想

重命名消除名字造成的假依赖，不能消除值造成的真依赖。判断性能关键路径时，先问“后者是否真的需要前者的结果”，再问寄存器名字是否只是复用。

Cache：为什么内存访问不是一个速度

程序员经常说“访问内存”，但 CPU 看到的是层级结构：

层级	大致特点	学习重点
寄存器	最靠近执行单元，数量少	分配、依赖、寄存器压力
L1 data cache	很小但很快	cache line、局部性、load/store 吞吐
L2 cache	更大但更慢	工作集大小、miss 代价
L3 cache	多核心共享或分片	跨核心影响、容量
主内存	容量大但延迟高	bandwidth、NUMA、预取

CPU 不是按单个 `int` 从内存取数据，而是按 cache line 在 cache 和更低层级之间移动数据。常见 cache line 大小是 64 字节。顺序扫描数组通常比随机访问快，一个重要原因就是顺序访问能利用 cache line 和硬件预取。

本章只建立直觉：内存访问速度取决于数据在哪一级，以及访问模式能否被 cache 和 prefetcher 利用。第二册的内存层级专题会用实验系统展开。

分支预测：CPU 为什么要猜

流水线越深、前端越宽，分支的代价越高。如果 CPU 不猜，它遇到条件跳转就必须等条件计算完成，然后才知道下一条从哪里取。这会让前端经常空转。

branch predictor 试图预测：

- 条件分支会不会跳。
- 跳转目标在哪里。
- `ret` 会返回到哪里。

如果预测正确，流水线保持忙碌。如果预测错误，CPU 必须丢弃错误路径上的推测工作，回到正确路径。这会造成明显性能损失。

这解释了为什么数据相关、难预测的分支会慢，也解释了为什么有时编译器或手写优化会使用条件移动、mask、SIMD blend 等方式减少分支。不是所有分支都坏，坏的是高频、难预测、代价明显的分支。

推测执行：先做了不代表正式发生

分支预测带来一个必须讲清楚的概念：推测执行。CPU 为了不让流水线空等，会沿着预测路径提前取指、解码甚至执行一些操作。如果预测正确，这些工作节省了时间；如果预测错误，CPU 会丢弃错误路径上的结果，并从正确路径重新开始。

从架构状态看，被丢弃的推测工作不应该留下正式结果。也就是说，错误路径上的寄存器写入、内存写入、`rip` 变化都不能成为程序可见状态。否则程序就无法按照 ISA 推理。

但微架构副作用可能更微妙。即使错误路径的架构结果被丢弃，它也可能影响 cache、TLB、分支预测器等内部状态。现代安全漏洞研究中，Spectre 一类问题正是利用了这种“架构上没发生、微架构上留下痕迹”的差异。第一册不会深入安全攻击，但必须让你知道：软件可见语义和微架构痕迹不是完全同一件事。

对普通性能学习来说，推测执行主要带来三个影响。

第一，分支预测正确时，程序可以像没有等待一样顺畅执行。循环分支如果规律稳定，预测器通常能学会，代价较低。

第二，分支预测错误时，错误路径上的工作被清空，已经占用的前端和后端资源浪费掉。高频难预测分支会显著降低吞吐。

第三，减少分支不一定总是更快。把分支改成条件移动或掩码计算，可能增加无条件执行的计算量，也可能拉长依赖链。正确判断要看输入分布、预测率、指令依赖和测量结果。

常见误区

不要把“CPU 可能推测执行”理解成“程序语义不可靠”。普通程序仍然按语言、ABI 和 ISA 规则定义可见结果。推测执行主要影响性能和某些安全边界；它不是随意违反程序语义的借口。

指令成本不能只看助记符

初学者读汇编时容易做一种错误比较：一段代码 5 条指令，另一段代码 8 条指令，于是认为 5 条一定更快。这种比较只在非常粗略的层面有参考价值，不能作为性能结论。

指令成本至少由下面因素共同决定：

- 解码后有多少 uops。
- uops 需要哪些执行资源。
- 操作数是否 ready。
- 是否访问内存，以及命中哪个层级。
- 是否形成循环携带依赖。
- 是否影响 flags 并约束后续指令。
- 是否改变控制流并依赖分支预测。
- 是否有序列化、原子、锁前缀或系统调用等特殊语义。

比如 `xor eax, eax` 常被用来把 `eax` 清零。它短、快，还能被许多 CPU 识别为不依赖旧 `eax` 的 zero idiom。相比之下，一条看似简单的内存 load 如果 miss 到主内存，可能让后续依赖链等待很久。

再比如 `idiv` 是整数除法，助记符只是一条指令，但延迟远高于普通加法。又比如带 `lock` 前缀的原子操作要参与跨核心一致性，成本和普通内存读写不是一类。读汇编时必须知道哪些指令属于“普通廉价操作”，哪些可能是长延迟或强同步操作。

第一册不会给出每条指令在每个 CPU 上的延迟表。更重要的是方法：遇到热点汇编，先按数据依赖和内存访问建立模型，再用工具和文档验证关键指令成本。不要让助记符数量代替分析。

为什么同一段源码会生成不同汇编

读者很快会发现：同一个 C++ 函数，在不同编译器、优化级别、目标 CPU、调试选项下生成的汇编可能完全不同。这不是异常，而是正常现象。

造成差异的原因包括：

- 优化级别不同。`-O0` 优先可调试，`-O2` 和 `-O3` 优先性能。
- 目标 CPU 不同。`-march` 会影响可用指令、调度模型和向量化策略。
- ABI 和平台不同。Linux System V、Windows x64、macOS 在调用约定和符号细节上有差异。
- 上下文不同。函数是否可内联、参数是否常量、指针别名是否可证明，都会改变优化。
- 调试、sanitizer、profiling、LTO 等选项会插入或改变代码。

因此，本书要求所有汇编观察都配套记录编译命令、编译器版本、目标平台和优化级别。没有这些信息的“我看到汇编是这样”不可复现，也不可审计。

这也解释了为什么实验常常同时看 `-O0` 和 `-O3`。`-O0` 帮你建立源代码、栈帧和寄存器的初始对应关系；`-O3` 更接近性能路径。两者都重要，但不能混用结论。用 `-O0` 解释真实性能，是学习底层时最常见的错误之一。

从 C++ 到 CPU 行为的阅读方法

以后看到一段 C++ hot loop，不要只停留在源代码层。建议按下面步骤阅读：

1. 写出这段代码的输入、输出和允许的 C++ 语义。
2. 编译成汇编，分别看 `-O0` 和 `-O3`，但以优化版本为性能分析对象。
3. 标出每条关键指令读写的寄存器和内存。
4. 找出循环携带依赖，例如 reduction 中的累加变量。
5. 找出内存访问模式，例如连续、跨步、随机、gather。
6. 判断控制流是否有难预测分支。
7. 用工具验证，不靠纯想象。

本书会不断训练这套方法。它看起来繁琐，但熟练以后会变成直觉。真正高手读性能代码时，脑子里会自动浮现数据路径、指令路径和瓶颈假设。

贯穿例子：求和循环

考虑：

```
extern "C" int sum_i32(const int* a, long n)
{
    int s = 0;
    for (long i = 0; i < n; ++i) {
        s += a[i];
    }
    return s;
}
```

一个标量汇编形态可能包含：

```
xor eax, eax
xor ecx, ecx
loop:
add eax, dword ptr [rdi + rcx*4]
inc rcx
cmp rcx, rsi
jl loop
ret
```

逐层理解：

- `rdi` 保存数组起始地址。
- `rsi` 保存长度 `n`。
- `eax` 保存累加和。
- `rcx` 保存循环下标。
- `[rdi+rcx*4]` 是数组元素地址。
- `add eax, dword ptr [...]` 同时做 load 和加法。
- `cmp` 和 `jl` 控制循环。

性能上，这段循环至少有几个问题可以讨论：

- 累加变量 `eax` 形成循环携带依赖。
- 每次迭代有一次 load。
- 每次迭代有循环分支。
- 如果编译器能证明安全并且目标支持 SIMD，优化版本可能向量化。

这就是从源代码到性能模型的第一步。你不需要在本章就优化它，但必须能把它拆成寄存器、内存、控制流和依赖。

给求和循环画一张简化依赖图

把循环体抽象成几条关键动作：

```
old rcx ----> address = rdi + rcx*4 ----> load a[i] ----+
                                                    |
old eax -----> add -> new eax

old rcx ----> inc rcx ----> cmp rcx, rsi ----> branch
```

这张图比汇编文本更接近性能分析需要的信息。它告诉我们：地址计算依赖旧下标，load 依赖地址，累加依赖 load 和旧累加值，下一轮累加又依赖这一轮的新累加值。循环控制则由下标递增、比较和分支组成。

如果数组数据在 cache 中，load 很快，累加依赖和循环控制开销可能更明显；如果数组很大且访问模式不友好，内存系统可能成为主要等待来源。若编译器能向量化或展开循环，它可能用多个累加器削弱单一 `eax` 链，并减少每个元素分摊的分支成本。但这些都是假设，必须看实际生成的机器码和测量结果。

再看一个指针追逐循环：

```
struct Node {
    Node* next;
    int value;
};

extern "C" int sum_list(Node* p)
{
    int s = 0;
    while (p) {
        s += p->value;
        p = p->next;
    }
    return s;
}
```

这里下一次迭代的地址依赖当前节点里的 `next`。即使 CPU 很宽，也很难同时发起很多未来节点的 load，因为未来地址还没有拿到。数组顺序求和和链表求和都叫“求和”，但底层依赖图完全不同。前者容易利用连续访问、预取和向量化；后者常常受 pointer chasing 延迟限制。

心智模型

源代码算法相似，不代表机器执行相似。数组循环、链表循环、哈希表查找、树遍历的关键差异，往往体现在地址是否可提前知道、load 是否可并行、分支是否可预测。

从执行模型到初步性能假设

到这里，我们已经有足够概念建立第一层性能假设。注意这里说的是假设，不是结论。合格的假设要被反汇编、计时和后续 PMU 证据支持或推翻。

第一类假设是前端供给问题。若热点代码指令很多、分支密集、跳转目标分散、函数调用无法内联，CPU 前端可能无法稳定提供足够 uops。症状可能是增加算术单元并不能提速，或者小幅改变代码布局就影响明显。第一册只要求你知道这种可能性；第三部分会强调测量证据。

第二类假设是计算关键路径问题。若循环每次迭代都依赖上一轮结果，例如单累加器 reduction、递推公式、加密链式状态，性能可能受延迟限制。增加独立工作、展开循环或分拆累加器有时能提高吞吐，但必须保证语义正确，尤其要注意整数溢出、浮点结合律和 C++ 未定义行为边界。

第三类假设是内存延迟问题。若程序访问大工作集、随机地址、链表或树结构，load 可能经常等待较远层级。此时减少一两条整数指令未必有意义，改善布局、访问顺序、批处理或缓存局部性才可能有效。

第四类假设是内存带宽问题。若程序顺序扫描大量数据，每个元素计算很少，瓶颈可能是单位

时间能从内存层级搬来多少字节，而不是单次 load 的延迟。此时要关注数据宽度、写入量、是否重复扫描、是否产生无用临时数组。

第五类假设是分支预测问题。若内层循环有高频且难预测的数据相关分支，错误预测会浪费前端和后端工作。替代方案可能是改变数据布局、预分类、使用条件移动、使用查表或掩码计算。但任何替代方案都可能增加其他成本，不能只凭“去掉分支”判断。

假设类型	常见线索	第一证据
前端供给	代码大、分支密、调用多	反汇编、循环形态、后续 PMU
关键路径	循环携带依赖明显	依赖图、展开对比实验
内存延迟	随机访问、指针追逐	访问模式、规模扫描
内存带宽	大数据顺序流式处理	字节量、吞吐估算
分支预测	数据相关高频分支	输入分布、分支版本对比

这张表的价值在于约束思考顺序。不要看见程序慢就立刻改代码。先问慢可能来自前端、依赖、内存延迟、内存带宽还是分支；再设计最小实验让这些解释互相区分。

一份合格的指令级分析应该写什么

从本章开始，报告中如果要分析一个函数的机器执行，不能只贴一段反汇编。贴反汇编只是证据的起点，不是结论。合格的指令级分析至少应该包含下面内容。

第一，说明构建上下文。包括编译器、优化级别、目标平台、是否开启调试信息、是否开启 sanitizer、是否使用 LTO。没有构建上下文，汇编不可复现。

第二，说明函数接口。参数从哪里来，返回值到哪里去，是否遵循 System V ABI，是否有 `extern "C"`，是否可能被内联。接口决定初始寄存器状态，也决定调用者和被调用者之间的责任。

第三，标出关键指令的状态变化。对每个重要计算，写清楚它读哪些寄存器、写哪些寄存器、是否读写内存、是否改 flags。不要试图逐字解释每条无关 prologue/epilogue 指令，但关键路径必须清楚。

第四，画出数据依赖。尤其是循环中的累加变量、指针递增、下标比较、load 结果使用。性能不是由文本顺序决定，而是由依赖图和资源共同决定。

第五，描述内存访问模式。连续、跨步、随机、间接寻址、写入输出、读写同一 buffer，这些都会影响 cache、TLB 和预取。

第六，描述控制流。分支条件来自哪里，是否形成循环，分支方向是否与输入数据相关，是否可能被编译器改成 `cmov` 或 `setcc`。

第七，说明当前分析的边界。比如“本章只基于反汇编判断可能瓶颈，尚未用 PMU 证明 cache miss 数量”；或者“只比较了 GCC 的 `-O3` 输出，尚未比较 Clang 和不同 `-march`”。

报告项	要回答的问题
构建上下文	这段机器码如何生成，别人能否复现
ABI 初始状态	参数、返回值、栈、调用者责任是什么
指令语义	关键指令读写哪些状态
依赖图	哪些操作必须等待前面的结果
内存模式	地址如何变化，是否有局部性
控制流	分支和循环如何决定 <code>rip</code>
证据边界	哪些是事实，哪些是假设，哪些还要测量

这套报告结构比“写一段感想”严格得多，但它能训练真正的底层阅读能力。后面讲 ABI、C++ 与汇编互操作、benchmark 设计时，也会使用类似结构。

架构可见状态和实现内部状态

读 CPU 执行模型时，必须分清架构可见状态和实现内部状态。架构可见状态是 ISA 承诺给软件的内容：通用寄存器、`rip`、`RFLAGS`、内存可见结果、异常行为。实现内部状态是某个 CPU 为了更

快执行而使用的结构：物理寄存器、重排序缓冲、保留站、load/store queue、分支预测器、cache、TLB、store buffer。软件通常不能直接依赖这些内部结构的具体形态。

这一区分解释了很多现象。同一份 x86-64 机器码，在不同 CPU 上必须给出相同的架构结果，但性能可以完全不同。因为 ISA 只规定“这条 add 对寄存器和 flags 做什么”，不规定它在某个微架构中用几个周期、几个 uops、哪个端口、怎样旁路、是否与相邻指令融合。性能工程关心实现内部状态，但正确性首先建立在架构状态上。

例如 `mov eax, edi` 的架构语义很简单：把 `edi` 的低 32 位写入 `eax`，并清零 `rax` 高 32 位。内部实现可能通过寄存器重命名把这个 move 消除，也可能真的执行一个 move uop，这取决于 CPU。报告中如果只分析正确性，写架构语义即可；如果分析性能，就需要进一步查目标 CPU 的实现特点或用测量验证。

为什么先学架构语义

初学者常急着问“这条指令几个周期”。这个问题有价值，但不应该排在第一位。你必须先知道指令读写什么状态、是否访问内存、是否改 flags、是否依赖上一条指令、是否改变控制流。否则周期数没有意义。一个低延迟指令如果在关键依赖链上，仍可能限制循环；一个高吞吐指令如果很少执行，可能不是瓶颈；一条 load 如果命中 L1 和如果 miss 到内存，成本差别巨大。

架构语义是跨机器稳定的学习基础。微架构性能是目标相关的工程细节。第一册先要求你能稳定读懂架构语义，并建立定性性能假设；第二册才会系统展开端口、uops、cache 层级、SIMD 吞吐和具体 CPU 差异。

异常、提交和“看起来执行过”的边界

现代 CPU 可以乱序执行和推测执行，但对普通程序来说，它必须维持精确的架构边界。所谓精确异常，是指当某条指令触发异常时，软件看到的状态像是异常之前的指令都已完成，异常之后的指令都没有完成。这条性质让操作系统、调试器和信号处理能够定位到一个清晰的故障点。

例如一段代码中有一个可能越界的 load。CPU 可能在内部提前发起某些后续计算，也可能沿预测路径做了后来会被丢弃的工作。但如果这个 load 触发 page fault，操作系统处理异常时需要看到一致的 `rip` 和寄存器状态。错误路径上的推测结果不能作为正式架构结果提交。

这也解释了“执行过”和“正式发生”的区别。推测执行可能让硬件内部短暂访问某些数据、占用 cache 或预测结构，但如果路径错误，架构寄存器和内存结果会被丢弃。安全侧信道研究正是利用了内部状态和架构状态之间的差异。第一册不展开这些攻击，但必须知道：ISA 层的程序结果和微架构内部痕迹不是同一个概念。

store 为什么需要更谨慎

load 读取内存，若路径错误，结果可以丢弃；store 改写内存，若过早对外可见，错误路径就会破坏程序状态。因此现代 CPU 对 store 的提交更谨慎。store 通常先进入 store buffer，等到指令确定可以提交时，再按内存一致性要求对外可见。这个机制能提高性能，也让观察 store 行为比观察寄存器写入更复杂。

从第一册角度，你只需要掌握两个结论。第一，普通单线程程序看到的架构结果仍按语言和 ISA 规则成立，不能因为 CPU 乱序就随意重排有依赖的操作。第二，性能上，store 可能因为 buffer、cache line ownership、store-to-load forwarding、写合并和内存顺序约束产生复杂成本。看到 store 密集代码时，不要只按“每条 store 一样快”理解。

依赖图比指令列表更接近性能

汇编文本按地址顺序排列，但性能常由依赖图决定。两条指令相邻，不代表它们互相依赖；两条指令隔得远，也可能因为寄存器、内存或 flags 形成关键路径。读性能时，要把线性文本转成图。

一个简单例子：

```
add eax, dword ptr [rdi]
add ecx, dword ptr [rsi]
```

这两条加法分别依赖不同地址和不同累加器。若内存命中且资源足够，它们可能有较多并行空间。另一个例子：

```
add eax, dword ptr [rdi]
add eax, dword ptr [rsi]
```

第二条必须等待第一条产生新的 `eax`，形成累加依赖。两段代码指令数量相同，但并行性不同。循环中这种差异会被重复放大。

`flags` 也会进入依赖图：

```
cmp ecx, esi
jl .Lloop
```

`jl` 依赖 `cmp` 产生的 `flags`。若中间有不写 `flags` 的 `lea`，依赖仍然连接到 `cmp`；若中间有写 `flags` 的 `add`，条件就变了。手写汇编和报告都必须把 `flags` 当作状态，而不是把它当成隐形注释。

内存依赖需要地址证据

内存依赖比寄存器依赖难，因为两个内存操作是否访问同一位置，可能要等地址算出来才知道。比如：

```
mov dword ptr [rdi], eax
mov ecx, dword ptr [rsi]
```

如果 `rdi` 和 `rsi` 指向同一地址，第二条 `load` 依赖第一条 `store`；如果它们一定不同，就可以更自由地重叠。编译器和 CPU 都会处理这类问题，但方式不同。编译器依赖类型、别名规则和静态分析；CPU 依赖运行时地址、`load/store queue` 和内存顺序预测。

性能报告中若要说“这两个内存访问独立”，必须有证据：不同数组且接口保证不重叠，或运行时检查分离，或地址范围计算表明不交叉。没有证据时，只能写“可能独立”。

从机器状态追踪到性能问题

机器状态追踪和性能分析不是两件事。正确的状态追踪会自然暴露性能问题的候选来源。

如果你追踪寄存器时发现每轮循环都要等待同一个累加器，可能是 `reduction` 关键路径。如果你追踪地址时发现下一轮地址来自当前 `load` 的结果，可能是 `pointer chasing`。如果你追踪 `flags` 时发现每轮都有数据相关分支，可能需要考虑预测。如果你追踪内存时发现每个元素只用一次且工作集巨大，可能是带宽或延迟问题。如果你追踪调用时发现热路径频繁跨动态库或函数指针，可能有间接调用和内联失败问题。

因此，本章的状态表不是只服务正确性。它也服务初步性能分类。一个好的状态表会记录：

- 循环携带寄存器：哪些值从上一轮传到下一轮。
- 地址来源：地址是否可提前计算，是否连续。
- `load` 使用距离：`load` 之后多久被使用。
- `store` 位置：是否与后续 `load` 可能重叠。
- 分支条件：是否由输入数据决定，是否可预测。
- 调用边界：是否破坏寄存器，是否阻止内联。

这些信息不直接给出周期数，但能告诉你下一步该设计什么实验。比如怀疑内存带宽，就做输入规模扫描和 `bytes/s` 计算；怀疑分支预测，就改变输入分布并看 `branch miss`；怀疑关键路径，就展开循环或拆多个累加器做对比；怀疑函数调用开销，就比较内联和非内联版本。

初学者应该避免的 CPU 模型

有几种 CPU 心智模型会误导学习。

第一，把 CPU 当成逐行解释器。现代 CPU 的内部执行不是简单地完成一条再开始下一条。它会流水、并行、预测和乱序。但架构提交又保持程序可见顺序。正确模型应同时包含“内部可以重叠”和“架构结果受规则约束”。

第二，把指令成本当成固定常数。某条 `load` 可能命中 `L1`，也可能等待内存；某条分支可能预测正确，也可能错误；某个函数调用可能被内联，也可能跨共享库；同一条指令在不同 CPU 上实现不同。成本要放在上下文中判断。

第三，把优化器当成机械翻译器。编译器不会忠实保留源码形状，它会在语言规则允许范围内

重写程序。源代码中有变量，不代表机器里有栈槽；源代码中有函数，不代表二进制里有调用；源代码中有循环，不代表 benchmark 中真的执行了循环。

第四，把硬件能做当成语言允许。CPU 可以执行某个越界地址计算，不代表 C++ 允许越界访问；CPU 有补码回绕，不代表 signed overflow 定义良好；CPU 可能屏蔽移位计数，不代表 C++ 源码里的越界移位合法。第一册反复强调语言语义和机器行为的边界，就是为了避免这种错误。

第五，把单个工具输出当成完整真相。反汇编告诉你机器码，不告诉你运行频率；GDB 告诉你某次状态，不告诉你统计分布；PMU 告诉你事件计数，不自动解释源码原因；源码告诉你意图，不保证二进制形态。底层分析需要多种证据互相校验。

核心理想

CPU 学习的第一目标不是背微架构术语，而是建立正确层次：架构语义保证程序结果，微架构实现决定成本，编译器决定机器码形态，测量决定结论可信度。

综合案例：同样求和，机器路径为什么不同

考虑三个求和函数：

```
extern "C" int sum_array(const int* p, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        s += p[i];
    }
    return s;
}

struct Node {
    Node* next;
    int value;
};

extern "C" int sum_list(Node* p)
{
    int s = 0;
    while (p) {
        s += p->value;
        p = p->next;
    }
    return s;
}

extern "C" int sum_if_positive(const int* p, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        if (p[i] > 0) {
            s += p[i];
        }
    }
    return s;
}
```

三个函数在算法描述上都很简单，甚至都可以说是“遍历并累加”。但机器路径完全不同。

sum_array 的地址按 **base+i*4** 递增。CPU 和编译器容易看出连续访问，可能展开、向量化或预取。关键路径可能是累加依赖，也可能在大输入下变成内存带宽。要验证，需要看汇编是否展开/向量化，再做规模扫描。

sum_list 的下一轮地址来自当前节点的 **next** 字段。未来地址无法提前知道，load 之间形成链式依赖。即使每个节点只做一次加法，性能也可能受内存延迟限制。优化方向通常不是减少加法，而是改变数据结构、批量化、改善局部性或避免链表。

sum_if_positive 在数组连续访问之外，还引入数据相关分支。若输入几乎全正或全负，分支可能可预测；若正负随机，分支 miss 可能明显。编译器也可能使用条件移动或向量 mask。性能取决于输入分布。没有记录 distribution 的 benchmark，不足以说明这个函数的行为。

这个案例说明，性能假设必须来自机器路径，而不是函数名或高层算法描述。三个函数都叫求

和，但一个偏向连续内存，一个偏向指针追逐，一个偏向分支分布。第一册的 CPU 执行模型，就是为了让你看出这些差异。

把案例写成报告

一份报告可以这样组织：

```
Question:
    Why do three sum-like functions have different performance shapes?

Binary:
    sum_array uses indexed load or pointer increment.
    sum_list loads next pointer before next iteration.
    sum_if_positive contains branch or conditional move.

Hypotheses:
    sum_array may become bandwidth-bound for large n.
    sum_list may be latency-bound due to pointer chasing.
    sum_if_positive may depend on input distribution.

Experiments:
    size scan for sum_array
    randomized linked list vs contiguous nodes for sum_list
    all-positive, all-negative, random signs for sum_if_positive

Limits:
    first pass only; no PMU evidence yet.
```

注意这个报告没有先给结论，而是给问题、二进制事实、假设和实验。CPU 模型的价值就在于把“慢”拆成可验证的机制。

自测清单：指令级分析是否合格

读完本章后，分析一个小函数时应能回答：

1. 入口参数在哪些寄存器或栈位置。
2. 每条关键指令读写哪些架构状态。
3. 哪些指令访问内存，地址如何计算。
4. 哪些指令生产 flags，哪些指令消费 flags。
5. 哪些值形成循环携带依赖。
6. 哪些 load 的地址可以提前知道，哪些必须等前一次 load。
7. 哪些分支由输入数据决定，哪些是固定循环控制。
8. 哪些性能判断只是静态假设，尚未测量。
9. 反汇编来自哪个构建和哪个二进制层级。
10. 当前分析是否区分了架构语义和微架构性能。

如果只能逐行翻译汇编，但不能回答这些问题，说明还停留在语法层。指令级分析的目标是恢复状态变化和依赖结构，而不是把助记符翻译成中文。

深入讲解：顺序语义和并行执行如何同时成立

很多读者第一次听到乱序执行，会产生一个疑问：如果 CPU 可以乱序，那程序顺序还有什么意义？答案是，乱序是实现策略，程序顺序是架构承诺。CPU 可以在内部提前执行不改变可见语义的工作，但提交给软件的结果必须像按规则执行一样。

例如：

```
mov eax, dword ptr [rdi]
add ecx, edx
imul r8d, r9d
```

如果三条指令互不依赖，CPU 内部可以重叠执行。load 等待 cache 的同时，加法和乘法可以执行。软件最终看到的寄存器结果与顺序执行一致。这里乱序提高了资源利用率，但没有改变架构语义。

再看有依赖的情况：

```
mov eax, dword ptr [rdi]
add eax, ecx
mov dword ptr [rsi], eax
```

第二条必须等待第一条 load 得到 `eax`，第三条必须等待第二条产生新 `eax`。CPU 不能随意把 store 的数据改成旧值。它可以提前准备地址、调度其他无关指令，但真依赖必须遵守。性能分析中，真依赖决定关键路径。

内存依赖更复杂。如果两个地址不同，load 和 store 可能重叠；如果地址相同，顺序可能影响结果。CPU 会使用 load/store queue 和预测机制处理，但一旦发现预测错误，就要修正。程序员在第一册只需要记住：内存操作的并行性取决于地址和别名，不能只看指令数量。

深入讲解：为什么 cache 不是一个“加速开关”

cache 经常被粗略解释成“CPU 旁边的高速缓存”。这个解释太浅。对性能分析来说，关键问题不是有没有 cache，而是数据访问模式如何与 cache line、容量、关联度、预取、写回和 TLB 交互。

顺序扫描数组时，CPU 每取一个 cache line，可以使用其中多个相邻元素。硬件预取器也更容易猜到后续地址。随机访问时，每次 load 可能带来一个新 cache line，只使用其中一个元素，空间局部性差。链表访问更糟，因为下一地址依赖当前节点内容，预取也困难。

cache 容量也重要。一个工作集在 L1 中、在 L2 中、在 LLC 中、超出 LLC，表现可能完全不同。因此 benchmark 必须扫描规模。只测一个大小，不知道结果代表哪个层级。一个优化在 L1 中快，不一定在内存带宽限制下有用；一个布局改变在大规模下有效，小规模可能看不出。

写入也不是免费。store 可能需要取得 cache line 的写权限，可能造成写分配，可能占用 store buffer，可能与其他核心共享状态交互。第一册不深入多核一致性，但要知道：读写模式、数据共享和写入量都会影响性能。

因此，当你看到内存访问相关性能问题时，不要只说“cache 问题”。更准确的问题是：访问是顺序还是随机？工作集多大？每个 cache line 使用多少有效数据？是否有重复利用？是否大量写入？是否跨页导致 TLB 压力？这些问题比“有没有 cache”更有工程价值。

深入讲解：分支预测不是只预测 if

分支预测不只影响源码中的 `if`。循环回边、函数返回、间接调用、虚函数、switch 跳转表、错误路径分支都会进入控制流预测。现代 CPU 需要提前知道下一段指令在哪里，否则前端供给会断。

循环分支通常很好预测，因为大多数迭代都回到循环头，只有最后一次退出。数据相关分支则取决于输入模式。全正、全负、周期性、随机、块状数据，对预测器难度不同。间接调用和虚函数还需要预测目标地址；目标集合越大、模式越乱，越难预测。

分支错误预测的代价不是“跳转指令慢”，而是 CPU 沿错误路径做了一些工作，发现错后丢弃，并重新从正确路径取指。代价与流水线、前端、后端和当前工作有关。第一册不要求给出固定周期数，但要求你知道：分支成本依赖可预测性，不是所有 `if` 都慢，也不是所有 branchless 都快。

这就是为什么控制流章节反复强调输入分布。一个 `if(x>0)` 在全正数据上几乎不造成预测问题，在随机正负数据上可能很明显。报告不写输入分布，就无法解释分支性能。

课堂讲解：硬件原理要从状态和约束讲起

初学 CPU 原理时，最容易被两种说法带偏。一种说法把 CPU 讲成“取指、译码、执行”的简单循环，好像每个时钟都规规矩矩完成一条指令。另一种说法一上来就讲乱序、重命名、保留站、提交队列和端口，好像不掌握所有微架构细节就无法理解程序执行。第一册采用中间路线：先把架构状态和合法约束讲清楚，再把实现技巧看成在这些约束内提高吞吐的方法。

架构状态包括寄存器、RIP、RFLAGS、可见内存、异常状态和一些控制状态。程序最终能观察到的，是这些状态按照 ISA 和操作系统合同发生变化。实现机器内部可以有更多结构，例如物理寄存器、乱序窗口、load queue、store buffer、分支预测器和 cache 层级，但这些结构不能随便改变架构语义。它们可以提前做、并行做、猜着做，也可以在猜错后丢弃，但提交给程序的结果必须像某种合法顺序发生过。

这个视角非常重要。它让读者既不会把 CPU 想得过于简单，也不会被实现细节淹没。遇到一

段机器码，先问架构上它改变什么状态；遇到一个性能现象，再问实现上这些状态变化受什么约束。正确性分析主要站在架构语义一侧，性能分析再逐步引入实现成本。把这两件事混在一起，会产生很多错误结论。

组合逻辑、时序逻辑和指令执行的联系

从硬件底层看，处理器由组合逻辑和时序逻辑组成。组合逻辑根据当前输入产生输出，时序逻辑在时钟边沿保存状态。寄存器文件、程序计数器、流水线寄存器和各种队列都是状态保存点；加法器、比较器、选择器、地址生成逻辑和译码逻辑则更接近组合计算。真实 CPU 远比这个模型复杂，但这个基础足够解释为什么状态变化需要被组织，为什么流水线可以把工作拆段，为什么依赖会限制并行。

一条加法指令看似只是把两个数相加，但在硬件中至少涉及取指地址、指令字节、译码结果、源操作数读取、执行结果、flags 生成、目的寄存器写入或提交。若下一条指令需要这个结果，它必须等待某个结果可转发或可读取的时刻。若下一条指令完全独立，则可以在流水线或乱序窗口中和它重叠。性能差异不来自“CPU 喜欢哪条源码”，而来自数据依赖和资源占用。

内存指令更能体现状态约束。load 需要地址，地址可能来自寄存器加偏移；虚拟地址要经过权限检查和地址翻译；cache 命中与否决定数据何时到达；如果前面有 store，处理器还要判断这个 load 是否应该读到刚写入但尚未写回 cache 的值。store 又涉及写入顺序、异常边界、store buffer 和可见性。第一册不深入展开所有细节，但要让读者知道：内存访问不是一个单一动作。

观察粒度：源码行、指令、微操作和事件

性能分析经常出错，是因为观察粒度混乱。源码行是程序员写下的表达，指令是 ISA 层的操作，微操作是实现层可能采用的内部拆分，硬件事件是 PMU 对某些现象的计数。它们之间有关联，但没有简单的一一对应。

一行源码可能生成零条指令，也可能生成很多条指令。零条指令不一定表示程序没有语义，可能是常量折叠、内联、死代码删除或调试信息映射变化。很多条指令也不一定表示源码复杂，可能是 ABI 传参、栈保护、异常处理、边界检查、地址计算或优化级别造成的。阅读时要从构建参数和函数身份开始，而不是凭行数判断。

一条 x86 指令也不一定等于一个微操作。某些指令会拆成多个内部动作，某些简单指令可以融合，某些寻址模式把地址计算和内存访问放在同一条文本指令里。微操作数量、端口使用和吞吐不是 ISA 文本直接给出的普遍真理，而是具体微架构的实现属性。第一册中我们会谨慎使用这类概念，只在能帮助建立性能直觉时引入。

硬件事件也不是最终答案。branch miss、cache miss、cycles、instructions、task clock、context switch 这些计数可以支持或推翻假设，但不能自动解释程序。比如 cache miss 增多，可能来自输入变大、访问变随机、结构体布局改变、预取失效、TLB 压力或系统干扰。看到事件后必须回到代码、数据和机器码，形成闭环。

为什么先要求读者会手动追踪状态

自动工具很有用，但第一册仍然反复要求手动追踪寄存器、内存、flags 和控制流。原因很简单：如果读者不能手动解释一个小例子，就无法判断工具摘要是否可信。工具会帮你节省时间，不会替你建立语义。

手动追踪并不是把每个真实程序都逐条模拟。它是一种校准训练。通过小函数，你会知道 32 位寄存器写入为什么清零高位，cmp 为什么本质是只更新 flags 的减法，call 为什么改变栈和 RIP，数组访问为什么变成 base 加 index 乘 scale，加法溢出为什么和 flags 有关。掌握这些小规则后，再看大程序时才能快速抓住关键路径。

综合案例：从错误性能直觉回到执行模型

考虑一个常见判断：“这个循环只有一个加法，所以一定很快。”这个判断缺少执行模型。假设循环每次从链表节点读取下一个指针，再把节点值累加。源码里确实只有一个加法，但每次迭代还包含一次依赖的指针 load。下一次迭代的地址必须等当前节点指针读出来以后才知道。这会形成跨迭代的内存依赖链，CPU 很难并行发起很多未来 load。

再看数组求和。它同样每次有一个加法，但地址是基址加递增偏移。处理器和编译器更容易看出连续访问，多个 load 可以在流水线中重叠，硬件预取也更容易工作。若编译器能向量化，还可以

一次处理多个元素。两个循环在算法大 O 上可能相同，在源码上也都很短，但机器执行路径完全不同。

如果读者只背“加法便宜，内存贵”，仍然不够。还要说清楚是哪一种内存访问贵：连续还是随机，独立还是依赖，工作集是否命中 cache，TLB 是否稳定，分支是否可预测，编译器是否改变循环形态。高质量解释要把直觉细化到可观察证据。至少应包含源码、关键汇编、地址公式、依赖图、输入规模和测量方式。

这个案例也说明为什么第一册把硬件原理、汇编原理和测量原理放在同一条线。只学硬件，会缺少源码和编译器边界；只学汇编，会缺少实验可信度；只学 benchmark，会不知道数字背后的机器原因。三者合在一起，才能把“感觉慢”变成“这个循环可能受跨迭代 load 依赖限制，需要用数组化、预取、分批或数据布局调整验证”的工程判断。

机器状态追踪报告

学习 CPU 执行模型时，建议把一个小函数写成机器状态追踪报告。报告不需要覆盖所有内部微架构细节，只要追踪架构可见状态：入口参数在哪些寄存器，关键指令读写哪些寄存器，flags 何时产生和消费，load/store 访问哪个地址，rip 如何移动，函数如何返回。

例如一个带条件的加法函数，可以按时间线写：

```
entry:
    edi = a
    esi = b

cmp edi, 0:
    updates flags according to edi - 0

jle .Lzero:
    consumes flags
    if a <= 0 jump to zero path

lea eax, [rdi + rsi]:
    computes a + b
    returns int in eax
```

这类报告的价值在于建立因果关系。flags 不是“顺便存在”，它连接比较和跳转；eax 不是“变量”，它在某一刻承载返回值；lea 不访问内存，只计算地址形式的整数表达式。把这些写清楚，后面读复杂汇编时才不会把状态搞混。

状态追踪不等于性能结论

状态追踪能说明程序做什么，不直接说明它快不快。知道某条指令是 load，只是开始；还要知道地址是否命中 cache、是否依赖前一条 load、是否可以并行、是否跨页、是否被分支保护。知道某个循环有分支，也只是开始；还要知道输入分布和预测稳定性。

因此，报告中应把语义结论和性能假设分开。语义结论可以比较确定，例如“该路径返回零”。性能假设应写成可验证形式，例如“随机输入下该分支可能难预测，需要用不同分布和 branch-miss 事件验证”。这能防止读者把静态阅读当成最终性能证明。

从硬件直感到可测假设

硬件直觉需要转化成可测假设。直觉说“随机访问慢”，假设应写成“在相同元素数量下，随机下标访问的 ns/element 高于顺序访问，且 cache-miss 或 TLB 相关事件更高”。直觉说“依赖链限制并行”，假设应写成“单累加器版本比多累加器版本慢，反汇编显示前者形成跨迭代依赖，后者有多个独立累加器”。直觉说“分支难预测”，假设应写成“随机分布比全真分布更慢，branch-miss 率更高”。

可测假设有三个要素：改变什么、观察什么、如何解释。改变的是输入分布、数据布局、循环形态或编译选项；观察的是时间、反汇编、硬件事件或错误率；解释必须能被反例推翻。若任何结果都能被你解释成“硬件复杂”，那就不是好假设。

现代 CPU 初学者的安全边界

第一册只建立现代 CPU 的基础直觉，不试图完整讲完微架构。读者应知道哪些话可以说，哪些话要保守。可以说：顺序语义和乱序执行可以同时成立；依赖限制并行；内存访问成本取决于层级和模

式；分支成本取决于可预测性；指令文本不能单独决定性能。不要轻易说：某条指令固定多少周期；某个 cache miss 一定解释了全部差异；某个 PMU 事件单独证明了瓶颈；所有 CPU 上某种写法都更快。

保守不是胆小，而是尊重证据。现代 CPU 有不同代际、不同缓存层级、不同前端、不同执行资源、不同预测器和不同功耗策略。同一份 ISA 程序在不同实现上的成本可能不同。第一册训练的是基础推理，第二册才会更系统地进入微架构和 SIMD 优化。现在只要把正确性、指令、依赖、内存、分支和测量连接起来，就已经打下重要基础。

本章学习输出

本章建议留下三份产物。第一份是机器码观察报告：从一个函数的机器码字节、汇编文本和源码对应关系出发，说明 CPU 实际执行的是字节而不是 C++。第二份是状态追踪报告：选择一个带分支或调用的小函数，逐条追踪寄存器、flags、内存和 rip。第三份是性能假设报告：比较数组顺序访问、随机访问或依赖链版本，写出假设、实验和限制。

这三份产物分别对应事实、语义和成本。事实层知道机器码是什么；语义层知道状态如何改变；成本层知道哪些现象需要测量。三层都掌握，后面进入汇编、ABI 和 benchmark 才能连贯。

从单核直觉走向系统现实

本章主要从单个 CPU 执行指令的角度建立模型，但真实程序运行在系统中。操作系统可能调度线程，处理中断，管理页面，迁移进程，处理缺页，响应系统调用。即使单条指令语义清楚，程序的运行时间仍可能受到系统现实影响。这并不推翻 CPU 模型，而是提醒读者区分执行语义和运行环境。

例如一个 load 在 ISA 层只是从内存地址读取值；在系统层，它可能触发缺页，由内核建立页面映射；在性能测量中，这次缺页会让一次样本异常变慢。一个循环在机器码层没有系统调用，但如果数据第一次访问，页面和 cache 状态仍会影响时间。一个线程本来在某个核心上运行，调度迁移后 cache 和分支预测状态可能改变。

因此，CPU 执行模型是必要基础，但不是性能结论的全部。后面测量章节会要求记录环境、warmup、CPU 绑定和样本分布，就是为了把系统现实纳入证据链。读者现在只要记住：语义由合同定义，成本由合同、实现和环境共同决定。

学习 CPU 时的两种错误极端

第一种极端是把 CPU 想得过于机械，认为程序文本顺序就是内部执行顺序，认为每条汇编固定耗时，认为 cache 只是简单加速。这种模型太弱，无法解释乱序、预测、cache 层级和依赖。第二种极端是过早进入微架构细节，试图用端口、队列和专有事件解释所有问题，却没有先证明代码语义、输入和二进制身份。

正确路线是分层推进。先能追踪架构状态，再能读依赖图，再能提出性能假设，再用测量验证。不要在还没确认循环是否被删除时讨论执行端口，也不要再在还没确认指针合法时讨论预取。深度不是从最复杂名词开始，而是从最可靠事实向下钻。

本章给出的模型足以支撑第一册后续内容。它不会替代第二册的微架构专题，但能保证读者进入高级主题时不再悬空。

综合案例：一次错误硬件解释如何被修正

设想一个学生测到两个求和函数速度不同。第一个函数按数组顺序求和，第二个函数按一个索引数组间接访问。学生最初解释为“第二个函数的加法更慢”。这个解释站不住，因为两个函数的整数加法语义相同，真正变化的是地址产生方式和内存访问路径。

合格分析应先回到机器码。顺序版本的地址通常是 base 加递增偏移，load 地址可以提前推导，硬件预取更容易工作。间接版本需要先从索引数组 load 下标，再用下标计算数据数组地址，第二次 load 依赖第一次 load 的结果。两者的关键差异不是加法，而是内存依赖和地址可预测性。若索引随机，cache line 利用率和 TLB 局部性也会变差。

然后把假设写成可验证实验。输入规模要扫描，因为小规模可能都在 cache 中，大规模才显示内存差异；索引分布要变化，顺序、块状、随机会得到不同结果；反汇编要保存，证明两个版本的核心循环形态；若能使用硬件事件，再观察 cache 和 TLB 相关计数。最终报告可以写：“当前证据支持间接 load 依赖和局部性下降解释差异”，而不是笼统说“CPU 不喜欢随机”。

这个案例训练的是纠错能力。看到性能差异时，不要把最近学到的术语套上去。先确认指令、地址、依赖、输入和测量，再给解释。硬件原理越深入，越要抵抗过度解释的冲动。

执行模型的学习边界

第一册中的执行模型是为了支持工程判断，而不是要求读者成为处理器设计者。你应该能解释寄存器、flags、RIP、load/store、call/ret、分支预测、cache 和依赖的基本关系；但不需要在此时背诵每个微架构的端口表、队列大小和预测器细节。那些内容会在第二册或专门资料中展开。

边界清楚以后，学习反而更有效。遇到未知性能现象，可以先用本章模型提出粗假设；若粗假设不足，再查更深资料。比如你发现循环受前端限制，再去研究指令长度、decode、uop cache 和分支布局；发现内存瓶颈，再去研究 cache 层级、TLB、预取和带宽。不要在没有问题驱动时盲目钻入细节，也不要再在证据需要时拒绝深入。

本章术语小结

- 架构状态：ISA 规定的软件可见状态，例如寄存器、flags、rip 和内存。
- 微架构状态：CPU 内部实现细节，例如物理寄存器、ROB、cache、TLB、预测器和队列。
- 机器码：CPU 取指单元读取的字节序列。
- 汇编：机器码的人类可读表示，不等于源码，也不等同于最终字节。
- 取指：根据 rip 获取指令字节，可能涉及 instruction cache 和权限检查。
- 解码：把机器码字节解释成指令语义和内部执行形式。
- 执行：在执行单元、load/store 单元或分支单元中完成实际动作。
- 提交：让执行结果按程序顺序成为架构可见状态。
- 精确异常：异常发生时，软件看到的状态像是异常前指令已完成、异常后指令未完成。
- 数据依赖：后一条操作需要前一条操作的结果。
- 真依赖：后一条操作读取前一条操作产生的值，是关键路径分析的核心。
- 假依赖：由于架构寄存器名字复用产生的反依赖或输出依赖，很多情况下可由重命名缓解。
- 内存依赖：load/store 是否必须保持顺序取决于地址、别名和内存模型约束。
- 控制依赖：下一条执行路径取决于分支、调用、返回或异常。
- 推测执行：CPU 沿预测路径提前执行，错误时丢弃架构结果。
- store buffer：暂存尚未完全写入缓存层级的 store，使写入和后续执行能够更好重叠。
- TLB：缓存虚拟地址到物理地址翻译的硬件结构，影响访存成本。

实验：观察机器码字节

目标：确认 CPU 执行的是机器码字节，汇编只是人类可读表示。

步骤：

```
mkdir -p scratch/ch02
cat > scratch/ch02/add.cpp <<'EOF'
extern "C" int add_i32(int x, int y)
{
    return x + y;
}
EOF

g++ -O0 -c scratch/ch02/add.cpp -o scratch/ch02/add_00.o
g++ -O3 -c scratch/ch02/add.cpp -o scratch/ch02/add_03.o

objdump -drwC -Mintel scratch/ch02/add_00.o
objdump -drwC -Mintel scratch/ch02/add_03.o
```

交付物：

- 截取 add_i32 的反汇编。
- 标出每条指令前面的机器码字节。
- 解释 -O0 和 -O3 的差异。

- 说明返回值为什么在 `eax`。

实验：用 GDB 单步观察寄存器

目标：用调试器观察 `rip`、`rsp`、`eax`、`edi`、`esi` 如何变化。

准备代码：

```
#include <stdio>

extern "C" __attribute__((noinline))
int add_i32(int x, int y)
{
    return x + y;
}

int main()
{
    int z = add_i32(10, 32);
    std::printf("%d\n", z);
}
```

编译：

```
g++ -O0 -g scratch/ch02/debug_add.cpp -o scratch/ch02/debug_add
gdb scratch/ch02/debug_add
```

GDB 命令：

```
set disassembly-flavor intel
break add_i32
run
disassemble /r add_i32
info registers rip rsp rax eax rdi edi rsi esi
si
info registers rip rsp rax eax rdi edi rsi esi
si
info registers rip rsp rax eax rdi edi rsi esi
```

交付物：

- 每次 `si` 后记录关键寄存器。
- 解释 `rip` 为什么变化。
- 解释 `rsp` 在函数入口和返回附近怎样变化。
- 解释调试版本为什么比优化版本多很多栈帧相关指令。

实验：flags 和条件跳转

目标：理解 `cmp`、`test`、条件跳转如何配合。

任务：写三个函数：

```
extern "C" int is_zero(int x)
{
    return x == 0;
}

extern "C" int max_i32(int a, int b)
{
    return a > b ? a : b;
}

extern "C" int abs_i32(int x)
{
    return x < 0 ? -x : x;
}
```

分别使用 `-O0`、`-O2`、`-O3` 编译，观察是否出现：

- `cmp`
- `test`
- `jcc`
- `cmov`
- `setcc`

交付物：

- 为每个函数画出控制流或数据流。
- 解释哪条指令设置 `flags`，哪条指令读取 `flags`。
- 判断优化版本是否减少了分支。

实验：call、ret 和栈

目标：理解函数调用如何使用栈保存返回地址。

任务：写三个 `noinline` 函数：

```
extern "C" __attribute__((noinline)) int f3(int x)
{
    return x + 3;
}

extern "C" __attribute__((noinline)) int f2(int x)
{
    return f3(x + 2);
}

extern "C" __attribute__((noinline)) int f1(int x)
{
    return f2(x + 1);
}
```

用 GDB 在 `f3` 里停住，执行：

```
bt
info registers rip rsp rbp
x/8gx $rsp
disassemble /r f1
disassemble /r f2
disassemble /r f3
```

交付物：

- 画出调用链。
- 标出每一层返回地址。
- 解释 `call` 和 `ret` 怎样配合。
- 比较 `-O0` 和 `-O2` 下调用链是否变化，解释 `inline` 的影响。

实验：顺序访问和随机访问的第一印象

目标：用简单实验感受 `cache` 和访问模式对性能的影响。本实验不要求建立完整 `cache` 模型，只要求形成问题意识。

任务：分配一个较大的 `std::vector<int>`，分别做：

- 顺序求和。
- 固定 `stride` 求和，例如 `stride = 16`。
- 按一个打乱后的 `index` 数组随机求和。

要求：

- 用相同数据规模。
- 防止编译器把循环优化掉。
- 每个 case 至少运行多次，报告中位数。
- 记录 CPU 型号和编译命令。

交付物：

- 三种访问模式的时间。
- 你对差异的解释。
- 你认为还需要哪些证据才能证明解释是正确的。

实验：依赖链和独立工作的差异

目标：观察真依赖链和独立计算对执行时间的影响，训练用依赖图解释结果。

任务：写两个循环，循环次数足够大，并确保结果被使用。

第一版使用单一累加链：

```
extern "C" std::uint64_t dep_chain(std::uint64_t n)
{
    std::uint64_t x = 1;
    for (std::uint64_t i = 0; i < n; ++i) {
        x = x * 1664525u + 1013904223u;
    }
    return x;
}
```

第二版使用多个独立累加器：

```
extern "C" std::uint64_t independent_chains(std::uint64_t n)
{
    std::uint64_t a = 1;
    std::uint64_t b = 3;
    std::uint64_t c = 5;
    std::uint64_t d = 7;
    for (std::uint64_t i = 0; i < n; ++i) {
        a = a * 1664525u + 1013904223u;
        b = b * 22695477u + 1u;
        c = c * 1103515245u + 12345u;
        d = d * 214013u + 2531011u;
    }
    return a ^ b ^ c ^ d;
}
```

要求：

- 使用 `-O2` 或 `-O3` 编译。
- 记录编译器版本、CPU 型号、编译命令和循环次数。
- 用反汇编确认结果没有被删除。
- 比较两个函数每次迭代的大致耗时。

交付物：

- 两个函数的关键反汇编。
- 手画依赖图，标出哪个版本有单一长链。
- 对计时差异的解释。
- 说明这个实验为什么不能单独证明所有 CPU 上的通用结论。

常见误区

常见误区

误区一：CPU 执行的是 C++。

不准确。CPU 执行机器码字节。C++ 是高层语言，编译器把它翻译到目标 ISA。调试信息可以帮助你吧机器状态映射回源代码，但那不是 CPU 的原生输入。

常见误区

误区二：汇编和机器码是一回事。

不准确。汇编是文本表示，机器码是字节。它们可以互相转换，但反汇编不一定恢复原始汇编源文件中的标签、注释和宏结构。

常见误区

误区三：一条汇编一定对应一个硬件动作。

不准确。复杂 x86 指令可能解码成多个 uops；简单指令也可能在流水线中经历多个阶段。性能上要区分 ISA 指令、uops、执行端口、延迟和吞吐。

常见误区

误区四：程序顺序就是内部执行顺序。

不准确。现代 CPU 可以乱序执行 ready 的操作，但 retire 时必须保持架构可见结果符合程序顺序。性能分析要看依赖图，而不只是文本顺序。

常见误区

误区五：内存访问速度固定。

不准确。同样是一条 load，数据在 L1、L2、L3、主内存时成本完全不同。访问模式、工作集大小、TLB、预取和 NUMA 都会影响性能。

常见误区

误区六：去掉分支一定更快。

不准确。条件移动、掩码计算或查表可能减少错误预测，但也可能增加无条件计算、拉长依赖链或增加内存访问。分支是否值得替换，要看预测率、输入分布和替代代码成本。

常见误区

误区七：寄存器重命名能解决所有等待。

不准确。重命名能缓解名字复用造成的假依赖，不能消除后一条指令真正需要前一条结果的真依赖，也不能让 cache miss 的数据提前出现。

本章作业

习题与作业

作业 1：手工执行指令。

给定函数入口状态：

- `edi = 7`。
- `esi = 5`。
- `edx = 3`。
- `rsp = 0x7fffffff000`。
- `memory[0x7fffffff000]` 保存返回地址 `0x401234`。

手工执行：

```
mov eax, edi
add eax, esi
sub eax, edx
ret
```

写出每条指令后的 `eax`、`rax`、`rsp`、`rip` 的变化，并说明哪条指令会更新 `flags`。

习题与作业

作业 2：解释一个数组访问。

对下面函数：

```
extern "C" int load2(const int* a, long i)
{
    return a[i + 2];
}
```

生成优化汇编。解释地址表达式中 `base`、`index`、`scale`、`displacement` 分别是什么。说明为什么 `int` 下标会乘以 4。

习题与作业

作业 3：比较 `add` 和 `lea`。

写两个或更多简单加法函数，尝试让编译器生成 `add` 或 `lea`。报告中必须包含：

- 源码。
- 编译命令。
- 反汇编。
- 你对编译器选择的解释。
- 这两类指令对 `flags` 的影响差异。

习题与作业

作业 4：调用链报告。

基于 `call`、`ret` 和栈实验，写一份不少于 800 字的报告，回答：

- 函数调用前参数在哪里？
- 返回值在哪里？
- 返回地址在哪里？
- `rsp` 如何变化？
- 优化级别改变后，为什么调用链可能消失？

习题与作业

作业 5：第一次性能解释。

完成顺序访问和随机访问实验，写一份性能报告。报告不能只写“随机访问慢”，必须包含：

- 数据规模。
- 编译器版本和命令。
- CPU 型号。
- 每种访问模式的时间统计。
- 你基于 `cache line` 和局部性的解释。
- 你认为本实验还有哪些不严谨之处。

习题与作业

作业 6：依赖图报告。

任选一个你熟悉的小函数，例如数组求和、查找最大值、条件计数、链表求和或字符串扫描。分别生成 `-O0` 和 `-O3` 的反汇编，然后以优化版本为主写一份报告。

报告必须包含：

- 源码和编译命令。
- 关键反汇编，不要求贴完整目标文件。
- 参数和返回值对应的寄存器或内存位置。
- 循环体中的关键 `load`、`store`、算术和分支指令。
- 至少一张手画依赖图，标出真依赖、内存依赖和控制依赖。
- 一个初步性能假设，例如受累加链限制、受随机 `load` 限制、受分支预测限制或受前端供给限制。
- 说明还需要哪些测量证据才能验证这个假设。

评分重点不是结论是否“高级”，而是证据链是否清楚：源码语义、编译产物、指令语义、依赖图和性能假设必须互相对应。

验收标准

学完本章，你应该能够做到：

- 解释 CPU 不直接执行 C++，而是执行机器码字节。
- 区分汇编文本、机器码、ISA 语义和微架构实现。
- 说清楚 `rip`、通用寄存器、`rsp`、`RFLAGS` 的基本作用。
- 手工执行简单整数指令，追踪寄存器和 `rip` 变化。
- 解释 `call` 和 `ret` 如何用栈保存和恢复控制流。
- 解释一条 `load` 指令为什么可能涉及地址生成、TLB 和 cache。
- 区分真依赖、假依赖、内存依赖和控制依赖。
- 解释 `flags` 为什么是一条隐含数据流。
- 初步说明 `store buffer`、`store-to-load forwarding` 和 TLB 对访存路径的意义。
- 用 `objdump` 观察机器码字节和反汇编。
- 用 GDB 单步观察寄存器变化。
- 初步解释 `pipeline`、`cache`、`branch prediction`、`out-of-order execution` 为什么存在。
- 为一个小循环写出依赖图和可验证的性能假设。

如果你能完成这些目标，后面正式进入 x86-64 汇编、ABI、性能测量和微架构时，就不会把术语当成孤立概念，而能把它们放回真实执行链路中理解。

第 4 章

数据表示、内存、指针和对象布局

问题与目标：a[i] 到底访问了什么

刚学 C/C++ 时，数组访问通常被解释为“取第 i 个元素”。这个说法对编写程序有帮助，但不足以支撑机器执行和性能分析。系统分析、性能测量和高性能 kernel 开发中的许多问题，都可以追溯到一个更底层的问题：

某条指令到底访问了哪个地址上的哪些字节？

先看一个简单函数：

```
int get(int const* a, int i)
{
    return a[i];
}
```

在 x86-64 System V ABI 下，编译器可能生成类似汇编：

```
movsxd rsi, esi
mov    eax, dword ptr [rdi + rsi*4]
ret
```

这三行值得仔细拆解。

- rdi 保存第一个参数 a，也就是数组首元素地址。
- esi 保存第二个参数 i 的低 32 位。
- movsxd rsi, esi 把 32 位有符号整数扩展成 64 位索引。
- [rdi+rsi*4] 是地址表达式，意思是 a+i*4。
- dwordptr 说明这次从内存读取 4 字节。
- eax 是返回值寄存器的低 32 位。

因此，a[i] 在机器层面不是抽象的“数组操作”，而是一次地址计算加一次内存读取：

```
address = base_address(a) + i * sizeof(int)
load 4 bytes from address
interpret those 4 bytes as int
```

核心思想

机器没有“数组第几个元素”的概念。机器看到的是地址、字节数、寄存器和指令。C/C++ 类型系统把“第几个元素”翻译成“从哪个基地址开始，按多大步长计算地址，读写多少字节，并按什么规则解释这些字节”。

本章要把这个过程讲清楚。后续性能优化会反复依赖这些基础：cache line 为什么按固定粒度搬运数据，SIMD load 为什么偏好连续布局，结构体字段顺序为什么影响性能，未定义行为为什么会扩大编译器优化空间，AoS 和 SoA 为什么差异显著，GEMM packing 为什么本质上是在重排内存访问。

本章学习路线：从一个字节走到一个 kernel

本章内容容易学散，因为它同时涉及位、整数、指针、数组、结构体、padding、cache line 和汇编寻址。学习时不要把它们当成孤立知识点背诵；它们之间存在一条明确的依赖链：

```
bit -> byte -> object representation -> type rule
      -> address -> pointer expression -> layout
```

-> x86-64 addressing mode -> cache line behavior
-> SIMD / HPC / AI kernel data movement

这条链路的意思是：

1. **bit 和 byte** 让你知道机器保存信息的最小粒度。
2. **对象表示** 让你知道一个 C++ 对象最终落成哪些字节。
3. **类型规则** 让你知道这些字节如何被解释，编译器能假设什么。
4. **地址和指针** 让你把源码表达式变成具体内存位置。
5. **布局** 让你知道数组元素、结构体字段、padding 在地址空间里的相对位置。
6. **x86-64 寻址模式** 让你把 C++ 访问表达式对应到 $[base + index * scale + displacement]$ 。
7. **cache line** 让你解释为什么同样的运算量会因为访问模式不同而差很多。
8. **SIMD 和 kernel 优化** 则要求你主动设计数据布局，让硬件以更少指令、更少 cache miss、更高带宽效率完成同样计算。

如果只是记住“int 通常 4 字节”“double 通常 8 字节”，本章价值很有限。真正要训练的是一种稳定能力：看到任意 C++ 表达式，能回答它访问哪些字节；看到任意内存操作汇编，能反推它可能来自什么数据布局；看到一个慢循环，能判断它是否受布局、步长、对齐或别名影响。

本章的四个观察层次

同一个程序可以从四个层次观察。初学者常犯的问题，是只停在第一层或把四层混在一起。

层次	你看到什么	本章要你学会的问题
C++ 源码层	<code>a[i]</code> 、 <code>p->x</code> 、 <code>sizeof(S)</code>	这个表达式的类型是什么？对象生命周期存在吗？访问是否越界？
对象布局层	字段 offset、padding、对齐、数组步长	目标字节从基地址偏移多少？一次访问读写几字节？
汇编层	<code>moveax, [rdi+rsi*4]</code>	基址、索引、缩放和常量偏移分别来自源码里的什么？
微架构预告层	cache line、步长、连续性、带宽	这些访问会顺序命中 cache，还是浪费 cache line、破坏预取和 SIMD？

学习时必须强迫自己每次都跨层解释。例如：

```
float z = particles[i].z;
```

不要只说“取第 i 个粒子的 z ”。更完整的解释应该是：

1. `particles` 是 `Particle*`，所以 `particles+i` 以 `sizeof(Particle)` 为步长。
2. 如果 `Particle` 大小为 16，`z` 字段 offset 为 8，那么目标地址是“首元素地址 + $i*16+8$ ”。
3. 目标字段是 `float`，因此 load 宽度是 4 字节。
4. 在 x86-64 汇编里，因为 `scale` 不能直接取 16，编译器可能先计算 $i*16$ ，再访问 `dword ptr [base+offset+8]`。
5. 如果循环只读 `z`，AoS 布局中每 16 字节只用 4 字节，cache line 利用率可能只有四分之一。

对应的汇编形态可以放进单独代码块观察：

```
shl rax, 4
movss xmm0, dword ptr [rdi + rax + 8]
```

这种解释方式看起来繁琐，但它是性能优化的基本功。后面做矩阵乘、卷积、attention、quantization、packing、prefetch、SIMD load/store 时，所有复杂技术都只是这条链路的规模化版本。

符号约定和推理习惯

为了让后面的推导不混乱，本章使用下面几个约定：

- `addr(x)` 表示对象或子对象 x 的起始地址。

- `sizeof(T)` 表示类型 `T` 的对象大小，单位是字节。
- `offset(S, field)` 表示字段 `field` 相对结构体 `S` 起始地址的字节偏移。
- `stride` 表示相邻两个逻辑元素之间的地址距离，单位通常是字节。
- `loadwidth` 表示一次 `load` 指令读取多少字节。

你每次推导内存访问时，都应该把“元素下标”和“字节偏移”分开写。下标是源码层概念，偏移是机器地址层概念。例如：

```
a[3] index      = 3
a[3] byte_offset = 3 * sizeof(int)
                = 12
```

把这两个数混在一起，是阅读汇编时最常见的早期错误。

习题与作业

随堂检查：把一句话说完整

请不要查资料，直接回答下面三个问题。每个答案至少写出“类型、基地址、字节偏移、访问宽度”四个词。

1. `double*p;p[i]` 访问的字节地址如何计算？
2. `struct V { float x, y, z, w; }; V* v; v[i].z` 访问的字节地址如何计算？
3. 下面两条汇编合起来可能对应什么 C++ 访问？

```
shl    rax, 4
movss xmm0, dword ptr [rdi + rax + 8]
```

如果你的答案里只有“取第几个元素”，说明你还停留在源码层；本章后面要反复训练到机器层。

位、字节和机器字：数据表示的最小地基

位是信息，字节是寻址单位

bit (binary digit) 是二进制位，只能表示 0 或 1。一个 bit 可以表达一个真假状态，但真实程序中的整数、地址、浮点数和指令都需要多个 bit 组合。

byte (字节) 在现代通用机器上通常是 8 bit。C++ 标准中 `char` 是最小可寻址对象单位；在我们学习的 x86-64/Linux 环境中，一个 byte 就是 8 bit。

单位	大小	常见用途
bit	1 个二进制位	标志位、掩码的一位
byte	8 bit	最小寻址单位、 <code>char</code> 、对象表示
word	依上下文变化	CPU 自然处理宽度或 ISA 术语
dword	32 bit	x86 汇编中的双字
qword	64 bit	x86 汇编中的四字

注意 **word** (字) 是一个容易混淆的词。在 x86 历史语境里，word 常指 16 bit，dword 是 32 bit，qword 是 64 bit；在体系结构教材中，word 有时指机器自然处理宽度；在内存系统里，cache line 又是另一种粒度。读文档时必须看上下文。

二进制、十六进制和为什么程序员爱用十六进制

二进制直接对应 bit，但写起来太长。十六进制每一位刚好表示 4 bit，因此两个十六进制数字刚好表示 1 byte：

十六进制	二进制	十进制
0x0	0000	0
0x1	0001	1
0x7	0111	7
0x8	1000	8
0xF	1111	15

例如：

```
0x12 = 0001 0010
0x34 = 0011 0100
0x1234 = 0001 0010 0011 0100
```

十六进制非常适合观察内存、机器码和地址。后面你会经常看到：

- 对象字节：78563412。
- 机器码字节：8b04b7。
- 地址：0x7ffe...。
- 位掩码：0xff、0xffff0000。

内存的第一模型：按字节寻址的巨大数组

从最简单的机器模型看，内存可以被想象成一个按地址索引的字节数组：

```
address  byte
0x1000  -> 0x78
0x1001  -> 0x56
0x1002  -> 0x34
0x1003  -> 0x12
0x1004  -> ...
```

地址是“某个字节的位置编号”。如果一个 int 占 4 字节，地址 0x1000 处的 int 会覆盖：

```
0x1000
0x1001
0x1002
0x1003
```

但是这个模型还不完整，因为普通 Linux 进程看到的是虚拟地址，不是物理内存条上的真实位置。更准确的说法是：对用户态程序而言，指针值通常是虚拟地址；CPU 执行 load/store 时会通过地址翻译机制把虚拟地址转换为物理地址，期间会用到 TLB 和页表。

本章先使用“内存是字节数组”的模型来理解对象表示、指针和布局。虚拟内存、页表和 TLB 会在后续系统与硬件专题中展开。

心智模型

先把内存想成字节数组,但永远记住它只是第一模型。性能优化时,你还要逐步叠加 cache line、cache set、TLB、page、NUMA、预取器和内存带宽。

类型不是字节本身，而是解释字节的规则

内存中的 byte 本身没有“这是 int”或“这是 float”的标签。类型存在于程序语言、编译器和指令语义中。

例如同样 4 个字节：

```
00 00 80 3f
```

如果按 32 位整数解释，它是某个整数位模式；如果按 IEEE 754 float 解释，它是 1.0f；如果按 4 个 unsignedchar 解释，它只是 4 个字节。

C++ 的类型系统至少做了四件事：

1. 决定对象需要多少字节，例如 `sizeof(int)`。
2. 决定对象要求什么对齐，例如 `alignof(double)`。
3. 决定表达式如何生成地址，例如 `p+1` 前进多少字节。
4. 决定编译器可以做哪些优化，例如别名规则和对象生命周期规则。

常见误区

“内存只是字节，所以我想怎么解释就怎么解释”是非常危险的半真半假。硬件层面确实是字节；C++ 语言层面有对象、类型、生命周期、对齐和别名规则。高性能代码经常需要看字节，但必须用语言允许的方式看。

对象表示：C++ 对象在内存中的字节

C++ 中的对象有类型、存储期、生命周期和值。对象在内存中的底层字节叫作 **object representation**（对象表示）。对于一个可平凡复制的对象，你可以通过 `unsignedchar`、`std::byte` 或 `char` 观察它的对象表示。

对象表示和值表示不是同一个概念

对象表示是对象占用的全部字节；值表示是这些字节中真正参与表示对象值的部分。对许多整数类型，二者非常接近，读起来像一回事；但一旦出现 padding、浮点特殊值、结构体填充字节、未初始化存储或某些平台相关表示，二者就不能混用。

例如一个结构体可能占 8 字节，其中只有 5 字节对应字段，剩下 3 字节是 padding。对象表示包含这 8 字节；结构体的语义值通常只由字段决定。两个结构体对象字段完全相等，但 padding 字节不同，它们的语义值仍然相等。若你用 `memcmp` 比较整个对象表示，就可能把这两个语义相等的对象判成不相等。

这个区分会影响三类工程场景。第一，序列化时不能默认把结构体原始字节直接写进文件，因为 padding、字节序和 ABI 都可能让文件格式不可移植。第二，哈希时不能随便哈希整个对象字节，因为 padding 可能不稳定。第三，性能优化时不能把“字节搬运成功”误认为“类型语义正确”，尤其是跨语言、跨进程、跨文件格式和跨硬件平台时。

可平凡复制不是随便改类型

对可平凡复制类型，把对象表示复制到另一块同类型对象存储中，通常可以保持值。这是 `std::memcpy`、文件读写缓冲、网络包组装和 SIMD 批量搬运能够工作的基础之一。但这不等于可以用任意类型指针解释任意字节。

比较下面两种说法：

- “把一个 `float` 的对象表示复制到 `std::uint32_t` 中，用来观察位模式。”这应使用 `std::bit_cast` 或 `std::memcpy`。
- “把 `float*` 强行转成 `std::uint32_t*` 后解引用。”这会碰到别名、对齐和对象生命周期问题，不能作为可靠写法。

底层程序常常需要观察位模式，但观察位模式和改变对象类型是两件事。前者可以通过字节视图、`std::memcpy` 或 `std::bit_cast` 表达；后者必须符合 C++ 对象模型。把这条边界搞清楚，后面理解 strict aliasing、对象生命周期和编译器优化会轻松很多。

常见误区

对象表示提供“这些字节长什么样”的证据；类型和值提供“这些字节按什么规则解释”的语义。系统工程必须能看字节，但不能忘记 C++ 程序仍然受对象、类型、生命周期和别名规则约束。

下面的程序打印一个 32 位整数的字节：

```
#include <stdint>
#include <iomanip>
#include <iostream>

int main()
```



```

{
    std::uint32_t x = 0x12345678U;
    auto const* p = reinterpret_cast<unsigned char const*>(&x);

    for (std::size_t i = 0; i < sizeof(x); ++i) {
        std::cout << std::hex << std::setw(2) << std::setfill('0')
            << static_cast<unsigned char>(p[i]) << '\n';
    }
}

```

在 x86-64 上，你通常会看到：

```

78
56
34
12

```

这就是小端字节序。

字节序：为什么 0x12345678 在内存里像反过来

endianness（字节序）描述多字节对象的字节如何放进连续地址。

小端和大端

如果最低有效字节放在最低地址，叫 **little-endian**（小端）。x86-64 使用小端。对 0x12345678：

地址偏移	字节
+0	0x78
+1	0x56
+2	0x34
+3	0x12

如果最高有效字节放在最低地址，叫 **big-endian**（大端）。同一个值会存成：

地址偏移	字节
+0	0x12
+1	0x34
+2	0x56
+3	0x78

字节序影响什么，不影响什么

字节序影响“多字节对象在内存中的排列”。它不影响整数的数学值，也不影响寄存器里你写的 0x12345678 这个值本身。

字节序经常在这些场景中变重要：

- 打印对象字节。
- 解析二进制文件格式。
- 网络协议和磁盘格式。
- 手写 SIMD shuffle 或处理 packed data。
- 调试机器码和内存 dump。

多数纯 C++ 算法不需要直接关心字节序，但系统性能工程师必须能看懂字节序，否则看到内存 dump 时会误读数据。

整数表示：无符号模运算和有符号补码

无符号整数是按位宽取模的环

一个 N 位无符号整数可以表示 0 到 $2^N - 1$ 。运算按模 2^N 回绕。例如 8 位无符号整数：

255 + 1 == 0
0 - 1 == 255

C++ 对无符号整数溢出有明确规定：按模回绕。这让无符号整数适合写位运算、哈希、掩码、循环计数的一些底层逻辑。

有符号整数通常用补码表示

现代 x86-64 使用 **two’s complement**（二进制补码）表示有符号整数。以 8 位为例：

位模式	无符号解释	有符号补码解释
00000000	0	0
00000001	1	1
01111111	127	127
10000000	128	-128
11111111	255	-1

补码的好处是加法电路可以同时服务有符号和无符号加法。比如 -1 的 8 位补码是 11111111，加 1 得到 00000000，自然回到 0。

语言规则和机器行为不能混为一谈

机器补码加法会回绕，但 C++ 有符号整数溢出是未定义行为。也就是说，下面代码在 C++ 语言层面不能被解释成“最大值加一变最小值”：

```
int f(int x)
{
    return x + 1;
}
```

如果 `x==INT_MAX`，表达式 `x+1` 发生有符号溢出，程序有未定义行为。编译器可以假设这种情况不会发生，并据此优化。

这不是抠字眼。性能优化中，编译器对循环边界、条件判断、向量化和死代码删除的很多决定，都依赖“程序没有未定义行为”的假设。

深入理解

后续学习编译器优化时，你会看到 signed overflow、strict aliasing、out-of-bounds、未初始化读取、对象生命周期等规则如何影响优化。底层工程不是绕开语言规则，而是理解语言规则如何给编译器提供证明空间。

浮点数只先建立直觉

本章重点是整数、地址和对象布局。浮点数会在 AI 算子、数值稳定性、SIMD 和 FMA 章节中深入。本章先建立三个直觉：

- 1. float 和 double 也是固定字节数的对象表示。
- 2. 它们通常使用 IEEE 754 格式，包含符号、指数和尾数。
- 3. 浮点运算不是实数运算，存在舍入、特殊值、非结合性和 fast-math 语义问题。

例如 `float x=1.0f;` 的对象字节在小端 x86-64 上通常是：

00 00 80 3f

按整数看这只是一个位模式；按 IEEE 754 float 看它表示 1.0。这个例子再次提醒你：字节没有类型，解释规则来自类型和上下文。

指针：地址值加类型语义

指针值通常是地址

在普通 x86-64/Linux 用户态程序中，指针值通常可以理解为虚拟地址。比如：

```
int x = 42;
int* p = &x;
```

p 保存的是对象 x 的地址。打印 p 可能得到类似：

```
0x7ffdf2c3a6bc
```

但指针不仅是一个整数地址。C++ 指针还携带类型语义：`int*` 指向 `int` 对象，`double*` 指向 `double` 对象，`Particle*` 指向 `Particle` 对象。类型决定了解引用读取多少字节、指针加法前进多少字节、编译器如何判断别名。

解引用是一次按类型解释的内存访问

表达式 `*p` 不是“取地址里的东西”这么笼统。更准确地说：

1. 读取指针表达式 `p` 的值，得到一个地址。
2. 根据 `p` 的静态类型知道目标对象类型是 `int`。
3. 从该地址开始读取 `sizeof(int)` 个字节。
4. 按 `int` 的对象表示解释这些字节。

如果地址没有对齐、对象生命周期不存在、类型不匹配或越界，语言层面可能已经出错。硬件是否“碰巧能读”不是 C++ 程序正确性的充分条件。

指针的有效范围和 one-past-the-end

C++ 指针不是可以任意加减的通用整数。对数组对象，指针可以指向数组中的某个元素，也可以指向最后一个元素之后的 `one-past-the-end` 位置。这个 `one-past` 指针可以用于比较和循环终止判断，但不能解引用。

```
int a[4] = {1, 2, 3, 4};

int* first = &a[0];
int* last = &a[0] + 4; // one-past-the-end

for (int* p = first; p != last; ++p) {
    // *p is valid inside the loop
}
```

在机器地址层，`last` 的数值通常等于 `addr(a[0])+4*sizeof(int)`。但这个地址不是一个活着的 `int` 对象起点。它只是语言允许你用来表示范围结束的位置。若写 `*last`，即使硬件可能从那里读到 4 字节，C++ 层面仍然是未定义行为。

这条规则解释了为什么很多高性能循环喜欢使用半开区间：

```
[begin, end)
```

它既适合地址计算，也适合语言规则：`begin` 到 `end` 之前的每个元素可访问，`end` 只表示边界。你在汇编里看到循环把指针每次加 4 或 8，再与结束指针比较，本质上就是把下标循环改写成地址范围循环。

把指针当整数时要保持克制

`std::uintptr_t` 可以在许多平台上保存指针转换得到的整数值，但这不表示任意整数运算后再转回指针都一定有效。语言层的指针还和对象来源、有效范围、对齐和生命周期有关。底层调试时把指针打印成十六进制地址很有用；工程实现中把指针长期当普通整数算来算去，则需要非常明确的理由。

例如，内存分配器、arena、JIT、序列化运行时、垃圾回收器和硬件映射代码确实会做大量地址整数运算。但它们通常会把“原始字节存储”“对象构造”“对齐修正”“边界检查”“释放责任”分成清楚步骤。普通业务代码和性能 kernel 更应该优先使用类型化指针、`span`、数组引用、迭代器或明确的字节视图，而不是随意把所有指针变成整数。

深入理解

现代 C++ 标准和编译器实现中，pointer provenance 是一个复杂话题。本书第一册不展开规范争议，只给出工程底线：指针值看起来像地址，但正确访问对象还需要对象来源、生命周期、类型、对齐和边界都成立。

空指针、野指针和悬垂指针

指针错误是底层学习必须严肃对待的内容。

空指针 不指向任何对象。解引用空指针是错误。

野指针 未初始化或保存无效地址的指针。它可能指向任意位置。

悬垂指针 原来指向有效对象，但对象生命周期已经结束。

例如：

```
int* bad()
{
    int x = 1;
    return &x;
}
```

x 是局部自动对象，函数返回后生命周期结束。返回的指针值可能仍然看起来像一个地址，但它不再指向活着的 int 对象。用它做性能实验，结果没有意义。

有效访问的五个条件

到目前为止，我们已经能把指针看成地址加类型语义。但在 C++ 中，一次内存访问要可靠成立，至少需要同时满足五个条件。缺任何一个，都可能让程序进入未定义行为、实现相关行为或不可移植边界。

1. 指针值必须指向可访问的存储区域。
2. 目标对象的生命周期必须已经开始且尚未结束。
3. 访问类型必须符合对象的有效类型和别名规则。
4. 地址必须满足访问类型的对齐要求。
5. 访问范围不能越过对象或数组允许的边界。

很多初学者只检查第一条：地址是否看起来落在某个映射区域里。可是硬件“能读到字节”并不等于 C++ 程序“有权用这个类型读这个对象”。性能优化中，编译器正是利用后四条规则做更大胆的重写。

条件一：可访问的存储区域

如果指针指向未映射地址，CPU 访问时通常会触发 page fault，操作系统把它转成 SIGSEGV。这种错误比较容易观察。比如空指针解引用常见地访问地址 0 附近，用户进程通常没有映射这个区域。

但可映射不代表正确。一个悬垂指针可能仍然落在当前栈或堆映射里；一个越界指针可能落在同一页的另一个对象上；一个错误类型指针可能读到看似合理的字节。它们未必崩溃，却仍然不可靠。

条件二：对象生命周期

对象生命周期定义了某块存储中什么时候真的存在某个类型的对象。局部变量离开作用域后生命周期结束；delete 后动态对象生命周期结束；一块原始字节缓冲区在构造对象之前，还不是那个对象。

例如：

```
alignas(int) unsigned char storage[sizeof(int)];
int* p = reinterpret_cast<int*>(storage);
```

这段代码只是得到一个可能满足对齐的地址，并不自动在 storage 中创建 int 对象。现代 C++ 中，在原始存储中创建对象需要明确的构造动作，例如 placement new 或相关库工具。初学阶段不需要掌握全部对象模型细节，但要知道：存储空间和对象生命周期不是同一个概念。

这个边界对 allocator、arena、对象池、序列化、JIT 和高性能框架很重要。许多性能代码为了

减少分配，会复用一块大 buffer；如果只把 buffer 当字节，却没有正确管理对象构造和销毁，就会在语言层给优化器错误前提。

条件三：访问类型和别名规则

别名规则回答一个关键问题：两个不同类型的指针，编译器是否必须假设它们可能指向同一个对象？如果编译器能证明或假设它们不别名，就可以把 load/store 重排、缓存到寄存器、向量化或删除重复访问。

看一个简化例子：

```
int f(int* pi, float* pf)
{
    *pi = 1;
    *pf = 2.0f;
    return *pi;
}
```

若编译器根据别名规则认为 `int*` 和 `float*` 不能指向同一个有效对象，它可以把返回值优化成 1，而不必在写 `*pf` 后重新从 `*pi` 读取。这个优化不是编译器任性，而是语言规则提供了证明空间。

但 `char`、`unsignedchar` 和 `std::byte` 视图有特殊用途，可以观察对象表示。它们适合做字节级检查、序列化中间缓冲和调试输出。即便如此，从字节视图回到类型化对象，也仍然要处理生命周期、对齐和构造问题。

条件四：对齐

如果一个类型要求 8 字节对齐，而你用不满足对齐的地址构造或访问它，语言层面可能已经错误。某些硬件会容忍未对齐普通 load/store，但 C++ 优化器仍然可以假设对象满足其对齐要求。编译器生成向量指令、合并访问或选择特定 load/store 形式时，都可能依赖对齐信息。

例如把字节 buffer 中任意偏移强行转成 `double*`：

```
unsigned char buffer[32];
double* p = reinterpret_cast<double*>(buffer + 1);
double x = *p;
```

这里 `buffer+1` 很可能不是 8 字节对齐。即使 x86-64 某次运行没有崩溃，这也不能作为可靠 C++。正确做法通常是用 `std::memcpy` 从字节中复制出 `double`，或保证 buffer 偏移满足对齐并正确建立对象。

条件五：边界

数组边界和对象边界决定访问范围。读取一个 `int` 通常读取 4 字节；如果地址只剩 2 字节属于当前对象，硬件也许能跨过去读到别的对象字节，但语言层面没有这种权限。结构体字段访问也不能越过对象边界去读取邻居对象，除非你通过允许的字节视图观察对象表示。

边界规则是优化器理解循环的基础。若编译器能假设数组访问不越界，就可以消除某些检查、重排循环、展开或向量化。反过来，如果你的程序越界，优化器基于“不会越界”的前提做出的变换可能让错误表现得很奇怪。

核心理想

一次内存访问是否有效，不只看地址数值。它还要满足生命周期、类型、别名、对齐和边界。高性能 C++ 不是绕过这些规则，而是用清晰接口把这些前提告诉编译器和读者。

别名如何影响优化和测量

别名是从本章走向编译器优化的关键桥梁。两个指针若可能指向同一块存储，编译器就必须保守，因为一次写入可能改变另一个指针后续读取的值。若能证明不别名，编译器就能保留更多值在寄存器中，也更容易向量化。

一个三数组循环

考虑：

```
void add_arrays(float* out,
```



```

    float const* a,
    float const* b,
    std::size_t n)
{
    for (std::size_t i = 0; i < n; ++i) {
        out[i] = a[i] + b[i];
    }
}

```

从数学上看，这只是逐元素加法。但 C++ 接口没有明确说明 `out` 是否可能和 `a` 或 `b` 重叠。如果 `out==a`，这是 in-place 更新，仍然可能有合理语义；如果 `out` 和 `a` 部分重叠，某些写入会影响后续读取，循环重排和向量化就要谨慎。

编译器可能生成运行时别名检查：先判断几个指针范围是否相互重叠；如果不重叠，走向量化快速路径；如果可能重叠，走保守标量路径。这种“多版本循环”在优化汇编中很常见。初学者看到大量比较和跳转时，不要立刻以为编译器生成了无用复杂代码，它可能是在保护别名语义。

restrict-like 信息

C++ 标准没有标准 `restrict` 关键字，但一些编译器支持扩展，例如 GCC/Clang 的 `__restrict__`。它能告诉编译器：在这个指针访问范围内，不会通过其他受限制指针访问同一对象。正确使用时，它可以释放优化空间；错误使用时，程序语义就不可靠。

例如：

```

void add_arrays_restrict(float* __restrict__ out,
    float const* __restrict__ a,
    float const* __restrict__ b,
    std::size_t n);

```

这个接口把“不重叠”作为调用者必须遵守的合同。它不是单纯性能开关，而是语义承诺。若调用者传入重叠范围，后果不能用“编译器错了”解释。

接口设计比局部技巧重要

很多性能问题不是在循环体里多写一条指令，而是接口没有表达足够前提。比如：

- 指针是否可以为空。
- 输入输出是否允许重叠。
- 数据是否按某个字节数对齐。
- `n` 是否为向量宽度的倍数。
- 输入是否连续，还是有 stride。
- 输出 buffer 是否已经分配且足够大。

如果这些前提只存在于作者脑子里，编译器和调用者都无法可靠利用。高质量 kernel 通常会把前提写进 wrapper、断言、文档、类型、span、参数块或专门函数名里。第 14 章讲 C++ 与汇编互操作时，会把这种思想用于手写汇编 kernel 的边界设计。

常见误区

不要为了让编译器生成更快代码而随便添加不真实的别名前提。优化前提是合同，不是建议。合同不成立时，性能数字没有意义。

字节视图、类型转换和安全观察

底层学习经常需要观察字节，但不同方法语义不同。

用字节视图观察对象表示

如果只是打印对象表示，可以使用 `std::byteconst*` 或 `unsignedcharconst*`：

```

template <class T>
void print_bytes(T const& value)
{
    auto const* bytes =
        reinterpret_cast<unsigned char const*>(&value);
}

```

```
for (std::size_t i = 0; i < sizeof(T); ++i) {
    print_one_byte(bytes[i]);
}
```

这种做法适合观察对象占用的字节。它不表示你可以通过任意类型指针修改对象，也不表示 padding 字节有稳定语义。

用 `std::bit_cast` 解释位模式

如果要在两个大小相同、可平凡复制的类型之间查看位模式，C++20 提供 `std::bit_cast`：

```
#include <bit>
#include <cstdint>

std::uint32_t bits_of_float(float x)
{
    return std::bit_cast<std::uint32_t>(x);
}
```

`std::bit_cast` 表达的是“复制对象表示并按目标类型得到一个值”，不是通过错误类型指针去访问原对象。它通常会被优化成零成本代码，但语义比手写指针转型清楚。

用 `std::memcpy` 跨越原始字节边界

在处理二进制文件、网络包或手工 buffer 时，`std::memcpy` 是非常重要的工具。它可以把字节复制到一个正确类型的对象中：

```
std::uint32_t load_u32_le(unsigned char const* p)
{
    std::uint32_t value = 0;
    std::memcpy(&value, p, sizeof(value));
    return value;
}
```

这段代码还没有处理字节序转换，只是展示“从字节复制到对象”的基本方式。编译器通常能把固定大小的 `memcpy` 优化成 `load`，但源码语义保持清楚：没有通过不对齐的 `std::uint32_t*` 解引用，也没有假设字节 buffer 中原本就存在 `std::uint32_t` 对象。

为什么不推荐随手 `reinterpret_cast`

`reinterpret_cast` 是底层工具，但它不会替你解决生命周期、对齐、别名和边界。许多错误代码看起来像这样：

```
float x = 1.0f;
std::uint32_t bits =
    *reinterpret_cast<std::uint32_t*>(&x);
```

这段代码常常在某些机器上“能跑”，也可能生成期望位模式，但它给编译器和读者的语义都很差。更好的写法是 `std::bit_cast<std::uint32_t>(x)`。如果你的目标是性能，清楚语义通常更容易让编译器优化，而不是更难。

核心思想

观察字节、复制字节和把字节当成某个对象访问，是三件不同的事。使用 `std::byte/unsignedchar`、`std::bit_cast` 和 `std::memcpy`，可以把这三件事表达得更清楚。

指针加法：类型决定缩放

指针加法不是简单整数加法。若 `p` 类型是 `T*`，则 `p+k` 的地址通常是：

$$\text{address}(p + k) = \text{address}(p) + k * \text{sizeof}(T)$$

例如：

```
int* p = reinterpret_cast<int*>(0x1000);
```

```
int* q = p + 3;
```

如果 `sizeof(int)==4`，则 `q` 的数值地址是 `0x100c`，不是 `0x1003`。
这解释了 `a[i]`：

```
a[i] == *(a + i)
```

如果 `a` 是 `int*`，`a+i` 自动按 4 缩放；如果 `a` 是 `double*`，按 8 缩放；如果 `a` 是 `Particle*`，按 `sizeof(Particle)` 缩放。

常见误区

`reinterpret_cast<std::uintptr_t>(p)+1` 和 `p+1` 不是同一个概念。前者是整数地址加 1 字节；后者是指针按元素大小前进 1 个对象。写底层代码时必须分清“字节偏移”和“元素偏移”。

数组：连续对象序列

C/C++ 数组是连续存放的同类型对象序列：

```
int a[4] = {10, 20, 30, 40};
```

如果 `a[0]` 地址为 `0x1000`，且 `sizeof(int)==4`，则布局通常是：

元素	起始地址	覆盖字节范围
<code>a[0]</code>	<code>0x1000</code>	<code>0x1000..0x1003</code>
<code>a[1]</code>	<code>0x1004</code>	<code>0x1004..0x1007</code>
<code>a[2]</code>	<code>0x1008</code>	<code>0x1008..0x100b</code>
<code>a[3]</code>	<code>0x100c</code>	<code>0x100c..0x100f</code>

数组名退化为指针

在很多表达式中，数组名会退化为指向首元素的指针：

```
int a[4];
int* p = a;    // p == &a[0]
```

但数组本身不是指针。下面三个 `sizeof` 结果不同：

```
int a[4];

sizeof(a);        // 4 * sizeof(int)
sizeof(&a[0]);    // pointer size
sizeof(int*);     // pointer size
```

这一区别对性能代码很重要。模板、`span`、数组引用、指针和长度参数会影响编译器是否知道长度、是否能展开、是否能做边界相关优化。

越界不是“多走几个字节”这么简单

从机器地址角度看，`a[4]` 似乎只是访问 `a[0]` 后面第 16 个字节。但从 C++ 语言角度，访问数组范围外对象是未定义行为。

```
int a[4] = {1, 2, 3, 4};
int x = a[4];    // undefined behavior
```

它可能读到某个栈上的值，可能触发崩溃，也可能被编译器假设为永不发生并重写代码。性能工程中不要用越界行为做“技巧”。

四步推导法：从 C++ 表达到机器访问

前面已经讲了指针、数组和类型，但真正读汇编时，你需要一个固定动作。本节给出一个可以反复使用的四步推导法。以后看到任何内存访问表达式，都按这个顺序走：

1. **确定表达式的对象类型和访问宽度。**它最终读写的是 `int`、`double`、`float`，还是某个结构体字段？访问宽度是 1、2、4、8、16、32 还是 64 字节？
2. **确定基地址。**这个访问从哪个指针、数组首元素、结构体对象或栈帧位置出发？
3. **计算字节偏移。**把下标、字段 `offset`、数组步长、结构体大小都换成字节单位。
4. **映射到 x86-64 寻址。**尽量写成 `[base+index*scale+displacement]`，同时标出 `load/store` 宽度。

这四步看似简单，但它可以把 C++ 源码、对象布局和汇编连接成一条线。

例 1: `inta[5];a[3]`

假设 `a[0]` 的地址是 `0x1000`，并且 `sizeof(int)==4`。

步骤	推导
对象类型	<code>a[3]</code> 是 <code>int</code> 左值，访问宽度 4 字节。
基地址	数组首元素地址 <code>addr(a[0])=0x1000</code> 。
字节偏移	下标 3，元素大小 4，所以 <code>offset = 3*4=12=0x0c</code> 。
最终地址	<code>0x1000+0x0c=0x100c</code> ，覆盖字节 <code>0x100c..0x100f</code> 。

如果 `a` 作为第一个参数传入函数，`i` 作为第二个参数，汇编可能是：

```
mov eax, dword ptr [rdi + rsi*4]
```

这里 `rdi` 是基地址，`rsi` 是元素下标，`scale 4` 来自 `sizeof(int)`，`dword ptr` 表示访问 4 字节。

例 2: `doublea[5];a[3]`

同样是下标 3，但类型换成 `double` 后，字节偏移不同。先把条件单独写清楚：

```
addr(a[0])    = 0x1000
sizeof(double) = 8
```

因此：

```
offset = 3 * sizeof(double) = 3 * 8 = 24 = 0x18
addr(a[3]) = 0x1000 + 0x18 = 0x1018
```

对应汇编可能是：

```
movsd xmm0, qword ptr [rdi + rsi*8]
```

这次 `scale` 是 8，`load width` 是 `qword`，返回值进入 `xmm0`。同一个下标，因为类型不同，机器地址计算不同；同一个地址，因为解释规则不同，得到的值也可能不同。

例 3: `p[i].z` 如何拆成两层偏移

看一个对 AI/HPC 很常见的结构：

```
struct Vec4 {
    float x;
    float y;
    float z;
    float w;
};

float get_z(Vec4 const* p, std::size_t i)
{
    return p[i].z;
}
```

在常见 ABI 下，四个 `float` 连续排布：

字段	offset	大小	说明
x	0	4	第一个 float
y	4	4	第二个 float
z	8	4	第三个 float
w	12	4	第四个 float

所以：

```
addr(p[i].z) = addr(p[0]) + i * sizeof(Vec4) + offset(Vec4, z)
              = base      + i * 16          + 8
```

这个地址公式不能直接写成单条 x86-64 内存操作数，因为 x86-64 的 scale 只有 1、2、4、8。真实编译器可能先把 $i*16$ 算到寄存器里，再做 load：

```
shl    rsi, 4
movss  xmm0, dword ptr [rdi + rsi + 8]
```

这里第一条指令把元素下标变成字节偏移。对于 $i*16$ 这种 2 的幂乘法，左移通常最直观。对于 12、24、40 这类非 2 的幂步长，后面会专门用 `leaq` 解释。

不要死记某一种汇编形态。关键是看出地址公式里有三个来源：数组基址、元素步长、字段 offset。

例 4：二维数组下标不是两个 load

考虑真正的 C++ 二维数组：

```
int a[3][5];
int x = a[i][j];
```

`a` 是 3 行 5 列的连续数组。按行主序存放，`a[i][j]` 的线性元素下标是：

$$i * 5 + j$$

字节地址是：

```
addr(a[i][j]) = addr(a[0][0]) + (i * 5 + j) * sizeof(int)
```

这不是“先找到第 i 行指针，再从那个指针找第 j 个元素”。真正的 `inta[3][5]` 是一整块连续内存，行长度 5 是类型的一部分。只有 `int**` 或“指针数组”才是另一种布局。

常见误区

`inta[3][5]`、`int(*p)[5]`、`int**p` 是三种不同模型。前两者知道每行 5 个 `int`，可以用线性地址公式；`int**` 是指向指针的指针，通常需要先读取行指针，再访问该行的数据。它们的性能、局部性和汇编形态都可能不同。

例 5：把 `a[i]+=b[i]` 展开成 load-compute-store

一个赋值表达式往往不止一次内存访问：

```
void add_into(float* a, float const* b, std::size_t n)
{
    for (std::size_t i = 0; i < n; ++i) {
        a[i] += b[i];
    }
}
```

循环体在标量层面可以拆成：

```
tmp_a = load 4 bytes from addr(a[0]) + i * 4
tmp_b = load 4 bytes from addr(b[0]) + i * 4
tmp   = tmp_a + tmp_b
store 4 bytes to  addr(a[0]) + i * 4
```


这对性能很关键。你不能只数“一次加法”。真实循环每个元素至少有两个 load、一个 store、一个浮点加法、一个循环控制，以及可能的别名检查、边界处理和向量化 remainder。后面做 roofline 模型时，算术强度就是从这种拆解开始的。

习题与作业

随堂检查：按四步推导法写答案

对下面表达式分别写出对象类型、基地址、字节偏移公式和可能的 x86-64 内存操作形式：

1. `std::uint64_t*p;p[i]`
2. `float*x;x[i+1]`
3. `struct Pair { int a; int b; }; Pair* p; p[i].b`
4. `doublea[4][8];a[i][j]`
5. `struct Node { int key; Node* next; }; Node* p; p->next`

要求：不要只写 C++ 解释，必须写成字节地址公式。

x86-64 寻址模式：数组访问如何进入指令

x86-64 内存操作数常见形式是：

```
[base + index * scale + displacement]
```

其中：

- **base** 是基址寄存器。
- **index** 是索引寄存器。
- **scale** 通常只能是 1、2、4、8。
- **displacement** 是编码在指令中的常量偏移。

这非常适合表达 C/C++ 的数组和结构体字段访问。

访问 `inta[i]`

```
mov eax, dword ptr [rdi + rsi*4]
```

含义：

```
load 4 bytes from address rdi + rsi * 4
```

如果执行到这条指令时寄存器是：

寄存器	值	含义
<code>rdi</code>	<code>0x1000</code>	<code>a[0]</code> 的地址
<code>rsi</code>	<code>3</code>	下标 <code>i</code>

那么有效地址是：

```
effective_address = 0x1000 + 3 * 4
                  = 0x100c
```

`mov eax, dword ptr [0x100c]` 的语义是从 `0x100c` 开始读取 4 字节，写入 `eax`。在 x86-64 中，对 `eax` 的写入还会清零 `rax` 的高 32 位。这一点读汇编时经常重要：`int` 返回值写 `eax`，但调用者看到的完整 `rax` 高位已经被清零。

访问结构体字段

如果有：

```
struct S {
```

```
int a;
double b;
int c;
};

int get_c(S const* p)
{
    return p->c;
}
```

汇编可能是：

```
mov eax, dword ptr [rdi + 16]
ret
```

16 就是字段 `c` 相对结构体起始地址的 `offset`。编译器根据类型布局规则算出它，然后把常量写进指令。

这个例子可以完整展开。常见 x86-64 ABI 下，`S` 的布局通常是：

字节范围	内容	大小	原因
0..3	a	4	int 字段
4..7	padding	4	让 b 满足 8 字节对齐
8..15	b	8	double 字段
16..19	c	4	int 字段
20..23	tail padding	4	让 sizeof(S) 是 8 的倍数

假设 `rdi` 保存 `p==0x2000`，那么：

```
addr(p->c) = addr(*p) + offset(S, c)
            = 0x2000 + 16
            = 0x2010
```

因此：

```
mov eax, dword ptr [rdi + 16]
```

就是从 `0x2010..0x2013` 读取 4 字节。这里没有 `index` 和 `scale`，因为没有数组下标；只有结构体基地址和字段偏移。

结构体数组字段访问：scale 和 displacement 同时出现

更接近真实 kernel 的情况是结构体数组：

```
struct Pixel {
    std::uint8_t r;
    std::uint8_t g;
    std::uint8_t b;
    std::uint8_t a;
};

std::uint8_t load_b(Pixel const* p, std::size_t i)
{
    return p[i].b;
}
```

`sizeof(Pixel)==4`，字段 `b` 的 `offset` 是 2，所以地址公式是：

```
addr(p[i].b) = addr(p[0]) + i * sizeof(Pixel) + offset(Pixel, b)
              = base      + i * 4          + 2
```

可能的汇编是：

```
movzx eax, byte ptr [rdi + rsi*4 + 2]
ret
```

这条指令有几个细节：

- `byteptr` 表示从内存读取 1 字节。
- `movzx` 表示 zero extend，读取 8 位后零扩展到 32 位 `eax`。
- `rsi*4` 来自 `sizeof(Pixel)`。
- `+2` 来自字段 `b` 的 offset。

假设：

寄存器	值	含义
<code>rdi</code>	<code>0x3000</code>	<code>p[0]</code> 的地址
<code>rsi</code>	<code>5</code>	下标 <code>i</code>

则：

```
effective_address = 0x3000 + 5 * 4 + 2
                  = 0x3016
```

CPU 会读取 `0x3016` 这个地址上的 1 字节，把它零扩展到 32 位返回值。这个案例比 `inta[i]` 更接近图像处理、量化张量、packed struct、RGBA buffer 这类真实工作负载。

当 `scale` 不够用时：`sizeof(T)` 大于 8 怎么办

x86-64 内存操作数的 `scale` 只有 1、2、4、8。如果结构体大小是 16、24、32，编译器必须用额外指令先算出字节偏移。

例如：

```
struct Vec4 {
    float x;
    float y;
    float z;
    float w;
};

float load_z(Vec4 const* p, std::size_t i)
{
    return p[i].z;
}
```

地址公式是：

```
addr(p[i].z) = base + i * 16 + 8
```

一种可能汇编是：

```
shl    rsi, 4
movss  xmm0, dword ptr [rdi + rsi + 8]
ret
```

如果寄存器初值是：

寄存器	值	含义
<code>rdi</code>	<code>0x4000</code>	<code>p[0]</code> 的地址
<code>rsi</code>	<code>3</code>	下标 <code>i</code>

执行 `shl rsi, 4` 后，`rsi` 变成 48，也就是 `3*16`。随后：

```
effective_address = 0x4000 + 48 + 8
                  = 0x4038
```

`movss` 从 `0x4038..0x403b` 读取 4 字节到 `xmm0` 的低 32 位。读汇编时要注意：有些寄存器一开始表示“元素下标”，经过 `shl` 或 `lea` 后变成“字节偏移”。如果不跟踪这种语义变化，就会把地址算错。

lea：地址计算指令不一定访问内存

lea 的名字是 load effective address，但它不从内存读取数据，只做地址形式的整数计算。例如：

```
lea rax, [rsi + rsi*2]
```

这条指令计算的是：

```
rax = rsi + rsi * 2
    = 3 * rsi
```

它没有访问 [rsi+rsi*2] 这个内存地址。编译器经常用 lea 做乘法常数、数组偏移和指针计算，因为它可以把“加法 + scale + 常量”合到一条指令里。

例如结构体大小为 24 时：

```
struct Item24 {
    double a;
    double b;
    int c;
    int d;
};

int load_d(Item24 const* p, std::size_t i)
{
    return p[i].d;
}
```

若 sizeof(Item24)==24，offset(Item24,d)==20，地址公式是：

```
addr(p[i].d) = base + i * 24 + 20
```

编译器可能生成类似：

```
lea eax, [rsi + rsi*2]      ; eax = i * 3
mov eax, dword ptr [rdi + rax*8 + 20]
ret
```

这里第二条指令的 scale 是 8，所以总乘数是 3*8=24。这种组合在真实汇编里很常见。读它时不要只看最后一条内存操作，要把前面的 lea 一起纳入地址公式。

核心理想

C++ 的数组下标、指针解引用、结构体字段访问，在机器层面都落到地址计算。理解地址计算，是读汇编、理解 cache 行为和优化数据布局的共同入口。

对象的存储期：栈、堆、静态区只是第一步分类

C++ 对象可以有不同存储期。存储期决定对象的存储何时获得、何时释放。常见分类如下：

存储期	示例	生命周期大致特点
自动存储期	局部变量	进入作用域创建，离开作用域销毁
动态存储期	new、malloc	程序显式申请和释放
静态存储期	全局变量、static	程序启动到结束
线程存储期	thread_local	线程开始到线程结束

初学者常说“局部变量在栈上，new 出来的在堆上，全局变量在静态区”。这作为入门直觉可以，但要知道它不是完整语言规则。编译器优化可能把局部变量放在寄存器里，甚至完全消除；动态分配器背后可能使用 mmap；静态对象还涉及 ELF 段、重定位和运行时初始化。

栈对象

局部自动对象常见实现是在栈帧中分配：

```
void f()
```

```
{
    int x = 1;
    int y = 2;
}
```

未优化时，编译器可能给 `x` 和 `y` 分配栈槽。优化后，如果它们不需要真实地址，可能只存在于寄存器甚至常量传播结果中。

堆对象

动态对象常由分配器管理：

```
auto* p = new int[1024];
delete[] p;
```

堆分配有额外成本：分配器元数据、锁或线程缓存、对齐、碎片、page fault、NUMA 位置等。性能关键路径中频繁分配小对象通常很危险。

静态对象

全局变量和静态变量通常在可执行文件或共享库的段中描述，并在进程加载时映射：

```
int global_counter = 0;

int f()
{
    static int local_static = 1;
    return ++local_static;
}
```

静态对象会牵涉链接、加载、初始化顺序、线程安全局部静态初始化等问题。本书后面讲 ELF、链接和运行时时会展开。

对齐：对象地址不是随便放的

alignment（对齐）是对象地址必须满足的约束。若一个类型 `T` 的对齐要求是 `A`，则 `T` 对象地址通常必须是 `A` 的倍数。

例如在常见 x86-64 ABI 下：

类型	常见大小	常见对齐
<code>char</code>	1	1
<code>int</code>	4	4
<code>float</code>	4	4
<code>double</code>	8	8
<code>void*</code>	8	8

你可以用 `alignof` 查询：

```
#include <iostream>

int main()
{
    std::cout << alignof(char) << '\n';
    std::cout << alignof(int) << '\n';
    std::cout << alignof(double) << '\n';
    std::cout << alignof(void*) << '\n';
}
```

为什么需要对齐

对齐来自硬件和 ABI 的共同需要。硬件访问自然对齐的数据通常更简单、更快。有些架构上未对齐访问会触发异常；x86-64 通常允许许多未对齐访问，但可能带来性能代价，尤其是跨 cache line、跨页或用于某些 SIMD 指令时。

例如，一个 8 字节 load 如果地址是 8 字节对齐，且不跨 cache line，硬件处理更直接。如果它从 cache line 最后 4 字节开始，就会跨两个 cache line，可能需要两个 cache line 都可用。

深入理解

“x86 支持未对齐访问”不等于“对齐不重要”。标量普通 load/store 可能只是轻微变慢，但跨 cache line、跨 page、原子操作、SIMD 对齐指令、false sharing 场景下，对齐会变成明显性能因素。

对齐要求、分配器保证和过度对齐

对齐不是只在结构体内部发生。任何对象被创建时，它的起始地址都必须满足该类型的对齐要求。局部对象、全局对象、new 创建的对象和标准容器中的元素，都要满足各自类型的对齐要求。若一个分配器返回的地址不能满足对象类型对齐，随后构造对象或访问对象就可能违反语言规则。

普通 operator new 会返回适合常规对象的对齐地址；对 alignas(64) 这类过度对齐类型，现代 C++ 有对应的对齐分配机制。标准容器在保存过度对齐类型时也应通过 allocator 提供正确对齐。自己写 arena、memory pool 或固定缓冲区时，必须显式处理这件事。

例如：

```
struct alignas(64) CacheLineBlock {
    std::uint64_t data[8];
};
```

这里 alignas(64) 要求对象起始地址按 64 字节对齐。这个要求可能用于避免 false sharing、配合 cache line 边界、或满足某些向量化数据布局。但它也会增大数组中相邻元素的步长，可能浪费空间。对齐越大不一定越快；它改变的是布局约束，是否提升性能要看访问模式和硬件行为。

自然对齐、cache line 对齐和 SIMD 对齐不要混用

“对齐”这个词在不同场景里有不同粒度：

- **类型自然对齐**：例如 double 通常要求 8 字节对齐。
- **cache line 对齐**：常见 x86-64 cache line 是 64 字节，用于减少跨行访问或隔离多线程写入。
- **SIMD 数据对齐**：128-bit、256-bit、512-bit load/store 可能希望数据按 16、32、64 字节边界组织。
- **页面对齐**：操作系统页通常是 4 KiB 或更大，用于虚拟内存映射、mmap、大页和 DMA 等场景。

这些层次有关联，但不能互相替代。一个地址 8 字节对齐，不代表它 64 字节对齐；一个 buffer 64 字节对齐，也不代表每个结构体字段都避免跨 cache line；一个数组首地址满足 SIMD 对齐，也不代表从任意下标开始的子数组仍然满足同样对齐。

写报告时要说清“对齐到多少字节、为了哪一层问题”。例如“这个 double 字段放在 offset 8，是为了满足字段 8 字节对齐”；“这个任务描述符用 alignas(64)，是为了让不同线程写的计数器落在不同 cache line”；“这个 microkernel 的 packed buffer 按 64 字节对齐，是为了方便宽向量 load 和预取”。三句话都讲对齐，但工程目的不同。

常见误区

不要把对齐当成泛泛的性能魔法。对齐首先是正确性和 ABI 布局规则，其次才是性能工具。过度对齐会增加空间占用，可能降低 cache 密度；是否值得，要用访问模式和测量证明。

结构体布局：字段、padding 和 ABI

结构体不是简单把字段大小相加。编译器需要为每个字段选择 offset，并在必要时插入 padding（填充字节），以满足字段和整个结构体的对齐要求。

先掌握一个机械算法

学习结构体布局时，不要一上来凭感觉。对普通标准布局结构体，可以先用下面这个机械算法训练。真实 ABI 有更多细节，例如继承、虚函数、空基类优化、位域、向量类型、显式 `alignas`、`packing pragma`，但本算法足够支撑本章和大量性能分析。

1. 令当前 `offset` 为 0。
2. 从第一个字段开始，按声明顺序处理。
3. 对当前字段，先查出字段大小 `sizeof(field)` 和对齐 `alignof(field)`。
4. 如果当前 `offset` 不是字段对齐的倍数，就插入 `padding`，把 `offset` 向上补齐到下一个满足对齐的位置。
5. 当前 `offset` 就是该字段的 `offset`。
6. `offset` 增加字段大小。
7. 所有字段处理完后，结构体整体对齐通常是所有字段对齐要求的最大值。
8. 最终 `sizeof(struct)` 必须补齐到结构体整体对齐的倍数，因此末尾可能有 `tail padding`。

其中“向上补齐”可以写成一个函数：

$$\text{align_up}(x, A) = \text{smallest } y \text{ such that } y \geq x \text{ and } y \% A == 0$$

如果 A 是 2 的幂，机器代码和系统代码里经常写成：

```
std::size_t align_up(std::size_t x, std::size_t a)
{
    return (x + a - 1) & ~(a - 1);
}
```

例如 `align_up(5,4)==8`, `align_up(16,8)==16`。第一个例子需要补 3 字节，第二个例子本来已经对齐，不需要补。

核心思想

`padding` 不是编译器“随便浪费空间”。`padding` 是为了让每个字段和数组中每个结构体元素都满足对齐约束。结构体大小必须包含尾部 `padding`，否则 `arr[1]` 的起始地址可能不满足结构体自身对齐。

看第一个例子：

```
struct A {
    char c;
    int x;
};
```

在常见 x86-64 ABI 下，`char` 大小 1、对齐 1；`int` 大小 4、对齐 4。若 `c` 放在 `offset 0`，那么 `x` 不能放在 `offset 1`，因为 `offset 1` 不是 4 的倍数。编译器会插入 3 字节 `padding`：

offset	内容	大小	说明
0	c	1	字段
1..3	padding	3	让下个字段 4 字节对齐
4..7	x	4	字段

所以 `sizeof(A)` 通常是 8，不是 5。

完整例题：逐字段模拟布局

现在用算法推一个稍复杂的例子：

```
struct Record {
    char tag;
    double score;
    int count;
```

```
short flags;
};
```

假设常见 x86-64 ABI 下：

类型	大小	对齐
char	1	1
short	2	2
int	4	4
double	8	8

逐字段推导如下：

字段	进入前 offset	对齐动作	字段 offset	写入后 offset
tag	0	已满足 1 对齐	0	1
score	1	补到 8, 插入 7 字节 padding	8	16
count	16	已满足 4 对齐	16	20
flags	20	已满足 2 对齐	20	22

字段处理完后，最大对齐是 8，所以结构体大小必须补到 8 的倍数。当前 offset 是 22，`align_up(22,8)==24`，因此尾部 padding 是 2 字节：

字节范围	内容	大小	说明
0	tag	1	字段
1..7	padding	7	让 score 8 字节对齐
8..15	score	8	字段
16..19	count	4	字段
20..21	flags	2	字段
22..23	tail padding	2	让 sizeof(Record) 是 8 的倍数

如果有数组：

```
Record r[2];
```

那么：

```
addr(r[1]) = addr(r[0]) + sizeof(Record)
            = addr(r[0]) + 24
```

24 是 8 的倍数，因此 `r[1].score` 仍然可以落在 8 字节对齐地址上。如果结构体大小只是字段结束时的 22，数组第二个元素就会破坏字段对齐，这就是尾部 padding 存在的根本原因。

把字段重排也算法化

如果只是追求更小的结构体大小，一个常见启发式是把对齐要求大的字段放前面：

```
struct RecordPackedOrder {
    double score;
    int count;
    short flags;
    char tag;
};
```

逐字段推导：

字段	进入前 offset	对齐动作	字段 offset	写入后 offset
score	0	已满足 8 对齐	0	8
count	8	已满足 4 对齐	8	12
flags	12	已满足 2 对齐	12	14
tag	14	已满足 1 对齐	14	15

最大对齐仍然是 8，最终大小补到 16。和原来的 24 相比，减少了 8 字节。如果这种对象有一千万个，节省的是约 80 MB，不是小数。

但这仍然不是“所有结构体都按大到小排序”的命令。性能结构体设计要同时考虑三类目标：

- **空间密度**：减少 padding，让更多对象进入 cache。
- **访问局部性**：把同一个 hot loop 经常一起使用的字段放近，把冷字段挪远。
- **并发隔离**：多线程频繁写的字段有时要故意分开，避免 false sharing。

字段顺序是数据布局设计的一部分，不只是“省几个字节”。对于 AI runtime 的 tensor metadata、operator descriptor、thread task、work queue node，字段布局会影响 cache 占用、预取效果、false sharing 和 ABI 稳定性。

尾部 padding

再看：

```
struct B {
    int x;
    char c;
};
```

x 放在 offset 0，占 4 字节；c 放在 offset 4，占 1 字节。看起来总共 5 字节。但结构体整体对齐是其成员最大对齐，通常是 4。为了让 `Barr[2]` 中每个元素都满足对齐，`sizeof(B)` 必须是 4 的倍数，于是尾部会有 3 字节 padding：

offset	内容	大小	说明
0..3	x	4	字段
4	c	1	字段
5..7	tail padding	3	让数组中下个元素对齐

字段顺序会影响大小

比较：

```
struct Bad {
    char a;
    double b;
    char c;
    int d;
};

struct Better {
    double b;
    int d;
    char a;
    char c;
};
```

Bad 可能因为多次对齐插入大量 padding；**Better** 把大对齐字段放前面，通常更紧凑。但字段重排不是无脑优化。你还要考虑：

- **ABI 兼容**：公开结构体布局不能随便改。
- **序列化和文件格式**：布局变化会破坏二进制兼容。
- **热字段和冷字段**：有时为了 cache，把常用字段聚在一起比最小 `sizeof` 更重要。
- **false sharing**：多线程写的字段可能需要故意分离到不同 cache line。

字段布局设计的三步判断

真实工程中设计结构体字段顺序，可以按三步判断，而不是只按字段大小排序。

第一步，判断结构体是否跨 ABI 或持久化边界。如果这个结构体出现在公开头文件、动态库接口、文件格式、网络协议、共享内存、插件接口或跨语言 FFI 中，字段顺序就是契约的一部分。为了省几个 padding 字节随意重排，可能让旧二进制、旧文件或其他语言绑定全部失效。此时应优先使用显式版本、大小字段、不透明句柄或稳定序列化格式，而不是把内部结构体裸露出去。

第二步，判断访问模式。若热循环每次只读少数字段，应把热字段聚在一起，甚至考虑 SoA；若每次都完整消费一个对象，紧凑 AoS 可能更好；若某些字段只在错误处理、日志、调试或初始化阶段使用，它们是冷字段，可以与热路径字段分开。空间最小的布局不一定让热路径最快，因为 cache line 搬运的是连续字节，热字段密度比总大小更贴近性能。

第三步，判断并发写入。多线程程序中，如果两个线程频繁写同一 cache line 上的不同字段，会发生 false sharing。为了避免这种情况，有时要故意插入 padding 或使用 `alignas(64)`，让不同线程写的字段分开。这个选择会增大对象大小，但可能显著降低缓存一致性流量。

因此，字段布局设计至少要写出下面四项证据：

1. 每个字段的大小、对齐和 offset。
2. 热路径实际访问哪些字段，访问频率如何。
3. 结构体是否是公开 ABI、文件格式或共享内存布局。
4. 是否有多线程频繁写入同一对象或相邻对象的风险。

这四项比“把大的放前面”更接近工程现实。对于教材练习，先学会手算 padding；对于生产代码，要把 padding 分析放进访问模式和兼容性分析里。

布局改变会改变汇编形态

字段 offset 会直接进入汇编指令的 displacement。结构体大小会直接进入数组访问的 scale、`shl` 或 `lea`。因此字段重排不只是 `sizeof` 变化，也会改变反汇编中出现的常量。

例如一个字段从 offset 16 移到 offset 8，原来的访问可能是：

```
mov eax, dword ptr [rdi + rsi*4 + 16]
```

重排后可能变成：

```
mov eax, dword ptr [rdi + rsi*4 + 8]
```

如果结构体大小从 24 变成 16，数组字段访问的步长也会改变。原本可能需要：

```
lea rcx, [rsi + rsi*2]      ; i * 3
mov eax, dword ptr [rdi + rcx*8 + 20]
```

变成 16 字节步长后，可能只需要：

```
shl rsi, 4
mov eax, dword ptr [rdi + rsi + 12]
```

这类差异会影响代码尺寸、地址计算指令、寄存器压力和 cache 密度。并不是说某种形态一定更快，而是说明布局设计会被编译器翻译成具体地址计算。读优化报告时，如果只看源代码字段顺序，不看反汇编中的 offset 和步长，就少了一半证据。

sizeof、alignof 和 offsetof

研究布局时，三个工具非常重要：

```
#include <cstddef>
#include <iostream>

struct S {
    char a;
    double b;
    int c;
};
```



```
int main()
{
    std::cout << "sizeof(S)  = " << sizeof(S) << '\n';
    std::cout << "alignof(S) = " << alignof(S) << '\n';
    std::cout << "offset a  = " << offsetof(S, a) << '\n';
    std::cout << "offset b  = " << offsetof(S, b) << '\n';
    std::cout << "offset c  = " << offsetof(S, c) << '\n';
}
```

`offsetof` 对标准布局类型可靠。学习阶段你应该先手工预测，再用程序验证。不要只看输出，因为真正的训练是形成布局推导能力。

padding 字节不一定有稳定值

结构体 padding 是对象表示的一部分，但不一定参与对象的值。它可能未初始化，也可能包含旧栈数据。比较结构体对象时，不能简单用 `memcmp` 替代字节级比较，除非你非常清楚类型性质和初始化方式。

例如：

```
struct A {
    char c;
    int x;
};

bool equal_bad(A const& lhs, A const& rhs)
{
    return std::memcmp(&lhs, &rhs, sizeof(A)) == 0;
}
```

即使两个对象的 `c` 和 `x` 相等，padding 字节也可能不同，`memcmp` 可能返回不相等。这个问题在哈希、序列化、网络传输和 SIMD 批量比较中很常见。

常见误区

结构体的“值”和结构体的“对象字节”不是总能一一对应。padding、未初始化字节、浮点 NaN、不同但等价的表示，都可能让字节级比较和语义级比较不一致。

安全地观察和搬运字节

用 `unsignedchar` 或 `std::byte` 观察对象表示

观察对象字节时，可以使用 `unsignedcharconst*`：

```
template <class T>
void dump_bytes(T const& value)
{
    auto const* p = reinterpret_cast<unsigned char const*>(&value);
    for (std::size_t i = 0; i < sizeof(T); ++i) {
        std::cout << std::hex << static_cast<unsigned>(p[i]) << ' ';
    }
    std::cout << '\n';
}
```

用 `std::memcpy` 或 `std::bit_cast` 搬运位模式

如果你想在两种类型之间复制位模式，不要随使用错误类型指针解引用。C++20 提供 `std::bit_cast`：

```
#include <bit>
#include <stdint>

float f = 1.0f;
std::uint32_t bits = std::bit_cast<std::uint32_t>(f);
```

也可以用 `std::memcpy`：

```
std::uint32_t bits;
std::memcpy(&bits, &f, sizeof(bits));
```

这两种方式都表达“复制对象表示”，比用不匹配类型的指针解引用更可靠。

别名规则：编译器如何知道两个指针会不会指同一块内存

别名问题是从 C++ 到高性能优化的关键门槛。看下面代码：

```
void f(int* a, int* b)
{
    *a = 1;
    *b = 2;
    *a = *a + 3;
}
```

如果 `a` 和 `b` 指向不同对象，最后 `*a==4`。如果它们指向同一个 `int`，执行过程是：

1. 执行 `*a=1`，该对象变成 1。
2. 执行 `*b=2`。因为 `a` 和 `b` 指向同一对象，所以 `*a` 也变成 2。
3. 执行 `*a=*a+3`，最终该对象变成 5。

编译器如果不能证明 `a` 和 `b` 不别名，就必须保守处理。保守处理会影响寄存器缓存、load/store 重排、循环向量化和 common subexpression elimination。

不同类型通常提供别名信息

看另一个例子：

```
void g(int* a, double* b)
{
    *a = 1;
    *b = 2.0;
    *a = *a + 3;
}
```

在正常 C++ 别名规则下，`int*` 和 `double*` 通常不能指向同一个活着的对象。编译器可以利用这个事实，把 `*a` 保存在寄存器中，而不必担心 `*b=2.0` 改了同一个 `int`。

这就是为什么错误的类型双关不仅可能不正确，还可能导致编译器生成你无法预期的代码。

别名如何阻止优化

别名不是抽象语言洁癖，它会直接改变循环汇编。考虑：

```
void scale_add(float* y, float const* x, float a, std::size_t n)
{
    for (std::size_t i = 0; i < n; ++i) {
        y[i] = y[i] + a * x[i];
    }
}
```

如果编译器不知道 `x` 和 `y` 是否重叠，就必须考虑很多情况：`x==y`，`x` 指向 `y` 中间，`y` 指向 `x` 中间，或者二者完全不重叠。对于标量循环，这可能只是多几条保守 load/store；对于向量化，情况会更复杂，因为一次向量 store 可能覆盖后续向量 load 还没读取的数据。

编译器常见策略有三种。第一，完全保守，不做某些重排或向量化。第二，生成运行时别名检查：先判断两个区间是否重叠，不重叠走快速向量路径，可能重叠走保守路径。第三，利用源码给出的额外信息，例如不同类型、`restrict` 扩展、编译器内建假设、`span` 的不可重叠约定或更高层算法保证。

这就是为什么同样的数学循环，在 C、C++、Fortran、Rust 或手写汇编中可能得到不同优化结果。不是硬件变了，而是编译器掌握的别名事实不同。Fortran 的数组语义长期对数值优化友好，一个重要原因就是别名假设更强；C/C++ 则需要程序员和接口设计主动表达不重叠关系。

运行时别名检查不是坏事

看到优化后汇编里多了一段指针范围比较，不要立刻认为编译器“变复杂了”。它可能是在做 loop versioning：用少量入口检查换取主循环的向量化。

概念上，编译器可能生成类似逻辑：

```
if ranges_do_not_overlap(x, y, n):
    run vectorized loop
else:
    run scalar conservative loop
```

这段检查本身有成本，但如果 n 足够大，向量化主循环节省的成本远大于入口判断。相反，如果 n 很小，检查成本可能显得明显。后面做 benchmark 时，你要知道：性能曲线里小尺寸和大尺寸表现不同，可能就来自这种版本化、边界处理和 remainder loop。

表达不别名关系的工程方式

在 C++ 中表达“不别名”要谨慎。常见方式包括：

- 通过 API 设计避免同一个 buffer 同时作为输入和输出。
- 在文档和断言中明确输入输出区间不重叠。
- 使用编译器支持的 `__restrict__` 或相关属性，但要保证调用者真的遵守。
- 将只读输入标成 `Tconst*`，虽然 `const` 本身不证明不别名，但能表达写入方向。
- 在高层容器或 tensor view 中记录区间、shape、stride，并在调试构建检查重叠。

最危险的做法是为了让编译器优化而撒谎。例如把可能重叠的指针声明成 `restrict` 风格，然后在调用时真的传入重叠区间。这样得到的错误往往只在某些优化级别、某些长度或某些 CPU 上出现，调试成本很高。底层优化依赖契约，契约必须真实。

深入理解

C 的 `restrict`、C++ 编译器扩展中的 `__restrict__`、LLVM IR 的 `alias metadata`、Fortran 的数组别名语义，都会显著影响高性能循环能否向量化。后面讲 auto-vectorization 和 AI kernel 时会深入。

常见混淆：底层学习最容易卡住的地方

本章内容多，但很多错误可以归结为几组概念混淆。这里集中整理一次。你后面看汇编、写 SIMD、调 benchmark 时，只要结果不符合预期，就应该回到这些问题检查。

混淆 1：指针变量和被指对象不是同一个东西

看下面代码：

```
int x = 42;
int* p = &x;
```

x 是一个 `int` 对象，通常占 4 字节。 p 也是一个对象，它的类型是 `int*`，在 x86-64 上通常占 8 字节。 p 的值是 x 的地址，但 p 自己也需要存储空间。

如果假设：

```
addr(x) = 0x1000
addr(p) = 0x2000
value(p) = 0x1000
```

那么：

- 访问 x ，读取的是 `0x1000..0x1003`。
- 访问 p 这个指针变量，读取的是 `0x2000..0x2007`。
- 访问 `*p`，先从 p 的值拿到 `0x1000`，再读取 `0x1000..0x1003`。

汇编里也会体现这个差异。如果 `rdi` 直接保存 p 的值，也就是 x 的地址，那么：

```
mov eax, dword ptr [rdi]
```

读取的是 `*p`。但如果 `rdi` 保存的是指针变量 `p` 自己的地址，也就是 `&p`，那么通常需要两次 load：

```
mov rax, qword ptr [rdi]    ; load p itself
mov eax, dword ptr [rax]    ; load *p
```

第一条读取指针值，第二条读取被指对象。链表、树、稀疏矩阵和 `int**` 的性能问题，很多都来自这种“先读地址，再按地址读数据”的间接访问。

混淆 2：元素偏移和字节偏移不是同一个单位

下面两行很像，但单位不同：

```
int* q1 = p + 3;
auto q2 = reinterpret_cast<char*>(p) + 3;
```

如果 `p==0x1000`，则：

```
q1 = 0x1000 + 3 * sizeof(int) = 0x100c
q2 = 0x1000 + 3              = 0x1003
```

`p+3` 的 3 是元素个数；`reinterpret_cast<char*>(p)+3` 的 3 是字节数。读汇编时，所有地址最终都要落到字节偏移；读 C++ 时，指针加法先按类型缩放。

混淆 3：数组对象、数组首元素指针、指向整个数组的指针

看代码：

```
int a[4];
int* p0 = a;
int (*p1)[4] = &a;
```

它们的数值地址可能相同，但类型不同：

表达式	类型	数值上常见地址	+1 的含义
<code>a</code>	表达式中常退化为 <code>int*</code>	<code>addr(a[0])</code>	下一个 <code>int</code>
<code>&a[0]</code>	<code>int*</code>	<code>addr(a[0])</code>	下一个 <code>int</code>
<code>&a</code>	<code>int(*)[4]</code>	<code>addr(a)</code>	下一个 4 元素数组

因此：

```
addr((&a[0]) + 1) = base + 4
addr((&a) + 1) = base + 16
```

这类区别在多维数组、固定 shape 张量、小矩阵 tile、编译期已知长度的 kernel 中非常重要。编译器知道更多类型信息时，往往能生成更直接的地址计算和更强的优化。

混淆 4：硬件能执行不代表 C++ 定义良好

很多初学者会用“我运行没崩”来判断底层代码对不对。这在系统编程和性能优化里非常危险。看下面例子：

```
int a[4] = {1, 2, 3, 4};
int x = a[4];
```

硬件层面，这可能只是从 `a[0]` 后面 16 字节的位置发起一次 4 字节 load。但 C++ 层面，`a[4]` 越界，行为未定义。编译器可以假设这种情况不会发生，并据此重排、删除或改写代码。

再看类型双关：

```
int x = 0;
double* p = reinterpret_cast<double*>(&x);
double y = *p;
```

硬件也许能从 `&x` 开始读 8 字节，但 C++ 对象模型、对齐和别名规则都可能已经被破坏。正确观察位模式时，应优先使用：

```
std::bit_cast<T>(value)
std::memcpy(&dst, &src, sizeof(dst))
unsigned char const* bytes = reinterpret_cast<unsigned char const*>(&obj)
```

底层工程的目标不是“骗过编译器和硬件”，而是在语言规则允许的范围内，把编译器、ISA 和微架构都用到极限。

混淆 5：汇编里的地址公式不等于源码里的唯一写法

同一个源码可能生成多种汇编，同一个汇编片段也可能来自多种源码。比如：

```
mov eax, dword ptr [rdi + rsi*4 + 8]
```

它可能来自：

```
return p[i + 2];
```

也可能来自：

```
struct S {
    int a;
    int b;
    int c;
};

return p[i].c;
```

区别要靠上下文判断：`rdi` 是不是数组首地址？`rsi` 是不是原始下标？结构体大小是否为 12？前面是否有 `lea` 改变了 index？函数参数 ABI、调试信息、周围 load/store 和循环结构都要一起看。

心智模型

读汇编不是把每条指令机械翻译回一行 C++。更可靠的方法是先恢复数据流和地址公式，再结合类型、ABI、循环结构和访问宽度推断源码意图。

对象生命周期：有地址不等于有对象

底层程序员容易犯的另一个错误是把“有一块字节存储”当成“有一个对象”。C++ 区分存储和对象生命周期。

例如，分配一块原始存储：

```
alignas(int) unsigned char buffer[sizeof(int)];
```

这块字节存在，但其中还没有一个 `int` 对象。要在其中创建对象，需要 placement new 或符合标准的对象创建方式：

```
int* p = new (buffer) int(42);
```

在高性能代码中，自定义 allocator、memory pool、arena、SIMD buffer 和序列化反序列化都会碰到对象生命周期问题。初学阶段不要求你立刻掌握所有细节，但必须建立警觉：地址、字节、对象不是同一个概念。

存储、对象和值的三层关系

可以把底层内存问题分成三层：

1. **存储**：有一段足够大、足够对齐的字节空间。
2. **对象**：在这段存储中，一个特定类型对象的生命周期已经开始。
3. **值**：这个对象已经被初始化到某个可读取的状态。

这三层不能跳。分配器返回一段存储，不等于其中已经有 `T` 对象；对象生命周期开始了，不等

于每个值都已经初始化；对象值可读，也不等于它的 padding 字节有稳定意义。

例如：

```
alignas(double) std::byte storage[sizeof(double)];
```

这行代码只给出一段按 `double` 对齐、大小足够的字节存储。若要在其中创建 `double`，需要构造对象；若要结束对象生命周期，需要调用合适的析构过程。对 `double` 这种平凡类型，析构看起来没有动作，但“生命周期是否存在”这个语言事实仍然影响你能否通过 `double*` 合法访问它。

为什么 allocator 和 arena 要关心生命周期

许多高性能系统会把内存分配拆成两步：先批量拿一大块原始存储，再在其中按需构造对象。这样可以减少系统调用、减少分配器锁、改善局部性，并让对象集中在少数页里。但是这种做法把责任交给了库作者：必须保证对齐正确、对象构造正确、异常时清理正确、析构顺序正确、释放时不再有悬垂引用。

一个 arena 分配器如果只返回 `void*` 或 `std::byte*`，调用者还要把它变成某个类型对象。正确抽象通常会提供类似“分配并构造 T”的接口，而不是让业务代码到处手写 placement new。否则很容易出现：地址对了但对象没构造、对象构造了但析构没执行、存储复用了但旧指针仍被使用。

这也是为什么底层性能库虽然追求速度，反而更需要清楚的对象模型。越是接近内存池、tensor buffer、operator workspace、JIT code cache 这类区域，越不能把“我拿到一段地址”当作全部正确性证明。

复用存储时旧指针不一定还能用

当一段存储中的对象生命周期结束，并在同一地址构造另一个对象时，旧指针、旧引用和旧类型假设可能失效。即使数值地址相同，语言层对象已经换了。某些场景需要使用 `std::launder` 或重新获取指针，具体规则较细，本册不展开；你现在要记住底线：地址相同不代表对象连续相同。

这个问题在对象池中很常见：

```
// conceptual only
T* p = pool.create<T>(...);
pool.destroy(p);
U* q = pool.create<U>(...); // may reuse the same address
```

如果后来还用旧的 `p` 访问，就不是“访问同一地址上的新对象”这么简单，而是使用悬垂指针。工具上，AddressSanitizer、UBSan、调试 allocator、对象生命周期分析都可以帮助发现这类错误；工程上，所有权和句柄设计要尽量避免旧指针长期流散。

常见误区

在 C++ 中，合法访问对象需要同时满足：地址落在有效存储内、对齐满足、对象生命周期已经开始、访问类型允许、访问范围不越界。缺一项，都不能只凭“硬件能读写”证明程序正确。

数据布局如何进入性能：cache line 预告

CPU 不是每次只从内存拿你请求的那几个字节。缓存层级通常以 cache line 为单位搬运数据。现代 x86-64 常见 cache line 大小是 64 字节。

如果你顺序遍历 `int` 数组：

```
for (std::size_t i = 0; i < n; ++i) {
    s += a[i];
}
```

每个 `int` 4 字节，一个 64 字节 cache line 可以容纳 16 个 `int`。当 CPU 加载 `a[i]` 所在 cache line 后，后面的 `a[i+1]` 到 `a[i+15]` 很可能已经在 cache 中。这叫空间局部性。

如果你随机访问：

```
for (std::size_t i = 0; i < n; ++i) {
    s += a[index[i]];
}
```

每次访问可能落在不同 cache line。即使只需要 4 字节，也可能为它搬来 64 字节，造成带宽浪费和高延迟。

步长访问

步长访问是理解 cache 的第一个实验：

```
for (std::size_t i = 0; i < n; i += stride) {
    s += a[i];
}
```

当 `stride==1` 时，空间局部性最好。当 `stride` 很大时，每次访问都可能使用一个新 cache line。访问元素数看似减少了，但每个访问的 cache line 利用率也下降了。

cache line 利用率的粗算

第一册先用一个粗略但非常有用的模型：如果 cache line 是 64 字节，而每次只真正使用其中 4 字节，那么理论上每条 cache line 的有效利用率只有 $4/64 = 1/16$ 。这不是精确性能公式，但能快速解释很多“运算量很少却很慢”的循环。

例如顺序读取 `float` 数组：

```
float a[n]
```

每个元素 4 字节，连续访问时一条 64 字节 cache line 可以提供 16 个元素。若循环确实用到这 16 个元素，带宽利用较好。若你每隔 16 个元素读一个：

```
for (std::size_t i = 0; i < n; i += 16) {
    s += a[i];
}
```

每次访问都可能落在新的 cache line，只用其中 4 字节，剩下 60 字节被搬来却不用。这个循环元素访问次数少了，但单位有效数据需要搬运的 cache line 更多，未必按访问次数线性变快。

结构体字段也是同理。若 `Particle` 是 32 字节，而循环只读其中一个 4 字节字段，一条 64 字节 cache line 只能容纳两个 `Particle`，有效使用 8 字节，利用率约八分之一。换成 SoA 后，所有目标字段连续存放，同一条 cache line 能提供 16 个目标字段。这就是布局选择和 cache 带宽之间的第一层关系。

局部性有空间局部性和时间局部性

空间局部性是“用到一个地址后，很快会用到附近地址”。连续数组遍历、结构体内相邻字段读取、按行访问矩阵，通常空间局部性较好。时间局部性是“用过的数据很快会再次使用”。小工作集反复迭代、循环内复用某个标量、矩阵块被多次参与计算，通常时间局部性较好。

许多高性能算法的布局设计，就是在这两类局部性之间取平衡。GEMM blocking 让一个矩阵块在 cache 中被多次复用，是时间局部性；packing 把原本跨步访问的数据复制成连续 buffer，是空间局部性；SoA 把同一字段聚到连续数组，是提高单字段循环的空间局部性；hot/cold split 把热字段从大对象里分离出来，是提高热路径 cache 密度。

本章不要求你会设计完整 cache blocking 算法，但要能从最简单的字节地址公式看出：循环每次访问是否连续、每条 cache line 有多少有效字节、某个字段是否被冷字段稀释、某个指针链是否让下一次地址依赖上一次 load。后续所有性能模型都会从这几个问题展开。

AoS 与 SoA：布局决定 SIMD 和 cache 效率

看一个粒子数组：

```
struct Particle {
    float x;
    float y;
    float z;
    float mass;
};
```

```
Particle* particles;
```

这是 **AoS** (Array of Structs): 数组中的每个元素是一个结构体。如果只计算所有 **x** 的和:

```
float sx = 0.0f;
for (std::size_t i = 0; i < n; ++i) {
    sx += particles[i].x;
}
```

内存中 **x** 被 **y**、**z**、**mass** 间隔开。每次 cache line 搬入很多不需要的字段。
另一种布局是 **SoA** (Struct of Arrays):

```
struct Particles {
    float* x;
    float* y;
    float* z;
    float* mass;
};
```

此时所有 **x** 连续存放。对只读 **x** 的 kernel, SoA 更利于 cache 和 SIMD。
但如果每次都同时使用 **x, y, z, mass**, AoS 可能更直观, 也可能有不错局部性。真正的布局选择必须围绕访问模式, 而不是口号。

AoS, SoA 和 AoSoA

除了 AoS 和 SoA, 真实高性能库中还常见 **AoSoA** (Array of Structs of Arrays)。它把对象按小块分组, 每个块内部用 SoA, 块与块之间形成数组:

```
struct ParticleBlock8 {
    float x[8];
    float y[8];
    float z[8];
    float mass[8];
};

ParticleBlock8* blocks;
```

AoSoA 的动机是把 SIMD 宽度、cache line 和对象语义折中起来。若向量宽度一次处理 8 个 float, 块内 **x[8]** 正好连续; 同时一个 block 又保留了“8 个粒子的一组字段”这个局部结构。很多物理模拟、图形、推理 runtime 和 tensor kernel 都会出现类似分块布局。

布局选择可以按访问模式粗分:

布局	适合访问模式	主要代价
AoS	每次消费一个对象的大多数字段	单字段扫描浪费 cache line, SIMD gather/permute 可能多
SoA	批量处理同一字段或同一属性	对象整体搬运不直观, 接口和所有权更复杂
AoSoA	固定向量宽度或 tile 内批量处理	需要处理块大小、尾部元素和布局转换

这张表不是绝对规则。真正判断要看: 一次循环访问哪些字段, 是否顺序访问, 是否需要 SIMD, 是否多线程写入, 是否跨 ABI, 是否要和外部库共享数据。教材训练的目标, 是让你能从地址公式和 cache line 利用率出发解释布局选择, 而不是背“布局 A 一定比布局 B 快”。

核心思想

数据布局不是“代码风格”, 而是性能模型的一部分。一个 kernel 的访问模式决定它需要什么布局; 布局决定 cache line 利用率、SIMD load/store 形态、预取效果和多线程 false sharing 风险。

从布局回到汇编：offset 和 scale 是布局的影子

读反汇编时，不要只盯着某一条 `mov`。你要把它前面的地址计算指令也一起看。下面几组模式非常常见。

模式 1：纯数组，scale 直接等于元素大小

当你看到：

```
movss xmm0, dword ptr [rdi + rax*4]
```

它很可能表示读取一个连续 `float` 数组：

```
float x = arr[i];
```

解释如下：

汇编部分	含义	可能来源
<code>rdi</code>	基地址	<code>arr</code>
<code>rax</code>	下标	<code>i</code>
<code>*4</code>	元素大小	<code>sizeof(float)</code>
<code>dwordptr</code>	访问 4 字节	<code>float load</code>

模式 2：结构体数组字段，先算大步长

如果结构体大小是 16，x86 内存操作数不能直接写 `rax*16`。真实汇编更可能是：

```
shl rax, 4
movss xmm0, dword ptr [rdi + rax + 4]
```

它可能表示：

```
struct Particle {
    float x;
    float y;
    float z;
    float mass;
};

float y = particles[i].y;
```

推导过程是：

```
sizeof(Particle)    = 16
offset(Particle, y) = 4
byte_offset_for_i   = i * 16
addr(particles[i].y) = base + i * 16 + 4
```

所以 `shl rax, 4` 把元素下标变成字节偏移，`+4` 是字段 `y` 的 `offset`。

模式 3：用 `lea` 拼出非 2 的幂步长

当结构体大小是 24 时，常见组合是：

```
lea rcx, [rax + rax*2]
mov edx, dword ptr [rdi + rcx*8 + 20]
```

这可能表示：

```
struct Item24 {
    double a;
    double b;
    int c;
    int d;
};

int d = items[i].d;
```

因为：

```
rcx = rax + rax * 2 = i * 3
effective_address = base + (i * 3) * 8 + 20
                  = base + i * 24 + 20
```

20 是字段 d 的 offset。这个例子训练的是：不要看到最后一条里只有 rcx*8 就误判结构体大小是 8；rcx 本身已经被前一条 lea 变成了 i*3。

模式 4：二维连续数组，行长进入地址公式

如果源码是：

```
int get_2d(int const a[3][5], std::size_t i, std::size_t j)
{
    return a[i][j];
}
```

一种可能汇编形态是：

```
lea eax, [rsi + rsi*4]
add rax, rdx
mov eax, dword ptr [rdi + rax*4]
ret
```

其中：

```
rax = i + i * 4 = i * 5
rax = i * 5 + j
address = base + (i * 5 + j) * 4
```

这说明每行 5 个 int 这个类型信息已经进入机器地址计算。如果是 int**，则通常会先 load 行指针，再访问行内元素，汇编形态完全不同。

布局审查：同时看正确性、ABI 和性能

数据布局不是单纯的内存知识。一个结构体的字段顺序、对齐、padding、大小和访问模式，会同时影响正确性、二进制接口和性能。第一册不要求你立即设计复杂数据结构，但要学会做布局审查。布局审查不是“看看 sizeof 多大”，而是把源码类型、C++ 对象规则、ABI 传参、汇编地址公式和访问模式放在同一张表里。

一个布局审查表可以包含：

审查项	要问的问题	常用证据
字段 offset	每个字段从对象起点偏移多少	offsetof、调试输出、反汇编 displacement
对象大小	数组步长是多少，尾部 padding 多少	sizeof、地址打印、scale/LEA 公式
对齐	对象和字段需要什么边界	alignof、分配器保证、指令宽度
生命周期	访问时对象是否真实存在	源码构造/析构、allocator 逻辑
别名	两个指针是否可能指向同一区域	函数契约、类型、运行时检查
ABI	类型如何传参和返回	函数签名、ABI 规则、反汇编
性能	访问是否连续、跨步、随机或浪费 cache line	地址公式、输入规模、benchmark

例如一个结构体：

```
struct Particle {
    float x;
    float y;
}
```



```
float z;
int id;
bool active;
};
```

初学者可能只问 `sizeof(Particle)`。更完整的审查会问：`active` 后面是否产生 padding？数组中相邻元素的步长是多少？热循环是否每次只读取 `x, y, z` 却把 `id` 和 `active` 一起带入 cache line？如果跨 C ABI 传递，字段和 padding 是否成为公开契约？如果汇编 kernel 按固定 offset 读取 `z`，未来重排字段是否会破坏它？如果多个线程更新 `active`，是否会造成同一 cache line 上的写竞争？这些问题都从布局出发，但影响远超布局本身。

布局改变不是局部修改

修改结构体字段顺序看起来只是源码重排，但它可能改变：

- `sizeof` 和数组步长。
- 每个字段 offset。
- 函数传参和返回的 ABI 分类。
- 序列化、文件格式或网络协议。
- 手写汇编和 SIMD kernel 的地址公式。
- cache line 利用率和预取效果。
- 调试器显示和旧二进制兼容性。

因此，内部私有结构体可以较自由地为性能重排；公共 ABI、文件格式、共享内存、网络协议和汇编参数块则必须谨慎。公共布局一旦发布，字段 offset 就像函数签名一样成为合同。要修改，应通过版本字段、新结构体或 wrapper 转换，而不是静默改变。

安全审查和性能审查共用地址公式

内存安全和性能经常使用同一套地址公式。安全审查问：访问是否在对象边界内，生命周期是否有效，对齐是否满足，类型访问是否合法。性能审查问：访问是否连续，步长是否友好，cache line 是否充分利用，是否有多余 load/store。二者都从同一条机器地址公式开始。

例如：

```
mov eax, dword ptr [rdi + rcx*12 + 8]
```

安全角度要问：`rcx` 是否在有效下标范围内？`+8` 是否确实是某个 4 字节字段？对象数组是否至少有 `rcx+1` 个元素？地址是否满足 4 字节对齐？性能角度要问：结构体步长 12 是否导致访问字段时 cache line 利用率怎样？若只读取这个字段，是否 SoA 更合适？scale 12 不能直接编码，前面是否有额外 `lea` 成本？同一条地址公式服务两个方向。

核心理想

布局审查的核心是把 `sizeof/alignof/offsetof`、C++ 生命周期、ABI 规则和汇编地址公式连起来。只会算大小，不等于理解布局。

综合案例：一次字段重排引发的事故

假设项目中有一个参数块：

```
struct KernelArgs {
    const float* x;
    const float* y;
    float* out;
    std::int64_t n;
    std::int64_t stride;
};
```

手写汇编按固定偏移读取字段：

```
mov r8, qword ptr [rdi + 0] # x
mov r9, qword ptr [rdi + 8] # y
```

```
mov r10, qword ptr [rdi +16] # out
mov rcx, qword ptr [rdi +24] # n
mov r11, qword ptr [rdi +32] # stride
```

后来有人为了“把整数放前面更清楚”，把结构体改成：

```
struct KernelArgs {
    std::int64_t n;
    std::int64_t stride;
    const float* x;
    const float* y;
    float* out;
};
```

C++ wrapper 重新编译后，字段访问当然使用新 offset；但手写汇编仍然按旧 offset 读取。于是 r8 读到 n，被当成指针；rcx 读到 y 的地址，被当成长度。程序可能立即崩溃，也可能写坏内存。这个事故不是“汇编突然错了”，而是布局合同被静默改变。

正确预防方式有三层。

第一，在 C++ 头文件中写 static_assert：

```
static_assert(sizeof(KernelArgs) == 40);
static_assert(offsetof(KernelArgs, x) == 0);
static_assert(offsetof(KernelArgs, y) == 8);
static_assert(offsetof(KernelArgs, out) == 16);
static_assert(offsetof(KernelArgs, n) == 24);
static_assert(offsetof(KernelArgs, stride) == 32);
```

第二，在汇编文件注释中写字段表，让 reviewer 能看到偏移来自哪里。

第三，把参数块视为 ABI。若必须改变布局，新增 KernelArgsV2 和新符号，而不是静默复用旧符号。

性能事故：字段重排也会改变 cache 行为

字段重排不只影响 correctness。假设热循环只读取 x,y,out,n，很少读取 debug_counter 和 name_ptr。如果冷字段夹在热字段中间，数组形式的参数块或对象数组可能浪费 cache line。反过来，把所有字段按大小排序虽然减少 padding，却可能把一起访问的字段拆散。布局优化不能只看 sizeof，还要看访问模式。

一个布局报告应写：

```
Correctness:
    offsets fixed by static_assert

ABI:
    assembly reads documented offsets

Performance:
    hot fields x/y/out/n are read at function entry
    cold fields are not in the hot loop

Risk:
    changing field order requires updating asm and ABI version
```

这类报告把布局从“内存大小问题”提升为“工程契约问题”。

深入讲解：对象、存储和值为什么要分开

C++ 底层学习中，最容易混淆的是对象、存储和值。存储是一段字节区域；对象是在某段存储中具有类型和生命周期的实体；值是按类型解释对象表示得到的抽象结果。三者相关，但不能互相替代。

例如，operator new 返回一段原始存储，这段存储有地址和大小，但还没有构造出某个具体对象。placement new 可以在这段存储中开始对象生命周期。对象析构后，存储可能仍然存在，但对象生命周期结束。此时旧指针的地址值可能还指向同一段字节，但不能随意按旧类型解引用。

这一区分对 allocator、arena、对象池、序列化和手写内存管理非常重要。很多 bug 不是“地址不对”，而是“地址仍然在，但对象已经不存在”或“存储复用了，但旧指针语义失效”。汇编只能看到地址和 load/store，看不到 C++ 生命周期。读底层代码时，必须把语言对象规则补回来。

值也不是字节的唯一解释。同一组字节可以作为 `std::uint32_t`、`float` 位模式、四个字符或结构体字段的一部分。C++ 允许通过 `unsignedchar` 或 `std::byte` 观察对象表示，但不允许随意把任意字节按任意类型访问。`std::bit_cast` 和 `std::memcpy` 提供了更安全的位模式搬运方式。

为什么这会影晌优化

编译器依赖对象生命周期和类型规则优化。若两个不同类型指针按规则不能别名，编译器可以重排 load/store；若对象生命周期已经结束，编译器不需要保留旧对象的值；若 signed overflow 未定义，编译器可以假设它不发生。底层程序员若只看硬件能不能读写某个地址，就会和编译器语义模型冲突。

这就是为什么“我在机器上试过能跑”不是 C++ 合法性的证据。未定义行为可能在 `-O0` 下看起来正常，在 `-O3` 下被优化器利用前提后失败。底层教材必须同时讲机器字节和语言规则，否则读者会写出只在当前构建偶然工作的代码。

深入讲解：布局优化要先问访问模式

结构体字段重排可以减少 padding，但不一定提升性能。性能取决于访问模式。如果热循环总是同时读取字段 `x, y, z`，把它们放在一起有利；如果一个阶段只读所有 `x`，另一个阶段只读所有 `y`，SoA 可能更好；如果对象很大但每次只用一个字段，AoS 会浪费 cache line；如果对象需要频繁按 id 查找，哈希表或索引结构可能比字段排列更重要。

布局优化应按步骤做：

1. 写出 hot path 访问哪些字段。
2. 统计这些字段是一起访问，还是分阶段访问。
3. 计算对象大小、字段 offset 和每个 cache line 携带的有效数据。
4. 判断是否存在跨步访问、随机访问或 false sharing 风险。
5. 设计 AoS、SoA 或参数块版本，并用 benchmark 验证。

不要只因为 `sizeof` 变小就宣布更快。对象变小通常有利于容量和带宽，但如果重排破坏了 ABI、增加了转换成本、让热字段分散，收益可能消失。布局是正确性、接口和性能共同决策。

深入讲解：指针不是整数的替代品

在很多机器上，指针值看起来就是一个整数地址；也可以把指针转换成 `std::uintptr_t` 做打印或某些底层操作。于是初学者容易以为指针就是整数。C++ 语义上，指针携带的不只是数值，还有指向对象的关系、可比较范围、可做指针运算的数组边界和解引用类型。

例如，同一个数值地址，如果它不指向一个生命周期内的对象，就不能合法解引用。两个来自不同数组对象的指针，即使数值上可以比较大小，也不一定有定义良好的关系。指针加法按元素缩放，并且只在同一个数组对象及 one-past 范围内定义。把指针转成整数再转回来，也有实现相关边界，不能把它当成普通数学整数随意运算。

这并不是语言故意复杂，而是为了让编译器能优化，也为了让程序有可移植语义。硬件只看到地址，编译器还要知道对象和类型。底层工程师需要同时理解两层：机器地址解释为什么某条 load 访问某个位置；C++ 指针规则解释这次访问是否定义良好。

什么时候可以看作地址

在调试和汇编阅读中，把指针值看作虚拟地址是必要的。你需要计算 `base+index*scale+displacement`，需要判断地址落在哪个映射，是否对齐，是否跨 cache line。这个层面上，指针确实表现为地址。

但在写 C++ 源码时，不能把所有地址技巧都写成指针整数运算。若目的是观察字节，用 `std::byte` 或 `unsignedchar` 视图；若目的是保存地址数值，用 `std::uintptr_t` 并承认平台边界；若目的是对象访问，用正确类型和生命周期；若目的是底层 allocator，用明确的构造和析构管理对象。不同目的应使用不同工具。

深入讲解：内存安全错误的共同形状

越界、use-after-free、未初始化读取、类型别名违规、未对齐访问、double free，看起来是不同 bug，但它们有共同结构：某次内存访问不满足有效访问条件。前文列出的五个条件可以作为统一诊断框

架。

第一，可访问存储区域。地址必须落在进程可访问映射中，且权限允许读或写。否则会发生 page fault 或保护错误。

第二，对象生命周期。地址所在存储中必须有对应类型的活对象。free 后的内存、析构后的对象、未构造的 raw storage，都可能地址但没有对象。

第三，访问类型。用不允许的类型解释对象字节，可能违反别名规则。短期看可能读到预期字节，长期看会被优化器破坏。

第四，对齐。某些类型和指令要求地址满足对齐。未对齐可能在 x86 上能执行但变慢，也可能在其他平台异常。向量指令和原子操作尤其要注意。

第五，边界。地址范围必须落在对象或数组有效范围内。访问一个元素前后的 padding、相邻对象或分配器元数据，即使硬件能读，也不是合法 C++ 对象访问。

把内存 bug 归到这五类，排查会更清楚。比如段错误通常先看可访问存储和边界；ASan use-after-free 指向生命周期；奇怪的优化后错误可能来自别名或未定义行为；特定 SIMD 指令崩溃可能来自对齐；大输入才失败可能来自整数溢出导致边界错误。

反汇编阅读清单

看到一条内存操作时，按下面清单写笔记：

1. 这条指令是 load 还是 store?
2. 访问宽度是多少：byte、word、dword、qword、xmmword 还是 ymmword?
3. base 寄存器可能是哪一个指针或对象起点?
4. index 寄存器现在是元素下标，还是已经被 shl/lea 变成字节偏移?
5. scale 来自元素大小、字段数组大小，还是地址计算的中间步骤?
6. displacement 是结构体字段 offset、固定下标、栈槽偏移，还是全局对象偏移?
7. 如果有前置 lea，它把下标乘成了多少?
8. 这次访问落到一个 cache line 内，还是可能跨 cache line?

核心思想

offset 和 scale 是数据布局在汇编里的影子；shl 和 lea 则经常是“影子被折叠之前”的地址计算过程。读 AI 算子汇编时，先恢复地址公式，再讨论 SIMD、cache、带宽和流水线。

深入讲解：对象、存储和生命周期是优化边界

内存章节容易被误读成“会算地址就够了”。地址当然重要，但 C++ 程序中的内存访问从来不只是地址算术。一个访问是否合法，至少要同时满足存储存在、对象生命周期存在、类型访问规则允许、对齐满足、范围没有越界这几类条件。编译器正是依据这些条件优化程序。若程序违反条件，机器上偶尔跑出结果也不能反过来证明代码合法。

对象和存储的区别尤其关键。存储是一段字节区域，可以来自自动变量、静态对象、堆分配、内存映射或外部缓冲。对象是在某段存储中按某种类型和生命周期建立起来的实体。分配了足够字节，不等于其中已经有目标类型对象；对象生命周期结束，也不等于字节立刻消失。很多底层错误就发生在“字节还在，所以对象也还在”的错误想法里。

优化器关心这些边界，因为它必须判断两个访问是否可能指向同一对象，某个 load 是否可以复用旧值，某个 store 是否会影响后续读取，某个对象是否可能被外部函数改动。严格别名、有效类型、生命周期、逃逸和可见副作用都会进入判断。读者不需要在第一册背完所有标准条款，但必须建立一个原则：语言层合法性不是装饰，它会直接改变机器码。

安全观察字节不等于任意解释字节

通过 `std::byte`、`unsigned char` 或 `std::memcpy` 观察对象表示，是学习布局和字节序的常用方法。这种观察通常是安全的，因为语言允许以字节方式查看对象表示。但把同一段字节强行当成另一个无关类型对象来解引用，就进入了另一类问题。前者是在看表示，后者是在声明“这里有一个这种类型的对象，并按这种类型访问它”。

例如把一个整数对象的地址转成浮点指针后解引用，不只是解释方式变了，还可能违反类型访

问规则和对齐要求。即使某次运行打印出某个浮点数，也不能说明程序定义良好。正确做法通常是用 `std::bit_cast` 在值层面表达位模式转换，或用 `std::memcpy` 把字节复制到目标类型对象中。这样写不仅更安全，也让编译器更清楚你的意图。

padding 也需要谨慎。结构体中的 padding 字节是布局的一部分，但它们未必具有稳定值。两个逻辑字段相等的对象，padding 可能不同。因此，用 `memcmp` 比较含 padding 的结构体作为逻辑相等判断通常是错误的。反过来，在二进制协议或文件格式中，如果你希望字节级稳定，就必须显式定义每个字段、填充、字节序和版本，而不能把普通 C++ 结构体直接当作外部格式。

课堂讲解：布局决策要从访问模式反推

结构体字段顺序、数组组织方式、对齐策略和指针关系，不能只按“看起来整齐”决定。布局首先服务访问模式。一个程序如果总是同时读取同一对象的多个字段，AoS 可能直观且局部性较好；如果程序经常只扫描某一个字段，SoA 往往能减少无用字节搬运，并给向量化创造条件。若访问模式会在不同阶段变化，还可能需分阶段布局或转换成本分析。

以粒子更新为例。若每一步都读取并更新位置、速度和质量，单个对象集中存放便于代码表达，也减少索引管理。但若渲染阶段只需要位置，物理阶段只需要速度和质量，统计阶段只扫描质量，把所有字段绑在一个大结构体里会让 cache line 携带大量当前阶段不需要的数据。性能问题不是“结构体慢”或“数组快”，而是每次 cache line 搬运中有效字节比例是多少。

布局还影响 ABI 和兼容性。对公开头文件中的结构体重新排序字段，可能改变 `sizeof`、`alignof`、字段 `offset` 和调用约定。如果这个结构体跨动态库边界、文件格式边界或网络协议边界，改字段顺序就不再是内部优化，而是破坏接口。即使只在单个程序内部，布局变化也可能改变序列化、调试脚本、二进制补丁、汇编互操作和性能测试结果。

从 cache line 角度估算有效载荷

初步布局分析可以从 cache line 有效载荷开始。假设 cache line 是 64 字节，一个结构体大小为 64 字节，其中热循环只读取 8 字节字段。顺序扫描时，每条 cache line 只提供 8 字节有用数据，有效载荷比例大约是八分之一。若把这个热字段拆成独立数组，同样一条 cache line 可以容纳 8 个元素，扫描效率明显不同。

这个估算不是最终结论，因为真实性能还受预取、写回、TLB、分支、向量化和并发影响。但它是很好的第一性检查。若一个布局让绝大多数搬运字节都无用，就必须有其他理由支持它，例如减少间接访问、保持对象封装、降低转换成本或满足 ABI 稳定。工程决策不是只追求某个单点 benchmark，而是在清楚成本后选择。

对齐不是越大越好

对齐可以让硬件访问更自然，也可能满足 SIMD 或原子操作要求。但过度对齐会增加对象大小、浪费 cache、扩大工作集。一个 24 字节对象如果因为某个字段或手动对齐变成 64 字节，顺序数组的内存占用可能接近三倍。若程序瓶颈在内存带宽或 cache 容量，这种变化会明显伤害性能。

因此，对齐决策要回答三个问题：目标硬件或指令是否要求这种对齐；不对齐时代价是否真的可见；增加的 padding 是否会扩大热工作集。很多时候，自然对齐已经足够。需要更高对齐时，应把它局限在真正的热数组、DMA 缓冲、SIMD 数据块或并发共享结构上，并用 benchmark 验证。

综合案例：一次字段重排为什么会引发线上问题

设想一个项目中有结构体表示请求记录，字段包括用户标识、时间戳、状态码、若干标志位和一个指向扩展信息的指针。有人为了减少 padding，把字段重新排序，本地单元测试全部通过，简单 benchmark 也略快。上线后，一个旧版本插件开始读取错误字段，日志解析脚本也把状态码解释成时间戳的一部分。

这个事故的根因不是“字段重排不好”，而是没有区分内部布局 and 外部合同。如果结构体只在一个翻译单元内部使用，字段重排通常是可控优化；如果它出现在公共头文件、共享库接口、二进制日志、内存映射文件、插件接口或手写汇编中，它就是 ABI 和数据格式的一部分。任何变化都需要版本化、兼容层或同步升级。

排查这类问题时，要建立四层证据。第一层是源码差异，确认字段顺序、类型和对齐属性变化。第二层是编译期证据，用 `sizeof`、`alignof` 和 `offsetof` 打印布局。第三层是二进制证据，检查访问指令中的 displacement 是否改变。第四层是边界证据，确认哪些外部模块、文件或脚本依赖旧 offset。

只有四层合起来，才能判断这是性能优化、接口破坏，还是两者同时发生。

内存相关改动的审查方法

任何涉及指针、布局、生命周期、别名或缓冲区大小的改动，都应该按审查清单处理。第一，明确对象所有权：谁创建对象，谁结束生命周期，谁可以保存指针或引用。第二，明确范围：每个指针能访问多少元素，长度单位是字节还是元素，边界是否包含尾端。第三，明确类型：是否通过合法类型访问对象，是否需要字节级观察，是否存在违反别名规则的转换。

第四，明确对齐：对象地址是否满足目标类型要求，手写汇编或 SIMD 是否额外要求对齐。第五，明确布局：字段 offset 是否被外部依赖，padding 是否影响比较、序列化或哈希。第六，明确并发：是否有另一个线程可能同时读写同一存储，原子性和内存顺序是否定义清楚。第七，明确测量：若改动声称提升性能，是否给出正确性、反汇编和输入规模证据。

这份清单看起来严格，但它能防止许多高成本错误。内存 bug 的危险在于它们常常不稳定：Debug 下不复现，Release 下偶发；小输入不复现，大输入崩溃；本机不复现，换编译器后出现。审查越早把边界写清楚，后面用工具定位的成本越低。

分配器和对象所有权的第一层模型

本章主要讲对象、字节和布局，但真实程序中的内存还涉及分配器。分配器负责把较大的虚拟内存区域切分成可用块，并在释放后复用。程序员看到的指针通常指向用户可用区域，分配器还可能在附近保存元数据。越界写有时没有立刻崩溃，是因为写到了相邻对象或分配器元数据；下一次分配或释放才暴露问题。

所有权模型是理解堆内存的第一步。一个对象由谁创建，谁负责销毁，谁只是借用，谁能延长生命周期，谁可以跨线程保存指针，都必须清楚。C++ 的 RAII、智能指针、容器和 span 都是在表达所有权或借用关系。裸指针本身不表达所有权；它只表示一个地址和访问类型。底层代码可以使用裸指针，但接口文档要补足所有权语义。

arena 或 pool 分配器常用于性能场景，因为它们能减少单个对象分配成本，提高局部性，并让批量释放更便宜。但 arena 也改变生命周期模型：单个对象可能不再单独析构，指针在 arena reset 后全部失效。若把 arena 分配出的对象指针保存到 arena 外部，就会产生悬空访问。性能优化不能绕过生命周期说明。

释放后地址仍然像地址

use after free 的迷惑性在于，释放后的指针值通常仍然是一个看起来合理的地址。硬件不知道这个地址曾经属于哪个 C++ 对象；分配器也可能暂时没有复用这块内存。所以程序可能在测试中继续读到旧值。语言层却已经结束了对象生命周期，继续访问就是错误。

ASan 能把这类错误变得更容易观察，因为它会在释放后标记内存状态，并在访问时报告。但 ASan 改变内存布局 and 性能，不应把 ASan 下的时间数字当成发布性能。功能诊断和性能测量要分开。这正是第 4 章工具边界和第 15 章测量边界的交叉点。

布局演化：内部优化和外部格式要分开

一个常见工程原则是：内部布局可以为了性能演化，外部格式必须稳定并版本化。内部布局指进程内临时结构体、热循环数据组织、缓存友好的 SoA 数组。外部格式指文件、网络协议、共享内存、数据库记录、插件 ABI、动态库公共结构体。把普通 C++ 结构体直接当作外部格式，短期方便，长期危险。

外部格式应该显式定义字段宽度、字节序、对齐、版本和保留字段。读取时把外部字节解析成内部对象，写出时再从内部对象序列化。这样内部布局可以按访问模式优化，不会破坏历史数据。性能敏感时，可以设计专门的 packed wire format 和内部 hot layout，并测量转换成本，而不是让一个结构体同时承担所有职责。

内部布局也不应随意改。热路径布局变化需要 benchmark，ABI 边界布局变化需要兼容审查，调试和运维工具依赖 offset 时需要同步更新。布局不是纯代码风格，它既是性能选择，也是接口选择。

AoS、SoA 和混合布局的教学案例

假设有一百万个点，每个点有 `x`、`y`、`z` 和 `color`。渲染阶段只需要位置，统计阶段只需要 `z`，调试阶段需要完整对象打印。AoS 写起来最自然，每个点一个结构体；SoA 把每个字段放成独立数组；混合布局可以把热字段放在一起，把冷字段放到旁表。

选择时先列访问矩阵：每个阶段读取哪些字段，是否顺序扫描，是否随机访问，是否写入，是否需要同时传给外部 API。若大部分热循环只读取 `x`、`y`、`z`，把 `color` 和调试字段放进同一 cache line 就浪费带宽。若外部 API 要求数组结构体，完全 SoA 可能增加转换成本。混合布局往往是工程折中。

这类案例不能只靠口号决定。报告应包含结构体大小、字段 offset、cache line 有效载荷估算、关键循环汇编和 benchmark。若 SoA 版本快，要说明是减少无用字节、改善向量化、降低 stride，还是其他原因。若 AoS 版本更快，也要解释是否因为转换成本、预取、对象封装或输入规模较小。教材的目标不是让读者永远选择某一种布局，而是学会推导。

内存章节的学习输出

学完本章后，建议读者整理三份产物。第一份是布局报告：选择两个结构体，打印 `sizeof`、`alignof`、`offsetof`，解释 padding 来源，并说明字段重排是否安全。第二份是地址公式报告：从一个数组或结构体访问的汇编中恢复 base、index、scale、displacement，并映射回源码。第三份是内存安全报告：选择一个越界、use after free 或别名错误，用工具证明它属于哪一类。

这三份产物分别训练布局、汇编和语义。只会打印地址不够，只会读汇编也不够，只会运行 sanitizer 也不够。内存能力来自三者合一：知道对象在语言层是什么，知道字节在机器层怎么访问，知道工具如何证明错误。

内存模型和性能模型不要混淆

内存安全讨论和性能讨论经常使用同一些词，例如地址、对象、cache line、对齐、别名，但它们回答的问题不同。内存模型首先问访问是否合法：对象是否存在、类型是否允许、范围是否正确、生命周期是否有效。性能模型问访问成本如何：是否连续、是否复用、是否跨 cache line、是否命中、是否能预取。合法访问也可能很慢，非法访问也可能在某次运行中看起来很快。

例如越界读取相邻对象，硬件可能顺利返回一个值，性能甚至很好，但语言层已经错误。又例如按合法方式随机访问大数组，程序完全正确，却可能因为 cache 和 TLB 成本很高而慢。把这两类问题混在一起，会导致危险结论：用“跑得快”证明合法，或用“合法”证明高效。

工程报告应先写合法性，再写性能。先证明指针、范围、生命周期和类型成立；再讨论地址模式、工作集和 cache。若合法性不成立，性能结论停止。若合法性成立但性能差，再进入布局和访问模式分析。这条顺序贯穿后面的汇编和 benchmark 章节。

调试内存问题时的证据组合

内存问题很少只靠一种证据解决。ASan 可以指出越界或释放后使用，GDB 可以显示崩溃现场，objdump 可以说明访问宽度和地址公式，sizeof 和 offsetof 可以证明布局，/proc/<pid>/maps 可以说明地址属于哪个映射，日志和输入可以说明路径条件。把这些证据组合起来，才能从“某地址崩溃”走到“哪个对象、哪个字段、哪条路径、哪个前置条件失败”。

如果只看崩溃地址，可能误判根因；只看源码，可能忽略优化后的访问；只看 sanitizer，可能不知道发布构建中的二进制形态；只看 benchmark，可能把内存错误当成性能差异。内存章节要求读者多工具交叉验证，就是为了避免单点误判。

综合案例：把结构体改小以后为什么反而变慢

结构体变小通常有利于 cache，但不是绝对。假设一个结构体原来 32 字节，字段自然对齐；优化后通过压缩字段和重新排序变成 24 字节。理论上每个 cache line 能容纳更多对象，但 benchmark 却显示某个热循环变慢。此时不能简单说“cache 理论错了”，而要继续拆。

第一种可能是对齐变化。原来某个热字段在 8 字节对齐位置，改动后变成跨边界访问或需要额外指令组合。第二种可能是访问模式变化。若循环每次访问两个相邻对象的某些字段，24 字节步长可能让访问跨 cache line 的模式更复杂。第三种可能是编译器向量化受影响。字段排列改变后，load/store

组合、别名判断或对齐假设不同，机器码可能变差。第四种可能是 benchmark 输入规模正好落在不同 cache 层级边界，导致比较不稳定。

分析这类问题需要四份证据：布局报告，确认 `sizeof`、`alignof` 和 `offset`；反汇编报告，确认访问指令和地址公式；输入规模扫描，确认趋势是否稳定；正确性检查，确认压缩字段没有截断语义。只有这些证据齐备，才能判断结构体变小到底是好是坏。

内存设计的长期维护问题

内存布局一旦进入公共接口，就会变成长期承诺。项目早期为了方便暴露结构体字段，后期可能发现无法重排；为了性能使用裸指针数组，后期可能发现所有权难以追踪；为了快速序列化直接写结构体字节，后期可能被字节序、padding 和版本升级困住。内存设计的代价经常延后出现。

因此，设计时要区分热路径内部布局和长期稳定边界。内部布局可以频繁实验，用 benchmark 验证；外部边界要保守、版本化、文档化。性能工程不是把所有东西都暴露给底层优化，而是让需要优化的部分有自由度，让需要稳定的部分有合同。

实验：打印地址并区分存储期

创建 `labs/ch03_data_layout/address_map.cpp`：

```
#include <cstdlib>
#include <iostream>
#include <memory>

int global_x = 1;

int main()
{
    static int static_x = 2;
    int stack_x = 3;
    auto heap_x = std::make_unique<int>(4);

    std::cout << "global_x = " << &global_x << '\n';
    std::cout << "static_x = " << &static_x << '\n';
    std::cout << "stack_x = " << &stack_x << '\n';
    std::cout << "heap_x = " << heap_x.get() << '\n';

    int stack_arr[4] = {1, 2, 3, 4};
    for (int i = 0; i < 4; ++i) {
        std::cout << "stack_arr[" << i << "] = "
                  << &stack_arr[i] << '\n';
    }
}
```

编译运行：

```
clang++ -std=c++20 -O0 -g address_map.cpp -o address_map
./address_map
./address_map
```

交付物：

1. 记录两次运行的地址，说明哪些地址变化明显。
2. 解释栈数组相邻元素地址差值为什么等于 `sizeof(int)`。
3. 说明这些地址为什么是虚拟地址，不应直接理解为物理内存地址。

实验：对象字节、字节序和 `std::bit_cast`

创建 `labs/ch03_data_layout/dump_bytes.cpp`，实现一个通用 `dump_bytes`：

```
#include <bit>
#include <cstdint>
#include <cstring>
#include <iomanip>
#include <iostream>
```

```

#include <type_traits>

template <class T>
void dump_bytes(char const* name, T const& value)
{
    static_assert(std::is_trivially_copyable_v<T>);

    auto const* p = reinterpret_cast<unsigned char const*>(&value);
    std::cout << name << " (" << sizeof(T) << " bytes): ";
    for (std::size_t i = 0; i < sizeof(T); ++i) {
        std::cout << std::hex << std::setw(2) << std::setfill('0')
            << static_cast<unsigned char>(p[i]) << ' ';
    }
    std::cout << std::dec << '\n';
}

struct A {
    char c;
    int x;
};

int main()
{
    std::uint32_t u = 0x12345678U;
    float f = 1.0f;
    A a{'Z', 0x12345678};

    dump_bytes("u", u);
    dump_bytes("f", f);
    dump_bytes("a", a);

    auto f_bits = std::bit_cast<std::uint32_t>(f);
    dump_bytes("f_bits", f_bits);
}

```

交付物：

1. 标出 `u` 的小端字节序。
2. 解释 `A` 中哪些字节是字段，哪些可能是 padding。
3. 比较 `f` 和 `f_bits` 的输出，说明 `bit_cast` 的含义。
4. 用 `-00` 和 `-03` 都运行一次，观察输出是否变化。

实验：手工预测结构体布局

对下面结构体，先手工预测 `sizeof`、`alignof` 和每个字段 `offset`，再写程序验证：

```

struct S1 {
    char a;
    int b;
    double c;
    char d;
};

struct S2 {
    double c;
    int b;
    char a;
    char d;
};

struct S3 {
    char tag;
    float x;
    float y;
    float z;
};

```

验证程序必须使用：

```
sizeof(T)
alignof(T)
offsetof(T, field)
```

交付物：

1. 每个结构体的布局表。
2. 手工预测和实际输出的对比。
3. 解释字段重排为什么改变或没有改变 `sizeof`。
4. 说明如果这些结构体已经写入二进制文件，为什么不能随便重排字段。

实验：指针加法和汇编寻址模式

创建 `labs/ch03_data_layout/pointer_scale.cpp`。本实验不是只看一条 `load` 指令，而是训练你把源码、结构体布局、汇编寻址和具体地址模拟连起来。

```
#include <stddef>
#include <stdint>

extern "C" int load_int(int const* p, int i)
{
    return p[i];
}

extern "C" double load_double(double const* p, int i)
{
    return p[i];
}

struct Pair {
    int a;
    int b;
};

extern "C" int load_pair_b(Pair const* p, int i)
{
    return p[i].b;
}

struct Pixel {
    std::uint8_t r;
    std::uint8_t g;
    std::uint8_t b;
    std::uint8_t a;
};

extern "C" std::uint8_t load_pixel_b(Pixel const* p, std::size_t i)
{
    return p[i].b;
}

struct Vec4 {
    float x;
    float y;
    float z;
    float w;
};

extern "C" float load_vec4_z(Vec4 const* p, std::size_t i)
{
    return p[i].z;
}

extern "C" int load_2d(int const (*a)[5], std::size_t i, std::size_t j)
{
    return a[i][j];
}
```

生成汇编：


```
clang++ -std=c++20 -O3 -S -masm=intel pointer_scale.cpp -o pointer_scale.s
clang++ -std=c++20 -O3 -c pointer_scale.cpp -o pointer_scale.o
objdump -d -Mintel pointer_scale.o > pointer_scale.objdump.txt
```

如果系统上没有 GNU `objdump`，可以尝试：

```
llvm-objdump -d --x86-asm-syntax=intel pointer_scale.o > pointer_scale.objdump.txt
```

第一部分交付物：为每个函数写一张汇编阅读表。

函数	load 指令	base	index/scale	displacement
<code>load_int</code>	你填写	你填写	你填写	你填写
<code>load_double</code>	你填写	你填写	你填写	你填写
<code>load_pair_b</code>	你填写	你填写	你填写	你填写
<code>load_pixel_b</code>	你填写	你填写	你填写	你填写
<code>load_vec4_z</code>	你填写	你填写	你填写	你填写
<code>load_2d</code>	你填写	你填写	你填写	你填写

第二部分交付物：为每个函数写地址公式。格式必须像下面这样明确：

```
load_pair_b:
    sizeof(Pair) = 8
    offset(Pair, b) = 4
    addr(p[i].b) = base + i * 8 + 4

load_vec4_z:
    sizeof(Vec4) = 16
    offset(Vec4, z) = 8
    addr(p[i].z) = base + i * 16 + 8
```

第三部分交付物：选择三个函数做具体地址模拟。至少包含一个 `scale` 为 4 或 8 的例子、一个结构体字段例子、一个需要 `shl` 或 `lea` 的例子。

示例格式：

```
Function: load_pixel_b
Assume:
    rdi = 0x3000
    rsi = 5
Formula:
    addr(p[i].b) = 0x3000 + 5 * 4 + 2
                  = 0x3016
Access:
    Load 1 byte from 0x3016
```

第四部分交付物：记录目标文件中的机器码字节。报告里必须贴出每个函数的反汇编片段，例如：

```
0000000000000000 <load_int>:
 0: 48 63 f6          movsxd rsi, esi
 3: 8b 04 b7          mov  eax,DWORD PTR [rdi+rsi*4]
 6: c3               ret
```

如果你的机器生成的指令不同，不要照抄本书示例。你要解释自己的编译器为什么可能选择不同指令，例如用 `lea`、`movsxd`、`shl`、不同寄存器或不同指令顺序。

最终报告必须回答：

1. 哪些函数的元素大小可以直接放进 `scale`？
2. 哪些函数需要额外地址计算？额外地址计算由哪条指令完成？
3. 哪些 `displacement` 来自字段 `offset`？
4. `movzx` 和普通 `mov` 在 `std::uint8_t` 返回值中有什么区别？

5. `movss` 和 `movsd` 分别对应什么标量浮点类型?
6. `load_2d` 的行长 5 如何进入汇编?
7. 修改 `Pair` 字段顺序或添加字段后, `sizeof(Pair)`、字段 `offset` 和汇编如何变化?

实验：别名假设如何改变汇编

目标：观察同一个循环在“可能别名”和“承诺不别名”两种接口下，编译器生成的汇编是否不同。本实验不要求你得出绝对性能结论，只要求你用反汇编说明编译器掌握了哪些信息。

创建 `labs/ch03_data_layout/alias_effect.cpp`:

```
#include <stddef>

extern "C" void saxpy_maybe_alias(float* y,
                                  float const* x,
                                  float a,
                                  std::size_t n)
{
    for (std::size_t i = 0; i < n; ++i) {
        y[i] = y[i] + a * x[i];
    }
}

extern "C" void saxpy_restrict(float* __restrict y,
                              float const* __restrict x,
                              float a,
                              std::size_t n)
{
    for (std::size_t i = 0; i < n; ++i) {
        y[i] = y[i] + a * x[i];
    }
}
```

生成汇编和目标文件:

```
clang++ -std=c++20 -O3 -march=native -S -masm=intel alias_effect.cpp -o alias_effect.s
clang++ -std=c++20 -O3 -march=native -c alias_effect.cpp -o alias_effect.o
objdump -drwC -Mintel alias_effect.o > alias_effect.objdump.txt
```

交付物:

1. 比较两个函数入口附近是否存在指针范围比较或运行时别名检查。
2. 比较两个函数主循环是否出现向量 `load/store` 或展开。
3. 记录每个函数中对 `x[i]` 和 `y[i]` 的 `load/store` 指令。
4. 解释 `__restrict` 给编译器承诺了什么，以及调用者违反承诺会有什么风险。
5. 把结论写成“汇编证据”，不要写成“某版本一定更快”。若要谈性能，必须另做 benchmark。

可选扩展：去掉 `-march=native`、改用 GCC、或者把 `float` 改成 `double`，观察汇编形态是否变化。报告里要说明工具链和参数，因为自动向量化高度依赖编译器版本、目标 CPU 和优化开关。

实验：AoS 与 SoA 的第一次 benchmark

实现两个版本，计算所有粒子 `x` 坐标之和。

```
struct Particle {
    float x;
    float y;
    float z;
    float mass;
};
```

```

float sum_x_aos(Particle const* p, std::size_t n)
{
    float s = 0.0f;
    for (std::size_t i = 0; i < n; ++i) {
        s += p[i].x;
    }
    return s;
}

float sum_x_soa(float const* x, std::size_t n)
{
    float s = 0.0f;
    for (std::size_t i = 0; i < n; ++i) {
        s += x[i];
    }
    return s;
}

```

要求：

1. 使用相同的 `x` 数据初始化 AoS 和 SoA。
2. 至少测试 `n=1024`、`n=1048576`、`n=67108864`。
3. 每个规模重复运行至少 20 次。
4. 防止结果被编译器优化掉。
5. 输出最小值、中位数、最大值。
6. 查看 `-O3` 汇编，记录是否自动向量化。

报告必须回答：

1. 哪个版本更快？是否所有规模都一样？
2. 从 cache line 利用率解释结果。
3. 从汇编解释编译器是否生成了不同访问形态。
4. 如果同时使用 `x, y, z, mass`，你的结论是否可能改变？

本章作业

习题与作业

作业 1：字节级对象报告

选择 5 个对象：

- 一个 `std::uint32_t`。
- 一个负的 `std::int32_t`。
- 一个 `float`。
- 一个含 padding 的结构体。
- 一个你自己设计的嵌套结构体。

对每个对象提交：

1. 源码定义。
2. `sizeof` 和 `alignof`。
3. 字节 dump。
4. 手工解释每个字段或位模式。
5. 哪些字节不应该被当作稳定语义值。

作业 2：结构体布局优化设计

设计一个结构体表示简化版神经网络张量元数据，至少包含：

- 数据指针。
- 维度数量。
- 4 个维度。

- 4 个 stride。
- dtype。
- flags。
- 引用计数或状态字段。

提交两个版本：

1. 直觉字段顺序版本。
2. 经过布局推导优化后的版本。

要求给出每个版本的 `sizeof`、字段 offset、padding 分析，并解释你是为了减小大小、改善 cache 局部性，还是为了 ABI 可读性做取舍。

作业 3：指针和地址推导题

假设：

```
struct P {
    float x;
    float y;
    float z;
    float w;
};

P* p = reinterpret_cast<P*>(0x1000);
```

在 `sizeof(P)==16` 的前提下，手工计算：

1. `&p[0].x`
2. `&p[0].z`
3. `&p[3]`
4. `&p[3].w`
5. `reinterpret_cast<char*>(p)+3`

然后写程序在真实数组上验证相对偏移。报告中必须明确区分元素偏移和字节偏移。

作业 4：行为分类

判断下列行为属于 well-defined、implementation-defined、unspecified 还是 undefined，并解释原因：

1. 读取 `std::uint32_t` 的对象字节。
2. 有符号 `int` 溢出。
3. 无符号 `uint32_t` 溢出。
4. 访问 `a[n]`，其中数组合法下标为 `0..n-1`。
5. 用 `std::bit_cast<std::uint32_t>(float_value)` 读取位模式。
6. 用 `reinterpret_cast<double*>(&int_obj)` 后解引用。
7. 比较两个含 padding 的结构体对象的 `memcmp` 结果。
8. 打印指针值并观察 ASLR 导致每次运行不同。

作业 5：别名与向量化观察

基于前面的别名观察实验，另外设计一个三数组循环：

```
z[i] = x[i] + y[i]
```

分别写可能别名版本和 restrict 风格版本，生成 `-O3` 汇编。报告必须包含：

- 函数签名和你给编译器的别名信息。
- 是否出现运行时别名检查。
- 是否出现向量化主循环。
- 指令中每次迭代的 load/store 数量。

- 为什么不能在调用者可能传入重叠区间时谎称 restrict。

作业 6: 小型性能报告

完成 AoS vs SoA benchmark。报告不少于 2000 字，必须包括：

- 机器信息。
- 编译器和参数。
- 数据规模和初始化方式。
- correctness 验证方式。
- benchmark 方法。
- 汇编证据。
- 时间结果表。
- 用 cache line 利用率解释的性能模型。
- 至少一个失败或反直觉观察。

验收标准

完成本章后，你应该能做到：

- 解释位、字节、对象表示、类型和值之间的关系。
- 读懂小端内存 dump。
- 区分无符号回绕、有符号补码表示和 C++ 有符号溢出 UB。
- 从 `a[i]` 推导出地址计算。
- 从 x86-64 内存操作数识别 base、index、scale、displacement。
- 手工推导简单结构体的 offset、padding、sizeof 和 alignof。
- 知道如何安全观察对象字节，什么时候应使用 `std::bit_cast` 或 `std::memcpy`。
- 解释别名和对象生命周期为什么影响编译器优化。
- 用 cache line 利用率初步解释连续访问、步长访问、随机访问、AoS 和 SoA 的性能差异。

如果你能做到这些，第二册里的 cache、AoS/SoA、SIMD load/store、GEMM blocking 和 AI 算子布局优化，才会真正建立在可推导的基础上，而不是建立在经验口号上。

第 5 章

Linux、工具链、调试器和学习 workflow

问题与目标

前面几章已经建立三条基础链路：源码经过工具链变成可执行程序，CPU 执行机器码字节，数据在内存中以字节、对象、地址和布局的形式存在。现在需要把这些概念转化为可操作能力；否则，后续汇编、ABI 和 benchmark 学习仍然缺少证据基础。你必须具备一种基本能力：

能在 Linux 环境里独立构建、观察、调试、记录和复现实验。

底层学习常见的隐形障碍是：概念似乎看懂了，一到自己的机器上却无法验证。命令不知道怎样组织，编译参数不知道放在哪里，`objdump` 输出看不懂，GDB 里的寄存器和源码对不上，benchmark 结果没有保存，第二天也复现不了前一天的实验。这样学下去，底层知识只会停留在“看过许多名词”，而不能转化为“解释真实程序行为”的能力。

本章的目标不是把 Linux、CMake、GDB、ELF 工具和性能工具写成手册；手册可以随时查阅。本章要建立的是学习 workflow：面对一个 C/C++ 小程序时，知道怎样从源码得到编译命令，从编译命令得到二进制，从二进制得到反汇编，从反汇编进入 GDB 观察，再把观察整理成实验报告。

核心思想

工具不是附属品。对底层学习来说，工具就是证据获取系统。没有命令、输出、反汇编、调试记录和报告，任何“我理解了”的说法都缺少验证。

本章的最低目标

学完本章后，你应该能做到：

- 在命令行创建目录、编辑文件、编译并运行 C++ 程序。
- 区分源码文件、头文件、目标文件、可执行文件、调试信息和构建目录。
- 用 GCC 或 Clang 生成 `-O0`、`-O2`、`-O3`、带调试信息和带 sanitizer 的构建。
- 用 `objdump`、`readelf`、`nm` 观察机器码、ELF 结构和符号。
- 用 GDB 设置断点、单步源码、单步指令、查看寄存器、查看内存和反汇编。
- 知道 Debug、Release、sanitizer、benchmark build 的用途不同，不能混用结论。
- 为每个实验保存命令、环境、输入、原始输出和结论。

这些能力看似朴素，却是后续章节的地基。不会使用这些工具，就很难真正学懂汇编；不会保存实验记录，就很难写出可信 benchmark；不会区分构建类型，就容易把调试版本的行为误当成性能事实。

命令行不是障碍，而是可复现实验接口

图形化 IDE 可以提高编码效率，但底层实验必须学会命令行。原因很简单：命令行是可复制、可粘贴、可记录、可自动化的实验接口。报告里写“我点了某个按钮”没有复现价值；写出完整命令，别人才能在一台机器上重复你的实验。

先熟悉最基本的命令：

```
pwd          # 当前目录
ls -la       # 列出文件和权限
cd path/to/dir # 切换目录
mkdir -p build # 创建目录
cp a b       # 复制文件
mv a b       # 移动或重命名
rm file      # 删除文件
rg "pattern" . # 搜索文本
find . -name '*.cpp'
```

```
uname -a      # 操作系统和内核信息
lscpu         # CPU 信息
```

这些命令的价值不在于“会背”。你要知道它们在实验报告里承担什么角色：

- `pwd` 和目录结构说明实验在哪里发生。
- `rg` 和 `find` 帮你定位源码、汇编和结果文件。
- `uname` 和 `lscpu` 是环境记录的一部分。
- `mkdir-presults/...` 让每次实验的输出有固定归档位置。

心智模型

把命令行看成“实验脚本的草稿”。一次手敲命令如果有价值，就应该能被整理成脚本、Makefile、CMake preset 或报告中的复现步骤。

Shell 的执行模型：命令、参数、环境和退出状态

很多工具学习失败，不是因为工具本身复杂，而是因为没有理解 Shell 到底在做什么。你在终端输入的一行内容，通常会被 Shell 拆成命令名、参数、重定向、管道和环境变量，然后创建进程执行。

例如：

```
clang++ -std=c++20 -O2 src/main.cpp -o build/app
```

这里 `clang++` 是要执行的程序，后面的每一项是参数。编译器看到的是参数数组，而不是你脑子里的“编译这个程序”。参数顺序、空格、引号、路径都会影响结果。路径中如果有空格而没有正确引用，Shell 会把它拆成多个参数；通配符如果被 Shell 展开，程序收到的也不是星号本身。

环境变量是进程继承的一组键值。常见例子包括 `PATH`、`CC`、`CXX`、`CFLAGS`、`CXXFLAGS`、`LD_LIBRARY_PATH`。构建系统、编译器、动态链接器都可能读取环境变量。报告中如果某个环境变量影响结果，就必须记录。比如同一个 CMake 命令，如果 `CC` 和 `CXX` 指向不同编译器，生成的构建系统就可能完全不同。

每个命令结束时都会返回退出状态。通常 0 表示成功，非 0 表示失败。Shell 变量 `\protect\TU\textdollar?` 保存上一条命令的退出状态。脚本中如果忽略退出状态，就可能在编译失败后继续运行旧二进制，得到完全错误的实验结果。因此实验脚本建议使用：

```
set -euo pipefail
```

这不是魔法安全开关，但它能让许多错误尽早暴露：命令失败时退出，使用未定义变量时报错，管道中某一步失败时不被最后一个命令的成功掩盖。

常见误区

最危险的实验错误之一是“编译失败了，但旧的可执行文件还在，所以运行看起来成功”。每次报告都要保存构建输出或至少确认构建命令成功，并在必要时先清理构建目录。

脚本化：把手工实验变成可重复过程

手工命令适合探索，脚本适合复现。一个实验只要需要第二次运行，就应该逐步脚本化。脚本不是为了炫技，而是为了减少遗漏：创建目录、记录环境、编译、保存反汇编、运行程序、保存结果，都应该有固定顺序。

一个好的实验脚本有几个特征：

- 从固定工作目录运行，或在开头切换到脚本所在目录。
- 明确创建输出目录，不把结果散落在项目根目录。
- 打印或保存关键命令，便于审计。
- 构建失败时停止，不继续运行旧程序。
- 输出文件名包含构建类型、输入规模或时间戳等必要维度。
- 不依赖终端历史或个人 Shell 配置。

例如一次章节实验可以有这样的流程：

```
#!/usr/bin/env bash
set -euo pipefail

cd "$(dirname "$0")"
mkdir -p build-release results

{
    date
    uname -a
    clang++ --version
} > results/environment.txt 2>&1

clang++ -std=c++20 -O3 -DDEBUG \
    src/main.cpp -o build-release/app

objdump -drwC -Intel build-release/app \
    > results/app.objdump.txt

./build-release/app > results/run.txt 2>&1
```

脚本中的命令仍然要被读懂。不要把所有内容藏进复杂函数或过度抽象里。教学实验脚本的第一目标是清楚，第二目标才是复用。

Makefile：给小实验一个稳定入口

脚本适合一次完整流程，Makefile 适合给常用动作提供稳定入口。很多章节实验只有几个源文件，不值得引入完整工程系统；但如果每次都手敲长命令，迟早会出现参数遗漏、输出覆盖、构建类型混乱。一个小 Makefile 可以把这些约定固定下来。

例如：

```
CXX := clang++
CXXFLAGS_COMMON := -std=c++20 -Wall -Wextra

SRC := src/main.cpp
DEBUG_BIN := build-debug/app
RELEASE_BIN := build-release/app

.PHONY: all debug release inspect clean

all: debug release

debug:
    mkdir -p build-debug
    $(CXX) $(CXXFLAGS_COMMON) -O0 -g $(SRC) -o $(DEBUG_BIN)

release:
    mkdir -p build-release
    $(CXX) $(CXXFLAGS_COMMON) -O3 -DDEBUG $(SRC) -o $(RELEASE_BIN)

inspect: release
    mkdir -p results
    objdump -drwC -Intel $(RELEASE_BIN) > results/release.objdump.txt
    nm -C $(RELEASE_BIN) > results/release.nm.txt
    readelf -S $(RELEASE_BIN) > results/release.sections.txt

clean:
    rm -rf build-debug build-release results
```

然后运行：

```
make debug
make release
make inspect
```

Makefile 的价值不是替代 CMake，而是让实验入口稳定。报告里写 `makeinspect` 比写一长串散落命令更不容易出错，但前提是 Makefile 本身要清楚、短小、可读。教学项目里不要把复杂逻辑藏进多层变量和隐式规则；读者应该能一眼看出 Debug 和 Release 到底用了哪些参数。

常见误区

Makefile 也是实验协议的一部分。修改 Makefile 的编译参数，就等于修改了实验条件。报告中若使用 **make**，应保存 Makefile 或记录其版本。

路径、工作目录和实验目录

底层实验经常失败在路径混乱上。比如同一个源码文件被复制到多个目录，编译的是旧文件，运行的是旧二进制，报告引用的是另一个输出。为了避免这种混乱，本书建议每个实验都使用固定结构：

```
labs/
  ch04_tools/
    src/
      main.cpp
      add.cpp
      add.hpp
    build-debug/
    build-release/
    results/
      environment.txt
      commands.txt
      objdump.txt
      gdb-session.txt
      report.md
```

这个结构表达了几条规则：

- 源码放在 **src/**，不要和构建产物混在一起。
- Debug 和 Release 分开构建，避免互相覆盖。
- 实验输出放在 **results/**，不要只留在终端滚动历史里。
- 每个实验至少保存 **commands.txt** 和 **environment.txt**。

相对路径和绝对路径都要能看懂。相对路径依赖当前工作目录，绝对路径从根目录开始。报告中可以使用相对路径，但必须说明执行命令时所在目录。例如：

```
cd labs/ch04_tools
clang++ -std=c++20 -O2 src/main.cpp -o build-release/app
./build-release/app
```

如果没有第一行 **cd**，后面的相对路径就不完整。

Linux 进程：实验真正运行的对象

当你在终端输入：

```
./build-release/app input.txt
```

运行的不是“一个源文件”，也不是“一个函数”，而是一个进程。进程是操作系统给正在运行的程序建立的执行容器。它至少包含：

- 一份虚拟地址空间，里面映射了程序代码、共享库、堆、栈和其他内存区域。
- 一组线程，最简单的程序只有一个主线程。
- 当前工作目录。
- 命令行参数，也就是 **argv**。
- 环境变量，也就是 **envp**。
- 打开的文件描述符表，例如标准输入、标准输出、标准错误和已经打开的文件。
- 资源限制、信号处理状态、用户和权限信息。

这解释了为什么同一个可执行文件在不同目录、不同环境变量、不同输入和不同权限下，行为可能不同。比如程序用相对路径打开 **data/input.txt**，实际查找位置取决于进程当前工作目录；程序加载共享库，搜索路径可能受动态链接器配置、**LD_LIBRARY_PATH**、**RPATH** 或 **RUNPATH** 影响；程序写日志失败，可能不是 C++ 逻辑错误，而是进程没有目录写权限。

从底层学习角度看，进程是把前面几章连接起来的地方。可执行文件提供机器码和元数据；加载器把它映射到进程地址空间；CPU 按 `rip` 执行这个地址空间里的指令；系统调用让进程请求内核打开文件、分配内存、创建线程或退出。工具观察也围绕进程展开：GDB 控制进程，`strace` 记录系统调用，`/proc` 暴露内核维护的进程信息，`perf` 统计进程运行时事件。

心智模型

一次实验不是“运行某个源码”，而是在特定环境中创建一个进程。要复现实验，就必须复现可执行文件、参数、工作目录、环境变量、输入数据、权限和关键系统设置。

权限、可执行位和 PATH 查找

Linux 文件有权限。一个文件是否能执行，不只取决于它里面是不是 ELF 或脚本，还取决于权限位。常用查看：

```
ls -l build-release/app
file build-release/app
```

如果看到类似：

```
-rwxr-xr-x 1 user user 123456 Jun 14 10:00 app
```

其中 `x` 表示可执行权限。若没有执行权限，可以：

```
chmod +x build-release/app
```

运行当前目录下的程序时通常写：

```
./build-release/app
```

这里的 `./` 很重要。Shell 只会在 `PATH` 环境变量列出的目录里查找没有斜杠的命令名。当前目录通常不在 `PATH` 里，因此直接输入 `app` 可能找不到，或者更糟：执行了另一个同名程序。

查看 `PATH`：

```
printf '%s\n' "$PATH" | tr ':' '\n'
which clang++
command -v clang++
```

报告中若使用某个工具，最好记录它的真实路径和版本。比如同一台机器上可能同时安装系统 GCC、手动安装 Clang、Conda 环境里的编译器、交叉编译器。只写“用 `clang++` 编译”有时不够，至少要保存 `clang++--version`；如果结果异常，还要保存 `command-vclang++`。

脚本还有一个特殊入口：shebang。脚本第一行如果是：

```
#!/usr/bin/env bash
```

内核执行脚本时会用指定解释器运行它。若脚本没有可执行位，可以用：

```
bash scripts/run.sh
```

这时执行的是 `bash`，脚本只是参数；若脚本有可执行位，可以：

```
./scripts/run.sh
```

这时 shebang 决定解释器。教学实验里，脚本开头建议写清 shebang，并在报告中保存脚本内容或版本。

常见误区

路径和权限问题经常伪装成程序错误。看到权限拒绝、命令不存在、文件不存在这类提示时，先检查执行路径、可执行位、工作目录、解释器和动态库依赖，不要马上改 C++ 逻辑。

标准输入、标准输出、标准错误和重定向

Linux 程序通常有三个基本流：

- **stdin**：标准输入，文件描述符 0。
- **stdout**：标准输出，文件描述符 1。
- **stderr**：标准错误，文件描述符 2。

重定向让输出可以保存下来：

```
./app > results/stdout.txt
./app 2> results/stderr.txt
./app > results/stdout.txt 2> results/stderr.txt
./app > results/all.txt 2>&1
```

这对实验很重要。终端输出一滚过去就很难审计；保存到文件后，报告可以引用，别人可以检查。比如保存反汇编：

```
objdump -drwC -Mintel build-release/app > results/app.objdump.txt
```

管道把一个命令输出交给另一个命令：

```
nm -C build-release/app | rg "sum|main"
readelf -S build-release/app | sed -n '1,80p'
```

管道适合快速观察，但报告里最好同时保存完整原始输出。只保存过滤后的几行，可能会丢失上下文。

从单文件编译开始

先建立一个最小程序：

```
#include <stdint>
#include <stdio>

extern "C" std::int32_t add_i32(std::int32_t a, std::int32_t b)
{
    return a + b;
}

int main()
{
    std::printf("%d\n", add_i32(1, 2));
    return 0;
}
```

用 GCC 或 Clang 编译：

```
g++ -std=c++20 -Wall -Wextra -O0 -g main.cpp -o app_debug
clang++ -std=c++20 -Wall -Wextra -O3 main.cpp -o app_release
```

这两条命令不是等价的。它们对应不同学习目的：

- **-O0-g**：保留更接近源码的调试结构，适合 GDB 学习。
- **-O3**：启用强优化，更接近性能路径，适合阅读优化汇编。
- **-Wall-Wextra**：打开常见警告，帮助发现代码问题。
- **-std=c++20**：固定语言版本，减少不同默认设置造成的差异。

常见误区

不要用 **-O0** 汇编讨论性能。Debug 构建的目标是可调试，不是快。它可能把变量放到栈上，保留多余的 load/store，避免某些优化，生成和真实热路径完全不同的代码。

分阶段编译：看清工具链边界

为了理解工具链，不能只会一条 `g++main.cpp-oapp`。要能把编译拆开：

```
g++ -std=c++20 -E main.cpp -o main.i
g++ -std=c++20 -S -O2 -masm=intel main.cpp -o main.s
g++ -std=c++20 -c -O2 main.cpp -o main.o
g++ main.o -o app
```

每一步的产物不同：

命令阶段	产物	观察重点
-E	.i	宏、include、条件编译展开后编译器看到什么
-S	.s	编译器选择了哪些汇编指令和标签
-c	.o	目标文件、机器码、符号、重定位
link	executable	最终可执行文件、入口、动态链接信息

学习时要同时看 `.s` 和 `objdump`。前者是编译器输出给汇编器的文本，后者是从目标文件或可执行文件中的机器码反汇编出来的文本。它们经常接近，但不是同一层证据。

多文件工程：头文件、定义和链接

真实程序很少只有一个源文件。考虑三个文件：

```
// add.hpp
#pragma once
#include <stdint>

std::int32_t add_i32(std::int32_t a, std::int32_t b);
```

```
// add.cpp
#include "add.hpp"

std::int32_t add_i32(std::int32_t a, std::int32_t b)
{
    return a + b;
}
```

```
// main.cpp
#include "add.hpp"
#include <cstdio>

int main()
{
    std::printf("%d\n", add_i32(1, 2));
}
```

编译：

```
g++ -std=c++20 -O2 -c add.cpp -o add.o
g++ -std=c++20 -O2 -c main.cpp -o main.o
g++ add.o main.o -o app
```

如果忘了链接 `add.o`：

```
g++ main.o -o app
```

链接器会报 `undefined reference`。这个错误不是语法错误，也不是运行时错误，而是链接阶段找不到某个符号定义。底层学习必须能区分：

- 编译错误：单个翻译单元内部语法或语义不成立。
- 链接错误：多个目标文件合成程序时符号无法解析或重复定义。
- 运行时错误：程序已经生成并启动，但执行过程中出现错误。

这一区分后面看 ABI、符号、重定位和 C++ 与汇编互操作时非常重要。

CMake 的最低工作流

大型工程不应该长期手写一串编译命令。第一册不需要深入 CMake，但至少要学会最小用法：

```
cmake_minimum_required(VERSION 3.20)
project(ch04_tools LANGUAGES CXX)

add_executable(app
  src/main.cpp
  src/add.cpp
)

target_compile_features(app PRIVATE cxx_std_20)
target_compile_options(app PRIVATE -Wall -Wextra)
```

构建：

```
cmake -S . -B build-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build-debug

cmake -S . -B build-release -DCMAKE_BUILD_TYPE=Release
cmake --build build-release
```

CMake 的价值不是“看起来专业”，而是把构建规则显式化。报告里可以写：

```
cmake -S . -B build-release -DCMAKE_BUILD_TYPE=Release
cmake --build build-release --verbose
```

如果需要查看真实编译命令，可以打开：

```
cmake -S . -B build-release \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

然后查看 `build-release/compile_commands.json`。这个文件对编辑器、语言服务器、静态分析和复现实验都很有用。

CMake Presets：把配置写成文件

如果一个项目需要反复生成 Debug、Release、sanitizer 和 benchmark 构建，命令行参数会越来越长。CMake Presets 可以把常用配置写进 `CMakePresets.json`，让配置可复现。

一个简化示例：

```
{
  "version": 6,
  "configurePresets": [
    {
      "name": "debug",
      "generator": "Ninja",
      "binaryDir": "build-debug",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "CMAKE_EXPORT_COMPILE_COMMANDS": "ON"
      }
    },
    {
      "name": "release",
      "generator": "Ninja",
      "binaryDir": "build-release",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release",
        "CMAKE_EXPORT_COMPILE_COMMANDS": "ON"
      }
    }
  ]
}
```

使用：

```
cmake --preset debug
cmake --build --preset debug

cmake --preset release
cmake --build --preset release
```

Presets 的好处是减少“我这次忘了传某个参数”的问题。它也让报告更清楚：你可以写“使用 **release** preset 构建”，并把 preset 文件作为源码的一部分保存。大型项目还可以为 sanitizer、benchmark、不同编译器和交叉编译准备不同 preset。

深入理解

Ninja 不是必须的生成器，但它输出简洁、增量构建快，常用于 CMake 工程。没有安装 Ninja 时，可以使用默认生成器；关键是把生成器和构建目录记录下来。

构建目录审计：确认你运行的是刚生成的程序

底层实验中，构建目录不是临时垃圾堆，而是证据来源。你应该能回答：这个可执行文件是什么时候生成的，由哪些目标文件链接而来，使用了哪些编译参数，是否来自当前源码。

最基本的审计包括：

- 查看可执行文件时间戳，确认它晚于源码修改。
- 使用 `verbose build` 保存真实编译和链接命令。
- 保存 `compile_commands.json`，确认每个源文件的参数。
- 使用 `nm`、`readelf`、`objdump` 验证二进制内容。
- 记录 Git commit 和 dirty status。

如果你使用 CMake，可以运行：

```
cmake --build build-release --verbose \
> results/build.verbose.txt 2>&1
```

这个文件经常比你手写的“我用了 `-O3`”更可靠，因为它记录了真实调用。大型工程里，目标可能继承了很多参数，最终命令未必等于你以为的命令。

还要注意增量构建。增量构建会跳过未变化目标，这本来是好事；但如果依赖关系写错，或者源码生成步骤没有被构建系统追踪，就可能运行旧产物。遇到难以解释的结果时，应该做一次干净构建：

```
rm -rf build-release
cmake -S . -B build-release -DCMAKE_BUILD_TYPE=Release
cmake --build build-release --verbose
```

这不是每次实验都必须做，但当结论很重要或结果异常时，干净构建能排除一类常见错误。

深入理解

很多“编译器怎么生成了奇怪代码”的问题，最后发现根本不是当前源码生成的二进制。审计构建目录的目的，就是在进入深层分析前先确认观察对象正确。

编译数据库：让工具看到同一个工程

`compile_commands.json` 是一个 JSON 文件，记录每个翻译单元的真实编译命令。它对底层学习有两个价值。

第一，它让编辑器和语言服务器使用与构建系统一致的 include path、宏定义和语言标准。没有编译数据库，编辑器可能用错误参数解析源码，显示不存在的错误，或漏掉真实错误。

第二，它让静态分析工具、格式化工具和审计脚本知道项目如何编译。比如一个文件只有在某个宏定义下才启用一段代码；如果工具没有看到相同宏定义，分析结论就可能不对应真实构建。

生成方式：

```
cmake -S . -B build-debug \
-DMAKE_BUILD_TYPE=Debug \
-DMAKE_EXPORT_COMPILE_COMMANDS=ON
```

报告里引用编译数据库时,不必贴完整 JSON,但应说明它来自哪个构建目录。Debug 和 Release 的编译数据库可能不同,因为优化级别、宏定义和 include path 都可能变化。

链接命令也是性能证据

初学者常只关注编译命令,忽略链接命令。可是链接阶段会影响最终二进制形态。

链接阶段可能决定:

- 使用哪些目标文件和库。
- 是否启用静态链接或动态链接。
- 是否使用 LTO。
- 是否生成 PIE。
- 符号是否被导出、隐藏或丢弃。
- 库搜索路径和运行时库路径。

例如开启 LTO 后,编译器可以跨翻译单元内联和优化;没有 LTO 时,一个源文件里的函数体可能对另一个源文件不可见。又比如动态链接的函数调用可能经过 PLT/GOT,手写汇编互操作时如果没有考虑 PIC/PIE,就可能在链接阶段出错或生成不合适的寻址。

因此,后面分析 ABI 和 C++ 与汇编互操作时,不能只说“源文件编译通过”。必须看链接命令、符号表、重定位和最终可执行文件。链接命令是二进制证据链的一部分。

快速识别二进制: file、ldd、size 和 strings

在进入反汇编之前,有几条轻量命令能快速回答“这个文件大概是什么”。

```
file build-release/app
ldd build-release/app
size build-release/app
strings -a build-release/app | rg "version|error|usage"
```

file 会告诉你文件类型、目标架构、是否 PIE、是否 stripped、是否包含调试信息等线索。看到 x86-64、dynamicallylinked、notstripped 这些词时,要能把它们和后面的 ELF、动态链接、符号表联系起来。

ldd 显示动态库依赖。它能帮助你确认程序运行时会加载哪些共享库,例如 libstdc++、libc、libm。如果程序在一台机器上能运行、另一台机器上找不到库,ldd 是第一批要看的工具之一。注意不要对不可信二进制随意运行 ldd;教学实验里使用自己构建的程序即可。

size 给出 text、data、bss 大小。它不能替代性能分析,但能帮助发现明显异常。例如一个小程序的 text 段突然变大很多,可能是模板实例化、静态链接、sanitizer、调试辅助或大段内联导致。代码大小变化会影响 instruction cache 和前端压力,后续学习前端与代码尺寸时会再次用到。

strings 从二进制中提取可打印字符串。它适合做粗略检查,例如确认版本字符串、错误消息、usage 文本是否进入二进制。但它不是结构化解析工具,不能替代 readelf 或符号表。

工具	回答的问题	主要限制
file	文件类型、架构、PIE、strip 状态。	只给摘要,不解释内部结构。
ldd	动态库依赖。	不适合不可信二进制。
size	代码和数据段大小。	不能说明热路径在哪里。
strings	二进制中可见字符串。	可能误判,也可能漏掉非文本信息。

这些工具适合放在实验脚本开头,作为二进制档案的一部分。它们给出的不是最终结论,而是后续深入分析的导航线索。

动态链接的第一印象：程序不只包含自己的代码

大多数普通 Linux C++ 程序不是把所有代码都静态放进一个文件。它们会依赖共享库，例如 C 标准库、C++ 标准库、数学库、线程库或项目自己的 `.so`。程序启动时，动态链接器会把需要的共享库映射进进程地址空间，并解析一部分符号引用。

查看依赖：

```
ldd build-release/app
readelf -d build-release/app
```

`ldd` 给出直观列表；`readelf -d` 查看 ELF 的 dynamic section，例如 `NEEDED` 项。若程序运行时报共享库找不到，常见原因包括：

- 共享库没有安装。
- 库路径不在系统搜索路径里。
- `LD_LIBRARY_PATH` 没有设置或设置错误。
- 二进制中 `RPATH/RUNPATH` 不符合预期。
- 架构不匹配，例如拿 ARM 库给 x86-64 程序加载。

动态链接还会影响反汇编。调用外部函数时，最终可执行文件里可能出现 PLT/GOT 相关代码。第一册不要求现在就手写动态链接汇编，但要知道：看到 `call printf@plt` 并不奇怪，它表示调用路径经过过程链接表。目标文件、最终可执行文件和运行时实际跳转路径之间还有动态链接器参与。

如果想看动态链接器加载库的过程，可以使用：

```
LD_DEBUG=libs ./build-release/app 2> results/ld-debug-libs.txt
```

这个输出可能很长，只适合诊断，不适合每次都保存。报告中如果使用它，要说明目的：例如确认某个 `.so` 实际从哪个目录加载。

常见误区

不要对不可信程序随意运行 `ldd` 或设置复杂的动态链接环境。教学实验使用自己构建的小程序即可。工程环境中分析第三方二进制时，应遵循安全隔离和最小权限原则。

静态链接、动态链接和性能结论

静态链接把更多库代码放入最终可执行文件，动态链接在运行时映射共享库。它们各有用途，也会影响文件大小、部署方式、符号解析、启动行为和反汇编形态。

性能实验中，链接方式必须记录。比如一个 benchmark 如果静态链接了大型运行时库，text size 和 instruction cache 行为可能不同；如果动态调用某个库函数，第一次调用可能涉及 lazy binding；如果启用 LTO，跨库或跨目标文件优化能力又会变化。大多数基础实验不需要静态链接，但报告必须写清实际链接方式。

可以用这些线索判断：

```
file build-release/app
ldd build-release/app
readelf -d build-release/app | rg 'NEEDED|RUNPATH|RPATH'
```

如果 `ldd` 显示它不是动态可执行文件，说明这个程序不走普通动态链接路径。若 `file` 显示它是静态链接，也要记录。后面学习 ABI 和互操作时，链接方式会影响符号解析和调用边界。

构建类型：不要混淆目的

同一个程序可以有多种构建方式。每种构建方式回答的问题不同：

构建类型	常见参数	用途
Debug	-O0-g	源码级调试、变量观察、单步学习。
RelWithDebInfo	-O2-g	优化后仍保留调试信息，适合结合 GDB 和性能观察。
Release	-O2 或 -O3	性能路径和最终二进制审计。
Sanitizer	-fsanitize=address, undefined	检查越界、use-after-free、未定义行为。
Benchmark	-O3-DNDEBUG 等	稳定测量，通常关闭 sanitizer，记录全部参数。

Debug 构建不能用于性能结论，sanitizer 构建也不能直接用于性能结论。sanitizer 会插入大量检查代码，改变内存布局、调用路径和运行时间。但 sanitizer 对正确性非常重要。正确流程通常是：

1. Debug 或 sanitizer 构建验证 correctness。
2. Release 构建阅读优化汇编。
3. Benchmark 构建做性能测量。
4. 报告中明确写出每一步使用的构建类型。

警告和 sanitizer：优化前先保证程序有定义

性能学习不能建立在未定义行为上。下面的代码可能看起来能跑：

```
int f(int* p)
{
    return p[10];
}
```

如果调用时数组只有 4 个元素，这就是越界。优化器可以利用语言规则做出你意想不到的改写。先用 sanitizer 抓这类问题：

```
clang++ -std=c++20 -O1 -g \
-fsanitize=address,undefined \
main.cpp -o app_asan_ubsan
./app_asan_ubsan
```

常用检查：

- AddressSanitizer：越界、use-after-free、栈/堆内存错误。
- UndefinedBehaviorSanitizer：有符号溢出、错误移位、空指针解引用等。
- ThreadSanitizer：数据竞争，代价较大，后续并发专题再深入。

sanitizer 的结论也要保守。它能抓很多错误，但不能证明程序完全正确；它会改变运行行为，所以不能直接拿 sanitizer 下的时间当优化结果。

objdump：从机器码回到可读指令

objdump 是本书最常用工具之一。基本命令：

```
objdump -drwC -Mintel app > results/app.objdump.txt
```

参数含义：

- -d：反汇编可执行段。
- -r：显示重定位信息，目标文件阶段尤其有用。
- -w：宽输出，不截断长行。
- -C：demangle C++ 符号名。
- -Mintel：使用 Intel 语法。

观察一个函数时，先找标签：

```
objdump -drwC -Mintel app | sed -n '/<add_i32>/,+20p'
```

不要只看指令文本，还要看三列信息：

address:	bytes	instruction
1140:	8d 04 37	lea eax, [rdi+rsi]
1143:	c3	ret

地址告诉你指令位于二进制中的位置；bytes 是 CPU 真正取到的机器码字节；instruction 是反汇编器给人的解释。读汇编时要记住：CPU 执行 bytes，不执行这一行文字。

阅读 objdump 的顺序

阅读 objdump 输出时，不建议从文件第一行一直读到最后。更有效的顺序是：

1. 先找到目标函数标签。
2. 确认函数是否真的存在独立代码。
3. 看函数入口和结尾，判断是否有栈帧、保存寄存器、调用其他函数。
4. 找到核心循环或关键分支。
5. 标出内存访问指令和调用指令。
6. 对照源码，判断哪些语句被内联、删除、展开或改写。

如果找不到函数，不要立刻认为编译失败。它可能被内联了，也可能因为未使用被删除，或者 C++ 符号名被 name mangling 改写。此时应配合 `nm-C`、`readelf-s` 和编译选项判断。

阅读反汇编时还要区分目标文件和最终可执行文件。目标文件中的 `call` 或地址引用可能仍然带重定位信息，最终地址要等链接后才确定。最终可执行文件中，链接器已经解析了许多符号，但动态链接相关引用可能仍然通过 PLT/GOT 间接完成。

常见误区

不要从反汇编里复制几行孤立指令就下结论。至少要确认这几行属于哪个函数、哪个构建类型、哪个地址范围，是否在热路径中，是否来自最终可执行文件。

源码行号和指令地址不是一一对应

带 `-g` 的反汇编或 GDB 可能显示源码行和指令的对应关系，但这个对应关系在优化后会变得复杂。一个源码行可能生成多条指令；多行源码可能合并成一段指令；某些行可能完全没有指令；一条指令也可能被调试信息关联到看似奇怪的源码位置。

这不是工具错误，而是优化器改变了程序结构。比如内联会把被调用函数的代码放进调用者；循环展开会复制循环体；常量传播会删除分支；死代码删除会让某些源码行没有机器实现。

所以，优化代码分析应以机器控制流和数据流为准，再回头解释源码语义。源码行号是导航线索，不是最终证据。

nm：符号表告诉你名字是否还存在

nm 查看符号：

```
nm -C app | rg "add|main"
```

常见符号类型：

符号类型	含义
T	text 段中的全局函数符号。
t	text 段中的局部函数符号。
U	undefined，需要链接时从别处解析。
D	已初始化全局数据。
B	BSS，未初始化全局数据。
W	weak symbol，弱符号。

如果优化后某个函数在 `nm` 中消失，不一定是错误。它可能被内联、删除、合并或变成局部符号。性能报告不能只说“源码里有这个函数”，必须确认二进制里是否有独立函数体，以及热路径是否真的调用它。

符号不是函数存在的全部证据

符号表回答的是“二进制中有哪些名字和地址关系”，但它不是程序行为的完整描述。一个函数可能存在符号，却从来不在热路径被调用；一个函数可能没有独立符号，因为已经被内联到多个调用点；一个模板函数可能实例化出多个版本；一个 `weak symbol` 可能被别的定义覆盖。

因此，使用 `nm` 时要把问题问具体：

- 我关心的函数是否有定义。
- 调用者目标文件是否仍有未解析引用。
- 最终可执行文件中调用是否已经被链接到某个地址。
- C++ `name mangling` 是否导致符号名和源码名不同。
- 符号可见性是否影响动态链接边界。

后面讲 C++ 与汇编互操作时，`extern "C"` 的一个重要作用就是控制符号名，让汇编文件和 C++ 文件能在链接阶段说同一种名字。但它只解决名字问题，不解决参数寄存器、栈对齐、返回值、异常和对象生命周期问题。

`readelf`: ELF 文件结构

`readelf` 直接查看 ELF 元数据：

```
readelf -h app
readelf -S app
readelf -l app
readelf -s app
readelf -r app
```

最常看的几类信息：

- ELF header：文件类型、机器架构、入口地址。
- sections：`.text`、`.rodata`、`.data`、`.bss`、`.eh_frame` 等。
- program headers：加载器如何把文件映射进进程地址空间。
- symbol table：符号定义和引用。
- relocations：哪些地址需要链接器或动态链接器修补。

第一册不要求你精通 ELF，但必须知道：可执行文件不是“纯机器码数组”。它是一个结构化文件，包含代码、数据、符号、重定位、调试/展开信息和加载描述。后续学习 ABI、动态链接、C++ 与汇编互操作时，这个认识会反复用到。

section 和 segment 的区别

ELF 里有两个容易混淆的概念：section 和 segment。

section 更偏向链接视角，例如 `.text`、`.rodata`、`.data`、`.bss`、`.symtab`、`.rela.text`。链接器使用 section 组织代码、数据、符号和重定位。

segment 更偏向加载视角。程序启动时，加载器按照 program header 描述把文件的一些范围映射到进程地址空间，并设置读、写、执行权限。多个 section 可以落在同一个 load segment 中。比如 `.text` 和某些只读元数据可能都在只读可执行或只读映射附近。

为什么这对底层学习重要？因为“文件里有什么”和“运行时映射成什么权限”不是同一层问题。你用 `readelf -S` 看 section，用 `readelf -l` 看 loader 如何映射。调试执行权限、只读数据、栈不可执行、动态链接和地址布局时，这个区别很关键。

重定位说明地址还没有完全固定

目标文件里的某些地址无法在编译单个源文件时确定。比如调用另一个翻译单元的函数，编译器可以生成一个占位引用，并让 relocation 记录告诉链接器以后如何修补。

查看目标文件：

```
readelf -r build-release/main.o
objdump -drwC -Mintel build-release/main.o
```

你可能看到某条 `call` 附近带有重定位项。这意味着反汇编中的立即数或目标地址不能按最终地址理解。链接后再看最终可执行文件，目标可能已经变成具体地址，或者变成 PLT 项。

这也是为什么学习链接不能只读源码。源码层写的是函数名；目标文件层保存符号引用和重定位；可执行文件层才接近运行时控制流。

GDB 的基本心智模型

GDB 不是魔法调试器。它依赖三类信息：

- 可执行文件中的机器码和符号。
- 编译器生成的调试信息，例如 `-g`。
- 运行中进程的寄存器、内存和控制权。

最小调试命令：

```
g++ -std=c++20 -O0 -g main.cpp -o app_debug
gdb -q ./app_debug
```

GDB 中：

```
break main
run
next
step
print x
info locals
info registers
x/10i $rip
x/32xb $rsp
disassemble /m main
quit
```

这些命令分成几类：

- 控制执行：`run`、`continue`、`next`、`step`。
- 观察源码变量：`print`、`info locals`。
- 观察机器状态：查看寄存器、当前指令和栈内存。
- 观察代码：`disassemble`、`layoutasm`。

源码级调试和指令级调试要区分。源码级 `next` 以源代码行为单位，指令级 `stepi` 以机器指令为单位。学习汇编和 ABI 时，必须会用 `stepi`。

断点、观察点和条件断点

最常见的断点是函数断点和源码行断点：

```
break main
break src/main.cpp:42
```

指令级学习时，也可以在地址上下断点：

```
break *0x401140
```

星号表示把后面的值当成地址，而不是函数名或源码行。使用地址断点前，要确认当前二进制是否启用 PIE，以及地址是否会因为 ASLR 变化。调试教学中如果地址变化造成困扰，可以临时关闭随机化：

```
set disable-randomization on
```

条件断点适合循环：


```
break sum_i32 if n == 1024
break src/main.cpp:30 if i == 100
```

但条件断点可能显著减慢程序，因为调试器需要频繁停下检查条件。性能测量时不能带着 GDB 条件断点运行后下结论。

观察点用于监控某个内存位置何时被读写：

```
watch variable
rwatch variable
awatch variable
```

它对定位“谁改坏了这个值”很有用。但硬件观察点数量有限，复杂表达式也可能退化成很慢的软件观察。使用时要把它当作调试工具，而不是性能工具。

崩溃时先看四件事

程序崩溃时，初学者容易直接猜原因。更好的流程是先收集四类证据：

1. 信号是什么，例如 SIGSEGV、SIGILL、SIGABRT。
2. `rip` 在哪里，当前指令是什么。
3. 关键寄存器是什么，尤其是参与地址计算的寄存器。
4. 调用栈是什么，最近的函数调用路径是否合理。

GDB 中常用：

```
run
bt
info registers
x/10i $rip
disassemble /r $rip-32, $rip+32
```

如果当前指令是 `load` 或 `store`，要计算它访问的地址，并判断该地址是否合理。比如当前指令从 `rdi` 指向的位置读取 32 位整数，第一眼应该看 `rdi`。如果 `rdi` 是 0 或很小的值，可能是空指针或野指针；如果地址看似合理，还要看页面权限、对象生命周期和是否越界。

如果当前指令是 `ret` 崩溃，要怀疑返回地址或栈被破坏。此时 `rsp`、栈顶内容、调用约定和前面的 `store` 都是重点。

常见误区

崩溃定位不要先从“代码大概想做什么”开始，而要先从“CPU 正在执行哪条指令、它访问哪个地址、哪个状态不合理”开始。底层调试的证据顺序很重要。

/proc：从内核视角观察进程

Linux 的 `/proc` 提供了一个观察进程状态的接口。它不是普通磁盘目录，而是内核导出的伪文件系统。对底层实验来说，`/proc/<pid>` 能回答很多“进程现在到底是什么状态”的问题。

先启动一个程序，并找到进程号：

```
./build-release/app &
pid=$!
echo "$pid"
```

然后观察：

```
cat /proc/$pid/cmdline | tr '\0' ' '
tr '\0' '\n' < /proc/$pid/environ | sort
readlink /proc/$pid/cwd
readlink /proc/$pid/exe
ls -l /proc/$pid/fd
cat /proc/$pid/maps
```

这些文件分别告诉你：

- **cmdline**: 进程启动时的参数。
- **environ**: 进程环境变量。
- **cwd**: 当前工作目录。
- **exe**: 实际运行的可执行文件。
- **fd**: 打开的文件描述符。
- **maps**: 虚拟地址空间映射。

maps 对理解 GDB 和崩溃尤其有用。它会显示可执行文件、共享库、堆、栈、匿名映射等地址范围和权限。例如一行里可能有 **r-xp**, 表示这段映射可读、可执行、私有; 也可能有 **rw-p**, 表示可读写但不可执行。若 **rip** 落在没有执行权限的区域, 或者崩溃地址不属于任何映射, 原因就更清楚。

这也解释了 PIE 和 ASLR 为什么影响调试。启用位置无关可执行文件和地址随机化后, 同一个函数每次运行的绝对地址可能变化。反汇编文件中的地址、GDB 中的运行时地址和 `/proc/<pid>/maps` 中的加载基址要结合起来看。教学实验中如果需要稳定地址, 可以在 GDB 中临时使用:

```
set disable-randomization on
```

但报告中要说明这一点, 因为关闭随机化改变了运行环境。

深入理解

`/proc` 是工具背后的证据来源之一。GDB、ps、top、lsof 等工具都可能从内核接口读取进程信息。学会直接看少量 `/proc` 文件, 可以帮助你判断工具输出到底来自哪里。

core dump: 让崩溃现场留下来

交互式 GDB 适合本地调试, 但程序有时在脚本、CI 或长时间运行后崩溃。此时 core dump 可以保存崩溃时的进程快照, 让你事后用 GDB 分析。

先查看和开启 core dump 限制:

```
ulimit -c
ulimit -c unlimited
```

运行程序崩溃后, 如果系统生成了 core 文件, 可以这样打开:

```
gdb -q build-debug/app core
```

进入 GDB 后仍然先看:

```
bt
info registers
x/10i $rip
disassemble /r $rip-32, $rip+32
```

不同 Linux 发行版对 core dump 的保存方式不同。有的会在当前目录生成 **core** 文件。有的会把崩溃信息交给 **systemd-coredump** 管理。如果当前目录没有看到 core 文件, 可以尝试:

```
coredumpctl list
coredumpctl gdb
```

core dump 不是万能的。若二进制和 core 不匹配, 调试信息缺失, 或程序被 strip, 分析会困难很多。因此保存崩溃现场时, 也要保存对应二进制、构建参数和源码版本。

核心思想

崩溃报告至少要绑定三样东西: core dump、对应二进制、对应源码/构建信息。只有 core 文件, 没有二进制和调试信息, 证据链是不完整的。

GDB 单步函数调用

考虑:

```
extern "C" int add_i32(int a, int b)
{
    return a + b;
}

int main()
{
    return add_i32(10, 32);
}
```

用 `-O0-g` 编译后，在 GDB 中：

```
break main
run
disassemble /m main
info registers rip rsp rdi rsi rax
stepi
stepi
x/4gx $rsp
```

观察重点不是“按了多少次回车”，而是回答：

- 调用前参数在哪里。
- `call` 执行后 `rsp` 如何变化。
- 返回地址保存在哪里。
- 进入 `add_i32` 后 `rip` 指向哪里。
- 返回后 `rax` 或 `eax` 是否保存返回值。

这就是把 ABI 表格变成机器证据的第一步。

调试信息不是程序语义本身

`-g` 会让编译器在二进制中写入调试信息。调试信息描述源码文件、行号、变量、类型、作用域和机器地址之间的关系。GDB 依赖这些信息把机器状态解释回源码世界。

但调试信息不是程序语义本身，它可能不完整，也可能因为优化而变得难读。比如变量被调试器标成 `optimized out`，并不表示程序没有这个概念上的变量，而是当前机器位置无法提供一个简单的存放位置。又比如一个变量在函数前半段位于寄存器，后半段被消除，调试器显示可能随位置变化。

所以调试时要区分三件事：

- 源码变量：C++ 语义层的名字。
- 调试信息位置：编译器告诉调试器变量此刻可能在哪里。
- 机器状态：寄存器、内存和指令地址的真实值。

当三者看起来矛盾时，先相信机器状态，再检查调试信息和优化。比如 GDB 显示某个局部变量值奇怪，但反汇编证明变量已经被常量传播或拆分，这时不能用调试器显示直接否定二进制。

工具会改变被观察对象

工具提供证据，但工具本身也可能改变程序行为。高质量实验必须说明这种影响。

GDB 会控制被调试进程。断点通常通过修改指令字节或使用硬件断点实现；单步执行会频繁让进程停下并交还控制权；条件断点可能让循环慢几个数量级。因此 GDB 适合解释状态，不适合直接给出性能结论。

`sanitizer` 会在编译期插入检查代码。`AddressSanitizer` 会改变内存分配、增加红区、插入访问检查；`UndefinedBehaviorSanitizer` 会插入运行时诊断；`ThreadSanitizer` 会大幅改变线程调度和访存成本。因此 `sanitizer` 适合发现错误，不适合直接代表 Release 性能。

`strace` 会拦截系统调用并记录参数和返回值。它对系统调用密集程序影响明显。它适合判断文件、路径、权限、进程、网络 and 系统调用错误，不适合测量正常运行速度。

`perfstat` 的干扰通常比 GDB 和 `strace` 小，但也不是绝对零开销。计数器复用、权限限制、虚拟化环境、CPU 频率变化、调度迁移都会影响结果。第 11 章会系统讲这些限制。

因此，报告里最好把工具角色写清楚：

工具	适合回答的问题	主要干扰
GDB	某一刻寄存器、内存、调用栈是什么。	断点、单步、调试信息和优化错位。
sanitizer	程序是否触发内存错误或未定义行为。	插桩改变代码和内存布局。
strace	程序请求了哪些系统调用。	拦截系统调用，显著变慢。
perfstat	运行时事件计数大致如何。	权限、复用、调度、虚拟化、频率。

核心理想

没有无扰动观察。工程上要做的是理解工具扰动，把工具用于它擅长的问题，并在报告中说明结论边界。

调试优化代码为什么困难

很多读者会问：既然性能路径是 `-O3`，为什么还要学 `-O0-g`？答案是两者回答不同问题。Debug 构建适合建立源码和机器状态的初始对应关系；优化构建适合研究真实性能路径。

优化代码调试会遇到：

- 变量被放进寄存器或直接消失。
- 函数被内联，调用栈看起来和源码不同。
- 循环被展开、向量化或重排。
- 源码行号和指令地址不再一一对应。
- 某些变量显示为 `optimized out`。

这不是 GDB 坏了，而是优化器改变了程序结构。正确做法是：

1. 用 `-O0-g` 学习基本控制流和状态变化。
2. 用 `-O2-g` 或 `-O3-g` 观察优化后的真实路径。
3. 用 `objdump` 作为最终二进制证据。
4. 报告里说明当前观察属于哪种构建。

time、perf 和本册的测量边界

本章只给最小测量工具意识，系统 benchmark 放到第三部分。

`time` 可以粗略看程序整体时间：

```
/usr/bin/time -v ./app
```

但它测的是整个进程，不是某个 kernel。启动、动态链接、输入输出、初始化都可能混进去。

`perfstat` 可以尝试获取硬件或内核计数：

```
perf stat ./app
perf stat -e cycles,instructions,branches,branch-misses ./app
```

注意几个限制：

- 虚拟机、容器、WSL2 里 PMU 可能不可用或不准确。
- 不同 CPU 的事件名称和语义可能不同。
- 权限设置可能禁止普通用户读取某些事件。
- `perfstat` 默认测整个进程，不自动知道你的热路径在哪里。

所以本章只要求你能运行、保存输出、知道限制。第 11 章和第 12 章会把测量结构系统化。

strace：观察程序和内核的边界

有些问题不是单靠源码和反汇编能快速看出来的。程序为什么找不到文件？为什么启动时读了很多路径？为什么一直阻塞？为什么动态库加载失败？这类问题常发生在用户程序和操作系统边界，`strace`

可以记录系统调用。

基本用法：

```
strace -o results/app.strace.txt ./app
strace -f -o results/app.strace.txt ./app
strace -e trace=openat,read,write,execve ./app
```

strace 输出的每一行大致是一次系统调用及其返回值。例如打开文件、读取、写入、创建进程、映射内存、退出。若程序报“文件不存在”，可以在 **strace** 中搜索 **ENOENT**；若程序启动很慢，可以看看是否反复查找动态库、配置文件或字体文件；若程序卡住，可以看最后停在哪个系统调用。

strace 的定位价值在于分层。比如程序运行失败，GDB 看到的可能只是某个库函数返回错误；**strace** 能告诉你底层系统调用返回了什么错误码。再比如 benchmark 结果异常慢，**strace** 可以帮助确认 timed region 外是否意外发生大量 I/O 或系统调用。

但 **strace** 会显著改变程序运行速度，不适合直接做性能测量。它是诊断工具，不是 benchmark 工具。报告中使用 **strace** 时，应说明它用于解释行为，不用于给出时间结论。

常见误区

strace 看到的是用户态程序请求内核服务的边界，不是 CPU 执行的全部细节。它能解释文件、进程、内存映射和系统调用问题，但不能替代反汇编、GDB 或 PMU。

环境记录：实验从机器档案开始

每次实验都应该有环境记录。最低内容：

```
mkdir -p results
{
  date
  uname -a
  lscpu
  g++ --version
  clang++ --version
  cmake --version
} > results/environment.txt 2>&1
```

如果使用 Git，还要记录：

```
git rev-parse HEAD >> results/environment.txt
git status --short >> results/environment.txt
```

为什么要记录 dirty status？因为未提交修改会影响实验可复现性。报告里写一个 commit，但工作区还有未记录改动，别人按 commit 复现时可能得到不同结果。

命令记录：让实验能重放

建议每个实验维护一个 **commands.txt**：

```
# build
clang++ -std=c++20 -O3 -DDEBUG src/main.cpp -o build-release/app

# inspect
objdump -drwC -Mintel build-release/app > results/app.objdump.txt
nm -C build-release/app > results/app.nm.txt
readelf -S build-release/app > results/app.sections.txt

# run
./build-release/app --size 1048576 --repetitions 31 \
> results/run.txt 2>&1
```

注意：命令记录不是日记。它应该能被另一个人复制执行。不要写“运行程序看看”，要写完整命令、参数、工作目录和输出路径。

报告结构：把观察变成知识

本章开始，所有实验报告至少包含：

- 实验目标：本次要验证什么。
- 环境：机器、系统、编译器、构建类型。
- 源码：相关文件路径或关键片段。
- 命令：完整编译、观察、运行命令。
- 工具输出：反汇编、符号表、GDB 记录或测量数据。
- 解释：把输出和模型对应起来。
- 限制：哪些结论不能从本实验推出。
- 下一步：还需要什么实验验证。

好的报告不一定长，但必须可审计。比如“-O3 更快”不是合格结论；“在这个输入规模下，-O3 版本被内联并生成了更短的循环，原始样本显示 median 从 X 变到 Y，但本实验没有控制 CPU 频率，所以只能作为初步趋势”才像工程结论。

证据链模板：从问题到工具输出

底层学习最需要训练的是证据链，而不是命令记忆。一个合格证据链应该能回答：我为什么运行这个工具，工具输出里的哪一行支持我的判断，还有哪些可能解释没有排除。

下面给出几类常见问题的证据链模板。

证明一个函数是否存在独立机器码

问题：源码里有 `foo`，优化后二进制里是否还有独立函数体？

建议证据：

1. 保存编译命令和构建类型。
2. 用 `nm-Cpp|rg"foo"` 查看符号是否存在。
3. 用 `objdump-drwC-Mintelapp` 搜索 `<foo>` 标签。
4. 在调用者反汇编中查找是否有 `call` 到 `foo`。
5. 若没有独立符号，检查是否被内联、删除或名称改写。

结论写法示例：在当前 -O3 构建下，`foo` 没有出现在最终可执行文件符号表中，调用者也没有 `call` 到 `foo`，因此当前证据支持“该函数在此构建中被内联或删除”。如果要区分内联和删除，还需要看调用点语义和生成指令。

证明一次崩溃来自某个非法地址

问题：程序 SIGSEGV，是否因为当前指令访问了非法地址？

建议证据：

1. 在 GDB 或 core dump 中记录信号。
2. 记录 `rip` 和当前指令。
3. 记录参与地址计算的寄存器。
4. 计算有效地址。
5. 查看 `/proc/<pid>/maps` 或 core 对应映射，判断地址是否属于合法映射和权限是否允许。
6. 若使用 sanitizer，也保存 sanitizer 报告作为补充证据。

结论写法示例：当前指令是从 `rdi` 指向地址读取 4 字节，崩溃时 `rdi` 为 0，因此这是一次空指针附近地址的非法读取。这个结论来自机器状态，不只是源码猜测。

证明性能实验测的是目标代码

问题：benchmark 变快了，如何证明测的是刚刚修改的目标代码？

建议证据：

1. 记录 Git commit、dirty status 和 diff 摘要。
2. 保存完整构建命令或 verbose build。

3. 保存可执行文件时间戳和路径。
4. 保存目标函数反汇编。
5. 保存 benchmark 输入规模、重复次数和原始样本。
6. 若结论依赖硬件事件，保存 `perfstat` 命令和输出。

结论写法示例：当前结果对应 `build-release/app`，该文件在本次构建日志之后生成；反汇编显示目标函数包含四路展开循环；raw samples 保存于 `results/raw.csv`。因此至少可以确认测量对象不是旧二进制。至于变快是否来自展开减少分支，还需要更多控制实验。

心智模型

工具输出不是越多越好。每一份输出都应该回答一个明确问题，并在报告中被引用到具体判断。

工具选择决策树：先问问题再选命令

初学者常见做法是遇到问题就把所有工具都跑一遍：`file`、`ldd`、`objdump`、`readelf`、`gdb`、`strace`、`perf`。这样看起来很努力，但输出越多，越容易迷失。更好的方法是先把问题分类，再选择最短证据路径。

程序没有生成

如果程序没有生成可执行文件，先不要看 GDB，也不要看 `perf`。问题还停留在构建阶段。

检查顺序：

1. 编译命令是否真的执行，退出码是否为 0。
2. 错误来自编译阶段还是链接阶段。
3. 若是编译错误，看具体翻译单元、include path、宏定义、语言标准。
4. 若是链接错误，用 `nm-C` 检查目标文件里符号定义和引用。
5. 若是构建系统问题，查看 `verbose build` 和 `compile_commands.json`。

典型工具：

```
cmake --build build-release --verbose
nm -C build-release/*.o | rg "symbol_name"
rg "symbol_name" src include
```

这类问题的证据是构建日志和符号表，不是运行时输出。

程序生成了但无法启动

如果可执行文件存在，但运行时报 “permission denied” “command not found” “No such file or directory” “cannot open shared object file”，通常先看文件身份、权限、解释器和动态库。

检查顺序：

1. 路径是否正确，是否运行了旧文件。
2. 文件是否有执行权限。
3. `file` 是否显示目标架构匹配当前机器。
4. 动态库是否能找到。
5. 当前工作目录和环境变量是否符合预期。

典型工具：

```
pwd
ls -l build-release/app
file build-release/app
ldd build-release/app
readelf -d build-release/app | rg 'NEEDED|RPATH|RUNPATH'
```

不要直接改源码逻辑。程序还没真正进入你的业务代码，错误可能来自启动环境。

程序启动后崩溃

如果程序启动后 **SIGSEGV**、**SIGABRT** 或 sanitizer 报错，问题进入运行时状态。此时需要定位崩溃点、调用链、寄存器、内存地址和对象生命周期。

检查顺序：

1. 用 Debug 或 sanitizer 构建复现。
2. GDB 中查看 backtrace 和当前指令。
3. 查看寄存器和参与地址计算的值。
4. 若有 core dump，确认 core 和二进制匹配。
5. 使用 sanitizer 报告补充对象边界和生命周期信息。

典型工具：

```
gdb -q ./build-debug/app
bt
info registers
x/10i $rip
disassemble /r $rip-32, $rip+32
```

如果崩溃只在 Release 出现，不要马上说“编译器 bug”。先检查未定义行为、未初始化数据、越界、use-after-free、数据竞争和 ABI 破坏。

程序结果不对

结果错误不一定会崩溃。它可能来自算法 bug、输入错误、未定义行为、浮点误差、ABI 不匹配、链接到错误库、读取了错误文件。此时要先建立 reference，再比较数据流。

检查顺序：

1. 输入文件、命令行参数和工作目录是否正确。
2. reference 实现是否给出不同结果。
3. Debug 和 Release 结果是否一致。
4. sanitizer 是否报告错误。
5. 若涉及跨语言或手写汇编，检查 ABI 和寄存器保存。

典型工具：

```
strace -e openat ./app input.txt
diff expected.txt actual.txt
gdb -q ./app
objdump -drwC -Mintel ./app > results/app.objdump.txt
```

这里 **strace** 可以帮助确认程序实际打开了哪个文件；GDB 和反汇编帮助确认数据如何流过函数边界。

程序结果正确但性能异常

性能异常是最后一类问题，不应跳过前面的正确性和二进制身份检查。先确认测的是当前二进制、结果正确、timed region 干净，再讨论 cache、分支、频率和 PMU。

检查顺序：

1. 构建命令、Git 状态、二进制时间戳是否匹配。
2. benchmark 是否保存 raw samples。
3. 结果是否通过 reference。
4. 反汇编是否显示目标代码存在。
5. 输入规模、distribution、batch、warmup 是否记录。
6. 环境和 PMU 证据是否支持性能解释。

典型工具：

```
git rev-parse HEAD
git status --short
objdump -drwC -Mintel build-release/bench > results/bench.objdump.txt
```

```
perf stat -r 10 ./build-release/bench
```

性能问题最容易被讲成故事。工具选择决策树的作用，就是让你先排除低层错误，再进入高层解释。

核心思想

工具选择不是“会不会用某个命令”，而是能否把问题定位到构建、启动、运行、正确性或性能层。问题层次错了，工具再多也只会制造噪声。

把事实、解释和建议分开写

实验报告最常见的问题不是数据少，而是把事实、解释和建议混在一起。工程报告应该让读者清楚知道：哪些是工具直接给出的事实，哪些是你基于模型做出的解释，哪些是下一步行动。

事实是可以被复查的内容。例如：

- 编译命令是某一条具体命令。
- `objdump` 显示某个函数中有一条 `call`。
- `nm` 显示某个符号是 `undefined`。
- GDB 中 `rdi` 在崩溃时等于某个地址。
- benchmark 原始样本有若干个数值。

解释是你对事实的模型化理解。例如：

- 这个 `call` 存在，说明该构建下函数没有被内联到调用点。
- `rdi` 是空值，所以当前崩溃可能来自空指针解引用。
- Release 版本更快，可能是循环被展开并减少了分支开销。
- 随机访问更慢，可能是 cache miss 和 TLB miss 增多。

建议是下一步要做什么。例如：

- 增加 sanitizer 构建确认是否存在越界。
- 保存最终可执行文件的反汇编，而不只看目标文件。
- 用更大的样本数重新测量。
- 增加 PMU 事件验证 cache miss 假设。

报告里应当避免把解释伪装成事实。比如“这个程序慢是因为 cache miss”如果没有计数器、访问模式分析或控制实验支持，就应该写成“当前假设是 cache miss 增多”。这种写法不是软弱，而是严谨。

最小可复现实验包

后续每个章节实验都建议形成一个最小可复现实验包。它不一定要复杂，但至少包含：

- 源码文件或源码路径。
- 构建脚本或完整命令。
- 环境记录。
- 原始工具输出。
- 原始运行结果。
- 简短报告。

目录可以这样组织：

```
results/
  README.md
  environment.txt
  commands.txt
  build.verbose.txt
  app.objdump.txt
  app.nm.txt
  app.readelf-sections.txt
```

```
gdb-session.txt
raw-samples.csv
report.md
```

README.md 用来说明如何重新运行实验；commands.txt 保存命令；report.md 写事实、解释和限制。原始数据不要只嵌在报告截图里。截图适合展示，不适合审计和后续处理。

核心思想

可复现不是“我还能再做一遍”，而是“另一个人在相近环境中可以根据记录复查我的观察，并知道哪些差异来自环境变化”。

Git：给实验一个源码身份

实验不只依赖机器和命令，也依赖源码状态。Git 的作用不是只在提交代码时才出现；它应该进入实验记录。

最低记录：

```
git rev-parse --short HEAD
git status --short
git diff --stat
```

HEAD 告诉你当前提交；status--short 告诉你工作区是否有未提交修改；diff--stat 告诉你修改大致涉及哪些文件。若工作区很脏，报告只写 commit hash 是不够的，因为实验运行的源码不等于该 commit。

对严肃实验，可以保存补丁：

```
git diff > results/working-tree.patch
```

这样即使没有提交，别人也能看到实验相对 commit 做了什么修改。对于教学项目，保存 patch 能帮助教师判断学生到底运行了哪份代码；对于性能项目，保存 patch 能避免“我记得只改了一行”的不可靠记忆。

不要把生成文件和源码混为一谈

Git status 里经常同时出现源码修改和生成文件修改。源码修改影响程序语义；生成文件可能只是构建产物、PDF、日志、CSV、索引。报告中要区分它们。比如 LaTeX 工程里 main.pdf 是构建产物，chapters/...tex 才是正文源文件；C++ 工程里 build/ 下的目标文件通常不应当被当作源码证据。

这一区分对实验复现很重要。源码和脚本决定如何重新生成产物；产物用于审计某次运行结果。两者都可以保存，但角色不同。不要把“我有一个二进制文件”当成“我能复现这个二进制”。

提交粒度和实验粒度

一个实验最好对应清晰的源码变化。如果一次提交同时改算法、改输入、改构建参数、改 benchmark harness，再看到性能变化，就很难归因。工程实践中，建议把语义修改、测量框架修改和报告修改分开，至少在报告里说明哪些变化属于被测对象，哪些变化属于实验系统。

例如：

```
code change:
  replace scalar sum with unrolled loop

harness change:
  add CSV output and environment record

report change:
  add result table
```

只有 codechange 才应作为性能比较的主要自变量；harnesschange 可能改变测量方法，应单独说明。

工具选择的优先级

底层学习工具很多，但初学阶段不应该陷入工具收集。先建立优先级。

第一优先级是构建和运行工具：编译器、构建系统、Shell、Git。没有稳定构建，后面所有观察都不可靠。

第二优先级是二进制观察工具：`objdump`、`nm`、`readelf`。它们回答“最终程序里有什么”。

第三优先级是动态调试工具：GDB、sanitizer、必要时 core dump。它们回答“程序运行到某个状态时发生了什么”。

第四优先级是测量工具：`time`、benchmark harness、`perfstat`、后续更专业的 profiling 工具。它们回答“运行代价是多少，瓶颈假设是否成立”。

不要反过来。一开始就追求复杂 profiler，却没有保存编译命令和反汇编，很容易得到无法解释的图表。优秀的性能工程不是工具越多越好，而是每个工具回答一个明确问题。

故障定位地图：按现象选择工具

工具学习最终要服务问题定位。面对一个异常现象时，先判断它属于哪一类，再选工具。

现象	先问的问题	首选工具
编译失败	哪个翻译单元、哪个诊断、哪个参数。	编译器输出、 <code>compile_commands.json</code>
链接失败	哪个符号未定义或重复定义。	链接命令、 <code>nm</code> 、 <code>readelf-s</code>
启动失败 崩溃	是否缺库、权限、路径或入口。 哪条指令、哪个地址、什么调用栈。	<code>ldd</code> 、 <code>strace</code> 、 <code>file</code> GDB、core dump、sanitizer
结果错误	reference 是否可靠，输入是否一致。	单元测试、sanitizer、GDB
汇编不符预期 性能异常	是否构建错、被内联、被删除。 是否测错对象，环境是否变化。	<code>objdump</code> 、 <code>nm</code> 、构建日志 raw data、 <code>objdump</code> 、 <code>perfstat</code>

这个地图不是死规则，但能避免乱用工具。比如链接失败时开 GDB 没意义，因为程序还没有生成；崩溃时先看 `bt` 和 `rip`，不要先改优化参数；性能异常时先确认二进制和计时区域，再谈 cache 和分支预测。

从外到内排查

一个实用顺序是：

1. 命令是否成功，退出状态是否为 0。
2. 构建产物是否是刚生成的，路径是否正确。
3. 构建参数是否符合实验目标。
4. 程序输入是否正确，输出是否保存。
5. 二进制内容是否符合预期。
6. 运行时状态是否合理。
7. 性能数据是否能支撑结论。

这个顺序看似保守，但能节省大量时间。很多复杂解释最后都败在第一步：命令失败、路径错、旧二进制、输入错、构建类型错。

核心思想

底层调试的第一原则是先确认观察对象，再解释观察结果。对象错了，解释越精密越危险。

诊断报告模板：让排障过程可复查

工具命令只有进入报告，才会变成可复查的工程知识。一个诊断报告不需要很长，但结构必须稳定。推荐模板如下：

```
Title:
  one-line symptom

Context:
  repository / commit / dirty state
  build type and command
  machine and OS

Expected:
  what should happen

Observed:
  exact command
  exit status
  key output

Scope:
  which inputs fail
  which builds fail
  whether failure is deterministic

Evidence:
  compiler diagnostics / linker output
  objdump / nm / readelf
  gdb backtrace / registers
  sanitizer / strace / perf as needed

Hypothesis:
  current explanation and why

Action:
  next experiment or fix

Boundary:
  what has not been checked
```

这个模板强制你把症状、证据和解释分开。很多排障失败，是因为一开始就写“应该是 cache 问题”“应该是编译器 bug”“应该是链接错了”，然后只寻找支持这个想法的证据。模板要求先写 observed，再写 hypothesis，可以减少这种偏差。

症状要具体

“程序坏了”不是可诊断症状。更好的写法是：

- `clang++-02` 链接时报 `undefinedreferencetoasm_dot_i32`。
- Release 构建在输入 `n=0` 时 SIGSEGV，Debug 构建未复现。
- `sum_u64` candidate 在 64 MiB 输入上 median 比 baseline 慢 18%。
- 动态库版本从 `libfoo.so.1` 变成 `libfoo.so.2` 后结果不同。

具体症状包含命令、构建类型、输入、错误形式和对比对象。症状越具体，第一批证据越明确。模糊症状会导致工具乱用。

复现矩阵比单次复现更有用

当问题不明显时，建立小复现矩阵：

构建	输入	机器/环境	结果
Debug	small	local	pass
Release	small	local	pass
Release	n=0	local	crash
Release+ASan	n=0	local	heap-buffer-overflow

矩阵能帮助你定位变量。若只有 Release 失败，考虑优化暴露未定义行为或构建差异；若只有

某个输入失败，考虑边界条件；若只有某台机器失败，考虑 ABI、ISA、库版本或环境；若 sanitizer 失败，优先修 correctness，再谈性能。

从证据到修复：不要跳过最小验证

找到疑似原因后，不要直接做大改。先设计最小验证。最小验证的目标是证明“如果这个解释正确，那么改变一个小条件应该改变现象”。

例一：链接失败显示未定义符号。假设是源文件没加入构建。最小验证不是重写函数，而是查看构建日志中是否编译了对应 `.cpp` 或 `.S`，查看目标文件中是否有该符号，临时把源文件加入 `target` 后链接是否通过。

例二：崩溃发生在 `movaps`。假设是栈或内存未对齐。最小验证是用 GDB 在崩溃前打印 `rsp%16` 或目标地址低位，检查调用点栈调整，而不是立即把所有 SIMD 指令换成 `unaligned` 版本。

例三：benchmark 变快 100 倍。假设是循环被优化器删除。最小验证是查看反汇编是否还有循环，给结果加 `do_not_optimize` 或 `checksum` 后是否恢复正常，而不是直接接受 `speedup`。

例四：随机访问慢。假设是 cache/TLB。最小验证是做输入规模扫描、比较顺序访问、收集 PMU 事件。不要直接改数据结构，因为可能真正瓶颈是分支或哈希计算。

修复后要回归到证据

修复完成后，报告要更新证据，而不是只写“已修复”。链接问题要保存新的链接命令和 `nm` 输出；崩溃问题要保存 sanitizer 通过或 GDB 状态；ABI 问题要保存寄存器/栈对齐检查；性能问题要保存 raw samples 和反汇编。修复是否有效，要由同一类证据验证。

如果修复改变了实验系统，也要说明。例如把 benchmark 从 batch 1 改成 auto batch，会改变测量协议；把 Debug 改成 Release，会改变二进制；把输入从随机改成全零，会改变 workload。修复不能偷偷改变问题。

工具输出的保存规则

工具输出要保存原始版本，也要写摘要。只保存摘要，后续无法复查；只保存原始输出，读者不知道重点。建议每个实验目录同时包含：

- `commands.txt`：完整命令。
- `environment.txt`：机器、系统、编译器、Git 状态。
- `raw/`：原始工具输出。
- `summary.md`：人工摘要。
- `report.md`：问题、证据、解释、限制。

原始输出文件命名要包含工具和对象，例如：

```
app.objdump-drwC-Mintel.txt
app.readelf-dynamic.txt
app.readelf-relocations.txt
app.nm-C.txt
gdb-crash-session.txt
perf-stat-r10.txt
strace-openat.txt
```

这样的命名看起来繁琐，但半年后你仍然能知道每个文件怎么来的。不要用 `output.txt`、`log2.txt` 这类名字，它们会很快失去意义。

输出过大时保存关键切片

有些输出很大，例如完整 `objdump`、完整 `strace`、长时间 `perfrecord`。可以同时保存完整文件和关键切片。关键切片应说明截取规则：

```
objdump excerpt:
    function asm_dot_i32 and its callers

strace excerpt:
    openat failures for config file

perf excerpt:
```

```
summary stat and top 20 symbols
```

不要只贴截图。截图无法搜索、无法 diff、无法被脚本处理。截图可以作为报告辅助，但原始文本更适合工程审计。

调试中的语言边界

工具看到的是机器状态，程序员关心的是语言对象。调试的难点在于把两者正确对应。GDB 中一个变量显示为 `<optimizedout>`，不代表程序没有这个概念；它可能被优化器删除、合并、放在寄存器或无法从当前位置恢复。反汇编中一个栈槽存在，也不代表一个 C++ 对象仍然处于生命周期内；它可能只是复用的存储。

sanitizer 报告通常更接近语言边界。AddressSanitizer 可以告诉你越界、use-after-free、栈使用后返回等问题；UndefinedBehaviorSanitizer 可以发现某些 signed overflow、越界移位、空指针引用等问题。但 sanitizer 也会改变二进制、内存布局 and 性能。它适合 correctness 诊断，不适合直接代表性能。

调试优化代码时，要接受源码行和机器指令不再一一对应。一个源码行可能生成多条指令，一个指令可能来自多个源码表达式，一个变量可能在不同路径上有不同位置。此时更可靠的方法是：用函数签名建立入口状态，用反汇编追踪寄存器和内存，用源码语义解释为什么这种机器路径合法。不要强求调试器把优化后的程序还原成未优化源码。

从工具熟练到工程熟练

会敲命令只是工具熟练；能选择证据、组织报告、判断边界，才是工程熟练。两者的区别可以这样看：

场景	工具熟练	工程熟练
链接失败	会运行 <code>nm</code>	能判断声明、定义、库顺序和名字修饰哪一层错
崩溃	会打开 GDB	能从 <code>rip</code> 、地址公式和对象生命周期定位证据
性能异常	会运行 <code>perf</code>	先确认二进制、计时区域、correctness 和 raw samples
汇编阅读	会看 <code>objdump</code>	能恢复 ABI 入口、数据流、控制流和推论边界
报告	会贴输出	能区分事实、解释、建议和未验证假设

第一册的工具章节目标不是让你背命令大全，而是让你建立工程熟练度。以后遇到新工具，也应先问它回答什么问题、需要什么输入、输出有什么边界、会不会扰动被观察对象。

综合案例：定位一次 Release-only 崩溃

假设程序 Debug 正常，Release 在某个输入下崩溃。不要先说“优化器 bug”。按工具流程排查。

第一步，确认复现命令：

```
./build-release/app --input cases/bad.bin
echo $?
```

记录退出状态和 stderr。确认 Debug 使用同一个输入：

```
./build-debug/app --input cases/bad.bin
```

第二步，确认二进制身份：

```
realpath build-release/app
ls -l build-release/app
file build-release/app
```

```
git rev-parse HEAD
git status --short
```

如果路径错或工作区脏，先记录，不要继续假设。
第三步，用 sanitizer 构建缩小 correctness 问题：

```
cmake --preset asan
cmake --build build-asan
./build-asan/app --input cases/bad.bin
```

如果 ASan 报 use-after-free 或越界，优先修语言错误。Release-only 崩溃常常是未定义行为在优化后暴露，而不是优化器错误。

第四步，如果 sanitizer 没有复现，用 GDB 看崩溃点：

```
gdb -q --args ./build-release/app --input cases/bad.bin
run
bt
info registers
x/8i $rip
disassemble /r $rip-32,$rip+32
```

记录 rip、参与地址计算的寄存器、当前函数和调用栈。若崩溃指令是 load/store，计算有效地址；若是 ret，怀疑栈破坏；若是间接 call 或 jmp，追踪函数指针或虚表来源。

第五步，检查反汇编和源码对应：

```
objdump -drwC -Mintel build-release/app > results/app.objdump.txt
nm -C build-release/app > results/app.nm.txt
```

确认崩溃函数是否被内联、是否有多个版本、是否走了某个 dispatch。如果源码行和机器码不直观，按寄存器和地址公式分析，不要强行相信源码单步。

第六步，写结论边界。一个合格的临时结论可能是：

```
Observed:
  Release build crashes on cases/bad.bin at mov eax, [rdi+rax*4].

Evidence:
  rdi points to buffer base.
  rax = -1 sign-extended to 64-bit.
  ASan build reports heap-buffer-overflow in the same logical path.

Explanation:
  Lower-bound check for index is missing. Release exposes the invalid load.

Action:
  Add explicit 0 <= index check and regression test for index = -1.
```

这个案例展示了工具链如何协作：命令确认复现，Git 和 file 确认对象，sanitizer 提供语言边界证据，GDB 提供机器现场，objdump 提供二进制结构，报告把证据变成修复行动。

综合案例：性能异常但功能正确

另一个常见场景：程序结果正确，但新版本 benchmark 慢。排查顺序不同。

第一步，确认 benchmark 输出包含 correctness：

```
status = ok
checksum = expected
```

如果 correctness 不通过，停止性能分析。

第二步，检查 raw samples。是所有样本都慢，还是只有少数离群点？如果只有少数离群点，先看调度、page fault、系统干扰；如果整体慢，再看二进制和输入。

第三步，检查构建参数。确认两个版本使用同一个优化级别、同一个 -march、同一个链接方式。不同构建参数会让比较失去归因。

第四步，审计热路径反汇编。确认新版本是否内联失败、向量化失败、走到 fallback、引入额外

分支或调用。若反汇编不同，要把差异写成事实，不要直接归因。

第五步，设计对照实验。如果怀疑分支，改变输入分布；如果怀疑内存，扫描规模；如果怀疑 dispatch，强制实现；如果怀疑动态库，记录 ldd 和 maps；如果怀疑频率，固定核心并重复。

第六步，输出结论等级。若只有时间差，没有二进制证据，只能说“观察到变慢”；若有二进制证据但无 PMU，只能提出机制假设；若有 raw、二进制、PMU 和对照，才可以更强地解释原因。

核心思想

功能调试和性能调试使用许多相同工具，但问题顺序不同。功能问题先找定义良好和状态错误；性能问题先确认 correctness、测量对象和二进制路径。

深入讲解：工具输出为什么必须可重复

调试时，很多人会在终端里反复试命令，看到一个结果后凭记忆写结论。这在小问题上可以凑合，但在底层工程中风险很高。因为工具输出依赖当前目录、环境变量、构建目录、时间、进程 PID、ASLR、输入文件和系统状态。没有记录，别人无法复查，你自己第二天也可能复现不了。

可重复的工具输出至少满足三个条件。第一，命令完整，包括工作目录、参数、重定向和环境。第二，输入可得，包括源码、二进制、数据文件和配置。第三，输出保存为文本或结构化文件，而不是只存在终端滚动缓冲或截图。

例如，objdump 输出必须说明对象是目标文件还是最终可执行文件，是否使用 -d、-r、-w、-C、-Mintel。不同选项显示的信息不同。perfstat 输出必须说明重复次数、事件列表和命令。gdb 会话至少要保存关键命令和输出，例如 backtrace、寄存器和崩溃指令。

可重复不是形式主义。它能让你比较两次构建的差异，能让 reviewer 检查证据，能让未来的你复盘错误。很多性能争议最后不是输在算法，而是输在“没人知道当时到底怎么测的”。

深入讲解：最小复现不是越小越好，而是刚好包含关键条件

最小可复现实例常被误解为“代码越短越好”。真正的最小复现，是删除无关因素后仍然保留问题触发条件。若问题来自动态库路径，复现必须包含动态库；若问题来自优化级别，复现必须保留相同优化；若问题来自 ABI 和手写汇编，复现必须保留跨文件调用边界；若问题来自输入规模，复现必须保留足够大的数据。

一个过度简化的复现可能把 bug 删掉。比如 Release-only 崩溃，如果你把代码改成单文件 -O0，问题消失，不代表修复了；只是删除了优化和链接条件。一个好的复现会明确写：

```
kept:
-03
separate translation units
same struct layout
same failing input boundary

removed:
logging
unrelated command-line parser
UI code
network dependency
```

这样别人知道哪些条件是必要的，哪些只是原项目噪声。最小复现的价值不在于短，而在于能隔离原因。

深入讲解：工具学习的迁移方法

本章讲了 objdump、nm、readelf、GDB、strace、perf 等工具。以后你会遇到更多工具，比如 llvm-objdump、eu-readelf、bpftrace、heaptrack、valgrind、VTune、uprofile、perfetto。不要把工具学习变成背命令。迁移方法是问四个问题。

第一，它观察哪一层？源码、目标文件、可执行文件、进程、系统调用、硬件事件、内存分配还是时间线。第二，它需要什么输入？二进制、调试信息、root 权限、PMU 权限、符号文件、运行命令还是 trace。第三，它会扰动什么？是否改变时间、内存布局、调度、优化、系统调用路径。第四，它的输出边界是什么？是精确事件、采样估计、符号化结果、还是工具推断。

掌握这四个问题，新工具也能快速归类。你不一定立刻熟悉所有选项，但能判断它适不适合当

前问题。工具越高级，越需要边界意识。没有边界意识，漂亮图表和复杂报告只会让错误结论更难被发现。

深入讲解：Linux 命令行的真正价值是可组合

命令行工具的价值不只是“能敲命令”，而是可组合。一个复杂问题通常需要多个工具串起来：用 `file` 确认文件类型，用 `ldd` 看动态库，用 `readelf` 看动态段，用 `objdump` 看机器码，用 GDB 看运行状态，用 `strace` 看系统调用，用 `perf` 看计数。每个工具只回答一小类问题，组合起来才形成证据链。

可组合的前提是文本输出和文件化记录。把输出重定向到文件，可以搜索、diff、归档和写入报告。比如：

```
objdump -drwC -Mintel ./app > results/app.objdump.txt
readelf -Ws ./app > results/app.symbols.txt
readelf -r ./app > results/app.relocations.txt
nm -C ./app > results/app.nm.txt
```

后续你可以用 `rg` 搜函数名，用 `diff` 比较两个构建，用脚本提取某些符号。若只在终端里看一眼，就失去了这些能力。

命令行还鼓励小步验证。与其打开一个大型 IDE 面板猜测，不如先问一个具体问题：“这个符号是否存在？”运行 `nm`；“这个库从哪里加载？”运行 `ldd` 或看 `maps`；“这个函数有没有 `call`？”运行 `objdump` 搜索。每个问题小，证据清楚，排查速度反而更快。

管道不是为了炫技

管道可以把一个工具输出变成另一个工具输入，例如：

```
readelf -Ws ./app | rg 'asm_dot|sum_u64'
objdump -drwC -Mintel ./app | rg -n 'call|ret|sum_u64'
cat raw_samples.csv | rg ',checksum_failed,'
```

这不是为了写复杂一行命令，而是快速缩小观察范围。正式报告仍应保存原始完整输出，管道结果可以作为摘要。不要只保存过滤后的输出，因为过滤条件可能漏掉重要信息。

深入讲解：调试器中的地址如何回到源码

GDB 中看到一个地址，例如 `0x5555555561a0`，它本身只是虚拟地址。要把它变成源码理解，需要几步。

第一，判断地址属于哪个映射。用 `info proc mappings` 或 `/proc/<pid>/maps`。如果地址落在主程序 `r-xp` 映射中，它可能是主程序代码；如果落在共享库映射中，就要找对应库；如果落在栈或堆，说明它不是普通代码地址。

第二，找到对应符号。GDB 的 `info symbol 0x...` 或 `addr2line` 可以利用符号和调试信息。没有调试信息时，仍可用模块基址加偏移，在 `objdump` 中定位附近指令。

第三，确认二进制匹配。ASLR 会改变运行时加载地址，但模块内偏移应与该二进制一致。如果你用另一个构建的二进制查地址，源码行可能错。`build-id` 和文件时间能帮助确认。

第四，回到源码语义。即使找到源码行，也要记住优化可能让一条指令对应多个源码表达式。最终仍要看寄存器、内存和反汇编，不能只看源码行号。

这个过程在崩溃分析中非常重要。很多错误报告只给了地址和 `backtrace`；你要能把它们连接到正确二进制和源码，否则修复可能瞄错目标。

课堂讲解：为什么工具链学习要从普通命令开始

有些读者希望直接学习高级 profiler 或大型 IDE 调试功能，觉得 `pwd`、`ls`、`file`、`nm`、`readelf` 这些命令太基础。实际工程中，许多严重问题恰恰败在这些基础事实上：运行了旧路径，链接了旧库，二进制架构不对，符号不存在，动态库搜索路径错误，输入文件不是以为的那个。高级工具无法弥补观察对象错误。

普通命令的优势是透明。`pwd` 告诉你当前目录；`realpath` 告诉你真实路径；`file` 告诉你文件类型和架构；`ldd` 告诉你动态库解析；`nm` 告诉你符号；`readelf` 告诉你 ELF 结构；`objdump` 告诉

你机器码。每个命令回答一个朴素问题。朴素问题都回答清楚，复杂工具才有意义。

这也是本章强调命令记录的原因。底层工程不是靠记忆维护的，而是靠可复查事实维护。你可以在报告里写：“当前运行文件是 `/abs/path/build-release/app`，`file` 显示 `x86-64 ELF`，`ldd` 显示加载本地 `libfast.so`，`nm` 显示 `asm_dot_i32` 存在，`objdump` 显示热循环包含预期指令。”这段话比“我确认过了”强很多。

课堂讲解：调试时如何避免破坏现场

调试会改变被调试对象。重新编译可能改变优化和布局；加日志可能改变时序；GDB 断点可能改变线程调度；sanitizer 会改变内存布局；打印变量可能触发函数调用或锁。调试时要有“现场保护”意识。

第一，先保存原始失败证据。包括命令、输入、二进制、core dump、日志、raw samples。不要在没有保存的情况下直接改代码，因为改完可能无法复现原问题。

第二，分离观察和修复。观察阶段尽量少改被测对象；修复阶段再做最小修改；修复后用同一复现命令验证。若观察过程中必须改变构建，比如启用 ASan，要明确这是一种新实验，不是原现场。

第三，记录每次改变。加了一个环境变量、换了输入、改了优化级别、切换了动态库路径，都可能改变结果。调试日志要写这些变化。否则最后即使问题消失，也不知道是修好了，还是改变了触发条件。

第四，对性能问题尤其谨慎。任何日志、断点、sanitizer、调试构建都可能让性能数据失效。功能诊断可以使用扰动工具；性能结论必须回到发布构建和可信测量协议。

工具不是按钮，而是证据系统

初学者常把工具当成按钮：崩溃就按 GDB，慢就按 perf，找不到库就按 ldd，链接失败就按 nm。这样使用工具很容易卡住，因为工具输出不会自动变成结论。工程上更可靠的方式是把工具看成证据系统。每个工具回答一类问题，也有自己的盲区。你要先提出问题，再选择工具，再解释输出。

例如 nm 能告诉你目标文件或库中有哪些符号，符号是定义、未定义、弱符号还是本地符号。但 nm 不能证明运行时一定加载了这个库，也不能告诉你某个路径搜索为什么失败。要回答运行时加载问题，需要 ldd、readelf-d、动态链接器日志、环境变量和实际启动命令。把符号问题和加载问题混在一起，就会在错误工具上浪费时间。

再例如 GDB 能看到某次崩溃的寄存器、栈和指令，但它不能直接证明根因。崩溃点可能只是错误被发现的位置，真正写坏内存的位置可能早得多。此时 sanitizer、watchpoint、core dump、反汇编、日志和最小复现都可能参与。工具链的目标不是让某一个工具说出答案，而是让多个证据互相约束。

把工具输出分成事实、解释和建议

一份高质量调试记录必须区分事实、解释和建议。事实是工具直接给出的内容，例如信号是 SIGSEGV，rip 在某个地址，readelf-d 显示需要某个共享库，objdump 显示某条指令读取 rax 指向的内存。解释是你基于事实做出的判断，例如 rax 为零导致空指针解引用，或者运行时库搜索路径没有包含目标目录。建议是下一步行动，例如增加空指针检查、修复所有权、设置 RPATH、调整部署包。

如果三者混在一起，报告会很难复核。写“程序因为指针错了而崩溃”既不是足够事实，也不是足够解释。更好的写法是：“core 中 rip 指向一条从 rax 偏移 16 读取的指令；崩溃时 rax 为零；调用栈显示该值来自配置解析失败后的返回对象；因此当前证据支持空对象被继续使用。下一步用 sanitizer 和单元测试覆盖配置缺失路径。”这样的结论可复查，也可行动。

综合案例：Release 崩溃怎样从日志追到机器状态

设想一个服务只在 Release 构建中偶发崩溃，Debug 构建无法复现。低质量排查会停在“优化器有 bug”或“指针有问题”。高质量排查要建立证据链。第一步保存崩溃进程的版本身份：提交号、构建参数、二进制校验和、运行命令、配置文件和环境变量。没有这些，后续看到的反汇编可能不是崩溃那份二进制。

第二步获取现场。若有 core dump，用 GDB 打开同一份可执行文件和 core，记录 bt、inforegisters、当前指令、栈顶内存和共享库映射。若没有 core，至少记录日志中的信号、地址、线程、最后操作和部署版本。第三步把崩溃地址映射到模块。通过 `/proc/<pid>/maps` 或 core 中的

映射，确认地址属于主程序还是共享库，再用基址加偏移找到反汇编位置。

第四步解释当前指令。若当前指令是 `load`，要找源寄存器的值和来源；若是 `store`，要找目的地址和写入值；若是 `ret`，要怀疑返回地址或栈被破坏；若是间接调用，要找函数指针或虚表来源。第五步把机器状态回溯到源码路径，检查哪些前置条件没有满足。这里不要因为 GDB 显示了源码行就停止，优化构建下源码行可能对应多个机器位置。

第六步设计复现和验证。若怀疑越界写，使用 `sanitizer`、边界输入和 `watchpoint`。若怀疑 `use after free`，保留分配栈、启用地址检测或替换 `allocator`。若怀疑 ABI 错误，检查调用约定、结构体布局、栈对齐和寄存器保存。修复后必须用同类 Release 构建验证，并保留新旧二进制反汇编差异。这样处理，结论才不是猜测。

综合案例：两台机器结果不同怎么排查

另一类常见问题是同一程序在两台机器上结果不同或性能差异巨大。排查不能从“机器不好”开始。首先要确认程序身份是否相同：源码提交、构建系统、编译器版本、编译参数、链接库、运行参数、输入数据和配置文件。很多所谓机器差异，最后发现是其中一台加载了旧共享库，或者环境变量改变了插件路径。

其次确认二进制身份。使用 `file` 查看架构和调试信息，使用 `sha256sum` 比较文件内容，使用 `ldd` 和 `readelf-d` 比较动态依赖，使用 `objdump` 抽查关键函数。若二进制不同，就先解释为什么不同；若二进制相同，再进入运行环境和硬件差异。

第三步确认运行身份。工作目录、相对路径、权限、文件描述符、CPU 亲和性、容器限制、频率策略、NUMA、内核版本、透明大页、系统负载，都可能影响结果。对于 `correctness` 差异，要优先检查未初始化数据、未定义行为、并发数据竞争、浮点环境和输入文件差异。对于性能差异，要记录 CPU 型号、核心数量、cache、频率、内存通道、编译目标和系统干扰。

最终报告应把差异分层写清：哪些差异已排除，哪些差异被证据支持，哪些还只是可能原因。比如“机器 B 慢”不是结论；“两台机器二进制相同，输入相同，机器 B 的 CPU 频率上限较低且 `perf` 显示 `task clock` 相近但 `cycles` 明显减少，因此当前证据支持频率差异是主要原因”才是可讨论结论。

如何阅读陌生工具输出

面对一个陌生工具，先不要急着复制网上命令。可以按四步阅读。第一，确认工具的输入对象是什么：源码、目标文件、可执行文件、进程、core、系统调用流、硬件事件还是结果文件。第二，确认输出单位是什么：地址、字节、符号、样本、事件计数、时间、比例还是状态码。第三，确认默认过滤和默认汇总。很多工具默认只显示一部分信息，或者把多个线程、进程、事件汇总在一起。

第四，确认输出能回答的问题和不能回答的问题。比如 `strace` 能告诉你系统调用层发生了什么，但不能看到用户态函数内部为什么传了这个路径。反汇编能看到机器码，但不能还原所有源码意图。PMU 事件能显示某类硬件现象，但不能自动判定算法是否合理。工具输出越强大，越需要明确边界。

读工具手册时，优先看三类内容：输入选择、输出字段含义、会改变被观察对象的选项。输入选择决定你是否在看正确对象；字段含义决定你是否读懂数字；扰动选项决定观察是否影响结论。比如采样频率过高可能扰动性能，GDB 单步会彻底改变时序，`sanitizer` 会改变内存布局和执行成本。工具不是中立窗口，它也是实验系统的一部分。

最小复现实验包

调试报告最终要落到一个别人能运行的实验包。最小复现不是把项目删到只剩一行，也不是随便贴一段代码。它的标准是：保留触发问题所需的语义条件，删除和问题无关的工程噪声，并让另一个人在同类环境下能复现观察。

一个合格实验包通常包含源文件、构建脚本、运行脚本、输入数据、预期输出、实际输出、环境记录和报告。若问题是链接失败，实验包要保留多个目标文件或库之间的关系；若问题是动态链接，实验包要保留库路径和 `RPATH/RUNPATH` 设置；若问题是崩溃，实验包要保留触发输入和调试命令；若问题是性能异常，实验包要保留计时协议和 `raw samples`。

最小复现的关键是“可验证地最小”。删除一部分代码后，必须重新运行确认问题仍存在。若问题消失，说明被删部分包含必要条件；若问题仍在，就继续删。这个过程本身也是学习：每删掉一个无关因素，问题边界就更清楚。很多复杂 bug 在最小化过程中已经被定位，因为读者被迫明确哪些

条件真正参与。

最小复现不能破坏问题性质

有些最小化会把问题改坏。例如把 Release 崩溃改成 Debug 构建后不再崩溃，却继续分析 Debug；把多线程竞态改成单线程后问题消失，却仍然声称复现；把动态库问题改成静态链接后，库搜索路径已经不存在；把 benchmark 输入缩小到 L1 cache 后，原来的内存带宽问题被改成了小规模问题。这些都不是有效最小复现。

因此，最小化时要保护问题性质。可以删功能，但不能删掉触发条件。报告应写清保留了哪些条件：优化级别、线程数、输入规模、库边界、符号可见性、CPU 特性、环境变量。若为了方便观察改变了条件，也要说明这是新实验，不是原问题本身。

从证据到修复：不要跳过验证

工具证据支持某个解释后，还需要修复和验证。修复不是“让现象消失”这么简单，而是改变了正确的因果点。若空指针崩溃来自配置解析失败，随手在崩溃处加空指针判断可能避免崩溃，但可能掩盖了配置错误；若链接错误来自库顺序，复制一个库到系统目录可能让本机通过，却破坏部署一致性；若性能异常来自 benchmark 被优化删除，加入 `volatile` 也许能保住代码，却可能引入不真实的内存语义。

修复验证应回到原始复现条件。原问题在什么命令、输入、构建和环境下出现，就尽量用同一条件验证。然后再增加回归测试。功能 bug 可以加单元测试或集成测试；ABI bug 可以加寄存器保存、栈对齐、结构体布局测试；动态链接 bug 可以加部署检查；性能 bug 可以加 benchmark harness 测试和二进制审计。没有回归保护，修复很容易在后续改动中消失。

修复报告应说明排除了什么

高质量修复报告不仅说明改了什么，还说明排除了什么。例如“不是编译器优化器错误，因为用旧二进制和新源码不匹配可以解释现象；不是输入文件损坏，因为 sha256 一致；不是动态库版本问题，因为 `ldd` 和 `/proc/<pid>/maps` 显示加载同一库；最终定位为长度字段溢出，因为边界输入稳定复现，ASan 指向小分配大写入”。这种报告让 reviewer 能跟着证据走。

排除项不要写成主观判断，要写证据。底层问题的排查路径常常比最终补丁更有价值。未来遇到类似问题，团队可以复用这条路径，而不是重新从猜测开始。

自动化工具不能替代工程判断

IDE、sanitizer、静态分析、profiling UI、CI 报警都能提高效率，但它们不能替代工程判断。自动化工具擅长发现模式和收集信息，不擅长理解项目语义。一个 sanitizer 报告能指出越界位置，但不知道这个长度是否来自协议字段；一个 profiler 能显示热点函数，但不知道这个热点是否符合业务目标；一个静态分析 warning 可能是真 bug，也可能是当前抽象无法表达的前置条件。

使用自动化工具时，要把它们纳入证据链。工具发现问题后，仍要确认输入对象、构建参数、路径条件、源码语义和修复策略。工具没有报警，也不等于代码正确；工具报警很多，也不代表都同等重要。工程师的责任是根据风险排序：会改变程序输出、破坏 ABI、导致内存错误、污染性能结论的问题，应优先于纯风格问题。

这也是本章放在汇编和测量之前的原因。后面每个高级主题都会产生更多工具输出。没有证据链思维，工具越多，误判越多；有了证据链思维，工具才会成为放大能力的系统。

调试记录的写作格式

建议把调试记录写成固定格式：现象、环境、复现、观察、假设、验证、修复、回归。现象描述用户看见的问题；环境绑定版本和机器；复现给出命令；观察保存工具输出；假设解释可能原因；验证说明如何支持或推翻；修复说明改动；回归说明以后如何防止。

这种格式能避免“边查边忘”。调试过程中，假设会多次变化。如果不记录，最后很容易只记得成功路径，忘记哪些可能性已被排除。团队协作时，别人也无法接上你的思路。底层问题尤其需要记录，因为证据来自多个工具，分散在终端、日志、core、反汇编和构建输出中。

不要把聊天记录当作唯一报告

聊天工具适合快速沟通，不适合作为唯一证据存档。命令输出会折叠，文件会过期，结论缺少上下文。重要问题应沉淀到仓库中的报告、issue、PR 描述或结果目录。聊天里可以贴摘要，正式证据要有稳定路径。

这条规则对性能问题尤其重要。几周后回看一个性能决策，团队需要看到 raw data、summary、环境和二进制证据，而不是一句“当时测过”。工具章节的目标，就是让每次观察都能留下可追溯痕迹。

从工具学习到 workflow 设计

掌握单个命令只是第一步，更重要的是把命令组织成 workflow。面对崩溃，workflow 可能是：保存 core，确认二进制身份，用 GDB 查看现场，用 maps 定位模块，用 objdump 解释指令，用 ASan 构造复现，用测试锁定修复。面对链接问题，workflow 可能是：查看完整链接命令，定位 undefined symbol，检查目标文件和库，确认符号名和架构，调整依赖关系。面对性能异常，workflow 可能是：确认正确性，保存 raw，审计 timed region，查看反汇编，再看 PMU。

workflow 的价值在于降低临场混乱。底层问题往往信息量大，若每次都从头想工具，很容易遗漏。把常见问题写成 workflow 清单，可以让团队形成共同语言。新人也能按清单收集第一轮证据，资深工程师再基于证据深入分析。

workflow 也要允许修正。若某个项目大量使用插件，就把插件路径和符号可见性加入启动排查；若项目大量手写汇编，就把 ABI 检查加入 code review；若项目重视性能，就把 benchmark artifact 加入 PR 模板。工具章节不是固定命令大全，而是 workflow 设计的起点。

工具输出的保存粒度

保存工具输出有一个平衡。输出太少，无法复查；输出太多，没有索引也难以使用。推荐做法是保留原始完整输出，同时写一个摘要报告。原始输出放在文件中，例如 `objdump.txt`、`readelf.txt`、`gdb.txt`、`perf_stat.txt`；摘要报告引用关键行并解释含义。

对于长输出，应记录生成命令和工具版本。不同版本的 `objdump`、`perf`、编译器和链接器可能格式不同。若输出包含绝对路径或敏感信息，公开前可以脱敏，但原始内部记录应尽量完整。对性能数据，raw CSV 比截图重要；对崩溃，core 和符号文件比手抄 backtrace 更重要。

保存粒度的原则是：未来有人能重建你的观察过程。若未来读者只能看到结论，看不到证据，记录就不够；若能从报告追到命令和原始输出，记录就是可审计的。

工具章节的综合训练

本章最后建议做一次综合训练：故意制造一个小项目，其中包含一个动态库、一个会崩溃的输入、一个 Release 构建和一个简单 benchmark。训练目标不是写复杂程序，而是把证据目录组织完整。目录中至少要有源码提交、构建命令、二进制身份、动态库解析、崩溃现场、反汇编、修复前后测试和 benchmark 原始数据。

完成后，读者应能回答：如果把这个目录交给另一个人，他能否复现崩溃，能否确认运行的是同一二进制，能否理解修复依据，能否判断性能数字是否可信。若答案是否定的，就继续补证据。工具能力的最终形态不是“我会用很多命令”，而是“我能组织一套别人可复查的排查过程”。

案例：把一个模糊 bug 报告改写成工程报告

模糊 bug 报告常写成：“程序偶尔崩溃，怀疑是优化器问题。”这样的报告几乎不能行动。工程报告应改写为：“在某个提交、Release 构建、固定输入和固定命令下，程序多次运行中偶发 SIGSEGV。core 中 rip 位于解析函数偏移处，当前指令从 rax 加偏移读取，崩溃时 rax 为零。Debug 构建未复现。ASan 构建在相同输入下报告 use after free，释放点位于缓存清理路径。当前证据不支持优化器 bug，支持对象生命周期错误。”

这个改写并没有增加神秘技巧，只是把现象、环境、复现、工具证据和解释分开。读者可以检查命令，可以打开 core，可以查看反汇编，可以复跑 ASan。工程报告的价值在于让下一步明确：应审查对象所有权和缓存清理路径，而不是盲目调整优化选项。

同样，性能报告也可以从“新版本快很多”改写成：“在固定输入、固定编译参数、交错运行下，

新版本 median 降低，但 correctness 在两个边界输入失败，因此性能结论无效。”能写出这种报告，说明工具能力已经服务于工程判断。

工具链学习的常见阶段

第一阶段是会运行命令。读者知道 `gdb`、`objdump`、`readelf`、`strace`、`perf` 怎么启动。第二阶段是会解释字段。读者知道符号表、重定位、寄存器、系统调用、事件计数分别表示什么。第三阶段是会组合证据。读者能把崩溃地址、映射、反汇编和源码路径连起来。第四阶段是会设计实验。读者能为了验证一个假设构造最小程序和输入。

本章希望读者至少进入第三阶段，并开始第四阶段。后面章节会不断要求你组合工具，而不是孤立使用工具。汇编阅读需要 `objdump` 和 ABI；互操作需要 `nm`、`readelf` 和测试；benchmark 需要反汇编、raw data 和环境记录。工具链越熟，底层概念越能落地。

工具证据的复盘问题

每次使用工具后，都可以问五个复盘问题。第一，我看的对象是否正确，是源码、目标文件、可执行文件、进程还是 core。第二，输出中的事实是什么，哪些只是我的解释。第三，这个工具有没有改变被观察对象，例如调试器、sanitizer 或 profiler 的扰动。第四，还需要哪个独立证据来支持当前解释。第五，当前结论能否写进报告并让别人复查。

这些问题能防止工具使用变成仪式。运行 `perfstat` 不等于理解性能，打开 GDB 不等于知道根因，生成反汇编不等于读懂汇编。工具只是证据入口，工程判断来自证据之间的连接。复盘问题越固定，排查越稳定。

建议读者在每次实验目录中保留一个 `notes.md`，专门回答这些问题。它不需要华丽，只要诚实。几周后回看，你会看到自己的判断如何从猜测变成证据链，这比记住更多命令更重要。

工具输出如何进入代码审查

底层相关 PR 不应只提交源码差异。若改动影响 ABI、汇编、链接、启动、崩溃或性能，PR 描述应附上相应工具证据。ABI 改动要有符号和调用点证据；汇编改动要有反汇编和寄存器保存说明；动态链接改动要有 `readelf-d` 或运行时加载路径；性能改动要有 raw data、summary 和二进制审计；崩溃修复要有复现和修复后验证。

这样做不是增加流程负担，而是把 review 从主观争论变成证据讨论。reviewer 可以直接检查：“这个符号是否仍导出”“这个调用点是否仍按同一 ABI 传参”“这个 benchmark 是否真的运行了新实现”“这个修复是否覆盖原始复现”。没有工具证据，reviewer 只能凭经验猜，底层改动的风险会被低估。

项目可以把证据要求写成模板。小改动不需要复杂报告，但越接近二进制边界和性能结论，证据要求越高。工具章节的训练最终要服务这种协作方式：每个重要判断都有可点击、可复查、可重新生成的证据。

从个人排查到团队知识库

个人排查解决的是一次问题，团队知识库解决的是同类问题反复出现。每次完成一个有代表性的排查，都可以沉淀成知识库条目：问题现象、适用场景、首选工具、关键命令、典型证据、常见误判、最终修复。比如“动态库加载旧版本”“Release 才崩溃”“benchmark 快得不合理”“结构体布局破坏 ABI”，都适合形成条目。

知识库不需要很长，但要具体。不要写“使用 GDB 调试”，而要写“core 中先确认二进制 build id，再用当前 `rip` 对照 maps 和 `objdump`”。不要写“检查性能”，而要写“先确认 correctness，再看 timed region，再看反汇编，再看 PMU”。具体条目能让新人少走弯路，也能让团队形成统一排查语言。

随着本书后续章节展开，读者可以把每章的常见问题都加入知识库。到第一册结束时，这个知识库会覆盖构建、链接、加载、内存、汇编、ABI、互操作和测量。它将成为继续学习第二册时最实用的基础资产。

常见误区

常见误区

误区一：只要程序能运行，就可以开始优化。

不对。程序能运行不代表没有未定义行为，也不代表结果正确。优化前必须有 reference、测试或不变量。

误区二：GDB 看到的变量就是机器里真实存在的变量。

不准确。变量是源码和调试信息层概念。优化后变量可能在寄存器里、被拆开、被合并，甚至完全不存在。

误区三：objdump 输出和编译器生成的 .s 文件一定一样。

不一定。.s 是汇编器输入，objdump 是机器码反汇编。链接、重定位、汇编器选择、LTO 都可能改变最终形态。

误区四：Release 构建通过了，就不需要 Debug 或 sanitizer。

不对。Release 适合性能路径，Debug 和 sanitizer 适合发现错误。它们互补，不能互相替代。

误区五：终端里看过输出就算记录了实验。

不对。没有保存命令、环境和原始输出，实验就很难复现，也很难审计。

实验：建立你的机器档案

目标：为后续所有实验建立环境基线。

创建目录：

```
mkdir -p labs/ch04_tools/results
cd labs/ch04_tools
```

保存环境：

```
{
  date
  pwd
  uname -a
  lscpu
  g++ --version
  clang++ --version
  cmake --version
  perf --version
} > results/environment.txt 2>&1
```

报告必须回答：

1. 当前 CPU 型号是什么。
2. 有多少 core 和 hardware thread。
3. L1/L2/L3 cache 信息是否能从 lscpu 看到。
4. 当前编译器版本是什么。
5. 当前环境是裸机、虚拟机、容器还是 WSL2；这对性能实验有什么影响。

实验：单文件四种构建

目标：理解 Debug、Release、sanitizer 和 benchmark build 的差异。

写 src/build_modes.cpp：

```
#include <stdint>
#include <stdio>

extern "C" std::int64_t sum_i32(std::int32_t const* a, int n)
{
    std::int64_t s = 0;
    for (int i = 0; i < n; ++i) {
```

```

        s += a[i];
    }
    return s;
}

int main()
{
    std::int32_t a[4] = {1, 2, 3, 4};
    std::printf("%lld\n", static_cast<long long>(sum_i32(a, 4)));
}

```

分别构建：

```

mkdir -p build-debug build-release build-sanitize results

g++ -std=c++20 -Wall -Wextra -O0 -g \
    src/build_modes.cpp -o build-debug/app

g++ -std=c++20 -Wall -Wextra -O3 -DNDEBUG \
    src/build_modes.cpp -o build-release/app

g++ -std=c++20 -Wall -Wextra -O1 -g \
    -fsanitize=address,undefined \
    src/build_modes.cpp -o build-sanitize/app

```

保存反汇编：

```

objdump -drwC -Mintel build-debug/app > results/debug.objdump.txt
objdump -drwC -Mintel build-release/app > results/release.objdump.txt
objdump -drwC -Mintel build-sanitize/app > results/sanitize.objdump.txt

```

报告必须比较：

1. `sum_i32` 在三种构建里是否存在独立函数体。
2. Debug 版本是否使用更多栈访问。
3. Release 版本是否内联或优化循环。
4. sanitizer 版本多了哪些检查调用。
5. 哪个构建适合调试，哪个适合性能结论。

实验：目标文件、符号和重定位

目标：把多文件构建和二进制工具连接起来。

创建：

```

src/add.hpp
src/add.cpp
src/main.cpp

```

分开编译：

```

g++ -std=c++20 -O2 -c src/add.cpp -o build-release/add.o
g++ -std=c++20 -O2 -c src/main.cpp -o build-release/main.o
g++ build-release/add.o build-release/main.o -o build-release/app

```

观察：

```

nm -C build-release/add.o > results/add.nm.txt
nm -C build-release/main.o > results/main.nm.txt
readelf -r build-release/main.o > results/main.reloc.txt
objdump -drwC -Mintel build-release/main.o > results/main.objdump.txt
objdump -drwC -Mintel build-release/app > results/app.objdump.txt

```

报告必须回答：

1. `add.o` 中 `add_i32` 是定义还是未定义引用。

2. `main.o` 中对 `add_i32` 的引用如何表现。
3. 未链接目标文件中的 `call` 为什么可能没有最终地址。
4. 链接后可执行文件中调用目标如何变化。

实验：GDB 观察寄存器和栈

目标：用 GDB 把函数调用从源码层追到寄存器和栈。
使用前面的 Debug 构建：

```
gdb -q build-debug/app
```

建议命令：

```
set disassembly-flavor intel
break sum_i32
run
info registers rip rsp rdi rsi rax
disassemble /m sum_i32
x/8gx $rsp
stepi
info registers rip rsp rax
next
continue
```

报告必须包含：

- 断点命中时 `rdi` 和 `esi` 的含义。
- `rsp` 指向的内存大致是什么。
- 至少三条指令执行前后的寄存器变化。
- GDB 源码单步和指令单步的差异。

实验：写一份可复现实验报告

目标：把本章工具串成一份完整报告。
报告目录建议：

```
results/
environment.txt
commands.txt
debug.objdump.txt
release.objdump.txt
app.nm.txt
app.sections.txt
gdb-session.txt
report.md
```

`report.md` 至少包含：

1. 实验目标。
2. 机器环境。
3. 源码文件列表。
4. 编译命令。
5. Debug 与 Release 的差异。
6. `objdump` 证据。
7. `nm` 或 `readelf` 证据。
8. GDB 观察结果。
9. 当前结论的限制。

实验：用 strace 定位文件路径问题

目标：理解用户程序和内核之间的系统调用边界。

创建程序：

```
#include <errno>
#include <stdio>
#include <string>

int main()
{
    const char* path = "data/input.txt";
    FILE* file = std::fopen(path, "r");
    if (file == nullptr) {
        std::fprintf(stderr,
            "failed to open %s: %s\n",
            path,
            std::strerror(errno));
        return 1;
    }
    std::fclose(file);
    return 0;
}
```

任务：

1. 不创建 data/input.txt，直接运行程序。
2. 用 strace-oresults/open.strace.txt./app 保存系统调用。
3. 在输出中搜索 openat 和 ENOENT。
4. 创建文件后重新运行，比较两次 strace。
5. 解释 C 库函数 fopen 和 Linux 系统调用之间的关系。

交付物：

- 两份 strace 输出。
- 程序 stderr 输出。
- 300 字说明：为什么源码只写了 fopen，工具里看到的是系统调用。

实验：保存并分析一次 core dump

目标：把崩溃现场保存下来，并用 GDB 事后分析。

创建程序：

```
int main()
{
    int* p = nullptr;
    *p = 42;
    return 0;
}
```

任务：

1. 用 -O0-g 编译。
2. 执行 ulimit-cunlimited。
3. 运行程序触发崩溃。
4. 如果系统生成 core 文件，用 gdb./appcore 打开；如果系统使用 systemd-coredump，尝试 coredumpctlgdb。
5. 在 GDB 中记录 bt、info registers、x/10i \$pc。
6. 解释当前指令为什么会崩溃。

交付物：

- 构建命令。

- core dump 获取方式。
- GDB 输出。
- 说明 rip、当前指令和空指针地址之间的关系。

实验：用 Git 和 Makefile 固化实验入口

目标：让一个小实验具备稳定构建入口和源码身份。

任务：

1. 为前面的单文件构建实验写一个 Makefile, 至少包含 debug、release、sanitize、inspect、clean 目标。
2. inspect 目标生成 objdump、nm、readelf、file 和 size 输出。
3. 在 results/environment.txt 中记录 gitrev-parse--shortHEAD、gitstatus--short 和 gitdiff--stat。
4. 修改一行源码但不提交，重新运行实验。
5. 在报告中说明 dirty worktree 为什么会影响复现。

交付物：

- Makefile。
- results/environment.txt。
- results/release.objdump.txt。
- 一段说明：构建规则、源码状态和实验结果之间的关系。

实验：用 /proc 验证进程身份

目标：确认一次运行中的程序对应哪个可执行文件、哪个工作目录、哪些参数和哪些地址映射。

创建一个会等待输入的小程序：

```
#include <stdio>

int main(int argc, char** argv)
{
    std::printf("argc=%d\n", argc);
    std::printf("press enter to exit\n");
    std::getchar();
    return 0;
}
```

构建并后台运行：

```
g++ -std=c++20 -O2 -g src/proc_probe.cpp -o build-release/proc_probe
./build-release/proc_probe alpha beta &
pid=$!
```

观察：

```
cat /proc/$pid/cmdline | tr '\0' ' ' > results/proc.cmdline.txt
tr '\0' '\n' < /proc/$pid/environ | sort > results/proc.environ.txt
readlink /proc/$pid/cwd > results/proc.cwd.txt
readlink /proc/$pid/exe > results/proc.exe.txt
ls -l /proc/$pid/fd > results/proc.fd.txt
cat /proc/$pid/maps > results/proc.maps.txt
```

交付物：

- 解释 cmdline、cwd、exe 和 maps 分别证明什么。
- 在 maps 中找出主程序、libc、heap 和 stack 的映射。
- 说明这些证据如何帮助排查“运行了旧二进制”或“相对路径找错位置”的问题。

实验：动态链接证据链

目标：观察一个普通 C++ 程序依赖哪些共享库，以及调用外部函数时二进制里出现什么线索。

使用包含 `std::printf` 或 `std::puts` 的程序，构建 Release 版本。保存：

```
file build-release/app > results/app.file.txt
ldd build-release/app > results/app.ldd.txt
readelf -d build-release/app > results/app.dynamic.txt
objdump -drwC -Mintel build-release/app > results/app.objdump.txt
```

任务：

1. 在 `ldd` 输出中找出 `libc` 和 C++ 标准库依赖。
2. 在 `readelf-d` 输出中找出 `NEEDED` 项。
3. 在反汇编中搜索 `@plt`。
4. 解释 `ldd`、`readelf` 和 `objdump` 分别站在哪一层提供证据。

交付物中必须说明：动态链接证据不能只靠一个工具；库依赖、ELF 元数据和调用点反汇编回答的是不同问题。

实验：工具扰动对比

目标：观察 GDB、sanitizer、`strace` 和普通运行的区别，形成“工具有扰动”的边界意识。

选择一个运行时间至少数百毫秒的小程序，分别执行：

```
/usr/bin/time -v ./build-release/app > results/run.normal.txt 2>&1
strace -o results/run.strace.log ./build-release/app \
> results/run.strace.txt 2>&1
g++ -std=c++20 -O1 -g -fsanitize=address,undefined \
src/main.cpp -o build-sanitize/app
/usr/bin/time -v ./build-sanitize/app > results/run.sanitize.txt 2>&1
```

报告要求：

- 比较普通运行、`strace` 运行和 sanitizer 运行的时间差异。
- 说明这些时间不能直接作为算法性能比较。
- 解释每个工具适合回答什么问题。
- 若差异不明显，也要说明程序规模、系统调用数量和测量方法可能导致观察不到明显差异。

本章作业

习题与作业

作业 1：工具链分层图。

画出从 `main.cpp` 到 `app` 的完整路径。图中必须包含：

- preprocessor。
- compiler frontend。
- optimizer。
- backend。
- assembler。
- linker。
- loader。

每一层写出输入、输出、常用观察命令和一个可能出错的例子。

作业 2：构建错误分类。

故意制造三类错误：

1. C++ 语法或类型错误。
2. undefined reference 链接错误。
3. 运行时越界错误。

保存错误输出，并解释每个错误发生在哪个阶段、应当用什么工具定位。

作业 3：GDB 调用链报告。

写一个三层函数调用：

```
int f(int);
int g(int);
int h(int);
```

用 GDB 在 `-O0-g` 下观察调用链、返回地址、`rsp` 变化和返回值。提交不少于 1200 字报告。

作业 4：Debug 与 Release 二进制对比。

选择本书前面任意一个小函数，用 `-O0-g` 和 `-O3-DNDEBUG` 构建。比较：

- 函数是否存在独立符号。
- 栈帧是否存在。
- 局部变量是否落在栈上。
- 循环是否被展开或改写。
- 哪个版本更适合调试，哪个版本更接近性能路径。

作业 5：实验可复现性审查。

拿自己之前写过的一份实验记录，检查是否缺少下面信息：

- 工作目录。
- 源码版本。
- 编译命令。
- 编译器版本。
- CPU 和 OS。
- 输入规模。
- 原始输出。
- 反汇编或工具证据。

把它补成一份别人可以复现的报告。

作业 6：工具选择诊断表。

给出下面 8 个现象，为每个现象选择首选工具，并写出第一条应该运行的命令：

1. 编译器找不到头文件。
2. 链接器报告 undefined reference。
3. 程序启动时报 shared library not found。
4. 程序读取配置文件失败。
5. 程序发生 SIGSEGV。
6. Release 版本中某个函数找不到反汇编标签。
7. benchmark 数字突然快得不合理。
8. 两台机器结果差异很大。

要求说明为什么选择这个工具，而不是只列命令。

作业 7：core dump 调试报告。

找一个会崩溃的小程序，可以是空指针、越界写或手写汇编破坏栈。生成 core dump，并提交一份报告：

- 构建命令和构建类型。

- core dump 保存方式。
- GDB 中的 `bt`、`info registers` 和当前指令。
- 崩溃地址的解释。
- 如果使用 sanitizer，同一错误的 sanitizer 报告。

重点是从机器状态解释崩溃，不要只写“因为代码有 bug”。

作业 8：最小实验包审计。

为本章任意实验整理一个完整结果目录，至少包含：

- `README.md`。
- `commands.txt`。
- `environment.txt`。
- 构建输出或 verbose build 日志。
- `objdump`、`nm`、`readelf` 输出。
- 如果涉及崩溃，包含 GDB 或 core dump 记录。
- `report.md`。

然后写一段自评：另一个人能否根据这个目录复现你的观察？还缺哪些信息？

作业 9：进程运行身份报告。

选择一个自己写的小程序，用 `/proc/<pid>` 收集运行时身份信息。报告必须包含：

- 启动命令和进程号获取方式。
- `cmdline`、`cwd`、`exe`、`fd`、`maps` 的关键输出。
- 当前工作目录如何影响相对路径。
- `maps` 中主程序、共享库、heap、stack 的地址范围。
- 如果程序崩溃，如何把 `rip` 和 `maps` 对应起来。

作业 10：动态链接故障推演。

假设一个程序在自己机器上能运行，到另一台机器上报错：

```
error while loading shared libraries: libexample.so: cannot open shared object file
```

写一份排查方案，至少包含：

- 使用 `ldd`、`readelf-d`、`file` 的命令。
- 检查 `LD_LIBRARY_PATH`、`RPATH/RUNPATH` 和库安装路径的方法。
- 如何确认库架构和程序架构一致。
- 为什么不能只把错误归咎于 C++ 源码。

作业 11：证据链改写。

把下面三个不合格结论改写成合格工程结论：

1. “这个函数没有了，因为编译器优化掉了。”
2. “程序崩溃是因为指针错了。”
3. “Release 比 Debug 快，所以算法优化成功。”

每个改写都必须列出需要的工具证据、哪些是事实、哪些是解释、哪些还需要进一步验证。

验收标准

完成本章后，你应该能够做到：

- 不依赖 IDE，在 Linux 命令行创建、编译、运行和调试 C++ 程序。
- 解释 Debug、Release、sanitizer、benchmark build 的用途和边界。
- 使用 `objdump-drwC-Mintel` 找到函数反汇编，并解释地址、机器码字节和指令文本的区别。
- 使用 `nm-C` 判断符号是否定义、未定义或被优化消除。

- 使用 `readelf` 查看 ELF header、section、program header、symbol 和 relocation 的基础信息。
- 使用 GDB 设置断点、单步源码、单步指令、查看寄存器和查看栈内存。
- 使用 `strace` 判断文件路径、系统调用和错误码相关问题。
- 使用 `core dump` 保存崩溃现场，并用 GDB 事后分析。
- 使用 `/proc/<pid>` 观察进程参数、工作目录、可执行文件、文件描述符和地址映射。
- 用 `file`、`ldd`、`size`、`readelf-d` 快速建立二进制档案和动态链接证据。
- 保存环境、命令、工具输出和报告，让实验可以复现。
- 解释 GDB、sanitizer、`strace` 和 `perf` 的主要扰动与适用边界。
- 把工具输出整理成事实、解释、建议分离的证据链。
- 在性能讨论中主动说明构建类型和实验限制，不把调试观察当成性能结论。
- 面对编译、链接、启动、崩溃、错误结果和性能异常，能选择合适工具而不是盲目猜测。
- 能用 Git 状态、构建日志和结果目录说明一次实验对应的源码身份。

如果这些能力稳定，后面进入 x86-64 汇编、ABI、C++ 与汇编互操作、可信性能测量时，读者就不会只是在读概念，而是在用工具不断验证概念。

第零部分

汇编原理：x86-64、ABI 与二进制接口

第 6 章

x86-64 汇编语法总览

问题与目标

前面几章已经建立了三条基础线：

```
C++ source -> compiler -> assembly text -> machine code bytes
object/type/layout -> address formula -> memory operand
instruction bytes -> CPU state transition -> performance behavior
```

现在正式进入 x86-64 汇编。初学阶段常见两种相反误区：一种把汇编当成“更难看的 C”，另一种把汇编当成不可读的符号堆。两者都不准确。汇编不是 C，它更接近 ISA 暴露给软件的机器动作；但汇编也不是随机符号，只要掌握语法、寄存器、操作数、指令宽度和 ABI 上下文，就可以逐条推导。

本章目标不是一次背完所有 x86 指令，而是建立读汇编的基本方法。完成本章后，应该能做到：

- 看懂一条 Intel 语法汇编指令的基本结构。
- 区分立即数、寄存器操作数、内存操作数。
- 识别通用寄存器及其 64/32/16/8 位别名。
- 理解 `byteptr`、`dwordptr`、`qwordptr` 等宽度标记。
- 从 `[base+index*scale+displacement]` 还原 C++ 数组和结构体访问。
- 用“读哪些状态、写哪些状态”的方式注释汇编。
- 比较 `-00` 和 `-03` 汇编为什么差异巨大。
- 知道 Intel 语法和 AT&T 语法的关键差异，避免读资料时混淆。

核心思想

读汇编的核心不是把每条指令机械翻译成一行 C++，而是恢复数据流、控制流和地址公式。C++ 源码只是汇编的一种可能来源；真正可靠的依据，是逐条说明指令读写了哪些寄存器、内存和 flags。

读汇编前先分清四个层次

汇编入门容易混乱，不是因为语法本身多难，而是因为几个层次的知识经常同时出现。看到一行汇编时，里面至少包含四类信息：汇编语法、ISA 语义、ABI 约定和编译器选择。把这四类信息分清，后面读任何函数都会更稳定。

第一类是汇编语法。它回答“这行文本怎么读”。例如 Intel 语法中通常写成目的操作数在前、源操作数在后；方括号表示内存操作数；`dwordptr` 说明内存访问宽度。语法本身不解释为什么第一个参数在 `rdi`，也不解释一条指令在某个 CPU 上快不快。

第二类是 ISA 语义。它回答“这条指令对架构状态做什么”。例如 `add eax, esi` 读取 `eax` 和 `esi`，把低 32 位加法结果写回 `eax`，并更新一部分 flags；`ret` 从栈顶取返回地址，改变 `rsp` 和 `rip`。ISA 语义关心软件可见状态，不要求你知道 CPU 内部把它拆成几个微操作。

第三类是 ABI 约定。它回答“函数之间如何协作”。为什么第一个整数参数常在 `edi`，为什么返回值在 `eax`，为什么某些寄存器由调用者保存、某些由被调用者保存，这不是 `mov` 或 `add` 本身决定的，而是 System V AMD64 ABI 决定的。同一条指令在不同函数位置有不同含义，常常就是因为 ABI 上下文不同。

第四类是编译器选择。它回答“为什么这段源码生成了这种形状”。编译器可以选择内联、删除、展开、合并、使用 `lea` 代替乘法、使用条件移动代替分支，也可以因为调试构建而保留大量栈槽。编译器选择受优化级别、目标 CPU、语言规则、别名信息和可见函数体影响。你不能只凭一段汇编判断源码原来一定长什么样，只能判断这段汇编实际做了什么。

心智模型

看到一行汇编时，按顺序问：语法上谁是目的操作数，ISA 上它读写什么状态，ABI 上这些寄存器在函数入口或出口代表什么，编译器为什么可能选择这种写法。四个问题分开回答，错误会少很多。

汇编不是更低级的 C

很多初学者会把汇编读成“一条指令对应一行 C++”。这在极小例子里偶尔能用，但作为方法是危险的。C++ 是带类型、作用域、对象生命周期、别名规则和未定义行为的语言；汇编面对的是寄存器、内存地址、flags 和控制流。两者之间经过优化器，形状可以完全不同。

一个 C++ 局部变量未必对应内存中的一个槽。它可能一直在寄存器里，也可能被常量传播后消失，也可能和另一个临时值复用同一个寄存器。相反，-O0 汇编里的许多栈槽也不代表真实性能路径中的对象，只是为了调试器能在源代码行之间看到变量。把调试构建的栈槽当成语言对象的真实运行形态，会误导后面的性能判断。

一个 C++ 表达式也未必对应一条指令。表达式可以被拆成多条指令，也可以和相邻表达式合并，还可以被代数化简。比如乘以常数可能用 `leq`，比较可能只留下 flags，条件表达式可能变成 `cmov`，数组访问可能和加法合并成带内存操作数的指令。读汇编时不要急着把每条指令翻译回某一行源码，而要先恢复数据如何流动、地址如何形成、控制流如何选择。

最可靠的读法是把汇编看成状态转移。每条指令都在读某些状态、写某些状态。状态可以是通用寄存器，可以是内存，可以是 flags，也可以是 `rip` 和 `rsp`。如果你能完整说明这些读写，再结合函数入口的 ABI 约定，就能解释这段机器码。至于它最初来自哪一行 C++，要在最后再谨慎推断。

常见误区

不要把“恢复 C-like 伪代码”当作第一步。伪代码是分析结果，不是分析依据。先做寄存器含义、内存地址、访问宽度和 flags 依赖，再写伪代码。

先生成一份你能控制的汇编

学习汇编不能只看网上片段。你要从自己写的 C++ 生成汇编，知道编译命令、优化级别、目标语法和反汇编工具。先创建 `labs/ch10_assembly_syntax/minimal.cpp`：

```
extern "C" int add3(int x, int y, int z)
{
    return x + y + z;
}

extern "C" int load_i32(int const* p, int i)
{
    return p[i];
}
```

生成 Intel 语法汇编：

```
clang++ -std=c++20 -O3 -S -masm=intel minimal.cpp -o minimal.s
```

生成目标文件并反汇编：

```
clang++ -std=c++20 -O3 -c minimal.cpp -o minimal.o
objdump -d -Mintel minimal.o > minimal.objdump.txt
```

如果你的系统主要使用 LLVM 工具链，也可以用：

```
llvm-objdump -d --x86-asm-syntax=intel minimal.o > minimal.objdump.txt
```

`minimal.s` 是汇编器输入，通常包含标签、伪指令、调试/展开信息等。`minimal.objdump.txt` 是从目标文件机器码反汇编出来的文本，会显示机器码字节和指令地址。学习时两者都要看：

- `.s` 更接近编译器输出，适合看函数边界和汇编器伪指令。
- `objdump` 更接近最终二进制，适合看机器码字节、真实指令地址和链接前符号。

这两份文件回答的问题不同。`.s` 适合观察编译器想让汇编器看到什么，包括函数标签、可见性、段切换、对齐和一些调试或展开信息。反汇编适合观察目标文件里真实存在的指令字节，尤其适合验证指令长度、相对跳转、重定位位置和符号地址。高质量报告最好同时保留两者，因为它们能互相校验。

还要保存完整编译命令。没有命令，汇编没有可复现性。优化级别、目标 CPU、是否启用位置无关代码、是否保留帧指针、是否启用链接时优化、使用 GCC 还是 Clang，都会改变输出。一个报告只贴几行汇编，却不说明命令，读者无法判断这些指令是调试路径、发布路径，还是某个特定工具链版本的偶然结果。

生成汇编时也要避免一次引入太多库和模板。第一份练习最好使用 `extern"C` 的小函数，避免 C++ 名字改编、异常表、标准库初始化和模板实例化把观察对象淹没。等你能读懂小函数，再逐步加入结构体、数组、循环、函数调用和标准库代码。教材训练的顺序应该是先控制变量，再扩大复杂度。

常见误区

不同编译器、版本、优化级别、目标 CPU、命令行参数都会生成不同汇编。本书的汇编是“合理示例”，不是要求你的机器逐字一致。学习重点是解释自己的输出。

汇编器到底做什么

很多读者第一次看到 `.s` 文件，会把它理解成“CPU 直接执行的文本”。这不准确。汇编器的任务是把汇编文本翻译成目标文件中的机器码、符号和重定位信息。CPU 最终执行的是内存中的机器码字节，不是 `.s` 文件。

从编译器到 CPU，中间至少有三个可观察产物：

产物	谁产生或消费	主要用途
<code>.s</code>	编译器产生，汇编器消费	人类可读的汇编文本、标签、伪指令、段信息
<code>.o</code>	汇编器产生，链接器消费	机器码、符号表、重定位、调试/展开信息
可执行文件	链接器产生，加载器消费	最终程序映像、动态链接信息、入口和段映射

汇编器做的事包括：

- 把助记符和操作数编码成机器码字节。
- 计算同一汇编文件内标签之间的相对位移。
- 把函数名、全局对象名等记录为符号。
- 对当前还不能确定的地址生成重定位记录。
- 按伪指令切换 section、设置对齐、生成常量数据和元数据。

这解释了为什么 `.s` 文件里会有很多不是 CPU 指令的行。比如 `.text`、`.globl`、`.type`、`.size`、`.cfi_startproc`、`.p2align` 都是给汇编器、链接器、调试器或异常展开机制看的元信息。它们会影响目标文件，但不是 CPU 在函数热路径中逐条执行的指令。

例如：

```
.text
.globl add2
.type add2,@function
add2:
    mov eax, edi
    add eax, esi
    ret
.size add2, .-add2
```

这里真正的指令只有三条：`mov`、`add`、`ret`。标签 `add2:` 是地址名字，不是指令；`.globl` 让符号对链接器可见；`.type` 和 `.size` 帮助工具知道这是函数以及函数范围。

核心理念

读汇编文件时要先把“CPU 指令”和“汇编器/链接器元信息”分开。伪指令不是没用，但它回答的是目标文件组织和工具链问题，不是每周期执行问题。

section：代码、常量和未初始化数据怎样分区

汇编文件不是一串扁平指令。汇编器输出目标文件时，会把不同性质的内容放入不同 section。常见 section 包括 `.text`、`.rodata`、`.data` 和 `.bss`。这些名字看起来像格式细节，其实会影响链接、加载、权限和运行时内存映像。

section	常见内容	运行时性质
<code>.text</code>	函数机器码、跳转填充、只执行的代码	通常可读可执行，不应可写
<code>.rodata</code>	字符串字面量、只读常量表、跳转表	通常可读不可写
<code>.data</code>	有初始值的全局变量和静态变量	可读可写，文件中保存初始值
<code>.bss</code>	零初始化的全局变量和静态变量	可读可写，文件中通常不保存大块零字节

例如：

```
.text
.globl read_answer
.type read_answer,@function
read_answer:
    mov eax, dword ptr [rip + answer]
    ret

.data
.globl answer
answer:
    .long 42

.section .rodata
message:
    .asciz "hello"
```

这里 `read_answer` 的指令在 `.text`；`answer` 的初始值在 `.data`；`message` 的字节在只读数据 section。CPU 执行时不会“执行 `.data`”。但链接器和加载器会根据 section 属性把它们映射到进程虚拟地址空间的不同区域，并设置合适的页权限。

这也解释了为什么同样是字节，工程含义完全不同。`.text` 里的字节会被反汇编器当作指令候选；`.rodata` 里的字节可能是字符串、浮点常量、查表数据或跳转表；`.bss` 里的对象在目标文件中可能只记录大小，不真的存储一堆零。

读目标文件时，如果不知道当前字节属于哪个 section，就很容易把数据当代码、把常量当指令，或者把未初始化存储误认为文件里漏了内容。

section 切换不是运行时跳转

伪指令 `.text`、`.data`、`.section.rodata` 的作用是告诉汇编器“接下来输出到哪个区域”。它们不是运行时控制流。下面这段文本：

```
.text
f:
    ret

.data
x:
    .long 7
```

不是说函数 `f` 返回后会跳到 `x`，也不是说 CPU 会执行 `.data`。汇编器只是在构造目标文件时把 `ret` 编码到代码 section，把 4 字节整数常量放到数据 section。运行时控制流由 `call`、`ret`、`jmp`、条件跳转和异常/信号等机制决定，不由汇编文本中 section 出现的先后决定。

这个区分对阅读编译器输出尤其重要。一个 `.s` 文件可能先写某个函数，随后切到 `.rodata` 输出字符串，再切回 `.text` 输出下一个函数。文本顺序只是汇编器输入顺序，最终链接器还可能重排 section、合并相同常量、丢弃未引用 section、插入对齐填充或把热冷代码分开。报告中如果要说明运行时地址顺序，应使用链接后反汇编或 section header，而不是只凭 `.s` 的文本顺序。

数据伪指令写的是字节，不是高级类型

汇编里的数据伪指令常见有 `.byte`、`.word`、`.long`、`.quad`、`.ascii`、`.asciz`、`.zero`。它们告诉汇编器输出特定宽度的字节序列：

```
.byte 1
.word 0x0203
.long 0x04050607
.quad 0x08090a0b0c0d0e0f
.asciz "ok"
.zero 16
```

这些伪指令不携带完整 C++ 类型语义。`.long 1` 只是输出 4 字节整数编码，可能表示 `int`，也可能表示无符号整数、位掩码、浮点位模式的一部分、跳转表项或某个结构体字段。是否能把它解释成 C++ 类型，要看 section、符号、对齐、引用它的指令、调试信息和源码上下文。

字符串也一样。`.asciz` 通常输出带结尾零字节的 C 风格字符串；`.ascii` 不自动追加结尾零。看到 `.asciz "hello"`，可以推断它很可能服务于字符串字面量或 C 接口，但仍要看是否有代码用 `lea rdi, [rip + ...]` 取它的地址，或者是否被作为常量表的一部分引用。

常见误区

不要把数据伪指令直接当作“变量声明”。汇编层只输出字节、对齐和符号；高级语言的类型、生命周期、`const` 语义和所有权，要靠源码和 ABI 共同解释。

对齐和填充：为什么会出现看似多余的字节

目标文件中经常出现对齐伪指令，例如 `.p2align`、`.align`、`.balign`。编译器可能在函数入口、循环入口、常量表或对象之间插入填充，让后续地址满足某种边界要求。

```
.p2align 4
hot_loop:
    add eax, dword ptr [rdi + rcx*4]
    inc rcx
    cmp rcx, rsi
    jl hot_loop
```

`.p2align 4` 常表示按 $2^4 = 16$ 字节边界对齐。汇编器可能在 `hot_loop` 前插入若干字节。若填充位于 `.text`，它可能使用一种或多种 `nop` 编码；若填充位于数据 section，可能是零字节或指定填充值。

对齐的目的有几类。代码入口对齐可能改善取指和分支目标布局；循环对齐可能减少跨取指块边界；数据对齐可能满足类型要求或让后续 `load/store` 更自然；向量数据可能需要更严格的边界。第一册不深入 CPU 前端和 cache line 性能，但要先知道：填充字节通常是工具链有意放置的布局内容，不一定是“无用垃圾”。

反汇编时，对齐填充会带来两个常见误判。第一，把函数之间的 `nop` 当作源代码逻辑。普通 `nop` 不改变程序语义，它可能只是对齐填充。第二，把数据 section 中的填充字节反汇编成指令。反汇编器如果从错误位置开始解释字节流，任何数据都可能被显示成看似合法的 x86 指令。x86 是变长指令集，很多随机字节序列都能被解释出某种指令文本，所以“反汇编器显示了助记符”本身不证明这些字节真会被执行。

深入理解

后续学习性能时，对齐不应被神化。对齐可能影响取指、解码、cache line、分页和向量访问，但具体效果依赖代码位置、循环形状、CPU、数据规模和运行路径。当前阶段只要求你能识别对齐伪指令和填充字节，并在报告里把它们与真实业务指令分开。

局部标签、全局符号和可见性

汇编文件中有些名字只服务于当前文件内部跳转，有些名字要暴露给链接器和其他目标文件。前者通常是局部标签，后者通常是全局符号。分清二者，可以避免把编译器内部控制流标签误认为公共 API。

编译器生成的局部标签常见形式包括 `.LBB0_1`、`.Ltmp3`、`.L.str`。不同工具链命名不同，但一般以 `.L` 开头的符号默认不会作为普通全局符号导出。它们可能标记基本块、常量池、异常处理范围、调试临时位置或跳转表位置。

```
.globl sum_positive
.type sum_positive,@function
sum_positive:
    xor eax, eax
    test esi, esi
    jle .Ldone
.Lloop:
    add eax, dword ptr [rdi]
    add rdi, 4
    dec esi
    jne .Lloop
.Ldone:
    ret
```

这里 `sum_positive` 是外部可见函数符号；`.Lloop` 和 `.Ldone` 是函数内部控制流标签。机器码中的 `jne .Lloop` 通常会变成相对位移，运行时不需要保存字符串 `.Lloop`。如果链接器最终丢弃局部符号，程序仍能运行，因为跳转目标已经被编码成位移或重定位修补后的地址。

`.globl` 只解决链接可见性

`.globlname` 的意思是让 `name` 成为链接器可见的全局符号。它不自动保证这个符号是函数，不自动保证 ABI 正确，也不自动保证 C++ 端能按期望名字找到它。函数符号通常还配合 `.typename`、`@function` 和 `.size name, .-name`，让工具知道符号类型和范围。

对手写汇编来说，只写标签而忘记 `.globl`，C++ 代码可能链接不到该函数；写了 `.globl` 但 C++ 声明没有 `extern "C"`，也可能因为 C++ 名字修饰导致符号名不匹配；符号名匹配但参数寄存器、栈对齐或 callee-saved 寄存器规则错了，程序仍然可能崩溃。符号可见性只是跨文件调用的第一道门，ABI 正确性还要在后面章节系统检查。

符号范围影响工具输出

使用 `nm`、`readelf-s` 或 `objdump-t` 时，你会看到符号的绑定、类型、section 和地址偏移。一个典型目标文件可能同时包含：

```
0000000000000000 T sum_positive
                  U printf
0000000000000000 r .L.str
```

这类输出中，`T` 常表示定义在代码 section 的全局文本符号，`U` 表示 undefined symbol，需要链接器从别处解析，`r` 可能表示只读数据中的局部符号。不同工具输出格式会变，但核心问题一样：这个名字是本文件定义，还是外部引用；是函数、对象，还是 section 内局部位置；链接后是否仍然可见。

读符号表时，不要只看名字。还要看绑定属性、类型、所在 section、大小和是否 undefined。一个 `printf` 出现在符号表里，通常说明当前目标文件引用了外部 `printf`，并不说明 `printf` 的代码已经在当前目标文件中。一个 `.L.str` 出现在只读 section，通常说明它是字符串或常量位置，不应被当作可调用函数。

标签、符号和地址不是一回事

汇编里经常看到名字，例如函数标签、局部标签、全局对象名和跳转目标。它们都可能显示成“符号”，但含义要分清。

标签是汇编文本中的一个位置标记：

```
loop:
    add eax, dword ptr [rdi + rcx*4]
    inc rcx
    cmp rcx, rsi
    jl loop
```

`loop` 标记某条指令的地址。条件跳转 `jl loop` 在机器码里通常不会保存字符串 `loop`，而是保存从当前指令到目标位置的相对位移。汇编器负责把标签名转成位移或重定位。

符号是目标文件层面记录的名字。函数名、全局变量名、外部引用都可能出现在符号表中。使用 `nm-C` 或 `readelf-s` 可以看到符号。一个标签不一定成为外部可见符号；一个符号也可能没有完整函数体，例如 `undefined symbol` 只表示当前目标文件引用了外部定义。

地址是运行时或目标文件中的位置数值。目标文件中的地址常常只是 section 内偏移；最终可执行文件里链接器会分配虚拟地址；启用 PIE 和 ASLR 后，运行时装载基址还可能变化。报告里说“函数在某地址”，必须说明这是目标文件偏移、可执行文件虚拟地址，还是当前进程运行时地址。

这个区分对调试非常重要。GDB 中的 `breakadd2` 是按符号下断点；`break*0x401140` 是按地址下断点；汇编中的 `add2:` 是标签；反汇编里的 `0000000000000000<add2>`：是工具把地址和符号名结合显示出来。它们有关联，但不是同一个概念。

常见误区

不要把“看见名字”理解成“机器码里保存了这个名字”。大多数普通指令执行时只处理位模式、寄存器编号、立即数、位移和地址。名字主要服务于汇编、链接、调试和报告。

一条汇编指令的基本结构

Intel 语法里，一条普通指令大致长这样：

```
mnemonic destination, source
```

例如：

```
mov eax, edi
add eax, esi
ret
```

这段代码可以来自：

```
extern "C" int add2(int x, int y)
{
    return x + y;
}
```

在 System V AMD64 ABI 下，前两个 `int` 参数在 `edi` 和 `esi`，返回值在 `eax`。逐条解释：

指令	读什么	写什么
<code>moveax,edi</code>	<code>edi</code>	<code>eax</code> ，同时清零 <code>rax</code> 高 32 位
<code>addeax,esi</code>	<code>eax</code> 、 <code>esi</code>	<code>eax</code> 和部分 flags
<code>ret</code>	栈顶返回地址、 <code>rsp</code>	<code>rip</code> 、 <code>rsp</code>

这里有一个初学阶段必须接受的事实：`add eax, esi` 的目的操作数既是输入也是输出。它不是“产生一个新变量”，而是把结果写回 `eax`：

```
eax = eax + esi
```

因此读汇编时，你应该维护一个“当前每个寄存器代表什么”的笔记。比如：

```

before:
    edi = x
    esi = y

mov eax, edi
    eax = x

add eax, esi
    eax = x + y

ret
    return eax

```

这个例子虽然小，但已经包含读汇编的核心动作。函数入口时，**edi** 和 **esi** 的含义不是从指令本身来的，而是从 ABI 和函数签名来的。第一条 **mov** 之后，**eax** 才开始代表 **x**。第二条 **add** 之后，**eax** 的含义变成 **x+y**。最后 **ret** 并不“返回 **eax**”这个动作本身；返回值放在 **eax** 是 ABI 约定，**ret** 做的是控制流返回。

因此，逐条注释时不能只写“移动”“相加”“返回”。这种注释没有信息量。好的注释要写出入口假设、读写状态和状态含义的变化。尤其要注意目的操作数既可能是输入也是输出，例如 **add**、**sub**、**and**、**or** 都会读取旧目的值，再写回新目的值。如果忘记这一点，就会把数据依赖链看断。

深入理解

现代性能分析非常重视依赖链。即使两条指令看起来都很短，只要后一条必须等待前一条写出结果，就不能完全并行。读汇编时维护寄存器含义，不只是为了理解语义，也是为了后续判断 latency、throughput 和乱序执行空间打基础。

操作数：立即数、寄存器和内存

x86-64 指令的操作数常见三类：立即数、寄存器、内存。

立即数

immediate（立即数）是编码在指令里的常量：

```

mov eax, 42
add rdi, 16
and eax, 255

```

立即数不是内存地址，也不需要额外 load。它是机器码字节的一部分。比如 **addrdi, 16** 中的 **16** 会被编码进指令。

寄存器操作数

寄存器操作数直接读写 CPU 的架构寄存器：

```

mov rax, rdi
add eax, esi
xor ecx, ecx

```

寄存器访问通常比内存访问快得多，但寄存器数量有限。编译器优化很大一部分工作，就是尽量把热数据保存在寄存器中，减少 load/store。

内存操作数

方括号表示内存操作数：

```

mov eax, dword ptr [rdi]
mov eax, dword ptr [rdi + rsi*4]
mov qword ptr [rsp + 8], rax

```

方括号里的表达式计算地址；**byteptr**、**dwordptr**、**qwordptr** 表示访问宽度。注意：

```

mov eax, edi      ; register -> register
mov eax, [rdi]    ; memory at address rdi -> register

```


少一个方括号，语义完全不同。第一条把 `edi` 的值复制到 `eax`；第二条把 `rdi` 当作地址，从该地址读取内存。

一般不能内存到内存

多数普通 x86 指令不能同时有两个内存操作数。例如下面这种通常不合法：

```
mov dword ptr [rdi], dword ptr [rsi] ; invalid for ordinary mov
```

需要用寄存器中转：

```
mov eax, dword ptr [rsi]
mov dword ptr [rdi], eax
```

这对性能分析很重要。源码里看起来像一次赋值，机器层面可能是一次 load 加一次 store。

内存操作数不是对象本身

方括号里的表达式只计算地址，访问的对象含义要靠上下文恢复。看到 `[rdi+rsi*4]`，只能先说它访问了从 `rdi` 加 `rsi` 乘 4 得到的地址，访问宽度由指令或宽度标记决定。它可能是 `int` 数组元素，也可能是某个结构体数组里的字段，也可能是编译器临时布置的数据表。只有结合函数签名、类型布局、访问宽度、相邻指令和源码，才能进一步推断它对应什么对象。

这个原则能避免许多错误。初学者看到乘 4 就直接说“这是 `int` 数组”，但乘 4 也可能来自四字节字段、索引表、浮点数组或某种压缩布局。看到 `+8` 也不能马上说“这是第三个元素”，它可能是结构体字段偏移、栈槽偏移、对象头之后的数据，或调用约定要求的保存位置。汇编里的地址公式是证据，C++ 对象解释是推论，二者不要混为一谈。

另一个常见错误是把内存操作数都当成“慢”。内存访问的成本取决于是否命中 cache、地址是否已知、是否和前后指令存在依赖、是否与 store 冲突、是否跨 cache line 或页面等。第 6 章不展开这些微架构细节，但你要先养成准确标注 load/store 的习惯。只有先知道哪里真正访问内存，后续才谈得上判断内存层级成本。

通用寄存器和别名

x86-64 有 16 个常用通用整数寄存器。每个寄存器可以用不同宽度的名字访问低位部分：

64 位	32 位	16 位	8 位低位	常见 ABI 含义
<code>rax</code>	<code>eax</code>	<code>ax</code>	<code>al</code>	返回值、临时
<code>rbx</code>	<code>ebx</code>	<code>bx</code>	<code>bl</code>	callee-saved
<code>rcx</code>	<code>ecx</code>	<code>cx</code>	<code>cl</code>	第 4 参数、移位计数
<code>rdx</code>	<code>edx</code>	<code>dx</code>	<code>dl</code>	第 3 参数、乘除辅助
<code>rsi</code>	<code>esi</code>	<code>si</code>	<code>sil</code>	第 2 参数
<code>rdi</code>	<code>edi</code>	<code>di</code>	<code>dil</code>	第 1 参数
<code>rbp</code>	<code>ebp</code>	<code>bp</code>	<code>bpl</code>	帧指针或 callee-saved
<code>rsp</code>	<code>esp</code>	<code>sp</code>	<code>spl</code>	栈指针

`r8` 到 `r15` 也有类似名字：

64 位	32 位	16 位	8 位
<code>r8</code>	<code>r8d</code>	<code>r8w</code>	<code>r8b</code>
<code>r9</code>	<code>r9d</code>	<code>r9w</code>	<code>r9b</code>
<code>r10</code>	<code>r10d</code>	<code>r10w</code>	<code>r10b</code>
<code>r11</code>	<code>r11d</code>	<code>r11w</code>	<code>r11b</code>
<code>r12</code>	<code>r12d</code>	<code>r12w</code>	<code>r12b</code>
<code>r13</code>	<code>r13d</code>	<code>r13w</code>	<code>r13b</code>
<code>r14</code>	<code>r14d</code>	<code>r14w</code>	<code>r14b</code>
<code>r15</code>	<code>r15d</code>	<code>r15w</code>	<code>r15b</code>

32 位写入会清零高 32 位

x86-64 一个非常重要的规则是：写 32 位寄存器会把对应 64 位寄存器高 32 位清零。

```
mov eax, edi
```

这条指令写 `eax`，也会让 `rax` 的高 32 位变成 0。可以这样理解：

```
rax = zero_extend_64(edi)
```

但写 16 位或 8 位寄存器不会自动清零高位：

```
mov ax, di      ; only low 16 bits are written
mov al, dil     ; only low 8 bits are written
```

因此部分寄存器写入可能保留旧值的高位。现代编译器通常更喜欢用 32 位写入来打破不必要的旧值依赖。

常见误区

不要把 `eax`、`ax`、`al` 当成完全独立寄存器。它们是 `rax` 的不同宽度视图。读汇编时必须知道一条指令到底写了多少位。

寄存器别名还会影响你对“值是否仍然存在”的判断。写 `eax` 会清零 `rax` 高 32 位，所以之后读 `rax` 得到的是零扩展后的 64 位值。写 `al` 只改变最低 8 位，高位仍然保留旧内容。假如你在笔记里把 `al` 写成独立变量，就会忘记 `rax` 的其余位仍然有历史值。

编译器倾向于使用 32 位写入清零寄存器，也和依赖有关。写完整的 32 位子寄存器能让硬件知道高 32 位确定为零，减少对旧 `rax` 的依赖。某些老旧微架构上，部分寄存器写入和随后读取完整寄存器可能引发额外依赖或合并成本。现代编译器和 CPU 已经处理了很多历史问题，但作为阅读者，你仍然要知道：宽度不是排版细节，它影响语义和依赖。

心智模型

寄存器不是变量名，而是一组可用不同宽度观察和写入的位。读汇编时，先确定写入宽度，再决定整个 64 位寄存器的新含义。

操作数宽度：机器到底读写几个字节

Intel 语法中，访问内存时常用宽度标记：

标记	字节数	常见 C++ 类型或用途
<code>byteptr</code>	1	<code>char</code> 、 <code>std::uint8_t</code>
<code>wordptr</code>	2	<code>std::uint16_t</code> 、x86 word
<code>dwordptr</code>	4	<code>int</code> 、 <code>float</code> 、 <code>std::uint32_t</code>
<code>qwordptr</code>	8	<code>long</code> 、指针、 <code>double</code> 、 <code>std::uint64_t</code>
<code>xmmwordptr</code>	16	128-bit SIMD memory operand
<code>ymmwordptr</code>	32	256-bit AVX memory operand

例如：

```
movzx eax, byte ptr [rdi]
mov  eax, dword ptr [rdi]
mov  rax, qword ptr [rdi]
movss xmm0, dword ptr [rdi]
movsd xmm0, qword ptr [rdi]
```

这些指令即使地址相同，访问宽度和解释规则也不同：

- `movzx eax, byte ptr [rdi]` 读取 1 字节并零扩展。
- `mov eax, dword ptr [rdi]` 读取 4 字节整数到 `eax`。
- `mov rax, qword ptr [rdi]` 读取 8 字节整数或指针。

- `movss xmm0, dword ptr [rdi]` 读取单精度浮点标量。
- `movsd xmm0, qword ptr [rdi]` 读取双精度浮点标量。

核心理想

内存地址只说明从哪里开始，操作数宽度说明读写多少字节，指令语义和寄存器类型说明如何解释这些字节。三者缺一不可。

C++ 类型信息在汇编中大多已经消失。你看到 `dword ptr [rdi]`，只能知道访问 4 字节，不能仅凭这一点判断它一定是 `int`。它也可能是 `float`，可能是无符号整数，可能是结构体中四字节字段，甚至可能只是某种位模式。真正区分整数和浮点的，往往是使用的寄存器类别和指令语义：通用寄存器上的 `mov`、`add` 更像整数或地址语义；XMM 寄存器上的 `movss`、`addss` 更像单精度浮点语义。

访问宽度也不等于对象大小。有时编译器只读取对象的一部分，例如读取一个字节字段或布尔值；有时会使用更宽的加载再用掩码取出需要的位；有时为了向量化会一次读取多个元素；有时结构体拷贝会变成若干宽 `load/store`。后续分析对象布局时，必须把“这条指令访问了多少字节”和“源语言对象有多大”分开。

宽度还直接影响正确性。读取 1 字节后如果要变成 32 位整数，就必须决定高位如何处理：零扩展还是符号扩展。读取 4 字节到 `eax` 会清零 `rax` 高位，但读取到 `al` 不会。很多 C/C++ 类型转换、数组元素读取和字符处理的汇编差异，都来自这个问题。下一章讲整数指令时会系统展开 `movzx`、`movsx` 和各种扩展规则，本章先把“宽度必须标清”作为基本习惯。

内存寻址模式：地址公式如何写进指令

x86-64 最常见的内存操作数形式是：

```
[base + index * scale + displacement]
```

其中 `scale` 只能是 1、2、4、8。不是所有部分都必须出现：

形式	含义
<code>[rdi]</code>	地址是 <code>rdi</code>
<code>[rdi+8]</code>	地址是 <code>rdi</code> 加 8 字节
<code>[rdi+rsi*4]</code>	地址是 <code>rdi</code> 加 <code>rsi</code> 乘 4
<code>[rdi+rsi*4+12]</code>	基址、索引缩放、常量偏移同时出现
<code>[rip+symbol]</code>	RIP-relative，常用于全局对象或常量池

案例：数组 load

C++：

```
extern "C" int load_i32(int const* p, int i)
{
    return p[i];
}
```

可能汇编：

```
movsxd rsi, esi
mov    eax, dword ptr [rdi + rsi*4]
ret
```

假设执行前：

寄存器	值	含义
<code>rdi</code>	<code>0x1000</code>	<code>p[0]</code> 地址
<code>esi</code>	<code>3</code>	下标 <code>i</code>

执行：

```

movsxd rsi, esi
    rsi = sign_extend_64(3)
    = 3

mov eax, dword ptr [rdi + rsi*4]
    effective_address = 0x1000 + 3 * 4
    = 0x100c
    load 4 bytes from 0x100c..0x100f into eax

```

案例：结构体字段

C++:

```

struct S {
    int a;
    double b;
    int c;
};

extern "C" int get_c(S const* p)
{
    return p->c;
}

```

常见布局:

字节范围	内容	大小	说明
0..3	a	4	字段
4..7	padding	4	让 b 8 字节对齐
8..15	b	8	字段
16..19	c	4	字段
20..23	tail padding	4	让 sizeof(S) 为 8 的倍数

可能汇编:

```

mov eax, dword ptr [rdi + 16]
ret

```

这里 **+16** 不是魔法数字, 而是 **offset(S, c)**:

```

addr(p->c) = addr(*p) + offset(S, c)
            = rdi      + 16

```

案例：结构体数组字段

C++:

```

struct Pair {
    int a;
    int b;
};

extern "C" int get_b(Pair const* p, int i)
{
    return p[i].b;
}

```

公式:

```

sizeof(Pair)    = 8
offset(Pair, b) = 4
addr(p[i].b)    = base + i * 8 + 4

```

可能汇编:

```
movsxd rsi, esi
mov    eax, dword ptr [rdi + rsi*8 + 4]
ret
```

如果 `rdi` 是 `0x2000`，`rsi` 是 5：

```
effective_address = 0x2000 + 5 * 8 + 4
                  = 0x202c
Load 4 bytes from 0x202c..0x202f
```

地址公式要同时解释三件事

一个内存操作数的地址公式至少有三层含义。第一层是机器层的有效地址，也就是寄存器、缩放和位移如何相加。这一层必须严格按照汇编文本写出来，不能省略。第二层是布局层的来源，也就是为什么缩放是 4、8 或其他形式，为什么常量偏移是 8、12、16。它可能来自元素大小、字段偏移、栈槽位置或表项大小。第三层才是源码层解释，也就是它可能对应 `p[i]`、`obj->field`、`items[i].count` 或某个临时 `spill`。

这三层的证据强度不同。机器层是确定的；布局层通常可以通过类型大小、对齐和相邻访问推导；源码层则需要结合函数签名和编译器优化。高质量汇编报告应该把三层分开写。例如不要只说“访问数组第 `i` 个元素”，而要写“有效地址为 `rdi` 加 `rsi` 乘 4；若 `rdi` 是 `int` 数组基址且 `rsi` 是下标，则它对应 `p[i]`；访问宽度为 4 字节”。

地址公式还能暴露数据布局问题。结构体数组字段访问常常出现“索引乘结构体大小再加字段偏移”；二维数组访问会出现“行号乘行跨度再加列偏移”；栈上局部变量常常表现为 `rsp` 或 `rbp` 加减常量。以后读更复杂代码时，你会靠这些公式恢复编译器看到的数据布局。第 6 章的目标不是背所有模式，而是训练你看到地址公式就能问出正确问题。

常见误区

不要把 displacement 叫作“随机常数”。汇编里的常量偏移通常有来源：字段偏移、数组元素偏移、栈槽位置、全局对象地址差、跳转目标位移或对齐填充。找不到来源时，应说“暂未解释”，不要编造一个源码形状。

RIP-relative：全局对象和常量池的常见形态

x86-64 代码里经常出现 `[rip+...]` 形式：

```
mov eax, dword ptr [rip + global_value]
lea rdi, [rip + message]
```

这里的 `rip` 不是普通基址寄存器用法，而是 RIP-relative addressing。它用“下一条指令地址附近的相对位移”来定位全局对象、字符串字面量、常量池或跳转表。这种形式对位置无关代码非常重要，因为代码被加载到不同基址时，相对距离可以由链接器和加载器处理。

考虑：

```
extern int global_value;

extern "C" int read_global()
{
    return global_value;
}
```

目标文件阶段可能看到类似：

```
mov eax, dword ptr [rip + 0x0]
                R_X86_64_PC32 global_value-0x4
ret
```

这不是说全局变量地址就是 0。目标文件还不知道 `global_value` 最终放在哪里，所以在指令里留下占位位移，并在 relocation 中记录“链接时把这里修成相对 `global_value` 的位移”。链接后再看最终可执行文件，位移可能变成具体数值。

RIP-relative 常见于：

- 读取只读常量或字符串地址。
- 访问全局变量。
- 访问 GOT 表项。
- 访问跳转表或常量表。
- 生成位置无关代码中的相对地址。

读这种地址时要小心两点。第一，`[rip+disp]` 的 `rip` 通常指向下一条指令，而不是当前指令起始地址。第二，反汇编器可能把目标符号名显示出来，也可能只显示数值位移；是否显示符号取决于符号表、重定位信息和工具选项。

心智模型

普通 `[rdi+rsi*4]` 常常来自对象或数组地址；`[rip+disp]` 常常来自“代码附近的全局/只读/链接时符号”。看到 RIP-relative，优先想到链接、位置无关代码和全局数据。

重定位：目标文件里尚未完成的地址

汇编阅读不能只看最终指令文本，还要知道哪些地址在目标文件阶段尚未确定。重定位就是工具链记录“这里以后需要修补”的机制。

例如一个文件调用另一个文件里的函数：

```
extern "C" int callee(int);

extern "C" int caller(int x)
{
    return callee(x + 1);
}
```

编译成目标文件后，`caller.o` 中可能有一条 `call`，但它还不知道 `callee` 的最终地址。反汇编可能显示：

```
83 c7 01      add edi, 1
e8 00 00 00 00 call 0x8
               R_X86_64_PLT32 callee-0x4
c3           ret
```

`e800000000` 中的 4 字节位移暂时是 0；重定位项告诉链接器，这里要修成到 `callee` 的相对位移，可能经过 PLT。链接后，这条 `call` 的位移才变成最终值。

因此，目标文件反汇编里的地址有三类：

- 已经确定的本地偏移，例如同一个函数内跳转到局部标签。
- 需要链接器修补的外部符号引用。
- 需要动态链接器或加载器参与的动态重定位。

读报告时，如果作者只贴目标文件中的 `call 0x0` 或 `mov [rip+0x0]`，却不贴 relocation，就不完整。因为这些 0 很可能只是占位，不是实际目标地址。正确证据应同时包含 `objdump-drwC-Mintel` 的指令和重定位行，或者直接分析链接后的可执行文件。

核心思想

目标文件不是最终程序。看到外部函数、全局对象、RIP-relative 占位或 `call` 目标异常时，先查重定位，再下结论。

如何组合 `objdump`、`readelf` 和 `nm`

单个工具很少给出完整答案。读目标文件时，`objdump` 擅长显示指令、机器码字节和重定位旁注。`readelf` 擅长显示 ELF section、符号表和重定位表的结构化信息。`nm` 擅长快速列出符号定义和未定义引用。高质量分析应根据问题组合工具，而不是把某个工具输出当作全部真相。

工具	常用命令	回答的问题
objdump	objdump-drwC...	指令、字节、反汇编、重定位旁注
readelf	readelf-S-s-rfile.o	section、符号表、重定位表
nm	nm-Cfile.o	哪些符号已定义，哪些仍未定义
llvm-objdump	llvm-objdump-dr...	LLVM 工具链下的反汇编和重定位

完整命令可以写在代码块中，避免表格因为长选项变得难读：

```
objdump -drwC -Mintel file.o
llvm-objdump -dr --x86-asm-syntax=intel file.o
```

比如你看到目标文件中有：

```
e8 00 00 00 00      call 0x5
R_X86_64_PLT32 printf@0x4
```

这时不能说“它调用地址 0x5”。0x5 是当前目标文件里占位位移导致的显示形式；重定位行才说明这里引用 printf。如果再运行 nm-Cfile.o，可能看到：

```
U printf
```

这说明 printf 在当前目标文件中未定义，链接器需要从 C 运行时库或动态库中解析它。若运行 readelf-rfile.o，会看到重定位表中有对应条目，说明哪个 offset 需要修补、重定位类型是什么、关联哪个符号。

如果问题是“函数里实际有哪些指令”，优先看 objdump。如果问题是“为什么 call 目标是 0”，必须看重定位。问题是“这个外部名字是否已经定义”，看 nm。问题是“这个常量放在哪个 section，section 是否可写”，看 readelf-S。问题是“符号大小、绑定和类型是什么”，看 readelf-s 或 objdump-t。同一个二进制事实经常要从多个角度交叉确认。

目标文件、可执行文件和运行时地址

同一个函数在三个阶段会有不同“地址语境”。在目标文件中，地址通常是 section 内偏移，很多外部地址还没解析；在链接后的可执行文件或共享库中，链接器已经给 section 分配虚拟地址，许多符号引用已经确定或变成动态链接入口；在运行中的进程里，PIE、ASLR 和动态加载会让实际装载基址变化。

因此下面三句话不是一回事：

- “目标文件中 add2 位于 .text offset 0。”
- “链接后可执行文件中 add2 的虚拟地址是 0x401140。”
- “当前进程这次运行中 add2 实际映射在某个运行时地址。”

调试器、perf、反汇编和链接器 map 文件可能使用不同地址语境。若报告中把它们混在一起，就会出现“objdump 里是 0，但 GDB 里不是 0”“上次运行地址和这次不一样”这类困惑。正确做法是每次写地址都标明阶段：目标文件 offset、链接后虚拟地址，还是运行时地址。

不要忽略工具选项

工具选项会改变你看到的信息。只用 objdump-d，可能看不到重定位；加上 -r 才显示 relocation；加上 -w 可以减少长行截断；加上 -C 可以反修饰 C++ 符号名；加上 -Mintel 可以显示 Intel 语法。不同平台的 objdump 和 llvm-objdump 选项略有差异，报告里应写明使用的命令。

这不是形式主义。一个只贴 objdump-d 的报告可能漏掉所有外部符号修补信息；一个没有 -C 的报告可能把 C++ 符号名写成难读的 mangled name；一个没有说明 Intel/AT&T 语法的报告可能把操作数方向解释反。工具命令就是证据链的一部分。

核心理想

反汇编不是“把二进制还原成源码”。它只是把字节解释成指令文本，并尽量附上符号和重定位信息。真正的分析要同时看指令、section、符号、重定位、编译命令和 ABI。

外部函数、PLT 和 GOT 的第一册认识

当代码调用外部函数或访问动态库中的全局对象时，反汇编中可能出现 `@PLT`、`@GOTPCREL`、`GOT load`、`PLT stub` 等字样。完整动态链接机制比较大，后续可以单独成章；本章先建立阅读边界，避免把这些名字当作普通业务函数或普通数组访问。

例如：

```
call printf@PLT
mov rax, qword ptr [rip + global@GOTPCREL]
```

PLT 通常与过程链接表有关，用来支持对动态库函数的间接解析和跳转；GOT 通常与全局偏移表有关，用来保存位置无关代码访问外部符号所需的地址。你不需要在第 6 章掌握动态链接全部细节，但看到这些标记时，至少要知道：这里不是一个普通的局部 `call` 或普通的数组 `load`，它经过链接器、动态链接器和位置无关代码约定。

外部函数调用在不同构建方式下形态会变。静态链接、动态链接、PIE、非 PIE、默认可见性、隐藏可见性、链接时优化、是否允许符号插桩或 `interposition`，都可能影响最终指令。有时你看到 `call printf@PLT`；有时看到对 GOT 表项的间接跳转；有时链接器能把调用放松成更直接的相对 `call`。报告里不要把某一种形态当作所有 Linux 程序的唯一形态。

从学习路径看，第一册只要求你做到三点。第一，看到 `@PLT` 或 `@GOTPCREL` 时意识到这是链接和动态加载相关机制。第二，能通过 `objdump-drwC` 和 `readelf-r` 找到对应重定位。第三，不把 PLT/GOT 开销直接归因成业务算法瓶颈，除非有运行时采样或计数证据证明它确实在热路径中频繁发生。

常见误区

PLT/GOT 是二进制链接边界的一部分，不是 C++ 源码里的普通函数体。读外部调用时，要把“业务函数语义”和“动态链接入口机制”分开。

栈上的地址公式：rsp、rbp 和栈槽

汇编里除了数组和结构体地址，还经常看到以 `rsp` 或 `rbp` 为基址的内存操作数：

```
mov dword ptr [rsp + 12], edi
mov eax, dword ptr [rbp - 4]
mov qword ptr [rsp + 24], rax
```

这些通常和栈帧、局部对象、保存寄存器、溢出临时值或调用参数有关。初学者容易把每个栈槽都对应成一个源码局部变量，这在 `-O0` 下有时近似成立，在优化构建中经常不成立。

栈地址阅读要先回答两个问题。

第一，当前函数是否使用 `frame pointer`。如果函数序言里有：

```
push rbp
mov rbp, rsp
```

那么 `rbp` 常被用作稳定基准，局部变量可能位于 `[rbp-offset]`，调用者相关数据或栈上传参可能位于 `[rbp+offset]`。如果省略 `frame pointer`，编译器通常用 `rsp` 加偏移访问栈槽，但 `rsp` 可能随 `push`、`call`、栈空间调整而变化，阅读时要追踪它的当前值。

第二，这个栈槽承担什么角色。可能是：

- 调试构建中为局部变量保留的位置。
- 寄存器不够时 `spill` 出去的临时值。
- 保存 `callee-saved` 寄存器。
- 为函数调用准备的栈上传参区域。
- 对齐填充或 `red zone` 中的临时空间。
- 编译器为了调试信息、异常展开或 `sanitizer` 插入的辅助数据。

例如：

```
sub rsp, 16
```

```
mov dword ptr [rsp + 12], edi
mov eax, dword ptr [rsp + 12]
add rsp, 16
ret
```

在 `-00` 中，这可能只是把参数 `x` 放到栈槽再读回来，方便调试器观察。优化后同样语义可能直接使用 `edi` 或 `eax`，栈槽完全消失。因此不能从 `-00` 栈访问推断性能路径一定有内存访问。

栈访问还有 ABI 约束。函数调用前栈要满足对齐要求；`call` 会压入返回地址；`ret` 从栈顶取回返回地址；某些寄存器若属于 callee-saved，被调用者使用前要保存，返回前要恢复。这些内容将在 ABI 章节展开。本章先建立阅读规则：看到 `rsp` 或 `rbp` 的地址公式，不要马上写“局部变量”，而要结合函数序言、调用点、寄存器保存和优化级别判断。

常见误区

栈槽是机器层存储位置，不是源码变量本身。一个源码变量可能没有栈槽，一个栈槽也可能在不同时间保存不同临时值。

lea：看起来像内存，实际不访问内存

`lea` 是学习 x86 汇编必须尽早掌握的指令。它的名字是 load effective address，但它只计算地址表达式，不读取内存。

```
lea rax, [rdi + rsi*4 + 8]
```

语义是：

```
rax = rdi + rsi * 4 + 8
```

不是：

```
rax = memory[rdi + rsi * 4 + 8]
```

对比：

```
lea rax, [rdi + rsi*4 + 8] ; compute address-like integer
mov eax, dword ptr [rdi + rsi*4 + 8] ; load 4 bytes from memory
```

编译器经常用 `lea` 做整数乘法常数：

```
lea eax, [rdi + rdi*2] ; eax = x * 3
lea eax, [rdi + rdi*4] ; eax = x * 5
lea eax, [rdi*8 + 16] ; eax = x * 8 + 16
```

这解释了为什么你在非指针代码里也会看到 `lea`：

```
extern "C" int mul5_add1(int x)
{
    return x * 5 + 1;
}
```

可能生成：

```
lea eax, [rdi + rdi*4 + 1]
ret
```

心智模型

看到 `lea` 时先问：它的结果后面被当作地址使用，还是只是整数计算结果？`lea` 本身不产生 cache miss，因为它不访问内存。

`lea` 的教学价值在于提醒我们：汇编语法中的方括号形式不总是意味着访存。对大多数指令，方括号表示内存操作数；但 `lea` 使用同样的寻址语法来计算有效地址，并把这个计算结果写进寄存器。

它复用了硬件和编码中表达地址公式的能力，却不执行 `load`。

编译器喜欢 `lea`，因为它可以在一条指令中表达若干简单整数运算。它常用于计算指针，也常用于计算整数表达式，尤其是乘以 3、5、9 这类可以表示成基址加索引乘缩放的常数。读到 `lea` 时，不要被名字中的 `load` 误导；真正的判断标准是它是否读取内存。答案是否定的。

当然，`lea` 不是万能乘法器。它支持的缩放只有 1、2、4、8，并且表达式形状有限。复杂常数乘法可能需要多条 `lea`、移位、加减或 `imul`。这部分细节留到整数指令章节。第 6 章只要求你建立底线：`lea` 计算数值，不访问内存；它可能服务于地址，也可能服务于普通整数。

flags：有些指令会留下条件信息

x86 有 RFLAGS/EFLAGS。很多整数指令会更新 flags，例如 `add`、`sub`、`cmp`、`test`。后续条件跳转或条件移动会读取这些 flags。

本章先只建立直觉：

flag	名称	常见用途
ZF	zero flag	结果是否为 0，相等比较
SF	sign flag	结果最高位是否为 1
CF	carry flag	无符号进位/借位
OF	overflow flag	有符号溢出

例如：

```
cmp edi, esi
setg al
```

`cmp edi, esi` 本质上计算 `edi-esi` 来设置 flags，但不保存减法结果。`setg al` 根据 flags 把 `al` 设置为 0 或 1。控制流章节会系统讲条件码；这里只要记住：读汇编时，有些数据依赖不在通用寄存器里，而在 flags 里。

flags 是读汇编时最容易漏掉的隐式状态。很多指令没有把结果写进普通寄存器，却把条件信息写进 flags；后面的条件跳转、条件设置或条件移动再读取这些信息。若你只跟踪通用寄存器，就会看不懂为什么一条 `jle` 或 `setne` 能做出选择。

还要注意 flags 会被覆盖。两个会写 flags 的指令如果相邻出现，后者会覆盖前者留下的条件信息。于是条件跳转通常要紧跟产生条件的 `cmp`、`test` 或算术指令，除非中间指令不影响 flags。读控制流时，必须找到每个条件跳转读取的是哪一次 flags 写入。第 8 章会把这件事作为控制流恢复的核心方法。

标签、函数和汇编器伪指令

看一段 `clang++-S` 生成的汇编，除了真正的机器指令，还会看到伪指令：

```
.text
.globl add2
.type add2,@function
add2:
mov    eax, edi
add    eax, esi
ret
.size  add2, .-add2
```

常见项：

写法	含义
<code>.text</code>	后面进入代码段
<code>.globladd2</code>	符号 <code>add2</code> 对链接器可见
<code>.typeadd2,@function</code>	标记符号类型是函数
<code>add2:</code>	标签，代表当前位置地址
<code>.sizeadd2,.-add2</code>	记录函数大小

这些伪指令不是 CPU 执行的指令。它们给汇编器、链接器、调试器和异常展开机制使用。后续讲 ELF、链接、ABI 和 unwinding 时会展开。

初学阶段可以把汇编文件分成两类行：一类是 CPU 将来会执行的指令，一类是给工具链看的描述。标签介于两者之间：标签本身不是指令，但它命名了一个地址，跳转、调用、符号表和调试信息都可能引用它。伪指令通常以点开头，它们告诉汇编器如何组织输出，而不是告诉 CPU 做一次计算。

这个区分会影响反汇编阅读。你在 .s 文件中看到的伪指令，不一定会出现在 objdump 的指令列表里；反过来，反汇编会显示机器码地址和字节，这些在普通 .s 文件中未必直接出现。若报告要解释机器真正执行了什么，应以反汇编中的指令为准；若报告要解释符号、段和汇编器输入，则需要看 .s。

函数标签也不等于高级语言函数的全部语义。一个标签只是一个地址名字。函数边界、栈展开、调试行号、异常处理和符号可见性，需要伪指令、调试信息、ABI 和链接器共同配合。后续 ABI 章节会讲为什么函数调用不仅是 call 和 ret，还包括参数、返回值、栈对齐和寄存器保存约定。

Intel 语法和 AT&T 语法

本书默认使用 Intel 语法，因为它和 Intel/AMD 手册、Compiler Explorer 的 Intel 输出、很多性能资料更接近。你仍然必须认识 AT&T 语法，因为 GNU 工具链和很多历史资料会用它。

同一条指令：

```
mov eax, dword ptr [rdi + rsi*4] ; Intel
movl (%rdi,%rsi,4), %eax      ; AT&T
```

关键差异：

差异	Intel	AT&T
操作数顺序	destination, source	source, destination
寄存器写法	rax	%rax
立即数写法	42	\protect\TU\textdollar42
内存写法	[rdi+rsi*4]	(%rdi,%rsi,4)
宽度表示	dwordptr	指令后缀 movl

再看几个对应关系：

```
Intel: mov rax, rdi
AT&T : movq %rdi, %rax

Intel: add eax, esi
AT&T : addl %esi, %eax

Intel: mov qword ptr [rsp + 8], rax
AT&T : movq %rax, 8(%rsp)
```

常见误区

最容易错的是操作数顺序。Intel 是 dst,src，AT&T 是 src,dst。读资料时先确认语法，否则会把数据流方向看反。

从函数签名建立入口状态

读一个函数的汇编，第一步不是看第一条指令，而是看函数签名。函数签名告诉你参数个数、参数类型和返回类型；ABI 告诉你这些参数在函数入口时大致放在哪里。没有入口状态，后面的寄存器含义就没有根。

例如一个函数签名是“接收一个指针、一个整数下标、一个整数增量，返回整数”。在常见 System V AMD64 ABI 下，前三个整数或指针类参数通常从 rdi、rsi、rdx 进入；如果参数类型是 32 位 int，你常会看到它的低 32 位别名，例如 esi 或 edx。因此在读第一条指令前，你应该先写入口笔记：rdi 是基址指针，esi 是下标，edx 是增量，返回值最终应在 eax。

这个入口笔记不是最终答案，而是分析起点。编译器可以马上把参数复制到别的寄存器，可以把 32 位下标符号扩展到 64 位，可以把某个参数与常量合并，也可以完全删除某个未使用参数。只要你从入口状态开始逐条更新，就能跟上这些变化。反过来，如果你看到 `rax` 就猜“这一定是返回值”，很容易出错；`rax` 在函数中间也可能只是临时寄存器。

入口状态还要包含你不知道的内容。比如函数接收一个指针，你通常不知道它是否为空、是否对齐、指向多大对象、是否与另一个指针别名。汇编只告诉你程序按某个地址读取或写入，不能自动证明这个地址在 C++ 语义上合法。读汇编时，要把“寄存器里有一个地址值”和“这个地址一定指向有效对象”分开。后者需要语言规则、调用约定和程序上下文共同证明。

心智模型

函数签名给出源码层入口，ABI 给出机器层入口，逐条指令更新当前状态。读汇编时不要从中间的某个寄存器名字开始猜，要从入口状态开始推。

确定事实、合理推论和待验证假设

高质量汇编分析必须区分三类说法：确定事实、合理推论、待验证假设。三者都可以出现在报告里，但不能混在一起。

确定事实来自汇编文本和 ABI 规则。例如某条指令读取 `rdi`，写 `eax`；某个内存操作数的有效地址是 `rdi` 加 `rsi` 乘 4；某条 `mov` 访问 4 字节；函数返回值按 ABI 放在 `eax`。这些事实可以直接从指令、宽度标记和调用约定读出。

合理推论是在事实之上结合上下文得到的解释。例如 `[rdi+rsi*4]` 很可能是四字节元素数组访问；`[rdi+rax*8+16]` 可能是结构体数组中的字段；连续的 `cmp` 和条件跳转可能来自一个 `if`；`lea eax, [rdi + rdi*4 + 1]` 可能来自 `x*5+1`。这些推论有依据，但不是机器文本本身直接写出来的。

待验证假设是还缺证据的解释。例如“这里慢是因为 cache miss”，“这里一定来自某个类成员函数”，“这个 +24 一定是第三个字段”，“这条间接调用一定调用某个虚函数”。这些说法可能正确，但需要更多证据：源码、类型布局、调试信息、符号、运行时采样、硬件计数器或调试器观察。没有证据时，应该把它写成假设，而不是结论。

这个区分看似麻烦，其实能显著提高分析质量。很多汇编报告的问题不是看错了指令，而是把推论写成事实。比如看到 `mov eax, dword ptr [rdi + 4]` 就说“读取结构体第二个字段”，这可能对，也可能不对。更严谨的写法是：这条指令从 `rdi` 加 4 的地址读取 4 字节；若 `rdi` 指向一个首字段大小为 4 字节且第二字段也是 4 字节的结构体，则它可能读取第二字段。

核心理想

汇编分析的可信度来自证据等级。机器事实要写准，源码解释要标明推论条件，性能判断要说明还需要哪些测量证据。

从汇编反推源码不是唯一答案

很多练习会让你“根据汇编写出可能的 C 代码”。这类练习有价值，但要知道：从汇编反推源码通常不是唯一答案。不同源码可以生成相同或相近汇编。

例如：

```
lea eax, [rdi + rdi*4 + 1]
ret
```

它可能来自：

```
return x * 5 + 1;
```

也可能来自：

```
return x + 4 * x + 1;
```

还可能来自某个被内联函数、模板常量表达式、数组下标计算或优化器代数化简后的结果。汇编确定的是“返回 `edi` 乘 5 加 1 的 32 位结果”，不确定原始源码究竟写成哪种表达式。

再比如：

```
mov eax, dword ptr [rdi + 16]
ret
```

它可能是结构体字段读取，也可能是数组偏移读取，也可能是某种对象头后面的值。只有当你知道函数签名、类型布局、调用上下文和源码，才能更有把握地解释。

因此，汇编阅读报告不应写得过度确定。可以使用下面的表达方式：

- “机器事实是访问 `rdi` 加 16 处的 4 字节。”
- “若 `rdi` 指向 `S` 对象，且 `c` 字段偏移为 16，则这对应读取 `p->c`。”
- “当前尚未证明该地址一定来自结构体字段；需要源码或调试信息确认。”

这种谨慎不是多余文字，而是专业习惯。性能调试中，过度确定的错误解释比“不知道”更危险。

证据强度排序

同一个判断可能有不同强度的证据。比如判断某个函数是否在热路径中：

- 源码中出现调用：证据弱，因为可能被内联、删除或走不到。
- 优化后反汇编有 `call`：证据较强，说明该二进制路径里有调用指令。
- profiling 显示采样落在该函数：证据更强，说明运行时确实执行并消耗时间。
- 控制输入和计数器后多次复现：证据最强，说明结论稳定。

判断一个地址公式来源也是类似：

- 看到乘 4：只能说明地址中有 `scale 4`。
- 结合访问宽度 4 和函数签名 `intconst*`：可以推论是 `int` 数组访问。
- 结合源码和调试信息：推论更强。
- 用 GDB 打印地址和内存，或用 `offsetof` 验证字段偏移：证据更完整。

本书要求你在报告里主动标注证据强度。尤其是性能解释，不要因为看见一条 `load` 就断言瓶颈是内存；不要因为看见一条 `branch` 就断言瓶颈是分支预测；不要因为指令条数变少就断言一定更快。汇编给出事实，瓶颈还需要测量。

-00 和 -03 为什么差异巨大

同一个 C++ 函数，在不同优化级别下可以完全不像同一个程序。看：

```
extern "C" int f(int x)
{
    int y = x + 1;
    int z = y * 3;
    return z - 2;
}
```

-00 的目标是容易调试，不是快。它可能生成栈帧，把局部变量放在栈上：

```
push rbp
mov rbp, rsp
mov dword ptr [rbp - 4], edi
mov eax, dword ptr [rbp - 4]
add eax, 1
mov dword ptr [rbp - 8], eax
mov eax, dword ptr [rbp - 8]
imul eax, eax, 3
mov dword ptr [rbp - 12], eax
mov eax, dword ptr [rbp - 12]
sub eax, 2
pop rbp
ret
```

-03 会做代数化简和寄存器分配，可能变成：

```
lea eax, [rdi + rdi*2 + 1]
ret
```

因为：

```
y = x + 1
z = y * 3 = 3*x + 3
return z - 2 = 3*x + 1
```

这个案例说明：

- -00 适合观察源代码结构、栈帧和调试信息。
- -03 适合观察真实性能路径。
- 性能优化不能根据 -00 汇编下结论。
- -03 汇编可能已经和源码结构差很多，需要恢复语义而不是逐句对应。

-00 和 -03 的差异不是“一个粗糙、一个聪明”这么简单。它们服务的目标不同。-00 尽量保留源代码级调试体验，让调试器能在某个源码行上看到局部变量，能单步经过看似对应的语句，能较少因为重排和删除让读者困惑。因此它常常把变量放入栈槽，频繁从内存读回，再写回内存。这个形态对学习栈帧和调试器很有用，但不能代表发布程序的热路径。

-03 则以优化后的可观察行为为准，不承诺保留源码形状。局部变量可以消失，语句顺序可以改变，函数可以内联，分支可以合并，循环可以展开，甚至整个计算可以被常量折叠。读 -03 汇编时，你的任务不是寻找“哪条指令对应哪行源码”，而是证明优化后的机器行为仍然实现了源码要求的结果，并找出真正执行的热路径。

在教材训练中，两种输出都要看，但用途不同。-00 适合观察编译器如何为调试构造保守形态，也适合初学栈帧和局部变量位置。-03 适合观察性能相关路径，适合学习编译器如何利用寄存器、地址公式和代数化简。把两者混用会产生错误结论：用 -00 判断性能会夸大内存访问，用 -03 学源码级单步调试又会觉得变量“莫名其妙消失”。

常见误区

报告中必须说明你分析的是哪种优化级别。没有优化级别，汇编分析不完整；用调试构建证明性能结论，通常不合格。

机器码字节和指令边界

汇编文本是给人看的，机器最终执行的是字节。x86-64 是变长指令集，一条指令的长度不固定。短的指令可能只有 1 字节，带寄存器编码、位移、立即数、前缀或 REX 扩展的指令会更长。反汇编工具把字节流重新切分成指令，并显示地址、字节和助记符。

学习机器码字节不是为了让你手写编码表，而是为了建立两个关键直觉。第一，rip 顺序前进时，下一条指令地址等于当前地址加当前指令长度。控制流章节讲跳转时，这个事实会解释为什么相对跳转目标要根据当前指令结束位置计算。第二，指令长度会影响取指和前端压力。第二册深入 CPU 前端时，变长指令、解码带宽和代码尺寸会成为性能因素。

立即数和位移常常让指令变长。例如把一个小寄存器加到另一个寄存器，编码可能很短；如果指令还带有 32 位立即数，机器码就要额外存放这个常量；如果内存操作数带有较大的 displacement，也要把位移编码进指令。你在 objdump 里看到一条指令前面有很多十六进制字节，不要只看助记符，要问哪些字节来自操作码，哪些来自寄存器和寻址编码，哪些来自立即数或位移。

机器码字节还能帮助你理解反汇编和汇编文件的区别。汇编文件中的标签没有固定地址，直到汇编和链接后才有位置；反汇编中的地址和字节则来自已经生成的目标文件或可执行文件。若你要证明某个函数中真的存在某条机器指令，反汇编比源码和 .s 更接近证据。若你要解释编译器输出给汇编器的符号和伪指令，.s 又更直接。

深入理解

本章不要求手工编码 x86 指令，但要求你不再把汇编文本当作最终执行物。机器码字节、指令长度和地址，是后续理解 rip、跳转、代码布局和前端性能的基础。

读汇编的固定流程

以后每次读一个函数，按下面流程做笔记：

1. 写出函数签名和 ABI 参数位置。
2. 标出每个入口寄存器的含义，例如 `rdi` 是第一个指针，`esi` 是第二个 `int`。
3. 从上到下维护寄存器含义表。
4. 对每条内存操作写出有效地址公式。
5. 标出访问宽度和读写字节范围。
6. 标出哪些指令更新 `flags`，后面哪里读取 `flags`。
7. 区分真正 `load/store` 和纯地址计算 `lea`。
8. 最后再尝试恢复 C-like 伪代码。

例如：

```
movsxd rsi, esi
mov    eax, dword ptr [rdi + rsi*4 + 8]
add    eax, 1
ret
```

笔记应该像这样：

```
entry:
    rdi = base pointer p
    esi = int index i

movsxd rsi, esi
    rsi = sign_extend_64(i)

mov    eax, dword ptr [rdi + rsi*4 + 8]
    address = p + i * 4 + 8
    load width = 4 bytes
    possible source = p[i + 2] if p is int*

add    eax, 1
    eax = loaded_value + 1

ret
    return eax
```

这种笔记比“这大概是数组访问”更有价值。高性能工程里，证据链必须精确到地址公式和访问宽度。

这个流程看起来繁琐，但它的作用是减少猜测。初学者常常一看到熟悉的指令就跳到结论，例如看到 `lea` 就说“取地址”，看到 `+8` 就说“第三个元素”，看到 `ret` 就说“返回上一行”。固定流程强迫你先写事实：入口寄存器是什么，当前寄存器含义如何变化，内存地址公式是什么，访问宽度是多少，`flags` 在哪里产生和消费。事实写清楚后，结论通常自然出现。

流程中的最后一步才是恢复 C-like 伪代码。伪代码应该尽量贴近机器行为，而不是强行恢复原始源码格式。比如一个结构体数组字段更新，伪代码可以写成“读取地址 `base+i*12+8` 的 4 字节整数，加上 `delta`，写回同一地址，再把结果作为返回值”。这比直接写“`items[i].count+=delta`”更能展示你真的理解了地址公式和访存行为。

如果某一步缺证据，就把缺口写出来。例如你不知道 `rdi` 的源类型，可以先说它是第一个参数指针；如果没有源码和调试信息，不能断言它一定是某个结构体。工程报告不要求每个推论都完美，但要求推论边界清楚。

案例：从 C++ 到汇编逐条解释

创建函数：

```
struct Item {
    int id;
    float score;
    int count;
};
```



```
extern "C" int update(Item* items, int i, int delta)
{
    items[i].count += delta;
    return items[i].count;
}
```

常见布局：

字段	offset	大小	说明
id	0	4	int
score	4	4	float
count	8	4	int

`sizeof(Item)` 通常是 12。因为 x86 scale 没有 12，编译器可能使用 `lea` 生成 `i*3`，再乘 4：

```
movsxd rax, esi
lea    rax, [rax + rax*2]
add    dword ptr [rdi + rax*4 + 8], edx
mov    eax, dword ptr [rdi + rax*4 + 8]
ret
```

逐条解释：

```
entry:
    rdi = items
    esi = i
    edx = delta

movsxd rax, esi
    rax = sign_extend_64(i)

lea rax, [rax + rax*2]
    rax = i * 3

add dword ptr [rdi + rax*4 + 8], edx
    address = items + (i * 3) * 4 + 8
              = items + i * 12 + 8
              = addr(items[i].count)
    load old 4-byte count
    add delta
    store new 4-byte count
    update flags

mov eax, dword ptr [rdi + rax*4 + 8]
    reload 4-byte count into eax as return value

ret
```

为什么第二条 `mov` 要 reload？因为 `add dword ptr [mem], edx` 是 read-modify-write 内存指令，它把结果写回内存，但不把结果写入 `eax`。返回值需要在 `eax`，所以还要再 load。编译器也可能生成不同形式，比如先 load 到寄存器、加、store、再 return 寄存器。你要解释自己的输出。

深入理解

read-modify-write 指令表面是一条汇编，但微架构内部通常涉及 load、ALU、store，并会占用 load/store 相关资源。后面讲 uops、ports、store buffer 和原子指令时，这一点会变得非常重要。

汇编阅读报告的最低质量线

本章实验要求写报告，不是为了形式，而是训练证据链。一份合格的汇编阅读报告至少要让另一个读者在没有你口头解释的情况下复现你的判断。

报告开头应写清楚源码文件、编译器版本、编译命令、优化级别、目标平台和使用的反汇编工具。然后给出函数签名和 ABI 入口状态。对每个被分析函数，先列出参数寄存器和返回寄存器，再

摘出核心指令。不要把整个反汇编大段粘进去；应该选择与分析相关的指令，并说明省略了什么。

逐条注释要包含读写状态。最低要求是：操作数类型、读取寄存器或内存、写入寄存器或内存、访问宽度、是否更新 flags、指令后的寄存器含义变化。遇到内存操作数，必须写有效地址公式；遇到结构体或数组解释，必须说明元素大小或字段偏移依据；遇到 `lea`，必须明确它是否访问内存；遇到 `cmp` 或 `test`，必须说明后续哪条指令读取 flags。

报告还要包含推论边界。能从汇编直接看出的，写成事实；需要源码类型支持的，写成“若某类型布局成立，则”；需要运行时数据支持的，写成待验证假设。比如“这条指令从 `rdi+16` 读取 4 字节”是事实；“这可能是读取结构体字段 `c`”是推论；“这里慢是因为缓存未命中”则需要性能计数器或实验支持。

最后，报告要说明 `-00` 和 `-03` 的用途差异。如果你同时展示两者，应分别解释它们服务什么问题：调试构建帮助理解栈帧和变量可见性，优化构建帮助理解真实性能路径。不能把两份汇编混在一起得出一个没有上下文的性能判断。

核心思想

汇编阅读报告不是“贴代码加中文翻译”。它应当证明你能从函数签名和 ABI 建立入口状态，从指令读写恢复数据流，从内存操作数恢复地址公式，并清楚区分事实、推论和假设。

把汇编读成证据链，而不是读成故事

真正可靠的汇编阅读，最后应该形成一条证据链。证据链不是把源码、汇编、反汇编、运行输出都贴上去，而是说明这些材料分别证明了什么。源码证明程序员写下了哪些语言层意图；编译命令证明编译器在什么规则下工作；`.s` 文件证明编译器交给汇编器的文本是什么；目标文件证明哪些机器码、符号和重定位已经生成；链接后的可执行文件证明链接器最终把符号绑定到哪里；调试器或运行日志证明某次运行时寄存器和内存状态怎样变化。每一层都有自己的边界，混成一团就会出现看似合理、实则无法复现的结论。

例如，你在 `.s` 文件里看到：

```
call external_addr@PLT
```

这可以证明编译器输出中存在一次对外部符号的调用意图，但不能直接证明运行时一定跳到哪个共享库地址。要证明目标文件阶段还没有确定最终地址，需要看 `objdump-drwC` 的重定位旁注或 `readelf-r` 的重定位项。要证明链接后地址，需要看可执行文件反汇编或动态链接信息。要证明某次运行绑定到哪个共享库实现，还要看 `ldd`、`LD_DEBUG`、`gdb` 中的符号解析，或者运行时映射。把这些层次分开，报告才有审查价值。

再看一个更常见的例子。你在反汇编里看到：

```
mov eax, dword ptr [rdi + rsi*4 + 8]
```

这条指令直接证明的事实只有几个：读取 `rdi`、`rsi`，计算地址 `rdi+rsi*4+8`，从该地址读取 4 字节，写入 `eax`，并因为写 32 位寄存器而清零 `rax` 高 32 位。它不能单独证明这里一定是 `p[i+2]`，也不能单独证明 `rdi` 一定指向 `int` 数组。若你有函数签名 `intf(intconst*p, longi)`，则可以把 `rdi` 和 `rsi` 分别解释成 `p` 和 `i`，并推导 `+8` 可能是两个 `int` 元素的偏移。若源码是结构体数组，则 `+8` 也可能是字段 `offset`。结论要依赖签名、类型布局和上下文。

心智模型

证据链写法的核心句式是：“从某个材料可以直接看到什么；结合哪个规则可以推出什么；仍然不能断言什么”。这比直接写“这一行表示访问数组”更慢，但更接近工程分析。

四层材料的常见用途

第一层材料是编译输入和命令。它包括源码、头文件、编译器版本、优化级别、目标 CPU、是否使用 `-fPIC`、是否开启调试信息、是否保留帧指针、是否链接时优化。没有这一层，汇编输出没有环境。相同源码在 `-00`、`-02`、`-03` 下可以完全不同；相同 `-03` 在不同编译器版本下也可能选择不同指令形态。报告里不要只写“生成汇编”，而要写完整命令。

第二层材料是 `.s`。它适合观察编译器给汇编器的结构意图：函数标签、section 切换、伪指令、对齐、局部标签、常量池、展开信息和符号可见性。对于理解“编译器为什么把某个常量放进 `.rodata`”“为什么某个函数被标为全局符号”“为什么循环入口前有对齐伪指令”，`.s` 往往比反汇编更直观。

第三层材料是目标文件。它适合观察已经编码的机器码、符号表、重定位表和 section header。目标文件仍未完成所有地址绑定。外部符号、位置无关代码中的全局对象访问、跨 section 引用，都可能留下重定位项。读目标文件时要习惯同时看三类输出：反汇编、符号表、重定位表。只看反汇编会漏掉“这条 `call` 的目标还要链接器修补”这种关键信息。

第四层材料是链接后产物和运行时观察。链接后，函数和对象有了更具体的地址布局，PLT/GOT、动态重定位、可执行文件入口、只读段和可写段关系更清楚。运行时，地址还会受到 ASLR、动态加载器、共享库版本和内核映射影响。若分析目标是性能或崩溃，最终还需要运行时证据：寄存器、栈、内存、分支路径、采样位置、性能计数器。反汇编解释的是可能执行什么，运行时证据说明某次或某类输入实际执行了什么。

不要把不同层次的地址混用

地址是初学者最容易混淆的对象。汇编文件里的标签没有最终虚拟地址；目标文件里的 section 偏移不是进程运行时地址；链接后可执行文件中的虚拟地址可能还会被 PIE 和 ASLR 改变；GDB 里看到的地址是某次运行的加载地址。它们都可以写成十六进制数，但含义不同。

一个严谨的报告应该说明地址属于哪一层。例如“目标文件 `foo.o` 的 `.text` section 中 offset `0x20` 处有一条 `call`”和“运行时进程地址 `0x7f...` 处执行 `call`”不是同一句话。前者用于解释目标文件结构，后者用于解释某次运行。若要把两者对应起来，需要知道 section 加载基址、符号地址和调试信息。

同样，跳转目标也有层次。目标文件反汇编里，某个 `jmp` 可能显示为相对当前 section 的标签；链接后可能显示为函数内偏移；运行时采样工具可能显示为模块名加偏移。不要因为三个输出显示的十六进制不同，就认为程序变了；也不要因为助记符相同，就认为地址已经一致。高质量分析要说明“这些地址如何对应”。

从机器状态表开始读陌生函数

陌生函数最稳的读法不是先猜源码，而是先建立机器状态表。机器状态表记录每个关键时刻寄存器、内存别名、flags 和栈指针的大致含义。它不需要记录所有寄存器，但必须记录当前函数会读取、写入或跨调用保持的状态。

函数入口状态来自 ABI。对于常见 System V AMD64 整数参数，`rdi`、`rsi`、`rdx`、`rcx`、`r8`、`r9` 是前六个整数或指针参数的通道；返回整数通常在 `rax`。如果函数使用浮点参数，还要看 `xmm0` 等寄存器。入口状态不是从第一条 `mov` 猜出来的，而是由函数签名和 ABI 共同给出。

接着从上到下维护“当前含义”。例如：

```
entry:
    rdi = p
    esi = i
    edx = delta

movsxd rax, esi
    rax = sign_extend_64(i)

lea rcx, [rax + rax*2]
    rcx = i * 3

mov eax, dword ptr [rdi + rcx*4 + 8]
    eax = load32(p + i * 12 + 8)
```

这种记录的价值在于，它把“寄存器当前是什么”从脑子里拿出来，变成可以审查的材料。只要中间某一步写错，后续地址公式就会暴露矛盾。比如你把 `rcx` 误记成 `i*4`，下一条地址就会变成 `p+i*16+8`，与结构体大小不符。机器状态表能迫使你及时发现这种错误。

状态表要记录失效，而不只记录赋值

寄存器含义不是永久有效的。任何写寄存器的指令都会覆盖旧含义；任何普通函数调用都可能破坏 caller-saved 寄存器；任何写 flags 的指令都会覆盖前一个比较结果；任何 store 都可能改变某个内存位置。状态表必须记录这些失效。

例如：

```
cmp edi, esi
lea eax, [rdi + 1]
jl .Lless
```

这里 lea 不写 flags，所以 jl 仍然读取 cmp 产生的 flags。若中间换成：

```
cmp edi, esi
add eax, 1
jl .Lless
```

那么 add 会写 flags，jl 读取的就不再是 cmp 的结果。真实编译器不会无意生成这种错误，但手写汇编和人工改写很容易犯。状态表若只记录寄存器值，不记录 flags 生命周期，就读不出这个问题。

函数调用也是同样道理。看到 call 之后，除非 ABI 或内联上下文能证明，否则不要继续相信 rax、rcx、rdx、rdi、rsi、r8 到 r11 的旧值还在。若调用前某个值调用后还要用，编译器通常会把它放到 callee-saved 寄存器、栈槽，或者在调用后重新计算。报告中若跨 call 使用旧寄存器含义，必须解释保存证据。

内存状态要写别名假设

寄存器状态比较容易记录，内存状态更难，因为多个地址可能指向同一对象。假设你看到：

```
mov dword ptr [rdi], 1
mov eax, dword ptr [rsi]
```

如果 rdi 和 rsi 可能相等，那么第二条 load 可能读到刚写入的 1；如果根据函数契约、类型或 restrict-like 信息可以证明不别名，第二条 load 才能独立解释。汇编本身只显示两个地址来源，不显示语言层别名证明。报告要写清楚：这里是否假设两个指针不重叠，证据是什么。

别名问题在结构体字段、数组切片和手写汇编包装函数中尤其常见。两个地址公式看起来不同，不代表一定不重叠；同一个 base 加不同 offset，若访问宽度交叉，也可能部分重叠。读汇编时，内存记录至少要包含 base、index、scale、displacement、访问宽度和已知对象范围。只有这样，才能判断 store 是否可能影响后续 load。

常见误区

“寄存器含义表”如果不记录 flags 失效、调用破坏和内存别名，只是半张表。完整的机器状态分析必须同时覆盖寄存器、flags、栈和内存。

用反例校正常见直觉

汇编入门最危险的不是不会，而是会了一半后形成过度简化的直觉。本节列出几个经常导致错误报告的反例。

第一，lea 不等于“取地址”。它的操作数写法像内存，但不访问内存。编译器常用它做加法、常数乘法和地址中间量。若 lea eax, [rdi + rsi*4] 出现在整数函数里，可能只是计算 $x+y*4$ ，并没有形成可解引用指针。是否是地址，取决于后续如何使用结果，而不取决于方括号外形。

第二，mov 不等于 C++ 赋值。C++ 赋值可能调用重载赋值运算符，可能写多个字段，可能涉及对象生命周期；mov 只是复制固定位宽的位模式。反过来，一个 C++ 赋值也可能完全消失，因为值被常量传播或保存在寄存器里。把 mov 机械翻译成“赋值语句”，会漏掉语言和机器之间的巨大差异。

第三，一个栈槽不等于一个源码局部变量。在 -O0 下，编译器可能为调试可见性给每个局部变量安排栈槽；在 -O3 下，多个临时值可能复用同一栈位置，或者根本没有栈槽。栈槽是编译器实现选择，不是 C++ 对象存在的唯一证据。对象生命周期要结合源码语义和调试信息判断，不能只看

[rbp-4]。

第四，反汇编的函数边界不总是源代码函数边界。内联会让被调用函数的指令进入调用者；尾调用会让一个函数以 `jmp` 结束；链接器可能插入 PLT、thunk 或安全检查序列；异常展开和冷路径分离会让函数代码不完全连续。看到一个符号名，只能说明工具按某种符号信息展示了一个范围，不能保证它和源码函数体一一对应。

第五，运行输出正确不代表汇编接口正确。一个手写汇编函数可能碰巧返回正确 `eax`，但破坏了 `rbx`；一个函数可能在小输入下栈未对齐也没崩溃，但在调用库函数或开启向量化后出错；一个参数扩展错误可能在正数测试中看不出来，一遇到负数或高位非零就失败。汇编正确性要检查契约，不只看样例输出。

汇编注释的层级写法

教材和工程报告里的汇编注释应该分层，而不是每行后面写一句模糊中文。建议使用三层注释。

第一层是机器语义注释。它只描述指令读写状态。例如“从 `[rdi+rsi*4]` 读取 4 字节到 `eax`”“写 `eax` 并清零 `rax` 高 32 位”“更新 ZF、SF、OF、CF 等 flags”。这一层要尽量客观，不引入源码猜测。

第二层是类型和布局注释。它结合函数签名、结构体布局、数组元素大小，把地址公式解释成对象访问。例如“若 `rdi` 是 `intconst*p`，`rsi` 是下标 `i`，则访问 `p[i]`”“若结构体大小为 12，offset 8 是字段 `count`，则访问 `items[i].count`”。这一层要说明前提。

第三层是意图和性能注释。它解释编译器为什么可能选择这种形态，以及这个形态对后续分析有什么影响。例如“使用 `lea` 是为了计算 `i*3`，不写 flags”“内存目的 `add` 是 RMW，结果没有自动进入返回寄存器”“这里选择 `movzx` 是因为无符号字节提升到 `int`”。这一层可以提出推论，但要避免把推论写成事实。

一个高质量注释块可以写成：

```
movzx eax, byte ptr [rdi + rsi]
machine:
    load 1 byte from rdi + rsi
    zero-extend to 32 bits
    write eax, clear high half of rax
layout:
    if rdi = p and rsi = i, this reads p[i]
source-level:
    matches uint8_t load promoted to int
boundary:
    does not prove i is in bounds; bounds are a C++ precondition
```

这种写法比单行“读取数组元素”长，但它一次性说明了机器宽度、扩展方式、类型依据和边界条件。对复杂函数，不需要每条指令都写四层，但关键 `load`、`store`、`flags`、`call`、`ret`、地址计算和跨边界指令必须达到这个精度。

优化汇编阅读：先接受源码形状已经消失

读 -03 汇编时，最大的心理障碍是源码形状消失。局部变量消失、函数消失、循环形态改变、分支变成条件移动、乘法变成 `lea`、小函数被内联、常量被提前算出，这些都不是异常，而是优化构建的正常结果。优化器的目标不是保留源码外形，而是在语言规则允许的范围内生成更合适的机器程序。

因此，优化汇编阅读要从“机器现在做什么”开始，而不是从“源码原来长什么样”开始。你可以把分析分成三轮。

第一轮只读机器事实。列出函数边界、入口寄存器、核心基本块、`load/store`、`call`、`ret`、`flags` 生产和消费。此时不要急着写 C++ 伪代码。机器事实应能在没有源码的情况下成立。

第二轮结合源码类型。把入口寄存器映射到参数，把 `displacement` 映射到字段 `offset`，把 `scale` 映射到元素大小，把条件码映射到 `signed/unsigned` 语义，把返回寄存器映射到返回类型。此时可以写“若源码类型为某结构，则该地址公式对应某字段”。

第三轮解释优化形态。问编译器为什么可以这样做：是否内联、是否常量传播、是否删除无用计算、是否利用未定义行为前提、是否因为别名信息受限而保留 `load`、是否因为 ABI 边界无法继续优化。优化解释必须建立在前两轮事实之上。

案例：函数消失不等于代码没执行

考虑：

```
static int twice(int x)
{
    return x * 2;
}

extern "C" int caller(int x)
{
    return twice(x) + 1;
}
```

优化后你可能找不到独立的 `twice` 符号，`caller` 中只剩：

```
lea eax, [rdi + rdi + 1]
ret
```

这不表示 `twice` “没有被执行”，而是它的语义被内联并合并到调用者中。报告应写：“目标二进制中没有独立 `twice` 函数符号；`caller` 的指令直接计算 `x*2+1`；因此当前构建下 `twice` 的函数调用开销不存在。”不要写“编译器把函数删错了”。

如果实验目标是测函数调用开销，就需要阻止内联或把函数放到单独编译单元，并说明这样做改变了实验问题。如果实验目标是真实发布性能，则内联是正常路径，应接受它并分析内联后的机器码。

案例：常量折叠让循环消失

考虑：

```
extern "C" int sum_const()
{
    int s = 0;
    for (int i = 0; i < 100; ++i) {
        s += i;
    }
    return s;
}
```

优化后可能直接返回常量：

```
mov eax, 4950
ret
```

机器事实是：当前二进制没有循环，只有一次立即数写入返回寄存器。源码解释是：循环边界和每次加数都是编译期常量，且无副作用，编译器可以在编译期算出结果。性能解释是：这个函数不能用于测循环加法吞吐，因为被测循环不存在。

这类例子是 benchmark 中最常见的陷阱。你以为测了一亿次加法，其实测的是返回一个常量。防止它的方法不是关闭所有优化，而是让输入来自运行时、让结果被观察、并用反汇编确认热路径存在。关闭优化会把实验变成调试构建性能，也不是你想测的发布路径。

案例：符号存在不代表热路径调用它

有时 `nm` 显示某个函数符号存在，但调用点仍然没有调用它。原因可能是其他调用点需要该符号，或函数被导出供外部使用，但当前热路径已经内联了它。符号表只能证明“二进制中有这个符号”，不能证明“当前输入执行了这个符号”。

要证明热路径调用某函数，需要更强证据：

- 调用点反汇编中存在 `call` 到该符号或其 PLT 入口。
- 运行时采样显示指令指针落在该函数内。
- GDB 断点在该函数上能被当前输入触发。
- 计数器或 tracing 记录该函数调用次数。

证据强度不同，结论也不同。`nm` 只能支持“函数存在”；反汇编调用点支持“机器码包含调用”；

GDB 或采样支持“这次运行走到了那里”。报告必须匹配证据强度。

从反汇编恢复函数边界的谨慎规则

反汇编工具通常根据符号表、section、调试信息和指令流显示函数。但函数边界并不是天然写在每条机器码旁边。优化、链接器布局、热冷拆分、尾调用、异常表和手写汇编都可能让边界看起来不直观。

恢复函数边界时，先看符号表。全局函数、局部函数、弱符号、PLT 条目、thunk 和冷路径符号可能都出现在输出中。再看 `.type` 和 `.size` 信息，判断工具认为函数范围从哪里到哪里。若没有大小信息，反汇编器可能用下一个符号作为边界，这在手写汇编或 stripped 二进制中可能不可靠。

其次看控制流。函数入口通常是调用目标或导出符号；函数结束可能是 `ret`，也可能是尾调用 `jmp`，也可能是不返回调用。一个 `ret` 后面出现字节，不代表同一函数继续执行；可能是下一个函数、对齐填充、跳转表或冷路径。反过来，一个函数也可能有多个返回点。

尾调用让调用栈外观改变

如果函数最后只返回另一个函数的结果，优化器可能把 `call` 加 `ret` 改成 `jmp`。这会让当前函数不再建立普通返回帧。反汇编中你会看到：

```
wrapper:
    jmp target
```

这不是缺少 `ret`，而是尾调用。读者要问：参数是否已经按目标函数 ABI 放好？当前函数是否已经恢复栈和 callee-saved 寄存器？如果都满足，尾跳合法。调试和 profiling 中，`wrapper` 可能不显示为普通调用帧，这也是机器码形态导致的正常现象。

热冷拆分让一个函数不连续

优化器和链接器可能把不常执行的错误路径、异常路径或冷分支放到远离热路径的位置。这称为热冷拆分或类似布局优化。源码上一个函数，在二进制中可能显示为热部分和冷部分，符号名也可能带有 `cold`。性能分析时，热路径和冷路径要分开看。冷路径代码大，不代表核心循环大；热路径短，也不代表函数语义简单。

若报告分析函数成本，应说明分析的是热路径、完整函数，还是某个输入触发的路径。把冷错误处理路径和热循环混在一起，会误判代码尺寸、分支结构和指令计数。

汇编阅读中的证据等级

为了避免过度推断，可以把汇编阅读结论分成四个等级。

第一等级是文本事实。比如“反汇编中存在 `mov eax, edi`”“目标文件有 `R_X86_64_PLT32` 重定位”“符号表中 `foo` 是 `undefined`”。这些可以直接从工具输出看到。

第二等级是机器语义。比如“该指令把 `edi` 的低 32 位复制到 `eax` 并清零高 32 位”“该 `call` 目标需要链接器修补”“该内存操作读取 4 字节”。这些来自 ISA 和目标文件规则。

第三等级是源码映射。比如“若 `rdi` 是参数 `p`，则该 `load` 读取 `p[i]`”“这个 `+8` 对应结构体字段 `count`”。这些需要函数签名、类型布局和源码上下文。

第四等级是性能解释。比如“这里可能受内存带宽限制”“这个 `cmov` 可能避免分支错误预测但增加数据依赖”。这些需要测量或至少明确假设。不能把第四等级写成第一等级。

等级	例子	需要的证据
文本事实	存在某条指令	反汇编输出
机器语义	指令读写哪些状态	ISA 语义
源码映射	地址对应哪个字段	源码类型和布局
性能解释	为什么快或慢	测量、计数器、对照实验

这个分级能保护报告。比如你可以确定地写“这里有一次 4 字节 `load`”，谨慎地写“结合源码它应是读取 `count` 字段”，再把“它可能因随机访问造成 `cache miss`”列为待验证假设。

读优化汇编时的十个固定问题

每次分析优化构建，建议按下面十个问题检查：

1. 这个函数是否仍有独立符号，还是被内联或删除。
2. 调用点是否真的调用它，还是调用被优化掉。
3. 入口寄存器是否能从 ABI 和函数签名解释。
4. 局部变量是否仍有对应存储，还是只在寄存器或完全消失。
5. 循环是否仍存在，是否被展开、向量化或常量折叠。
6. 分支是否仍是分支，还是变成 `setcc/cmovcc/mask`。
7. 内存访问是否对应源码对象，是否有额外 reload 或 store。
8. 是否存在外部调用、PLT/GOT、间接调用或尾调用。
9. 反汇编来自目标文件、可执行文件还是运行时映射。
10. 当前结论是事实、源码映射还是性能假设。

这十个问题看似繁琐，但它们能防止最常见的误读：把源码函数当成机器函数，把符号存在当成执行路径，把 `-O0` 形态当成性能路径，把一次地址公式当成唯一源码写法，把反汇编片段当成完整性能证据。

综合案例：读一个陌生函数的完整过程

假设你只拿到下面反汇编和函数声明：

```
struct Rec {
    std::int32_t id;
    std::int32_t score;
    std::int64_t total;
};

extern "C" std::int64_t update_score(Rec* p, int i, int delta);
```

反汇编核心片段：

```
movsxd rax, esi
shl    rax, 4
add    dword ptr [rdi + rax + 4], edx
movsxd rcx, dword ptr [rdi + rax + 4]
add    qword ptr [rdi + rax + 8], rcx
mov    rax, qword ptr [rdi + rax + 8]
ret
```

第一轮写机器事实。入口 `rdi` 是第一个参数，`esi` 是第二个参数低 32 位，`edx` 是第三个参数。第一条把 `i` 符号扩展到 64 位，第二条把它乘以 16。第三条对地址 `rdi+i*16+4` 做 4 字节 read-modify-write，加上 `delta`。第四条从同一地址读取 4 字节并符号扩展到 64 位。第五条将该值加到地址 `rdi+i*16+8` 的 8 字节整数。第六条读取该 8 字节整数作为返回值。

第二轮结合布局。`Rec` 的字段 offset 很可能是：

```
id    offset 0, size 4
score offset 4, size 4
total offset 8, size 8
sizeof(Rec) = 16
```

因此 `rdi+i*16+4` 对应 `p[i].score`，`rdi+i*16+8` 对应 `p[i].total`。注意这个结论依赖结构体布局；若源码类型不同，同样地址公式可能有别的解释。

第三轮写 C-like 语义：

```
p[i].score += delta;
p[i].total += p[i].score;
return p[i].total;
```

第四轮写边界。汇编不能证明 `i` 在范围内，不能证明 `p` 非空，不能证明 `p` 指向至少 `i+1` 个对象，也不能证明并发访问安全。它只能证明当前机器码按该地址公式访问内存。C++ 合法性来自调

用者前置条件。

第五轮写性能假设。该函数对同一结构体元素做一次 4 字节 RMW、一次 4 字节 load、一次 8 字节 RMW、一次 8 字节 load。若在循环中随机访问 `p[i]`，可能受 cache 影响；若顺序访问，结构体大小 16 对 cache line 比较规整。但这只是静态假设，不能替代测量。

这个案例展示了完整报告的层次：机器事实、布局解释、伪代码、合法性边界、性能假设。缺任何一层，报告都不完整。

自测清单：汇编语法是否真正过关

完成本章后，应能独立回答：

1. `.s` 文件和 `objdump` 输出分别来自哪个阶段。
2. 伪指令、标签和 CPU 指令如何区分。
3. `.text`、`.data`、`.rodata`、`.bss` 的作用。
4. Intel 语法中目的操作数和源操作数顺序。
5. 内存操作数的 base、index、scale、displacement 如何组合。
6. 写 32 位寄存器为什么会影响 64 位寄存器。
7. `lea` 为什么不一定访问内存。
8. flags 生产者和消费者如何配对。
9. 函数入口寄存器如何从 ABI 推导，而不是从第一条指令猜。
10. 哪些结论能从反汇编直接看出，哪些必须依赖源码类型。

深入讲解：反汇编不是反编译

反汇编把机器码字节翻译成人类可读的指令文本；反编译试图恢复高级语言结构。两者目标不同。初学者常把反汇编当成“低级版源码”，于是期待每条指令都能对应一行 C++。这种期待会导致误读。

机器码没有 C++ 的变量名、作用域、模板、重载、对象生命周期、异常语义和完整类型信息。调试信息可以补充一部分，但优化后仍然可能不完整。反汇编能告诉你寄存器、内存、跳转和调用；不能单独告诉你某个地址一定属于哪个 C++ 对象，也不能证明某个源码变量仍然存在。

因此，反汇编阅读应保持双向克制。向下，要尊重机器事实：指令读写什么、地址怎么算、宽度是多少。向上，要谨慎恢复源码：只有结合函数签名、类型布局、符号、调试信息和上下文，才能写 C-like 伪代码。伪代码是分析产物，不是证据本身。

为什么工具显示会不同

不同反汇编工具可能显示不同文本。同一条机器码，`objdump` 和 `llvm-objdump` 的符号命名、立即数格式、地址显示、重定位旁注、伪指令风格可能不同。Intel 和 AT&T 语法也不同。工具显示不同不代表机器码不同。

严肃比较时，要保存机器码字节和工具命令。若两个输出指令文本不同，先比较字节；若字节相同，只是显示差异。若字节不同，再回到构建命令、链接阶段和工具版本查原因。不要只凭视觉差异判断程序变化。

深入讲解：汇编学习要避免“指令表崇拜”

很多资料把汇编学习变成背指令表。指令表有用，但不是核心。真正的核心是状态变化和上下文。同一条 `mov`，在函数入口可能是参数搬运，在循环中可能是 load，在返回前可能是准备返回值，在 ABI 边界可能是保存调用者状态。同一条 `lea`，可能是地址计算，也可能是整数乘法。

学习一条指令时，要同时学：

- 它读哪些状态。
- 它写哪些状态。
- 它是否访问内存。
- 它是否改 flags。
- 它的操作数宽度如何决定。
- 它常出现在什么编译器模式里。

- 它在 ABI 或控制流上下文中可能承担什么角色。

这样学习，指令才会进入工程能力。单独背“`mov` 是传送指令”“`lea` 是装入有效地址”，反而容易形成误解。

课堂讲解：读汇编要先建立坐标系

很多人第一次打开反汇编，会从第一条指令开始逐行翻译。这样很容易迷路，因为你还不知道这段文本处在什么坐标系里。读汇编前要先建立四个坐标：文件坐标、函数坐标、控制流坐标和数据坐标。

文件坐标回答“这段指令来自哪个二进制”。同一个源码可以生成多个目标文件和多个可执行文件，目标文件中的地址、重定位和符号与最终可执行文件不同。报告中应写清工具命令、输入文件、构建参数和是否经过链接。只贴一段反汇编，不说明来源，读者无法判断重定位、地址和符号是否可信。

函数坐标回答“函数边界在哪里”。符号表、标签、`.type`、`.size`、反汇编分隔和调用目标都可以帮助确认函数范围。但优化后函数可能被内联、拆分、合并或冷路径外提。若函数标签不存在，不一定表示源码函数没有执行；若存在标签，也不一定表示它没有被其他路径绕过。函数坐标要结合构建和优化级别判断。

控制流坐标回答“执行路径如何移动”。线性地址顺序只是布局，不是语义顺序。条件跳转、无条件跳转、`fallthrough`、`call`、`ret`、跳转表和异常边都会改变路径。读者应先标出基本块，再读块内指令。否则容易把冷路径当成热路径，或者把跳过的代码当成必经代码。

数据坐标回答“每个寄存器和内存地址扮演什么角色”。入口寄存器来自 ABI，栈槽可能是局部变量或保存寄存器，全局地址可能通过 `RIP-relative` 访问，结构体字段通过 `displacement` 暴露。数据坐标不是一次性确定的，寄存器角色会随基本块变化。读汇编报告应记录角色变化，而不是给某个寄存器贴永久标签。

综合案例：从陌生函数恢复最小语义

假设你拿到一个没有源码上下文的小函数，只知道它属于 `x86-64 System V` 程序。高质量阅读可以按六步进行。第一步，确认入口和出口。查看函数标签、`ret`、尾调用 `jmp`、栈调整和可能的不返回调用。第二步，确认 ABI 入口寄存器。若看到 `rdi`、`rsi`、`rdx` 在函数开始被使用，先把它们标成候选参数，而不是直接命名成具体变量。

第三步，分基本块。每个条件跳转和目标标签都可能产生新块。把块画出来后，先写路径条件，例如“若 `esi` 小于等于零则返回零”。第四步，恢复数据流。找出循环变量、累加器、指针递增、`load/store` 宽度和地址公式。第五步，结合类型猜测写 C-like 伪代码，但要标注不确定性。比如“可能是 `intconst*`，因为 `load` 宽度为 4 且下标步长为 4”。

第六步，写边界。反汇编不能证明指针非空，不能证明长度合法，不能证明别名关系，也不能证明源语言类型。它只能证明这份机器码按某些地址公式和控制路径执行。若报告把伪代码写得像源码原文，就过度解释了。正确写法是“在假设第一个参数是指向 32 位整数的指针、第二个参数是元素个数时，该函数等价于求和非负长度数组”。

把不确定性写出来

底层报告不是越肯定越好，而是证据和结论匹配越好越专业。你可以把不确定性分级。一级证据是机器事实：某指令读 `rdi` 加偏移，某块由 `jle` 跳入。二级解释是 ABI 和布局推断：`rdi` 可能是第一个指针参数，偏移 8 可能是结构体第二个字段。三级假设是源码恢复：这个字段可能叫 `size` 或 `total`。不同层级不能混写。

当证据不足时，可以列出需要补的观察：函数声明、调用点、调试信息、结构体定义、更多输入输出样本、GDB 单步、反编译辅助、符号表或重定位信息。写清需要什么证据，说明你知道当前结论的边界。

汇编阅读报告模板

本书建议每份汇编阅读报告固定包含以下字段：

1. 目标身份：源码提交、构建命令、二进制路径、工具命令。

2. 函数身份：符号名、地址范围、是否内联、是否外部可见。
3. ABI 假设：平台、调用约定、入口参数、返回值位置。
4. 基本块：入口、分支、循环、退出、冷路径。
5. 寄存器角色：每个阶段主要寄存器保存什么。
6. 内存访问：宽度、地址公式、可能对象、对齐和边界前提。
7. 控制依赖：哪些检查保护哪些危险操作。
8. C-like 伪代码：只写证据支持的最小语义。
9. 性能假设：指令类型、依赖链、分支、load/store 和可能瓶颈。
10. 限制：哪些结论不能从当前反汇编证明。

模板的目标不是增加形式感，而是避免遗漏关键问题。很多错误汇编解读只写了伪代码，没有写 ABI；只写了指令含义，没有写控制流；只写了性能猜测，没有写输入和边界。模板能强迫读者把这些层次分开。

常见视觉陷阱

反汇编有一些视觉陷阱。第一，地址相邻不代表路径相邻。编译器可能把冷路径放在热路径之后，也可能把多个函数的冷块集中到其他 section。第二，短指令不一定便宜，长指令不一定慢。性能要看依赖、吞吐、内存和微架构。第三，`lea` 看起来像地址指令，但常用于整数乘加。第四，`xor eax, eax` 不是逻辑意图上的“异或”，常是清零。

第五，栈槽不一定对应源码局部变量。优化后栈槽可能用于溢出寄存器、保存调用参数、对齐、异常处理或临时存储。第六，`nop` 不一定是无意义垃圾，可能用于对齐或热补丁。第七，同一个源码变量可能在不同路径上位于不同寄存器，甚至不存在。第八，反汇编中的符号名可能来自最近的符号，不一定是精确标签。

这些陷阱的共同原因是：反汇编是机器码视角，不是源码视角。读者要尊重它的事实，同时控制向源码层推断的欲望。

符号、重定位和地址注释

汇编阅读不能只看指令助记符，还要看符号和重定位。目标文件中的许多地址还没有最终确定，反汇编旁边的重定位记录说明链接器以后会填什么。若一条指令访问 `rip+0`，旁边有指向某个全局符号的重定位，不能按字面把偏移零解释成真实地址。要结合 `objdump-dr` 的重定位注释。

函数调用也是如此。目标文件里对外部函数的 `call` 可能显示相对偏移占位，重定位记录指向符号；最终可执行文件中可能变成 PLT 调用，也可能在静态链接或 LTO 后变成直接地址或被内联。阅读目标文件和阅读最终可执行文件，结论层级不同。报告中要说明你看的是哪一层。

RIP-relative 寻址是 x86-64 常见模式。它让代码可以位置无关地访问附近数据或 GOT 表项。看到 `[rip+disp]`，要计算的是下一条指令地址加 displacement，而不是当前指令起始地址加 displacement。初学者常在这里算错地址。若有符号注释，以符号和重定位为准；若没有，就用地址手工验证。

调试信息和反汇编的配合

带 `-g` 的二进制可以把指令映射回源码行，但这种映射不是完美逆变换。优化后，一条源码行可能对应多段指令，一条指令可能服务多个源码表达式，变量可能被优化掉或位于多个位置。GDB 显示的源码行是线索，不是完整解释。

阅读时可以先用调试信息定位函数和源码区域，再切换到指令级视角。若 GDB 显示变量为 `optimized out`，不要立刻认为变量不存在；它可能被常量传播、拆成寄存器片段，或者根本不需要存储。若源码单步跳来跳去，可能是优化重排和内联造成的。此时应回到反汇编和寄存器状态。

调试信息还可能和二进制不匹配。若符号文件来自另一构建，源码行会误导。正式报告应记录 build id 或二进制校验和，确保调试信息和机器码对应。反汇编阅读的第一原则仍然是：先确认对象身份。

从汇编到问题单

真实工程中，汇编阅读常常服务一个问题单：为什么崩溃，为什么慢，为什么调用错库，为什么 ABI 破坏，为什么某个函数消失。报告应围绕问题，不必把每条指令都翻译。先列与问题相关的块，再说明它们如何支持或推翻假设。

例如性能问题关注热循环、load/store、分支和调用；崩溃问题关注当前指令、寄存器、内存地址和保护条件；ABI 问题关注入口寄存器、栈对齐和保存寄存器；链接问题关注符号和重定位。目的不同，阅读重点不同。会读汇编，不是把所有指令翻成中文，而是从机器事实中提取和问题相关的证据。

汇编注释应该解释层次，不要复述指令

给汇编写注释时，低质量注释只是把指令翻译成中文，例如“把 `eax` 赋值为 0”。这种注释帮助很小。高质量注释应解释层次：这个寄存器此刻代表哪个逻辑值，这个分支保护什么条件，这个地址公式对应哪个字段，这个调用跨过哪个 ABI 边界，这个块为什么是冷路径。

例如：

```
xor eax, eax      ; return value = 0 for n <= 0 path
test esi, esi     ; check n before any load from p
jle .lreturn      ; zero/negative length does not access memory
```

这里注释说明了控制流保护关系，而不只是复述 `test` 和 `jle`。读者能从注释中知道为什么这几条指令重要。汇编报告中的注释也应如此：解释证据如何连接到语义和边界。

从手写汇编反推编译器生成汇编

学习汇编有两条路线：读编译器生成的汇编，写少量手写汇编。手写能帮助理解语法、标签、section、符号和 ABI；读编译器汇编能帮助理解优化、寄存器分配、控制流和真实代码形态。两者要互相校准。只写手写汇编，容易形成不符合编译器输出的风格；只读编译器汇编，容易缺少对目标文件结构的掌控。

建议读者手写最小函数后，马上写等价 C++，比较两者目标文件。看符号、section、`.note.GNU-stack`、`.type`、`.size`、栈帧和返回值是否一致。这个练习能把“汇编文本”与“可链接目标文件”联系起来，为第 13 和第 14 章做准备。

阅读优化汇编的耐心

优化汇编常常比 `-O0` 难读，因为源码变量消失、循环形态改变、分支反转、常量折叠、内联和公共子表达式消除都会发生。读者不要把这种变化理解成“编译器乱写”。优化器是在语言合同允许范围内重写程序，使机器执行更合适。

阅读时可以先用 `-O0` 建立源码到机器的粗映射，再用 `-O2` 或 `-O3` 看真实发布形态。两者都要看，但回答不同问题。`-O0` 帮助学习语义，`-O3` 帮助分析性能。若报告要支持性能结论，最终必须回到优化构建；若报告要解释语言结构，可以用低优化辅助理解。

综合复盘：从一段汇编写出三层结论

读者可以拿任意一个短函数练习三层结论。第一层是机器事实：指令、寄存器、内存宽度、地址公式和跳转。第二层是源码解释：在某些类型假设下，它可能对应什么 C-like 语义。第三层是工程判断：它是否满足 ABI，是否有边界风险，是否有性能假设。三层结论必须分开写。

例如一条 `moveax, dwordptr[rdi+rsi*4]`，机器事实是从地址读取 4 字节到 `eax`；源码解释可能是读取 `int` 数组第 `i` 个元素；工程判断还要问 `rdi` 是否非空、`rsi` 是否在范围内、读取是否受检查保护、访问是否顺序。只写“读取数组元素”省略了很多前提。

这个练习能训练克制。汇编阅读最重要的品质不是想象力，而是知道哪些结论有证据，哪些只是合理猜测。

汇编阅读的长期训练方法

每周选择一个小函数，保存源码、构建命令、反汇编和一页分析。函数可以很简单：加法、数组求和、结构体字段更新、短路判断、switch、函数调用。简单函数重复做，能让模式稳定进入直觉。等

模式熟悉后，再读复杂函数就不会被局部指令淹没。

长期训练还要保留失败样例。哪些指令曾被你误解，哪些条件码曾读反，哪些地址公式曾算错，哪些函数边界曾判断错，都应该写进错误笔记。汇编能力不是靠一次性背完指令表形成，而是靠不断校正误读形成。

汇编报告的审阅方式

审阅一份汇编报告时，不要先挑指令翻译是否漂亮，而要先看证据层次。报告是否说明二进制身份，是否说明平台 ABI，是否区分机器事实和源码推断，是否写出无法证明的部分，是否把控制流和数据流连起来。如果这些层次缺失，即使单条指令翻译正确，报告仍然不可靠。

其次看边界。报告是否说明指针和下标的合法性来自哪里，是否说明结构体 offset 的依据，是否说明优化级别和内联对函数边界的影响，是否说明性能假设需要测量。汇编报告最常见的问题不是少翻译一条指令，而是把推断写成事实。审阅时要鼓励作者把不确定性写出来。

最后看可复现性。别人能否用同一命令生成同一段反汇编，能否找到同一函数，能否验证同一地址公式。如果不能，报告就只是一次口头解释。汇编学习从个人能力变成工程能力，关键就在可复现性。

实验：建立第一份汇编阅读报告

创建 labs/ch10_assembly_syntax/basic_reading.cpp:

```
#include <cstdint>
#include <cstdint>

extern "C" int add2(int x, int y)
{
    return x + y;
}

extern "C" int mix(int x)
{
    return x * 5 + 17;
}

extern "C" std::uint8_t load_u8(std::uint8_t const* p, std::size_t i)
{
    return p[i];
}

extern "C" int load_i32(int const* p, int i)
{
    return p[i + 2];
}
```

生成汇编和反汇编：

```
clang++ -std=c++20 -O0 -S -masm=intel basic_reading.cpp -o basic_reading_00.s
clang++ -std=c++20 -O3 -S -masm=intel basic_reading.cpp -o basic_reading_03.s
clang++ -std=c++20 -O3 -c basic_reading.cpp -o basic_reading.o
objdump -d -Mintel basic_reading.o > basic_reading.objdump.txt
```

交付物：

1. 对每个函数列出 ABI 参数寄存器和返回值寄存器。
2. 从 basic_reading_03.s 中摘出每个函数的核心指令。
3. 对每条核心指令写“读什么、写什么、是否访问内存、是否更新 flags”。
4. 对 load_u8 解释为什么可能出现 movzx。
5. 对 mix 解释编译器是否使用 lea。
6. 对 load_i32 写出地址公式，并说明 +8 从哪里来。
7. 从 objdump 中记录机器码字节，说明汇编文本和机器码字节的关系。

实验：Intel 和 AT&T 语法互译

用同一个源码生成两种语法：

```
clang++ -std=c++20 -O3 -S -masm=intel basic_reading.cpp -o intel.s
clang++ -std=c++20 -O3 -S basic_reading.cpp -o att.s
```

交付物：

1. 从两个文件中各选 8 条对应指令。
2. 写成三列表：Intel、AT&T、语义。
3. 标出操作数顺序差异。
4. 标出内存寻址写法差异。
5. 标出宽度写法差异，例如 `dwordptr` 和 `movl`。

报告中必须包含下面这种格式：

Intel	AT&T	语义
<code>moveax,edi</code>	<code>movl%edi,%eax</code>	<code>eax=edi</code>
<code>addeax,esi</code>	<code>addl%esi,%eax</code>	<code>eax+=esi</code>

实验：-O0 和 -O3 对比

创建 `labs/ch10_assembly_syntax/o0_o3_compare.cpp`：

```
extern "C" int formula(int x)
{
    int a = x + 1;
    int b = a * 3;
    int c = b - x;
    return c + 7;
}
```

生成：

```
clang++ -std=c++20 -O0 -S -masm=intel o0_o3_compare.cpp -o formula_00.s
clang++ -std=c++20 -O3 -S -masm=intel o0_o3_compare.cpp -o formula_03.s
```

交付物：

1. 注释 -O0 中的栈帧建立、局部变量栈槽和返回路径。
2. 注释 -O3 中的代数化简结果。
3. 用数学式证明 `formula(x)` 可以化简成什么。
4. 解释为什么不能用 -O0 汇编评估性能。
5. 说明 -O0 对调试有什么价值。

实验：符号、section 和重定位证据

目标：把 `.s`、`.o`、符号表和重定位表连起来看，理解目标文件里哪些地址已经确定，哪些还要等链接器修补。

创建 `labs/ch10_assembly_syntax/relocation_a.cpp`：

```
#include <stdio>

extern "C" int external_add(int, int);

int global_counter = 7;

extern "C" int call_external(int x)
{
```

```

    return external_add(x, global_counter);
}

extern "C" void print_value(int x)
{
    std::printf("value=%d\n", x);
}

```

创建 labs/ch10_assembly_syntax/relocation_b.cpp:

```

extern "C" int external_add(int a, int b)
{
    return a + b;
}

```

先只编译第一个目标文件:

```

clang++ -std=c++20 -O2 -fPIC -S -masm=intel relocation_a.cpp -o relocation_a.s
clang++ -std=c++20 -O2 -fPIC -c relocation_a.cpp -o relocation_a.o
objdump -drwC -Mintel relocation_a.o > relocation_a.objdump.txt
readelf -S -s -r relocation_a.o > relocation_a.readelf.txt
nm -C relocation_a.o > relocation_a.nm.txt

```

再把两个目标文件链接成可执行文件:

```

clang++ -std=c++20 -O2 relocation_a.o relocation_b.cpp -o relocation_app
objdump -drwC -Mintel relocation_app > relocation_app.objdump.txt
readelf -S -s -r relocation_app > relocation_app.readelf.txt
nm -C relocation_app > relocation_app.nm.txt

```

交付物:

1. 在 `relocation_a.s` 中标出 `.text`、`.data`、`.rodata` 或相关 section 切换。
2. 用 `nm` 判断 `external_add`、`printf`、`global_counter` 哪些是已定义符号，哪些是未定义符号。
3. 在 `objdump-drwC` 输出中找出访问 `global_counter` 的指令及其重定位旁注。
4. 找出调用 `external_add` 或 `printf` 的 `call`，解释目标文件阶段为什么可能显示占位位移。
5. 用 `readelf-r` 把重定位 offset、重定位类型、符号名和反汇编中的指令位置对应起来。
6. 对比目标文件和链接后可执行文件，说明哪些地址或调用目标发生了变化。
7. 说明 `-fPIC` 对全局变量访问形态可能产生什么影响。

报告要求：不能只贴输出。每个结论都要写清“证据来自哪个文件、哪个命令、哪一行输出”。例如“`external_add` 在 `relocation_a.o` 中是 undefined symbol，证据是 `nm-Crelocation_a.o` 显示 `Uexternal_add`；对应 `call` 的修补证据来自 `objdump-drwC` 中的 `relocation` 行”。

实验：手写最小汇编目标文件

目标：亲手写一个最小 `.S` 文件，观察标签、全局符号、函数类型、大小和不可执行栈标记如何进入目标文件。

创建 labs/ch10_assembly_syntax/handwritten.S:

```

.intel_syntax noprefix
.text
.globl asm_add2
.type asm_add2,@function
asm_add2:
    mov eax, edi
    add eax, esi
    ret

```



```
.size asm_add2, .-asm_add2

.section .note.GNU-stack,"",@progbits
```

创建 labs/ch10_assembly_syntax/use_handwritten.cpp:

```
#include <stdio>

extern "C" int asm_add2(int, int);

int main()
{
    std::printf("%d\n", asm_add2(20, 22));
}
```

构建并观察:

```
clang++ -c handwritten.S -o handwritten.o
clang++ -std=c++20 -c use_handwritten.cpp -o use_handwritten.o
clang++ handwritten.o use_handwritten.o -o use_handwritten
./use_handwritten
objdump -drwC -Mintel handwritten.o > handwritten.objdump.txt
readelf -S -s handwritten.o > handwritten.readelf.txt
nm -C handwritten.o > handwritten.nm.txt
```

交付物:

1. 解释 `.intel_syntaxnoprefix` 改变了什么。
2. 解释 `.globlasm_add2` 为什么对跨文件链接必要。
3. 解释 `.type` 和 `.size` 对工具输出有什么帮助。
4. 在 `objdump` 中记录 `mov`、`add`、`ret` 的机器码字节。
5. 在 `readelf-s` 中找到 `asm_add2` 的符号类型、section 和大小。
6. 说明 `.note.GNU-stack` 的目的, 以及为什么它不是 CPU 执行路径。
7. 删除 `.globlasm_add2` 后重新构建, 记录链接错误, 再恢复该行。

这个实验故意只写一个加法函数。目标不是展示复杂汇编, 而是让你看到: 可链接的汇编函数不仅有三条机器指令, 还需要正确的符号可见性和目标文件元信息。

本章作业

习题与作业

作业 1: 逐条注释 20 条汇编

从本章实验生成的 -03 汇编中选择至少 20 条真实指令。每条指令必须写:

- 指令文本。
- 操作数类型: 立即数、寄存器、内存。
- 读取的寄存器或内存。
- 写入的寄存器或内存。
- 访问宽度。
- 是否更新 flags。
- 你认为它对应的 C++ 语义。

作业 2: 地址公式恢复

对下面汇编片段恢复可能的地址公式和 C-like 伪代码:

```
movsxd rax, esi
mov    ecx, dword ptr [rdi + rax*4 + 12]
add    ecx, edx
mov    dword ptr [rdi + rax*4 + 12], ecx
mov    eax, ecx
```

```
ret
```

要求至少给出两种可能来源：

1. `int*` 数组访问版本。
2. 结构体数组字段访问版本。

每种版本都必须解释 `+12` 的来源。

作业 3：语法互译

把下面 Intel 语法翻译成 AT&T 语法，并写出语义：

```
mov rax, rdi
add eax, esi
movzx eax, byte ptr [rdi + rsi]
mov qword ptr [rsp + 8], rax
lea rax, [rdi + rsi*8 + 16]
```

再把下面 AT&T 语法翻译成 Intel 语法：

```
movl %edi, %eax
addl %esi, %eax
movzbl (%rdi,%rsi), %eax
movq %rax, 8(%rsp)
leaq 16(%rdi,%rsi,8), %rax
```

作业 4：机器码和指令长度

用 `objdump` 记录至少 10 条指令的机器码字节。回答：

1. 哪些指令是 1 字节？
2. 哪些指令长度超过 4 字节？
3. 立即数和 displacement 如何让指令变长？
4. 为什么 x86 前端需要处理变长指令？

作业 5：目标文件证据链

选择一个包含外部函数调用、字符串字面量、全局变量访问的小程序，分别生成 `.s`、`.o` 和可执行文件。提交一份目标文件证据链报告，必须包含：

- `objdump-drwC-Mintel` 输出中至少 5 条带机器码字节的指令。
- `readelf-S` 中相关 section 的名称、类型和权限。
- `readelf-r` 中至少 2 条重定位记录。
- `nm-C` 中至少 2 个已定义符号和 2 个未定义符号。
- 对一个 `call` 占位位移的解释。
- 对一个 `[rip+...]` 全局或常量访问的解释。
- 目标文件阶段和链接后阶段地址语境的区别。

报告必须明确标注“确定事实、合理推论、待验证假设”。例如“重定位类型是什么”是工具输出事实；“它可能通过 PLT 调用动态库函数”是结合构建方式的推论；“这部分开销影响性能”则需要运行时测量，不能仅凭反汇编断言。

作业 6：小型汇编阅读报告

选择一个你自己写的 10 到 20 行 C++ 函数，要求包含：

- 至少一个数组访问。
- 至少一个结构体字段访问。
- 至少一个整数表达式。
- 至少一个条件判断或比较。

生成 `-O0` 和 `-O3` 汇编。提交不少于 2000 字报告，必须包含：

- 源码。
- 编译命令。
- 函数签名和 ABI 参数寄存器。
- -O0 和 -O3 汇编对比。
- 至少 15 条指令逐条注释。
- 所有内存访问的地址公式。
- 至少 5 条机器码字节记录。
- 你遇到的一个误判，以及如何纠正。

验收标准

完成本章后，你应该能做到：

- 熟练区分 Intel 和 AT&T 语法。
- 识别立即数、寄存器和内存操作数。
- 解释通用寄存器别名和 32 位写入清零高位。
- 从内存操作数写出有效地址公式。
- 解释 `byteptr`、`dwordptr`、`qwordptr`、`xmmwordptr` 的访问宽度。
- 解释 `lea` 为什么不访问内存。
- 区分 `.text`、`.rodata`、`.data`、`.bss` 的基本用途。
- 说明 `.globl`、`.type`、`.size`、局部标签和全局符号的作用。
- 用 `objdump-drwC`、`readelf-r`、`nm-C` 交叉验证外部调用和全局访问。
- 解释目标文件阶段的重定位占位为什么不能直接当作最终地址。
- 对简单函数生成 -O0 和 -O3 汇编，并逐条注释核心指令。
- 从反汇编中记录机器码字节，知道汇编文本最终对应指令编码。
- 不再把 -O0 汇编当作性能路径。

如果这些能力稳定，本册后面的整数指令、控制流、ABI 和 C++ 汇编互操作才有可靠基础；第二册的自动向量化和 AVX microkernel 也才真正读得进去。

整数指令、位运算和地址计算

问题与目标

上一章建立了阅读 x86-64 汇编的基本语法：指令、寄存器、立即数、内存操作数、操作数宽度和 `lea`。本章进入最常见、也最容易被低估的一组机器动作：整数数据移动、加减、乘法、除法、符号扩展、零扩展、移位、位运算、比较、溢出检测和地址计算。

这些指令本身不复杂，但它们构成了后续章节的底层材料。数组下标、结构体字段、循环计数、边界判断、指针递增、类型转换和掩码处理，最终都会落到这些整数操作上。若这一层只靠助记符记忆，后面阅读控制流、ABI、C++ 互操作和性能报告时就会不断误判。

- 数组下标和结构体字段访问需要整数地址计算。
- 循环计数、边界判断、指针递增都依赖加减和比较。
- 类型转换会生成符号扩展或零扩展。
- 对齐、掩码、量化、哈希、SIMD lane 处理会大量使用位运算。
- 性能优化里经常要判断一条指令是否更新 flags、是否破坏依赖、是否可被 `lea` 替代。

本章不要求背诵完整 Intel 手册，而是训练一套可复查的读法。看到一个整数汇编片段时，你能逐项回答：

1. 每条指令读哪些寄存器、内存和 flags?
2. 每条指令写哪些寄存器、内存和 flags?
3. 操作数宽度是多少?
4. 这是有符号语义、无符号语义，还是纯位模式语义?
5. 如果它来自 C++，语法规则是否允许这种变换?
6. 它对性能有什么直接影响?

核心思想

整数指令直接处理的是固定位宽的位模式。有符号和无符号的差异，通常不在于寄存器中保存了两类对象，而在于指令如何解释这些位、编译器如何根据 C++ 类型选择指令，以及语法规则允许优化器作出哪些假设。

学习整数指令时最容易混淆的三件事

整数指令表面上比控制流、ABI 和浮点更直观，但它反而容易形成错误直觉。原因在于“整数”同时存在于三种语境：C++ 类型系统中的整数类型，ISA 中的整数寄存器和整数指令，数学中的无限精度整数。三者有关联，却不能直接等同。

第一，C++ 整数类型带有语法规则。一个表达式是 `int` 还是 `unsigned int`，会影响整型提升、比较规则、溢出规则、移位规则和编译器能做的优化假设。尤其是有符号整数溢出，在 C++ 中不是普通回绕，而是未定义行为。编译器可以利用这个前提消除分支、改写循环或做代数化简。

第二，ISA 指令处理的是固定宽度位模式。寄存器里没有“这个值是 signed”的标签。许多指令只关心低 N 位结果，例如普通加法、减法、按位与、按位或、异或。另一些指令或后续条件码会以 signed 或 unsigned 方式解释 flags。扩展指令、除法指令、右移指令和条件跳转尤其会暴露这种解释差异。

第三，数学整数是无限精度概念，而机器整数有宽度。数学上 $2^{147483647}+1$ 是 `2147483648`；32 位补码低位结果是 `0x80000000`；C++ `int` 加法如果溢出则没有定义良好的程序语义；`std::uint32_t` 加法则按模 2^{32} 回绕。读汇编时必须同时知道这三种视角，否则会把机器现象误认为语言保证。

心智模型

整数分析的顺序是：先看位宽，再看指令如何处理位模式，再看 C++ 类型给编译器什么语义约束，最后才谈数学含义和性能。不要反过来从数学直觉直接解释机器指令。

补码为什么成为本章默认模型

现代 x86-64 整数使用二进制补码表示有符号整数。补码的核心好处是同一套低位加法硬件可以同时服务有符号和无符号加法。比如 8 位里，`0xff` 如果按无符号解释是 255，按有符号解释是 -1；但它和 `0x01` 相加的低 8 位结果都是 `0x00`。区别不在加法器本身，而在你如何解释结果以及 flags。

补码还能让取负具有统一形式：按位取反再加一。这个性质解释了许多底层技巧，例如用 `neg` 产生借位信息，用 `sbb` 生成全零或全一掩码，或者用位运算处理符号扩展。初学阶段不需要记住所有技巧，但必须理解它们来自补码表示和 flags 语义的共同作用，而不是某种脱离规则的特殊写法。

补码模型也解释了为什么 signed 和 unsigned 的低位乘法结果常常相同。两个 N 位数相乘后取低 N 位，许多情况下同一条机器指令就足够；只有当你需要高半部分、完整双倍宽度结果、除法、比较或溢出判断时，signed 和 unsigned 的区别才会变得明显。

不过，补码机器不等于 C++ signed 溢出有定义。现代 C++ 标准已经越来越明确地贴近补码现实，但优化器仍然依赖语言层规则。你在 x86 上观察到某条 `add` 产生回绕，只能说明机器执行了低位加法，不能证明原始 C++ signed 表达式在所有输入上定义良好。高质量教材必须把这条边界反复讲清楚。

整数寄存器里的值：位模式先于解释

先看一个 32 位寄存器：

`eax = 0xffffffff`

这个位模式可以解释成：

解释方式	值	说明
<code>std::uint32_t</code>	4294967295	无符号解释
<code>std::int32_t</code>	-1	二进制补码解释
位掩码	所有 32 位都是 1	mask 解释

寄存器本身没有贴着“这是 signed int”的标签。不同指令会以不同方式解释这些位：

- `add` 对低 N 位加法而言，有符号和无符号使用同一个加法器；区别主要体现在 flags 如何解释。
- `cmp` 后接有符号条件跳转和无符号条件跳转时，读取的 flags 组合不同。
- `movsx` 和 `movzx` 明确区分符号扩展和零扩展。
- `sar` 是算术右移，保留符号位；`shr` 是逻辑右移，高位补 0。

常见误区

不要说“这个寄存器是 signed”。更准确的说法是：“这段代码把这个寄存器的位模式按 signed 语义使用”。寄存器存的是位，解释来自指令、类型和上下文。

这个原则会贯穿本章。一个寄存器里的低 32 位全 1，可以作为 -1 参与有符号比较，也可以作为四十多亿这个无符号数参与无符号比较，还可以作为所有位都为 1 的掩码。编译器选择哪种指令，不是因为寄存器本身变了，而是因为源码类型、后续用途和语言规则要求不同解释。

因此，读整数汇编时不要只问“值是多少”，还要问“位宽是多少”“后续按什么语义使用”。`eax` 的 `0xffffffff` 作为 32 位 `int` 是 -1；如果写入 `eax` 后再读 `rax`，高 32 位会被清零，64 位值变成 `0x00000000ffffffff`，按 64 位 signed 解释就是正数 4294967295。这个细节在参数扩展、地址计算和比较中非常重要。

C++ 类型如何影响汇编选择

许多初学者以为编译器只是把 `+` 翻译成 `add`，把 `*` 翻译成 `imul`，把 `<` 翻译成某个固定跳转。实际情况要严格得多：同一个源码符号在不同类型下可能生成不同指令；同一条机器指令也可能服务不同源码语义。

以小整数加载为例。读取 `std::uint8_t` 后转换成 `int`，高位必须补 0；读取 `std::int8_t` 后转换成 `int`，高位必须复制符号位。因此一个看似相同的“读一个字节”操作，可能分别生成 `movzx` 和 `movsx`。如果你只看内存访问宽度，会漏掉类型语义。

比较也是如此。`intx<inty` 和 `unsignedx<unsignedy` 都可能先用 `cmp` 设置 flags，但后续读取 flags 的条件码不同。有符号比较关心符号位和溢出位组合，无符号比较关心进位或借位。控制流章节会系统展开条件码，本章先要求你知道：`cmp` 本身不是 signed 或 unsigned 的全部答案，后续如何解释 flags 才决定比较语义。

算术优化也受类型影响。无符号加法按模回绕定义良好，编译器不能假设它不会溢出；有符号溢出未定义，编译器通常可以假设它不会发生，从而进行某些代数化简。比如某些边界检查、循环终止条件和表达式比较，在 signed 与 unsigned 下可优化空间不同。读汇编时如果发现 signed 和 unsigned 版本不一样，不要只从指令表找原因，要回到语言规则。

核心理想

C++ 类型不会作为标签保存在寄存器里，但它会影响编译器选择扩展方式、比较条件、溢出假设和优化形状。汇编里看不见类型，不代表类型不重要。

整型提升：小整数通常先变成 `int`

C++ 里很多小整数运算并不是在 8 位或 16 位上直接完成。表达式求值前会发生整型提升：`bool`、`char`、`signedchar`、`unsignedchar`、`short`、`unsignedshort` 这类小类型，在通常算术表达式中会先提升到 `int`，如果 `int` 不能表示该类型所有值，才提升到 `unsignedint`。在常见 x86-64/Linux 环境里，`std::uint8_t` 和 `std::int8_t` 参与普通加法时，后续算术通常发生在 32 位寄存器上。

这解释了一个常见现象：源码看起来只处理字节，汇编却出现 `eax`、`ecx` 这样的 32 位寄存器。编译器不是把字节“扩大成了变量”，而是在执行 C++ 的表达式规则。读取无符号字节后用 `movzx` 得到 0 到 255 的 `int` 值，读取有符号字节后用 `movsx` 得到 -128 到 127 的 `int` 值，然后再做 32 位加法、比较或返回。

```
#include <stdint>

extern "C" int add_u8(std::uint8_t a, std::uint8_t b)
{
    return a + b;
}
```

这段代码的返回类型是 `int`，表达式 `a+b` 也按提升后的整数语义求值。汇编中可能先零扩展两个参数，再做 32 位加法。不要因为源码参数是字节，就期待最终加法一定是 8 位 `add`。如果程序员真正需要 8 位回绕，应当在源码里把结果转换回 `std::uint8_t`，并清楚说明这是窄化后的低 8 位结果。

整型提升还会影响布尔和字符处理。普通 `char` 是否有符号由实现决定，`charc=0xff` 在不同实现上的提升结果可能不同。底层教材和实验代码应避免依赖普通 `char` 的符号性。你要研究字节数值，就使用 `std::uint8_t`；要研究有符号小整数，就使用 `std::int8_t`；要研究文本字符，就把字符编码和字节表示分开说明。

usual arithmetic conversions：混合类型先统一语义

两个操作数类型不同时，C++ 会通过 usual arithmetic conversions 把它们转换到一个共同类型。这个规则比“谁更大就转成谁”复杂，但在底层阅读里最容易出错的是 signed 和 unsigned 混合。若一个 `int` 和一个 `unsignedint` 比较，`int` 往往会转换成 `unsignedint`。因此 `-1<1u` 的结果不是数学直觉里的真，而是假，因为 `-1` 转换成无符号后成为很大的值。

```
extern "C" bool mixed_less(int x, unsigned y)
{
    return x < y;
}
```

```
}

```

这段源码在语义上不是“有符号 x 小于无符号 y ”。比较前 x 会按规则转换到无符号类型，后续比较按 `unsigned` 语义进行。汇编里常见表现是同样的 `cmp` 后面接无符号条件码，例如 `setb` 或 `jb`，而不是有符号的 `setl` 或 `jl`。如果报告只写“比较 x 和 y ”，就漏掉了最关键的语义转换。

`std::size_t` 是另一个常见陷阱。容器大小、数组长度、内存字节数通常用无符号的 `std::size_t` 表示。如果循环变量是 `int`，条件里又和 `std::size_t` 混合，比较可能转成无符号语义。负数下标在比较时可能变成极大的无符号数，控制流就会和直觉相反。高质量底层代码通常会让索引类型、长度类型和边界检查保持一致，或者在转换处写出明确的前提。

这条规则也解释了为什么从汇编反推源码类型并不唯一。你看到无符号条件跳转，可能是源码本来使用 `unsigned`，也可能是 `signed` 与 `unsigned` 混合后被转换，也可能是编译器把某个范围已知非负的 `signed` 值当作无符号比较处理。汇编提供证据，但最终结论需要结合源码类型、编译选项和上下文推导。

常见误区

混合 `signed` 和 `unsigned` 不是小风格问题，它会改变比较语义、循环边界和越界检查。读汇编时看到无符号条件码，要回到源码确认是否发生了 `usual arithmetic conversions`。

本章的读法：每条整数指令都问五个问题

本章后面会看到很多指令。不要把它们背成“助记符表”。每遇到一条整数指令，都按五个问题做笔记。

第一，操作数宽度是多少。是 8 位、16 位、32 位还是 64 位？是通用寄存器、内存，还是立即数？写 32 位寄存器是否会清零高 32 位？部分寄存器写入是否留下旧高位？

第二，它读取和写入哪些状态。状态不只是通用寄存器，还包括内存和 `flags`。`cmp` 不写通用寄存器，但写 `flags`；`test` 不保存按位与结果，但写 `flags`；内存目的操作数的 `add` 同时读内存、写内存、写 `flags`。

第三，它如何解释位模式。是纯位运算、模加法、符号扩展、零扩展、算术右移、逻辑右移，还是乘法低位结果？如果后续有条件跳转或条件设置，要看 `flags` 按 `signed` 还是 `unsigned` 语义被消费。

第四，它在 C++ 里需要什么前提。移位计数是否合法？`signed` 加法是否可能溢出？指针下标是否越界？小整数转换是否需要符号扩展？这些问题决定编译器输出是否只是机器习惯，还是语法规则允许的优化。

第五，它对性能的直接影响是什么。它是寄存器 ALU，还是 `load/store`，还是 `read-modify-write`？它是否产生依赖链？是否写 `flags` 并影响后续指令？是否可以被 `leaq` 替代？这里先建立定性判断，第二册再做定量微架构分析。

心智模型

整数指令的学习目标不是“看到助记符就说中文名”，而是能回答宽度、读写、解释、语言前提和性能影响。

`mov`：复制位模式，不是高级赋值

`mov` 是最常见的指令之一，但它的名字容易误导。它不是“移动后源消失”，而是复制位模式：

```
mov eax, edi

```

语义：

```
eax = edi
rax high 32 bits = 0

```

因为写 32 位寄存器会清零高 32 位。

寄存器到寄存器

C++：

```
extern "C" int id(int x)
{
    return x;
}
```

可能汇编：

```
mov eax, edi
ret
```

`edi` 是参数，`eax` 是返回值。这里没有内存访问。

内存到寄存器：load

C++：

```
extern "C" int load(int const* p)
{
    return *p;
}
```

可能汇编：

```
mov eax, dword ptr [rdi]
ret
```

解释：

```
address = rdi
load 4 bytes from address
write eax
```

寄存器到内存：store

C++：

```
extern "C" void store(int* p, int x)
{
    *p = x;
}
```

可能汇编：

```
mov dword ptr [rdi], esi
ret
```

解释：

```
address = rdi
store low 32 bits of esi to address
```

立即数到寄存器或内存

```
mov eax, 123
mov dword ptr [rdi], 123
```

第一条把立即数写入寄存器；第二条把立即数写入内存。第二条是 `store`，会修改内存。

为什么 `mov` 不是“赋值语句”

把 `mov` 理解成 C++ 赋值语句，会在两个地方出错。第一，C++ 赋值有类型、对象生命周期、重载运算符、`volatile`、原子对象和别名规则；`mov` 只是复制某个宽度的位模式。第二，C++ 赋值的源和目标是语言对象；`mov` 的源和目标是寄存器、立即数或内存地址。它们可能对应某个对象，也可能只是临时值或保存槽。

例如 `mov eax, edi` 可以来自函数返回参数，也可以来自清理高位，也可以来自把某个中间结

果换到返回寄存器。你不能只根据 `mov` 断言源码有一条赋值语句。反过来，源码的一条赋值也可能变成多条 `load/store`，或者在优化后完全消失。

`load` 和 `store` 也要分开看。`load` 把内存中的字节复制到寄存器；`store` 把寄存器或立即数的字节写回内存。`load` 可能因为 `cache miss`、缺页或依赖而等待；`store` 可能进入 `store buffer`，稍后才真正对其他核心可见。第 7 章不展开完整内存系统，但从现在开始，报告里要明确写出一条 `mov` 是否访问内存，访问方向是什么。

常见误区

看到 `mov` 不要自动写“赋值”。应写清楚：复制多少位，从哪里读，写到哪里，是否访问内存，是否因为写 32 位寄存器清零高位。

清零寄存器：`xor reg, reg` 和 `mov reg, 0`

你经常看到：

```
xor eax, eax
```

它把 `eax` 清零。因为任何位和自己异或都是 0：

```
x ^ x = 0
```

也可以写：

```
mov eax, 0
```

但编译器常用 `xor eax, eax`，原因包括编码短、CPU 识别 `zero idiom`、可打破旧值依赖等。注意 `xor` 会更新 `flags`，而 `mov` 通常不更新 `flags`。上下文不同，编译器选择也可能不同。

深入理解

现代 x86 核心通常能识别 `xor reg, reg`、`sub reg, reg` 这类 `zero idiom`，把它们当成不依赖旧寄存器值的清零操作。依赖打断对乱序执行和寄存器重命名很重要，后面讲微架构时会展开。

清零指令还有一个容易忽略的语义细节：写 `eax` 会把 `rax` 高 32 位清零。因此 `xor eax, eax` 不只是把低 32 位清零，它让整个 `rax` 都成为 0。类似地，`mov eax, 0` 也会让 `rax` 成为 0。若你看到 `xor al, al`，情况就不同了，它只清最低 8 位，高位仍然保留旧值。

为什么编译器偏爱清零 `idiom`，还和寄存器重命名有关。普通指令通常依赖旧目的寄存器值，例如 `add eax, esi` 必须读取旧 `eax`。但 `xor eax, eax` 的数学结果不依赖旧值，硬件可以把它识别为“生成一个新的零值”。这样后续读取 `eax` 的指令不必等待旧 `eax` 的生产者。这个机制本质上是性能话题，但在读汇编时先知道它，有助于理解为什么编译器不用看起来更直观的 `mov`。

不过，不能机械地把所有清零都改成 `xor`。如果后续还需要保留之前的 `flags`，`xor` 会覆盖 `flags`，而 `mov` 通常不会。手写汇编时，这种差异会导致条件判断错误。编译器会根据上下文选择；读者要能解释选择背后的状态影响。

加法和减法：结果、`flags` 和语言规则

`add` 和 `sub`

C++：

```
extern "C" int add_i32(int x, int y)
{
    return x + y;
}

extern "C" int sub_i32(int x, int y)
{
    return x - y;
}
```

可能汇编：

```
add_i32:
    lea eax, [rdi + rsi]
    ret

sub_i32:
    mov eax, edi
    sub eax, esi
    ret
```

add 和 sub 都会更新 flags。编译器有时用 lea 做加法，因为 lea 不更新 flags：

```
lea eax, [rdi + rsi]    ; eax = x + y, flags unchanged
add eax, esi            ; eax = eax + y, flags updated
```

如果后续不需要 flags，lea 可能更合适。如果后续需要根据结果跳转，add 或 sub 设置 flags 就 有价值。

add 和 sub 的目的操作数既是输入也是输出。读 add eax, esi 时，要写成“读取 eax 和 esi， 把结果写回 eax”，不能写成“把 esi 加到某个新变量”。这种原地更新形式会形成依赖链：下一 条读取 eax 的指令必须等待这条加法结果。

内存目的操作数更要小心。比如 add dword ptr [rdi], esi 表面是一条指令，但语义是从内 存读 4 字节，加上 esi，再写回同一地址，并更新 flags。它不是普通寄存器加法，也不是单纯 store。 若以后加上 lock 前缀，它还会变成更重的原子 read-modify-write。第 7 章的最低要求是：看到内 存目的操作数，就必须把 load、计算和 store 都写进解释。

为什么编译器有时用 lea 表达加法？除了不更新 flags，lea 还能在一条指令里完成某些加法组 合，例如一个基址、一个缩放索引和一个常量位移。如果这个表达式只是为了产生整数或地址，且 不需要 flags，lea 可以避免破坏前面比较留下的 flags。这个细节连接到控制流章节：flags 是隐式状 态，被覆盖就会改变后续条件指令的意义。

有符号溢出和无符号回绕

机器的 32 位加法天然按低 32 位结果回绕。但 C++ 语言规则不同：

C++ 表达式	溢出规则
std::uint32_t+a	定义良好，按模 2 ³² 回绕
int+a	有符号溢出是 undefined behavior

所以同样可能生成一条 add，编译器对 signed 和 unsigned 源码能做的优化假设并不一样。不 要把“机器会回绕”直接等同为“C++ 程序定义良好”。

这个边界值得多说一点。机器加法总会产生某个低 N 位结果，flags 也会记录进位、溢出、零值、 符号等信息。但 C++ 优化器在生成这条机器加法之前，已经根据语言规则做过推理。对于 unsigned， 回绕是程序定义的一部分；对于 signed，如果溢出发生，程序已经进入未定义行为，编译器可以假 设这种情况不会出现在合法执行中。

因此，读到 signed 加法生成 add 时，不要说“C++ signed 加法就是补码回绕”。更准确的说法 是：在编译器认为不会发生 signed 溢出的合法路径上，这条机器指令实现了加法；如果硬件实际输 入导致低位回绕，那是某次机器执行现象，不是 C++ 对所有输入的语义承诺。性能工程里大量边 界错误、越界检查消失和循环优化惊讶，都来自忽视这点。

如果实验目的就是研究溢出，应显式使用无符号类型、编译器内建溢出检查，或能定义溢出行 为的选项和库函数。教材里的普通性能实验不应依赖 signed 溢出结果，因为那样测到的是未定义行 为下的编译器选择，而不是稳定程序语义。

CF 和 OF：同一条加法的两种溢出视角

add 和 sub 会同时更新多种 flags，其中最容易混淆的是 CF 和 OF。CF 常用于 unsigned 语义，表示 最高位之外发生进位或借位；OF 常用于 signed 语义，表示补码有符号结果超出了可表示范围。它 们来自同一条机器加法或减法，但回答的问题不同。

以 8 位加法为例，0xff+0x01 的低 8 位结果是 0x00。按 unsigned 解释，这是 255 加 1，超出

0 到 255，发生回绕，所以 CF 为 1。按 signed 解释，`0xff` 是 -1，加 1 得到 0，结果完全可表示，所以 OF 为 0。同一组位，同一条加法，unsigned 溢出和 signed 溢出结论不同。

再看 `0x7f+0x01`。低 8 位结果是 `0x80`。按 unsigned 解释，这是 127 加 1 得到 128，没有超过 255，所以 CF 为 0。按 signed 解释，127 加 1 试图得到 128，但 8 位 signed 最大值是 127，结果位模式 `0x80` 解释为 -128，所以 OF 为 1。这个例子正好和前一个例子相反。

8 位运算	低 8 位结果	unsigned 视角	signed 视角
<code>0xff+0x01</code>	<code>0x00</code>	CF=1, 回绕	OF=0, -1+1 合法
<code>0x7f+0x01</code>	<code>0x80</code>	CF=0, 未超过 255	OF=1, 127+1 越界

这张表不是让你背 flag 公式，而是训练区分问题。问“无符号加法是否产生模回绕”，看 CF；问“补码有符号数学结果是否超出范围”，看 OF。条件跳转也遵循这个分工：无符号比较用 CF/ZF 组合，有符号比较用 SF/OF/ZF 组合。后面控制流章节会系统化，本章先把概念边界打牢。

减法里的 CF 更容易让人误解。x86 的 `cmp a, b` 本质上计算 `a-b` 并设置 flags。对于 unsigned 语义，如果 `a < b`，减法需要借位，CF 会反映这个借位，因此 `jb`、`jc` 这类条件码能表达 unsigned 小于。对于 signed 语义，是否小于不能只看最高位，因为溢出会改变符号位含义，需要结合 SF 和 OF。

心智模型

CF 回答 unsigned 是否进位或借位，OF 回答 signed 补码结果是否越过可表示范围。不要把“溢出”当成一个单一概念。

显式溢出检测：让语言语义和 flags 对齐

如果程序确实需要检测整数溢出，应该把这个需求写进源码，而不是依赖未定义行为。C++ 标准库和编译器内建能力在不同版本里有所差异，工程中常见做法包括使用无符号边界推导、宽类型临时值、手写范围检查，或使用编译器提供的 `__builtin_add_overflow`、`__builtin_sub_overflow`、`__builtin_mul_overflow` 这类内建函数。

```
extern "C" bool add_overflow_i32(int a, int b, int* out)
{
    return __builtin_add_overflow(a, b, out);
}
```

这类源码把“我要知道是否溢出”变成了明确语义。编译器就可以生成 `add` 后接 `seto`，或者使用其他等价序列来保存 OF 结果。关键是：这里不是让 signed 溢出先发生再观察硬件回绕，而是请求编译器生成定义良好的溢出检测代码。源码语义和机器 flags 在这里对齐了。

无符号溢出检测常见地使用 CF。例如两个 `std::uint32_t` 相加后，如果结果小于任一加数，说明发生了模回绕；汇编也可能用 `add` 后接 `setb` 或 `adc` 之类序列。signed 溢出检测则关注 OF。两者都叫 overflow，但对应 flags 和语言语义不同。

手写溢出检查时还要避免检查表达式本身已经触发 UB。下面这种写法看似直观，但如果 `a+b` 先以 signed `int` 求值并溢出，程序已经没有定义良好语义：

```
int c = a + b;    // 如果这里溢出，后面的检查已经太晚
if (c < a) { ... }
```

正确的思路是先用不会溢出的条件判断、宽类型，或内建溢出检测，让“检查”发生在危险运算之前或由编译器以定义良好方式生成。底层教材强调这一点，是为了避免把汇编实验变成 UB 实验。

常见误区

硬件 flags 可以表达溢出现象，但 C++ signed 溢出不能先发生再被普通代码补救。要检测溢出，就在源码语义里显式表达检测。

inc 和 dec

inc 加 1，dec 减 1：

```
inc eax
```

```
dec ecx
```

它们会更新大多数 flags，但不更新 CF。现代编译器在一些场景下更偏向 `add reg, 1` 或 `sub reg, 1`，因为 flags 行为更规则，也可能更利于后续优化。你看到哪种要按上下文解释，不要死记“循环一定用 `inc`”。

`inc` 和 `dec` 的历史很长，写法也很自然，但现代编译器并不总是优先用它们。原因之一就是 flags 行为不完全等同于 `add` 和 `sub`。它们不更新 CF，这在多精度算术或依赖进位的代码中很重要。对于普通循环计数，如果后续条件只关心 ZF 或 SF/OF，差异可能不影响结果；但编译器统一使用 `add` 或 `sub` 可以让 flags 模型更规整。

读汇编时看到 `inc`，仍然要按完整规则写：它读取并写回目的操作数，更新大多数 flags，但保留 CF。不要因为它“只是加一”就忽略 flags。后续控制流如果读取 flags，就必须确认读取的是哪条指令留下的状态。

乘法：imul、常数乘法和 lea

常数乘法经常不用乘法指令

C++:

```
extern "C" int mul5(int x)
{
    return x * 5;
}
```

可能汇编:

```
lea eax, [rdi + rdi*4]
ret
```

因为:

```
x * 5 = x + x * 4
```

`lea` 可表达 `base+index*scale+displacement`，其中 `scale` 为 1、2、4、8。因此乘以 3、5、9 等常数经常可用一条 `lea`。

C++ 表达式	可能汇编	语义
<code>x*3</code>	<code>leaeax,[rdi+rdi*2]</code>	<code>x+2*x</code>
<code>x*5</code>	<code>leaeax,[rdi+rdi*4]</code>	<code>x+4*x</code>
<code>x*8+7</code>	<code>leaeax,[rdi*8+7]</code>	<code>8*x+7</code>

imul 的常见形式

当乘法不能简单用 `lea` 或 `shift` 表达时，会看到 `imul`:

```
imul eax, edi, 37    ; eax = edi * 37
imul eax, esi         ; eax = eax * esi, low 32 bits
```

对低 N 位乘法结果来说，补码 signed 和 unsigned 的低 N 位相同。因此很多普通乘法可以使用同一类指令。但当需要完整宽度结果、高半部分、溢出判断时，signed/unsigned 就会影响指令选择。

性能直觉

整数乘法通常比简单加法、位运算、`lea` 延迟更高、吞吐更低。现代 CPU 乘法已经很快，但在 tight loop 里仍然要关注。如果乘以常数，编译器经常自动替换成 `lea`、`shift` 或加法组合。不要手写“聪明替换”之前先看编译器输出。

乘法优化也是一个很好的例子，说明“源码运算符”和“机器指令”不是一一对应。源码里写 `x*5`，编译器可以生成 `lea`；源码里写数组或结构体地址计算，编译器也可能生成 `lea`，但这不代表源码里有显式乘法。汇编里的 `lea` 是地址形式的整数计算工具，既可以服务源码乘法，也可以服务内存寻址。

什么时候常数乘法适合用 `lea`、移位或加法组合，取决于常数形状、目标 CPU 指令代价、寄存器压力和是否需要 `flags`。现代编译器通常比手写替换更可靠。初学者不应该为了“乘法慢”而盲目把所有乘法改写成移位加法；这样可能降低可读性，也可能不比编译器输出更好。正确做法是先看优化后汇编，再用性能实验验证。

乘法还提醒我们注意结果宽度。一个 32 位乘法的数学结果可能需要 64 位保存，但很多指令形式只保留低 32 位。若源码类型只需要低位回绕，低位结果足够；若需要检查溢出或得到完整结果，指令选择就不同。读到 `imul` 时，要问它是哪种形式，结果保存在哪里，是否只取低位。

常见误区

不要把“没有 `imul`”解释成“没有乘法语义”，也不要把“有 `imul`”解释成“性能一定瓶颈”。先恢复表达式和结果宽度，再结合目标 CPU 和测量判断。

除法：div、idiv 和高半寄存器

整数除法比加法、乘法更难读，因为 x86 的除法指令有强烈的隐式寄存器约定，延迟通常也更高。普通二操作数风格里，你习惯看到源和目的都写在指令文本里；但 `div` 和 `idiv` 会隐式使用 `rax`、`rdx`，结果也隐式写回这些寄存器。读这类汇编时，如果只看显式操作数，会漏掉大部分语义。

无符号除法使用 `div`，有符号除法使用 `idiv`。以 64 位除法为例，被除数不是单独的 `rax`，而是 `rdx:rax` 组成的 128 位值；显式操作数是除数；商写入 `rax`，余数写入 `rdx`。32 位形式类似，使用 `edx:eax` 作为 64 位被除数，商在 `eax`，余数在 `edx`。

形式	被除数	除数	结果
<code>div r/m32</code>	<code>edx:eax</code>	显式 32 位操作数	商 <code>eax</code> ，余数 <code>edx</code>
<code>idiv r/m32</code>	<code>edx:eax</code>	显式 32 位操作数	商 <code>eax</code> ，余数 <code>edx</code>
<code>div r/m64</code>	<code>rdx:rax</code>	显式 64 位操作数	商 <code>rax</code> ，余数 <code>rdx</code>
<code>idiv r/m64</code>	<code>rdx:rax</code>	显式 64 位操作数	商 <code>rax</code> ，余数 <code>rdx</code>

这就是为什么除法前经常看到 `xor edx, edx`、`cdq` 或 `cqo`。无符号 32 位除法前，如果被除数本来只有 `eax` 的 32 位值，高半 `edx` 通常要清零，否则 `edx:eax` 会形成一个完全不同的 64 位被除数。有符号 32 位除法前，`cdq` 会把 `eax` 的符号位扩展到 `edx`；有符号 64 位除法前，`cqo` 会把 `rax` 的符号位扩展到 `rdx`。

```
; unsigned eax / esi
xor edx, edx
div esi

; signed eax / esi
cdq
idiv esi

; signed rax / rsi
cqo
idiv rsi
```

`cdq` 和 `cqo` 不应该被解释成普通“把某个变量赋给另一个变量”。它们是在构造除法要求的双倍宽度被除数。若 `eax` 是负数，`cdq` 会让 `edx` 变成全 1；若 `eax` 非负，`edx` 变成 0。这样 `edx:eax` 才是正确的有符号扩展被除数。

除法还有异常边界。除数为 0 会触发硬件异常；商无法放入目标寄存器也会触发异常。典型例子是最小 signed 整数除以 -1，数学结果超过同宽 signed 最大值。在 C++ 中，整数除以 0 是未定义行为；signed 最小值除以 -1 这类不可表示结果也不能当作普通定义良好的运算。编译器可以基于这些前提优化，所以读汇编时不要把硬件异常路径当作 C++ 保证的错误处理机制。

常见误区

`div` 和 `idiv` 的显式操作数只是除数。被除数、商和余数都在隐式寄存器里。解释除法指令必须同时追踪 `rax` 和 `rdx` 的定义。

商和余数：同一次除法产生两个结果

C++ 的 `/` 和 `%` 看起来是两个运算符，但机器除法常常一次产生商和余数。如果源码同时需要 `x/y` 和 `x%y`，编译器可能只生成一次 `idiv` 或 `div`，然后分别使用 `rax` 和 `rdx`。如果源码只需要余数，也仍然可能需要完整除法，因为余数是除法结果的一部分。

```
extern "C" int quotient_plus_remainder(int x, int y)
{
    return x / y + x % y;
}
```

可能汇编会把 `x` 放入 `eax`，用 `cdq` 构造 `edx:eax`，执行 `idiv esi`，然后把 `edx` 加到 `eax`。这里 `eax` 和 `edx` 都是除法输出。报告里不能只写“`idiv` 得到除法结果”，要写清商和余数分别在哪里。

C++ 对 signed 除法和取余有明确关系：商向零截断，余数满足 $x == (x/y) * y + x \% y$ ，并且余数符号与被除数相关。这个规则与数学里某些向下取整除法不同。负数场景下，汇编中的修正序列往往正是为了实现向零截断，而不是向负无穷取整。

除以 2 的幂：unsigned 简单，signed 需要修正

对 unsigned 整数，除以 2 的幂通常可以用逻辑右移实现。例如 `x/8u` 可以变成 `shr eax, 3`，`x%8u` 可以变成 `and eax, 7`。这是因为 unsigned 值非负，二进制右移和向下取整除法一致。

对 signed 整数，情况不同。C++ signed 除法向零截断，而 `sar` 对负数更接近向负无穷取整。比如 -3 除以 2，在 C++ 中结果是 -1；算术右移一位得到 -2。编译器要把 signed `x/2` 优化成移位时，通常会先根据符号加一个偏置，再执行算术右移。

```
; 一种常见思想，不代表唯一输出
mov eax, edi
shr eax, 31      ; 提取符号形成 0 或 1
add eax, edi     ; 负数时加偏置
sar eax, 1
```

这个序列的目的不是“多余操作”，而是修正舍入方向。对于非负数，偏置为 0，右移等价于除以 2；对于负奇数，先加 1，再算术右移，可以得到向零截断结果。除以 4、8、16 等 2 的幂时，偏置通常会变成 $2^k - 1$ ，并且只在负数路径上生效。

因此，看到 signed 除以 2 的幂生成多条位运算，不要急着说编译器没有优化。它已经把高延迟除法替换成了移位和加法，只是还必须遵守 C++ 的舍入语义。教材里反复要求说明语言规则，原因就在这里：如果只从硬件位移直觉看，你会误删必要修正。

除以常数：乘法、移位和 magic number

除以非 2 的幂常数也不一定生成 `idiv`。编译器经常把除以编译期常数改写成乘以一个预先计算的常数，再取高位、移位和修正。这类常数常被称作 magic number。它不是近似计算，而是在固定整数宽度和特定舍入规则下等价的整数算法。

```
extern "C" unsigned div10(unsigned x)
{
    return x / 10u;
}
```

优化后汇编可能出现一个看起来很大的立即数、`mul` 或 `imul`、右移和若干修正，而不是 `div`。初学者常把这误认为哈希、随机常数或混淆代码。正确读法是：编译器在用乘高位近似并精确恢复整数除法商。常数如何推导需要数论和编译器优化知识，本册只要求能识别这种模式，不把它误判为源码里真的写了乘法。

signed 常数除法更复杂，因为要满足向零截断，还要处理负数。编译器可能使用算术右移、符号位修正、加减偏置等组合。你在报告里不必手推完整 magic number 证明，但要说明：源码除法语义、目标位宽、signed/unsigned 和舍入规则共同决定了这段序列。

什么时候仍会看到真实 `div` 或 `idiv`？常见情况是除数运行期才知道，编译器无法把它化成常数乘法；或者优化级别较低；或者目标架构和代价模型判断直接除法更合适。除法通常比简单 ALU 指令重得多，但是否构成瓶颈仍要看循环结构、数据依赖和内存行为。

核心思想

整数除法的核心不是背 `idiv`，而是看清：隐式高半被除数、商和余数寄存器、除零和不可表示结果、signed 舍入规则，以及编译器用乘法和移位替代除法的条件。

符号扩展和零扩展：movsx、movzx、movsxd

从小整数类型转换到大整数类型时，编译器必须决定高位如何填充。

扩展指令是 C++ 类型语义最直观地出现在汇编里的地方。内存里同样一个字节 `0xff`，如果来源类型是 `std::uint8_t`，转换到 `int` 应该得到 255；如果来源类型是 `std::int8_t`，转换到 `int` 应该得到 -1。硬件不能只“读取一个字节”就结束，因为后续计算通常在 32 位寄存器上进行，高位必须有确定含义。

零扩展表示高位补 0，保留无符号数值；符号扩展表示复制源最高有效位，保留补码有符号数值。扩展不是“改变原来的内存”，而是决定装入更宽寄存器时高位如何构造。很多 bug 就来自错误地把字节当 signed 或 unsigned，例如图像像素、网络包字段、量化权重、字符处理和二进制文件解析。

扩展还和寄存器写入规则重叠。写 `eax` 会清零 `rax` 高 32 位，所以许多 32 位结果自然变成零扩展到 64 位。但从 8 位或 16 位变成 32 位时，编译器仍然需要 `movzx` 或 `movsx` 这类指令来决定高位。读报告时要把“32 位写清零高位”和“小宽度到大宽度的扩展”同时考虑。

零扩展

C++:

```
#include <stdint>

extern "C" int from_u8(std::uint8_t x)
{
    return x;
}
```

`std::uint8_t` 到 `int` 要保留 0..255 的值，因此高位补 0。可能汇编：

```
movzx eax, dil
ret
```

语义：

```
eax = zero_extend_32(dil)
```

如果是从内存读取：

```
movzx eax, byte ptr [rdi]
```

表示读取 1 字节，再零扩展到 32 位。

符号扩展

C++:

```
#include <stdint>

extern "C" int from_i8(std::int8_t x)
{
    return x;
}
```

`std::int8_t` 到 `int` 要保留 -128..127 的值，因此复制符号位。可能汇编：

```
movsx eax, dil
ret
```

语义：


```
eax = sign_extend_32(dil)
```

举例：

8 位输入	movzx 到 32 位	movsx 到 32 位
0x7f	0x0000007f	0x0000007f
0x80	0x00000080	0xffffffff80
0xff	0x000000ff	0xffffffffff

movsxd: 32 位到 64 位符号扩展

数组下标是 `int`，地址计算需要 64 位时，经常看到：

```
movsxd rsi, esi
mov    eax, dword ptr [rdi + rsi*4]
```

`movsxd rsi, esi` 把 32 位有符号下标扩展成 64 位。如果 `i == -1`，扩展后 `rsi` 是 `0xffffffffffffffff`，地址计算就会向前移动 4 字节。C++ 中真正访问负下标通常越界，是 UB；但从指令语义看，它就是补码地址计算。

常见误区

`char` 是否 `signed` 在 C++ 中与实现有关。写底层实验时需要明确使用 `std::int8_t` 或 `std::uint8_t`，不要靠普通 `char` 猜测编译器会生成 `movsx` 还是 `movzx`。

扩展错误会怎样污染后续计算

扩展错误不是局部小问题，它会改变后续所有算术和地址计算。假设一个字节是 `0xff`。零扩展到 32 位得到 255；符号扩展到 32 位得到 -1。之后无论是加法、比较、数组下标还是 `mask` 生成，都会沿着完全不同的路径走。

在数组下标中，这尤其危险。如果 32 位下标被 `movsxd` 扩展到 64 位，负数会变成高位全 1 的 64 位值，地址公式会向基址前方移动。机器可以照样计算地址，但 C++ 中越界访问通常是未定义行为。不要把“汇编能算出地址”当成“源码访问合法”。本书反复强调语言语义，是因为优化器会利用这些合法性前提。

在性能代码里，扩展指令也可能影响依赖和指令数量。循环里反复从 `uint8_t` 数据加载并扩展，是图像、文本、量化模型和压缩数据处理的常见模式。后续如果进入 SIMD，扩展会变成向量 `unpack`、`zero extend`、`sign extend` 等操作。第 7 章把标量扩展讲清楚，是第二册低精度计算的基础。

移位：乘除 2 的幂、位域和符号

移位指令表面简单，但它同时涉及位模式、乘除法直觉、符号处理和语言规则。左移经常被理解为乘以 2 的幂，右移经常被理解为除以 2 的幂；这种说法只在特定前提下成立。对无符号整数，左移低位结果按模宽度保留，右移逻辑补零；对有符号整数，负数、溢出和舍入方向都需要格外谨慎。

机器移位只移动位。C++ 表达式是否定义良好，还要看移位计数和被移位值。计数为负或大于等于位宽是未定义行为；`signed` 负数左移和溢出也存在规则限制。编译器可以假设合法程序不会触发这些情况。底层代码如果没有证明计数范围，就容易写出在某台机器上“看起来能跑”、但语言层不可靠的程序。

shl 和 sal

`shl` 是逻辑左移，`sal` 是算术左移；在 x86 上它们对整数位模式的效果相同：

```
shl eax, 3 ; eax = eax << 3
```

如果没有溢出语义问题，左移 3 位相当于乘以 8。但 C++ 对 `signed` 左移有规则限制，尤其是溢出和负数左移。机器指令只是移位，语言层面未必定义良好。

左移的性能通常很好，编译器也常把乘以 2 的幂替换成左移或地址计算。但你不需要手工把所有乘法写成移位。写 `x*8` 通常更清楚，优化器会选择合适指令。手工移位只有在表达位级协议、`mask`、哈希或对齐时更自然。教材训练的重点是读懂编译器输出，而不是用低级写法替代清晰源码。

shr 和 sar

右移分两种：

```
shr eax, 1 ; logical right shift, high bits filled with 0
sar eax, 1 ; arithmetic right shift, high bits filled with sign bit
```

例子：

输入 8 位	shr 1	sar 1
01111110	00111111	00111111
10000000	01000000	11000000
11111110	01111111	11111111

shr 适合 unsigned 语义；**sar** 适合补码 signed 语义。C++ 中 signed 负数右移的具体行为在现代主流实现通常是算术右移，但学习标准规则时要注意实现定义历史和标准版本细节。本书在 x86-64/Linux 实验中按补码和算术右移观察。

右移与除法的关系更容易误导。对非负整数，右移一位通常等价于除以 2 向下取整。对负数，算术右移会向负无穷方向靠近，而 C++ 整数除法通常向零截断。编译器在把 signed 除以 2 的幂优化成移位时，常常需要额外修正偏置，不能简单地把 $x/2$ 变成 **sar**。如果你在汇编里看到若干加法、移位和符号位处理组合，很可能是在实现这种舍入规则。

这也说明为什么“右移就是除法”不是合格解释。合格解释要说明数据是否无符号、是否可能为负、舍入方向是什么、语言规则要求什么。对 unsigned，**shr** 很自然；对 signed，**sar** 保留符号位，但是否等价于源码除法要看具体表达式和编译器附加修正。

移位计数

x86 中变量移位计数通常使用 **cl**：

```
shl eax, cl
shr rdx, cl
sar edi, cl
```

C++ 中如果移位计数小于 0 或大于等于位宽，行为未定义：

```
std::uint32_t x = 1;
auto y = x << 32; // undefined behavior in C++
```

硬件可能会对计数取模或屏蔽低位，但这不是你依赖的 C++ 语义。性能代码要么证明计数合法，要么显式处理。

变量移位中使用 **cl** 还有一个实际阅读意义：如果你看到编译器把某个参数移动到 **ecx** 或 **cl**，随后出现移位指令，这通常不是 ABI 参数位置本身的要求，而是 x86 指令编码对变量移位计数的要求。读汇编时要把这种目标指令约束与函数参数传递约定区分开。

移位计数合法性也会影响优化。若编译器能证明计数范围在 0 到位宽减一之间，它可以生成直接移位；若无法证明，源码又要求定义良好行为，程序员可能需要显式 **mask** 或分支处理。硬件会屏蔽某些计数位，并不意味着 C++ 自动允许越界计数。

位运算：mask、alignment 和 flags

位运算是底层编程的基本语言。它看起来像“技巧”，但本质上是在利用二进制表示的结构：某些低位表示对齐，某个位表示布尔标志，某些连续位表示字段，某个全零或全一模式表示选择掩码。理解位运算，不是为了写晦涩代码，而是为了读懂系统代码、运行时库、内存分配器、压缩格式、SIMD **mask** 和量化数据。

位运算通常不关心 signed 或 unsigned 的数学解释，而关心每一位的模式。比如 **and** 清掉某些位，**or** 设置某些位，**xor** 翻转或比较差异，**test** 用按位与结果设置 **flags**。即使操作数在 C++ 里写成整数，底层意图可能是 **mask**，而不是普通数量。

and、or、xor、not

常见位运算指令：

指令	简化语义	常见用途
and	<code>dst=dst&src</code>	清位、取低位、对齐 mask
or	<code>dst=dst src</code>	置位、合并 flags
xor	<code>dst=dst^src</code>	翻转位、清零寄存器、差异检测
not	<code>dst=~dst</code>	位取反，不更新 flags

例如判断偶数：

```
extern "C" bool is_even(unsigned x)
{
    return (x & 1U) == 0;
}
```

可能汇编：

```
test dil, 1
sete al
ret
```

`test a, b` 计算 `a&b` 并设置 flags，但不保存结果。这里用最低位判断奇偶。

对齐向下：清掉低位

如果 `align` 是 2 的幂，对地址向下对齐可写成：

```
std::uintptr_t align_down(std::uintptr_t x)
{
    return x & ~std::uintptr_t(63);
}
```

对应位运算：

```
and rax, -64
```

因为 64 字节对齐要求低 6 位为 0。清掉低 6 位就是向下对齐到 cache line 边界。

例子：

```
x = 0x1234
mask = ~0x3f = ...ffc0
x & mask = 0x1200
```

为什么 64 字节对应低 6 位？因为 $64=2^6$ 。一个地址按 64 字节对齐，意味着它能被 64 整除；在二进制中，这等价于最低 6 位全为 0。向下对齐就是保留高位、清掉低 6 位。mask `~63` 的低 6 位为 0，其余位为 1，所以按位与会把低 6 位清零。

这个知识会在很多地方出现。cache line 常见大小是 64 字节，SIMD buffer 可能要求 16、32 或 64 字节对齐，页大小常见是 4096 字节，对应低 12 位。无论具体数字是什么，只要对齐值是 2 的幂，低位清零模型都成立。后面讲内存层级和 allocator 时，这会成为基本语言。

对齐向上：加再清低位

向上对齐：

```
std::uintptr_t align_up_64(std::uintptr_t x)
{
    return (x + 63) & ~std::uintptr_t(63);
}
```

地址例子：

```
x = 0x1234
x + 63 = 0x1273
(x + 63) & ~0x3f = 0x1240
```

这个模式在 allocator、SIMD buffer、cache line padding、tensor storage 对齐中非常常见。

向上对齐比向下对齐多一个步骤：先加上 $A-1$ ，再清低位。直觉是，如果地址已经对齐，加上 $A-1$ 后仍落在同一个对齐块内，清低位会回到原地址；如果地址未对齐，加上 $A-1$ 会把它推到下一个对齐块内，清低位得到下一个边界。这个公式只对 2 的幂对齐直接成立，任意整数对齐不能直接用同一个 mask 技巧。

写底层代码时还要考虑溢出。表达式 $x+A-1$ 如果超过整数宽度，会发生回绕或未定义行为，取决于类型。地址计算通常使用无符号整数类型如 `std::uintptr_t`，但即便如此，回绕是否是你想要的结果也要小心。教材例子先讲正常范围内的位模型，工程代码还需要边界检查。

位标志

位运算经常用来维护 flags：

```
constexpr unsigned kReadable = 1u << 0;
constexpr unsigned kWritable = 1u << 1;
constexpr unsigned kPinned   = 1u << 2;

bool writable(unsigned flags)
{
    return (flags & kWritable) != 0;
}
```

可能汇编：

```
test dil, 2
setne al
ret
```

test 不保存 `flags&2`，只设置 ZF。后续 **setne** 根据 ZF 生成布尔值。

位标志的优点是紧凑，一个整数可以同时携带多个布尔状态。代价是可读性和正确性需要纪律：每个位的含义要有命名常量，设置、清除、测试要使用明确 mask，不要把 magic number 散落在代码里。汇编层看到的只是 **test dil, 2**，但源码层应当让人知道 2 代表哪个标志位。

并发场景下，位标志还可能牵涉原子操作和内存序。普通 **or** 或 **and** 修改内存中的 flags 不是自动线程安全的。第 7 章不展开并发，但要知道：位运算本身只是位模式操作，线程间可见性和竞态需要额外机制。第二册讲 atomics 和 false sharing 时会回到这个边界。

cmp 和 test：只设置 flags 的计算

cmp a, b 类似计算：

```
a - b
```

但不保存结果，只更新 flags。

test a, b 类似计算：

```
a & b
```

也不保存结果，只更新 flags。

对比：

```
sub eax, esi    ; eax = eax - esi, flags updated
cmp eax, esi    ; compute eax - esi only for flags

and eax, 1      ; eax = eax & 1, flags updated
test eax, 1     ; compute eax & 1 only for flags
```

控制流章节会系统讲条件跳转。本章先掌握一个读法：看到 **cmp** 或 **test**，不要找“结果存哪里”；结果不存，flags 才是输出。

cmp 和 **test** 是理解“隐式输出”的最好例子。它们没有显式目的寄存器，但绝不是没有输出。输出在 flags 里。后面如果紧跟 **je**、**jne**、**setl**、**setb**、**cmovg** 这类条件指令，条件指令读取的就是前面留下的 flags。读汇编时要把 flags 当成一组真实状态，而不是注释。

cmp 的 signed/unsigned 区别不在 **cmp** 本身，而在后续如何解释 flags。比如同样比较两个 32 位

位模式，`0xffffffff` 和 `1`：按 unsigned，前者大于后者；按 signed，前者代表 -1，小于后者。前面的减法设置了同一组 flags，后面的条件码选择不同组合，得到不同结论。这是 C++ signed/unsigned 比较在汇编里分叉的关键。

test 常用来判断零、判断某个位是否设置、或者检查两个 mask 是否有交集。它的好处是不用破坏原操作数。比如 **test eax, eax** 可以判断 `eax` 是否为 0，同时保留 `eax` 的值；**test dil, 2** 可以判断某个标志位，同时不保存中间 mask 结果。这种“只生产条件信息”的模式在控制流和布尔返回里非常常见。

常见误区

看到 **cmp** 或 **test** 后，必须向下寻找消费 flags 的指令。只解释它本身不够；如果找不到消费者，可能是 flags 后来被覆盖，或者这段反汇编不是完整热路径。

同一个 cmp，不同条件码

有符号比较和无符号比较最适合用“同一个 **cmp**，不同消费者”来理解。假设两个 32 位位模式分别是 `0xffffffff` 和 `0x00000001`。执行 **cmp eax, esi** 后，硬件只知道它做了一次低 32 位减法并设置 flags。它并不知道源码想比较 **int** 还是 **unsigned**。真正的分叉发生在后续条件指令。

源码语义	常见条件码	直觉
<code>intx < y</code>	j l 或 set l	signed 小于，看 SF 与 OF
<code>intx <= y</code>	j le 或 set le	signed 小于等于，看 ZF/SF/OF
<code>unsignedx < y</code>	j b 或 set b	below, unsigned 小于，看 CF
<code>unsignedx <= y</code>	j be 或 set be	below or equal, 看 CF/ZF
<code>x == y</code>	j e 或 set e	相等，只看 ZF
<code>x != y</code>	j ne 或 set ne	不相等，只看 ZF

jl 里的 less 是 signed less；**j**b 里的 below 是 unsigned below。很多反汇编器还会显示 **j**c、**j**nc、**j**a、**j**g 等别名。读者不需要在第一遍背完所有名字，但必须知道 signed 和 unsigned 使用的条件码族不同。看到 **j**b，优先想到 unsigned 小于或借位；看到 **j**l，优先想到 signed 小于。

比较常量时也要注意操作数顺序。Intel 语法的 **cmp eax, 10** 表示用 `eax` 减 10 设置 flags。随后 **j**l 表示 `eax` 按 signed 小于 10，**j**b 表示 `eax` 按 unsigned 小于 10。如果换成 **cmp 10, eax**，逻辑方向就反过来。实际汇编中立即数不能总是作为目的操作数，所以编译器会选择可编码形式；反汇编报告必须根据真实操作数顺序解释条件。

布尔返回常用 **setcc**。例如：

```
cmp edi, esi
setl al
ret
```

这里 **setl al** 只写低 8 位 `al`，随后返回 **bool** 时 ABI 只关心低 8 位结果。若返回 **int**，编译器通常还会用 **movzx eax, al** 或直接生成能把高位清零的序列，确保 `eax` 是 0 或 1。不要把 **setcc** 写成“跳转”，它是条件置字节；也不要忽略它只写一个字节的宽度。

min/max、条件移动和比较证据

比较不一定生成分支。一个简单的 **min** 或 **max** 可能生成 **cmp** 加条件移动 **cmovcc**，也可能生成分支，或者在优化后变成其他等价序列。

```
extern "C" int min_i32(int a, int b)
{
    return a < b ? a : b;
}
```

可能汇编：

```
mov eax, esi
cmp edi, esi
cmovl eax, edi
```



```
ret
```

这段汇编没有分支，但仍然有比较。条件移动读取 flags，如果条件成立，就把源寄存器复制到目的寄存器；如果条件不成立，目的寄存器保持旧值。这里 `cmovl` 的 `l` 仍然是 signed less。若源码是 `unsigned`，可能变成 `cmovb` 或其他 `unsigned` 条件移动。

条件移动的性能好坏取决于场景。它避免了分支预测失败，但也可能让两边值都需要提前计算，并引入数据依赖。第一册不展开完整分支预测模型，但要建立读法：没有 `jcc` 不代表没有条件语义，看到 `cmovcc` 和 `setcc` 同样要追踪 flags 来源。

test reg, reg 和零比较

判断一个寄存器是否为 0，经常看到 `test eax, eax`，而不是 `cmp eax, 0`。两者都能设置 ZF 来表达是否为零，但 `test reg, reg` 编码紧凑，也清楚表达“按位与自己，只关心是否全零”。它还会清除 CF 和 OF，设置 SF/ZF/PF 等 flags。具体 flags 行为在手写汇编中可能重要。

```
test eax, eax
je zero_case
```

这段代码的条件不是“按位与的结果被保存了”，而是“按位与结果为 0，所以 ZF=1”。如果 `eax` 非零，`eax&eax` 非零，ZF=0。对于指针判空也类似，常见 `test rdi, rdi` 后接 `je` 或 `jne`。

`test` 也常用于 mask 判断。例如 `test eax, 7` 可以判断低 3 位是否全为 0，用于检查 8 字节对齐；`test eax, eax` 是其中的特殊情况，mask 等于自身。报告里应说明它检查的是哪几位，而不是笼统写“测试 `eax`”。

核心思想

`cmp` 和 `test` 的意义要和后续 `jcc`、`setcc`、`cmovcc` 连起来读。条件码才告诉你 signed、unsigned、相等、零值或 mask 的真正语义。

read-modify-write：一条指令不等于一次微操作

看：

```
add dword ptr [rdi], esi
```

ISA 语义可以写成：

```
tmp = load 4 bytes from [rdi]
tmp = tmp + esi
store low 4 bytes of tmp to [rdi]
update flags
```

它是一条汇编，但语义包含 load、ALU、store。对于普通内存操作，这已经比寄存器加法重；对于原子 read-modify-write，例如 `lockadd`，还会牵涉 cache coherence 和内存序。

性能工程里要区分：

```
add eax, esi           ; register ALU
add dword ptr [rdi], esi ; memory RMW
lock add dword ptr [rdi], esi ; atomic RMW, much heavier
```

本章只要求你看到内存目的操作数时提高警觉：这不是普通寄存器加法。

read-modify-write 指令还有一个重要边界：ISA 把它描述成一条指令，但性能上它不是“免费合并”。处理器仍然要获得内存中的旧值，完成计算，再安排写回。旧值如果不在 L1 cache 中，load 部分会等待；store 部分也要进入 store buffer，并遵守内存顺序规则。它比寄存器上的 `add eax, esi` 复杂得多。

从正确性角度看，普通 read-modify-write 也不是线程安全的原子操作。两个线程同时执行普通 `add dword ptr [mem], 1`，可能发生丢失更新，因为 load 和 store 之间没有原子性保证。带 `lock` 前缀的原子 RMW 才会引入缓存一致性协议层面的互斥效果，但代价也高得多。第 7 章只建立这个边界，后面并发章节再展开。

从汇编阅读角度看，内存目的操作数要明确写出“读内存”和“写内存”。很多报告只写“给内

存加上 `esi`”，这仍然太粗。应该写：从有效地址读取指定宽度的旧值，与寄存器或立即数计算，把结果写回同一地址，并更新 `flags`。这样才能为后续性能分析留下足够证据。

核心理想

一条汇编指令是一条 ISA 指令，不等于一次内部操作，也不等于一次内存访问。读性能相关汇编时，要把语义拆成 `load`、计算、`store` 和 `flags` 更新。

部分寄存器：为什么编译器偏爱 32 位写

x86-64 的通用寄存器有多个名字表示不同宽度。例如 `rax` 是 64 位，`eax` 是低 32 位，`ax` 是低 16 位，`al` 是低 8 位。它们不是四个独立寄存器，而是同一个物理/架构寄存器的不同视图。这个设计让旧代码兼容新架构，但也给汇编阅读带来细节。

名字	宽度	写入效果	典型用途
<code>rax</code>	64 位	写完整 64 位	指针、 <code>std::uint64_t</code> 、返回 64 位值
<code>eax</code>	32 位	写低 32 位，并清零高 32 位	<code>int</code> 运算、清高位、返回 32 位值
<code>ax</code>	16 位	只写低 16 位，高位保留	窄整数、特殊 ABI 或指令约束
<code>al</code>	8 位	只写低 8 位，高位保留	<code>bool</code> 、字节操作、 <code>setcc</code>

最重要的规则是：写 32 位寄存器会把对应 64 位寄存器的高 32 位清零。写 `eax` 后，`rax` 的高半一定为 0；写 `edi` 后，`rdi` 的高半一定为 0。这个规则在 x86-64 中非常关键，编译器经常利用它产生零扩展效果和打断旧高位依赖。

相比之下，写 8 位或 16 位寄存器不会自动清高位。执行 `mov al, 1` 只改变 `rax` 的最低 8 位，`ah`、`ax` 的高 8 位、`eax` 的高 24 位和 `rax` 的高 32 位都保留旧值。若后续把完整 `eax` 或 `rax` 当成新值读取，就必须确认旧高位是否仍然合法。编译器通常会通过 `movzx` 等指令把窄结果扩展成干净的 32 位值。

```
setne al
movzx eax, al
```

这两条常见序列表示先根据 `flags` 产生 0 或 1 的字节，再零扩展到完整 `eax`。如果源码返回 `bool`，只写 `al` 通常足够；如果返回 `int`，高位必须明确为 0。这个差异来自 ABI 和返回类型，不是编译器随意多生成了一条指令。

部分寄存器还牵涉性能历史。较老或某些微架构上，先写低 8 位或 16 位、再读完整 32/64 位，可能需要硬件合并旧值和新值，形成额外依赖。现代 CPU 已经改善了很多场景，但编译器仍然倾向于使用 32 位写来产生完整、干净、依赖简单的值。这也是你经常看到 `xor eax, eax` 而不是 `xor al, al` 的原因之一。

高 8 位寄存器 `ah`、`bh`、`ch`、`dh` 还和 REX 前缀存在编码限制，现代 64 位代码中较少主动使用。初学者不需要记住所有编码细节，但要知道：如果反汇编出现 `al`、`ax`、`eax`、`rax`，它们的写入宽度和高位效果不同。报告里不能只写“写 `rax`”，应写明实际写的是哪一部分。

常见误区

写 `eax` 会清零 `rax` 高 32 位；写 `al` 或 `ax` 不会。很多整数汇编 bug 都来自把部分寄存器写入误认为完整寄存器赋值。

案例：从 C++ 类型转换到扩展指令

创建：

```
#include <cstdint>

extern "C" int sum_bytes(std::uint8_t const* a, std::int8_t const* b, int i)
{
    return a[i] + b[i];
}
```

可能汇编：

```
movsxd rax, edx
movzx ecx, byte ptr [rdi + rax]
movsx eax, byte ptr [rsi + rax]
add eax, ecx
ret
```

逐条解释：

```
entry:
    rdi = a
    rsi = b
    edx = i

movsxd rax, edx
    rax = sign_extend_64(i)

movzx ecx, byte ptr [rdi + rax]
    load a[i] as uint8_t
    ecx = zero_extend_32(a[i])

movsx eax, byte ptr [rsi + rax]
    load b[i] as int8_t
    eax = sign_extend_32(b[i])

add eax, ecx
    eax = int(b[i]) + int(a[i])
```

这里 `movzx` 和 `movsx` 的差异来自 C++ 类型。换成两个 `std::uint8_t`，第二个 `load` 很可能也变成 `movzx`。

这个案例的重点不是记住四条指令，而是看清类型如何留下痕迹。两个数组都按字节访问，地址公式也几乎一样；真正区分它们的是扩展方式。无符号字节进入整数加法前必须零扩展，有符号字节进入整数加法前必须符号扩展。后面的 `add` 只是 32 位低位加法，它并不知道两个源值原来来自什么类型。

写报告时应该把这类案例分成三层。第一层是内存事实：两个 `load` 都读取 1 字节，地址分别是 `rdi` 加下标、`rsi` 加下标。第二层是扩展事实：一个零扩展到 32 位，一个符号扩展到 32 位。第三层是 C++ 推论：这对应 `std::uint8_t` 和 `std::int8_t` 的整数提升。三层都写清楚，结论才稳。

还要注意 `i` 的扩展。参数 `i` 是 `int`，地址计算需要 64 位索引，所以编译器用 `movsxd`。如果源码使用 `std::size_t`，入口参数可能已经是 64 位无符号值，汇编形态可能不同。不要把 `movsxd` 当成所有数组访问固定模板，它来自具体下标类型和目标地址宽度。

案例：地址计算、常数乘法和结构体大小

C++：

```
struct Node {
    int key;
    int value;
    int next;
};

extern "C" int get_value(Node const* nodes, int i)
{
    return nodes[i].value;
}
```

布局：

```
sizeof(Node)      = 12
offset(Node,value) = 4
addr(nodes[i].value) = base + i * 12 + 4
```

可能汇编：

```
movsxd rax, esi
```

```

lea    rax, [rax + rax*2]
mov    eax, dword ptr [rdi + rax*4 + 4]
ret

```

解释：

```

rax = sign_extend_64(i)
rax = rax + rax * 2 = i * 3
address = base + (i * 3) * 4 + 4
         = base + i * 12 + 4

```

这个案例很典型：结构体大小是 12，不能直接作为 x86 scale，于是编译器拆成 $i*3$ 再 $*4$ 。

这个案例还训练一个重要能力：不要被单个 scale 误导。最后的内存操作数里看起来是 $rax*4+4$ ，但 rax 已经被上一条 `lea` 改成了 $i*3$ 。完整地址公式必须沿着寄存器定义链往前追，不能只看最后一条内存指令。最终步长是 $3*4=12$ ，不是 4。

复杂数据布局经常会出现这种多步地址计算。结构体大小不是 1、2、4、8 时，x86 寻址模式无法直接表达 $i*sizeof(T)$ ，编译器就会先构造某个中间倍数，再在内存操作数中使用允许的 scale。二维数组、AoS/AoSoA 布局、张量 strides 也会出现类似模式。第 7 章只用小结构体示范，后面内存和性能章节会继续扩大。

如果要验证字段 offset，最好同时用 C++ 的 `sizeof` 和 `offsetof`，再对照汇编。不要只从汇编猜结构体，因为不同字段组合可能产生相同 offset。汇编能证明机器访问了某个偏移，源码和布局工具才能证明这个偏移对应哪个字段。

案例：branchless mask 的第一眼

后面会系统讲 branchless 技术，这里先看一个基础模式：

```

extern "C" unsigned mask_if_nonzero(unsigned x)
{
    return x != 0 ? 0xffffffffu : 0u;
}

```

一种可能汇编：

```

neg edi
sbb eax, eax
ret

```

简化解释：

- `neg edi` 计算 $0-x$ ，并根据是否借位设置 CF。
- 如果 $x!=0$ ，CF 通常为 1；如果 $x==0$ ，CF 为 0。
- `sbb eax, eax` 计算 $eax=eax-eax-CF$ 。
- 当 CF 为 1，结果是 $0xffffffff$ ；当 CF 为 0，结果是 0。

这个例子不要求你现在熟练使用 `sbb`，但要建立意识：整数指令和 flags 可以组合出没有分支的 mask。SIMD tail mask、选择、量化 clamp、条件累加等场景会反复出现这种思想。

这个例子容易让人兴奋，也容易让人过早滥用 branchless 写法。这里的教学目的不是让你立刻手写 `neg` 加 `sbb`，而是理解整数指令、flags 和补码可以合成布尔掩码。编译器在某些场景会自动选择这种形式，尤其当分支预测代价可能高于几条整数指令时。

但是 branchless 并不天然更快。它可能增加指令数量，延长依赖链，消耗执行端口，或者让本来很好预测的分支变成额外计算。是否值得使用，要看数据分布、分支可预测性、指令代价和整体热路径。第 8 章会更系统地讲分支和条件移动；本章只把 mask 生成作为整数指令组合能力的预告。

从报告角度看，这类代码更要写清 flags 依赖。不能只写“生成 mask”。应说明哪条指令设置 CF，哪条指令读取 CF，为什么结果变成全零或全一。否则读者无法判断这个技巧是否真的成立。

案例：signed 和 unsigned 比较只差一个条件码

下面两个函数源码结构几乎相同，唯一差别是类型：

```
extern "C" bool less_i32(int a, int b)
{
    return a < b;
}

extern "C" bool less_u32(unsigned a, unsigned b)
{
    return a < b;
}
```

优化后汇编可能都是先比较两个参数，再设置低字节返回值：

```
less_i32:
    cmp edi, esi
    setl al
    ret

less_u32:
    cmp edi, esi
    setb al
    ret
```

核心差异不是 `cmp`，而是 `setl` 与 `setb`。`setl` 按 signed less 解释 flags，`setb` 按 unsigned below 解释 flags。若 `edi` 的位模式是 `0xffffffff`，`esi` 是 `1`，第一个函数把它解释成 `-1` 小于 `1`，返回真；第二个函数把它解释成 `4294967295` 小于 `1`，返回假。

这个案例说明，汇编里没有类型标签，但类型仍然改变机器代码的条件码。报告必须写出“同一条 `cmp` 的 flags 被不同条件码消费”。如果只写“`cmp` 比较 `a` 和 `b`”，两个函数会被错误地解释成一样。

若把源码改成混合类型：

```
extern "C" bool less_mixed(int a, unsigned b)
{
    return a < b;
}
```

它通常会更接近 unsigned 版本，因为 usual arithmetic conversions 会把 `a` 转成 unsigned 后比较。这个结果对很多人反直觉，但它是 C++ 类型规则的直接后果。底层性能代码里，索引、长度、偏移、返回值的 signed/unsigned 选择必须有统一策略，否则边界判断很容易在汇编层变成另一种语义。

案例：除以常数为什么没有 `idiv`

考虑三个函数：

```
extern "C" unsigned u_div8(unsigned x) { return x / 8u; }
extern "C" int      i_div8(int x)      { return x / 8; }
extern "C" unsigned u_div10(unsigned x){ return x / 10u; }
```

第一个函数通常可以用 `shr`。第二个函数虽然也是除以 8，但 signed 负数要向零截断，编译器可能生成提取符号、加偏置、`sar` 的组合。第三个函数除数不是 2 的幂，优化后可能出现乘以大常数、取高位和右移，而不是 `div`。

这三个函数是非常好的课堂对照。它们都写着除法，但汇编形态分别体现了 unsigned 右移、signed 舍入修正和常数除法强度削弱。学生如果只记“除法对应 `idiv`”，会完全读错；如果只记“除以常数会优化”，也会漏掉 signed 舍入规则。正确答案必须把源码类型、除数形状和 C++ 除法语义连起来。

写这类报告时，不要求手工证明 magic number 的正确性，但要做三件事。第一，指出源码是否 signed。第二，指出除数是否为 2 的幂。第三，指出生成序列是为了避免高延迟除法，同时保持语言规定的商。若报告里出现“编译器把除法错编成乘法”这种说法，说明对整数优化还没有入门。

案例：溢出检测和普通加法的差别

再比较两个函数：


```
extern "C" int add_plain(int a, int b)
{
    return a + b;
}

extern "C" bool add_checked(int a, int b, int* out)
{
    return __builtin_add_overflow(a, b, out);
}
```

第一个函数只要求在定义良好的 signed 加法范围内返回数学结果。编译器可以生成一条 `lea` 或 `add`，并不需要保存 OF。第二个函数要求返回是否溢出，并把低位结果写入 `out`。汇编里通常会看到会设置 flags 的加法、store，以及 `seto` 或等价条件处理。两者看起来都做加法，但语义合同完全不同。

这个对照能纠正一个常见误解：不是所有加法都需要检查溢出。C++ 普通 signed 加法没有把溢出变成返回值；一旦溢出，程序语义不再定义良好。只有源码显式要求检查，编译器才需要把 OF 转化成可见结果。底层教材必须区分“硬件产生了 OF”与“程序语义需要观察 OF”。

在安全关键代码、解析外部输入、分配内存大小计算、图像尺寸和 tensor 元素个数计算中，显式溢出检查非常重要。例如计算 `n*sizeof(T)` 时，如果乘法溢出再传给 allocator，可能导致缓冲区过小。工程代码应当使用定义良好的检查方式，而不是依赖回绕后再猜测。后续内存章节会继续讨论这类问题。

性能注意点：延迟、吞吐、依赖和 flags

本章不做完整微架构表，但必须先建立几个判断：

1. 寄存器 ALU 指令通常很便宜。
2. 乘法通常比加法和位运算更重。
3. 内存操作比寄存器操作更容易受 cache、TLB、store buffer 影响。
4. 更新 flags 会产生 flags 依赖；如果后续不需要 flags，`lea` 可能避免不必要的 flags 写。
5. 32 位写入清零高位，经常用于打断旧高位依赖。
6. 小宽度部分寄存器写入可能引入额外依赖或需要合并旧值。

例如：

```
add eax, esi      ; writes eax and flags
lea eax, [rdi+rsi]; writes eax, does not write flags
```

如果后面紧跟条件跳转读取上一条 `cmp` 的 flags，那么中间插入会写 flags 的指令可能破坏条件。编译器会避开这种错误，但手写汇编必须自己负责。

这一节只给定性直觉，不给具体周期表，是有意的。不同 CPU 微架构上，整数加法、乘法、load、store、`lea` 的 latency 和 throughput 都可能不同；同一指令在不同寻址形式下也可能代价不同。第一册的目标是让你先能分类：哪些是寄存器 ALU，哪些访问内存，哪些产生 flags 依赖，哪些可能是 RMW，哪些只是地址计算。

真正的性能判断需要结合上下文。单独一条乘法可能不是瓶颈，如果循环受内存带宽限制；一条 load 也可能很便宜，如果数据稳定命中 L1；一条 `lea` 也可能不是免费，如果它在关键依赖链上或形式复杂。不要把“某指令通常更快”变成绝对结论。后续测量章节会要求用数据验证。

不过，有几个阅读习惯现在就要养成。看到寄存器到寄存器 ALU，先标记依赖链；看到内存操作，先标记 load/store 和地址来源；看到 flags 写入，检查后续是否读取；看到 `imul`，确认是否常数乘法已经无法用简单地址计算表达；看到 `movsx/movzx`，回到 C++ 类型。这样读出来的汇编，才有资格进入性能分析。

心智模型

第一册不让你背每条指令的周期数，而是训练分类和证据。分类正确，第二册查具体微架构数据才有意义；分类错误，周期表背得再熟也会判断错。

整数汇编报告应该怎样写

本章的实验报告要比上一章更严格。上一章主要训练语法和地址公式；本章必须把类型、位宽、扩展、flags 和语言规则写进去。

报告开头应列出源码中涉及的整数类型，尤其是 `std::int8_t`、`std::uint8_t`、`int`、`unsigned`、`std::size_t`、`std::uintptr_t`。不要只贴源码。你要说明这些类型的宽度、signed/unsigned 属性，以及它们为什么会影响扩展、比较、移位或地址计算。

逐条注释时，至少写出六项：指令读什么，写什么，操作数宽度，是否访问内存，是否更新 flags，位模式如何解释。遇到扩展指令，要说明高位如何填充；遇到移位，要说明计数是否合法以及是逻辑还是算术；遇到位运算，要说明 mask 的二进制含义；遇到 `cmp/test`，要说明后续消费者。

报告还要写语言边界。比如 signed 溢出不能当成定义良好回绕；负下标地址计算不代表 C++ 允许访问；移位计数越界不能靠硬件屏蔽解释成合法；普通内存 RMW 不等于原子操作。这些边界是本章质量要求的一部分。没有边界说明的报告，只能算指令翻译，不能算教材式分析。

核心思想

整数汇编报告的最低标准是：从 C++ 类型到机器位宽，从指令读写到 flags，从地址公式到语言合法性，都能闭环解释。

flags 不是附属品：它们是隐藏的数据流

很多读者学整数指令时只盯着通用寄存器，忽略 flags。这样读简单加法还勉强可以，一旦遇到比较、条件跳转、条件移动、溢出检查、进位链、多精度运算，就会失去主线。flags 不是装饰性副作用，而是一组隐含的数据流。某条指令写 flags，后面的 `jcc`、`setcc`、`cmovcc`、`adc`、`sbb` 等指令读取 flags。只要中间有另一条写 flags 的指令，原来的条件信息就被覆盖。

读 flags 要先抓住四个常用标志位：

标志位	直觉	常见用途
ZF	结果是否为 0	相等比较、零测试、循环结束
SF	结果最高有效位是否为 1	signed 结果符号的一部分证据
CF	无符号加法进位或减法借位	unsigned 比较、多精度加减
OF	signed 补码溢出	signed 比较、显式溢出检测

这四个标志位不能孤立理解。ZF 常常比较直观：`sub` 或 `cmp` 的结果为零，说明两个操作数相等。CF 和 OF 最容易混淆：CF 是无符号视角下低 N 位放不下或发生借位，OF 是有符号补码视角下数学结果超出范围。SF 只是结果最高位，不等于“signed 比较结论”，因为 signed 比较还要考虑 OF。条件码之所以有 `jnl` 和 `jb` 的区别，就是因为它们消费 flags 的方式不同。

把 flags 当成一个临时寄存器

可以把 flags 想成一个没有名字的临时寄存器。下面这段代码：

```
cmp edi, esi
setl al
```

可以理解为：

```
tmp_flags = compare_signed_related(edi - esi)
al = tmp_flags.signed_less ? 1 : 0
```

如果中间插入一条会写 flags 的指令：

```
cmp edi, esi
add ecx, 1
setl al
```

那么 `setl` 读取的是 `add` 的 flags，不再是 `cmp` 的 flags。这就是为什么编译器在调度指令时要尊重 flags 依赖，为什么手写汇编不能随意在比较和跳转之间插入指令。某些指令如 `lea` 不写 flags，

所以常常可以放在比较和条件跳转之间而不破坏条件；某些指令如 `inc`、`dec` 写一部分 flags 而不写 CF，也会带来更细的依赖问题。

flags 的生命周期通常很短。编译器喜欢让 `cmp` 或 `test` 紧贴 `jcc`，一个原因是读者容易看懂，另一个原因是减少 flags 活跃区间，给指令调度留下空间。若你看到比较和使用 flags 的指令隔得很远，不要只看最近的 `cmp`，要逐条检查中间是否有 `add`、`sub`、`and`、`or`、`xor`、`test`、`imul` 等可能写 flags 的指令。

cmp、sub 和条件码的关系

`cmp a, b` 本质上计算 `a-b` 并设置 flags，但不保存结果。Intel 语法中目的操作数在前，源操作数在后，所以：

```
cmp edi, esi
```

设置的是 `edi-esi` 的 flags。若后面是：

```
jl .Lless
```

语义是 signed `edi<esi`；若后面是：

```
jb .Lbelow
```

语义是 unsigned `edi<esi`。同一个 `cmp` 可以服务 signed 或 unsigned 条件，区别在消费者。报告中写“`cmp` 比较两个数”不够，必须写“后续条件码如何解释这次减法 flags”。

减法结果的低 N 位相同，但 signed 和 unsigned 结论不同。以 8 位直觉说明，`0xff-0x01` 的低位结果是 `0xfe`。按 unsigned，255 大于 1，不发生借位，CF 为 0；按 signed，-1 小于 1，signed less 条件应为真。硬件不需要把寄存器标成 signed，它只提供 flags，条件码负责选择解释。

为什么 test 常用于零测试

`test a, b` 本质上计算 `a&b` 并设置 flags，但不保存结果。最常见形式是：

```
test edi, edi
je .Lzero
```

一个数与自身按位与，结果仍是它自己。因此 ZF 说明 `edi` 是否为 0。编译器常用 `test reg, reg` 做零测试，因为它不需要额外立即数，也能设置后续条件跳转需要的 flags。若你看到 `test` 后接 `js`，则是在测试结果最高位；若 `test mask, value` 后接 `jne`，通常是在测试某些 bit 是否至少有一位为 1。

`test` 与 `cmp reg, 0` 常能表达相同零测试，但 flags 细节和编码可能不同。报告不需要宣称某一种一定更快，只要说明它生产了后续条件需要的 ZF/SF 等信息。若后续是 signed 大小比较，就不能把任意 `test` 误解成 `cmp` 的替代，因为 signed 大小比较通常需要减法相关的 OF 信息。

整数宽度是一种契约

每条整数指令都有宽度。宽度决定读取多少位、写入多少位、flags 按多少位结果计算、立即数如何扩展、内存访问几个字节、写入 32 位寄存器是否清零高位。宽度不是语法细节，而是机器契约。一个 8 位 `add al, 1` 和一个 32 位 `add eax, 1` 都叫 `add`，但它们影响的寄存器部分、flags 计算宽度和后续解释都不同。

读宽度时，先看寄存器名。`rax` 是 64 位，`eax` 是低 32 位，`ax` 是低 16 位，`al` 是低 8 位。内存操作数则需要从指令或显式 `ptr` 标记判断，例如 `byteptr`、`wordptr`、`dwordptr`、`qwordptr`。有些汇编行从寄存器操作数就能推出内存宽度，例如 `mov dword ptr [rdi], eax` 里的 `eax` 已经说明写 4 字节；有些行必须写 `ptr` 标记，否则汇编器无法判断。

写 32 位寄存器为什么常见

x86-64 中写 32 位通用寄存器会把对应 64 位寄存器的高 32 位清零。例如写 `eax` 会让 `rax` 高半部分变成 0。这条规则对读汇编非常重要。它解释了为什么编译器经常用 `xor eax, eax` 清零返回值，也解释了为什么读取 `std::uint32_t` 后返回 `std::uint64_t` 可能只需要写 `eax`。

但这也制造陷阱。假设入口 `rdi` 是 64 位指针，如果代码写：

```
mov edi, edi
```

这不是无效果操作，它会清零 `rdi` 高 32 位，从而破坏地址。编译器不会这样处理真实指针，但手写汇编或错误约束的内联汇编可能造成灾难。相反，若值的语义本来就是无符号 32 位扩展到 64 位，写 32 位寄存器正好表达了需要的零扩展。

部分寄存器写入要检查后续合并

写 8 位或 16 位部分寄存器不会自动清零高位。例如：

```
mov al, 1
```

只写 `rax` 的低 8 位，其余位保持旧值。若后续读取 `eax` 或 `rax`，必须知道高位来自哪里。编译器通常会在需要完整整数结果时使用 `movzx` 清高位，例如：

```
setne al
movzx eax, al
```

`setne al` 只产生低 8 位布尔结果，`movzx eax, al` 把它变成 0 或 1 的 32 位整数结果。若报告只写“`setne` 返回 `bool`”，就漏掉了后续扩展对返回寄存器正确性的作用。C++ `bool` 返回值和 `int` 返回值在 ABI 上都可能使用 `al/eax` 的有效部分，但调用者如何读取、编译器如何清理高位，仍要看具体代码和类型。

立即数宽度和符号扩展

立即数也有宽度。某些 x86 指令会把较小立即数符号扩展到操作数宽度，例如常见的 `add rax, imm32` 会把 32 位立即数符号扩展后参与 64 位加法。另一些形式有专门短立即数编码。反汇编中显示的 `addrax, -1` 可能编码为较短立即数，而不是完整 64 位常量。

这通常不影响高层语义，但会影响机器码字节和某些边界常量。若你要解释指令长度、重定位、机器码字节或手写汇编编码，就不能只看助记符。对于普通阅读，至少要知道：操作数宽度由寄存器或内存宽度决定，立即数会按指令规则扩展或截断。若立即数超出可编码范围，汇编器可能选择不同编码或报错。

核心思想

宽度决定结果，不只是显示风格。任何整数指令分析都必须先回答“这是几位运算”，再谈 `signed`、`unsigned`、溢出、返回值和性能。

从 C++ 整数表达式推到指令选择

读整数汇编的另一半工作，是从 C++ 表达式规则理解编译器为什么可以这么生成。C++ 源码中的 `+`、`-`、`*`、`/`、`<`、`>` 并不直接指定某条机器指令，而是指定语言语义。编译器只要保持定义良好程序的可观察行为，就可以选择不同机器形态。

普通加法与显式溢出检查

看两个函数：

```
extern "C" int add_plain(int a, int b)
{
    return a + b;
}

extern "C" bool add_checked(int a, int b, int* out)
{
    return __builtin_add_overflow(a, b, out);
}
```

`add_plain` 的语言合同是在不发生 `signed` 溢出的定义良好输入上返回和。编译器可以用 `lea`，也可以用 `add`，不需要保留 `OF` 给调用者。若输入导致 `signed` 溢出，C++ 层没有定义良好结果，不能用“硬件回绕了”反推源码合法。

`add_checked` 的合同完全不同。它要求把加法低位结果写入 `out`，并返回是否溢出。此时 `OF`

或等价溢出检测必须转化成可见布尔值。你可能看到 `add` 后接 `seto`，也可能看到其他序列。报告应写清楚：这里观察 OF 不是因为所有 signed 加法都必须检查，而是因为源码显式要求检查。

无符号回绕是语言语义，不是优化失败

`unsigned` 加法按模 2^N 回绕，这是 C++ 语言定义。编译器不能假设它不溢出，从而像 signed 那样使用“无溢出”前提做某些变换。例如：

```
extern "C" bool wrapped(unsigned x)
{
    return x + 1 < x;
}
```

对无符号整数，这个表达式只在 `x` 为最大值时为真。编译器可能把它优化成与最大值比较。若把类型改成 `int`，`x+1<x` 在定义良好 signed 语义下永远为假，因为 `x+1` 不允许溢出；编译器可以直接返回 `false`。两者的机器码差异来自语言规则，不是硬件加法器不同。

右移要区分算术和逻辑

右移是另一个类型驱动的典型例子。无符号右移应高位补 0，常对应 `shr`；有符号负数右移在现代编译器和目标上通常使用算术右移 `sar` 以复制符号位，但要注意语言标准和实现定义细节。对于非负 signed 值，`shr` 和 `sar` 的结果可能相同；对于负数，它们完全不同。

除以 2 的幂也不能简单等同右移。无符号除以 2^k 可以直接逻辑右移；signed 除法在 C++ 中向零截断，而算术右移对负数更接近向负无穷取整。编译器若把 signed 除以 8 改写成移位，通常需要先做偏置修正。看到几条看似多余的加法、比较、移位，不要急着说“编译器没有优化好”，它可能是在保持 C++ 舍入语义。

比较指令不携带类型，条件码携带解释

比较表达式的类型信息通常体现在条件码上。下面两个函数：

```
extern "C" bool less_i(int a, int b) { return a < b; }
extern "C" bool less_u(unsigned a, unsigned b) { return a < b; }
```

可能都使用 `cmp edi, esi`。差异在后续 `setl` 和 `setb`，或 `jle` 和 `jbe`。若是混合 signed/unsigned，C++ 的 usual arithmetic conversions 可能让比较转成 unsigned 条件码。报告必须把源码类型、转换规则和条件码三者连起来。

多精度整数的第一眼：adc 和 sbb

虽然第一册不展开大整数库实现，但理解 `adc` 和 `sbb` 能帮助你看懂 CF 的真实用途。普通 `add` 做低 `N` 位加法并设置 CF；`adc` 会把 CF 也加进去。这样可以把多个机器字拼成更宽的整数。

例如做 128 位无符号加法，低 64 位相加后产生的进位要传给高 64 位：

```
add rax, rdx
adc rcx, r8
```

可以理解为：

```
low  = low_a + low_b
carry = carry_out(low)
high = high_a + high_b + carry
```

这里 CF 就是跨指令的数据。它不是“顺便设置的标志”，而是高位加法的输入。类似地，`sbb` 会把借位纳入减法，常用于多精度减法，也常用于生成全零或全一掩码：

```
cmp edi, esi
sbb eax, eax
```

这类序列的具体含义取决于 CF。若比较后发生 unsigned 借位，`sbb eax, eax` 可以把 `eax` 变成 `0xffffffff`；否则变成 0。优化器和手写高性能代码有时利用这种性质构造 branchless mask。读到这类代码时，必须把 CF 从生产者追到消费者，否则完全看不懂。

常见误区

`adc` 和 `sbb` 说明 `flags` 可以参与真实数据计算。不要把 `flags` 只看成条件跳转的附属信息。

整数错误诊断：从症状回到位宽

整数 bug 在底层经常表现为“偶尔越界”“大输入才错”“负数才错”“跨平台才错”。诊断时不要先猜算法，而要回到位宽、扩展、比较和溢出四类问题。

第一类是扩展错误。无符号字节被符号扩展，`0xff` 变成 `-1`；有符号字节被零扩展，`-1` 变成 `255`。症状常见于解析二进制协议、图像像素、压缩数据、网络包和字符处理。检查方法是找 `movsx` 与 `movzx`，再回到源码类型确认 `char`、`signedchar`、`unsignedchar`、`std::byte`、`std::uint8_t` 是否使用正确。

第二类是比较语义错误。`signed` 与 `unsigned` 混合导致负数变成巨大无符号数，边界检查失效或循环不退出。检查方法是找 `setb/jb/ja` 这类 `unsigned` 条件码，以及源码中 `std::size_t` 和 `int` 的混合。不要看到源码里写了 `i < n` 就假设是 `signed` 比较；类型转换可能已经改变语义。

第三类是溢出错误。外部输入参与大小计算，例如 `width*height*channels`，若在 32 位中溢出，再用于分配内存，就可能产生小缓冲区和大写入。检查方法是确认乘法和加法的类型宽度，确认是否使用显式溢出检查或更宽类型，确认编译器是否有机会假设 `signed` 溢出不发生。机器上看到回绕不等于 C++ 程序合法。

第四类是移位错误。移位计数为负或大于等于类型宽度，在 C++ 中不是普通硬件屏蔽语义；`signed` 负数右移还要注意实现语义和目标假设。检查方法是确认计数来源是否受约束，确认源码是否先做范围检查，确认汇编中是否有 `mask` 或 `branch` 保护。若代码依赖 x86 对移位计数的硬件屏蔽，而源码没有合法化计数，就可能被优化器按未定义行为前提改写。

第五类是地址计算窄化。下标或字节数在 32 位里计算后再扩展到 64 位，可能在大输入下回绕。检查方法是找 `movsxd`、32 位 `imul`、写 `eax/ecx` 后用于地址的模式。地址公式中的 `index` 若来自 32 位结果，要问源码是否本来就限制了范围，还是意外截断了 `std::size_t`。

心智模型

整数问题的诊断顺序是：位宽是否足够，扩展是否正确，比较是否按预期 `signed/unsigned`，溢出是否定义良好，移位计数是否合法，地址计算是否在足够宽度上完成。

把整数指令和内存安全连起来

整数指令本身只处理位模式，但它们经常决定内存安全。数组下标、长度、容量、字节偏移、对齐、对象大小、分配尺寸，最终都要通过整数计算进入地址公式。一个整数语义错误，可能不会在算术位置崩溃，而是在很远的 `load/store` 处表现为越界访问。

例如：

```
extern "C" int load_bad(int const* p, int n, int i)
{
    if (i < n) {
        return p[i];
    }
    return 0;
}
```

这个检查没有保证 `i >= 0`。若 `i` 是 `-1`，`signed i < n` 可能为真，地址公式 `p+i*4` 会访问数组前面的内存。汇编可能清楚显示 `movsxd` 把 `i` 符号扩展到 64 位，再用于 `[rdi+rax*4]`。这里汇编没有错，错的是源码边界条件不完整。

再看无符号混合：

```
extern "C" int load_mixed(int const* p, int i, unsigned n)
{
    if (i < n) {
        return p[i];
    }
    return 0;
}
```

```
}

```

这里比较可能按 unsigned 语义进行，负数 `i` 会转换成很大的无符号值，通常使检查失败。这一次负数可能不会进入 load，但语义已经和许多程序员直觉不同。若后续又把 `i` 转成其他宽度，问题会更复杂。底层教材要训练的不是记住哪个例子安全，而是知道边界检查必须同时覆盖下界、上界、类型转换和地址计算宽度。

整数和内存安全还在分配大小上相遇：

```
std::size_t bytes = count * sizeof(T);
void* p = std::malloc(bytes);

```

若 `count*sizeof(T)` 溢出，分配可能小于实际需要。之后循环按 `count` 写入，会越过缓冲区。汇编里可能只看到一次乘法或移位，再传给 allocator。判断它是否安全，不能只看乘法指令，还要看源码是否检查 `count<=max/sizeof(T)`，或使用了能报告溢出的安全封装。性能优化和安全检查在这里不是对立关系；定义清楚的整数前提，反而让编译器和读者都更容易推理。

范围分析：整数值不是只有当前位模式

读整数汇编时，除了当前寄存器位模式，还要关心值的范围。范围分析回答“这个值可能是多少”。编译器会利用范围信息做优化；程序员也要利用范围信息判断访问是否合法、比较是否冗余、扩展是否安全、溢出是否可能。

例如：

```
extern "C" int load_checked(int const* p, int n, int i)
{
    if (0 <= i && i < n) {
        return p[i];
    }
    return 0;
}

```

进入 load 块时，路径条件给出范围：

```
i >= 0
i < n

```

如果还知道 `n<=capacity` 且 `p` 指向至少 `capacity` 个元素，那么 `p[i]` 是边界内访问。汇编里可能只看到一次 signed compare 或 unsigned trick，但安全性来自路径条件和对象范围共同成立。只看地址公式 `p+i*4` 不够。

无符号范围技巧

编译器常把两个 signed 边界检查合并成一次 unsigned 比较。源码：

```
if (0 <= i && i < n)

```

在某些前提下可以改写成：

```
cmp edi, esi
jae .Lout

```

若 `edi` 是 `i`，`esi` 是 `n`，unsigned `i>=n` 会同时处理负数 `i`：负数按无符号解释为很大的数，自然大于等于非负 `n`，进入 out。这个技巧依赖 `n` 非负或范围已知。报告中看到 unsigned 条件码，不要立刻说源码使用 unsigned；它也可能是编译器用 unsigned 比较实现 signed 下界加上界检查。

这种范围技巧说明条件码和源码类型不是一一对应。分析时要写：

```
machine:
    unsigned compare i and n
possible source meaning:
    range check 0 <= i < n if n is known non-negative
evidence needed:
    earlier path or type proves n >= 0

```

范围信息会让检查消失

如果编译器已经知道某个值范围，后续检查可能被删除。例如：

```
extern "C" int f(unsigned char x)
{
    if (x < 256) {
        return x;
    }
    return -1;
}
```

`unsignedchar` 的值范围已经是 0 到 255，条件永远为真，优化后可能只返回 `x`。这不是编译器忽略检查，而是检查在语言类型下冗余。类似地，数组循环中如果前面已经检查了 `n>0`，循环内部可能不再重复检查。

范围信息也可能来自循环。若循环写成 `for(int i=0; i<n; ++i)`，进入循环体时编译器知道 `i<n`，也知道 `i` 从 0 开始递增。但如果 `i` 可能 `signed` 溢出，语言规则又会影响优化。范围分析、溢出规则和循环优化紧密相关。

整数转换审查：从源类型到目标寄存器

跨函数、跨 ABI、跨内存加载时，整数经常发生转换。审查转换时，按下面链路：

1. 源值的类型和范围是什么。
2. 目标类型的宽度和 `signed/unsigned` 属性是什么。
3. 转换是保值、取模、符号扩展、零扩展还是实现定义。
4. 汇编中对应 `movsx`、`movzx`、写 32 位寄存器还是截断 `store`。
5. 后续按什么语义使用转换结果。

例如读取字节：

```
extern "C" int read_byte(unsigned char const* p)
{
    return *p;
}
```

汇编中应看到零扩展语义。若是：

```
extern "C" int read_sbyte(signed char const* p)
{
    return *p;
}
```

则应看到符号扩展语义。两者都从内存读取 1 字节，但高位填充不同。若后续用结果做数组下标、表查找或比较，扩展错误会变成真实 bug。

截断 store 也属于转换

写入窄类型时，常见汇编只存低位：

```
mov byte ptr [rdi], sil
```

这表示把 `sil` 的低 8 位写到内存。若源码是把 `int` 转成 `std::uint8_t`，这符合取低 8 位的无符号窄化语义。若程序员以为大值会报错，那就是源码语义误解。C++ 普通窄化转换在很多情况下不会自动检查范围。安全代码若需要拒绝大值，应显式检查：

```
if (x < 0 || x > 255) {
    return false;
}
*out = static_cast<std::uint8_t>(x);
```

汇编报告中看到窄 `store`，要问它是有意截断，还是缺少范围检查。

整数乘法和分配大小

内存分配中的整数乘法是安全审查重点。常见模式：

```
std::size_t bytes = count * sizeof(T);
void* p = std::malloc(bytes);
```

如果 `count*sizeof(T)` 溢出，分配到的字节数会变小。后续按 `count` 写入，就会越界。安全写法应检查：

```
if (count > std::numeric_limits<std::size_t>::max() / sizeof(T)) {
    return nullptr;
}
std::size_t bytes = count * sizeof(T);
```

汇编中安全版本通常会多出比较、除法常量或等价边界检查；不安全版本可能只有一次 `lea`、`shl` 或 `imul`。不要把少几条指令自动看成优化好。对于外部输入驱动的大小计算，缺少溢出检查是 `correctness` 风险。

左移不是总能替代乘法

当 `sizeof(T)` 是 2 的幂时，编译器可能用左移计算字节数：

```
shl rdi, 3
```

这相当于乘以 8 的低位结果。但在源码层，如果 `count*8` 需要检查溢出，单纯左移不提供检查。对于无符号类型，回绕定义良好但可能不符合分配意图；对于 `signed` 类型，溢出或非法移位可能是未定义行为。安全接口应使用无符号大小类型并显式检查上界。

从整数汇编审查漏洞模式

许多安全漏洞最终都能在整数汇编中看到模式。

第一种是检查后截断。源码先用宽类型检查某个范围，但随后把值截断成窄类型用于地址或长度。汇编表现为比较发生在 64 位，之后写 32 位寄存器或使用 `mov` 截断，再参与地址计算。要确认检查和使用的宽度一致。

第二种是 `signed/unsigned` 错配。边界检查用 `signed`，长度使用 `unsigned`，或反过来。汇编表现为 `jle/jge` 与 `jb/jae` 混合，或 `movsxd` 与零扩展路径混合。要回到源码确认转换规则。

第三种是乘法溢出后分配。汇编中只有低位乘法或左移，随后调用 `allocator`；后续循环使用原始 `count`。要查是否有上界检查。若没有，外部输入可能构造小分配大写入。

第四种是负数下标。汇编中出现 `movsxd` 把 32 位下标扩展到 64 位，再用于地址；前置检查只检查 `i < n`，没有检查 `i >= 0`。要看是否有 `unsigned` 合并检查或其他路径条件保护。

第五种是移位计数未检查。汇编中移位计数来自 `cl`，源码没有保证范围。x86 硬件会屏蔽计数，但 C++ 源码中计数过大可能未定义。优化器可利用未定义行为前提，因此不能用硬件屏蔽当作语言保证。

整数代码审查清单

审查整数相关 C++ 和汇编时，可以使用下面清单：

1. 所有外部输入进入整数计算时，类型宽度是否足够。
2. `signed` 与 `unsigned` 混合是否有意，是否记录原因。
3. 所有数组下标是否同时检查下界和上界，或有等价 `unsigned` 范围检查。
4. 所有分配大小乘法是否检查溢出。
5. 所有窄化转换是否有范围检查或明确取低位语义。
6. 所有移位计数是否保证在范围内。
7. 所有字节加载是否使用正确的符号扩展或零扩展。
8. 所有 32 位写后用于 64 位地址的值是否符合零扩展预期。
9. 所有显式溢出检测是否把 `flags` 或 `builtin` 结果转成可见错误处理。

10. 汇编条件码是否和源码比较语义一致。

这张清单既适用于安全，也适用于性能。整数前提清楚，优化器才能安全重写；整数前提混乱，性能测量可能只是测到未定义行为或错误结果。

综合案例：一个长度检查为什么仍然越界

考虑下面函数：

```
extern "C" int get_value(const int* p, int len, int index)
{
    if (index < len) {
        return p[index];
    }
    return 0;
}
```

很多初学者会认为它已经检查了边界。整数审查会指出至少三个问题。

第一，没有检查 `index >= 0`。若 `index = -1` 且 `len = 10`，`signed -1 < 10` 为真，程序访问 `p[-1]`。汇编中常见形态是 `movsxd` 把 `index` 符号扩展成 64 位，再用于 `[rdi+rax*4]`。机器忠实执行了源码允许的路径，但 C++ 访问越界。

第二，没有说明 `len` 是否非负。若 `len` 来自外部输入，负数长度本身就表示接口前置条件混乱。更稳的接口可以使用 `std::size_t len`，但这又引入 signed/unsigned 混合问题，需要把 `index` 先检查为非负再转换。

第三，没有证明 `p` 指向至少 `len` 个元素。边界检查只约束 `index` 和 `len` 的关系，不证明对象范围。调用者必须提供有效数组，或接口必须携带 span-like 契约。

安全版本可以写成：

```
extern "C" int get_value_safe(const int* p, int len, int index)
{
    if (p == nullptr || len <= 0) {
        return 0;
    }
    if (index < 0 || index >= len) {
        return 0;
    }
    return p[index];
}
```

优化器可能把两个 `index` 检查合并成 unsigned 范围检查。报告中若看到 unsigned 条件码，不要惊讶；要解释它如何同时排除负数和大于等于 `len` 的值。

从汇编验证修复

修复后应在汇编中看到三类保护之一：

- 显式检查 `index < 0` 和 `index >= len`。
- unsigned 合并范围检查。
- 上层 wrapper 已经保证范围，当前函数只作为内部 unchecked kernel。

如果当前函数是公共入口，却只有 `index < len` 一条 signed 检查，报告应标为边界缺失。若它是内部 kernel，文档必须写清调用者保证 `0 <= index < len`。同一段汇编，在公共 API 和内部 kernel 中风险等级不同。

综合案例：分配大小溢出

考虑：

```
void* make_buffer(std::uint32_t count)
{
    std::uint32_t bytes = count * 16;
    return std::malloc(bytes);
}
```

若 `count` 接近 $2^{32}/16$ 以上，`count*16` 在 32 位中回绕。汇编可能只是：


```
shl edi, 4
jmp malloc
```

这不是高性能优化，而是潜在漏洞。若后续代码按原始 `count` 写入对象，就可能小分配大写入。更安全的写法：

```
void* make_buffer_safe(std::uint32_t count)
{
    constexpr std::uint32_t MAX = UINT32_MAX / 16;
    if (count > MAX) {
        return nullptr;
    }
    std::size_t bytes = static_cast<std::size_t>(count) * 16;
    return std::malloc(bytes);
}
```

汇编里应出现上界比较，并且乘法在足够宽度上完成。这个案例说明，整数指令审查必须看源码意图。少一条比较可能是优化，也可能是缺少安全检查；判断依据是输入是否可信、接口是否声明前置条件、后续是否按原始 `count` 使用内存。

自测清单：整数语义是否过关

完成本章后，应能回答：

1. 寄存器中的位模式为什么本身没有 signed 标签。
2. `movsx` 和 `movzx` 分别对应什么转换。
3. 写 `eax` 后读取 `rax` 为什么高 32 位为 0。
4. CF 和 OF 分别代表什么溢出视角。
5. 同一条 `cmp` 后为什么可以接 signed 或 unsigned 条件码。
6. C++ signed overflow 和硬件回绕为什么不是一回事。
7. 移位计数为什么需要源码范围保证。
8. 分配大小乘法为什么必须审查溢出。
9. 窄化 store 为什么可能是有意取低位，也可能是缺少检查。
10. 地址计算中的下标宽度和扩展方式如何影响内存安全。

深入讲解：整数语义是优化器和安全的共同边界

整数规则看起来像语言细节，但它同时影响优化器和安全。优化器依赖规则删除不可能路径、合并比较、重写循环；安全审查依赖规则判断溢出、截断、越界和错误转换。若程序员和编译器对整数语义理解不同，bug 往往很隐蔽。

例如 signed overflow。程序员可能以为 `int` 加法在 x86 上自然回绕；硬件确实产生低 32 位结果；但 C++ 语义不把 signed overflow 定义为普通回绕。优化器可以假设定义良好程序中它不发生。于是某些检查在源码层看似合理，优化后可能消失。安全代码如果需要回绕，应使用无符号类型或显式定义良好的函数；如果需要报错，应使用 overflow builtin 或手工检查。

无符号整数则相反。它定义为模 2^N 回绕，所以非常适合位掩码、哈希、计数器和底层协议。但无符号也会带来边界陷阱，尤其是和 signed 混合比较。负数转成无符号会变成大数，可能让检查方向反直觉。使用 `std::size_t` 表示长度合理，但下标若来自 signed 输入，应先检查非负再转换。

移位也类似。硬件可能屏蔽移位计数，但 C++ 对越界移位计数有自己的规则。若输入来自外部，必须检查 `0 <= shift < width`。否则测试在某台 x86 上看似正常，优化器仍可能按未定义行为前提重写。

整数语义的工程原则是：外部输入先合法化，内部计算用足够宽度，边界转换显式写出，溢出策略明确选择。不要把硬件偶然表现当作语言合同。

深入讲解：为什么范围信息能提高性能

范围信息不只是安全检查，也能帮助性能。若编译器知道下标非负且小于长度，可以消除重复检查；若知道指针不为空，可以避免保护分支；若知道长度是向量宽度倍数，可以省掉 tail；若知道值在 0

到 255，可以使用窄表或零扩展。

程序员可以通过接口设计提供范围信息。例如把内部 kernel 的前置条件写成 $n > 0$ 且 n 是 4 的倍数，让 wrapper 处理空输入和尾部。这样 kernel 更简单，汇编更可预测。但这要求文档和测试严格，否则调用者一旦违反前置条件，错误会很严重。

另一个例子是把长度和下标都使用同一类型，减少混合比较。若 API 接收 `std::span<T>`，wrapper 可以从 span 得到指针和 `std::size_t` 长度，再在边界处转换给汇编。类型一致减少了隐式转换，也让范围证明更清楚。

性能和安全在这里并不冲突。清晰的范围合同让安全检查集中在边界，让内部热路径更简单；模糊的范围合同则既容易越界，又让编译器保守，导致更多检查和 reload。

课堂讲解：整数审计从“值域”开始

整数 bug 往往不是某条加法或比较本身难懂，而是值域没有写清。一个变量可能来自外部输入、文件长度、网络包字段、容器大小、数组下标、循环计数、字节数或元素数。不同来源有不同合法范围。若一开始不写值域，后面的扩展、截断、乘法、比较和地址计算都会变成猜测。

整数审计的第一步是给每个关键变量写三种范围：接口范围、内部期望范围、机器表示范围。接口范围是调用者可能传入什么；内部期望范围是函数正确执行需要什么；机器表示范围是当前类型能表示什么。比如参数类型是 `intn`，机器表示范围可能很大且包含负数；内部期望可能是 $0 \leq n \leq \text{len}$ ；接口若来自不可信输入，则必须在函数边界检查。

第二步是写单位。很多溢出来自字节和元素混用。数组长度是元素数，分配大小是字节数，指针差可能是元素差，也可能被转换成字节偏移。变量名 `size`、`len`、`count` 若没有单位，审查会很困难。高质量代码和报告应写 `elements`、`bytes`、`stride`、`capacity` 这类明确词。

第三步是检查所有边界转换。signed 到 unsigned、窄整数到宽整数、宽整数到窄整数、元素数到字节数、用户输入到内部下标、容器大小到 `int`，这些都是风险点。转换本身不是错误；隐式、无检查、无注释的转换才危险。

综合案例：长度字段如何变成越界写

考虑一个二进制协议，头部有 16 位长度字段，表示后续 payload 字节数。解析代码把它读入 `std::uint16_t len`，然后为了添加头部开销，计算 `std::uint16_t total = len + 8`，再分配 `total` 字节，最后复制 `len` 字节并写入 8 字节元数据。若 `len` 接近 65535，`len+8` 会在 16 位中回绕，分配很小缓冲区，却复制很大 payload。

在汇编中，这类错误可能表现为窄寄存器加法、零扩展、较小的 `malloc` 参数和后续较大的复制长度。单看指令，`add`、`movzx`、`call memcpy` 都很普通。问题来自值域和单位：总大小计算应该在足够宽度上进行，并检查是否超过最大允许容量。

安全修复不是简单把类型全改成 `std::size_t`。如果协议字段本来是 16 位，读取时保留 16 位语义是合理的；关键是提升到足够宽度后再计算总字节数，并检查上限。修复报告应写清：输入字段范围、最大允许 payload、头部开销、加法是否可能溢出、分配大小和复制大小是否一致。

从汇编找整数风险信号

阅读汇编时，下面信号值得警惕：

- 使用 `movsx` 还是 `movzx` 与源码期望不一致。
- 乘法或移位后直接作为分配大小，没有上界比较。
- 用 32 位寄存器计算字节数，再传给需要 64 位大小的接口。
- 比较使用 `unsigned` 条件码，但输入可能来自 `signed` 参数。
- 下标先参与地址计算，范围检查在后面或不支配访问。
- 窄化 store 后又按宽值使用同一逻辑变量。
- 负数被转换成很大的 `std::size_t` 后参与循环。

这些信号不是自动判错。底层代码有时故意使用回绕、截断和位掩码。审查要看接口合同。如果合同明确要求模 2^N 算术，并且后续使用也按这个合同解释，那么回绕可以是正确设计；如果合同是长度、容量、下标或分配大小，回绕通常需要检查。

整数和 ABI 的交叉点

整数宽度在 ABI 边界尤其重要。C++ 函数声明中的 `int`、`long`、`std::size_t`、指针和枚举类型，会决定调用者如何准备参数以及被调用者如何解释寄存器低位和高位。手写汇编若把 `int` 参数当成 64 位有符号值直接使用，可能读到未定义或不期望的高位。常见稳妥做法是在汇编入口显式使用 32 位寄存器或做符号/零扩展，让语义清楚。

返回值也类似。返回 32 位整数通常写 `eax`，这会清零 `rax` 高 32 位；返回 64 位整数则必须写完整 `rax`。如果声明和汇编实现不一致，调用者会按声明解释返回寄存器，错误可能非常隐蔽。C++ wrapper 和汇编符号之间必须保持签名、宽度和 signedness 文档一致。

结构体中的整数字段更要小心。字段 `offset`、字段宽度和 `padding` 决定汇编 `load/store`。若 C++ 端把字段从 `std::uint32_t` 改成 `std::size_t`，手写汇编仍按 4 字节访问，就会产生 ABI 破坏。公开参数块应使用固定宽度类型，并用 `static_assert` 检查 `offset` 和 `size`。

整数审查清单

任何涉及整数边界的代码，都可以按下面清单审查：

1. 输入来源是否可信，是否需要合法化。
2. 每个长度变量的单位是元素还是字节。
3. `signed` 与 `unsigned` 比较是否符合意图。
4. 乘法、加法、左移是否可能溢出。
5. 分配大小和后续写入大小是否使用同一检查结果。
6. 下标范围检查是否支配数组访问。
7. 窄化转换是否有显式上界检查。
8. 汇编入口是否按声明正确扩展参数。
9. 返回值宽度是否与声明一致。
10. `benchmark` 是否覆盖边界值，而不是只测普通中间值。

这份清单同时服务正确性、安全和性能。范围清楚后，优化器更容易生成紧凑代码；范围不清时，代码要么危险，要么保守。整数不是“小类型细节”，而是系统边界的一部分。

整数测试要覆盖边界而不是只测中间值

整数代码最容易在边界失败。只用普通中间值测试，例如长度 100、下标 3、正数 42，几乎不能证明整数逻辑稳健。测试应覆盖零、一、最大值、最大值减一、负一、类型边界、乘法临界点、对齐临界点、尾部长度和溢出前后。若函数接收外部输入，还要覆盖非法值。

例如分配大小为 `count*sizeof(T)`，就应测试 `count=0`、1、正常值、`max(sizeof(T))`、`max(sizeof(T))+1`。若代码把 `int` 转成 `std::size_t`，就应测试 `-1`。若循环按 4 个元素展开，就应测试长度 0、1、3、4、5、7、8、9。边界测试能暴露尾部、回绕、符号转换和范围检查错误。

汇编层也要覆盖边界。手写汇编可能在普通长度上正确，但在 `n==0` 时仍读取第一个元素；可能在尾部长度不是向量宽度倍数时越界；可能在负长度被解释为巨大无符号数时长时间运行。wrapper 应把非法输入挡在边界，kernel 测试应验证前置条件内的边界。

整数性能优化也要保留语义

整数优化常见做法包括用移位代替乘除、用掩码代替取模、用 `lea` 合并乘加、用查表代替分支、用窄类型减少内存带宽。这些优化只有在语义前提成立时才正确。除以 8 可以变成右移吗？对无符号数和非负有符号数通常容易；对负数，舍入方向可能不同。取模可以变成掩码吗？只有模数是 2 的幂，且语义是无符号或非负范围时才简单。

查表优化也有边界。索引必须范围检查，表项类型要足够宽，默认路径要正确。窄类型优化要考虑溢出和提升规则。把 `int` 中间值改成 `int16_t` 可能减少内存，但如果计算需要更大范围，就会改变结果。性能优化不能把“当前测试数据没溢出”当作语义保证。

因此，整数优化报告应同时包含语义证明和性能证据。语义证明写范围、单位和溢出策略；性能证据写反汇编、输入规模和时间。只给其中一半都不够。

整数错误的跨层传播

整数错误通常不会停留在整数层。一个长度溢出会变成小分配大写入，这是内存错误；一个 signed/unsigned 比较错误会变成控制流绕过，这是路径错误；一个参数宽度不一致会变成 ABI 错误；一个 benchmark 的计数溢出会变成错误吞吐指标；一个数组下标截断会变成错误地址公式。整数是很多底层问题的源头。

因此，排查复杂问题时主动寻找整数边界。崩溃地址很大，可能来自负数转无符号；malloc 分配很小，可能来自乘法回绕；循环次数异常，可能来自窄化；性能指标离谱，可能是 ops 或 bytes 计算溢出。不要把整数章节看成孤立的指令知识，它连接内存、控制流、ABI 和测量。

写给接口的整数合同

接口文档应明确整数合同。例如长度参数是否允许零，是否允许负数，最大值是多少，单位是字节还是元素；stride 是否可以为负；offset 是否必须对齐；返回值是否可能溢出；错误码如何表示。没有这些合同，调用者和实现者会各自猜测。

对于内部 hot kernel，可以把合同写得更严格，例如 $n > 0$ 且 n 是 8 的倍数；wrapper 负责把公共输入拆成符合合同的主循环和尾部。严格合同能简化汇编，但必须有测试防止误用。接口合同越明确，整数风险越低。

整数章节的综合训练

本章可以用一个缓冲区分配函数做综合训练。输入是元素数量和元素大小，输出是分配指针。读者要写三个版本：错误版本、带溢出检查版本、把检查放在 wrapper 并让 kernel 假设范围合法的版本。然后生成汇编，观察比较、乘法、扩展和调用。

报告要说明每个版本的合同、边界输入、汇编差异和测试结果。若安全版本多了几条比较，不要简单说它慢；要说明这些指令换来了什么语义保证。若内部 kernel 去掉检查，也要说明 wrapper 如何保证前置条件。这个训练能把整数、内存、ABI 和性能放到同一个小问题中。

整数报告中的反例表

整数分析报告可以附一张反例表。表中列出输入、期望行为、错误版本行为、安全版本行为、汇编证据。例如 $\text{count} = \text{max}/\text{elem} + 1$ 时，错误版本回绕并分配小缓冲区，安全版本返回错误； $n = -1$ 时，错误版本转成巨大无符号长度，安全版本拒绝； $\text{shift} = \text{width}$ 时，错误版本触发未定义行为，安全版本检查范围。

反例表能防止报告只覆盖顺利输入。整数规则的危险恰恰在边界，边界不列出来，审查者很难知道你是否真的考虑了风险。对安全相关代码，反例表比一段笼统说明更有说服力。

整数知识向第二册的迁移

第二册中的 SIMD、量化、矩阵乘法和 AI 算子会大量使用整数。索引、stride、tile size、字节偏移、对齐、尾部 mask、量化 scale、int8 累加、饱和和回绕，都会回到本章主题。若整数范围和单位不清，高级优化会非常危险。

因此，本章不是只为普通整数表达式服务。它为后续所有高性能 kernel 提供边界语言。你要能说清每个循环计数、每个 offset、每个 byte count、每个 accumulator 的范围，才能安全地写更复杂的优化。

实验：类型扩展指令观察

创建 labs/ch11_integer_instructions/extend.cpp:

```
#include <stdint>

extern "C" int from_u8(std::uint8_t x) { return x; }
extern "C" int from_i8(std::int8_t x) { return x; }
extern "C" int load_u8(std::uint8_t const* p, int i) { return p[i]; }
extern "C" int load_i8(std::int8_t const* p, int i) { return p[i]; }
```

生成汇编：

```
clang++ -std=c++20 -O3 -S -masm=intel extend.cpp -o extend.s
clang++ -std=c++20 -O3 -c extend.cpp -o extend.o
objdump -d -Mintel extend.o > extend.objdump.txt
```

交付物：

1. 找出所有 `movsx`、`movzx`、`movsxd`。
2. 解释每条扩展指令对应的 C++ 类型。
3. 对 `i=-1` 的情况，写出地址公式会如何变化，并说明 C++ 是否允许访问。
4. 记录机器码字节。

整数审查的最后一道问题

审查整数代码时，最后要问一句：如果输入取到边界，这段代码会发生什么。这个问题简单，却能暴露大量缺陷。长度为零是否访问内存，长度为一是否走尾部，长度为最大值是否溢出，负数是否被转换成巨大无符号数，乘法临界点是否被检查，返回值是否能表达错误，都是边界问题。

边界问题也要进入性能测试。很多优化只在普通长度上测过，到了非整倍尾部就走完全不同路径；很多汇编 kernel 在大输入上快，小输入上被固定开销支配；很多量化或累加代码在普通值上正确，在极值上溢出。整数边界不是安全章节的附属内容，而是性能 kernel 的日常条件。

如果读者能在写代码前先写出边界表，写汇编前先写出参数范围，写 benchmark 前先写出输入矩阵，那么本章目标就达到了。第二册中的 SIMD tail、tile size、量化累加和地址偏移，都会继续使用这套边界思维。

实验：lea、乘法和结构体步长

创建 `labs/ch11_integer_instructions/lea_patterns.cpp`：

```
extern "C" int mul3(int x) { return x * 3; }
extern "C" int mul5_add7(int x) { return x * 5 + 7; }

struct S12 {
    int a;
    int b;
    int c;
};

extern "C" int load_s12_c(S12 const* p, int i)
{
    return p[i].c;
}

struct S24 {
    double a;
    double b;
    int c;
    int d;
};

extern "C" int load_s24_d(S24 const* p, int i)
{
    return p[i].d;
}
```

交付物：

1. 找出所有 `lea`。
2. 解释每个 `lea` 是否访问内存。
3. 写出 S12 和 S24 的布局、`sizeof` 和字段 `offset`。
4. 从汇编恢复 `i*12+offset` 和 `i*24+offset`。
5. 对比编译器何时用 `imul`，何时用 `lea`。

实验：位运算和对齐

创建 labs/ch11_integer_instructions/bit_alignment.cpp:

```
#include <stdint>

extern "C" std::uintptr_t align_down_64(std::uintptr_t x)
{
    return x & ~std::uintptr_t(63);
}

extern "C" std::uintptr_t align_up_64(std::uintptr_t x)
{
    return (x + 63) & ~std::uintptr_t(63);
}

extern "C" bool has_flag(unsigned flags, unsigned bit)
{
    return (flags & bit) != 0;
}

extern "C" unsigned set_flag(unsigned flags, unsigned bit)
{
    return flags | bit;
}
```

交付物:

1. 找出 `and`、`or`、`test`。
2. 用二进制解释 `align_down_64(0x1234)` 和 `align_up_64(0x1234)`。
3. 解释 `test` 为什么不保存结果。
4. 说明 64 字节对齐为什么和低 6 位有关。

实验：比较条件码与 signed/unsigned

创建 labs/ch11_integer_instructions/compare_flags.cpp:

```
#include <cstdint>

extern "C" bool less_i32(int a, int b) { return a < b; }
extern "C" bool less_u32(unsigned a, unsigned b) { return a < b; }
extern "C" bool less_mixed(int a, unsigned b) { return a < b; }
extern "C" bool index_before(int i, std::size_t n) { return i < n; }
extern "C" int min_i32(int a, int b) { return a < b ? a : b; }
extern "C" unsigned min_u32(unsigned a, unsigned b) { return a < b ? a : b; }
```

交付物:

1. 找出每个函数中的 `cmp` 或 `test`。
2. 记录后续使用 flags 的 `setcc`、`jcc` 或 `cmovcc`。
3. 区分 `setl/cmovl` 与 `setb/cmovb`。
4. 解释 `less_mixed(-1, 1u)` 的 C++ 结果，并说明汇编条件码是否符合。
5. 对 `index_before` 写出 `int` 与 `std::size_t` 混合比较的转换过程。

实验：除法、余数与常数除法优化

创建 labs/ch11_integer_instructions/division.cpp:

```
#include <stdint>

extern "C" int div_i32(int a, int b) { return a / b; }
extern "C" int mod_i32(int a, int b) { return a % b; }
extern "C" int divmod_sum_i32(int a, int b) { return a / b + a % b; }

extern "C" unsigned div_u32(unsigned a, unsigned b) { return a / b; }
```

```
extern "C" unsigned mod_u32(unsigned a, unsigned b) { return a % b; }

extern "C" int div8_i32(int x) { return x / 8; }
extern "C" unsigned div8_u32(unsigned x) { return x / 8u; }
extern "C" unsigned div10_u32(unsigned x) { return x / 10u; }
```

交付物:

1. 找出 `idiv` 和 `div`, 说明显式操作数和隐式寄存器。
2. 找出 `cdq`、`cqo` 或清零 `edx/rdx` 的指令, 解释它们如何构造被除数高半部分。
3. 对 `divmod_sum_i32` 判断编译器是否复用一次除法结果。
4. 对比 `div8_i32` 和 `div8_u32`, 说明 signed 舍入修正是否出现。
5. 对 `div10_u32` 记录是否出现 magic number、乘法或右移, 并说明为什么这仍然是除法语义。

实验: 溢出检测和部分寄存器

创建 `labs/ch11_integer_instructions/overflow_partial.cpp`:

```
#include <stdint>

extern "C" int add_plain(int a, int b)
{
    return a + b;
}

extern "C" bool add_checked(int a, int b, int* out)
{
    return __builtin_add_overflow(a, b, out);
}

extern "C" bool is_nonzero(int x)
{
    return x != 0;
}

extern "C" int bool_to_int(int x)
{
    return x != 0;
}

extern "C" std::uint64_t write_u32(std::uint64_t x)
{
    std::uint32_t y = static_cast<std::uint32_t>(x);
    return y;
}
```

交付物:

1. 对比 `add_plain` 和 `add_checked`, 说明哪一个需要把 OF 转化成可见结果。
2. 找出 `seto` 或等价溢出检测序列。
3. 找出 `setne al` 这类只写低 8 位的指令。
4. 判断后续是否有 `movzx eax, al` 或其他清高位操作。
5. 解释写 `eax` 为什么会清零 `rax` 高 32 位, 并说明 `write_u32` 的返回值语义。

本章作业

习题与作业

作业 1: 整数指令注释表

从本章实验中选择至少 30 条整数指令, 建立表格:

- 指令文本。
- 分类: move、ALU、shift、bitwise、extension、compare、address calculation。

- 读寄存器/内存。
- 写寄存器/内存。
- 是否更新 flags。
- 对应 C++ 语义。

作业 2：扩展语义推导

给定输入字节：

```
0x00
0x7f
0x80
0xff
```

分别写出：

1. `movzx eax, byte ptr [p]` 的结果。
2. `movsx eax, byte ptr [p]` 的结果。
3. 对应 `std::uint8_t` 和 `std::int8_t` 到 `int` 的 C++ 转换结果。

作业 3：恢复地址公式

对下面汇编恢复结构体布局的可能信息：

```
movsxd rax, esi
lea    rax, [rax + rax*2]
mov    ecx, dword ptr [rdi + rax*8 + 20]
ret
```

要求：

1. 写出元素步长。
2. 写出字段 offset。
3. 设计一个可能的 C++ 结构体。
4. 解释为什么不能只看 `rax*8` 就说元素大小是 8。

作业 4：对齐函数证明

证明下面函数对 2 的幂 A 有效：

```
std::uintptr_t align_up(std::uintptr_t x, std::uintptr_t A)
{
    return (x + A - 1) & ~(A - 1);
}
```

要求用二进制低位解释，不少于 800 字，并举三个例子：

- 已经对齐的地址。
- 只差 1 字节对齐的地址。
- 随机未对齐地址。

作业 5：小型整数汇编报告

写一个 20 行以内 C++ 文件，包含：

- `std::uint8_t` 和 `std::int8_t` load。
- 一个结构体数组字段访问，结构体大小不是 1、2、4、8。
- 一个对齐函数。
- 一个位标志测试函数。

生成 -03 汇编并提交不少于 2500 字报告，必须包括：

- 源码和编译命令。
- 类型和结构体布局表。

- 每个函数的核心汇编。
- 所有地址公式。
- 扩展指令解释。
- flags 相关指令解释。
- 至少 10 条机器码字节记录。

作业 6: signed/unsigned 条件码对照报告

编写一个 30 行以内 C++ 文件，至少包含以下函数：

- `int` 小于比较。
- `unsigned` 小于比较。
- `int` 与 `unsigned` 混合比较。
- `std::size_t` 长度与 `int` 索引比较。
- `signed` 和 `unsigned` 的 `min` 或 `max`。

生成 -O3 汇编，提交不少于 2200 字报告。报告必须列出每个函数的源码类型、usual arithmetic conversions 结果、`cmp` 操作数顺序、后续 `setcc/jcc/cmovcc`，并解释为什么某些函数使用 `signed` 条件码，某些函数使用 `unsigned` 条件码。报告还要用 -1、0、1、`UINT_MAX` 设计至少四组输入，预测 C++ 结果和汇编条件码结果是否一致。

作业 7: 除法强度削弱分析

编写一组除法函数，包含：

- `signed` 变量除以变量。
- `unsigned` 变量除以变量。
- `signed` 除以 2、4、8、16。
- `unsigned` 除以 2、4、8、16。
- `unsigned` 除以 3、5、10、100。

生成 -O3 汇编，提交不少于 2500 字报告。报告必须说明哪些函数使用 `idiv` 或 `div`，哪些函数使用移位，哪些函数出现 magic number 和乘高位模式。对 `signed` 除以 2 的幂，必须解释负数向零截断为什么需要修正；对变量除法，必须说明 `edx:eax` 或 `rdx:rax` 的被除数构造。

作业 8: 溢出检测与 UB 边界

编写一组加法、减法、乘法函数，分别包含普通 `signed` 运算、普通 `unsigned` 运算和 `__builtin_add_overflow`、`__builtin_sub_overflow`、`__builtin_mul_overflow` 检测版本。提交不少于 2200 字报告，要求：

- 区分普通 `signed` 运算的语言语义和硬件低位结果。
- 说明 `unsigned` 回绕为什么定义良好。
- 找出检测版本中 `flags`、`seto`、`setb` 或等价序列。
- 解释为什么“先溢出再检查”不是可靠 C++ 写法。
- 至少举两个真实工程场景，说明整数溢出检查为什么影响内存安全或数据正确性。

作业 9: 部分寄存器读写表

从本章实验和你自己补充的例子里收集至少 20 条涉及 `al`、`ax`、`eax`、`rax` 或其他同类寄存器别名的指令，建立表格。每一行必须写出：实际写入宽度、是否清高位、后续读取宽度、是否需要 `movzx/movsx` 补充扩展、对应源码返回类型或表达式类型。最后用不少于 1200 字解释为什么现代编译器经常偏爱 32 位写入。

验收标准

完成本章后，你应该能做到：

- 区分位模式、`signed` 解释和 `unsigned` 解释。

- 解释 `mov`、`load`、`store` 的差异。
- 解释 `xor reg, reg` 清零和 32 位写清零高位。
- 读懂 `add`、`sub`、`imul`、`lea` 的基本语义。
- 解释 `movsx`、`movzx`、`movsxd` 对应的 C++ 类型转换。
- 区分 `shr` 和 `sar`。
- 用 `and`、`or`、`test` 解释 `mask`、`flag` 和 `alignment`。
- 看到 `cmp` 和 `test` 时知道真正输出是 `flags`。
- 区分 `jl/setl/cmovl` 和 `jb/setb/cmovb`。
- 解释 `signed/unsigned` 混合比较为什么可能转成 `unsigned` 条件码。
- 解释 `CF` 和 `OF` 分别回答什么问题。
- 说明普通 `signed` 溢出、`unsigned` 回绕和显式溢出检测的差异。
- 读懂 `div/ldiv` 的隐式寄存器、商和余数。
- 解释 `signed` 除以 2 的幂为什么可能需要偏置修正。
- 识别除以常数时的 `magic number`、乘法和移位模式。
- 解释部分寄存器写入与 32 位写清高位规则。
- 从 `lea` 组合恢复非 2 的幂结构体步长。
- 初步判断整数指令对性能的影响：寄存器 `ALU`、内存 `RMW`、`flags` 依赖、乘法和地址计算。

如果这些能力稳定，本册后面的控制流和 `ABI` 会更容易进入；第二册的 `loop optimization`、`alias/UB`、低精度量化和 `AVX microkernel` 也会有更好的汇编基础。

控制流：跳转、循环、调用与 switch

问题与目标

前两章已经建立了单条整数指令、寄存器、内存操作数、地址公式和 flags 的阅读方法。本章把这些材料组合成程序的执行路径。只有能解释路径选择，才能从“逐行看懂指令”进入“看懂程序行为”。

一个 C++ 程序不会从第一行机械地直线执行到最后一行。它会判断、跳转、循环、调用函数、返回调用点，也会通过函数指针、虚函数或跳转表进入运行时才确定的目标。机器层面并不存在高级语法中的 `if`、`while`、`switch`、`return`；机器真正维护的是下一条取指地址的选择：

```
normal instruction:
    rip = address_of_next_instruction

branch/call/ret:
    rip = some_other_address
```

`rip` 表示下一条要取指的地址。只要能够解释一段代码在什么条件下改变 `rip`、改变为哪个地址、为什么选择那个地址，就能把控制流从表面语法还原到机器行为。

本章目标是让你完成下面几件事：

- 把 C++ 的 `if/else`、条件表达式、循环、`switch` 和函数调用还原成基本块和边。
- 理解 `cmp`、`test` 如何生产 flags，`jcc`、`setcc`、`cmovcc` 如何消费 flags。
- 区分 `signed` 比较和 `unsigned` 比较使用的条件码。
- 能画出一段汇编的控制流图。
- 能解释循环中的入口、回边、退出条件、归纳变量和地址递推。
- 能看懂 `switch` 的比较链和跳转表。
- 能用具体地址解释 `call` 写入返回地址、`ret` 取回返回地址。
- 初步理解分支预测、间接跳转和返回预测为什么影响现代 CPU 性能。

核心思想

控制流的本质是对 `rip` 的选择。顺序执行是默认选择；条件跳转、无条件跳转、调用、返回和间接跳转，都是在明确改变下一条指令地址。

控制流位于哪一层

控制流同时存在于多个层次。源代码层有 `if`、`for`、`while`、`switch`、`return`、短路逻辑和函数调用；编译器中间层有基本块、边、支配关系、循环、`phi` 节点和控制依赖；机器层有 `cmp`、`test`、`jcc`、`jmp`、`call`、`ret` 以及 `rip`；微架构层还有分支预测、目标预测、取指队列、错误路径清空和返回地址预测。

学习时必须分清这些层次。C++ 源码中的一条 `if` 不一定对应机器码里的一条条件跳转。编译器可能把条件选择改写成 `cmovcc`，可能把布尔结果改写成 `setcc`，可能把小的 `switch` 改写成比较链，也可能把密集 `switch` 改写成跳转表或数据表。反过来，机器码中的一条 `jcc` 也不一定直接对应源码里的显式 `if`，它可能来自循环边界检查、数组范围保护、异常路径、空指针防御、编译器生成的运行时检查，或者链接器与安全机制插入的控制流。

本章的训练目标不是把每条跳转机械翻译回一行 C++，而是恢复控制流结构。你要能回答：有哪些基本块？每个块在什么条件下到达？块尾选择了哪条边？循环的回边在哪里？退出条件是什么？哪些指令生产 flags，哪些指令消费 flags？哪些选择改变 `rip`，哪些选择只改变数据？只有这些问题清楚，后面看 ABI、性能测量、分支预测、向量化和热路径报告才有基础。

心智模型

读控制流有三张图：

- 源码图：程序员写下的语法结构。
- CFG 图：基本块和边执行的执行可能性。
- 机器图：rip、flags、跳转目标和返回地址形成的实际路径。

三张图相关，但不能直接等同。

从源代码到基本块

先看一个很普通的 C++ 函数：

```
extern "C" int classify(int x)
{
    if (x < 0) {
        return -1;
    }
    if (x == 0) {
        return 0;
    }
    return 1;
}
```

源代码有三条返回路径。编译器降低它时，会先把代码切成若干 **basic block**（基本块）。基本块是一段只有一个入口、一个出口的连续指令序列。进入基本块之后，中间不会跳走；要跳走只能发生在块尾。

控制流图可以写成：

```
entry:
    test x
    if x < 0 goto neg
    goto check_zero

neg:
    return -1

check_zero:
    test x
    if x == 0 goto zero
    goto pos

zero:
    return 0

pos:
    return 1
```

真实编译器可能使用分支，也可能用 **setcc** 或 **cmovcc** 生成无分支代码。学习时不要预设“这段 C++ 必须长成某种汇编”。更可靠的训练方式是：

1. 找出基本块。
2. 找出每个基本块最后如何选择下一块。
3. 找出每条边的条件。
4. 找出每个块里产生或使用的数据。
5. 再分析编译器是否把控制流改写成数据流。

深入理解

编译器中间表示通常会显式表达基本块、边和控制依赖。后面讲 LLVM IR 时，你会看到 **br**、**switch**、**phi**。汇编阶段虽然没有 **phi**，但基本块和边仍然存在，只是隐藏在标签、跳转和寄存器赋值里。

为什么基本块是控制流分析的基本单位

基本块不是教材为了画图而发明的装饰概念，而是编译器、反汇编人员和性能工程师共同使用的最小结构单位。原因在于：如果一段指令中间没有其他入口，也没有提前出口，那么只要执行进入这段指令的第一条，后面的指令就会按顺序执行到块尾。这样，块内部主要是数据流问题，块与块之间才是控制流问题。

这种分离非常重要。分析块内部时，你可以追踪寄存器定义、内存访问、flags 生产和普通算术依赖；分析块尾时，你再判断 `rip` 会去哪个后继块。如果不切分基本块，所有指令混在一起，条件跳转、fallthrough、循环回边、早返回和跳转表会互相干扰，报告很快变成逐行翻译，无法证明程序结构。

编译器优化也依赖基本块。局部公共子表达式消除、死代码删除、寄存器分配里的活跃性传播、循环识别、分支概率估计、热冷分离、代码布局，都要先知道基本块和边。即使你暂时不写编译器，也应该用同一种结构读汇编。这样后面进入 LLVM IR、SSA 和优化器章节时，概念不会重新开始。

基本块还有一个实用价值：它能让你发现不完整分析。一个块如果有两个入口，说明切分错了；一个条件跳转如果只画了一条边，说明漏掉 fallthrough；一个循环如果没有回边，说明你可能只看到了展开后的部分代码或把 latch 误删了。教材式报告应该允许别人根据你的块表复查路径，而不是只能相信你的文字结论。

基本块切分的规则

手工切分基本块时，可以采用一个机械规则。第一，函数入口是基本块入口。第二，任何跳转目标标签都是基本块入口。第三，任何条件跳转、无条件跳转、`ret`、可能不返回的调用之后，下一条指令若仍可到达，也应作为新基本块入口。第四，一个基本块在遇到跳转、返回或函数结束时结束。

这个规则比凭缩进和标签更可靠。汇编标签不一定都代表源代码分支，有些标签只是调试、对齐或跳转表辅助；没有显式标签的位置也可能是基本块入口，例如条件跳转的 fallthrough 目标就是下一条指令。反汇编工具有时会把标签命名成 `.LBB0_2`、`<foo+0x17>` 或地址偏移，名字不同不影响基本块含义。

切分之后，每个块应只有一个入口。如果一段指令中间可以从别处跳入，那么它就不能和前面的指令属于同一个基本块。每个块也应只有一个控制出口：顺序落入下一个块、条件跳转形成两条边、无条件跳转形成一条边、`ret` 结束当前函数控制流、`call` 通常不是块出口，除非调用目标被证明不返回。

一个常见错误是把 `call` 一律当成控制流终点。普通 `call` 会临时转到被调用函数，但它预期会返回到下一条指令，所以在当前函数的 CFG 中，`call` 通常属于基本块内部指令。只有调用 `abort`、`exit`、抛出后不返回的函数，或者编译器知道 `[[noreturn]]` 时，调用才可能成为当前函数路径的终点。

fallthrough 是一条边

条件跳转有两条边：taken edge 和 fallthrough edge。taken edge 是条件满足时跳到目标标签；fallthrough edge 是条件不满足时顺序进入下一条指令所在块。初学者容易只画跳转箭头，忘记 fallthrough。这样画出来的 CFG 会少一条执行路径，后续数据流分析也会错。

例如：

```
test edi, edi
jle .Lelse
lea eax, [rsi+rsi]
ret
.Lelse:
lea eax, [rsi+3]
ret
```

`jle .Lelse` 的 taken edge 表示 `x<=0` 时进入 else；fallthrough edge 表示 `x>0` 时继续执行 then。源代码写的是 `if(x>0)`，但汇编用反条件跳走。这种布局很常见，因为编译器会把更适合顺序执行、代码布局或预测的路径放在 fallthrough 上。

fallthrough 对性能也有意义。顺序路径通常有更好的取指连续性，不需要跳转目标预测；但现代 CPU 会预测 taken 分支并提前取目标，所以不能简单说 fallthrough 永远更快。工程分析时要结合输入分布、代码布局 and 实际计数器。对本章而言，先把两条边画全，比急着讨论哪条快更重要。

核心思想

画 CFG 时，每个条件跳转都至少问两个问题：跳转条件成立时去哪里？条件不成立时顺序落到哪里？漏掉 fallthrough，就漏掉了一条真实路径。

合流点：多条路径在哪里重新汇合

控制流不只会分叉，也会汇合。一个 `if/else` 之后继续执行公共代码时，`then` 路径和 `else` 路径会在某个块重新汇合。这个汇合块常被称为 `join block`。读汇编时，合流点和分叉点同样重要，因为返回值、临时变量和寄存器活跃性经常在这里发生变化。

看一个简单例子：

```
extern "C" int choose_then_add(int x, int y)
{
    int v;
    if (x > 0) {
        v = y + 1;
    } else {
        v = y - 1;
    }
    return v * 3;
}
```

一种低优化形态可以写成：

```
test edi, edi
jle .Lelse
lea eax, [rsi + 1]
jmp .Ljoin
.Lelse:
lea eax, [rsi - 1]
.Ljoin:
lea eax, [rax + rax*2]
ret
```

这里 `.Ljoin` 就是合流点。进入 `.Ljoin` 时，`eax` 必须已经保存变量 `v`。`then` 路径把 `y+1` 放进 `eax`，`else` 路径把 `y-1` 放进 `eax`，合流点之后的代码不再关心它来自哪条路径，只依赖“`eax` 是 `v`”这个事实。

编译器中间表示用 SSA 时，会用 `phi` 节点描述这种“根据前驱边选择值”的语义。伪形式如下：

```
join:
v = phi(then: y + 1, else: y - 1)
return v * 3
```

汇编里没有显式 `phi` 指令。编译器会通过寄存器放置、`move` 插入、栈槽、`cmovcc` 或重排控制流来实现同样效果。因此，看到多条路径汇合时，要检查每条前驱边进入合流点前是否为后续代码准备了同一组寄存器或内存状态。

心智模型

分叉点回答“下一步走哪条边”；合流点回答“不同路径带来的值如何统一”。没有合流点思维，就很难读懂优化后的返回值、循环变量和 SSA 降低结果。

支配关系的直觉

控制流图里还有一个常用概念：如果每条从入口到某个块的路径都必须经过块 `A`，就说 `A` 支配这个块。第一册不要求形式化算法，但直觉很有用。函数入口支配所有可达块；循环 `preheader` 通常支配循环体；某个 `null check` 如果支配后续 `load`，就说明所有到达 `load` 的路径都已经检查过指针。

这个概念能帮助你判断编译器是否真的保留了保护条件。比如：

```
if (p != nullptr) {
    return *p;
}
return 0;
```

如果汇编中 load 所在块只可能从 `p!=nullptr` 的 fallthrough 或 taken 路径到达，那么检查支配了 load。若某个优化把 load 移到检查之前，就要问语言语义是否允许提前访问。对普通指针解引用而言，空指针路径不能执行 load，所以这种移动通常不合法。用支配关系描述，比笼统说“应该先判断”更精确。

循环里也一样。若结束指针在 preheader 里计算，且 preheader 支配循环体，那么每次进入循环时结束指针都已经可用。若某个寄存器只在某条分支里定义，却在合流后的所有路径使用，就要检查另一条路径是否也定义了它；否则可能是你漏读了初始化，或者这段路径实际上不可达。

RIP：CPU 正在执行哪里

从 ISA 视角看，CPU 有一个架构状态：通用寄存器、flags、内存可见状态和 `rip`。普通指令完成后，`rip` 指向下一条指令：

address	bytes	instruction
0x1000	89 f8	mov eax, edi
0x1002	01 f0	add eax, esi
0x1004	c3	ret

执行过程：

当前 rip	执行指令	下一步 rip
0x1000	moveax,edi	0x1002
0x1002	addeax,esi	0x1004
0x1004	ret	从栈顶取出的返回地址

为什么 0x1000 后面是 0x1002？因为 `mov eax, edi` 的机器码长度是 2 字节。x86 是变长指令集，一条指令可以是 1 字节，也可以更长。取指和解码前端的复杂性，一部分就来自“下一条指令地址不是固定加 4”。

分支指令则不同。比如：

0x2000:	85 ff	test edi, edi
0x2002:	7e 08	jle 0x200c
0x2004:	8d 04 36	lea eax, [rsi + rsi]
0x2007:	83 c0 01	add eax, 1
0x200a:	c3	ret
0x200c:	8d 46 03	lea eax, [rsi + 3]
0x200f:	c3	ret

`jle` 的条件为真时，下一条 `rip` 是 0x200c；条件为假时，下一条 `rip` 是顺序地址 0x2004。这就是控制流分叉。

心智模型

读控制流时，把每条跳转都看成一次对 `rip` 的赋值：

```
if condition:
    rip = target
else:
    rip = next_instruction
```

Flags 回顾

上一章已经见过 flags。本章要把它们放进控制流里使用。x86-64 的很多条件跳转不是直接读取 C++ 的布尔变量，而是读取前面指令设置的 flags。

cmp 的本质

Intel 语法：

cmp destination, source

语义近似是：

```
temporary = destination - source
set flags according to temporary
discard temporary
```

例如：

```
cmp edi, esi
jg .Lgreater
```

这表示先按 **edi-esi** 设置 flags，然后 **jg** 判断 signed 意义下 **edi>esi** 是否成立。
最常用 flags：

flag	名称	典型含义
ZF	zero flag	结果为 0；常用于相等判断
SF	sign flag	结果最高位为 1；常用于 signed 判断的一部分
CF	carry flag	unsigned 加法进位或减法借位
OF	overflow flag	signed 加减溢出
PF	parity flag	低 8 位奇偶；现代 C++ 性能代码较少直接关心

test 的本质

test 做按位与，只设置 flags，不保存结果：

```
test edi, edi
je .Lzero
```

语义近似：

```
temporary = edi & edi
set flags according to temporary
discard temporary
```

因为 **x&x==x**，所以 **test edi, edi** 常用于判断 **x==0**、**x<0** 或 **x>0**。它不会像 **cmp edi, 0** 那样编码一个立即数 0，机器码通常更短。

```
test edi, edi
je zero      ; ZF=1, x == 0
js negative  ; SF=1, x < 0 in signed interpretation
jne nonzero  ; ZF=0, x != 0
```

常见误区

flags 是隐式状态。很多指令会写 flags，很多条件指令会读 flags。手写汇编时，如果在 **cmp** 和 **jcc** 中间插入会改 flags 的指令，条件判断就会改变。编译器会维护这种依赖；人写汇编必须自己负责。

flags 的生命周期

flags 没有变量名，也很少在源码层显式出现。它们像一组隐藏寄存器，由某条指令写入，再被后面的条件指令读取。读汇编时必须建立“flags 定义到 flags 使用”的数据流，而不是只看 **jcc** 本身。

最简单情况是相邻的 **cmp** 和 **jcc**：

```
cmp edi, esi
jl .Lless
```

这里 flags 的生产者明显是 **cmp edi, esi**，消费者是 **jl**。但真实编译器可能在二者之间插入不改 flags 的指令，例如某些 **lea**、**mov**：

```
cmp edi, esi
lea eax, [rdi + rsi]
jl .lless
```

此时 `jl` 仍然读取 `cmp` 设置的 `flags`，因为 `lea` 不改 `flags`。相反，如果中间插入 `add`、`sub`、`test`、`and` 等会改 `flags` 的指令，后面的 `jcc` 读取的就不是原来那次比较结果。

这也是为什么 `lea` 在地址计算和算术优化里很有用。它能计算加法、乘以小常数和地址表达式，但不破坏 `flags`。编译器有时故意选择 `lea` 而不是 `add`，就是为了在保持 `flags` 可用的同时准备某个数据值。

读条件码的四步

看到条件跳转时，建议按四步推导。第一，找到最近的有效 `flags` 生产者，注意跳过不写 `flags` 的指令，也要警惕中间写 `flags` 的指令。第二，判断生产者的逻辑：`cmp a, b` 相当于计算 `a-b` 设置 `flags`；`test a, a` 相当于按位与后判断零和符号；`add`、`sub` 等算术指令也会设置 `flags`。第三，判断消费者使用 `signed`、`unsigned`、相等、非零还是符号位条件。第四，把机器条件翻译成当前程序里的语义，并写清楚操作数顺序。

操作数顺序尤其容易出错。Intel 语法 `cmpedi, esi` 是用 `edi-esi` 设置 `flags`；若后面是 `jl`，含义是 `signed edi<esi`。如果你把它误读成 `esi<edi`，分支方向就反了。AT&T 语法的操作数顺序又不同，所以本书统一使用 Intel 语法，减少不必要混乱。

常见误区

不要只看到 `jl` 就写“跳到小于”。必须写清楚“谁小于谁”，以及这是 `signed` 还是 `unsigned`。完整表述应类似：在 `cmpedi, esi` 之后，`jl` 表示 `signed edi<esi`。

条件跳转

`jcc` 是一族条件跳转。这里的 `cc` 是 condition code，例如 `je`、`jne`、`jl`、`jg`、`ja`、`jb`。

基本形态：

```
cmp a, b
jcc target
```

含义：

```
compute flags for a - b
if condition(flags):
    rip = target
else:
    rip = next_instruction
```

相等和不相等

相等判断只需要看 `ZF`：

指令	条件	常见 C++ 关系
<code>je</code> 或 <code>jz</code>	<code>ZF==1</code>	<code>a==b</code>
<code>jne</code> 或 <code>jnz</code>	<code>ZF==0</code>	<code>a!=b</code>

signed 比较

`signed` 比较需要考虑 `SF` 和 `OF`：

指令	读 <code>flags</code> 的条件	<code>cmpa, b</code> 后的含义
<code>jl</code> 或 <code>jnge</code>	<code>SF!=0F</code>	<code>signed a<b</code>
<code>jle</code> 或 <code>jng</code>	<code>ZF==1 SF!=0F</code>	<code>signed a<=b</code>
<code>jg</code> 或 <code>jnl</code>	<code>ZF==0 && SF==0F</code>	<code>signed a>b</code>
<code>jge</code> 或 <code>jnl</code>	<code>SF==0F</code>	<code>signed a>=b</code>

unsigned 比较

unsigned 比较主要看 CF 和 ZF：

指令	读 flags 的条件	cmp a, b 后的含义
jb 或 jnae	CF==1	unsigned $a < b$
jbe 或 jna	CF==1 ZF==1	unsigned $a \leq b$
ja 或 jnbe	CF==0 && ZF==0	unsigned $a > b$
jae 或 jnb	CF==0	unsigned $a \geq b$

核心理想

同一个 cmp 之后，接 signed 条件跳转还是 unsigned 条件跳转，会得到不同语义。硬件没有问变量类型；类型信息已经体现在编译器选择了哪条条件指令。

signed 和 unsigned 的分叉

下面两个函数源代码几乎一样，但类型不同：

```
extern "C" int less_i32(int a, int b)
{
    return a < b;
}

extern "C" int less_u32(unsigned a, unsigned b)
{
    return a < b;
}
```

可能汇编：

```
less_i32:
    xor eax, eax
    cmp edi, esi
    setl al
    ret

less_u32:
    xor eax, eax
    cmp edi, esi
    setb al
    ret
```

两段都用 `cmp edi, esi`，但是 signed 版本使用 `setl`，unsigned 版本使用 `setb`。这不是小细节，而是语义核心。

设：

```
edi = 0xffffffff
esi = 0x00000001
```

解释：

解释方式	edi 的值	edi<esi
std::int32_t	-1	true
std::uint32_t	4294967295	false

所以：

```
cmp edi, esi
setl al    ; signed: -1 < 1, true
setb al    ; unsigned: 4294967295 < 1, false
```

常见误区

很多性能 bug 和安全 bug 都来自 signed/unsigned 混用。看到 **jl**、**jb** 时要想到 signed；看到 **jb**、**ja** 时要想到 unsigned。尤其是数组长度、下标、**size_t**、负数检查混在一起时，要非常警惕。

if/else 的降低

考虑函数：

```
extern "C" int branchy(int x, int y)
{
    if (x > 0) {
        return y * 2 + 1;
    } else {
        return y + 3;
    }
}
```

为了教学，我们先看一种有显式分支的写法：

```
branchy:
    test edi, edi
    jle .Lelse
    lea eax, [rsi + rsi]
    add eax, 1
    ret
.Lelse:
    lea eax, [rsi + 3]
    ret
```

控制流图：

```
entry:
    test x
    if x <= 0 goto else
    goto then

then:
    eax = y * 2 + 1
    return

else:
    eax = y + 3
    return
```

注意编译器经常会把源代码条件取反。源代码写的是 $x > 0$ ，汇编却可能写 **jle .Lelse**。这不是语义反了，而是把不满足 then 的情况跳到 else，让 then 作为 fallthrough。

逐条解释一次：

```
entry:
    edi = x
    esi = y

test edi, edi
    flags = flags_of(x & x)

jle .Lelse
    if signed x <= 0:
        rip = .Lelse
    else:
        rip = next instruction

lea eax, [rsi + rsi]
    eax = y * 2
    flags unchanged

add eax, 1
    eax = y * 2 + 1
```

```

    flags updated, but no later use here

ret
    return eax

.Lelse:
lea eax, [rsi + 3]
    eax = y + 3

ret

```

带机器码地址的观察

反汇编里可能看到类似：

```

0000000000000000 <branchy>:
0: 85 ff          test    edi, edi
2: 7e 08          jle     c <branchy+0xc>
4: 8d 04 36       lea     eax, [rsi+rsi]
7: 83 c0 01       add     eax, 0x1
a: c3            ret
c: 8d 46 03       lea     eax, [rsi+0x3]
f: c3            ret

```

`jle` 的机器码 `7e08` 使用 8 位相对位移。位移不是从当前指令开头算，而是从下一条指令地址算：

```

jle address      = 0x0002
jle length       = 2
next instruction = 0x0004
encoded offset   = 0x08
target           = 0x0004 + 0x08 = 0x000c

```

这就是 `c<branchy+0xc>` 的来源。

深入理解

x86 条件跳转既可以使用短位移，也可以使用更长的相对位移。汇编器会根据目标距离选择编码。链接和重定位还可能调整最终地址。理解“相对下一条指令”的规则，对读机器码、二进制补丁和反汇编非常重要。

setcc：把条件变成 0 或 1

C++ 经常需要返回布尔值：

```

extern "C" int is_positive(int x)
{
    return x > 0;
}

```

可能汇编：

```

is_positive:
xor  eax, eax
test edi, edi
setg al
ret

```

`setg al` 表示如果 `signed x > 0`，就把 `al` 写成 1，否则写成 0。由于 `setcc` 只写 8 位目标，前面常见 `xor eax, eax`，先把整个 `eax` 清零，避免高位残留：

```

xor eax, eax
    eax = 0

test edi, edi
    flags = flags_of(x)

setg al

```



```
if x > 0:
    al = 1
else:
    al = 0
eax is now 0 or 1
```

常见 `setcc`:

指令	条件	常见含义
<code>sete</code>	<code>ZF==1</code>	等于
<code>setne</code>	<code>ZF==0</code>	不等于
<code>setl</code>	signed 小于	<code>a<b</code> for signed
<code>setg</code>	signed 大于	<code>a>b</code> for signed
<code>setb</code>	unsigned 小于	<code>a<b</code> for unsigned
<code>seta</code>	unsigned 大于	<code>a>b</code> for unsigned

`cmovcc`: 把控制选择变成数据选择

分支会改变 `rip`。有时候编译器不想改变控制流，而是先算出两个候选值，然后用 `cmovcc` 选择一个。
C++:

```
extern "C" int min_i32(int a, int b)
{
    return a < b ? a : b;
}
```

可能汇编:

```
min_i32:
    mov eax, esi
    cmp edi, esi
    cmovl eax, edi
    ret
```

逐条解释:

```
entry:
    edi = a
    esi = b

    mov eax, esi
    result candidate = b

    cmp edi, esi
    flags = flags_of(a - b)

    cmovl eax, edi
    if signed a < b:
        eax = a
    else:
        eax remains b

    ret
```

`cmovcc` 不改变 `rip`，但它读取 `flags`，并且目标寄存器会依赖条件和两个候选值。它适合分支难预测、两边计算都便宜的情况。

什么时候不要盲目追求 `branchless`

无分支不等于一定更快。考虑:

```
extern "C" int maybe_load(int cond, int const* p, int fallback)
{
    if (cond) {
        return *p;
    }
}
```

```
    return fallback;
}
```

这类代码不能简单改成“先 load 再 cmov”，因为 `cond==0` 时 `p` 可能为空或无效。C++ 语义只允许在条件为真时访问 `*p`。机器硬件可以执行 load，但语言层面和异常行为都不允许你随便提前访问。

性能判断也要看代价：

- 分支高度可预测时，普通分支可能非常快。
- 分支接近随机时，错误预测代价可能很高，`cmovcc` 可能更好。
- `cmovcc` 会保留数据依赖，可能拉长关键路径。
- 如果两边计算很重，`branchless` 可能让本来不需要执行的一边也产生代价。
- 如果一边包含潜在非法内存访问，不能用简单数据选择替代控制选择。

核心思想

分支优化不是“消灭所有分支”。正确问题是：这条分支是否可预测？错误预测代价多大？两边计算是否便宜？数据依赖是否变长？语言语义是否允许提前计算？

短路逻辑和早返回

C++ 的 `&&` 和 `||` 不是普通的按位运算。它们有短路语义：对 `a&&b`，如果 `a` 为假，`b` 不会求值；对 `a||b`，如果 `a` 为真，`b` 不会求值。这意味着它们天然包含控制流，尤其当右侧表达式有函数调用、内存访问或可能触发未定义行为时，编译器不能随意提前执行。

看一个边界明确的例子：

```
extern "C" int safe_read(int const* p, int n)
{
    if (p != nullptr && n > 0) {
        return p[0];
    }
    return 0;
}
```

一种可能控制流是：

```
test rdi, rdi
je .Lzero
test esi, esi
jle .Lzero
mov eax, dword ptr [rdi]
ret
.Lzero:
xor eax, eax
ret
```

这里不能先无条件执行 `moveax,[rdi]` 再根据条件选择返回值，因为当 `p==nullptr` 时，源码语义并不会访问 `p[0]`。短路保护的是右侧求值本身，不只是最终结果。这个例子也说明 `branchless` 改写必须服从语言语义，不能只看数学等价。

早返回也会形成多个出口块：

```
extern "C" int clamp_positive(int x)
{
    if (x < 0) {
        return 0;
    }
    if (x > 100) {
        return 100;
    }
    return x;
}
```

汇编可能有多个 `ret`，也可能把多个路径合并到一个共同尾声。多个 `ret` 不代表函数逻辑混乱，

只说明编译器认为直接返回更短。共同尾声则常见于需要恢复栈、弹出 callee-saved 寄存器或执行统一清理的函数。读汇编时要区分“源码有几个 return”和“机器码有几个 ret”，二者不必相同。

核心思想

短路逻辑是一种控制依赖。只要右侧表达式可能访问内存、调用函数、产生异常或触发未定义行为，就不能把它当成普通数据表达式随便提前计算。

控制依赖和数据依赖

控制流分析里有两个常用词：控制依赖和数据依赖。若某条指令是否执行取决于前面的条件跳转，它就处在控制依赖之下。若某条指令的输入值来自前面指令的输出，它就有数据依赖。编译器优化经常在这两者之间转换。

前面的 `min_i32` 可以用分支写：

```

cmp edi, esi
jge .Luse_b
mov eax, edi
ret
.Luse_b:
mov eax, esi
ret

```

也可以用 `cmovl` 写:

```
mov eax, esi
cmp edi, esi
cmovl eax, edi
ret
```

第一种形式中，`moveax,edi` 是否执行受控制依赖影响；第二种形式中，`cmovl` 总会执行，但 `eax` 的最终值依赖 `flags` 和两个候选数据。控制依赖减少了不必要计算，但可能带来分支预测风险；数据依赖消除了分支，但可能拉长关键路径。

这一区分对性能阅读很重要。一个看起来“没有分支”的版本，并不等于没有代价。它可能让本来不需要的计算变成每次都做，也可能让后续指令必须等待条件和两个数据源都准备好。相反，一个有分支的版本，如果输入分布稳定、预测准确，可能更快也更省功耗。

深入理解

编译器是否选择分支、`setcc`、`cmovcc` 或查表，取决于目标 CPU、优化级别、分支概率信息、两边计算成本、异常和内存安全语义。源码层的三目运算符并不强制生成 `cmovcc`。

循环的机器结构

循环不是一个神秘结构。它由入口判断、循环体、回边和退出边组成。

C++:

```
extern "C" int sum_i32(int const* p, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        s += p[i];
    }
    return s;
}
```

一种容易教学的汇编：

```
sum_i32:
    xor eax, eax
    test esi, esi
    jle .Ldone
    xor ecx, ecx
.Lloop:
```

```

    add eax, dword ptr [rdi + rcx*4]
    inc rcx
    cmp ecx, esi
    jl .Lloop
.Ldone:
    ret

```

这里假设 `rdi` 是 `p`，`esi` 是 `n`，`eax` 是累加器，`ecx/rcx` 是循环下标。
控制流图：

```

entry:
    s = 0
    if n <= 0 goto done
    i = 0
    goto loop

loop:
    s += p[i]
    i += 1
    if i < n goto loop
    goto done

done:
    return s

```

回边是：

```
jl .Lloop
```

它让 `rip` 回到更小地址处的循环体开头。后面讲分支预测时会看到，循环回边往往非常可预测：大多数迭代跳回，最后一次不跳。

地址和循环变量一起推导

对循环体核心指令：

```
add eax, dword ptr [rdi + rcx*4]
```

推导：

```

rdi = p
rcx = i
element width = 4 bytes

address = p + i * 4
load    = *(int*)(p + i * 4)
eax     = s + load

```

每轮迭代：

迭代	<code>rcx</code>	访问地址	语义
0	0	<code>p+0</code>	<code>s+=p[0]</code>
1	1	<code>p+4</code>	<code>s+=p[1]</code>
2	2	<code>p+8</code>	<code>s+=p[2]</code>
k	k	<code>p+4*k</code>	<code>s+=p[k]</code>

指针递增形式

编译器也可能不用显式下标，而是让指针前进：

```

sum_i32_ptr:
    xor eax, eax
    test esi, esi
    jle .Ldone
    movsxd rsi, esi
    lea rdx, [rdi + rsi*4]
.Lloop:

```

```

add eax, dword ptr [rdi]
add rdi, 4
cmp rdi, rdx
jne .Lloop
.Ldone:
ret

```

这里：

```

rdi = current pointer
rdx = end pointer = p + n * 4

loop:
    s += *current
    current += 4
    continue while current != end

```

这和源代码的 `i < n` 语义等价，但汇编里看不到 `i`。你要恢复的是数据流和地址递推，而不是强行找源代码变量名。

常见误区

不要看到 `cmp rdi, rdx` 就以为源代码一定写了两个指针比较。它也可能来自数组下标循环，编译器把下标形式优化成了指针递增形式。

循环的三个问题

读循环时，不要先问“这是 `for` 还是 `while`”。机器层面更重要的是三个问题。

第一，循环是否可能执行零次。若入口处先判断 `n <= 0` 并跳到 `done`，那么循环体可能一次不执行；若直接进入循环体，条件在尾部检查，那么至少执行一次。这个问题关系到空数组、边界输入和 benchmark 中小规模输入的行为。

第二，每次迭代改变什么状态。常见状态包括累加器、归纳变量、当前指针、剩余计数、循环内临时值。只要能写出每轮迭代前后的状态变化，就能判断循环是否等价于源码，而不需要强行找回源代码变量名。

第三，退出条件依赖什么。退出条件可能依赖计数器，也可能依赖指针到达 `end`，也可能依赖数据内容，例如扫描到零字节、找到匹配元素、遇到错误码。计数型循环通常更容易预测和向量化；数据依赖退出的循环更难批量处理，因为每一轮是否继续取决于刚读到的数据。

心智模型

循环可以写成四个槽位：

```

preheader:
    prepare invariants and initial state

header:
    decide whether to enter/continue

body:
    do useful work and update state

latch:
    jump back to header or exit

```

真实汇编可能合并这些槽位，但分析时可以按这个模型恢复。

preheader、header、latch 和 exit

编译器内部经常把循环拆成几个角色：`preheader` 是进入循环前只执行一次的块，`header` 是判断是否继续或承载循环入口的块，`body` 是主要工作，`latch` 是更新状态并跳回 `header` 的块，`exit` 是离开循环后的块。真实汇编可能把 `header` 和 `body` 合并，也可能把 `latch` 和 `body` 合并，但这些角色仍然存在。

为什么要有这些名字？因为它们回答不同问题。`preheader` 适合放循环不变量，例如结束指针、常量倍数、`mask`、表地址；`header` 适合放入口条件和从上一轮带来的状态；`body` 放每轮真正工作；

latch 放归纳变量更新和回边判断；exit 放返回值整理或后续代码。若报告只说“这里是循环”，就无法说明某条 lea 是每轮执行还是只执行一次，也无法判断优化是否减少了循环体成本。

以指针递增求和为例，lea rdx, [rdi + rsi*4] 在 preheader 里计算 end pointer。如果它在循环体里，每轮都要重新计算结束地址；移到 preheader 后，每轮只需要比较当前指针和 rdx。这就是循环不变量外提的基本形态。对性能来说，一条指令是否在循环内，影响会被迭代次数放大；对正确性来说，它是否支配循环体，也决定循环体能否安全使用它。

exit 也不能忽略。很多循环离开后还要处理 tail、返回累加器、恢复寄存器、处理没有进入循环的空输入，或者进入第二个清理循环。优化后的代码常常有多个 exit：一个处理 $n \leq 0$ ，一个处理主循环结束，一个处理提前找到目标。画 CFG 时要把每个 exit 分开标注，否则会把不同边界条件混成一个结论。

归纳变量替换：源码的 i 可能消失

归纳变量是循环优化的核心。源码里写的是 i，汇编里可能改成当前指针、剩余计数、偏移字节数，甚至同时维护多个等价变量。编译器这样做不是为了让人难读，而是为了减少每轮计算、适配寻址模式、方便向量化或方便退出判断。

一个下标循环：

```
for (int i = 0; i < n; ++i) {
    s += p[i];
}
```

可以用三种等价机器状态表达：

形态	每轮状态	退出判断
下标形态	$i=i+1$	$i < n$
指针形态	$cur=cur+4$	$cur \neq end$
剩余计数形态	$count=count-1$	$count \neq 0$

这三种形态都可能来自同一段源码。读者要学会证明它们等价。下标形态里地址是 $base+i*4$ ；指针形态里 cur 初始为 base，每轮加 4，因此第 k 轮 $cur=base+k*4$ ；剩余计数形态里可能同时维护当前指针和剩余元素数，退出时 count 归零。证明时写出初始值、每轮递推和退出条件，就能恢复源码含义。

归纳变量替换也会改变条件码。源码 $i < n$ 是 signed 比较，但如果编译器先证明 $n > 0$ 且 i 从 0 递增，就可能使用指针不等或 unsigned 范围检查。不要看到 jne .Lloop 就说源码写了 while($p \neq end$)。更准确的说法是：优化后循环使用当前指针和结束指针表达原来的计数边界。

循环退出证明：为什么不会多读一个元素

读循环汇编时，必须证明每次 load 都在合法迭代范围内。对于前面的下标版本，入口处 test esi, esi; jle .Ldone 证明 $n \leq 0$ 时不进入循环。进入循环后初始 $i=0$ ，每轮先读 $p[i]$ ，再执行 $i+=1$ ，然后用 $i < n$ 决定是否回边。这样最后一次读取发生在旧 $i=n-1$ 时，更新后 $i=n$ ，比较失败退出，没有读取 $p[n]$ 。

对于指针版本，preheader 计算 $end=p+n*4$ 。循环中先读 $*cur$ ，再执行 $cur+=4$ ，然后比较 $cur \neq end$ 。若初始 $n > 0$ ，第 0 次读 p，第 k 次读 $p+4k$ ，当读完 $p+4(n-1)$ 后，cur 加到 end，比较失败退出。这个证明比“看起来像数组循环”更可靠。

边界输入要单独检查。若 $n=0$ ，入口判断必须绕过循环；若 $n < 0$ ，源码里的循环条件 $i < n$ 对初始 $i=0$ 为假，也应绕过循环。若编译器使用 std::size_t 或 unsigned 长度，负数不再是可能值，但混合类型转换可能引入其他边界。高质量报告应至少覆盖零长度、单元素和最大常见长度三个场景。

常见误区

循环报告不能只写“回到.Lloop”。必须证明入口条件、每轮更新和退出判断共同保证访问范围正确。尤其是优化成指针递增后，更要说明当前指针和结束指针如何对应原始下标。

循环合并块和循环携带依赖

循环本身也是一种合流结构。每次回边回到 header 时，某些值来自第一次进入循环前，另一些值来自上一轮迭代后的 latch。SSA 里会用 **phi** 表达这种关系，例如：

```
header:
    i = phi(entry: 0, latch: i_next)
    s = phi(entry: 0, latch: s_next)
```

汇编里没有 **phi**，但你能看到寄存器扮演同样角色。**ecx** 初始为 0，回边后保存下一轮的 **i**；**eax** 初始为 0，回边后保存下一轮的累加和 **s**。这类值叫循环携带值，因为它们从一轮迭代传到下一轮。

循环携带依赖会影响性能。累加器 **s+=p[i]** 形成一条跨迭代依赖链：第 **k+1** 轮的 **s** 依赖第 **k** 轮加法结果。简单计数器也有依赖，但通常便宜。后续讲 ILP、展开和向量化时，会看到编译器通过多个累加器、循环展开或向量归约来缩短这种依赖链。第一册只要求你能在汇编里指出哪些寄存器跨迭代保留状态。

循环不变量

loop invariant（循环不变量）是循环执行期间不变化的值或计算。编译器会尽量把不变量移到循环外，减少每次迭代成本。例如数组基址、元素宽度、结束指针、某个常量系数，都可能在循环前准备好。

在指针递增版本里：

```
movsxd rsi, esi
lea rdx, [rdi + rsi*4]
```

rdx 是 end pointer，在循环中保持不变。循环体只递增 **rdi**，每轮比较当前指针和 end pointer。这个转换把每轮的 **i<n** 和 **p+i*4** 关系改写成“当前指针是否到达终点”。对机器来说，这可能减少地址计算压力，也让循环状态更直接。

读优化汇编时，要主动寻找 preheader 中的这些准备指令。很多看似不属于循环的 **lea**、**movsxd**、**shl**，其实是在构造循环不变量。如果只看标签之间的循环体，会漏掉循环正确性的前提。

break 和 continue

break 和 **continue** 会给循环增加额外边。看例子：

```
extern "C" int first_positive_sum_until_zero(int const* p, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        int v = p[i];
        if (v == 0) {
            break;
        }
        if (v < 0) {
            continue;
        }
        s += v;
    }
    return s;
}
```

CFG 中至少有这些边：

```
entry -> loop_header
loop_header -> done      if i >= n
loop_header -> body      if i < n
body -> done             if v == 0 ; break
body -> latch            if v < 0 ; continue
body -> add_positive      otherwise
add_positive -> latch
latch -> loop_header
```

break 的目标是循环外的 **done**，**continue** 的目标通常是 **latch** 或 **step** 位置，然后再回到条件判断。优化后标签名字可能不明显，但边的语义仍然存在。画 CFG 时，如果只找一条回边和一条退

出边，会漏掉这类多出口循环。

这类循环也更难优化。它的数据内容决定退出和跳过路径，分支预测依赖输入分布；向量化时还要处理提前停止和掩码选择。第二册讨论 SIMD 和扫描类算法时，会反复回到这些控制流结构。

循环形态：while、do-while 和 for

高级语言有不同循环语法，机器层面差异主要在第一次检查条件的位置。

while 循环

C++:

```
extern "C" int count_down(int n)
{
    int s = 0;
    while (n > 0) {
        s += n;
        --n;
    }
    return s;
}
```

结构：

```
entry:
    s = 0
    if n <= 0 goto done

loop:
    s += n
    n -= 1
    if n > 0 goto loop

done:
    return s
```

while 是先判断，再执行。若初始条件不成立，循环体一次都不执行。

do-while 循环

C++:

```
extern "C" int count_down_do(int n)
{
    int s = 0;
    do {
        s += n;
        --n;
    } while (n > 0);
    return s;
}
```

结构：

```
entry:
    s = 0

loop:
    s += n
    n -= 1
    if n > 0 goto loop

done:
    return s
```

do-while 至少执行一次。汇编里常见特征是循环体前面没有入口条件跳转，条件跳转在尾部。

for 循环

for(init;cond;step) 常见结构：

```

init
if not cond goto done

loop:
    body
    step
    if cond goto loop

done:

```

编译器会根据优化目标重排。比如把第一次条件判断放到循环尾，或者把不变量挪到循环外。读汇编时不要纠结源代码写法，先识别基本块和边。

归纳变量和循环优化预告

induction variable（归纳变量）是每次迭代按固定规律变化的变量。最常见：

```

i = i + 1
ptr = ptr + 4
offset = offset + stride

```

在 HPC 和 AI 算子里，归纳变量非常重要，因为它决定内存访问模式、向量化条件、预取距离和 cache line 利用率。

考虑：

```

extern "C" void axpy(float* y, float const* x, float a, int n)
{
    for (int i = 0; i < n; ++i) {
        y[i] += a * x[i];
    }
}

```

标量汇编的核心形态可能是：

```

.Lloop:
    movss xmm1, dword ptr [rsi + rcx*4]
    mulss xmm1, xmm0
    addss xmm1, dword ptr [rdi + rcx*4]
    movss dword ptr [rdi + rcx*4], xmm1
    inc rcx
    cmp ecx, edx
    jl .Lloop

```

控制流只是循环；性能核心则在数据流：

```

x address = x + i * 4
y address = y + i * 4
two loads + one store per element
one multiply + one add per element
branch once per scalar iteration

```

后面讲 SIMD 时，这个循环会变成每轮处理多个元素，回边次数减少，地址递增步长变大，tail 处理成为新问题。本章先把标量控制流看清楚。

switch：比较链和跳转表

switch 的降低取决于 case 值是否密集、数量是否多、每个分支体大小和编译器策略。

比较链

C++：

```

extern "C" int small_switch(int x)
{
    switch (x) {
        case 1: return 10;
        case 7: return 70;
    }
}

```

```

        default: return -1;
    }
}

```

可能汇编：

```

small_switch:
    cmp edi, 1
    je .Lcase1
    cmp edi, 7
    je .Lcase7
    mov eax, -1
    ret
.Lcase1:
    mov eax, 10
    ret
.Lcase7:
    mov eax, 70
    ret

```

控制流就是多次比较加条件跳转。case 稀疏时，这种形式很常见。

跳转表

case 密集时，编译器可能生成跳转表：

```

extern "C" int dense_switch(int x)
{
    switch (x) {
        case 0: return 3;
        case 1: return 5;
        case 2: return 8;
        case 3: return 13;
        case 4: return 21;
        default: return -1;
    }
}

```

如果每个 case 只是返回常量，编译器可能生成数据表而不是代码跳转表。为了看清跳转表，可以让每个分支调用不同函数。教学形式如下：

```

dense_switch:
    cmp edi, 4
    ja .Ldefault
    mov eax, edi
    jmp qword ptr [rip + .LJTI0_0 + rax*8]

.Lcase0:
    mov eax, 3
    ret
.Lcase1:
    mov eax, 5
    ret
.Lcase2:
    mov eax, 8
    ret
.Lcase3:
    mov eax, 13
    ret
.Lcase4:
    mov eax, 21
    ret
.Ldefault:
    mov eax, -1
    ret

.LJTI0_0:
    .quad .Lcase0
    .quad .Lcase1
    .quad .Lcase2
    .quad .Lcase3
    .quad .Lcase4

```


分解：

```
cmp edi, 4
ja .Ldefault
    unsigned if x > 4 goto default

mov eax, edi
    rax = zero-extended x

jmp qword ptr [rip + table + rax*8]
    target_address = *(table + x * 8)
    rip = target_address
```

为什么使用 `ja` 而不是 `jg`? 因为这里常见技巧是用 `unsigned` 范围检查同时处理负数：

```
x = -1 as int32 bits = 0xffffffff
as unsigned          = 4294967295
4294967295 > 4       = true
```

所以 $x < 0$ 和 $x > 4$ 都会跳到 `default`。

核心思想

跳转表把多路选择变成一次范围检查、一次表访问、一次间接跳转。它减少比较次数，但引入间接分支和额外取表访问。它是性能优化、二进制分析和安全缓解都会关心的结构。

编译器为什么有时不用跳转表

初学者常以为 `switch` 就应该生成跳转表。实际并不一定。编译器会综合考虑 `case` 的数量、取值密度、范围大小、分支体复杂度、是否能把结果折叠成数据表、目标平台的间接分支成本和代码大小。

如果 `case` 值很稀疏，例如 `1`、`1000`、`1000000`，直接建立从最小值到最大值的跳转表会浪费大量空间。此时比较链、二分比较树或哈希式策略可能更合理。如果每个 `case` 只是返回常量，编译器可能生成常量表：先做范围检查，再从表里 `load` 返回值，而不是跳到不同代码块。这样控制流更少，取表访问变成普通数据访问。

如果目标平台对间接分支预测很敏感，或者启用了某些安全缓解，跳转表的收益也可能下降。现代系统为了防御间接分支攻击，可能引入 `retpoline`、`IBT`、`CFI` 等机制；这些机制会改变间接跳转和间接调用的成本模型。第一册不展开安全缓解细节，但必须知道：跳转表不是无条件更快，它是在比较次数、代码大小、预测难度和安全约束之间做取舍。

default 路径是正确性的边界

跳转表之前通常必须有范围检查。没有范围检查，错误的 `x` 会把表外内存当成跳转目标，控制流直接失控。对于 `switch(x)` 中合法 `case` 为 `0..4` 的例子，`cmp edi, 4` 和 `ja .Ldefault` 不只是优化前奏，而是安全边界。

如果 `case` 从非零基准开始，编译器常先做归一化：

```
sub edi, 10
cmp edi, 4
ja .Ldefault
jmp qword ptr [rip + table + rdi*8]
```

这对应源代码 `case 10..14`。减去 10 后，合法范围变成 `0..4`，才能直接作为表下标。读这种汇编时，要把归一化前的源码值和归一化后的表索引分开。

常见误区

跳转表分析必须包含范围检查。只看到 `jmp [table + index*8]` 就开始列 `case`，容易漏掉负数、越界值和 `default` 路径。

数据表不是跳转表

`switch` 的另一个常见降低方式是数据表。若每个 `case` 只是返回常量，编译器没有必要跳到多个代码块再返回；它可以先做范围检查，然后从常量数组里 `load` 结果。

```
extern "C" int fib_case(int x)
{
    switch (x) {
        case 0: return 3;
        case 1: return 5;
        case 2: return 8;
        case 3: return 13;
        case 4: return 21;
        default: return -1;
    }
}
```

可能变成：

```
cmp edi, 4
ja .ldefault
mov eax, dword ptr [rip + table + rdi*4]
ret
.ldefault:
mov eax, -1
ret
```

这里表项宽度是 4 字节，表里放的是返回值，不是代码地址。控制流只有范围检查和 `default` 分支，没有间接跳转。若把它误判成跳转表，就会错误地寻找不存在的 `case` 标签和间接分支成本。

区分数据表和跳转表的方法很直接：看表项被当作什么使用。若表项被 `load` 到普通返回寄存器或参与计算，它是数据；若表项被 `load` 后写入 `rip`，或直接出现在 `jmp qword ptr [table + index*8]` 这样的控制转移里，它才是跳转目标。表项宽度也给线索：代码地址在 x86-64 上通常是 8 字节，`int` 常量表通常是 4 字节。

有时编译器还会生成“混合表”：先从表里取偏移，再加上基址形成目标；或者表里不是绝对地址，而是相对偏移。这和 PIE、重定位和代码模型有关。分析时不要只凭 `.quad` 或 `.long` 下结论，要看使用点。

稀疏 switch：比较链、二分树和位测试

`case` 稀疏时，比较链不一定按源码顺序排列。编译器可能按概率、数值范围或二分搜索形状组织比较。例如 `case` 为 1、100、1000、10000 时，线性比较四次不是唯一选择；编译器可能先比较 `x<1000`，再在左右两边继续比较。这时 CFG 更像一棵比较树。

比较树的好处是减少最坏比较次数，但会改变 `fallthrough` 和 `taken` 路径。若 `profile` 信息显示某个 `case` 特别常见，编译器也可能把它提前，让热路径更短。读这种汇编时，不要用源码 `case` 顺序判断逻辑。应当把每条比较产生的范围约束写在边上，例如“进入右子树意味着 `x>=1000`”，再继续细化。

某些小范围稀疏集合还可能用位测试。例如合法 `case` 是 0、2、5，编译器可能先检查范围，再用一个常量 `mask` 判断对应 `bit` 是否为 1。这种形式看起来像位运算而不是 `switch`，但语义仍然是集合成员测试。它适合 `case` 范围不大、分支结果可合并的场景。

教材在第一册不要求穷尽所有降低策略，但要建立一个原则：`switch` 是多路选择的语义，机器实现可以是比较链、比较树、跳转表、数据表、位测试，甚至被完全代数化。报告应描述你实际观察到的结构，而不是套用“`switch` 等于跳转表”的模板。

跳转表和重定位

目标文件阶段的跳转表常常不能包含最终运行时地址，因为函数和代码段还没有被链接器放到最终位置。于是表项会带有重定位记录。链接器根据最终布局修正表项，动态链接器还可能在程序加载时继续处理 PIE 或共享库中的相对地址。

这解释了实验里为什么要求同时看 `objdump-dr` 和 `readelf-r`。`objdump-d` 能看到指令和表附近的数据，`-r` 能把相关重定位显示出来；`readelf-r` 则列出目标文件里的重定位项。若只看反汇编，可能看到 `0x0`、占位偏移或看似奇怪的表项，于是误以为跳转目标是空地址。实际上，这些值要等链

接时才完整。

PIE 下还常见相对寻址和相对表项。代码不把绝对地址写死，而是通过 `rip` 相对地址找到表，再把表中偏移加到基址得到目标。这样同一份代码可以加载到不同虚拟地址。读这种形式时，要分清三层：指令里到表的位移、表项里到 case 的位移、运行时实际加载基址。

核心思想

跳转表是代码地址或代码偏移的数据结构。目标文件里看到的表项可能需要重定位；PIE 中的表项可能是相对偏移。分析跳转表必须把汇编、表内容和重定位记录一起看。

跳转表的地址推导

设：

```
rip at jmp next instruction base = 0x401050
.LJTI0_0 runtime address         = 0x402000
rax                             = 3
entry width                     = 8 bytes
```

则：

```
entry_address = 0x402000 + 3 * 8
               = 0x402018

target = *(uint64_t*)0x402018
        = address of .Lcase3

rip = target
```

反汇编时，`rip+displacement` 里的 `rip` 通常指下一条指令地址，不是当前指令开头地址。这一点和前面的相对跳转一致。

无条件跳转和尾调用

`jmp` 直接改变 `rip`：

```
jmp .Ltarget
```

语义：

```
rip = .Ltarget
```

它不写返回地址。和 `call` 的区别非常关键。
一个函数末尾直接返回另一个函数结果：

```
extern "C" int callee(int);

extern "C" int wrapper(int x)
{
    return callee(x);
}
```

优化后可能是：

```
wrapper:
    jmp callee
```

这叫 **tail call**（尾调用）或 `tail-call optimization`。因为 `wrapper` 调用 `callee` 后没有额外工作，编译器可以复用调用者给 `wrapper` 的返回地址。这样 `callee` 返回时直接回到 `wrapper` 的调用者。

控制流：

```
caller -> wrapper -> callee -> caller
```

尾调用优化后：

```
caller -> wrapper
        jmp callee
callee -> caller
```

尾调用成立的边界

尾调用不是“函数最后一行有调用”就一定发生。编译器必须确认当前函数在目标函数返回后没有任何工作要做，并且目标调用可以复用当前调用帧。若调用后还要调整返回值、执行析构、释放动态资源、恢复某些栈状态、处理异常清理，或者调用约定不兼容，就不能简单改成 `jmp`。

C++ 里对象生命周期尤其重要。下面函数看似最后返回被调函数结果，但如果局部对象有析构函数，调用返回后还必须执行析构，就不再是简单尾调用：

```
extern "C" int callee(int);

int wrapper_with_cleanup(int x)
{
    SomeGuard g;
    return callee(x);
}
```

这里 `g` 的析构是语言语义的一部分。即使源码最后一行是 `return callee(x);`，编译器也要保证离开函数前执行清理。优化器可能通过内联或其他方式改变形态，但不能丢掉析构语义。读汇编时，如果没有看到尾调用，不要立即认为编译器能力不足，先检查函数是否还有隐藏清理工作。

尾调用还受 ABI 约束。参数数量、栈上传参、栈对齐、返回类型、调用约定、可变参数函数、异常处理元数据，都可能影响能否复用当前栈帧。下一章讲 ABI 后，这些限制会更清楚。本章先记住：`jmp callee` 作为尾调用，意味着当前函数不再需要自己的返回点，目标函数将直接返回到更上层调用者。

常见误区

看到函数末尾的 `jmp`，要判断它是普通跳转、错误路径合并，还是尾调用。尾调用没有压入新的返回地址，调用栈形状会和普通 `call` 不同。

call 和 ret

`call` 做两件事：

1. 把返回地址压入栈。
2. 把 `rip` 改成被调用函数入口地址。

`ret` 做相反动作：

1. 从栈顶取出返回地址。
2. `rsp` 增加 8。
3. 把 `rip` 改成取出的返回地址。

以具体地址说明。假设：

```
main:
0x401120: e8 5b 00 00 00    call 0x401180 <f>
0x401125: 83 c0 01          add eax, 1

f:
0x401180: 8d 04 3f          lea eax, [rdi+rdi]
0x401183: c3               ret
```

执行 `call` 前：

```
rip = 0x401120
rsp = 0x7fffffffcd90
```

执行 `call 0x401180` 后：

```
rsp = 0x7fffffffcd88
[rsi] = 0x401125
rip = 0x401180
```

返回地址是 0x401125，也就是 `call` 的下一条指令地址。函数 `f` 执行 `ret` 时：

```
target = *(uint64_t*)rsp = 0x401125
rsp = rsp + 8
rip = target
```

于是执行回到 `main` 里的 `add eax, 1`。

指令	栈动作	rip 动作
<code>call target</code>	<code>pushnext_rip</code>	<code>rip=target</code>
<code>ret</code>	<code>poptarget</code>	<code>rip=target</code>
<code>jmp target</code>	无返回地址动作	<code>rip=target</code>

常见误区

`ret` 信任栈顶的返回地址。如果栈上的返回地址被破坏，控制流就会跳到错误位置。这是理解栈溢出、ROP、控制流完整性和影子栈等安全机制的基础。

call 在当前函数 CFG 中怎么画

普通 `call` 会把控制流临时交给被调用函数，但如果它会返回，那么当前函数的下一条指令仍然是后继执行点。因此，在当前函数的 intraprocedural CFG 中，普通 `call` 通常画在基本块内部，而不是块尾的终止指令。块尾仍然由后面的 `jcc`、`jmp`、`ret` 或函数末尾决定。

这和跨函数调用图不同。调用图会画出 `caller->callee` 的边，用来表示函数之间可能调用关系；当前函数 CFG 则关注一个函数内部基本块如何互相到达。两张图解决的问题不同。把 `call` 当作 CFG 终点，会导致你误以为调用后的代码不可达，进而漏掉返回值使用、错误处理和后续分支。

但是，有些调用确实不返回。例如 `abort`、`exit`、某些抛异常后不回到当前路径的 `helper`，或者被编译器标记为 `[[noreturn]]` 的函数。对这类调用，当前函数的正常控制流在 `call` 处结束，下一条指令可能只是对齐填充、冷路径或不可达保护。反汇编里有时会在不返回调用后看到 `ud2`，用于表示不可达陷阱。

因此，报告里应写清调用类别：普通返回调用、不返回调用、可能抛异常调用、尾调用、间接调用。第一册不展开完整异常控制流，但必须知道 C++ 异常会引入隐藏边：调用可能正常返回，也可能转到异常处理和栈展开路径。若你只看普通 `ret` 路径，复杂 C++ 函数的真实控制流会被低估。

返回地址是控制数据

返回地址不是抽象概念，它是一段可读写内存里的 8 字节数据。函数调用嵌套越深，栈上保存的返回地址、旧帧指针、局部变量、临时溢出槽就越多。只要某次越界写覆盖了返回地址，后续 `ret` 就会把错误数据当成代码地址。

从程序正确性角度，返回地址损坏常表现为“函数 `return` 时崩溃”，但根因往往在更早的位置。比如局部数组越界写、错误的 `memcpy` 长度、手写汇编没有恢复 `rsp`、调用约定不匹配、把数据指针当函数指针调用，都可能让控制流在 `ret` 或间接 `call` 处失控。

从系统安全角度，返回地址是攻击和防御的核心对象。栈 `canary` 试图在返回前发现栈帧被覆盖；不可执行栈阻止把栈上数据当代码执行；ASLR 让代码地址难预测；CET shadow stack 用硬件维护一份受保护返回地址副本；CFI 检查间接控制流目标是否属于合法集合。这些机制各自复杂，本册只要求理解它们保护的是“控制数据不能被普通数据写坏”这一条主线。

call 不是普通 jmp

`call` 和 `jmp` 都会改变 `rip`，但它们的契约完全不同。`jmp` 只是跳走，不留下返回路径；`call` 会把下一条指令地址写入栈，建立一个预期的返回点。因此，`call` 同时属于控制流机制和函数调用协议的一部分。

这一区别解释了尾调用优化为什么可以用 `jmp`。如果当前函数已经不需要自己的栈帧，不需要在被调用函数返回后继续做事，那么它可以把控制流直接交给目标函数，让目标函数最终使用当前

栈上的返回地址回到更上层调用者。反过来，如果当前函数在调用后还要加一、恢复寄存器、释放栈空间或处理返回值，就不能简单替换成 `jmp`。

`call` 还和 ABI 紧密相连。执行 `call` 前，调用者必须按 ABI 放好参数并维护栈对齐；执行 `call` 后，被调用者入口看到的是返回地址位于 `rsp` 指向的位置，参数位于寄存器或返回地址上方的栈槽。下一章会系统讲这些规则；本章先抓住控制流事实：返回地址是调用点下一条指令地址，它是 `ret` 能回来的原因。

ret 的目标来自数据

`ret` 看起来像“返回到调用者”，但机器层面它只是从栈顶取一个地址写入 `rip`。这个地址通常由最近的 `call` 写入，但硬件不会验证它一定来自合法调用。只要栈顶被覆盖，`ret` 就会跳到被覆盖后的地址。

这让 `ret` 同时具有两面性。正常程序利用它实现函数嵌套和返回预测；攻击者可能利用栈破坏构造异常返回路径；防御机制则尝试验证返回地址，例如栈 canary、影子栈、控制流完整性和硬件 CET。作为第一册基础，读者至少要知道：返回地址是普通内存中的控制数据，保护它是系统安全的重要主题。

调试崩溃时，如果 `ret` 跳到无效地址，不要只看 `ret` 本身。真正的破坏通常发生在更早的位置：栈缓冲区越界写、错误的 `rsp` 恢复、push/pop 数量不匹配、手写汇编误弹参数、调用约定不匹配。要沿着栈状态回溯，找出返回地址何时被写坏。

直接调用和间接调用

直接调用目标在指令里：

```
call foo
```

间接调用目标来自寄存器或内存：

```
call rax
call qword ptr [rdi + 16]
```

C++ 函数指针：

```
using Fn = int (*)(int);

extern "C" int call_fn(Fn f, int x)
{
    return f(x) + 1;
}
```

可能汇编：

```
call_fn:
    push rbx
    mov  rbx, rdi
    mov  edi, esi
    call rbx
    add  eax, 1
    pop  rbx
    ret
```

这里 `rdi` 初始是函数指针，`esi` 是参数 `x`。调用前要把 `x` 放进第一个参数寄存器 `edi`，再 `call rbx`。

间接调用在 C++ 中还来自：

- 函数指针。
- 虚函数调用。
- `std::function` 或回调封装。
- 动态链接过程中的 PLT/GOT。
- JIT 生成代码或插件系统。

性能上，间接调用目标不固定，分支预测更难。安全上，间接调用目标需要控制流完整性保护。

后面讲分支预测、动态链接和生产库 dispatch 时会继续深入。

函数指针和虚函数的控制流形状

函数指针调用的目标来自数据值。这个数据值可能是参数、结构体字段、全局表项或运行时选择结果。对 CFG 来说，间接调用不是一条指向固定函数的边，而是一组可能目标。静态分析工具可能只能给出保守集合，运行时 profiler 则能记录实际热点目标。

虚函数调用通常还多一层内存读取。对象指针指向对象，对象中有虚表指针，虚表中某个槽位保存目标函数地址。机器层可能类似：

```
mov rax, qword ptr [rdi]      ; load vptr
call qword ptr [rax + offset] ; call virtual function slot
```

这不是“虚函数一定很慢”的证明，而是说明它的控制流目标要到运行时才能从对象动态类型中读出。若编译器能证明动态类型，可能去虚化，把间接调用变成直接调用甚至内联；若不能证明，就要保留间接调用。性能上要看目标稳定性、内联机会、对象布局 and 实际热点，不要用一句口号判断。

PLT/GOT 也是间接控制流

动态链接中的外部函数调用可能经过 PLT 和 GOT。源码里写 `printf`，反汇编里可能先跳到 PLT stub，再通过 GOT 中的地址跳到真实 libc 实现。第一次调用还可能触发动态链接器解析符号，之后 GOT 项被更新，后续调用路径变短。

这说明控制流不只由当前函数源码决定，还受链接方式、PIE、符号可见性、延迟绑定和共享库加载影响。做性能测量时，第一次调用和热身后的调用可能不是同一条路径；做二进制分析时，看到 `call printf@plt` 不代表目标函数体就在当前二进制中。

核心理想

直接调用的目标写在指令或重定位里；间接调用的目标来自寄存器或内存。目标越依赖运行时数据，越需要关注预测、内联机会、安全检查和动态链接路径。

现代 CPU 如何处理分支

ISA 语义很简单：条件满足就跳，不满足就不跳。但现代 CPU 为了性能不会慢慢等条件算完才取下一条指令。它会预测。

一个简化执行过程：

```
front-end:
  fetch bytes at predicted rip
  decode instructions

branch predictor:
  predict taken/not taken
  predict target address

back-end:
  execute compare/test
  resolve actual branch direction

retire:
  if prediction was correct:
    commit work
  else:
    flush wrong-path work
    restart from correct rip
```

常见预测结构包括：

- 方向预测：这次分支会不会跳。
- BTB：branch target buffer，预测跳转目标地址。
- RSB：return stack buffer，预测 `ret` 返回到哪里。
- 间接分支预测：预测 `jmp rax` 或 `call rax` 的目标。

这解释了为什么控制流会影响性能：

- 可预测分支通常成本低。
- 不可预测分支会造成错误路径清空，浪费前端和后端工作。
- 间接分支目标多时，预测更困难。
- 代码布局会影响取指、I-cache、BTB 和 fallthrough。
- **ret** 通常依赖 RSB 预测；异常调用模式可能让返回预测变差。

深入理解

这不是说每个分支都要手工消掉。高性能工程里要用数据证明：**perfstat** 的 **branch misses**、采样热点、输入分布、汇编路径长度、内存行为都要一起看。单独盯一条 **jcc**，很容易做出错误优化。

可预测性来自模式

分支预测器利用历史模式预测未来。一个循环回边通常很容易预测，因为它在绝大多数迭代中都跳回循环开头，只在最后一次退出。一个判断输入是否为空的分支也可能很好预测，因为生产环境中非空请求占绝大多数。相反，一个根据随机数据位决定走向的分支，可能接近不可预测。

“随机”也要说清楚。完全独立的随机位对方向预测器最困难；周期性模式如真、假、真、假可能被某些预测器学到；长段连续真再长段连续假通常比均匀随机容易；按排序数据扫描时，分支可能只在阈值附近改变一次。输入分布不是一个形容词，而是实验条件的一部分。

这意味着同一段代码在不同输入分布下性能可能完全不同。下面这种代码：

```
if (a[i] > threshold) {
    s += a[i];
}
```

如果 **a[i]>threshold** 几乎总为真或几乎总为假，分支预测通常容易；如果真假接近随机各一半，错误预测可能显著增加。把它改成 **branchless** 形式是否更快，要看 **load**、**add**、**mask**、**cmov** 或向量指令的成本，以及错误预测减少了多少。

因此，控制流性能实验必须报告输入分布。只说“**branchless** 版本更快”没有意义；应该说明数据是有序、随机、偏斜、周期性，还是来自真实业务采样。还要说明数据是否足够大到让 **cache** 行为成为瓶颈，是否经过热身，是否固定 **CPU** 频率和亲和性。后面测量章节会系统讲这些要求，本章先强调：分支性能不是代码文本单独决定的。

branch miss 计数怎么读

perfstat 里的 **branches** 和 **branch-misses** 很有用，但不能孤立解释。首先，不同 **CPU**、内核版本和权限配置对事件的含义与可用性可能不同。其次，**branch-misses** 统计的是硬件事件，不等于源码层 **if** 失败次数。循环回边、函数调用返回、间接跳转、编译器生成的边界检查，都可能贡献分支事件。

第三，分支错预测不是唯一瓶颈。一个版本 **branch miss** 很少，但如果它每次多做几次 **load** 或形成更长依赖链，仍然可能慢；另一个版本 **branch miss** 较多，但循环被向量化、内存访问更少，也可能快。性能结论必须把时间、指令数、分支数、**miss** 比例、**cache miss**、汇编形态和输入分布一起看。

第四，编译器可能改写你的实验。你写了 **if**，优化器可能生成 **cmovcc**、**mask** 或向量比较；你写了 **branchless**，优化器也可能把它识别成选择并重新生成分支。实验前必须检查汇编核心循环。若要对标量分支和标量 **branchless**，可能需要禁止向量化、禁止内联、使用合适的数据依赖或单独编译函数。

第五，测量顺序会影响结果。先运行的版本可能承担冷 **cache**、动态链接、页错误和分支预测器初始状态；后运行的版本可能受益于热数据。严谨实验应交错运行、重复多轮、报告方差或至少报告多次样本。后面测量章节会更严格地规范这些问题；本章先要求你不要用一次运行得出绝对结论。

常见误区

branch-misses 不是“源码 if 错了几次”的同义词。它是硬件计数器事件，必须和汇编、输入分布、循环形态、内存行为和测量方法一起解释。

代码布局也影响控制流成本

即使语义相同，基本块排列也会影响取指和预测。编译器可能把常见路径放在顺序 fallthrough 上，把冷错误路径放到远处；可能把异常处理块、断言失败块、慢路径 helper 放在函数末尾或单独的 cold section；也可能根据 profile-guided optimization 重新排列基本块。

代码布局影响 I-cache、指令 TLB、预取、BTB 项局部性和前端带宽。一个热循环如果被很少执行的错误处理代码插在中间，可能增加取指压力；把冷路径移开后，热路径更连续。对小函数来说，这种差异可能不明显；对大型服务、解释器、数据库执行引擎和机器学习 runtime，代码布局会成为真实性能因素。

读反汇编时，如果看到源代码相邻的分支在机器码中被拆远，不要立刻认为编译器“打乱了逻辑”。它可能是在做热冷分离、尾合并、跳转缩短或布局优化。分析报告应从实际地址和基本块边出发，而不是从源码行号顺序出发。

分支预测和安全缓解

现代 CPU 的预测执行提高性能，也带来安全挑战。间接分支预测、返回预测和错误路径执行曾经成为多类侧信道攻击的基础。系统和编译器为此引入了各种缓解，例如限制间接分支预测、插入序列避免某些推测路径、使用间接分支跟踪或控制流完整性检查。

第一册不要求展开这些安全机制，但要形成正确直觉：控制流性能不是纯 ISA 问题。相同源代码在不同编译选项、内核缓解状态、CPU 代际、动态链接方式下，间接调用和返回的成本可能不同。做高质量 benchmark 时，应记录系统环境和编译选项；做跨机器性能比较时，应避免把安全缓解差异误判成算法差异。

控制流阅读流程

读一段陌生控制流汇编，按下面顺序：

1. 标出函数入口和所有标签。
2. 按跳转、调用、返回切分基本块。
3. 给每个基本块编号，例如 B0、B1、B2。
4. 对每个块写入口寄存器含义。
5. 找出块尾的 `jcc`、`jmp`、`call`、`ret`。
6. 如果是 `jcc`，向上找最近一次设置 flags 的指令。
7. 判断条件是 signed、unsigned、相等、非零还是位测试。
8. 画边：taken edge 和 fallthrough edge。
9. 对循环找回边和退出边。
10. 对每条内存访问写地址公式。

报告里建议使用这种格式：

```
B0:
instructions:
    test edi, edi
    jle B2
state:
    edi = x
    esi = y
outgoing edges:
    if signed x <= 0 -> B2
    otherwise       -> B1

B1:
instructions:
    lea eax, [rsi+rsi]
    add eax, 1
    ret
```

```

meaning:
    return y * 2 + 1

B2:
    instructions:
        lea eax, [rsi+3]
        ret
    meaning:
        return y + 3

```

常见误区

第一，按源码缩进画 CFG。优化后源码块可能被合并、拆分、重排、内联或删除。正确做法是按机器跳转和标签切分，再回头解释它对应的源码意图。

第二，把每个 `call` 当成当前函数 CFG 的终点。普通调用会返回到下一条指令，它是基本块内部的一次控制转移；只有不返回调用或异常路径需要特殊处理。

第三，忘记 `fallthrough`。条件跳转的目标是一条边，下一条指令也是一条边。报告里应明确写出两条边的条件。

第四，只写“跳转到小于”而不写操作数顺序和 `signed/unsigned`。完整结论必须包含 `flags` 生产者、消费者和语义，例如“`cmpecx, esi` 后 `jle` 表示 `signed ecx < esi`”。

第五，看到 `cmovcc` 就说“无分支一定更快”。`cmovcc` 消除的是控制流分支，不消除数据依赖，也不允许提前执行有副作用或非法访问风险的表达式。

第六，把跳转表当成绝对地址数组而忽略 `rip` 相对寻址、索引归一化和范围检查。跳转表分析必须从 `range check` 开始。

高质量控制流报告

控制流报告应当证明你能恢复结构，而不是把反汇编逐行翻译成中文。建议包含六个部分。

第一，构建条件。写清编译器、优化级别、目标平台、是否 `PIE`、是否保留帧指针。不同条件会改变基本块布局和调用形式。

第二，函数签名和入口状态。列出参数寄存器和返回寄存器。即使本章重点是控制流，也不能离开 `ABI` 入口状态读寄存器含义。

第三，基本块表。每个块给出地址范围、核心指令、块尾控制指令。不要无筛选粘贴整段 `objdump`。

第四，边表。对每条边写明 `taken/fallthrough`、条件、`signed/unsigned`、对应 `flags` 生产者。循环要标出回边和退出边。

第五，数据流摘要。写清累加器、归纳变量、指针递推、地址公式和返回值来源。控制流和数据流必须合在一起，才能解释程序含义。

第六，性能解释边界。如果讨论分支性能，要说明输入分布、预测可能性、是否有间接分支、是否有内存瓶颈、是否有实际计数器证据。没有测量证据时，只能写“可能”，不能写成确定结论。

第七，合流点和循环携带值。对有分支后继续执行的函数，写清每条前驱边进入合流点时哪些寄存器已经准备好。对循环，写清哪些寄存器从上一轮传到下一轮，例如累加器、当前指针、剩余计数和 `mask`。没有这部分，报告只能说明“代码会走哪里”，还不能说明“值如何跨路径和跨迭代保持正确”。

第八，二进制证据。涉及跳转表、外部调用、`PLT/GOT` 或重定位时，应给出目标文件和最终可执行文件的差异。目标文件里看到的占位地址、重定位项和最终链接后的运行时地址不是同一层事实。报告要说明自己观察的是哪一层。

核心理想

控制流报告的核心不是“这条指令是什么意思”，而是“这些基本块和边共同允许哪些执行路径，每条路径上数据如何变化，哪些结论来自规则，哪些来自观察”。

把控制流和数据流一起读

控制流图只告诉你程序可能走哪些路径，数据流告诉你每条路径上值如何产生、传递和合流。只画 CFG 不追数据，会得到一张空图；只追寄存器不画边，又会在分支、循环和早返回处迷路。真实汇编阅读必须把两者合起来。

看一个简单形态：

```
test edi, edi
jle .Lnonpos
lea eax, [rsi + 1]
jmp .Ljoin
.Lnonpos:
lea eax, [rsi - 1]
.Ljoin:
add eax, edx
ret
```

CFG 说明入口根据 **edi** 分成两条路径，随后在 **.Ljoin** 汇合。数据流说明两条路径进入 **.Ljoin** 前都必须定义 **eax**：正数路径定义为 **y+1**，非正路径定义为 **y-1**。合流后 **add eax, edx** 不关心 **eax** 来自哪条路径，只要求每条可达路径都已经准备好它。

如果报告只写“条件满足跳到 **.Lnonpos**，否则继续”，还没有解释返回值。如果只写“**eax** 最后加上 **edx**”，又没有解释 **eax** 在不同路径上的来源。合格分析应该写成：

```
B0:
  condition: signed x <= 0
  if true -> B2
  if false -> B1

B1:
  eax = y + 1
  -> B3

B2:
  eax = y - 1
  -> B3

B3:
  incoming eax:
    from B1: y + 1
    from B2: y - 1
  eax = eax + z
  return eax
```

这种写法等价于在汇编层恢复一个隐含的 **phi**。机器码没有 **phi** 指令，但合流点确实存在“根据前驱路径选择值”的语义。理解这点之后，读优化后的分支、循环累加器和条件移动会容易很多。

路径条件是数据流的一部分

路径条件不是只用于画箭头，它还会约束后续数据范围。例如：

```
test edi, edi
jle .Ldone
mov eax, dword ptr [rsi]
.Ldone:
ret
```

若 **fallthrough** 进入 **load**，说明在该路径上 **signed edi > 0**。这个条件可以成为后续推理的前提。编译器优化也会利用这类事实：进入某个块后，某些值已知非零、非负、小于边界、等于某个常量或某个位已设置。人工读汇编时也应把路径条件写入状态表。

路径条件尤其影响循环。进入循环体前可能已经证明 **n > 0**；进入某个数组 **load** 前可能已经证明 **i < n**；进入 **default** 块前可能已经证明 **switch** 值不在若干 **case** 中。若你不记录这些条件，就会误以为某些访问缺少检查；若你记录错了，又可能把不安全路径误判为安全。

合流点要检查每个前驱

每个合流点都要检查所有前驱是否提供后续需要的状态。后续需要的状态包括寄存器值、栈指针位置、**callee-saved** 寄存器恢复情况、**flags** 是否仍有效、内存是否已经写入。很多隐藏错误都发生在“某条路径忘了准备某个值”。

例如手写汇编中：

```
test edi, edi
```

```

jle .Lskip
mov eax, esi
.Lskip:
add eax, edx
ret

```

若 `edi<=0`，执行会直接到 `.Lskip`，但 `eax` 没有在当前函数中定义。它可能保留调用者留下的旧值，导致结果不可预测。编译器生成的定义良好代码通常不会这样，但手写汇编、错误内联汇编、反汇编混淆和坏优化器 bug 都可能出现类似形态。合流点检查能及时发现问题。

常见误区

控制流分析不能只画边。每个合流点都要写“从每条前驱进入时，后续要用的寄存器和内存状态是什么”。

条件码选择表要结合操作数顺序

条件码最常见错误有两个：忘记 Intel 语法的 `cmp dst, src` 表示 `dst-src`，以及把 signed/unsigned 条件码混用。一个实用方法是每次看到条件跳转都写三行：

```

producer:
    cmp A, B
temporary:
    flags from A - B
consumer:
    jl means signed A < B
    jb means unsigned A < B

```

不要直接写“`jl` 跳到小于”。小于谁？按 signed 还是 unsigned？操作数顺序是什么？如果这些问题没有回答，条件码分析就没有完成。

常用条件码的阅读模板

下面模板适用于 `cmp A, B` 后的常见条件：

条件码	读法	适用语义
<code>je/jz</code>	<code>A==B</code>	相等，不区分 signed/unsigned
<code>jne/jnz</code>	<code>A!=B</code>	不相等，不区分 signed/unsigned
<code>jl/jnge</code>	<code>A<B</code>	signed 小于
<code>jle/jng</code>	<code>A<=B</code>	signed 小于等于
<code>jg/jnle</code>	<code>A>B</code>	signed 大于
<code>jge/jnl</code>	<code>A>=B</code>	signed 大于等于
<code>jb/jc/jnae</code>	<code>A<B</code>	unsigned 小于
<code>jbe/jna</code>	<code>A<=B</code>	unsigned 小于等于
<code>ja/jnbe</code>	<code>A>B</code>	unsigned 大于
<code>jae/jnb/jnc</code>	<code>A>=B</code>	unsigned 大于等于

这些助记符有多个别名。别名不是不同指令语义，而是同一条件的不同名字，强调角度不同。例如 `jb` 和 `jc` 都看 CF，只是前者强调 unsigned below，后者强调 carry。报告里建议使用能表达源码语义的名字，例如数组长度比较用 unsigned below/above，进位链用 carry/no carry。

test 后的条件码不要乱套比较模板

`test A, B` 产生的是 `A&B` 的 flags，不是 `A-B`。因此 `test` 后的 `je` 常表示“按位与结果为 0”，`jne` 表示“至少有一个被测试 bit 为 1”，`js` 表示结果最高位为 1。不要把 `test` 后的 `jl` 解释成“`A < B`”。虽然 flags 组合仍然有定义，但常规大小比较模板基于 `cmp/sub`，不能直接套到位测试上。

例如：

```

test edi, 8
jne .Lhas_bit3

```

这里不是比较 `edi` 和 8 的大小，而是测试 bit 3 是否设置。若写成“`edi!=8` 时跳转”，就是错误解释。完整说明应为：计算 `edi&8`，若结果非零，则说明 `edi` 的 bit 3 为 1，跳到 `.Lhas_bit3`。

循环分析要证明边界

循环是控制流和数据流最密集的结构。一个合格的循环分析不只是找出回边，还要证明循环从哪里开始、何时结束、每轮访问哪些内存、是否可能多读或少读一个元素。尤其是数组循环，边界证明直接关系到正确性和安全性。

四个循环块的工程含义

许多优化后的循环可以分成四类块：`preheader`、`header`、`latch`、`exit`。`preheader` 是进入循环前只执行一次的准备块，常计算结束指针、初始累加器、对齐信息或常量。`header` 是每次迭代开始处，常包含退出检查或循环体入口。`latch` 是每轮末尾，更新归纳变量并决定是否回到下一轮。`exit` 是循环结束后的块。

并非每个循环在机器码中都明显分成四个标签，但这个模型能帮助你找职责。若结束指针在循环前计算，要记录它是循环不变量；若下标在 `latch` 递增，要记录递增宽度；若退出检查在循环头，是 `while-like`；若退出检查在循环尾，是 `do-while-like` 或经过 `loop rotation` 的形态。编译器可能重排循环，使源码 `for` 看起来像 `do-while`。不要按源码语法判断机器循环形态。

证明数组访问范围

以典型求和循环为例：

```
xor eax, eax
test esi, esi
jle .Ldone
xor ecx, ecx
.Lloop:
add eax, dword ptr [rdi + rcx*4]
inc rcx
cmp ecx, esi
jl .Lloop
.Ldone:
ret
```

边界证明可以写成：

```
entry:
    p = rdi
    n = esi
    s = 0

precheck:
    if signed n <= 0, no load happens

loop invariant before load:
    0 <= i < n
    rcx = i
    eax = sum of p[0..i-1]

load:
    address = p + i * 4
    width = 4 bytes

latch:
    i = i + 1
    if i < n, continue
    otherwise exit
```

这个证明说明第一次 `load` 时 `i=0`，最后一次 `load` 时 `i=n-1`，当 `n<=0` 时没有 `load`。若没有 `precheck`，`n=0` 时可能仍执行一次循环，形态就不同。若比较条件是 `unsigned`，负数 `n` 的路径也不同。边界证明必须包含 `signed/unsigned` 条件。

指针递增循环的边界证明

优化器常把下标循环变成指针递增：

```

lea rax, [rdi + rsi*4]
xor ecx, ecx
.Lloop:
add ecx, dword ptr [rdi]
add rdi, 4
cmp rdi, rax
jne .Lloop
mov eax, ecx
ret

```

这里 `rdi` 不再是固定 base，而是当前指针；`rax` 是结束指针。报告应写：

```

initial:
    cur = p
    end = p + n * 4

each iteration:
    load 4 bytes from cur
    cur = cur + 4
    continue while cur != end

```

这种循环的正确性依赖进入循环前已经保证 `n>0`，否则 `cur==end` 时仍可能先 load 一次。若机器码没有显示 precheck，要查前面是否有单独路径处理空长度。指针递增形式特别容易让初学者忘记 base 已经变化，错误地把每轮访问都写成 `p[0]`。状态表必须在每轮更新 `rdi` 的含义。

短路逻辑是受保护的的控制流

C++ 的 `&&` 和 `||` 不只是布尔运算，它们有短路求值规则。机器层面常表现为控制流保护：只有前一个条件允许时，后一个表达式才会求值。这对于避免非法内存访问非常重要。

例如：

```

extern "C" int guarded_load(int const* p, int n)
{
    if (p != nullptr && n > 0) {
        return *p;
    }
    return 0;
}

```

一种机器形态可能是：

```

test rdi, rdi
je .Lzero
test esi, esi
jle .Lzero
mov eax, dword ptr [rdi]
ret
.Lzero:
xor eax, eax
ret

```

这里 load 所在块被两个条件共同支配：`p!=nullptr` 和 signed `n>0`。短路逻辑的安全性就在于，如果 `p==nullptr`，第二个条件和 load 都不会以需要解引用 `p` 的方式执行。若编译器把 load 提前到 null check 之前，就会改变定义良好路径上的行为或引入非法访问，因此通常不允许。

`||` 的保护方向不同：

```

if (p == nullptr || *p == 0) {
    return 0;
}
return *p;

```

这里只有当 `p!=nullptr` 时，才能执行 `*p`。机器码可能先判断空指针，若为空直接返回；若非空再 load。报告中应写“哪个条件保护哪个 load”。这比笼统写“短路逻辑会跳转”更有用。

branchless 不能破坏保护关系

短路逻辑有时可以被改写成 branchless 数据选择，但前提是被选择的表达式都可以安全求值，且没有副作用或异常边界。比如两个纯整数表达式：

```
return cond ? a + 1 : b + 1;
```

可能变成 `cmovcc`。但下面这种不能简单同时求值：

```
return p != nullptr ? *p : 0;
```

如果先无条件 load `*p`，空指针输入会出错。编译器可以使用某些受保护的投机或目标相关技巧，但必须保持语言语义。人工优化时尤其要谨慎，不要为了消除分支而把原本受保护的内存访问提前。

常见误区

branchless 不是无条件更高级。若条件保护了内存访问、除零、溢出检查、函数副作用或异常路径，把控制依赖改成数据依赖可能改变程序语义。

switch 分析要先找归一化和范围检查

跳转表是 `switch` 最容易吸引注意的部分，但分析跳转表不能从表开始，而要从归一化和范围检查开始。编译器通常会把 case 值转换成从 0 开始的索引，再检查索引是否在表范围内，最后才用索引读取跳转目标或结果表。

例如源码 case 是 10 到 14：

```
switch (x) {
case 10: return 1;
case 11: return 2;
case 12: return 3;
case 13: return 4;
case 14: return 5;
default: return 0;
}
```

机器码可能先做：

```
sub edi, 10
cmp edi, 4
ja .Ldefault
jmp qword ptr [rip + .LJTI + rdi*8]
```

这里 `sub edi, 10` 把原始 `x` 归一化成 `x-10`；`cmp edi, 4` 与 `ja` 使用 unsigned 检查。为什么 unsigned 检查可以同时挡住小于 10 和大于 14？因为若原始 `x<10`，归一化后在 32 位无符号视角下会变成很大的数，满足 `>4`，进入 default。这个技巧很常见，报告里必须写清楚，否则会误以为缺少下界检查。

跳转表、结果表和位测试不要混淆

并非所有 `switch` 都生成跳转表。若每个 case 只是返回常量，编译器可能生成结果表：

```
mov eax, dword ptr [rip + table + rdi*4]
ret
```

这里读取的是返回值，不是跳转目标。若 case 集合稀疏但可以用位集合表示，编译器可能生成 bit test。若 case 数量少，可能生成比较链或二分树。判断依据不是“看到表就是跳转表”，而是看表项如何被使用：表项被加载到 `rip` 或作为间接 `jmp` 目标，才是控制流跳转表；表项被加载到 `eax` 返回，就是数据表。

default 是安全边界

对于跳转表，default 不只是语义分支，也是安全边界。没有范围检查，任意输入都可以作为索引读取表外地址，形成任意间接跳转。编译器生成的定义良好 `switch` 会保护跳转表索引。手写汇编或二进制漏洞分析中，若看到间接跳转使用未检查索引，就要高度警惕。

报告中分析 **switch** 至少写四项：原始 **switch** 表达式在哪个寄存器，如何归一化，范围检查条件是什么，表项类型是什么。最后再列出 **case** 到目标块或返回值的映射。顺序不能反过来。

调用边也是控制流边

在当前函数的 CFG 中，普通 **call** 通常不是终点，因为它预期返回到下一条指令。但它仍然是一条重要控制流边：执行会暂时离开当前函数，进入另一个函数，可能读写内存、破坏 **caller-saved** 寄存器、抛异常、调用回调、阻塞、进行系统调用，甚至不返回。分析调用点时，不能只写“调用某函数”，还要说明调用前后状态。

调用点至少有五个问题：

1. 参数是否已经放到 ABI 要求的位置。
2. 调用前 **rsp** 是否满足对齐规则。
3. 调用后哪些寄存器仍可相信，哪些 **caller-saved** 已失效。
4. 返回值从哪里取得，宽度是多少。
5. 调用目标是否可能不返回、抛异常或通过指针修改内存。

例如：

```
mov edi, eax
call helper
add ebx, eax
```

若 **ebx** 在调用前保存累加器，它是 **callee-saved**，调用后仍应保持；**eax** 在调用后是返回值，不再是调用前的旧 **eax**；**edi** 是参数寄存器，调用后不能假设仍等于参数。若 **helper** 可能修改 **rdi** 指向的内存，调用前后的内存状态也要失效或重新证明。

间接调用需要目标集合

直接调用的目标在反汇编中通常是符号或相对地址；间接调用的目标来自寄存器或内存：

```
call rax
call qword ptr [rdi + 16]
```

这种调用不能只写“调用 **rax**”。报告应尽量恢复目标集合：**rax** 来自函数指针参数、虚表槽、回调表、PLT/GOT 入口，还是编译器生成的 **thunk**？如果无法确定，就写明未知。间接调用对性能、安全和可调试性都更复杂，因为目标预测、控制流完整性、动态绑定和 ABI 适配都可能参与。

不返回调用改变 CFG

若调用目标是 **abort**、**exit**、**__builtin_trap** 或被标注 **[[noreturn]]** 的函数，调用后没有正常 **fallthrough**。编译器可能在 **call** 后不生成返回路径，也可能放置不可达填充。手工画 CFG 时，要把这类调用作为终点。若误把它当成普通 **call**，会画出不存在的路径，后续数据流也会出现“未定义值继续使用”的假象。

核心思想

调用点是 ABI、控制流和数据流的交界。读 **call** 时要同时考虑参数、返回值、寄存器失效、内存副作用和是否返回。

控制流证据矩阵

控制流分析经常出现一种问题：报告画了一张 CFG，但没有说明哪些边来自静态反汇编，哪些边在实验输入中真的走过，哪些边只是理论可能，哪些边需要额外测试。为了解决这个问题，可以给每个函数建立控制流证据矩阵。

矩阵至少包含：

边	静态条件	动态证据	备注
B0 -> B1	fallthrough, $x > 0$	input $x=3$ hit	then path
B0 -> B2	taken, $x \leq 0$	input $x=0, -1$ hit	boundary covered
B1 -> B3	unconditional	covered with $x=3$	join
B2 -> B3	unconditional	covered with $x=0$	join

静态条件来自反汇编和 flags 分析；动态证据可以来自测试输入、GDB 断点、coverage、采样、日志或手工记录。对于性能报告，静态可能路径和实际热路径必须分开。一个错误处理分支静态存在，不代表它影响当前 benchmark；一个分支在测试中未覆盖，也不代表它不重要。

边覆盖不是语句覆盖

源代码语句覆盖不能替代机器边覆盖。优化器可能合并分支、删除分支、把分支改成 `cmov`，也可能把多个源码条件共享一个机器分支。若你要验证汇编控制流，最好直接以基本块和边为单位记录。

例如短路逻辑：

```
if (p != nullptr && n > 0) {
    return *p;
}
return 0;
```

测试只覆盖 $p \neq \text{nullptr}, n > 0$ 的路径，还不能证明空指针保护路径正确。至少要覆盖：

- $p == \text{nullptr}$ ，第一条件失败，不能执行 load。
- $p \neq \text{nullptr}, n \leq 0$ ，第二条件失败，不能执行 load。
- $p \neq \text{nullptr}, n > 0$ ，执行 load。

机器边覆盖能明确说明每个保护条件都被测试到。对安全关键代码和手写汇编包装函数，这比只看返回值更可靠。

路径条件和断言

控制流分析得到的路径条件可以转化为测试断言。若某个块只应在 $0 \leq i < n$ 时执行，你可以在调试构建中加入断言，或在 GDB 条件断点中验证。这样能把静态推理和动态检查连接起来。

例如：

```
Bload:
    path condition:
        i >= 0
        i < n
    memory:
        load p[i]
```

对应测试应覆盖：

- $i = -1$ 应走 out。
- $i = 0$ 应走 load。
- $i = n - 1$ 应走 load。
- $i = n$ 应走 out。

这四个点比随机挑几个正常输入更有价值，因为它们直接验证边界。控制流报告若能列出这些输入，说明作者不仅画了 CFG，还理解了路径条件的意义。

不可达路径也要说明原因

优化后汇编中可能缺少某条源码路径。原因可能是路径在语言规则下不可达、被常量折叠、被内联合并、被 `__builtin_unreachable` 或假设消除，也可能是你没看完整二进制。报告中看到路径消失，应说明原因。

例如：

```
extern "C" int g(unsigned char x)
{
```

```

    if (x > 255) {
        return -1;
    }
    return x;
}

```

$x > 255$ 在类型范围下不可能成立，优化后不会有这条分支。这里“不可达”来自类型范围。另一个例子，如果程序写了 signed overflow 后的检查，编译器可能基于未定义行为前提删除某些路径。这里“不可达”来自语言规则假设，更需要谨慎解释。

循环路径的三种正确性

循环分析至少有三种正确性：入口正确、迭代正确、退出正确。

入口正确指第一次进入循环体前，循环变量、累加器、当前指针和结束条件已经初始化。如果 $n \leq 0$ 时不应进入循环，入口前必须有检查，或循环形态必须保证先检查再 load。

迭代正确指每轮循环体在路径条件下执行正确工作。例如第 i 轮读取 $p[i]$ ，更新 s ，然后把 i 改成 $i+1$ 。如果循环展开，每一轮机器迭代可能处理多个源码元素，报告要写清每个 unrolled lane 对应哪个下标。

退出正确指循环结束时刚好处理了应处理的元素，没有多处理也没有少处理。退出条件可能在头部，也可能在尾部；可能比较下标，也可能比较指针；可能使用 `jl`、`jne`、`jae` 等不同条件码。必须把最后一次迭代和退出边界说清楚。

展开循环的边界

优化器可能把循环展开四次：

```

.Lloop:
    add rax, qword ptr [rdi + rcx*8]
    add rax, qword ptr [rdi + rcx*8 + 8]
    add rax, qword ptr [rdi + rcx*8 + 16]
    add rax, qword ptr [rdi + rcx*8 + 24]
    add rcx, 4
    cmp rcx, rsi
    jb .Lloop

```

这段主循环只适合处理能被 4 分块覆盖的部分。真实编译器还需要处理余数，可能在前面计算 $n \& 4$ ，后面有 tail loop，或使用条件处理。报告不能只解释主循环，还要找尾部路径。很多 SIMD 和展开 bug 都发生在尾部。

边界说明可以写成：

```

main loop:
    processes i, i+1, i+2, i+3
    advances i by 4
    valid only while i + 3 < n

tail:
    processes remaining n % 4 elements

```

如果没有 tail，而接口允许任意 n ，就可能是 bug；如果 wrapper 保证 n 是 4 的倍数，则需要文档证据。

控制流和错误处理路径

性能报告往往只关心热路径，但正确性报告不能忽略错误路径。错误路径包括空指针、长度为零、越界、格式错误、系统调用失败、分配失败、unsupported ISA、checksum failed 等。优化构建可能把错误路径放到冷 section，也可能用不返回调用结束。

读错误路径时，要问：

- 错误条件在哪里检测。
- 错误路径是否会继续使用未初始化或非法数据。
- 错误返回值或异常是否符合接口。
- 错误路径是否恢复栈和 callee-saved 寄存器。

- 错误路径是否影响 benchmark timed region。

手写汇编中，错误路径尤其容易忘记恢复寄存器或栈。C++ 中，错误路径可能抛异常、调用日志、分配字符串，导致性能测量中的长尾。报告中应把热路径和错误路径分开，但不能把错误路径当作不存在。

间接控制流的目标恢复

间接跳转、函数指针、虚函数、PLT/GOT 和跳转表都属于间接控制流。它们共同特点是：指令文本里不直接写最终目标，而是从寄存器或内存取得目标地址。分析间接控制流时，要恢复目标集合。

函数指针调用：

```
call rax
```

目标可能来自参数、表项、lambda 转换、回调注册或虚拟分发。你需要向上追踪 `rax` 的来源。如果来源是 `[rdi]` 或 `[rdi+offset]`，可能是对象中的函数指针或虚表槽；如果来源是 GOT，可能是动态链接；如果来源是跳转表，目标集合来自 case 表。

虚函数调用的第一眼

C++ 虚函数调用常见形态是先从对象读取 `vptr`，再从 `vtable` 读取函数地址：

```
mov rax, qword ptr [rdi]
call qword ptr [rax + 16]
```

这里 `rdi` 通常是 `this` 指针。第一条 `load` 读取对象开头的 `vptr`，第二条通过 `vtable offset` 调用某个虚函数槽。这个形态不是普通结构体字段访问那么简单，它连接了 C++ 对象模型、ABI 和间接调用。优化器若能证明动态类型，可能去虚化，把间接调用改成直接调用甚至内联。若不能证明，间接调用会保留。

报告中看到虚调用形态，应说明目标集合可能包含哪些动态类型，当前构建是否有去虚化证据，性能解释是否考虑间接调用和内联失败。不要只写“调用某个函数”，因为机器码中并没有直接目标。

控制流审查清单

分析一个函数控制流时，使用下面清单：

1. 是否列出所有基本块入口和出口。
2. 每个条件跳转是否写出 `taken` 和 `fallthrough` 两条边。
3. 每个条件是否注明 `flags` 生产者和 `signed/unsigned` 语义。
4. 每个合流点是否说明前驱路径提供的寄存器和内存状态。
5. 每个循环是否说明入口、迭代、回边、退出和尾部处理。
6. 每个短路逻辑是否说明哪个条件保护哪个危险操作。
7. 每个 `switch` 是否说明归一化、范围检查和表项类型。
8. 每个 `call` 是否说明参数、返回、寄存器失效和是否可能不返回。
9. 每个间接控制流是否追踪目标来源或承认未知。
10. 动态测试是否覆盖关键边界路径。

这张清单能把控制流报告从“画图”提升到“证明路径”。图画得漂亮但边界不清，仍然不能支持正确性和性能结论。

综合案例：短路保护不能被随意改写

考虑：

```
extern "C" int maybe_load(const int* p, int fallback)
{
    if (p != nullptr && *p > 0) {
        return *p;
    }
    return fallback;
}
```

控制流的关键不是有两个条件，而是第二个条件受第一个条件保护。机器码必须保证当 `p=nullptr` 时，不会执行 `*p` 的 load。一种合理形态是：

```
test rdi, rdi
je .Lfallback
mov eax, dword ptr [rdi]
test eax, eax
jle .Lfallback
ret
.Lfallback:
mov eax, esi
ret
```

CFG 分析应写：

```
B0:
  if p == nullptr -> Bfallback
  else -> Bload

Bload:
  path condition: p != nullptr
  v = *p
  if v <= 0 -> Bfallback
  else -> Breturn_v
```

如果有人为了“branchless”把它改成先无条件 load，再用条件选择返回值：

```
mov eax, dword ptr [rdi]
...
```

这会在 `p==nullptr` 时崩溃，改变语义。控制依赖在这里是安全条件，不只是性能成本。控制流优化必须证明被提前执行的操作对所有路径都合法。

测试如何覆盖这个案例

至少需要三个输入：

- `p==nullptr`，应返回 fallback，且不能 load。
- `p!=nullptr, *p<=0`，应返回 fallback。
- `p!=nullptr, *p>0`，应返回 `*p`。

若只测试第三个输入，错误 branchless 改写也能通过。控制流测试必须覆盖保护失败路径。

综合案例：switch 跳转表的安全边界

考虑：

```
extern "C" int classify(int x)
{
  switch (x) {
    case 10: return 100;
    case 11: return 110;
    case 12: return 120;
    case 13: return 130;
    default: return -1;
  }
}
```

优化后可能出现：

```
sub edi, 10
cmp edi, 3
ja .Ldefault
mov eax, dword ptr [rip + table + rdi*4]
ret
.Ldefault:
mov eax, -1
```


ret

这里不是跳转表，而是结果表。范围检查使用 `unsigned ja`，同时处理原始 `x<10` 和 `x>13`。报告要写清：

```
normalized index = x - 10
valid range      = 0..3
out of range     = unsigned index > 3
table kind       = data table, not jump target table
```

如果表项被用作 `jmp` 目标，才是控制流跳转表。无论结果表还是跳转表，范围检查都是安全边界。没有范围检查的表索引，可能变成越界读或任意跳转。

自测清单：控制流分析是否过关

完成本章后，应能回答：

1. 基本块如何切分，`fallthrough` 为什么也是边。
2. `cmp A, B` 后的条件码如何根据 `A-B` 解释。
3. `signed` 和 `unsigned` 条件码如何区分。
4. 合流点如何统一不同前驱路径上的值。
5. 循环入口、回边、退出和尾部如何证明。
6. 短路逻辑保护了哪些危险操作。
7. `cmovcc` 消除分支时有什么语义前提。
8. `switch` 的归一化、范围检查和表类型如何恢复。
9. 普通 `call` 在当前函数 CFG 中为什么通常不是终点。
10. 间接调用如何追踪目标集合。

深入讲解：控制流优化为什么会改变可读性

优化器经常把源码控制流改写成机器上更合适的形态。它可能反转条件，让热路径 `fallthrough`；可能把多个 `early return` 合并成一个出口；可能把小分支改成 `cmovcc`；可能把 `switch` 改成查表；可能把循环旋转成先执行一次再在尾部判断；可能把错误路径移到冷区域。所有这些都会降低源码形状和机器形状的一一对应。

这不是编译器故意让人难读，而是机器成本模型不同于源码排版。`fallthrough` 布局影响取指和分支；合并出口可以减少重复 `epilogue`；条件移动可以避免难预测分支；跳转表可以减少比较链；循环旋转可以让主循环更紧凑；冷路径分离可以改善热路径局部性。

读优化控制流时，不要问“为什么它不按我的 `if` 写”。要问：这个布局的基本块是什么，路径条件是什么，数据如何在合流点统一，热路径是否更短，错误路径是否仍正确，二进制证据是否支持。源码结构是起点，机器 CFG 才是实际执行对象。

深入讲解：控制流正确性和性能经常拉扯

有些优化看起来性能更好，但会碰到正确性边界。提前执行 `load` 可以隐藏延迟，但如果原路径中 `load` 受 `null check` 或边界检查保护，就不能提前。把分支改成 `cmov` 可以减少 `branch miss`，但两个候选值必须都能安全计算。把 `switch` 改成跳转表可以快，但索引必须先范围检查。把错误路径标为 `unreachable` 可以优化热路径，但前提必须真实。

因此，控制流优化的审查顺序是：先证明合法，再谈性能。合法性包括路径保护、对象生命周期、异常和副作用、除零、溢出、越界、函数调用副作用。只有候选路径都安全，才能把控制依赖变成数据依赖。

性能上也不能简单认为分支越少越好。可预测分支可能很便宜；`cmov` 可能延长依赖链；查表可能增加内存访问；跳转表可能引入间接分支；合并路径可能增加寄存器压力。真正的选择要看输入分布、热路径、依赖和测量。

课堂讲解：为什么 CFG 是编译器和人工分析的共同语言

控制流图不是画图作业，而是把程序路径变成可讨论对象的方法。编译器需要 CFG 来做可达性、支配关系、循环分析、数据流分析、异常路径、活跃变量和优化合法性判断。人工阅读汇编时也需要 CFG，因为线性反汇编只显示地址顺序，不显示每条路径在语义上如何分叉和汇合。

基本块是 CFG 的节点，边是控制可能从一个块流向另一个块的关系。这个抽象看似简单，却能统一很多现象。一个 `if/else` 是分叉后合流；一个循环是入口、回边和退出；一个早返回是直接通向函数出口；一个 `switch` 是范围检查后通向多个目标；一个函数调用在当前函数内通常是“调用后返回到下一条指令”的边；一个不返回调用则终止当前路径。

CFG 的价值在于它迫使读者说明路径条件。看到一个 `load`，不能只说“这里读取内存”，还要问到达这个 `load` 的路径上是否检查过指针非空、下标范围、对象生命周期和权限。看到一个除法，不能只说“这里除以寄存器”，还要问除数在所有前驱路径上是否可能为零。看到一个 `cmov`，要问两个候选值是否都已经安全计算。路径条件是正确性证明的一部分。

支配关系把保护条件说清楚

支配关系的直觉是：如果每条到达 B 的路径都必须经过 A，那么 A 支配 B。这个概念对安全检查特别有用。若一个边界检查块支配数组访问块，说明访问前一定经过检查；若它不支配，就可能存在绕过检查的路径。很多漏洞、崩溃和优化错误都可以用“保护块没有支配危险操作”来描述。

在优化后汇编中，保护条件可能被重排、合并或反转。源码中的 `if(p)use(p)` 可能变成先跳过空指针路径，也可能变成错误路径外提。人工分析时不要执着于源码排版，而要看机器 CFG 上是否所有到达 `use` 的路径都满足非空条件。若危险操作被提前到检查之前，就必须证明它在所有路径上都安全，否则优化不合法。

课堂讲解：控制流性能先问“可预测吗”

讨论分支性能时，第一问题不是“有没有分支”，而是“目标是否可预测”。一个总是走同一方向的分支，成本通常很低；一个随机方向的分支，可能频繁错误预测；一个和数据分布相关的分支，在排序输入、偏斜输入和随机输入上可能表现完全不同。没有输入分布，分支性能结论是不完整的。

可预测性也不只属于条件分支。间接调用的目标集合是否稳定，虚函数调用是否总是同一动态类型，`ret` 是否匹配返回地址栈，跳转表索引是否集中在少数 case，循环退出次数是否稳定，都会影响控制流预测。控制流越动态，越需要用数据分布和 PMU 事件支持判断。

`branchless` 改写也要放在这个框架里。把分支改成 `cmov` 或掩码运算，可能减少错误预测，但可能增加无条件计算、延长依赖链、增加寄存器压力，甚至破坏短路保护。若原分支高度可预测，`branchless` 可能更慢。若原分支难预测且两个候选计算都便宜，`branchless` 才可能有优势。结论必须来自具体 workload。

用路径热度读优化后的布局

优化器常把热路径放在 `fallthrough`，把冷路径放到远处，或把错误处理块集中到冷区域。线性反汇编中，地址相邻不一定表示语义关系最强；地址遥远也不一定表示不相关。阅读时要结合跳转方向、条件反转和剖析信息，恢复真实热路径。

例如一个边界检查可能写成“越界则跳到冷错误块，否则继续”。这时正常路径没有显式跳转，错误路径有一个 `taken branch`。若输入几乎都合法，这种布局有利于取指和预测。反过来，如果错误路径也很常见，布局假设就不成立。控制流性能报告应说明热路径假设来自哪里：代码语义、输入数据、profile、样本计数还是硬件事件。

综合案例：边界检查、数据选择和合法提前执行

考虑一个函数，它在下标合法时读取数组，否则返回默认值。源码上很容易写成一个 `if`。优化器可能尝试把条件变成数据选择，例如先计算默认值，再在合法路径读取数组，然后用条件移动选择结果。这个改写是否合法，关键不在于 `cmov` 本身，而在于数组读取是否仍受范围检查保护。

若机器码先执行数组 `load`，再用 `cmov` 选择是否采用 `load` 结果，那么当下标越界时，即使最终返回默认值，越界 `load` 也已经发生。这通常不等价于源码语义，可能触发崩溃、泄露信息或读到非法对象。合法的 `branchless` 改写必须保证所有无条件提前执行的操作在所有路径上都安全，或者只提前执行纯寄存器计算、常量选择等不会触发未定义行为的操作。

这个例子能帮助读者理解控制依赖和数据依赖的区别。原始 `if` 让 `load` 受控制依赖保护；改成数据选择后，选择本身可能是数据依赖，但候选值的计算如果提前，就可能失去保护。人工审查时要问：危险操作是否还在保护路径之后？保护块是否支配危险操作？如果不支配，有没有其他理由证明危险操作总是安全？

性能上也要谨慎。保留分支可能在合法输入上非常可预测；改成 `branchless` 可能引入额外计算和依赖。若默认路径很少出现，分支版本可能更好。若输入下标随机且两个路径都安全，`branchless` 才值得实验。正确报告必须同时包含合法性证明和测量证据。

路径覆盖：测试语句不等于测试边

控制流测试常见误区是只看语句覆盖。一个函数的每一行都执行过，不代表每条边都执行过；每条边都执行过，也不代表关键路径组合都覆盖了。底层错误往往藏在特定边上：错误路径没有释放资源，短路保护被绕过，循环零次执行没有初始化，`switch default` 没有处理，尾调用路径破坏栈。

边覆盖比语句覆盖更接近 CFG。对于 `if/else`，`then` 边和 `else` 边都要有；对于循环，要覆盖零次、一次、多次和退出；对于 `switch`，要覆盖每个 `case`、`default`、边界值和越界值；对于短路逻辑，要覆盖左操作数阻止右操作数执行的路径；对于错误处理，要覆盖每个返回错误码的边。测试设计应从 CFG 反推输入，而不是只从源码行数反推。

路径组合也重要。两个独立条件各自覆盖，不代表四种组合都覆盖。如果后续代码依赖两个条件共同成立，缺少组合测试会漏掉合流点错误。第一册不要求完整路径爆炸分析，但要求读者知道：控制流正确性不只是“跑到那一行”。

循环不变量和退出证明

读循环汇编时，要写出三个问题：进入循环前什么成立，每次迭代保持什么，退出时什么成立。进入条件决定零次循环是否安全；迭代不变量决定数组访问和累加状态是否正确；退出条件决定返回值或后续访问是否在合法范围内。

例如数组求和循环的不变量可以写成：在每次迭代头部，指针指向第 `i` 个元素，累加器等于前 `i` 个元素之和，且 $0 <= i <= n$ 。循环体读取第 `i` 个元素后，递增指针和 `i`，回到头部时不变量仍成立。退出时 `i == n`，累加器等于全部元素之和。汇编中的指针递增形式可能没有显式 `i`，但不变量仍可用元素计数表示。

优化后循环可能展开、剥离、向量化或拆成主循环和尾循环。不变量要相应分层。主循环每次处理多个元素，尾循环处理剩余元素；展开循环可能有多个 `load` 和累加器；向量化循环可能有向量累加和水平归约。无论形态如何，边界证明仍然要回答：不会读越界，不会漏元素，不会重复处理，零长度是否安全。

错误路径也是一等路径

性能代码常把错误路径视为冷路径，但冷不等于不重要。错误路径可能处理空指针、长度不匹配、分配失败、文件打开失败、CPU 不支持某指令、checksum 不一致、系统调用失败。若错误路径没有测试，程序可能在异常条件下泄露资源、返回未初始化值、破坏 ABI 或误报性能结果。

从控制流角度看，错误路径通常有早返回、跳到清理块、调用不返回函数、设置错误码或抛异常。阅读汇编时要确认错误路径是否仍满足函数出口约定：返回值是否设置，栈是否恢复，`callee-saved` 是否恢复，锁是否释放，已构造对象是否析构。C++ 源码中的 RAII 可以帮助管理资源，但手写汇编或 `goto` 清理逻辑仍要人工审查。

`benchmark` 中也有错误路径。若 `candidate checksum` 失败，`runner` 应把样本标为失败并停止性能结论，而不是继续汇总时间。若 `perf` 权限不足，报告应说明缺少硬件事件，而不是伪造解释。错误路径处理得好，实验系统才可信。

控制流报告模板

一份控制流分析报告建议包含：

1. 函数身份和构建命令。
2. 基本块列表和每个块的入口条件。
3. 边列表，包括 `fallthrough`、`taken branch`、`call`、`return`、`tail call` 和不返回调用。

4. flags 生产者和消费者配对。
5. signed/unsigned 条件码解释。
6. 循环入口、回边、退出和不变量。
7. 短路保护和危险操作的支配关系。
8. switch 的范围检查、索引归一化和表类型。
9. 热路径假设和输入分布。
10. 测试或实验如何覆盖关键边。

这份模板能防止两类错误。第一类是只翻译指令，不证明路径条件。第二类是只谈分支性能，不证明优化合法。控制流章节的核心就是把这两件事放在一起。

控制流和资源管理

控制流不仅决定计算路径，也决定资源生命周期。文件打开后哪些路径关闭，锁加上后哪些路径释放，内存分配后哪些路径释放，对象构造后哪些路径析构，都是控制流问题。C++ 的 RAII 能把很多资源释放绑定到对象生命周期，但手写汇编、C 接口、错误码风格代码和性能 kernel 仍然需要显式审查。

在 CFG 中，资源获取点和释放点应形成支配和后支配关系。简单说，所有成功获取资源的路径，最终都应经过释放或转移所有权的路径。若某个早返回绕过释放，就有泄漏；若某个错误路径重复释放，就有 double free；若异常路径没有 unwind 信息，析构可能不发生。控制流图能把这些问题画出来。

性能优化也可能改变资源路径。把错误检查外提、合并出口、使用尾调用、把分支改成条件移动，都可能影响清理逻辑。优化前后必须证明资源语义不变。对于底层库，资源路径错误往往比热路径慢一点更严重。

间接控制流的安全视角

函数指针、虚函数、跳转表、回调和 PLT 都属于间接控制流。它们的共同特点是目标地址来自数据，而不是指令中固定写死。阅读间接控制流时，要恢复目标集合：可能跳到哪些函数，目标由谁写入，是否经过范围检查或类型检查，是否可能被外部输入影响。

从安全角度，间接控制流需要特别谨慎。越界跳转表可能变成任意跳转；被破坏的函数指针可能劫持执行；虚表指针错误可能调用错误方法；返回地址被覆盖会影响 `ret`。第一册不展开漏洞利用，但要求读者知道：控制数据也是数据，保护它和保护普通内存同样重要。

从性能角度，间接控制流的目标集合越稳定，预测越容易；目标越多、模式越乱，预测越难。报告应区分安全目标恢复和性能可预测性分析。前者问“可能去哪”，后者问“通常去哪”。

控制流重构的审查原则

把多个 `return` 合并成一个出口、把嵌套 `if` 改成早返回、把分支改成查表、把错误路径移到冷函数、把循环拆成主循环和尾循环，都是控制流重构。审查这类改动时，不能只看代码更短或更快，要确认路径语义不变。

审查可按路径集合比较。重构前有哪些输入类别，每类走哪些路径，产生什么结果和副作用；重构后是否一一对应。尤其要检查边界输入、错误输入、零次循环、异常路径、资源清理和短路保护。若路径集合变化，应说明这是有意语义变化，而不是隐藏在重构里的 bug。

控制流重构还可能改变调试和测量。合并出口可能让 `backtrace` 和日志位置变化；冷路径外提可能影响反汇编布局；branchless 改写可能改变 PMU 事件。报告应说明这些变化，不要只写“等价重构”。

路径条件的文字表达

画 CFG 很重要，但文字表达同样重要。每个关键块旁边应写路径条件，例如“到达此块时 `p!=nullptr` 且 `0<=i<n`”，“此块只处理 `n==0`”，“此块是 default 路径，说明索引不在表范围内”。路径条件写清楚后，危险操作是否合法就容易检查。

路径条件也是测试设计的来源。若某块条件是 `p==nullptr`，就需要空指针测试；若某块条件是 `i==n-1`，就需要尾部测试；若某块条件是越界 default，就需要越界输入。控制流分析和测试不应分

离。

控制流章节的综合训练

综合训练可以选择一个带长度检查、空指针检查、循环、早返回和 `switch` 的函数。读者先画源码 CFG，再看 -03 汇编恢复机器 CFG，最后比较两者。报告应指出哪些源码块消失了，哪些路径合并了，哪些保护关系仍然成立，哪些输入分布可能影响性能。

这个训练能把本章所有主题连起来：flags、条件码、基本块、循环、switch、call、间接跳转、branchless 和预测。若读者能完成这样的报告，说明控制流已经从语法知识变成分析能力。

控制流知识向调试和优化迁移

调试时，控制流帮助你判断错误是否可能发生。若崩溃指令被某个检查支配，检查条件就要重新解释；若崩溃路径绕过检查，CFG 能直接指出漏洞；若 `backtrace` 停在错误处理函数，调用边能帮你回到真正来源。没有控制流，调试只能按线性代码猜测。

优化时，控制流帮助你判断改写是否合法和值不值得。把分支变成数据选择，要证明候选计算安全；把循环展开，要证明尾部正确；把 `switch` 改表，要证明范围检查仍在；把错误路径外提，要证明资源清理不变。控制流是正确性和性能之间的共同语言。

控制流章节的毕业标准

完成本章后，读者应能拿一段优化汇编，画出基本块和边，写出路径条件，解释 flags，恢复循环边界，判断 `switch` 表类型，区分直接和间接调用，并说明输入分布如何影响预测。若只能把 `jle` 翻译成“小于等于跳转”，还没有达到毕业标准。

毕业标准不要求读者预测每个 CPU 的精确分支代价，但要求读者知道哪些证据缺失。能说“当前只能证明路径形态，不能证明性能，因为缺少输入分布和测量”，也是合格能力。

控制流分析的最后一道问题

控制流分析结束前，也要问一句：有没有一条路径没有被我解释。未解释路径可能是错误处理、零长度、default、尾部循环、不返回调用、异常路径、间接跳转或编译器分离出的冷块。很多 bug 不在主路径，而在这些被忽略的路径上。

这句话对性能同样有用。若只解释热路径，不解释冷路径为什么冷，报告就缺少输入分布；若只解释 `branchless` 主体，不解释候选值是否安全计算，报告就缺少合法性；若只解释跳转表，不解释范围检查，报告就缺少安全边界。控制流分析的完整性，来自每条重要边都有名字、有条件、有证据。

当读者能把未解释路径主动列出来，说明他已经不再只是翻译跳转指令，而是在审查程序路径。这是进入 ABI、互操作和 benchmark 的重要门槛。

案例：从 C++ 到 CFG 到汇编

综合前面内容，分析一个带循环和条件的函数：

```
extern "C" int sum_positive(int const* p, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        int v = p[i];
        if (v > 0) {
            s += v;
        }
    }
    return s;
}
```

一种分支式汇编：

```
sum_positive:
    xor eax, eax
    test esi, esi
    jle .ldone
```



```
xor ecx, ecx
.Lloop:
    mov edx, dword ptr [rdi + rcx*4]
    test edx, edx
    jle .Lskip
    add eax, edx
.Lskip:
    inc rcx
    cmp ecx, esi
    jl .Lloop
.Ldone:
    ret
```

基本块：

```
B0 entry:
    s = 0
    if n <= 0 goto Bdone
    i = 0
    goto Bloop

Bloop:
    v = p[i]
    if v <= 0 goto Bskip
    goto Badd

Badd:
    s += v
    goto Bskip

Bskip:
    i += 1
    if i < n goto Bloop
    goto Bdone

Bdone:
    return s
```

地址公式：

```
p base      = rdi
i           = rcx
element width = 4
load address = rdi + rcx * 4
```

flags 生产和消费：

生产 flags	消费 flags	语义
testesi,esi	jle.Ldone	signed $n \leq 0$
testedx,edx	jle.Lskip	signed $v \leq 0$
cmpecx,esi	jl.Lloop	signed $i < n$

编译器也可能使用无分支形式：

```
.Lloop:
    mov edx, dword ptr [rdi + rcx*4]
    test edx, edx
    cmovle edx, r8d
    add eax, edx
    inc rcx
    cmp ecx, esi
    jl .Lloop
```

这里假设 r8d 已经是 0。含义是如果 $v \leq 0$ ，把 edx 变成 0，再无条件加到 s。这消除了循环体内部的数据相关分支，但引入 cmovle 和数据依赖。哪种更快取决于正数分布、分支可预测性、CPU 和内存瓶颈。

实验：signed 和 unsigned 条件码

创建 labs/ch12_control_flow/signed_unsigned.cpp:

```
#include <stdint>

extern "C" int less_i32(std::int32_t a, std::int32_t b)
{
    return a < b;
}

extern "C" int less_u32(std::uint32_t a, std::uint32_t b)
{
    return a < b;
}

extern "C" int in_range_i32(std::int32_t x)
{
    return x >= 0 && x <= 4;
}

extern "C" int in_range_u32(std::uint32_t x)
{
    return x <= 4;
}
```

生成:

```
mkdir -p labs/ch12_control_flow/build
clang++ -std=c++20 -O3 -S -masm=intel \
    labs/ch12_control_flow/signed_unsigned.cpp \
    -o labs/ch12_control_flow/build/signed_unsigned.s

clang++ -std=c++20 -O3 -c \
    labs/ch12_control_flow/signed_unsigned.cpp \
    -o labs/ch12_control_flow/build/signed_unsigned.o

objdump -d -Mintel \
    labs/ch12_control_flow/build/signed_unsigned.o \
    > labs/ch12_control_flow/build/signed_unsigned.objdump.txt
```

交付物:

1. 找出 signed 版本使用的 `setcc` 或 `jcc`。
2. 找出 unsigned 版本使用的 `setcc` 或 `jcc`。
3. 用 `a=-1, b=1` 的位模式解释 signed 和 unsigned 结果差异。
4. 对 `in_range_i32(-1)` 和 `in_range_u32(0xffffffff)` 解释 flags 和返回值。
5. 从 `objdump` 中记录至少 6 条机器码字节，并说明每条指令的地址。

实验：画控制流图

创建 labs/ch12_control_flow/cfg.cpp:

```
#include <stdint>

extern "C" int score(int const* p, int n, int threshold)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        int v = p[i];
        if (v > threshold) {
            s += v - threshold;
        } else {
            s -= threshold - v;
        }
    }
    return s;
}
```

分别生成 `-00` 和 `-03`：

```
clang++ -std=c++20 -00 -S -masm=intel cfg.cpp -o cfg_00.s
clang++ -std=c++20 -03 -S -masm=intel cfg.cpp -o cfg_03.s
clang++ -std=c++20 -03 -c cfg.cpp -o cfg.o
objdump -d -Mintel cfg.o > cfg.objdump.txt
```

交付物：

1. 对 `cfg_00.s` 切分基本块，至少标出入口块、循环条件块、then 块、else 块、回边块、退出块。
2. 对 `cfg_03.s` 切分基本块。如果编译器使用 `cmovcc` 或代数化简，要说明控制流如何变成数据流。
3. 画出 `-00` 和 `-03` 的控制流图。可以用文本图、Graphviz 或手绘截图。
4. 对每个 `jcc` 写出：生产 flags 的指令、条件码、taken edge、fallthrough edge。
5. 对循环体里的 `p[i]` 写地址公式。

实验：分支和 branchless 对比

创建 `labs/ch12_control_flow/branch_vs_cmov.cpp`：

```
#include <algorithm>
#include <chrono>
#include <cstdint>
#include <cstdio>
#include <random>
#include <vector>

extern "C" int sum_branch(int const* p, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        if (p[i] > 0) {
            s += p[i];
        }
    }
    return s;
}

extern "C" int sum_mask(int const* p, int n)
{
    int s = 0;
    for (int i = 0; i < n; ++i) {
        int v = p[i];
        int mask = -(v > 0);
        s += v & mask;
    }
    return s;
}

static std::uint64_t now_ns()
{
    auto t = std::chrono::steady_clock::now().time_since_epoch();
    return std::chrono::duration_cast<std::chrono::nanoseconds>(t).count();
}

int main()
{
    constexpr int n = 1 << 24;
    std::vector<int> data(n);

    // pattern 1: predictable, all positive
    std::fill(data.begin(), data.end(), 1);

    volatile int sink = 0;
    auto t0 = now_ns();
    for (int r = 0; r < 20; ++r) sink += sum_branch(data.data(), n);
}
```

```

auto t1 = now_ns();
for (int r = 0; r < 20; ++r) sink += sum_mask(data.data(), n);
auto t2 = now_ns();

std::printf("predictable branch=%llu ns mask=%llu ns sink=%d\n",
    (unsigned long long)(t1 - t0),
    (unsigned long long)(t2 - t1),
    sink);

// pattern 2: random signs
std::mt19937 rng(123);
for (int& x : data) {
    x = (rng() & 1) ? 1 : -1;
}

t0 = now_ns();
for (int r = 0; r < 20; ++r) sink += sum_branch(data.data(), n);
t1 = now_ns();
for (int r = 0; r < 20; ++r) sink += sum_mask(data.data(), n);
t2 = now_ns();

std::printf("random branch=%llu ns mask=%llu ns sink=%d\n",
    (unsigned long long)(t1 - t0),
    (unsigned long long)(t2 - t1),
    sink);
}

```

构建并观察汇编：

```

clang++ -std=c++20 -O3 -march=native \
    branch_vs_cmov.cpp -o branch_vs_cmov

clang++ -std=c++20 -O3 -march=native -S -masm=intel \
    branch_vs_cmov.cpp -o branch_vs_cmov.s

./branch_vs_cmov

perf stat -e branches,branch-misses ./branch_vs_cmov

```

交付物：

1. 找出 `sum_branch` 和 `sum_mask` 的汇编核心循环。
2. 说明编译器是否自动向量化。如果向量化了，额外用下面参数生成标量版本比较：

```
-fno-vectorize -fno-slp-vectorize
```

3. 比较 `predictable` 和 `random` 两种输入的时间。
4. 记录 `branches` 和 `branch-misses`。如果系统不允许访问 `perf` 事件，说明权限问题并至少报告运行时间。
5. 写出结论：什么时候分支更好，什么时候 `mask/branchless` 更好，本实验不能证明什么。

实验：switch 和跳转表

创建 `labs/ch12_control_flow/switch_table.cpp`：

```

#include <stdint>

extern "C" int f0(int x) { return x + 3; }
extern "C" int f1(int x) { return x + 5; }
extern "C" int f2(int x) { return x + 8; }
extern "C" int f3(int x) { return x + 13; }
extern "C" int f4(int x) { return x + 21; }

extern "C" int dense_dispatch(int op, int x)
{
    switch (op) {

```

```

        case 0: return f0(x);
        case 1: return f1(x);
        case 2: return f2(x);
        case 3: return f3(x);
        case 4: return f4(x);
        default: return -1;
    }
}

extern "C" int sparse_dispatch(int op, int x)
{
    switch (op) {
        case 1: return f0(x);
        case 17: return f1(x);
        case 101: return f2(x);
        default: return -1;
    }
}

```

生成：

```

clang++ -std=c++20 -O3 -fno-inline -S -masm=intel \
    switch_table.cpp -o switch_table.s

clang++ -std=c++20 -O3 -fno-inline -c \
    switch_table.cpp -o switch_table.o

objdump -dr -Mintel switch_table.o > switch_table.objdump.txt
readelf -r switch_table.o > switch_table.reloc.txt

```

交付物：

1. 判断 `dense_dispatch` 是否使用跳转表。
2. 判断 `sparse_dispatch` 是否使用比较链或其他策略。
3. 找出范围检查指令，并解释为什么可能使用 `unsigned` 条件。
4. 如果出现 `jmpq word ptr [rip+...+rax*8]`，写出表项地址公式。
5. 在 `readelf -r` 中找出和跳转表相关的重定位项，说明目标文件阶段为什么还需要重定位。

实验：call/ret 和返回地址

创建 `labs/ch12_control_flow/call_ret.cpp`：

```

#include <cstdio>

extern "C" int twice(int x)
{
    return x * 2;
}

extern "C" int caller(int x)
{
    int y = twice(x);
    return y + 1;
}

int main()
{
    std::printf("%d\n", caller(21));
}

```

构建带调试信息的版本：

```

clang++ -std=c++20 -O0 -g call_ret.cpp -o call_ret_00
objdump -d -Mintel call_ret_00 > call_ret_00.objdump.txt

```

用 GDB 观察：


```

gdb ./call_ret_00
(gdb) break caller
(gdb) run
(gdb) disassemble /r caller
(gdb) info registers rip rsp rbp
(gdb) x/4gx $rsp
(gdb) stepi
(gdb) info registers rip rsp rbp
(gdb) x/4gx $rsp

```

交付物：

1. 找到 `call` 指令地址和下一条指令地址。
2. 在执行 `call` 前记录 `rsp`。
3. 在进入 `twice` 后记录 `rsp` 和栈顶 8 字节。
4. 证明栈顶保存的是返回地址。
5. 执行到 `ret` 前后，记录 `rip` 和 `rsp` 的变化。
6. 对 -03 再构建一次，观察是否发生内联或尾调用，并解释差异。

本章作业

习题与作业

作业 1：条件码推导表

给定下面位模式：

```

a = 0xffffffff
b = 0x00000001

```

完成表格：

- 把 `a`、`b` 分别解释成 `std::int32_t` 和 `std::uint32_t`。
- 对 `cmpa, b` 写出 signed 小于、signed 大于、unsigned 小于、unsigned 大于的真假。
- 说明 `setl`、`setg`、`setb`、`seta` 各会产生 0 还是 1。
- 用文字解释为什么同一个位模式会得到不同结果。

作业 2：控制流图恢复

对下面汇编恢复控制流图和可能 C-like 伪代码：

```

foo:
    xor eax, eax
    test esi, esi
    jle .ldone
    xor ecx, ecx
.Lloop:
    mov edx, dword ptr [rdi + rcx*4]
    cmp edx, 10
    jl .lskip
    add eax, edx
.Lskip:
    inc rcx
    cmp ecx, esi
    jl .Lloop
.Ldone:
    ret

```

要求：

1. 切分基本块。
2. 给每条边标注条件。
3. 说明每个 `jcc` 使用 signed 还是 unsigned 语义。

4. 写出数组访问地址公式。
5. 给出至少一种可能的 C++ 源码。

作业：合流点和 phi 思维

编写一个函数，包含 `if/else` 后继续执行公共计算，例如两个分支分别计算临时值，最后统一乘以常数或参与第二次判断。分别生成 `-00` 和 `-03` 汇编，提交不少于 1800 字分析。要求：

- 画出 `then`、`else`、`join` 三类基本块。
- 说明每条前驱边进入 `join` 时，哪个寄存器或栈槽保存同一个逻辑变量。
- 用伪 `phi` 表示合流点的值选择。
- 如果 `-03` 使用 `cmovcc` 或代数化简，说明控制合流如何转成数据选择。
- 判断 `join` 块是否被两个分支共同支配，哪些检查支配后续内存访问。

作业 3: switch 反推

给定：

```
cmp edi, 5
ja .Ldefault
mov eax, edi
jmp qword ptr [rip + .Ltable + rax*8]
```

要求：

1. 解释为什么 `ja` 可以同时过滤负数和大于 5 的值。
2. 写出跳转表索引公式。
3. 假设 `.Ltable=0x5000`，`edi=4`，写出表项地址。
4. 说明这是直接跳转还是间接跳转。
5. 说明这种结构对分支预测有什么潜在影响。

作业：数据表、跳转表和重定位

写两个 `switch` 函数：第一个每个 `case` 只返回不同常量，第二个每个 `case` 调用不同的 `noinline` 函数。使用 `-03-fno-inline` 生成目标文件和可执行文件，提交不少于 2200 字报告。要求：

- 判断第一个函数是否生成数据表，表项宽度是多少，表项如何被使用。
- 判断第二个函数是否生成跳转表，表项是地址还是偏移。
- 对每个表写出范围检查、索引归一化和表项地址公式。
- 使用 `objdump-dr` 和 `readelf-r` 找出重定位记录。
- 对比目标文件和最终可执行文件中的表内容差异。

作业 4: call/ret 地址审计

从任意一个你自己编译的 `-00` 程序选择一个非内联函数调用，提交：

- `objdump-d-Mintel` 中调用者和被调用者的反汇编。
- `call` 的机器码字节。
- `call` 指令地址、下一条指令地址、目标函数地址。
- GDB 中 `call` 前后的 `rsp`、`rip` 和栈顶值。
- 用具体数值证明返回地址被压栈。

作业：普通调用、不返回调用和尾调用

编写三个小函数组：普通调用后继续加一；调用 `std::abort` 或自定义 `[[noreturn]]` 函数；函数末尾直接返回另一个函数结果。生成 `-00` 和 `-03` 汇编，提交不少于 1800 字报告。要求：

- 说明普通 `call` 在当前函数 CFG 中为什么不是终点。

- 找出不返回调用后的控制流形态，观察是否有 `ud2` 或不可达块。
- 判断尾调用是否生成 `jmp`，并解释为什么没有新返回地址入栈。
- 若尾调用没有发生，分析可能的 ABI、栈帧、清理或返回值原因。
- 用调用图和单函数 CFG 分别画出三类调用的差异。

作业 5：分支性能小报告

基于分支和 `branchless` 对比实验写一份 1800 字以上报告，必须包含：

- CPU 型号、编译器版本、编译参数。
- 输入数据分布。
- 汇编核心循环。
- 运行时间表。
- `branch` 和 `branch-miss` 数据；若无法获得，解释原因。
- 是否发生向量化、内联或 `branchless` 改写；必须用汇编证明。
- 至少三种输入分布：全真或全假、随机、偏斜或排序。
- 多次运行样本或交错运行设计，避免只用一次计时下结论。
- 结论和限制：不要只写“`branchless` 更快”或“`branch` 更快”，必须说明条件。

评分标准

本章作业按下面标准评价：

1. 是否能从 `flags` 生产指令追踪到 `flags` 消费指令。
2. 是否能正确区分 `signed` 和 `unsigned` 条件码。
3. 是否能切分基本块并画出完整边。
4. 是否能把地址公式、循环变量和内存访问联系起来。
5. 是否能用真实工具输出支持自己的结论。
6. 是否明确区分 C++ 语言语义、ISA 语义和微架构性能现象。

本章小结

本章把单条指令扩展成了程序路径。你应该把控制流理解成三层：

```
source level:
    if / while / for / switch / function call

ISA level:
    cmp/test -> flags -> jcc/setcc/cmov
    call -> push return address + jump
    ret -> pop return address + jump
    jmp -> direct or indirect rip assignment

microarchitecture level:
    prediction, speculation, BTB, RSB, misprediction recovery
```

从性能工程角度，控制流不是孤立问题。它和输入分布、数据布局、内存访问、向量化、代码布局、前端带宽和预测器状态交织在一起。下一章进入 System V AMD64 ABI，把函数调用的参数、返回值、寄存器保存、栈对齐和结构体传递规则讲清楚。到那时，`call` 和 `ret` 就不只是跳转，而会成为二进制接口的一部分。

第 9 章

System V AMD64 ABI

问题与目标

前面几章已经建立了阅读函数内部汇编的基础：整数指令、地址计算、条件跳转、循环、`call` 和 `ret`。但真实程序不是一个函数孤立运行，而是由大量函数、目标文件、静态库、动态库、运行时和操作系统共同组成。函数之间怎样交接参数、返回值、寄存器和栈状态，是本章必须解决的问题。

换句话说：一个 C++ 文件里编译出来的函数，为什么能被另一个 C 文件调用？一个手写汇编函数，为什么能正确返回给 C++ 调用者？动态库里的 `printf`，为什么知道实参放在哪里？异常展开、调试器栈回溯和 profiler 采样，为什么能够沿着调用链恢复上下文？

答案是 **ABI** (Application Binary Interface)。ABI 是二进制层面的契约。它规定：

- 函数参数放在哪些寄存器或栈位置。
- 返回值放在哪里。
- 哪些寄存器由调用者保存，哪些由被调用者保存。
- 栈在调用点如何对齐。
- 结构体、浮点、向量、变参函数如何传递。
- 函数入口、函数尾声、调试信息和异常展开如何约定。

本章讨论 Linux、macOS 等类 Unix x86-64 平台常用的 System V AMD64 ABI。Windows x64 使用另一套 ABI，参数寄存器、shadow space 和保存规则都不同。本册默认以 System V AMD64 为上下文，除非另有明确说明。

核心理想

ISA 告诉 CPU 一条指令怎么执行；ABI 告诉不同二进制代码如何互相调用。读函数级汇编时，如果不懂 ABI，就无法可靠判断参数、返回值、栈槽、寄存器保存和跨语言边界。

ABI 位于哪一层

学习 ABI 时，第一步不是背寄存器表，而是把它放到正确层次。源代码层有函数声明、类型、重载、命名空间、引用、对象生命周期和异常语义；ISA 层有寄存器、内存操作数、`call`、`ret` 和机器码编码；操作系统层有进程、虚拟地址空间、共享库映射、信号和系统调用。ABI 处在这些层之间，把源码中的“调用一个函数”约束成二进制模块可以共同遵守的接口。

因此，ABI 不是 C++ 标准的一部分。C++ 标准会描述表达式求值、对象生命周期、函数重载、异常传播等语言规则，但不会规定 Linux x86-64 上第一个整数参数一定放在 `rdi`。同样，ABI 也不是 CPU 硬件自动强制的规则。CPU 并不知道 `rbx` 是 callee-saved，也不会在函数返回时检查 `rsp` 是否恢复。ABI 的力量来自编译器、汇编器、链接器、运行时库、调试器和程序员共同遵守。

这一区分能解释许多初学者疑惑。一个破坏 `rbx` 的手写汇编函数，CPU 会照样执行，因为从 ISA 角度看 `mov rbx, ...` 完全合法；但从 ABI 角度看，它可能破坏调用者仍然依赖的值。一个栈未对齐的函数，简单算术测试可能通过，因为没有指令触及对齐敏感路径；但一旦调用库函数、使用某些 SIMD 指令、进入不同优化版本，程序就可能崩溃。ABI 错误的特点往往不是“立即非法”，而是“破坏了其他代码合理依赖的前提”。

心智模型

可以把一个函数调用看成四层同时发生：

- 源代码层：调用者认为自己在调用某个有类型的函数。
- ABI 层：调用者把参数放到约定位置，被调用者从约定位置读取。

- ISA 层：`call` 写入返回地址并跳转，`ret` 弹出返回地址并跳回。
- 工具层：调试器、profiler 和异常展开器依赖符号、栈帧和展开信息恢复调用链。

只看其中任意一层，都容易误判真实程序。

本章应该怎样学

ABI 很容易被学成表格背诵：前六个整数参数对应哪些寄存器，返回值放在哪里，哪些寄存器需要保存。表格必须熟悉，但本章的目标不止于此。你应该能从一个 C++ 函数签名出发，推导函数入口时每个参数的位置；能从一段反汇编出发，判断它是否遵守 ABI；能解释为什么某个看似多余的 `sub rsp, 8` 实际是在维护栈对齐；能看出某个大结构体返回函数为什么多了隐藏参数；还能把这些结论写成可复现实验报告。

学习时建议使用三种笔记。第一种是入口状态笔记：函数刚进入时，哪些寄存器和栈槽有 ABI 含义。第二种是保存责任笔记：函数体修改了哪些 callee-saved 寄存器，是否恢复；调用别的函数前是否保护了仍然需要的 caller-saved 值。第三种是边界证据笔记：结论来自函数签名、ABI 规则、反汇编、调试器单步，还是来自对编译器行为的推测。

本章不会把完整 psABI 规范逐条翻译。规范里的分类算法、对象文件细节、动态链接、线程局部存储、异常展开和向量扩展都很庞大。第一册采用经典教材式路径：先掌握函数调用最常见、最容易出错、最能支撑后续实验的规则；遇到边界情况时明确告诉你应回到正式 ABI 文档和编译器输出验证，而不是凭记忆硬猜。

ABI 为什么必须存在

考虑两个源文件：

```
// add.cpp
extern "C" int add2(int a, int b)
{
    return a + b;
}
```

```
// main.cpp
#include <cstdio>

extern "C" int add2(int, int);

int main()
{
    std::printf("%d\n", add2(10, 32));
}
```

它们可以分开编译：

```
clang++ -std=c++20 -O3 -c add.cpp -o add.o
clang++ -std=c++20 -O3 -c main.cpp -o main.o
clang++ add.o main.o -o app
```

`main.o` 编译时并不知道 `add2` 的机器码是什么，但它知道怎样调用 `add2`：

```
edi = 10
esi = 32
call add2
read return value from eax
```

`add.o` 编译时也不知道谁会调用它，但它知道入口时参数在哪里：

```
on entry:
    edi = first int argument
    esi = second int argument

on return:
    eax = int return value
```


这就是 ABI 的意义。调用者和被调用者不需要共享源代码，不需要由同一个编译器同时优化，只要遵守同一套二进制定约，就可以链接和运行。

心智模型

函数调用不是“跳到另一个 C++ 函数”。机器层面更像一次协议交接：

```
caller:
    arrange arguments
    preserve caller-owned live values if needed
    align stack
    call callee
    read return value

callee:
    assume arguments are in ABI-defined locations
    preserve callee-saved registers it modifies
    restore stack
    ret
```

从源码函数调用到二进制调用点

源代码中的函数调用通常只有一行：

```
long r = f(a, b, c);
```

但在二进制层，调用点至少要完成五件事。第一，计算每个实参表达式，得到要传递的值或地址。第二，按照 ABI 把这些值放入通用寄存器、浮点寄存器或栈参数区。第三，保护调用后还要继续使用、但可能被被调用者破坏的 caller-saved 寄存器值。第四，在执行 `call` 前维护栈对齐。第五，在返回后从约定寄存器或内存位置读取返回值。

被调用者看到的不是源代码实参表达式，而是调用点留下的机器状态。对它来说，入口状态是事实起点：`rdi` 里是什么，`xmm0` 里是什么，`rsp` 指向哪里，哪些栈槽属于参数，哪些寄存器必须恢复。被调用者不能假设调用者用哪个源文件写成，不能假设调用者保留了局部变量名字，也不能假设调用者和自己使用同一种优化级别。

这也是单独编译能够成立的原因。编译 `main.cpp` 时，编译器可能只看见 `extern "C" long f(long, long, long)`；这样的声明。它不知道 `f` 的函数体，但它知道函数签名和目标 ABI，于是能生成调用序列。编译 `f.cpp` 时，编译器不知道所有调用点，却知道入口处第一个 `long` 参数应从 `rdi` 读取。链接器最后只需要把符号引用接到符号定义，调用双方已经在 ABI 层达成一致。

真实工程里，ABI 边界往往就是编译优化的边界。编译器在一个函数内部可以重排指令、删除变量、内联小函数、把局部值放进任意临时寄存器；但当它生成一个外部可见函数或一个不能内联的调用点时，就必须把自由度收束成 ABI 规定的形状。理解这一点，才能解释为什么同一个计算在内联后可能完全看不见函数调用，而一旦跨动态库边界，参数传递、返回值、PLT 跳转和寄存器保存又全部回来了。

常见误区

不要把“源代码里有一个函数调用”和“机器码里一定有一条 `call` 指令”画等号。优化器可能内联、常量折叠、尾调用优化，甚至删除整个调用。ABI 规则约束的是实际存在的二进制调用边界；如果边界被优化掉，就不再有那次普通 ABI 调用。

从函数签名推导入口状态

读函数汇编时，可以按下面顺序推导入口状态。第一，看目标平台和调用约定，确认当前是 System V AMD64，而不是 Windows x64、内核系统调用约定或某个编译器扩展约定。第二，看函数返回类型，先判断是否可能有隐藏 `sret` 指针。第三，按从左到右的显式参数顺序分类：整数、指针、引用通常进入通用寄存器序列；标量浮点通常进入 `xmm` 序列；聚合类型要看大小、字段类别、对齐和 C++ 平凡性。第四，记录寄存器容量耗尽后的栈参数位置。第五，把返回值位置写在入口笔记旁边，因为很多函数会围绕返回寄存器组织数据流。

对一个普通函数：

```
extern "C" double f(long a, double b, int c, float d);
```

推导过程不是“第一个参数 `rdi`，第二个参数 `rsi`”这么简单。应该分通道计数：

```
long a   -> first integer-class argument -> rdi
double b -> first floating-class argument -> xmm0
int c    -> second integer-class argument -> esi
float d  -> second floating-class argument -> xmm1
return double -> xmm0 on return
```

注意返回时 `xmm0` 会从参数含义变成返回值含义。寄存器的语义不是永久标签，而是由时间点、ABI 和指令数据流共同决定。

对一个可能有隐藏返回指针的函数：

```
struct Big3 { long a; long b; long c; };
extern "C" Big3 make(long x, long y);
```

入口推导要先处理返回类型。若该类型走 `sret`，调用者会把返回对象地址作为隐藏参数放在第一个通用寄存器位置，显式参数整体后移：

```
rdi = hidden pointer to caller-provided Big3 storage
rsi = x
rdx = y
return convention may also copy hidden pointer to rax
```

如果你跳过返回类型，直接把 `rdi` 当作 `x`，后面的所有解释都会错。这类错误在逆向、调试崩溃和手写汇编互操作中非常常见。

核心思想

ABI 推导的基本顺序是：先定目标 ABI，再看返回类型，再分类参数，再分配寄存器和栈槽，最后结合指令数据流验证。不要只凭第一个出现的寄存器名猜源码变量。

先掌握最常见的整数参数

System V AMD64 ABI 中，前 6 个整数或指针类参数通常放在这些寄存器里：

参数序号	64 位寄存器	常见 32 位整数视图
第 1 个	<code>rdi</code>	<code>edi</code>
第 2 个	<code>rsi</code>	<code>esi</code>
第 3 个	<code>rdx</code>	<code>edx</code>
第 4 个	<code>rcx</code>	<code>ecx</code>
第 5 个	<code>r8</code>	<code>r8d</code>
第 6 个	<code>r9</code>	<code>r9d</code>

整数返回值通常放在 `rax`；如果返回 `int`，有效结果在 `eax`。

C++：

```
extern "C" int sum6(int a, int b, int c, int d, int e, int f)
{
    return a + b + c + d + e + f;
}
```

可能汇编：

```
sum6:
    lea eax, [rdi + rsi]
    add eax, edx
    add eax, ecx
    add eax, r8d
    add eax, r9d
    ret
```

入口状态：

```
edi = a
esi = b
edx = c
ecx = d
r8d = e
r9d = f
```

返回约定：

```
eax = a + b + c + d + e + f
ret
```

常见误区

`rcx` 在 System V ABI 中是第 4 个整数参数；在 Windows x64 ABI 中，`rcx` 是第 1 个整数参数。看资料或反汇编时必须先确认目标 ABI。

窄整数参数和高位问题

很多函数参数不是 64 位整数，而是 `bool`、`char`、`short`、`int`、枚举、小整数 typedef。它们在 ABI 层仍然占用通用寄存器位置，但有效位宽和高位解释需要小心。读汇编时，如果只看到 `rdi`、`rsi` 这类 64 位寄存器名，就直接把参数当成完整 64 位值，很容易误判。

例如：

```
extern "C" int add_char_signed(signed char a, signed char b)
{
    return int(a) + int(b);
}

extern "C" int add_char_unsigned(unsigned char a, unsigned char b)
{
    return int(a) + int(b);
}
```

两个函数都可能从第一个、第二个通用参数寄存器接收值，但函数体会用不同扩展方式解释低 8 位：

```
signed char:
    sign-extend low 8 bits before arithmetic

unsigned char:
    zero-extend low 8 bits before arithmetic
```

你可能看到 `movsx`、`movsbl`、`movzx`、`movzbl` 这类指令，也可能看到编译器利用 32 位写入清零高位的性质消除显式扩展。关键是：参数寄存器的物理宽度不等于源语言参数的有效宽度。源语言类型决定低多少位有效，以及这些位应该按有符号还是无符号解释。

调用者和被调用者都不能乱猜高位

ABI 会规定某些小整数参数如何传递，但不同类型、不同编译器优化和不同调用边界下，高位是否有确定值并不总能靠肉眼猜。安全的读法是：被调用者若需要 `signedchar` 语义，就看它是否从低 8 位符号扩展；若需要 `unsignedchar` 语义，就看它是否零扩展或只使用低 8 位。调用者传参时也应按函数原型准备实参，而不是把任意 64 位垃圾值塞进 `rdi` 后指望被调用者总能忽略高位。

对 `int` 参数，常见代码会使用 `edi`、`esi`、`edx` 等 32 位视图。x86-64 中写 32 位寄存器会清零对应 64 位寄存器高 32 位，所以调用点常通过写 `edi` 来自自然形成零扩展到 `rdi` 的效果。但这不意味着所有有符号语义都被取消。被调用者做 32 位算术时，溢出、比较、返回值解释仍然由源语言和指令决定。

看下面两个函数：

```
extern "C" bool is_negative_i32(int x)
{
```

```

    return x < 0;
}

extern "C" bool high_bit_u32(unsigned x)
{
    return (x & 0x80000000u) != 0;
}

```

二者都可能检查同一个最高位，但语义不同。第一个是有符号比较，第二个是无符号位测试。汇编里可能都涉及 `edi`，但不能只凭寄存器名判断源类型。要结合条件码、比较指令、返回值构造和函数签名。

返回窄整数也要看有效位

返回 `bool`、`char`、`short` 时，结果通常位于 `al` 或 `ax` 的低位部分；返回 `int` 时看 `eax`。调用者若把返回值继续提升为 `int` 或 `long`，会按源类型进行零扩展或符号扩展。手写汇编返回窄类型时，最稳妥的做法是明确设置返回寄存器低位，并在需要时清理高位，避免让调用者后续优化遇到不明确状态。

例如返回 `bool` 的函数，建议让 `eax` 为 0 或 1：

```

xor eax, eax
test edi, edi
setne al
ret

```

这样不仅低 8 位正确，高位也被清零，调用者把结果当作整数继续使用时更容易得到稳定、可读的机器码。编译器常生成类似模式。

常见误区

读 ABI 时不要把“参数放在某个 64 位寄存器”理解成“整个 64 位都是该参数的语义值”。窄整数的有效位宽、符号扩展、零扩展和返回低位都要结合源类型和指令判断。

超过 6 个整数参数：栈上传参

第 7 个及以后的整数或指针类参数通常通过栈传递。看例子：

```

extern "C" long sum8(long a, long b, long c, long d,
                    long e, long f, long g, long h)
{
    return a + b + c + d + e + f + g + h;
}

```

一种可能汇编：

```

sum8:
    lea rax, [rdi + rsi]
    add rax, rdx
    add rax, rcx
    add rax, r8
    add rax, r9
    add rax, qword ptr [rsp + 8]
    add rax, qword ptr [rsp + 16]
    ret

```

函数刚进入时，`rsp` 指向返回地址：

on entry to sum8:

```

[rsp + 0]  return address
[rsp + 8]  7th argument: g
[rsp + 16] 8th argument: h

rdi = a
rsi = b
rdx = c
rcx = d

```

```
r8 = e
r9 = f
```

为什么第 7 个参数是 `[rsp+8]` 而不是 `[rsp]`? 因为 `call` 已经把返回地址压到了栈顶。被调用函数入口处, 栈顶首先是返回地址, 栈上传递的第一个参数在返回地址上方 8 字节。

深入理解

如果被调用函数一进来就 `push rbp` 或 `sub rsp, ...`, 参数相对 `rsp` 的偏移会改变。因此读汇编时必须知道当前 `rsp` 已经移动了多少。使用帧指针时, 编译器常用 `rbp` 作为稳定基准。

栈参数不是局部变量

栈上传参容易和局部变量、spill slot 混在一起。第 7 个及之后的参数位于调用者准备好的调用帧区域中; 局部变量和临时溢出槽通常位于被调用者自己调整 `rsp` 后得到的区域中。二者都在栈上, 但所有权和解释方式不同。

从被调用者入口看, `[rsp+8]` 是第一个栈参数; 从调用者视角看, 它是在执行 `call` 之前就准备好的一个输出区域。执行 `call` 后, 硬件把返回地址写到当前 `rsp - 8`, 于是被调用者入口的 `rsp` 指向返回地址。这个返回地址不是“参数的一部分”, 却夹在栈顶和栈参数之间, 所以读偏移时必须把它算进去。

如果被调用者建立帧指针:

```
push rbp
mov rbp, rsp
```

那么入口时的返回地址从 `[rsp]` 变成 `[rbp+8]`, 第一个栈参数从 `[rsp+8]` 变成 `[rbp+16]`。很多低优化级反汇编都呈现这种形式。初学者常把 `[rbp+16]` 误认为局部变量, 因为它也是以 `rbp` 为基址; 事实上, 正偏移通常指向调用者方向, 负偏移通常指向被调用者自己的局部区域。

常见误区

不要看到 `[rbp+16]` 就机械说“栈上的局部变量”。在典型帧指针布局里, `rbp` 正偏移常常是返回地址和栈参数, 负偏移才更常见于局部变量、保存寄存器和 spill slot。

谁负责清理栈参数

在 System V AMD64 常见调用约定下, 调用者负责为栈参数准备空间, 并在调用返回后恢复自己的 `rsp`。被调用者读取栈参数, 但通常不会把调用者压入的参数弹掉。这个规则让被调用者不需要知道调用者在调用点周围还安排了哪些临时栈空间, 也让变参函数、尾调用优化和编译器自己的调用序列更容易统一处理。

例如调用一个有 8 个 `long` 参数的函数时, 调用者可能把第 7、第 8 个参数写到栈上, 然后执行 `call`。返回后, 调用者把 `rsp` 调回调用前状态。被调用者只保证自己返回时 `rsp` 回到入口约定状态, 也就是仍然指向返回地址所在的那一层, 随后 `ret` 会弹出返回地址。

这个规则也解释了为什么手写汇编函数不能随意 `pop` 栈参数。如果你在被调用者里把栈参数弹掉, `ret` 可能从错误位置取返回地址; 即使没有立刻崩溃, 调用者恢复 `rsp` 时也会基于错误前提继续破坏栈。

浮点和向量参数

浮点参数走 SSE/AVX 寄存器通道。常见规则:

- `float`、`double` 参数通常放在 `xmm0` 到 `xmm7`。
- 浮点返回值通常放在 `xmm0`。
- 整数参数寄存器序列和浮点参数寄存器序列分别计数。

C++:

```
extern "C" double mix_args(int a, double b, int c, float d)
{
```



```
return double(a + c) + b + double(d);
}
```

入口位置通常是：

```
edi = a
xmm0 = b
esi = c
xmm1 = d
```

注意 `c` 是第 2 个整数类参数，所以在 `esi`，不是 `edx`。因为 `doubleb` 消耗的是浮点寄存器 `xmm0`，不消耗整数参数寄存器。

可能汇编片段：

```
mix_args:
    add edi, esi
    cvtsi2sd xmm2, edi
    addsd xmm2, xmm0
    cvtss2sd xmm1, xmm1
    addsd xmm1, xmm2
    movapd xmm0, xmm1
    ret
```

返回值在 `xmm0`。这里的 `cvtsi2sd` 把整数转换成 `double`，`cvtss2sd` 把 `float` 扩展成 `double`。

常见误区

`xmm0` 既可能是第 1 个浮点参数，也可能是浮点返回值。函数入口和函数返回是两个不同时间点。读汇编时要区分“入口时 `xmm0` 是参数”和“返回时 `xmm0` 是结果”。

浮点通道和整数通道混用时的常见误判

混合参数最容易出错的地方，是把参数序号和寄存器序号混成一条线。System V AMD64 中，通用寄存器通道和浮点寄存器通道分别计数。一个 `double` 参数不会占掉 `rsi`，一个 `int` 参数也不会占掉 `xmm1`。因此第 4 个源代码参数不一定在第 4 个通用寄存器，也不一定在 `xmm3`。

例如：

```
extern "C" double f(double a, long b, double c, long d, double e);
```

入口通常应写成：

```
xmm0 = a
rdi = b
xmm1 = c
rsi = d
xmm2 = e
```

如果你只按源代码位置数“第一个、第二个、第三个”，就会把 `c` 错写成 `xmm2`，把 `d` 错写成 `rcx`。正确做法是给每个类别单独维护计数器。写 ABI 推导笔记时，可以画两列：

```
integer channel:
    b -> rdi
    d -> rsi

floating channel:
    a -> xmm0
    c -> xmm1
    e -> xmm2
```

这种写法比单列表更不容易错。

向量参数只做第一册边界认识

SSE、AVX 向量类型也会进入 ABI 讨论，例如 `__m128`、`__m256`、编译器向量扩展类型。它们的传递规则涉及向量寄存器、对齐、目标 ISA、编译参数和 psABI 扩展。第一册暂不展开 SIMD 编程，但要知道一个事实：向量参数不是普通指针，也不是普通结构体。它们可能直接走 `xmm` 或 `ymm` 寄存

器，调用者和被调用者还必须在同一目标 ISA 假设下编译。

如果你在 C++ 和手写汇编边界传递向量对象，必须确认：

- 函数声明使用的向量类型是什么。
- 编译参数是否启用了对应 ISA。
- 参数和返回值是否走向量寄存器。
- 是否存在 AVX 到 SSE 过渡相关的清理要求。
- 栈上保存向量时是否满足对齐要求。

后续第二册会系统讲 SIMD。当前阶段的原则是：公开 ABI 和手写汇编边界优先使用普通标量、指针和明确布局的平凡结构体；若必须传向量，先用编译器生成一个小例子并反汇编验证。

返回值位置

常见返回值约定：

返回类型	返回位置	说明
<code>int</code> 、 <code>unsigned</code>	<code>eax</code>	写 32 位会清零 <code>rax</code> 高 32 位
<code>long</code> 、指针	<code>rax</code>	64 位整数或地址
<code>double</code> 、 <code>float</code>	<code>xmm0</code>	标量浮点返回
两个整数机器字	<code>rax</code> 、 <code>rdx</code>	常见小结构体返回形式
大结构体	隐藏 <code>sret</code> 指针	调用者提供返回对象地址

小结构体例子：

```
struct Pair {
    long x;
    long y;
};

extern "C" Pair make_pair(long a, long b)
{
    return Pair{a + 1, b + 2};
}
```

可能汇编：

```
make_pair:
    lea rax, [rdi + 1]
    lea rdx, [rsi + 2]
    ret
```

返回时：

```
rax = result.x
rdx = result.y
```

调用者按 ABI 知道这两个寄存器共同组成返回对象。

大结构体返回：隐藏 `sret` 指针

大结构体通常不能只靠 `rax` 和 `rdx` 返回。调用者会先分配好返回对象空间，然后把地址作为隐藏参数传给被调用者。

C++：

```
struct Big {
    long a;
    long b;
    long c;
};

extern "C" Big make_big(long x)
{
```

```
return Big(x, x + 1, x + 2);
}
```

逻辑上像这样：

```
extern "C" Big make_big(long x);
```

机器层面更像：

```
extern "C" void make_big_hidden(Big* out, long x);
```

可能汇编：

```
make_big:
    mov rax, rdi
    mov qword ptr [rdi], rsi
    lea rcx, [rsi + 1]
    mov qword ptr [rdi + 8], rcx
    add rsi, 2
    mov qword ptr [rdi + 16], rsi
    ret
```

入口状态：

```
rdi = hidden pointer to caller-allocated result object
rsi = x
```

返回状态：

```
memory at rdi:
    [rdi + 0] = x
    [rdi + 8] = x + 1
    [rdi + 16] = x + 2

rax = rdi
```

这解释了一个重要现象：源代码里看起来只有一个参数 `x`，但汇编入口时 `rdi` 可能不是 `x`，而是隐藏返回对象指针。真正的 `x` 变成了 `rsi`。

核心思想

看到函数第一个参数“不对劲”时，不要马上怀疑编译器。先检查返回类型是否是大结构体，是否存在隐藏 `sret` 指针。

返回值不是只有一个寄存器问题

很多教材在介绍返回值时只说“整数返回值放在 `rax`，浮点返回值放在 `xmm0`”。这句话对最常见标量情况足够，但不足以解释真实 C++ 程序。C++ 返回值可能是标量、指针、引用、小结构体、大结构体、含浮点字段的聚合、非平凡对象，也可能涉及返回值优化、复制省略、移动构造和析构。ABI 只能描述二进制边界上对象如何被交接，不能替代 C++ 对象语义。

从 ABI 角度看，返回值大致有三种形态。第一种是寄存器返回：一个 `int`、一个指针、一个 `double`，或者能拆进一两个 `eightbyte` 的小聚合，直接用 `rax`、`rdx` 或 `xmm0`、`xmm1` 交接。第二种是内存返回：调用者提供一块结果对象存储，被调用者把结果写进去，也就是隐藏 `sret` 指针。第三种是优化后没有普通返回边界：函数被内联，返回对象被直接构造到调用者后续使用的位置，汇编里可能看不到独立的返回动作。

这三种形态要和源码语义分开。源码里 `return Big{...}`；仍然是返回一个 `Big` 对象；机器层面可能表现为写 `[rdi]`、`[rdi+8]`、`[rdi+16]`。源码里返回一个两字段结构体，机器层面可能表现为同时写 `rax` 和 `rdx`。如果函数被内联，源码返回值甚至可能只表现为调用者内部的几条算术指令。

分析返回值时，建议写下四个问题：

1. 返回类型在 ABI 中属于标量、可寄存器传递的小聚合，还是需要内存返回的对象。
2. 返回路径上哪些寄存器或内存地址被写入。

3. 调用者在返回后从哪里读取结果。
4. C++ 对象语义是否引入构造、析构、异常或非平凡复制移动。

这些问题比单独背“**rax** 返回整数”更可靠。尤其在调试跨语言接口时，调用者和被调用者若对返回类型理解不同，结果可能不是链接失败，而是静默破坏内存。例如 C++ 头文件声明返回小结构体，汇编实现却按大结构体隐藏指针写内存，双方都能生成机器码，但入口寄存器含义完全错位。

常见误区

返回类型是 ABI 推导的一部分，不是最后才看的附加信息。大结构体返回会改变显式参数的位置；小结构体返回可能占用两个寄存器；浮点返回会占用 **xmm0**。跳过返回类型，函数入口解释很容易从第一步就错。

结构体参数的简化分类规则

System V AMD64 ABI 对聚合类型有详细分类算法。完整规则涉及 **INTEGER**、**SSE**、**SSEUP**、**X87**、**COMPLEX_X87**、**MEMORY** 等类别，还会把对象按 8 字节分块。本书先给出足以读大多数性能代码的简化流程：

1. 把结构体按 8 字节为单位切成一个或多个 **eightbyte**。
2. 如果对象太大、字段未对齐、或 C++ 类型有非平凡拷贝/析构语义，通常走内存或隐藏引用。
3. 如果不超过两个 **eightbyte**，且每块能分类成 **INTEGER** 或 **SSE**，就可能通过寄存器传递。
4. **INTEGER** 类通常走通用寄存器。
5. **SSE** 类通常走 **xmm** 寄存器。
6. 一旦需要的寄存器不够，相关参数可能改走栈。

例子 1：

```
struct TwoLong {
    long a;
    long b;
};

extern "C" long use_two_long(TwoLong x)
{
    return x.a + x.b;
}
```

两个 8 字节整数块，可能入口为：

```
rdi = x.a
rsi = x.b
```

例子 2：

```
struct TwoDouble {
    double a;
    double b;
};

extern "C" double use_two_double(TwoDouble x)
{
    return x.a + x.b;
}
```

两个 8 字节 SSE 块，可能入口为：

```
xmm0 = x.a
xmm1 = x.b
```

例子 3：

```
struct Mixed {
```

```
int a;
double b;
};

extern "C" double use_mixed(Mixed x)
{
    return double(x.a) + x.b;
}
```

常见入口可能是：

```
edi = x.a
xmm0 = x.b
```

这说明结构体并不总是“整体放到内存”。小结构体可能被拆分到多个寄存器里。

常见误区

结构体 ABI 分类是容易出错的地方。字段顺序、padding、对齐、浮点/整数混合、C++ 非平凡类型都会改变传参方式。遇到公开二进制接口时，不要凭感觉改结构体布局。

为什么按 eightbyte 思考

System V AMD64 ABI 采用 eightbyte 分类，是因为二进制传参最终要落到机器寄存器和机器字大小附近的传输单元上。一个小结构体虽然在 C++ 层是一个整体对象，但在调用边界上，ABI 可以把它拆成一个或两个 8 字节块，再分别决定走通用寄存器、浮点寄存器，还是内存。这个设计在效率和通用性之间折中：常见小对象不用总是写入内存，复杂对象又能退回更保守的内存传递。

以两个 long 字段的结构体为例，两个字段刚好分别占两个 eightbyte，且都属于整数类，因此可以用两个通用寄存器传递。以两个 double 字段的结构体为例，两个字段也各占一个 eightbyte，但类别是 SSE，因此可以用两个 xmm 寄存器传递。若结构体包含一个 int 后跟一个 double，由于对齐和 padding，可能形成一个整数类块和一个 SSE 类块，于是入口状态可能同时使用 edi 和 xmm0。

这里的关键不是记住某个例子的答案，而是看到“结构体整体”与“ABI 传输块”之间的差异。源码层的结构体表达了字段关系、类型不变量和程序员意图；ABI 层只关心调用边界上这些字节如何分类、放在哪里、由谁负责复制。一个结构体越接近普通标量组合，越可能被拆进寄存器；一旦它变大、未对齐、类别复杂或带有非平凡 C++ 语义，就更可能回到内存路径。

padding 和字段顺序会改变 ABI 形状

结构体布局不是只影响 sizeof。它还可能改变 ABI 看到的类别。下面两个结构体包含相似字段，但排列方式不同：

```
struct A {
    int x;
    double y;
};

struct B {
    double y;
    int x;
};
```

在常见 x86-64 布局下，二者大小可能都被对齐到 16 字节，但字段 offset 不同。ABI 分类时会按 eightbyte 切块并合并字段类别。对于普通情况，二者都可能通过一个通用寄存器和一个 xmm 寄存器传递；但如果再加入小字段、显式对齐、packing pragma 或向量字段，分类结果可能变化。公开 ABI 中随意调整字段顺序，不只是二进制大小变化，也可能改变调用者如何传参。

这对库设计尤其重要。一个只在单个程序内部使用的结构体，字段重排通常可以随源码一起重新编译；一个暴露在动态库边界、插件接口或 FFI 边界上的结构体，字段 offset、对齐、大小和传参方式都成为兼容性的一部分。改动它可能要求所有调用方重新编译，甚至造成旧调用方按旧 ABI 传参、新库按新 ABI 解释的错误。

非平凡 C++ 类型不是普通字节包

有些结构体大小很小，但仍然不应该按“两个整数寄存器”这样简单理解。C++ 类型可能有非平凡构造、复制、移动、析构，可能包含虚函数、基类、引用成员，或者要求特殊对齐。ABI 在处理 C++ 类型时不仅看字节数量，还要考虑对象如何被创建、复制和销毁。非平凡类型常常会通过地址间接传递，让调用边界能够保持对象语义。

例如一个结构体只包含一个指针，但有自定义析构函数。它在内存大小上像一个机器字，但语言层并不是普通整数。调用者和被调用者必须就对象生命周期达成一致，否则析构次数、所有权转移和异常清理都会出错。ABI 规范和具体编译器 ABI 会规定这些对象在函数参数和返回值中如何处理。读汇编时，如果只凭 `sizeof` 判断“应该进 `rdi`”，就可能漏掉隐藏地址传递或临时对象构造。

本册后面手写汇编实验会优先使用 `extern "C"` 和普通平凡类型，不是因为真实工程只有这些类型，而是为了先把二进制调用约定学清楚。进入复杂 C++ ABI、异常、虚函数和模板库边界时，需要在这个基础上继续查看编译器 ABI 文档和真实反汇编。

核心思想

结构体 ABI 分析至少要同时看四件事：字段布局、`eightbyte` 分类、寄存器容量、C++ 类型是否平凡。只看 `sizeof` 或只看字段个数都不够。

caller-saved 和 callee-saved

寄存器数量有限。函数调用前后，哪些寄存器可以被破坏？哪些必须保持？ABI 用 `caller-saved` 和 `callee-saved` 分工。

类别	寄存器	含义
caller-saved	<code>rax</code> 、 <code>rcx</code> 、 <code>rdx</code> 、 <code>rsi</code> 、 <code>rdi</code> 、 <code>r8–r11</code>	调用者若还需要这些值，调用前自己保存
callee-saved	<code>rbx</code> 、 <code>rbp</code> 、 <code>r12–r15</code>	被调用者如果修改，返回前必须恢复
特殊	<code>rsp</code>	返回前必须恢复到入口约定状态
浮点/向量	<code>xmm0–xmm15</code>	System V 下通常 <code>caller-saved</code>

为什么这样分？如果所有寄存器都由调用者保存，每次调用都很重；如果所有寄存器都由被调用者保存，每个函数入口都很重。ABI 把一部分寄存器定义为 `caller-saved`，适合短生命周期临时值；把另一部分定义为 `callee-saved`，适合跨调用仍要保留的值。

保存责任的真正含义

`caller-saved` 和 `callee-saved` 不是寄存器用途分类，而是责任分类。`caller-saved` 寄存器不是“只能调用者使用”，`callee-saved` 寄存器也不是“只能被调用者使用”。任何函数都可以使用大多数通用寄存器，区别在于跨越一次调用边界时，谁必须承担让某个值继续有效的成本。

如果调用者有一个值放在 `rax`，并且调用另一个函数后还要使用这个值，那么调用者不能指望被调用者保持 `rax` 不变。它必须在调用前把这个值保存到别的地方，例如 `callee-saved` 寄存器、栈槽，或者重新计算。反过来，如果被调用者想使用 `rbx`，它可以使用，但必须在返回前把调用者原来的 `rbx` 恢复。调用者看到的是“调用前后的 `rbx` 没变”，至于被调用者中间怎样使用，调用者不关心。

这个责任模型给编译器寄存器分配器提供了成本选择。生命周期很短、不会跨调用的临时值，放在 `caller-saved` 寄存器通常便宜，因为不需要函数入口额外保存。生命周期较长、要跨多个调用的值，放在 `callee-saved` 寄存器可能更合适，因为只在函数入口和出口保存一次，而不是每个调用点保存一次。当然，具体选择还受优化级别、寄存器压力、内联决策、异常路径和调试信息影响。

手写汇编时，最稳妥的思路是先列出自己会修改的寄存器，再按 ABI 分组。对 `caller-saved` 寄存器，如果你的函数不需要在调用后保留其中某个值，可以直接使用；如果你自己在调用别的函数前仍然需要它，就由你作为调用者保存。对 `callee-saved` 寄存器，只要你修改了它，就必须保证返回给你的调用者时恢复原值。这个规则与函数是否崩溃无关，是二进制接口正确性的最低要求。

深入理解

保存 callee-saved 寄存器不一定非要用 **push** 和 **pop**。编译器也可能把它们保存到栈帧固定偏移，或者在 shrink wrapping 优化下只在真正需要的分支保存和恢复。判断是否遵守 ABI，要看所有返回路径上调用者原值是否恢复，而不是机械要求函数开头必须有某种序言。

callee-saved 示例

C++:

```
extern "C" long g(long);

extern "C" long keep_across_call(long x)
{
    long y = x * 3;
    long z = g(x);
    return y + z;
}
```

y 必须跨过对 **g** 的调用继续存在。编译器可能把 **y** 放在 callee-saved 寄存器 **rbx**:

```
keep_across_call:
    push rbx
    lea  rbx, [rdi + rdi*2]
    call g
    add  rax, rbx
    pop  rbx
    ret
```

解释:

```
push rbx
    save caller's original rbx

lea rbx, [rdi + rdi*2]
    rbx = x * 3

call g
    g may clobber caller-saved registers
    g must preserve rbx if it uses rbx

add rax, rbx
    rax = g(x) + x * 3

pop rbx
    restore caller's original rbx

ret
```

如果 **keep_across_call** 修改了 **rbx** 却没有恢复，调用它的上层函数可能崩溃或产生错误结果。

深入理解

callee-saved 不是说这个寄存器“不能用”。它的意思是：可以用，但你要把调用者看到的旧值还回去。常见保存方式是 **push** 和 **pop**，也可以保存到栈帧中的固定偏移。

caller-saved 示例

再看 caller-saved 的情况。假设一个函数要计算两个 helper 的结果，并且第一个结果在第二次调用后还要继续使用：

```
extern "C" long f(long);
extern "C" long g(long);

extern "C" long combine(long x)
{
    long a = f(x);
    long b = g(x + 1);
```

```
    return a + b;
}
```

f 的返回值在 **rax**。但随后调用 **g** 时，**rax** 属于 caller-saved，**g** 可以合法覆盖它。因此编译器必须在调用 **g** 前保存 **a**，可能放到 **rbx**，也可能放到栈槽：

```
combine:
    push rbx
    call f
    mov  rbx, rax
    lea  rdi, [original_x + 1]
    call g
    add  rax, rbx
    pop  rbx
    ret
```

这段伪汇编故意省略了 **x** 的保存细节，因为真实编译器可能用不同方式保留原始参数。重点是：**a** 不能指望留在 **rax** 里跨过 **g** 的调用。caller-saved 的“可被破坏”不是坏事，它让被调用者能自由使用这些寄存器，减少小函数保存成本。

常见误区

如果你在手写汇编里调用 C++ helper，然后继续使用 **rdi**、**rsi**、**rax**、**rcx** 等 caller-saved 寄存器中的旧值，必须先确认这些旧值已经被保存。helper 即使今天没有改它们，也没有 ABI 义务替你保留。

栈对齐规则

System V AMD64 ABI 要求调用点满足 16 字节栈对齐。更具体地说：

```
before call:
    rsp is 16-byte aligned

call pushes 8-byte return address

on callee entry:
    rsp + 8 is 16-byte aligned
    rsp itself is 8 mod 16
```

一个函数入口常见状态：

```
rsp points to return address
rsp mod 16 = 8
```

如果函数要再调用别的函数，它必须在自己的 **call** 前把 **rsp** 调整到 16 字节对齐。
例子：

```
caller:
    sub rsp, 8
    call callee
    add rsp, 8
    ret
```

假设 caller 入口时 **rsp mod 16 = 8**：

```
entry:
    rsp mod 16 = 8

sub rsp, 8:
    rsp mod 16 = 0

call callee:
    pushes return address
    callee entry rsp mod 16 = 8

add rsp, 8:
    restore caller's stack pointer
```

为什么要对齐？一方面 ABI 需要统一约定；另一方面 SIMD load/store、局部对象对齐、调用约定和编译器生成代码都依赖这个规则。一些指令如 `movaps` 对内存地址有对齐要求；即使现代 CPU 对很多未对齐访问能处理，错误的 ABI 栈对齐仍然可能导致崩溃或性能下降。

常见误区

手写汇编调用 C/C++ 函数时，最常见错误之一就是忘记在 `call` 前对齐 `rsp`。这种错误可能在简单测试里不暴露，换一个编译器版本、库函数或 CPU 状态就崩。

对齐规则怎样推导

栈对齐最容易背错，因为入口状态和调用点状态差 8 字节。记住一个事实就够：`call` 会把 8 字节返回地址压栈。ABI 要求调用点执行 `call` 之前 `rsp` 是 16 字节对齐；所以被调用者刚进入时，`rsp` 指向返回地址，数值自然变成 $8 \bmod 16$ 。换句话说，被调用者入口通常满足 `rsp+8` 是 16 字节对齐，而不是 `rsp` 本身 16 字节对齐。

如果一个函数不再调用别的函数，它可以在很多情况下不调整栈，只要自己的局部访问和返回满足要求。若它要调用其他函数，就必须在自己的调用点前把 `rsp` 调到 16 字节对齐。于是一个没有其它栈需求的非叶子手写函数，经常在入口后执行 `sub rsp, 8`，调用结束后再 `add rsp, 8`。这不是为了“申请一个有用变量”，而是为了抵消入口处的 8 字节偏移。

如果函数还要保存 `rbx` 或其它寄存器，`push` 本身也会改变对齐。例如入口 `rsp mod 16 = 8`，执行一次 `push rbx` 后，`rsp mod 16 = 0`，此时如果马上调用别的函数，栈对齐可能已经满足。若再额外 `sub rsp, 8`，反而会破坏调用点对齐。真实函数的栈调整必须把返回地址、保存寄存器、局部空间和调用前对齐一起计算。

栈对齐错误为什么难复现

栈对齐错误常表现得不稳定。一个错误函数可能在本机、当前优化级别、当前 libc 版本下运行正常，因为被调用路径没有使用对齐敏感指令，或者未对齐访问被硬件以较高代价处理了。换一台机器、换一个编译器版本、打开不同优化选项、链接到不同库实现，错误就可能变成崩溃或数据错误。

这类问题不能用“我运行了一次没崩”证明正确。正确证明应来自规则推导和反汇编检查：函数入口 `rsp` 余数是多少，执行每条 `push`、`sub rsp, ...` 后余数怎样变化，每个 `call` 前是否回到 $0 \bmod 16$ ，返回前是否恢复到入口对应状态。实验可以帮助暴露错误，但 ABI 正确性首先是静态契约问题。

核心思想

栈对齐分析要按时间线做模 16 计算。入口、保存寄存器、申请局部空间、调用点、恢复路径，每一步都要算；不能只看函数里有没有某条固定的 `sub rsp, 8`。

栈帧、帧指针和局部变量

带帧指针的函数常见形态：

```
push rbp
mov  rbp, rsp
sub  rsp, 32
...
add  rsp, 32
pop  rbp
ret
```

或等价尾声：

```
leave
ret
```

进入函数后栈可能长这样：

```
higher addresses

[rbp + 16] stack argument 2 if present
[rbp + 8]  return address
```

```
[rbp + 0] saved old rbp
[rbp - 8] local/spill slot
[rbp - 16] local/spill slot
[rbp - 24] local/spill slot
```

Lower addresses

`rbp` 的价值是稳定。函数里 `rsp` 可能因为 `push`、`sub rsp, ...`、调用准备而变化，但 `rbp` 通常保持不变，方便调试器、低优化级代码和人工阅读。

优化级较高时，编译器常省略帧指针：

```
clang++ -std=c++20 -O3 -fomit-frame-pointer ...
```

性能分析时，很多工程会保留帧指针以改善采样栈回溯：

```
clang++ -std=c++20 -O3 -fno-omit-frame-pointer ...
```

是否保留帧指针是性能、调试、profiling 之间的工程权衡。现代生产服务常为了 profiler 可用性保留帧指针。

栈帧不是语言变量表

栈帧经常被讲成“存放局部变量的地方”，这句话太粗。优化后的栈帧可能没有某些源码局部变量，因为它们被寄存器保存、被常量折叠、被完全消除，或者生命周期被拆散。栈帧里也可能有源码里看不见的内容，例如保存的 callee-saved 寄存器、对齐填充、spill slot、临时对象、异常处理需要的状态、调用其它函数前准备的栈参数区。

因此，读栈帧时不要从变量名出发，而要从用途出发。一个槽可能用于保存 `rbx`；一个槽可能用于把 `xmm0` 溢出到内存；一个槽可能是为了让局部数组有稳定地址；一个槽可能只是为了维持对齐。调试信息会把部分机器位置映射回源码变量，但高优化级下这种映射可能是分段的、变化的，甚至在某些指令范围内不可用。

这对 GDB 学习很重要。你在 `-O0` 下看到 `x` 总在 `[rbp-4]`，不代表 `x` 在机器模型中天然属于那个地址。到了 `-O3`，同一个值可能在 `eax`，可能被合并进另一条指令，可能根本没有单独实体。ABI 只要求函数边界遵守约定，不要求函数内部保留源码变量的外观。

帧指针、展开信息和调用链

调试器和 profiler 要恢复调用链，通常需要知道每一层函数的返回地址在哪里、上一层栈位置在哪里、哪些寄存器被保存到哪里。保留帧指针是一种简单而稳定的方式：每个函数把旧 `rbp` 保存起来，并让新 `rbp` 指向当前帧基准，形成一条链。沿着这条链，工具可以较容易地找到上一帧。

但现代编译器即使省略帧指针，也可以通过 DWARF CFI 等展开信息描述如何从当前指令位置恢复调用者状态。这些信息不是 CPU 执行用的，而是调试、异常处理、栈回溯和性能采样工具使用的元数据。只要展开信息完整，省略帧指针的程序也可以正确回溯；但在生产性能采样、异步采样、JIT、手写汇编或部分 stripped binary 场景中，展开信息可能不完整或工具支持不足，保留帧指针就变得很有价值。

手写汇编如果要参与可靠回溯和异常边界，不能只写能运行的指令，还要考虑函数符号、`.type`、`.size`、CFI 指令和栈变化描述。本册的基础实验先关注普通调用正确性；工程级汇编库还必须让调试器和 profiler 看得懂。

深入理解

ABI 的“函数边界”不只服务调用者和被调用者，也服务工具。一次采样落在函数中间时，profiler 需要根据当前 `rsp`、保存寄存器和展开规则找到上一层调用者。栈帧写得越偏离约定，工具越依赖你提供准确元数据。

red zone

System V AMD64 ABI 定义了 **red zone**（红区）：用户态函数入口时，`rsp` 下方 128 字节区域可以被叶子函数临时使用，而不必移动 `rsp`。

位置：


```
[rsp - 128] ... [rsp - 1]  red zone
[rsp + 0]                return address
```

叶子函数是不调用其他函数的函数。例子：

```
extern "C" long leaf(long x)
{
    long tmp[4];
    tmp[0] = x + 1;
    tmp[1] = x + 2;
    tmp[2] = x + 3;
    tmp[3] = x + 4;
    return tmp[0] + tmp[1] + tmp[2] + tmp[3];
}
```

如果优化器不能完全消除数组，叶子函数可能使用 `rsp` 下方空间，而不执行 `sub rsp, ...`。red zone 的限制：

- 只适合不调用其他函数的叶子函数。
- 一旦执行 `call`，新的返回地址会写到更低地址，可能覆盖调用者 red zone 中的内容。
- 内核、裸机、中断上下文通常不能依赖 red zone，常用 `-mno-red-zone`。
- 手写汇编使用 red zone 时必须确认目标环境允许。

```
clang++ -std=c++20 -O3 -mno-red-zone ...
```

red zone 的工程边界

red zone 的价值是减少叶子函数调整栈指针的开销。一个小叶子函数如果只需要少量临时内存，可以直接使用 `rsp` 下方空间，不必执行 `sub rsp, ...` 和 `add rsp, ...`。这能减少指令，也让某些短函数更紧凑。

但 red zone 不是“栈下面永远安全的 128 字节”。它成立的前提是 System V 用户态 ABI 保证异步信号处理不会破坏这片区域，并且当前函数不执行会覆盖该区域的普通调用。一旦进入内核代码、中断上下文、裸机环境，或者使用不遵守该约定的运行环境，这个前提就可能不存在。内核代码常禁用 red zone，因为中断和异常处理可能在当前栈上写入状态。

手写汇编里是否使用 red zone，要看目标环境和函数性质。若函数会调用其他函数，不应把仍然需要的值放在 red zone 里跨越 `call`。若函数需要被移植到 Windows x64 或内核环境，也不能假设 red zone 存在。若为了简单和可读性，基础实验完全可以不用 red zone，显式调整 `rsp` 反而更容易推导。

常见误区

red zone 是 System V 用户态的约定，不是 x86-64 硬件特性，也不是所有平台 ABI 都有。跨平台汇编、内核代码和中断相关代码应主动确认是否允许使用。

变参函数和 `va_list`

变参函数比普通函数复杂，因为被调用者不知道静态参数列表之后还有多少参数、类型是什么。

典型调用：

```
#include <stdio.h>

extern "C" void print_values(int x, double y)
{
    std::printf("x=%d y=%f\n", x, y);
}
```

调用 `printf` 时：

```
rdi = pointer to format string
esi = x
xmm0 = y
```

```
al = number of vector registers used for floating arguments
call printf
```

System V ABI 要求调用变参函数时，`al` 携带用于传递浮点/向量参数的向量寄存器数量上界。你经常在调用 `printf` 前看到：

```
mov eax, 1
call printf
```

如果没有浮点参数，可能看到：

```
xor eax, eax
call printf
```

变参函数内部使用 `va_start` 时，编译器会维护一个 `va_list`，里面通常包含：

```
gp_offset
fp_offset
overflow_arg_area
reg_save_area
```

直觉上：

- 前几个普通参数可能来自寄存器保存区。
- 超出寄存器容量的参数来自栈上的 `overflow` 区域。
- 浮点参数和整数参数有不同 `offset`。

深入理解

手写汇编调用变参函数时，不能只把参数放到寄存器就结束。尤其是调用带浮点参数的 `printf`，还要正确设置 `al`。这个细节非常适合实验，因为错误代码有时能运行，有时输出错，有时在不同 `libc` 或优化级别下表现不同。

为什么变参需要额外规则

普通函数的被调用者知道完整参数类型。它看到自己的函数签名，就知道第几个参数应该从哪个寄存器或栈槽读取。变参函数不同，省略号后面的参数在被调用者函数类型里没有逐项写出。被调用者只能通过格式串、约定或调用者与被调用者共同理解的协议来解释后续参数。

这会带来两个问题。第一，寄存器参数在函数入口后可能被覆盖，但 `va_start` 后仍要能依次访问变参。因此编译器通常要建立寄存器保存区，把可能参与变参访问的通用寄存器和浮点寄存器内容保存到一个已知布局中。第二，整数和浮点使用不同寄存器通道，被调用者需要知道浮点寄存器中到底有多少个有效变参或至少知道一个上界，才能正确初始化访问状态。System V AMD64 中 `al` 的约定就服务于这个目的。

这解释了为什么调用 `printf` 前常能看到 `xoreax,eax` 或 `moveax,1`。这条指令并不是给 `printf` 的第一个整数参数赋值，也不是清空返回值；它是在调用变参函数前设置 ABI 要求的隐藏调用状态。读汇编时如果不知道这条规则，很容易把它误解成无意义的优化器习惯。

变参函数也说明 ABI 不是只关心显式参数表。有些调用状态不会出现在 C++ 函数声明中，但仍然是二进制契约的一部分。手写汇编调用普通函数时，参数放对位置通常就够；手写汇编调用变参函数时，还要考虑 `al`、默认参数提升、格式串与实参类型匹配、栈对齐和寄存器保存区等细节。

常见误区

C/C++ 的变参函数没有运行时类型检查。格式串写 `%f`，但调用者没有按 ABI 传入 `double`，被调用者仍会按 `double` 解释某个位置的字节。许多变参错误不会在编译或链接阶段暴露，而是在运行时输出错值或破坏栈/寄存器解释。

默认参数提升和格式串必须一致

C 风格变参还有一个语言层规则：默认参数提升。传给省略号的 `float` 会提升为 `double`，小于 `int` 的整数类型通常提升为 `int` 或 `unsignedint`。这意味着 `printf("%f",x)` 期望的不是 `float` ABI 形态，而是 `double` 形态；`printf("%hhhd",c)` 的实参也通常按 `int` 传入，格式串再告诉 `printf` 如

何解释和打印。

例如：

```
float f = 1.5f;
std::printf("%f\n", f);
```

调用点会把 `f` 转成 `double`，再通过浮点参数通道传递。手写汇编若把 32 位 `float` 原样放进 `xmm0` 的低 32 位，却让格式串写 `%f`，`printf` 会按 `double` 读取，结果就是错误解释。类似地，格式串写 `%d` 时，`printf` 会按 `int` 从通用参数位置取值；你若只准备了低 8 位字符值，高位不明确，就可能输出奇怪结果。

因此，变参调用要同时满足三层规则：

1. C/C++ 默认参数提升后，实参类型是什么。
2. System V ABI 把提升后的实参放在哪里。
3. 被调用者的格式串或协议按什么类型读取。

这三层任何一层错了，编译器和链接器都未必能救你。许多现代编译器会对字面量格式串做警告检查，但手写汇编、动态格式串、跨语言 FFI 和错误声明仍然可能绕过检查。

C++ 名字修饰和 `extern"C`

ABI 不只包含寄存器和栈，也包含符号名约定。C++ 支持函数重载、命名空间、类成员函数，因此编译器会做 `name mangling`（名字修饰）。

C++：

```
int add(int, int);
double add(double, double);
```

两个函数源代码名都叫 `add`，目标文件里的符号名必须不同。编译器可能生成类似：

```
_Z3addii
_Z3adddd
```

如果希望 C++ 函数使用 C ABI 符号名，可以写：

```
extern "C" int add(int, int);
```

这样符号名通常就是 `add`。本书实验里大量使用 `extern"C`，不是因为 C++ 不好，而是为了让汇编符号名稳定、易读、易链接。

`extern"C` 改变什么，不改变什么

`extern"C` 最常见的作用是改变语言链接规则，让 C++ 编译器使用 C 风格符号名，避免名字修饰带来的链接不匹配。它也禁止同一 C 链接名下的 C++ 重载，因为 C 符号名无法表达参数类型列表。对手写汇编来说，这能让汇编文件导出 `asm_add2`，C++ 头文件也声明 `asm_add2`，链接器看到的是同一个符号。

但 `extern"C` 不会把函数体变成 C 语言执行，也不会关闭 C++ 类型系统，更不会改变目标平台的底层调用约定。一个 `extern"C"double f(double)` 在 System V AMD64 下仍然使用 `xmm0` 传参和返回；一个 `extern"C"Bigmake()` 如果返回类型需要 `sret`，仍然会出现隐藏返回指针。它解决的是符号名和语言链接问题，不是所有 ABI 问题。

还要注意，C++ 对异常、对象生命周期、模板实例、内联函数、命名空间和类成员函数有自己的 ABI 细节。把某个函数声明成 `extern"C`，通常意味着你在设计一个更简单、更稳定的二进制边界：参数和返回类型应尽量使用普通标量、指针或明确布局的平凡结构体；不要让 C++ 异常跨越不理解异常 ABI 的语言边界；不要把 STL 容器对象作为长期稳定 ABI 直接暴露给外部模块。

核心思想

`extern"C` 主要稳定符号名，不自动保证接口稳定。真正稳定的 ABI 还需要稳定的类型大小、对齐、字段布局、调用约定、内存所有权、错误处理方式和版本策略。

引用、指针和成员函数的隐藏参数

C++ 源码里有些东西看起来不是指针，但 ABI 层经常表现为地址。最常见的是引用、非静态成员函数的 **this**、大对象传参和某些非平凡对象。读汇编时，如果只盯着源码表面类型，会把地址当成数值，把隐藏参数当成显式参数。

引用通常以地址传递

考虑：

```
extern "C" void inc_ref(int& x)
{
    ++x;
}
```

源代码里 **x** 是引用，不是指针。但在 ABI 层，被调用者需要知道要修改哪个对象，入口通常会收到对象地址：

```
rdi = address of referenced int object
inc dword ptr [rdi]
ret
```

这和下面函数的机器形态很接近：

```
extern "C" void inc_ptr(int* x)
{
    ++*x;
}
```

区别在 C++ 语义层：引用必须绑定到对象，语法上不可重新绑定，通常不应为空；指针可以为空、可以重新赋值。ABI 层看到的常常都是地址。因此调试崩溃时，**rdi** 为 0 可能意味着调用者违反了引用前提，或者你实际分析的是指针接口；不能只从汇编判断完整源语义。

非静态成员函数的 **this**

非静态成员函数有一个隐藏的 **this** 参数。比如：

```
struct Counter {
    int value;

    int add(int x)
    {
        value += x;
        return value;
    }
};
```

在 System V AMD64 下，成员函数入口通常类似：

```
rdi = this pointer
esi = x
```

函数体可能访问：

```
add dword ptr [rdi], esi
mov eax, dword ptr [rdi]
ret
```

这说明源代码里看起来只有一个显式参数 **x**，但机器层第一个通用寄存器被 **this** 占用。若成员函数还返回大结构体，隐藏 **sret** 指针可能进一步占用第一个通用寄存器，**this** 再后移。复杂 C++ ABI 中，隐藏参数的顺序要以具体 ABI 和编译器输出为准。

成员函数指针不是普通函数指针

普通函数指针通常就是代码地址或可调用入口地址；C++ 成员函数指针可能要表达虚函数、基类调整、**this** 调整等信息，不能简单当作 **void*** 或普通函数地址处理。第一册不深入成员函数指针 ABI，但要建立底线：不要把成员函数指针传给期待 C 函数指针的接口，也不要手写汇编里凭直觉调用

它。

如果需要给 C 风格回调传入 C++ 对象，常见做法是使用普通函数指针加 `void*` 上下文：

```
using Callback = void (*)(void* context, int value);
```

调用者把对象地址放进 `context`，回调函数内部再把它转回具体类型。这种接口虽然朴素，但 ABI 形状清楚：一个函数指针，一个上下文指针，一个整数参数。许多 C 库和系统 API 都采用这种设计。

核心理想

C++ 源码里看不到的参数，机器层可能真实存在：引用对象地址、`this` 指针、`sret` 指针、异常和展开相关状态。分析函数入口时，要先问“有没有隐藏参数”，再给显式参数编号。

函数调用 ABI 和系统调用 ABI 不同

本章讨论的是用户态函数之间的 System V AMD64 调用约定。Linux 系统调用使用另一套寄存器约定。用户态函数调用使用 `call` 进入普通函数，返回地址在用户栈上；系统调用通常使用 `syscall` 指令进入内核，参数寄存器和破坏寄存器规则与普通函数调用不同。

常见 Linux x86-64 系统调用约定中，系统调用号放在 `rax`，参数使用 `rdi`、`rsi`、`rdx`、`r10`、`r8`、`r9`。注意第 4 个参数是 `r10`，不是普通函数 ABI 的 `rcx`。原因与 `syscall` 指令本身会使用 `rcx` 和 `r11` 保存返回相关状态有关。

这一区分很重要。你在 `libc` 的 `write` 包装函数外层看到的是普通 C 函数 ABI；在它内部最终进入内核的那一刻，才会按系统调用 ABI 摆放寄存器。把两套约定混在一起，会导致手写汇编系统调用、调试 `libc`、阅读 `strace` 与 `objdump` 对照时全部错位。

常见误区

System V AMD64 函数调用约定不等于 Linux `syscall` 约定。普通函数第 4 个整数参数常在 `rcx`，系统调用第 4 个参数常在 `r10`。看到 `syscall` 时必须切换规则。

动态库边界和 ABI 稳定性

ABI 最有工程重量的地方，是动态库和长期发布的二进制接口。一个静态链接、全量重新编译的小程序，很多类型和调用边界变化都能被编译器同时更新；一个已经发布的共享库则不同。外部程序可能只拿到头文件和 `.so`，按旧头文件编译好的二进制会在运行时加载你的新库。如果你改变了导出函数的参数类型、返回类型、结构体布局、符号名或异常边界，就可能破坏旧程序。

例如，库版本一导出：

```
extern "C" Result query(Context*, int);
```

如果版本二把 `Result` 从两个 `long` 字段改成三个 `long` 字段，源码看起来只是多了一个字段，但 ABI 形状可能从 `rax/rdx` 返回变成隐藏 `sret` 指针返回。旧调用者仍然按旧规则从寄存器取结果，新库却按新规则把第一个寄存器解释为输出地址，这就是灾难级不兼容。

再比如把结构体字段顺序改变，可能导致旧调用者写入的 `offset` 与新库读取的 `offset` 不一致；把 `int` 改成 `long`，可能改变寄存器中有效位宽和符号扩展预期；把 C 符号改成 C++ 重载符号，可能让动态链接器找不到旧符号；让异常跨越 C ABI 边界，可能让不理解 C++ 异常运行时的调用方无法正确清理。

因此，稳定 ABI 设计通常会采用更保守的接口风格：使用不透明指针隐藏内部结构体；用显式版本号和大小字段描述可扩展结构；通过函数返回错误码或明确的错误对象处理失败；避免把 C++ 标准库类型、模板类型和非平凡对象直接暴露在二进制边界上；需要升级时保留旧符号或使用符号版本机制。

本册不是动态链接专著，但学习 ABI 时必须形成这个工程意识：ABI 不是写汇编才需要关心的冷门知识，它是库作者、性能工程师、语言互操作工程师和系统调试人员共同依赖的契约。

不透明指针：把布局变化藏起来

稳定 C ABI 常用不透明指针：

```
extern "C" Context* context_create();
extern "C" void context_destroy(Context*);
extern "C" int context_query(Context*, int key, long* value_out);
```

调用者只知道 `Context*` 是一个句柄，不知道 `Context` 内部字段。库可以在新版本中改变 `Context` 的大小、字段顺序、内部缓存和锁，只要创建、销毁和查询函数保持 ABI 兼容，旧调用者就不需要重新理解内部布局。这比把结构体定义暴露在头文件里稳定得多。

不透明指针的代价是调用者不能在栈上直接分配对象，也不能内联访问字段；所有操作都要通过函数调用。但在动态库、插件和跨语言边界上，这个代价通常值得。它把“对象内部实现”从 ABI 中移出去，只把“句柄和函数”作为稳定接口。

带大小字段的可扩展结构

有时接口必须让调用者传入一个结构体，例如配置参数。为了允许未来扩展，常见设计是把结构体大小或版本号放在第一个字段：

```
struct Options {
    std::uint32_t size;
    std::uint32_t version;
    std::uint32_t flags;
    std::uint32_t reserved;
};

extern "C" int run_with_options(const Options* options);
```

调用者把 `size` 设置为自己编译时看到的 `sizeof(Options)`。库收到后，先检查 `size`，只读取调用者保证存在的字段。未来版本可以在结构体末尾添加字段，旧调用者传来的 `size` 较小，新库就不会越界读取；新调用者传给旧库时，旧库也可以根据版本和大小拒绝或忽略新字段。

这种设计不能随便改变已有字段的 `offset`、类型和含义。它只适合在末尾追加字段，并保持对齐和默认值策略清楚。若把中间字段重排，`size` 也救不了旧 ABI。

内存所有权也是 ABI

ABI 不只规定寄存器，还规定跨边界对象由谁分配、谁释放、用哪个分配器释放。例如：

```
extern "C" char* make_message();
extern "C" void free_message(char*);
```

如果库用自己的 `allocator` 分配字符串，调用者就不应该用另一个运行时的 `free` 或 `delete[]` 释放。跨动态库、跨语言、跨运行时边界时，分配器不匹配会导致堆损坏。稳定 ABI 通常让“创建”和“销毁”成对出现在同一库接口里，或要求调用者提供 `buffer`：

```
extern "C" int write_message(char* buffer, std::size_t buffer_size);
```

后者把内存所有权留给调用者，库只写入调用者提供的空间。接口更繁琐，但所有权更清楚。

核心理想

公开 ABI 设计的核心是减少二进制边界上的假设：隐藏内部布局，显式记录大小和版本，明确内存所有权，保留旧符号或旧入口，不让调用者猜实现细节。

尾调用和 ABI 边界

优化后的汇编中，函数结尾有时不是 `call` 后再 `ret`，而是直接 `jmp` 到另一个函数。这叫尾调用优化或 sibling call。它能复用当前栈帧，避免额外返回地址和调用开销。

例如：

```
extern "C" long target(long);

extern "C" long wrapper(long x)
```

```
{
    return target(x);
}
```

优化后可能是：

```
wrapper:
    jmp target
```

这不是普通跳转 bug，而是合法优化：wrapper 不需要在 target 返回后再做任何工作，所以可以让 target 直接返回给 wrapper 的调用者。

尾调用必须满足 ABI 前提

尾调用不是随便把 call 换成 jmp。编译器必须保证跳转目标看到的入口状态符合 ABI：参数已经放在正确位置，栈指针处于目标函数期望的调用边界状态，当前函数需要恢复的 callee-saved 寄存器已经恢复，局部栈空间已经释放，返回值语义等价。若这些条件不满足，就不能简单尾跳。

例如当前函数修改了 rbx，在尾跳前必须先恢复 rbx。当前函数申请了栈空间，尾跳前必须把 rsp 恢复到调用者期望状态。当前函数需要把参数重排给目标函数，必须在跳转前完成，且不能破坏仍需要的值。若目标函数参数比当前函数更多，或需要额外栈参数，尾调用可能仍然可行，但条件更复杂。

手写汇编里使用尾跳尤其要谨慎。你不能因为“最后一步就是调用另一个函数”就直接 jmp，而要检查所有 ABI 清单。错误尾跳常表现为调用栈缺失、返回地址错乱、callee-saved 寄存器破坏或目标函数读到错误栈参数。

尾调用对调试和 profiling 的影响

尾调用会改变调用栈外观。源码层看起来有 wrapper->target，但机器栈上可能没有 wrapper 的独立返回帧。GDB、profiler 或崩溃回溯中，wrapper 可能不出现，或只通过调试信息以特殊形式显示。性能分析时，这可能让热点归因直接落到 target，而不是包装函数。

这不是工具错误，而是机器码真的没有普通调用帧。若你需要在调试时保留完整调用链，可以降低优化级别或使用编译器选项限制 sibling call；若你在阅读优化构建，就要接受源码调用链和机器调用链可能不同。

常见误区

尾调用优化后的 jmp 仍然是 ABI 边界。目标函数必须看到合法入口状态；当前函数必须先完成自己的保存寄存器、栈恢复和参数重排责任。

手写汇编函数的最低正确性清单

如果你要写一个被 C++ 调用的 .S 函数，至少检查：

1. 符号名和 C++ 声明是否一致。
2. 参数寄存器是否按 System V ABI 读取。
3. 返回值是否写到正确寄存器。
4. 修改 callee-saved 寄存器前是否保存，返回前是否恢复。
5. 返回前 rsp 是否恢复。
6. 调用其他函数前 rsp 是否 16 字节对齐。
7. 调用变参函数前 al 是否正确。
8. 是否误用 red zone。
9. 是否需要 .type、.size 和 .note.GNU-stack。

一个最小汇编函数：

```
.intel_syntax noprefix
.text
.globl asm_add2
.type asm_add2, @function
asm_add2:
```

```

    lea eax, [rdi + rsi]
    ret
.size asm_add2, .-asm_add2
.section .note.GNU-stack,"",@progbits

```

对应 C++ 声明：

```
extern "C" int asm_add2(int, int);
```

ABI 错误的诊断方法

ABI 错误的麻烦之处在于，它们经常表现为“离错误很远的地方坏掉”。一个函数忘记恢复 **rbx**，真正崩溃的可能是它的调用者之后访问某个数据结构；一个函数调用前栈未对齐，崩溃点可能出现在 **libc** 内部的 **SIMD** 指令；一个返回大结构体的函数签名不匹配，表面现象可能是某个无关指针被写坏。诊断时不能只盯着崩溃那条指令，而要回到最近的二进制边界。

实用诊断步骤如下。第一，确认目标平台和调用约定，避免把 Windows x64、System V AMD64、Linux syscall、编译器特殊属性混用。第二，核对声明和定义：C++ 头文件、汇编符号、目标文件导出符号、链接器实际绑定的符号是否一致。第三，根据函数签名手工推导入口状态，并与反汇编调用点比较。第四，检查被调用函数是否修改了 callee-saved 寄存器、是否恢复 **rsp**、是否在所有返回路径上满足规则。第五，如果函数内部还调用别的函数，逐个调用点计算栈对齐和 caller-saved 值保存。第六，用 GDB 在调用前、入口、返回前、返回后三个位置记录关键寄存器和栈顶内容。

一个好的 ABI 调试记录不需要贴满整页反汇编。它应该抓住边界状态。例如：

```

before call:
    rdi = pointer p
    esi = n
    rsp mod 16 = 0

callee entry:
    rsp points to return address
    rsp mod 16 = 8
    rdi still equals p
    esi still equals n

before callee ret:
    rax = expected result
    rsp restored to entry value
    rbx equals original caller value

```

这类记录能证明函数是否遵守契约，而不是只证明某次输出碰巧正确。尤其是手写汇编实验，必须把“运行结果正确”和“ABI 规则正确”分开。一个坏函数可能在特定测试输入下返回正确数值，但仍然破坏调用者寄存器；这样的函数不能算通过。

常见错误模式

第一类错误是参数错位。常见原因包括忘记浮点和整数参数分通道计数、忘记大结构体返回的隐藏指针、把 Windows x64 寄存器顺序套到 System V、把系统调用约定套到普通函数。表现可能是第一个参数看起来变成地址、第二个参数变成第一个显式参数，或者浮点值全错。

第二类错误是保存责任错位。手写汇编修改 **rbx**、**r12** 到 **r15** 后直接返回，调用者后续状态被破坏；或者调用 C++ helper 后继续相信 **rax**、**rcx**、**rdi** 中旧值还在。表现可能是返回后某个循环变量、对象指针或累加器突然变化。

第三类错误是栈状态错误。包括调用前未对齐、返回前 **rsp** 没恢复、额外弹出栈参数、保存和恢复数量不匹配、某条错误路径提前返回。表现可能是 **ret** 跳到奇怪地址、GDB 回溯断裂、库函数内部崩溃、异常展开失败。

第四类错误是符号和链接边界错误。C++ 声明没有 **extern "C"**，汇编导出符号名与 C++ 期望不一致，动态库里导出了不同版本符号，或者链接到了另一个同名函数。表现可能是链接失败，也可能是运行时绑定到意外实现。

第五类错误是对象布局和所有权错误。调用双方对结构体大小、字段 offset、对齐、返回方式、内存所有权理解不同。表现可能是数值字段错位、隐藏指针写坏内存、释放不属于自己的内存，或者跨库析构时崩溃。

核心思想

ABI 调试要围绕边界状态，而不是围绕源码直觉。调用前、入口、调用内部、返回前、返回后，这几个时间点的寄存器和栈状态，比单独看最终输出更能说明问题。

ABI 实验报告怎么写

本章实验的目标不是把命令跑通，而是训练可复现的二进制接口分析。报告应包含五部分。

第一，环境和构建条件。写清楚 CPU 架构、操作系统、编译器名称和版本、优化级别、是否保留帧指针、是否使用 PIE、源文件列表和构建命令。ABI 现象会受平台和编译选项影响，缺少这些信息，别人无法复现实验。

第二，函数签名和手工 ABI 推导。对每个被分析函数，先列出返回类型、参数类型、预期参数位置、返回位置、是否存在栈参数、是否可能存在隐藏 sret 指针。这个推导要写在反汇编之前，因为它是你判断反汇编的标准。

第三，反汇编证据。只摘取与结论相关的指令：调用点如何放置参数，被调用者如何读取参数，返回值如何写入，保存寄存器如何入栈和出栈，调用前如何调整 rsp。不要把完整 objdump 无筛选粘贴进报告。大段原始输出会掩盖分析，也不会提高质量。

第四，动态观察。用 GDB 或同类工具在关键位置记录寄存器、栈顶、返回地址和内存写入。记录应对应你的推导。例如证明第 7 个参数在 [rsp+8]，就要在函数入口停住并查看该地址；证明 rbx 被恢复，就要记录调用前和返回后的值。

第五，结论和边界。说明哪些现象是 ABI 规则必然导致，哪些是当前编译器在当前优化级别下的选择，哪些只是实验中观察到但不能推广的现象。例如“第一个整数参数在 rdi”属于 ABI 规则；“编译器选择用 rbx 保存某个局部值”属于编译器选择；“这次错误栈对齐没有崩溃”只是实验现象，不能作为正确性结论。

常见误区

高质量实验报告不靠代码量取胜。真正重要的是：推导是否清楚，证据是否对应结论，是否区分规则、观察和推测，是否能让别人按同样条件复现。

ABI 推导要从函数类型开始

ABI 分析最常见的坏习惯，是先看反汇编，再反过来猜参数。正确顺序应该相反：先从函数类型推导调用边界应该是什么，再用反汇编验证编译器或手写汇编是否满足。函数类型包括返回类型、每个参数的类型、是否为成员函数、是否为变参函数、是否有隐藏返回指针、是否经过 extern"C" 暴露、是否跨动态库边界。缺少这些信息，只凭寄存器很容易误判。

一个 ABI 推导表至少包含：

项目	要问的问题	证据来源
返回类型	寄存器返回、内存返回还是无返回值	函数声明、类型大小、平凡性
显式参数	每个参数属于整数、指针、浮点、向量还是聚合	函数声明、类型定义
隐藏参数	是否有 this、sret、虚调用辅助信息	C++ 语义、返回类型、成员函数形式
栈参数	寄存器通道是否耗尽，是否有溢出到栈	参数列表、ABI 分类
保存责任	调用者和被调用者各自要保存什么	System V 规则、函数内部调用情况
可见符号	链接器看到的名字是什么	nm、readelf-s、extern"C"

推导表写完后，再看调用点。调用点应当把参数放到正确寄存器或栈位置，满足调用前栈对齐，处理变参所需的额外状态，并在调用后从正确位置取返回值。接着看被调用者入口。被调用者不需要知道调用者源码长什么样，它只依赖 ABI 状态：寄存器里有什么，栈顶是什么，哪些寄存器可自由使用，哪些必须恢复。

同一段机器码在不同声明下含义不同

假设有一个手写汇编函数：

```
asm_f:
    mov eax, edi
    add eax, esi
    ret
```

若 C++ 声明是：

```
extern "C" int asm_f(int, int);
```

它可以解释为返回两个 `int` 参数之和。若声明错误地写成：

```
extern "C" long asm_f(long, long);
```

机器码仍然只写 `eax`，高 32 位被清零，返回值语义就不是 64 位 signed 加法。若声明写成：

```
extern "C" double asm_f(double, double);
```

调用者会把参数放入 `xmm0`、`xmm1`，而汇编函数读取 `edi`、`esi`，完全错位。运行可能没有立刻崩溃，但结果没有 ABI 意义。

这说明 ABI 正确性是“声明、调用点、被调用实现”三方一致，不是单个函数看起来能运行。跨语言、跨文件、跨动态库时，头文件就是契约的一部分。任何一方对类型、返回方式、符号名、所有权理解不同，都会形成 ABI 错误。

栈对齐要按时间点计算

栈对齐规则容易背错，原因是很多资料只说“16 字节对齐”，却不说明是调用前、调用后还是函数入口。System V AMD64 的实用读法是：调用者在执行 `call` 前，要让栈满足被调用者 ABI 期望；`call` 会把 8 字节返回地址压栈，所以被调用者入口看到的 `rsp` 通常与 16 字节边界相差 8。被调用者若还要调用其他函数，就要在自己的调用点前重新调整。

用时间线写更清楚：

```
caller before call:
    rsp mod 16 = 0

call target:
    push return address

callee entry:
    rsp mod 16 = 8
    [rsp] = return address

callee before nested call:
    adjust stack if needed
    rsp mod 16 = 0
```

具体函数中，`push rbp`、保存 callee-saved 寄存器、`sub rsp, N` 都会改变对齐。不要只看 `sub rsp, 8` 或 `sub rsp, 16` 单条指令，要从入口一路累计。若函数是 leaf 且不调用其他函数，可能根本不调整栈；若使用 red zone，可能在 `rsp` 下方放临时数据但不移动 `rsp`；若需要对齐局部对象或调用需要栈参数的函数，调整会更复杂。

栈对齐错误为什么有时不崩溃

错误栈对齐不一定每次立即崩溃。若被调用函数只使用整数寄存器，可能暂时看不出问题；若库函数内部使用要求对齐的 SIMD load/store，才可能触发异常或显著变慢；若编译器选择了不要求对齐的指令，也可能只是性能下降。不能用“一次运行没崩溃”证明栈对齐正确。

诊断栈对齐时，要在每个调用点前记录 `rspmod16`，而不是只在函数入口看一次。手写汇编里尤其要检查所有路径：正常路径、错误路径、早返回路径、尾调用路径。一个路径多 `push` 少 `pop`，或在 `ret` 前没有恢复 `rsp`，可能直到返回很久以后才表现为崩溃。

栈参数和返回地址的相对位置

函数入口处，`rsp` 指向返回地址。若有栈上传入的参数，它们位于返回地址之上。也就是说，入口 `[rsp]` 是返回地址，`[rsp+8]` 才是第一个栈参数位置，具体还要考虑调用者传参时的布局和类型宽度。若被调用者执行 `push rbp` 或 `sub rsp, N`，这些相对 `rsp` 的偏移会变化；用 `rbp` 建帧时，栈参数常可以用正偏移访问。

初学者常把局部变量和栈参数混在一起。局部变量通常在当前函数调整后的栈空间中，属于被调用者自己的临时存储；栈参数由调用者放置，属于调用边界的一部分。两者都在栈上，但所有权和生命周期不同。报告中应分别写“入口栈参数位置”和“函数内部局部栈槽位置”。

聚合类型传参：不要只看大小

结构体、数组包装类型、`std::pair`、小向量和自定义类的 ABI 分类比整数参数复杂。第一册不求背完整 System V 分类算法，但必须避免一个错误直觉：不是“结构体小就一定放寄存器，大就一定放栈”这么简单。大小、字段类型、对齐、是否跨 `eightbyte`、是否包含浮点、是否非平凡拷贝/析构，都会影响传参和返回方式。

小结构体可能拆到多个寄存器

例如：

```
struct PairI {
    int a;
    int b;
};

extern "C" int sum_pair(PairI p)
{
    return p.a + p.b;
}
```

`PairI` 通常可以放在一个 `eightbyte` 中，通过一个整数寄存器传递。被调用者可能用移位或高低半部分提取字段。若是：

```
struct TwoLong {
    long a;
    long b;
};
```

它可能占两个 `eightbyte`，分别通过两个整数寄存器传递。报告中若只写“结构体参数在 `rdi`”，可能漏掉第二个寄存器。要根据结构体布局和 ABI 分类写清楚每个 `eightbyte` 的位置。

混合浮点和整数会走不同通道

例如：

```
struct Mix {
    double x;
    int tag;
};
```

它可能同时使用浮点寄存器和整数寄存器，具体取决于 ABI 分类和字段布局。调用点可能把 `x` 放到 `xmm0`，把 `tag` 放到某个通用寄存器。若你只按参数序号数通用寄存器，就会错位。混合参数列表也一样：整数通道和 SSE 通道分别计数，`int, double, int` 不等于三个都用通用寄存器。

非平凡 C++ 类型可能退化为地址传递

C++ 类型若有非平凡拷贝构造、析构或移动语义，ABI 可能不把它当成普通字节包直接拆寄存器传递，而是通过内存地址间接传递，保证对象生命周期语义。即使大小很小，也不能只凭 `sizeof` 判断。阅读 C++ 与汇编接口时，必须检查类型是否是平凡可复制、标准布局、是否有用户定义构造/析构、是否跨翻译单元暴露。

这也是为什么 C ABI 边界常建议使用简单整数、指针和不透明句柄，而不是直接暴露复杂 C++ 类。复杂类型的 ABI 不只与字段字节有关，还与编译器、标准库、异常模型、可见性和版本兼容有关。第一册先建立这个边界，后续工程章节再展开库 ABI 稳定性。

返回值位置要和调用者读取方式配套

返回值错误经常比参数错误更隐蔽。调用者按照声明决定从哪里读取返回值；被调用者按照实现写某个位置。两者不一致时，可能表现为返回值错、隐藏指针被写坏、栈被破坏，或对象析构异常。

普通整数返回通常在 `rax` 的有效低位。返回 `int` 只要求低 32 位有意义；返回 `bool` 可能只看低 8 位；返回指针或 `std::uintptr_t` 需要完整 64 位。若手写汇编函数声明返回指针，却只写 `eax`，高 32 位会被清零，地址可能被截断。若声明返回 `long`，但实现只做 32 位运算，也可能改变负数扩展语义。

浮点返回通常使用 `xmm0`；某些小聚合返回可能使用 `rax/rdx` 或 `xmm0/xmm1`；大结构体返回常使用隐藏 `sret` 指针。调用者会先准备一块返回对象存储，把指针作为隐藏参数传给被调用者；被调用者把结果写到那里，并可能把该指针也放回 `rax`。若手写汇编不知道隐藏参数，就会把第一个显式参数错认为 `rdi`，实际 `rdi` 可能是返回存储地址。

`sret` 错位的典型症状

假设 C++ 声明：

```
struct Big {
    long a;
    long b;
    long c;
};

extern "C" Big make_big(long x);
```

调用者可能做的是：

```
rdi = address of caller-provided return storage
rsi = x
call make_big
```

若汇编实现误以为 `rdi` 是 `x`，它就会把返回存储地址当整数参数使用；若它又写 `[rdi]`，可能刚好写到返回对象，但字段和参数全错。更糟的是，若它把 `rdi` 当普通指针参数解引用，可能破坏调用者栈上的返回对象区域。诊断这种问题的第一步，就是根据返回类型判断是否存在隐藏 `sret` 指针。

caller-saved 与 callee-saved 是活跃值协议

很多教材把 caller-saved 和 callee-saved 讲成死记表。更好的理解是：它们规定跨调用的活跃值由谁负责保护。caller-saved 寄存器不是“调用后一定被破坏”，而是“调用者不能要求它保持”。callee-saved 寄存器不是“被调用者不能用”，而是“用了就必须在返回前恢复”。

调用者若有一个值在 `call` 后还要用，有三种常见选择：放到 callee-saved 寄存器并在当前函数序言/尾声保存恢复；放到栈槽；在调用后重新计算；或者如果值本来在内存中，调用后重新 `load`。编译器会根据寄存器压力和成本选择。手写汇编必须自己做同样的责任分配。

被调用者若使用 `rbx`、`rbp`、`r12` 到 `r15`，通常要保存旧值并在所有返回路径恢复。例如：

```
push rbx
mov rbx, rdi
call helper
mov rdi, rbx
call other
pop rbx
ret
```

这里 `rbx` 用来跨调用保存原始参数。正确性不在于 `rbx` 有没有变过，而在于返回给调用者前是否恢复为入口值。若中间有早返回，也必须经过恢复路径。报告中应检查每条 `ret` 前 callee-saved 状态，而不是只看主路径。

caller-saved 失效也包括 flags 和向量寄存器

普通函数调用后，不仅通用 caller-saved 寄存器不能继续相信，flags 也不能继续相信。不要在 `cmp` 和 `jcc` 中间插入普通 `call` 后还期待 flags 保持。浮点和向量寄存器也有调用约定，某些寄存器属于

caller-saved。若函数调用前后要保留 `xmm` 中的值，调用者要自己安排。

内存状态还要看别名和副作用。调用一个未知外部函数后，编译器通常必须假设它可能通过指针、全局变量或可见别名修改内存。若某个 `load` 在调用前后都出现，不要急着说重复；可能是因为调用使内存值失效，编译器必须重新读取。ABI 规定寄存器保存责任，语言和别名规则规定内存可见性，两者要一起看。

变参函数是 ABI 的压力测试

变参函数把 ABI 的复杂性集中暴露出来。固定参数部分可以从函数原型知道类型；可变参数部分需要运行时按约定读取。调用者和被调用者必须约定整数寄存器、浮点寄存器和栈参数如何保存到可遍历区域，`va_list` 如何描述当前位置，以及调用变参函数时额外状态如何传递。

在 System V AMD64 中，调用变参函数时，`al` 常用于表示使用了多少个向量寄存器传递浮点参数。读 `printf` 调用汇编时，看到调用前设置 `eax` 或 `al`，不要误以为它是普通整数参数；它可能是变参 ABI 所需元信息。若手写汇编调用 `printf`，忽略这点可能在含浮点参数时出错。

格式串错误不是 ABI 能修复的

`printf` 这类函数依赖格式串解释后续参数。ABI 只规定参数如何传递，不知道格式串是否真实匹配。如果格式串写 `%f`，但调用者传了 `int`；或格式串写 `%ld`，但传了 `int`，被调用者会按错误类型从寄存器保存区或栈中取数据。结果可能是乱码、错位读取或崩溃。这不是 ABI 自动检测的错误，而是调用契约不一致。

默认参数提升也要注意。传给 C 风格变参的 `float` 会提升为 `double`，小整数会提升为 `int` 或 `unsignedint`。因此变参调用点的实际 ABI 类型可能不同于源码里字面写的小类型。报告分析变参时，要写“传入变参前经过了哪些提升”，再看寄存器或栈位置。

ABI 稳定性是工程设计问题

本章主要教你读单个调用点，但真实项目更关心 ABI 是否长期稳定。只要一个函数或类型跨目标文件、动态库、插件、JIT、脚本绑定、手写汇编、FFI 边界暴露，它就变成工程契约。改变参数顺序、返回类型、结构体字段、对齐、异常传播、所有权规则，都可能破坏已有二进制。

为什么 C 接口常用不透明句柄

不透明句柄是一种常见 ABI 设计：

```
extern "C" Engine* engine_create();
extern "C" void engine_destroy(Engine*);
extern "C" int engine_run(Engine*, char const* input);
```

调用者只知道 `Engine*` 是一个指针，不知道 `Engine` 内部布局。库可以在不改变 ABI 的情况下增删字段、调整容器、替换实现。若直接暴露：

```
struct Engine {
    int mode;
    double threshold;
    void* impl;
};
```

那么字段顺序、大小、对齐都进入 ABI。调用者可能在自己的编译单元中按旧 `offset` 访问字段，库升级后就会错位。不透明句柄牺牲一部分直接访问便利，换来布局演化空间。

所有权规则要写进 ABI 文档

ABI 不只包括寄存器和栈，还包括谁分配、谁释放、能否跨库释放、字符串是否以零结尾、缓冲区长度单位是什么、错误码如何返回、线程安全如何保证。很多跨语言崩溃不是参数寄存器错了，而是所有权错了：一个库用自己的 `allocator` 分配，另一个模块用不同运行库释放；调用者传入临时缓冲区，被调用者保存指针长期使用；返回字符串没有说明生命周期。

高质量接口文档应写清楚：

- 参数指针是否可为空。
- 缓冲区长度单位是字节、元素还是字符。

- 被调用者是否会写入缓冲区，最多写多少。
- 返回指针由谁释放，用哪个函数释放。
- 错误如何报告，返回值和输出参数在错误时是否有效。
- 回调是否可重入，是否跨线程调用。

这些内容看起来不像“汇编”，但它们决定二进制边界是否可用。底层工程不是只把参数放进正确寄存器，还要让调用双方对对象、内存和错误状态有同一个合同。

ABI 审查清单

分析或编写一个 ABI 边界函数时，可以按下面清单审查。

1. 声明是否唯一。所有调用方是否包含同一个头文件，是否存在手写重复声明。
2. 符号是否匹配。C++ 名字修饰、`extern "C"`、可见性、版本脚本是否符合预期。
3. 返回方式是否正确。小整数、指针、浮点、小聚合、大聚合、非平凡对象是否按 ABI 返回。
4. 参数通道是否正确。整数、指针、浮点、向量、聚合、隐藏参数和栈参数是否逐项推导。
5. 栈状态是否正确。入口、调用前、返回前的 `rsp` 是否满足规则。
6. 保存责任是否正确。callee-saved 寄存器是否所有路径恢复，caller-saved 是否调用后重新建立。
7. 变参是否正确。`al`、默认提升、格式串和实际参数是否一致。
8. 内存所有权是否明确。输入、输出、返回指针、错误路径是否有清晰生命周期。
9. 异常和不返回路径是否明确。是否允许 C++ 异常跨 C ABI，`[[noreturn]]` 是否被调用者和调用者一致理解。
10. 测试是否覆盖边界。负数、大值、空指针、栈参数、浮点参数、大结构体、错误路径是否都测到。

这张清单的目的不是让每个小函数都写长篇文档，而是在出现跨语言、手写汇编、动态库或性能关键包装时，避免遗漏最常见的灾难点。越是底层边界，越不能靠“看起来能跑”验收。

跨平台 ABI：不要把一套规则带到所有机器

本章以 Linux 上常见的 System V AMD64 ABI 为主。这个选择是为了让第一册有一个具体、可实验的目标，而不是因为所有 x86-64 平台都这样工作。Windows x64、Linux syscall ABI、macOS 平台约定、AArch64 ABI、RISC-V ABI 都可能在参数寄存器、栈空间、红区、向量寄存器、异常展开、名字修饰和系统调用入口上不同。

最常见的误用是把 Windows x64 的前四个整数参数寄存器 `rcx`、`rdx`、`r8`、`r9` 套到 System V 上。System V 常见整数参数顺序是 `rdi`、`rsi`、`rdx`、`rcx`、`r8`、`r9`。如果手写汇编按错平台读取参数，前两个参数就会错位。程序可能仍然运行，但结果毫无 ABI 保证。

另一个差异是栈约定。Windows x64 有 shadow space 概念，System V 有 red zone 概念；二者不能混用。一个在 Linux leaf function 中使用 red zone 的技巧，移植到 Windows x64 不成立；一个按 Windows shadow space 写的调用序列，放到 System V 上也不是同一套规则。跨平台汇编必须为每个 ABI 写独立入口，或通过 C++ wrapper 隔离平台差异。

系统调用 ABI 和函数调用 ABI 不同

即使在同一台 Linux x86-64 机器上，普通函数调用 ABI 和系统调用 ABI 也不同。普通函数把第四个整数参数放在 `rcx`；Linux syscall 使用 `r10` 传第四个参数，且 `syscall` 会破坏 `rcx` 和 `r11`。如果把普通 C 函数调用规则套到 `syscall` 上，参数会错。

第一册不鼓励初学者直接手写 `syscall` 包装，但要知道这条边界。调用 libc 的 `write` 是普通函数调用，最终 libc 内部可能执行 `syscall`；你直接写 `syscall`，就要遵守 `syscall` ABI。两个层次都叫“调用”，但合同不同。

跨 ABI 的安全做法

如果项目需要跨平台，建议把汇编实现藏在平台专用文件中：

```
src/
```

```
dot_ref.cpp
dot_dispatch.cpp
dot_sysv_x86_64.S
dot_win_x64.asm
dot_aarch64.S
```

公共 C++ 头文件只暴露稳定 wrapper:

```
std::int64_t dot_i32(std::span<const std::int32_t> a,
                    std::span<const std::int32_t> b);
```

wrapper 根据平台选择内部符号。内部符号可以各自遵守本平台 ABI，测试也按平台分别验证。不要试图写一份汇编同时假装满足所有 ABI，除非它只使用非常受限的公共子集且已经明确验证。

ABI 错误案例库

ABI 错误最好通过案例学习，因为它们的症状常常离根因很远。

案例一：忘记恢复 **rbx**

手写汇编：

```
bad:
    mov rbx, rdi
    call helper
    mov rax, rbx
    ret
```

这个函数用 **rbx** 保存参数，但没有保存和恢复旧 **rbx**。如果调用者正好把某个活跃值放在 **rbx**，返回后该值被破坏。小测试可能没问题，因为调用者未使用 **rbx**；优化构建或复杂调用者中才暴露。

诊断方法是在调用前后记录 **rbx**：

```
before call:
    rbx = sentinel
after call:
    rbx should still be sentinel
```

修复方式是：

```
good:
    push rbx
    mov rbx, rdi
    call helper
    mov rax, rbx
    pop rbx
    ret
```

修复后还要重新检查栈对齐。因为 **push rbx** 改变了 **rsp**，如果中间还有 **call helper**，调用前对齐可能需要额外调整。ABI 修复常常牵动多个规则。

案例二：返回指针只写 **eax**

错误实现：

```
get_ptr:
    mov eax, edi
    ret
```

若声明是：

```
extern "C" void* get_ptr(void*);
```

这个实现把输入指针低 32 位写入 **eax**，同时清零高 32 位。低地址测试可能碰巧通过，高地址或 ASLR 环境下会返回截断指针。正确实现应写完整 **rax**：

```
mov rax, rdi
ret
```


这个案例说明返回类型宽度必须和寄存器写入宽度一致。写 32 位寄存器的清零规则对无符号 32 位返回很方便，对指针返回则可能是灾难。

案例三：变参调用忘记 `al`

手写汇编调用 `printf` 时，如果格式串包含浮点变参，System V ABI 需要调用者在 `al` 中提供使用的向量寄存器数量。若忽略，某些调用可能读取错误的寄存器保存区。整数-only `printf` 可能看不出问题，含 `%f` 的格式才失败。

这个案例说明变参函数不能只按“参数放进寄存器”理解。变参 ABI 还需要额外元信息和默认提升。调用 `printf` 这类函数时，最稳妥做法是让 C++ wrapper 调用，或严格照 ABI 写调用序列。

案例四：栈对齐只在主路径正确

某函数主路径恢复了 `rsp`，错误路径提前返回：

```
f:
    sub rsp, 24
    test rdi, rdi
    je .Lerr
    call helper
    add rsp, 24
    ret
.Lerr:
    ret
```

`.Lerr` 路径没有 `add rsp, 24, ret` 会从错误位置取返回地址。若测试很少覆盖空指针路径，问题可能潜伏很久。控制流章节讲过，每个返回路径都要检查状态；ABI 章节在这里给出具体例子：栈状态必须所有路径一致。

修复方式通常是让错误路径跳到统一 epilogue：

```
f:
    sub rsp, 24
    test rdi, rdi
    je .Ldone
    call helper
.Ldone:
    add rsp, 24
    ret
```

ABI 与 C++ 对象语义的边界

ABI 能规定对象如何传递，但不能替你保证 C++ 对象语义正确。比如一个函数接收指针，ABI 只规定指针值在哪个寄存器里；它不保证指针非空、不保证对象生命周期有效、不保证类型别名合法、不保证指向足够大缓冲区。这些属于语言和接口契约。

同样，ABI 可以规定一个非平凡对象通过地址传递，但不会自动调用你忘记写的构造或析构；ABI 可以规定异常展开需要元数据，但裸汇编没有正确 CFI 时，异常穿过该帧仍可能失败；ABI 可以规定返回对象存储由调用者提供，但不会检查被调用者是否写满所有字段。

因此，跨 C++/汇编边界时，至少要分三层写文档：

- ABI 层：寄存器、栈、返回、保存责任、符号。
- 对象层：生命周期、所有权、空指针、长度、别名、对齐。
- 语义层：函数计算什么，错误如何处理，是否允许异常。

只写 ABI 层，函数可能被调用但语义不安全；只写语义层，手写汇编可能参数错位；只写对象层，性能和二进制边界仍不清楚。三层都写，接口才稳。

ABI 测试夹具设计

普通单元测试只检查返回值，不能覆盖 ABI 保存责任。可以设计专门夹具。

第一类夹具检查 callee-saved 寄存器。用一段汇编或 compiler-supported 机制在调用前设置 `rbx`、`r12` 到 `r15` 为哨兵值，调用目标函数后检查是否保持。若目标函数内部调用其他函数，也要覆盖。

第二类夹具检查栈对齐。写一个 helper，在入口读取 `rsp` 并返回 `rsp%16`，或在目标函数调用 helper 前让 helper 验证对齐。这样可以发现主路径和错误路径对齐不一致。

第三类夹具检查栈参数。构造超过六个整数参数、混合浮点和整数参数、大结构体返回、变参调用。若只测一两个整数参数，很难发现参数通道错位。

第四类夹具检查隐藏参数。测试成员函数、引用参数、大结构体返回、非平凡对象边界。确认第一个显式参数是否因为 `this` 或 `sret` 被后移。

第五类夹具检查动态库边界。把实现放进共享库，从另一个可执行文件调用，检查符号可见性、`RPATH/RUNPATH`、版本和异常边界。静态链接通过，不代表动态库部署正确。

核心理想

ABI 测试不应只问“返回值对不对”。它要故意检查调用者看不见的责任：寄存器保存、栈状态、隐藏参数、变参元信息、符号和动态库边界。

综合案例：一个函数签名的 ABI 推导

看一个混合签名：

```
struct Pair {
    double x;
    int tag;
};

extern "C" long f(int a,
                 double b,
                 long c,
                 Pair p,
                 int d,
                 double e,
                 long g);
```

不要直接从参数序号数寄存器。System V AMD64 中，整数/指针通道和浮点/SSE 通道分别计数，聚合类型还要分类。一个推导过程可以写成：

```
return:
    long -> rax

arguments:
    int a    -> integer channel 1 -> edi
    double b -> SSE channel 1    -> xmm0
    long c   -> integer channel 2 -> rsi
    Pair p   -> aggregate classification:
        double x likely SSE eightbyte
        int tag likely INTEGER eightbyte or memory depending classification
    int d    -> next integer channel
    double e -> next SSE channel
    long g   -> next integer channel
```

这里故意不把 `Pair` 的最终位置写死，因为完整分类需要按 ABI 规则仔细判断字段、padding 和 eightbyte。教材式报告应明确“需要分类”，而不是凭直觉把整个结构体塞进一个寄存器。实际实验可以通过编译小函数、查看调用点和被调用者读取位置来验证。

这个案例训练两点。第一，浮点参数不会占用整数寄存器通道。第二，聚合参数不能只看总大小，还要看字段类别和 eightbyte 分布。复杂签名的 ABI 推导应先在纸上列通道，再看反汇编验证。

如何用实验验证推导

可以写一个调用者：

```
extern "C" long f(int, double, long, Pair, int, double, long);

long call_f()
{
    Pair p{1.5, 7};
    return f(1, 2.5, 3, p, 4, 5.5, 6);
}
```

生成 `-O0` 和 `-O2` 汇编。`-O0` 可能更容易看到参数准备；`-O2` 更接近发布路径，但可能内联或优化。若要保留调用，可以把 `f` 放在另一个翻译单元，或只声明不定义。报告应记录采取了什么方法保留调用边界。

验证时看调用点如何写 `edi`、`rsi`、`xmm0` 等寄存器，以及是否有栈参数。再看被调用者入口如何读取。调用点和入口应一致。若不一致，可能是你看了不同构建、函数被内联、声明不一致，或 ABI 推导错了。

自测清单：ABI 是否真正掌握

完成本章后，应能回答：

1. 为什么 ABI 是调用者和被调用者的共同合同。
2. 前六个整数/指针参数在 System V AMD64 中通常在哪里。
3. 浮点参数为什么使用独立通道。
4. 第七个整数参数在入口栈上的相对位置如何判断。
5. `rsp` 在调用前和被调用者入口的对齐关系。
6. caller-saved 和 callee-saved 分别意味着什么责任。
7. 大结构体返回为什么可能引入隐藏 `sret` 指针。
8. 非平凡 C++ 类型为什么不能简单当字节包传递。
9. 变参函数为什么需要额外 ABI 规则。
10. Windows x64、System V 和 Linux syscall ABI 为什么不能混用。

如果只能背寄存器顺序，但不能从函数签名推导入口状态、不能检查栈对齐、不能发现隐藏参数，就还没有真正掌握 ABI。

深入讲解：ABI 是局部优化的边界

ABI 不只是正确性规则，也是优化边界。函数内部，编译器可以自由选择寄存器、重排指令、删除局部变量、内联 helper；函数边界处，它必须让调用者和被调用者看到约定好的状态。这个边界限制了优化，也让独立编译成为可能。

如果一个小函数被内联，ABI 边界消失，参数不一定真的放进 `rdi`、`rsi`，返回值也不一定经过 `rax`。值可以直接在调用者的寄存器中流动，甚至被常量折叠。若函数没有内联，ABI 边界保留，调用者必须准备参数，被调用者必须返回结果。性能上，内联常常消除调用开销并暴露优化机会，但也可能增加代码大小。

动态库边界更强。编译器通常不能假设外部可见函数不会被替换，也不能随意改变公共符号 ABI。隐藏符号和 LTO 可以放宽一些限制，但前提是构建系统和链接模型明确。库作者控制符号可见性，不只是为了整洁，也是在给优化器提供边界信息。

手写汇编则处在最硬的 ABI 边界。编译器看不到汇编内部语义，只能按声明和 clobber/符号规则处理。汇编实现必须完全遵守合同。你不能指望编译器帮你保存 `rbx`，也不能指望调用者猜到你的隐藏前置条件。

课堂讲解：ABI 推导要从函数类型开始

很多读者学习 ABI 时会先背“前六个整数参数放在 `rdi`、`rsi`、`rdx`、`rcx`、`r8`、`r9`”。这句话有用，但不够。真正的 ABI 推导必须从函数类型开始：返回类型是什么，参数类型分别属于整数、指针、浮点、向量还是聚合，是否有隐藏参数，是否是变参函数，是否有非平凡 C++ 语义，调用点是否保留函数边界。

推导流程可以固定为六步。第一，写出精确声明，包含命名空间、`extern "C"`、引用、指针、`const`、结构体定义和返回类型。第二，判断返回值位置：普通整数、指针、浮点、大聚合、非平凡类型分别不同。第三，逐个参数分类，并分别维护整数通道和 SSE 通道。第四，判断是否溢出到栈，栈上参数位置要结合调用前后的 `rsp` 变化。第五，考虑调用者和被调用者保存寄存器。第六，用反汇编验证调用点和入口是否一致。

这个流程比背表慢一点，但能处理真实签名。真实项目里的函数很少只有六个整数。它们可能有 `std::size_t`、指针、`double`、小结构体、返回结构体、变参、成员函数、引用、模板实例化和异

常边界。只靠寄存器顺序，遇到第一个聚合参数就会出错。

调用点和被调用者入口要成对看

ABI 是调用者和被调用者的共同合同，所以验证时不能只看一边。调用点显示参数如何准备、栈如何对齐、返回值如何使用；被调用者入口显示参数如何读取、哪些寄存器被保存、栈帧如何建立。两边合起来，才能确认合同一致。

若只看被调用者入口，可能误判优化器已经把某个参数常量传播或删除。若只看调用点，可能不知道被调用者是否破坏 callee-saved 寄存器或错误解释参数宽度。互操作问题尤其需要成对证据：C++ 声明、调用点反汇编、汇编入口、测试结果必须互相匹配。

ABI 和 C++ 类型系统的边界

ABI 描述机器级传递规则，但 C++ 类型系统还包含构造、析构、复制、移动、异常、访问控制和对象生命周期。不是所有 C++ 类型都适合直接跨汇编边界传递。平凡、标准布局、固定宽度字段的小结构体相对容易审计；含虚函数、非平凡析构、引用成员、内部指针、allocator 或不稳定布局的类型，通常应由 C++ wrapper 处理，不应直接交给裸汇编解释。

引用在 ABI 上通常以地址形式传递，但语言语义不是“普通可空指针”。若汇编把引用当成可空指针处理，说明接口层已经混乱。异常也类似。汇编函数如果不建立正确的 unwind 信息，就不应该让 C++ 异常穿过它。析构和资源清理依赖栈展开，手写汇编若破坏 unwind 规则，错误会很难排查。

成员函数还涉及隐藏的 `this` 参数。普通非静态成员函数的第一个隐含参数是对象指针；虚函数调用还可能经过 `vtable`；多重继承可能需要 `this` 调整。第一册不深入 C++ 对象模型，但要明确：看到“一个参数的成员函数”时，ABI 层可能有不止一个来源的值。做汇编互操作时，优先使用简单 C ABI 包装复杂 C++ 类型。

动态库 ABI：兼容比能链接更难

一个程序能链接到共享库，不代表 ABI 兼容长期成立。共享库一旦发布，外部程序可能保存旧头文件、旧二进制和旧调用方式。库作者改变函数签名、结构体大小、枚举宽度、异常策略、符号名或可见性，都可能破坏旧调用者。ABI 兼容是二进制层承诺，不是源码重新编译后能过就算。

保持动态库 ABI 稳定的常见做法包括：导出最小符号集合，隐藏内部函数；公共结构体使用不透明指针或版本字段；新增函数而不是改变旧函数签名；为参数块保留 `size` 字段；使用版本脚本管理符号；在 CI 中用 ABI 检查工具或反汇编审计关键符号。对小项目来说，不一定一开始全做，但必须知道问题存在。

调用者也有责任记录库身份。性能或崩溃报告中，应保存实际加载库路径和版本。若系统升级后共享库实现变了，同一可执行文件也可能表现不同。动态库边界把“程序身份”延伸到了运行环境，这是第一章和本章相互连接的地方。

ABI 错误的典型症状

ABI 错误不一定立刻崩溃。它可能表现为返回值偶尔错误、只在优化构建失败、调用后某个无关变量被破坏、异常路径崩溃、`printf` 参数错位、栈回溯断裂、只在某个输入规模出现性能异常。原因是 ABI 错误常常破坏调用者和被调用者之间的隐含状态，例如 callee-saved 寄存器、栈对齐、返回地址、隐藏参数或寄存器高位。

排查时要按合同逐项核对。参数位置是否一致；返回值宽度是否一致；`rsp` 在 `call` 前是否按要求对齐；被调用者是否保存并恢复 `rbx`、`rbp`、`r12` 到 `r15`；调用变参函数前是否设置需要的寄存器状态；结构体返回是否使用隐藏指针；调用声明是否和定义完全相同。不要只看功能测试是否通过，ABI 错误可能在另一个调用者处暴露。

ABI 审查清单

对任何跨翻译单元、跨语言或 C++ 调汇编接口，都应至少审查：

1. 声明是否唯一，调用者和实现是否包含同一头文件。
2. 符号名是否符合预期，是否需要 `extern"C`。

3. 参数类型、宽度、signedness、指针 const 和返回类型是否一致。
4. 结构体布局是否用 `static_assert` 固定。
5. 栈对齐是否在所有调用路径上成立。
6. callee-saved 寄存器是否完整保存。
7. 是否调用其他函数，若调用是否满足 non-leaf 责任。
8. 是否允许异常、回调、线程局部状态或全局状态穿过边界。
9. 是否有反汇编证据和运行测试覆盖边界参数。
10. 公共 ABI 变化是否版本化。

ABI 的难点不是规则多，而是错误经常跨文件、跨工具、跨运行时暴露。清单化审查能把隐含合同变成显式事实。

ABI 学习中的反例训练

ABI 最好通过反例学。写一个汇编函数故意不保存 `rbx`，再让调用者在调用前后依赖 `rbx` 保存的值；写一个函数故意破坏栈对齐，再调用需要对齐的 helper；写一个 C++ 声明返回 `long`，汇编却只写低 32 位；写一个参数块，C++ 端改变字段顺序而汇编 offset 不更新。每个反例都能把一条 ABI 规则变成可观察错误。

反例实验要在安全的小程序里完成，不要在生产代码中破坏 ABI。目标是观察症状：有的立即崩溃，有的只在优化构建下错误，有的只在特定调用者处暴露，有的只是返回值异常。观察后再修复，并保存修复前后的反汇编和测试。这样学习，比单纯背寄存器表更稳。

ABI 报告的最低标准

一份 ABI 报告至少应包含函数声明、平台和调用约定、参数分类表、返回值位置、栈对齐说明、保存寄存器责任、调用点反汇编、被调用者入口反汇编和测试结果。若涉及结构体，还要包含布局断言；若涉及变参，还要说明额外规则；若涉及动态库，还要说明符号可见性和加载库路径。

报告中不要只写“按 System V ABI”。这句话太宽。你要说明当前函数如何应用 ABI：哪个参数在哪个寄存器，哪个参数上栈，哪个隐藏参数存在，哪个寄存器必须保存。ABI 是文档，工程报告要把文档实例化到当前接口。

ABI 分析的最后一道问题

ABI 分析结束前，要问一句：调用者和被调用者是否真的在同一个合同上。声明一致不一定够，因为可能有不同头文件、不同编译选项、不同语言链接、不同结构体布局、不同动态库版本。反汇编看起来合理也不一定够，因为你可能只看了被调用者，没有看调用点。

完整检查要把四样东西放在一起：C++ 声明、调用点反汇编、被调用者入口反汇编、运行测试。若涉及共享库，还要加上实际加载路径和符号表。若涉及结构体，还要加上布局断言。只有这些证据一致，才能说 ABI 合同成立。

这个问题在第二册仍然重要。SIMD kernel、AI 算子库、运行时 dispatch、插件接口，都依赖 ABI 边界。越是追求性能，越不能让调用边界含糊。一个寄存器保存错误、一个隐藏参数误解、一个结构体 offset 漂移，都可能让高性能代码变成高风险代码。

综合项目：写一份 ABI 合同文件

本章结束时，可以选择一个真实或练习用函数，为它写一份 ABI 合同文件。文件不需要长，但必须精确。第一段写公共声明和用途；第二段写参数和返回值在 System V AMD64 下的位置；第三段写前置条件，例如指针非空、长度范围、对齐要求和是否允许别名；第四段写寄存器保存、栈对齐、是否调用其他函数、是否抛异常；第五段写测试和反汇编证据。

例如一个内部函数 `asm_sum_i32`，合同文件应写明 `rdi` 是指针，`rsi` 是元素数，`rax` 返回 64 位和，`n > 0`，若 `n == 0` 不访问内存，函数不调用其他函数，不修改 callee-saved 寄存器。若未来实现改成调用 helper，合同就必须更新栈对齐和调用责任。

这份文件能帮助多人协作。C++ 调用者知道如何声明，汇编作者知道要遵守什么，测试作者知道测哪些边界，reviewer 知道检查什么。ABI 合同文件是从“我懂规则”到“团队能维护接口”的关键一步。

ABI 变更的迁移策略

接口总会演化。长度参数可能从 `int` 改成 `std::size_t`，返回值可能增加错误码，参数块可能新增字段，内部实现可能需要对齐保证。ABI 变更不要悄悄发生。迁移策略可以是新增符号、保留旧符号 wrapper、参数块版本字段、编译期断言、运行时 size 检查和文档标记。

对内部接口，也建议把破坏性变化写进提交说明。因为旧目标文件、旧 benchmark、旧测试和旧文档可能仍在。若变更没有标记，性能数据也会混乱：v1 和 v2 不是同一个接口，不能直接画在一条趋势线上。ABI 迁移策略让变化可见，也让回滚和排查更容易。

深入讲解：ABI 文档应和测试一起存在

ABI 文档如果没有测试，很容易腐烂；测试如果没有文档，很难知道测什么。两者应该成对存在。文档写“callee-saved none modified”，测试就应检查寄存器；文档写“栈 16 字节对齐后调用 helper”，测试就应覆盖 helper；文档写“n 必须非负”，wrapper 测试就应拒绝负数；文档写“结构体 offset 固定”，编译期就应有 `static_assert`。

这种文档-测试对应关系能帮助 review。reviewer 不必凭信任接受注释，而能看到每条关键合同都有验证方式。对底层代码来说，这比单纯追求代码短更重要。一个汇编函数可能只有二十行，但它背后的 ABI 合同和测试可能比函数本身更长。

案例：C++ 调汇编

创建两个文件。

labs/ch13_system_v_abi/asm_add.S:

```
.intel_syntax noprefix
.text
.globl asm_add6
.type asm_add6, @function
asm_add6:
    lea eax, [rdi + rsi]
    add eax, edx
    add eax, ecx
    add eax, r8d
    add eax, r9d
    ret
.size asm_add6, .-asm_add6
.section .note.GNU-stack,"",@progbits
```

labs/ch13_system_v_abi/main.cpp:

```
#include <cstdio>

extern "C" int asm_add6(int, int, int, int, int, int);

int main()
{
    int r = asm_add6(1, 2, 3, 4, 5, 6);
    std::printf("%d\n", r);
}
```

构建：

```
clang++ -std=c++20 -O2 -c main.cpp -o main.o
clang++ -c asm_add.S -o asm_add.o
clang++ main.o asm_add.o -o abi_demo
./abi_demo
```

预期输出：

```
21
```

反汇编：

```
objdump -d -Mintel abi_demo > abi_demo.objdump.txt
```

你应该在 `main` 的调用点看到参数被放入 ABI 寄存器，在 `asm_add6` 中看到这些寄存器被读取。

案例：汇编调 C++

C++:

```
#include <stdio>

extern "C" int cpp_square(int x)
{
    return x * x;
}

extern "C" int asm_call_square(int);

int main()
{
    std::printf("%d\n", asm_call_square(9));
}
```

汇编:

```
.intel_syntax noprefix
.text
.globl asm_call_square
.type asm_call_square, @function
asm_call_square:
    sub rsp, 8
    call cpp_square
    add eax, 1
    add rsp, 8
    ret
.size asm_call_square, .-asm_call_square
.section .note.GNU-stack,"",@progbits
```

为什么要 `subrsp, 8`? 因为 `asm_call_square` 入口时 `rspmod16=8`。在它执行 `call cpp_square` 之前，必须让 `rspmod16=0`。

执行路径:

```
main:
    edi = 9
    call asm_call_square

asm_call_square:
    align stack
    call cpp_square
    eax = cpp_square(9) + 1
    restore stack
    ret
```

返回值:

```
cpp_square(9) = 81
asm_call_square(9) = 82
```

实验：参数寄存器和栈参数地图

创建 `labs/ch13_system_v_abi/args.cpp`:

```
#include <stdint>

extern "C" long sum8(long a, long b, long c, long d,
                    long e, long f, long g, long h)
{
    return a + b + c + d + e + f + g + h;
}

extern "C" double mix(int a, double b, int c, float d)
```

```

{
    return double(a + c) + b + double(d);
}

struct TwoLong {
    long a;
    long b;
};

extern "C" long use_two_long(TwoLong x)
{
    return x.a * 10 + x.b;
}

```

生成：

```

mkdir -p labs/ch13_system_v_abi/build
clang++ -std=c++20 -O0 -S -masm=intel args.cpp \
-o labs/ch13_system_v_abi/build/args_00.s
clang++ -std=c++20 -O3 -S -masm=intel args.cpp \
-o labs/ch13_system_v_abi/build/args_03.s
clang++ -std=c++20 -O3 -c args.cpp \
-o labs/ch13_system_v_abi/build/args.o
objdump -d -Mintel labs/ch13_system_v_abi/build/args.o \
> labs/ch13_system_v_abi/build/args.objdump.txt

```

交付物：

1. 对 `sum8` 列出 8 个参数分别在哪里。
2. 解释为什么第 7、第 8 个参数从栈读取。
3. 对 `mix` 列出整数参数和浮点参数的寄存器序列。
4. 对 `use_two_long` 判断结构体是否拆到寄存器。
5. 对比 `-O0` 和 `-O3` 栈帧差异。
6. 从 `objdump` 记录至少 8 条机器码字节。

实验：破坏 callee-saved 寄存器

创建 `labs/ch13_system_v_abi/callee_saved.S`：

```

.intel_syntax noprefix
.text

.globl bad_clobber_rbx
.type bad_clobber_rbx, @function
bad_clobber_rbx:
    mov rbx, 0x1234567812345678
    ret
.size bad_clobber_rbx, .-bad_clobber_rbx

.globl good_clobber_rbx
.type good_clobber_rbx, @function
good_clobber_rbx:
    push rbx
    mov rbx, 0x1234567812345678
    pop rbx
    ret
.size good_clobber_rbx, .-good_clobber_rbx

.globl check_bad
.type check_bad, @function
check_bad:
    push rbx
    mov rbx, 0x1122334455667788
    call bad_clobber_rbx
    mov rax, rbx
    pop rbx
    ret
.size check_bad, .-check_bad

```

```
.globl check_good
.type check_good, @function
check_good:
    push rbx
    mov rbx, 0x1122334455667788
    call good_clobber_rbx
    mov rax, rbx
    pop rbx
    ret
.size check_good, .-check_good

.section .note.GNU-stack,"",@progbits
```

创建 labs/ch13_system_v_abi/callee_saved.cpp:

```
#include <stdint>
#include <stdio>

extern "C" std::uint64_t check_bad();
extern "C" std::uint64_t check_good();

int main()
{
    std::printf("bad = 0x%llx\n",
        (unsigned long long)check_bad());
    std::printf("good = 0x%llx\n",
        (unsigned long long)check_good());
}
```

构建:

```
clang++ -std=c++20 -O2 -c callee_saved.cpp -o callee_saved.o
clang++ -c callee_saved.S -o callee_saved_asm.o
clang++ callee_saved.o callee_saved_asm.o -o callee_saved_demo
./callee_saved_demo
objdump -d -Mintel callee_saved_demo > callee_saved_demo.objdump.txt
```

交付物:

1. 解释 `bad_clobber_rbx` 违反了哪条 ABI 规则。
2. 解释 `good_clobber_rbx` 如何保存和恢复 `rbx`。
3. 说明 `check_bad` 为什么返回被破坏后的值。
4. 在 GDB 中单步观察 `rbx` 变化。
5. 把这个实验推广到 `r12`，写一个新的坏例子和修复例子。

实验：栈对齐和 `movaps`

创建 labs/ch13_system_v_abi/stack_align.S:

```
.intel_syntax noprefix
.text

.globl bad_movaps_store
.type bad_movaps_store, @function
bad_movaps_store:
    sub rsp, 16
    xorps xmm0, xmm0
    movaps xmmword ptr [rsp], xmm0
    add rsp, 16
    ret
.size bad_movaps_store, .-bad_movaps_store

.globl good_movaps_store
.type good_movaps_store, @function
good_movaps_store:
    sub rsp, 24
    xorps xmm0, xmm0
```

```

movaps xmmword ptr [rsp], xmm0
add rsp, 24
ret
.size good_movaps_store, .-good_movaps_store

.section .note.GNU-stack,"",@progbits

```

创建 labs/ch13_system_v_abi/stack_align.cpp:

```

#include <stdio>

extern "C" void bad_movaps_store();
extern "C" void good_movaps_store();

int main()
{
    std::puts("calling good");
    good_movaps_store();
    std::puts("calling bad");
    bad_movaps_store();
    std::puts("done");
}

```

构建并运行:

```

clang++ -std=c++20 -O2 -c stack_align.cpp -o stack_align.o
clang++ -c stack_align.S -o stack_align_asm.o
clang++ stack_align.o stack_align_asm.o -o stack_align_demo
./stack_align_demo

```

交付物:

1. 计算 bad_movaps_store 中 sub rsp, 16 后的栈对齐状态。
2. 计算 good_movaps_store 中 sub rsp, 24 后的栈对齐状态。
3. 解释 movaps 为什么要求目标地址对齐。
4. 如果程序崩溃, 用 GDB 或 shell 记录信号和崩溃位置。
5. 把 movaps 改成 movups, 观察行为是否变化, 并解释为什么不能把这当成“不需要 ABI 对齐”的证据。

实验: 大结构体返回和隐藏指针

创建 labs/ch13_system_v_abi/sret.cpp:

```

#include <stdint>

struct Small {
    std::uint64_t a;
    std::uint64_t b;
};

struct Big {
    std::uint64_t a;
    std::uint64_t b;
    std::uint64_t c;
};

extern "C" Small make_small(std::uint64_t x)
{
    return Small{x, x + 1};
}

extern "C" Big make_big(std::uint64_t x)
{
    return Big{x, x + 1, x + 2};
}

```

生成:


```
clang++ -std=c++20 -O0 -S -masm=intel sret.cpp -o sret_00.s
clang++ -std=c++20 -O3 -S -masm=intel sret.cpp -o sret_03.s
clang++ -std=c++20 -O3 -c sret.cpp -o sret.o
objdump -d -Mintel sret.o > sret.objdump.txt
```

交付物：

1. 说明 **Small** 如何返回。
2. 说明 **Big** 如何返回。
3. 找出 **make_big** 中隐藏 **sret** 指针的位置。
4. 解释为什么源代码参数 **x** 在汇编中可能不是第一个参数寄存器。
5. 画出调用者为 **Big** 分配返回对象并传入地址的过程。

实验：变参函数和 **al**

创建 `labs/ch13_system_v_abi/varargs.cpp`：

```
#include <stdio>

extern "C" void print_int(int x)
{
    std::printf("x=%d\n", x);
}

extern "C" void print_double(double x)
{
    std::printf("x=%f\n", x);
}

extern "C" void print_mix(int x, double y)
{
    std::printf("x=%d y=%f\n", x, y);
}
```

生成：

```
clang++ -std=c++20 -O2 -S -masm=intel varargs.cpp -o varargs.s
clang++ -std=c++20 -O2 -c varargs.cpp -o varargs.o
objdump -d -Mintel varargs.o > varargs.objdump.txt
```

交付物：

1. 找出每次调用 **printf** 前 **eax** 或 **al** 如何设置。
2. 解释 **print_int** 为什么通常设置为 0。
3. 解释 **print_double** 和 **print_mix** 为什么通常设置为 1。
4. 写一个手写汇编函数调用 **printf** 打印 **double**，先故意错误设置 **al**，再修复。
5. 记录错误版本和修复版本在你的系统上的表现。

实验：隐藏参数和尾调用

目标：观察引用、**this** 指针、窄整数扩展和尾调用优化如何改变函数入口解释。

创建 `labs/ch13_system_v_abi/hidden_and_tail.cpp`：

```
#include <cstdint>

extern "C" int add_schar(signed char a, signed char b)
{
    return int(a) + int(b);
}

extern "C" int add_uchar(unsigned char a, unsigned char b)
{
    return int(a) + int(b);
}
```

```

}

extern "C" void inc_ref(int& x)
{
    ++x;
}

struct Counter {
    int value;

    int add(int x)
    {
        value += x;
        return value;
    }
};

extern "C" int call_member(Counter* counter, int x)
{
    return counter->add(x);
}

extern "C" long target_tail(long x);

extern "C" long wrapper_tail(long x)
{
    return target_tail(x);
}

```

构建：

```

clang++ -std=c++20 -O3 -S -masm=intel \
    hidden_and_tail.cpp -o hidden_and_tail.s
clang++ -std=c++20 -O3 -c hidden_and_tail.cpp -o hidden_and_tail.o
objdump -drwC -Mintel hidden_and_tail.o > hidden_and_tail.objdump.txt

```

任务：

1. 比较 `add_schar` 和 `add_uchar` 使用的是符号扩展还是零扩展。
2. 说明 `inc_ref` 的引用参数在入口寄存器中表现为什么。
3. 说明 `Counter::add` 或 `call_member` 中 `this` 指针在哪里。
4. 判断 `wrapper_tail` 是否被编译成尾调用形式。
5. 若出现 `jmp`，解释它为什么仍然必须满足 ABI 边界。

交付物：

1. 相关反汇编片段。
2. 每个函数的入口状态表。
3. 对窄整数高位、引用地址、`this` 指针和尾调用的解释。

本章作业

习题与作业

作业 1：ABI 参数表

对下面函数签名，给出 System V AMD64 下每个参数的常见传递位置：

```

extern "C" long f1(long a, int b, void* p, long d,
                  long e, long f, long g);

extern "C" double f2(int a, double b, int c,
                    float d, double e);

struct P { long a; long b; };
extern "C" long f3(P p, long x);

```

要求：

1. 区分通用寄存器和 `xmm` 寄存器。
2. 标出哪些参数会走栈。
3. 对结构体参数说明你的分类依据。
4. 说明返回值位置。

作业 2：栈布局推导

给定函数入口汇编：

```
push rbp
mov  rbp, rsp
push rbx
sub  rsp, 24
```

要求画出执行完这些指令后的栈布局，至少包含：

- 返回地址。
- saved `rbp`。
- saved `rbx`。
- 24 字节局部区域。
- `rbp` 和 `rsp` 分别指向哪里。

作业 3：修复错误汇编

下面函数被 C++ 调用：

```
.intel_syntax noprefix
.globl broken
broken:
    mov rbx, rdi
    call helper
    add rax, rbx
    ret
```

要求：

1. 找出所有 ABI 风险。
2. 修复 callee-saved 寄存器问题。
3. 修复调用前栈对齐问题。
4. 给出修复后的完整汇编。
5. 解释每条新增指令为什么必要。

作业 4：sret 逆向

给定汇编：

```
make_obj:
    mov rax, rdi
    mov qword ptr [rdi], rsi
    mov qword ptr [rdi + 8], rdx
    mov qword ptr [rdi + 16], rcx
    ret
```

要求：

1. 判断是否存在隐藏返回对象指针。
2. 推测源代码返回类型大小。
3. 推测显式参数可能有哪些。
4. 解释为什么返回值还会写 `rax`。

作业 5：跨语言 ABI 小项目

写一个小项目，包含：

- 一个 C++ 文件声明并调用 3 个手写汇编函数。
- 汇编函数 1: 整数参数求和, 至少 8 个参数。
- 汇编函数 2: 调用一个 C++ helper, 并正确保存 callee-saved 寄存器。
- 汇编函数 3: 返回一个两字段结构体。
- CMake 或 Makefile 构建脚本。
- 单元测试或简单断言。
- `objdump` 报告, 解释每个函数的 ABI 行为。

作业 6: 公开 ABI 设计审查

设计一个小型 C ABI, 用于从动态库中导出一个查询引擎。要求至少包含:

- 创建和销毁上下文的函数。
- 一个查询函数。
- 一个可扩展配置结构体。
- 错误码或错误消息获取方式。
- 明确的内存所有权规则。

然后写审查报告, 回答:

1. 哪些类型暴露在 ABI 边界上。
2. 结构体如何通过 `size` 或 `version` 支持扩展。
3. 为什么不直接暴露 C++ 类或 STL 容器。
4. 如果未来增加字段, 旧调用者会怎样。
5. 哪些改动会破坏 ABI。

作业 7: 尾调用 ABI 分析

写三个 wrapper 函数:

1. 直接返回另一个函数的结果。
2. 调用另一个函数后还要加 1。
3. 调用另一个函数前后使用一个需要保存的局部值。

用 `-O2` 或 `-O3` 生成反汇编, 判断哪些能变成尾调用, 哪些不能。报告必须说明: 参数是否需要重排, 栈是否恢复, callee-saved 寄存器是否需要恢复, 以及为什么某个 `jmp` 是合法尾调用。

评分标准

本章作业按下面标准评价:

1. 参数位置是否和 ABI 一致。
2. 是否能区分寄存器参数和栈参数。
3. 是否能正确处理 caller-saved 与 callee-saved。
4. 手写汇编是否保持栈对齐。
5. 是否能识别隐藏 `sret` 指针。
6. 是否能识别引用、`this` 和窄整数扩展造成的入口状态变化。
7. 是否能解释尾调用优化对栈帧和 ABI 边界的影响。
8. 是否理解公开 ABI 的布局、版本和内存所有权风险。
9. 是否能用 GDB、`objdump` 和实际运行结果证明结论。
10. 是否明确区分 System V AMD64 和其他 ABI。

本章小结

ABI 是从“单个函数内部汇编”走向“真实二进制程序”的关键桥梁。本章你应该掌握:

```
integer/pointer args:
```

```
rdi, rsi, rdx, rcx, r8, r9, then stack

floating args:
  xmm0 - xmm7

return:
  rax / rdx for many integer-like results
  xmm0 for scalar floating results
  hidden sret pointer for large aggregate results

register ownership:
  caller-saved: caller protects live values
  callee-saved: callee restores what it modifies

stack:
  on entry, rsp points to return address
  before call, rsp must be 16-byte aligned
  leaf functions may use red zone in allowed environments
```

从高性能计算和 AI 算子开发角度，ABI 决定了 kernel 函数边界的成本和正确性：参数如何进入寄存器，哪些寄存器可以自由使用，调用 helper 前要保存什么，返回 tensor metadata 或小结构体会不会产生隐藏内存写。下一章会把这些规则用于 C++ 和汇编互操作，进一步进入可测试、可链接、可维护的手写汇编工程。

C++ 与汇编互操作

问题与目标

上一章讲清楚了 System V AMD64 ABI：参数放在哪里、返回值放在哪里、哪些寄存器要保存、栈如何对齐。这些规则看似只是表格，但真正价值会在工程边界上体现出来：一个 C++ 文件如何调用手写汇编？手写汇编如何再调用 C++ helper？链接器如何把两个目标文件连接到一起？内联汇编为什么经常比表面上更危险？为什么同样一条 `call`，在目标文件里还没有最终地址，链接之后才变成真实跳转？

本章把“能读汇编”推进到“能设计可维护、可测试、可审计的汇编边界”。你需要掌握的不只是语法，而是一套从声明到二进制证据的工程流程：

```
source declaration:
  C++ header says which symbol exists and what ABI contract it has

assembly implementation:
  .S file exports that symbol and obeys ABI

object file:
  assembler emits symbols, sections, relocations, debug/unwind hints

linking:
  linker resolves references and patches call/data addresses

runtime:
  CPU only executes bytes; correctness depends on the binary contract
```

从高性能计算和 AI 算子开发角度看，本章尤其关键。真实算子库经常出现这样的边界：

- C++ 调度层选择某个 kernel，再调用汇编或 intrinsic 实现。
- 汇编 kernel 接收指针、维度、步长、常量、尾部长度等参数。
- kernel 内部尽量不调用其他函数；一旦调用，就必须恢复 ABI 约束。
- profiler、debugger、sanitizer、异常展开和符号工具都要能看懂这段代码。

因此，本章的核心不是“写几条汇编指令”，而是建立下面这条可验证链路：

```
C++ type/signature
-> ABI-level parameter and return-value locations
-> assembly symbol and prologue/epilogue
-> object-file symbols and relocations
-> linked executable
-> debugger/profiler observable behavior
-> tests and performance evidence
```

核心思想

C++ 与汇编互操作的本质是二进制契约工程。C++ 编译器不会理解手写汇编的意图；汇编器也不会检查 C++ 类型语义。两边能正确合作，依赖的是符号名、ABI、目标文件格式、链接器、调试元数据和测试共同维持的边界。

互操作代码的设计原则

手写汇编边界最怕“聪明但含糊”。一段汇编函数可以很短，但它背后的接口必须保守、清晰、可测试。设计互操作接口时，应优先追求可证明正确，而不是一开始就追求最极限的指令布局。高质量的汇编工程，通常把复杂性放在 C++ 调度层，把汇编热路径压缩到少量明确前置条件下运行。

第一条原则是接口窄。汇编函数最好只接收必要的标量、指针、长度、步长和常量，不要直接暴露复杂 C++ 对象。复杂对象可以在 C++ 层拆解成普通字段，再传给汇编。这样做的好处是 ABI

形状稳定，测试容易，符号边界清楚，也减少手写汇编理解 C++ 对象生命周期的风险。

第二条原则是前置条件明确。汇编函数内部通常不适合做大量错误处理。比如一个点积 kernel 可以要求 **a** 和 **b** 指向至少 **n** 个元素，**n** >= 0，外层已经完成空指针和尺寸检查。若这些条件不成立，行为可以定义为调用方错误。这样热循环不需要每轮检查边界，也能让测试集中验证 wrapper 是否正确过滤非法输入。

第三条原则是边界稳定。一个已经被 C++ 调用的汇编函数，参数顺序、类型大小、返回方式和寄存器保存约定都不能随便改。你可以在内部重写指令，也可以增加新的符号提供新版实现，但不要让旧声明继续指向 ABI 形状已经变化的实现。对库来说，符号名就是承诺。

第四条原则是证据优先。互操作代码是否正确，不能依赖“寄存器看起来放对了”这种印象。每个函数都应有 C++ reference、确定性测试、随机测试、反汇编审计、必要的 GDB 记录和接口注释。进入性能优化之前，先证明它在二进制边界上没有违反合同。

心智模型

可以把汇编函数看成一个很小的硬件风格模块：

- 输入端口：ABI 参数寄存器和栈参数。
- 输出端口：返回寄存器或返回对象内存。
- 时序要求：调用前栈对齐，返回前恢复 **rsp**。
- 保持约束：callee-saved 寄存器必须还原。
- 环境假设：指针、长度、对齐和别名由外层保证。

接口文档就是这个模块的规格书。

先建立三层边界

许多手写汇编问题并不是由 **add**、**mov** 这类指令本身造成的，而是来自三层规则的混淆。

第一层：C++ 语言层

C++ 源代码里看到的是函数声明、类型、重载、命名空间、引用、对象生命周期、异常、访问控制和模板实例化。例如：

```
extern "C" long asm_sum8(long a, long b, long c, long d,
                        long e, long f, long g, long h);
```

这个声明告诉 C++ 编译器：

- 存在一个可调用函数。
- 函数名在链接层叫 **asm_sum8**，不要使用 C++ name mangling。
- 参数和返回值都是 **long**。
- 调用时按当前平台默认调用约定生成代码。

但这个声明没有告诉编译器函数体在哪里，也没有证明汇编实现真的返回 **long**。如果你的汇编函数把结果放进了 **rcx** 而不是 **rax**，C++ 编译器不会替你发现。

第二层：ABI 层

上一章的 ABI 表格在这里变成实际契约。对于上面的 **asm_sum8**，System V AMD64 下入口状态应当是：

```
on entry to asm_sum8:
    rdi    = a
    rsi    = b
    rdx    = c
    rcx    = d
    r8     = e
    r9     = f
    [rsp + 0] = return address
    [rsp + 8] = g
    [rsp + 16] = h
```

on return:

```
rax = result
```

如果函数修改了 `rbx`、`rbp`、`r12` 到 `r15`，它必须在返回前恢复。若它调用其他函数，调用点前还要满足栈对齐约定。

第三层：目标文件和链接层

汇编文件最终不是直接和 C++ 源码连接，而是先变成目标文件。目标文件里包含：

- section，例如 `.text`、`.rodata`、`.data`。
- symbol，例如 `asm_sum8`、`cpp_helper`。
- relocation，例如“这里有一个 `call`，目标符号稍后由链接器填”。
- debug/unwind metadata，例如调试器和异常展开需要的信息。

可以用下面工具观察：

```
nm -C file.o
readelf -Ws file.o
readelf -r file.o
objdump -drwC -Mintel file.o
```

心智模型

源代码层问“这个函数叫什么、类型是什么”；ABI 层问“参数和返回值在哪些寄存器或栈槽”；目标文件层问“哪个符号定义在哪里，哪个地址需要链接器以后修补”。这三层都正确，跨语言调用才正确。

接口类型应该怎样选

互操作接口的类型选择，比汇编指令本身更早决定工程质量。C++ 允许非常丰富的类型：类、模板、引用、异常、虚函数、容器、迭代器、`lambda`、智能指针。它们在源码层很有表达力，但在二进制边界上可能引入复杂 ABI 细节。手写汇编如果直接接触这些对象，就必须理解它们的布局、生命周期、构造析构、异常清理和版本兼容，这通常不是一个热路径 kernel 应承担的责任。

基础阶段建议优先使用下面几类接口类型：

- 固定宽度整数，例如 `std::int32_t`、`std::uint64_t`。
- 指向连续数组的裸指针，例如 `float*`、`const std::int32_t*`。
- 显式长度和步长，例如 `std::int64_tn`、`std::int64_tstride`。
- 平凡小结构体，且字段布局在报告中写清楚。
- 不透明指针，例如 `Context*`，由 C++ 层解释内部结构。

不建议直接把下面类型暴露给手写汇编热路径：

- `std::vector`、`std::string`、`std::span` 等库类型对象本身。
- 带虚函数、继承、非平凡析构的类对象。
- 抛出异常或需要异常展开穿过汇编帧的接口。
- 需要汇编负责构造或析构的 C++ 对象。
- 未明确布局和对齐的跨库结构体。

这不是说汇编永远不能和复杂 C++ 系统合作。正确做法通常是：C++ wrapper 接收复杂对象，完成检查、拆解、调度和生命周期管理；汇编 kernel 只接收已经规整好的地址、大小和标量参数。这样复杂语义留在编译器能理解的 C++ 层，汇编层专注执行密集计算。

显式长度比隐式约定更好

数组类接口应显式传入长度。不要让汇编函数通过哨兵值、全局状态或对象内部字段猜测长度，除非这是算法本身的必要部分。显式长度有几个优点：ABI 映射简单，测试能覆盖 `n=0`、`n=1`、边界尾部和大输入，性能报告能按元素数归一化，越界风险也更容易在 C++ wrapper 中检查。

如果存在步长，也应显式传入。矩阵、`tensor`、图像和切片常常不是简单连续数组。一个汇编 kernel 如果假设连续布局，却被调用方传入有 `stride` 的数据，结果会静默错误。相反，把 `stride` 作

为显式参数，会让地址公式在接口中可见，也让反汇编审计更直接。

对齐和别名要写进契约

性能 kernel 经常关心对齐。某些实现可以处理任意地址，只是未对齐时慢；某些 SIMD 指令或手写加载策略要求特定对齐；某些算法要求输入输出不重叠，才能安全重排 load/store。C++ 编译器可以通过类型、**restrict** 风格扩展、assume、alignment attribute 等信息进行优化；手写汇编没有这些自动语义，必须由接口文档和 wrapper 维护。

例如一个向量加法 kernel 的前置条件可以写成：

```
Preconditions:
  x, y, out point to at least n float elements
  n >= 0
  out may alias x, but must not partially overlap y
  implementation accepts unaligned pointers
```

或者：

```
Preconditions:
  x, y, out are 32-byte aligned
  n is a multiple of 8
  x, y, out do not overlap
```

这两种接口都可以合理，但它们是不同契约。前者更通用，汇编内部可能要处理尾部和未对齐；后者更快，但调用方必须提供更强保证。教材训练时要把这种取舍写出来，不能只在脑中默认。

核心思想

汇编接口类型越简单，二进制契约越容易证明。复杂 C++ 对象应由 C++ wrapper 处理，汇编 kernel 接收规整后的标量、指针、长度和步长。

推荐分层：C++ wrapper 和汇编 kernel

真实项目里，不建议让业务代码直接调用裸汇编符号。更稳妥的结构是两层：

```
public C++ API
  |
  v
C++ wrapper: validate, normalize, dispatch, document
  |
  v
extern "C" assembly kernel: small ABI-stable hot path
```

公共 C++ API 面向使用者，可以使用类型更丰富的接口。例如接收 `std::span<constfloat>`、矩阵描述对象、枚举参数、错误码或上下文对象。它负责把输入转换为汇编 kernel 能理解的简单形态：裸指针、长度、步长、标量常量和少数标志位。这样调用方不需要知道 ABI 细节，汇编也不需要理解复杂 C++ 类型。

C++ wrapper 至少承担五类责任。

第一，检查前置条件。比如指针是否为空，长度是否非负，输入输出是否允许重叠，是否满足对齐要求，尾部是否需要 fallback。汇编热路径可以假定这些条件已经成立，从而避免在内层循环里反复检查。

第二，规格化参数。比如把 `std::size_t` 转换为汇编约定中的 `std::int64_t`，把二维矩阵的行列和 stride 展开成显式整数，把布尔模式转换成小整数标志。转换时要处理溢出和范围，不能让一个超出汇编可表达范围的长度悄悄截断。

第三，选择实现。一个 wrapper 可以根据 CPU feature、数据规模、对齐、尾部长度选择不同 kernel。例如 `scalar`、`avx2`、`avx512` 可以是不同内部符号。第一册不展开 ISA dispatch，但要建立分层意识：dispatch 属于 C++ 控制层，单个 kernel 应保持简单。

第四，处理错误和异常。公共 API 可以返回错误码、抛异常或断言失败；汇编 kernel 通常不应该抛异常，也不应该直接构造复杂错误对象。这样异常传播、资源清理和日志记录仍由 C++ 编译器能理解的代码承担。

第五，提供测试锚点。测试可以分别覆盖 wrapper 的非法输入行为和 kernel 的合法输入行为。若直接测试裸汇编，非法输入可能导致崩溃；若只测试公共 API，又可能难以定位是 wrapper 选择错了 kernel，还是 kernel 本身计算错了。分层后两类问题可以分开验证。

一个简单 wrapper 形态

例如裸汇编 kernel 的契约是：

```
extern "C" std::int64_t asm_dot_i32_kernel(const std::int32_t* a,
                                         const std::int32_t* b,
                                         std::int64_t n);
```

公共 wrapper 可以写成：

```
#include <cstdint>
#include <span>
#include <stdexcept>

extern "C" std::int64_t asm_dot_i32_kernel(const std::int32_t* a,
                                         const std::int32_t* b,
                                         std::int64_t n);

std::int64_t dot_i32(std::span<const std::int32_t> a,
                   std::span<const std::int32_t> b)
{
    if (a.size() != b.size()) {
        throw std::invalid_argument("dot_i32: size mismatch");
    }

    if (a.size() > static_cast<std::size_t>(INT64_MAX)) {
        throw std::length_error("dot_i32: input too large");
    }

    return asm_dot_i32_kernel(a.data(),
                             b.data(),
                             static_cast<std::int64_t>(a.size()));
}
```

这个 wrapper 做了三件关键事：检查尺寸相等，检查长度能放进汇编接口的 `std::int64_t`，把 `std::span` 拆成指针和长度。汇编 kernel 不需要知道 `std::span` 的布局，也不需要知道异常类型。它只履行一个小契约：对两个长度为 `n` 的数组计算点积。

wrapper 不能替代 kernel 测试

有了 wrapper，并不表示 kernel 可以不测。wrapper 测试回答公共 API 是否按规格处理输入；kernel 测试回答 ABI 边界和汇编计算是否正确。两者的测试输入也不同。

wrapper 测试应包含：

- 尺寸不匹配时是否报错。
- 空 span 是否按规格处理。
- 超大长度是否被拒绝。
- 合法输入是否调用正确实现。

kernel 测试应包含：

- `n=0`、`n=1`、小规模和较大规模。
- 正数、负数、零、随机值。
- 多次调用和不同调用点。
- 反汇编审计和符号审计。

如果项目有多个 ISA kernel，还要保证每个 kernel 都能被单独测试，而不只是通过 dispatch 间接覆盖。否则某个 CPU feature 不可用时，测试可能永远没有跑到某个实现。

核心思想

C++ wrapper 负责语义、检查、调度和错误处理；汇编 kernel 负责小而稳定的机器级计算。把这两层混在一起，会同时破坏 C++ 可维护性和汇编可证明性。

符号可见性、ODR 和 ABI 稳定性

互操作项目一旦从单个实验文件进入库，就会遇到另一个问题：符号不仅要“能链接”，还要“按预期被链接”。C++ 有 ODR，也就是一个定义规则；链接器有强符号、弱符号、局部符号、全局符号、默认可见性、隐藏可见性；动态库还有导出符号集合。手写汇编如果不控制这些边界，很容易出现重复定义、意外覆盖、符号泄漏或链接到错误版本。

全局符号不是越多越好

.globl 会把符号导出给链接器，让其他目标文件可以引用。对于公共 ABI 函数，这是必要的；对于只在当前汇编文件内部使用的局部标签或 helper，则不应该导出。内部标签可以用局部名字，例如 .lloop、.ldone。这些标签通常不会出现在最终动态符号表里，也不会成为外部链接契约。

导出太多符号有几个问题。第一，命名冲突风险增加。一个通用名字如 loop、helper、kernel 很可能和其他目标文件撞名。第二，动态库 ABI 面变大，未来修改更困难。第三，profiler 和符号工具输出变噪，真正公共入口不容易识别。第四，安全审计更复杂，因为外部可见入口越多，外部代码越可能直接绕过 wrapper 调用内部实现。

工程中常见做法是：公共 C++ 头文件只声明少量稳定入口，汇编内部实现使用带前缀的私有符号，动态库构建时默认隐藏符号，只显式导出 API。对第一册实验来说，你至少应养成前缀习惯，例如所有实验符号使用 asm_ 或项目名前缀，而不是裸 add、sum。

ODR 和重复定义

如果两个目标文件都定义同一个强符号，链接器通常会报 multiple definition。若一个是弱符号，链接器可能选择强符号。这个机制在运行时库和默认实现里有用途，但对初学汇编项目来说，弱符号会增加不确定性。你可能以为调用的是自己的汇编实现，实际链接到了另一个同名实现。

模板、inline 函数和头文件实现也会影响 ODR。C++ 编译器可以为模板实例生成多个 COM-DAT 弱定义，让链接器合并；手写汇编则通常没有这种高级语义。不要把同一个 .S 文件通过多个路径重复加入多个库，再把这些库同时链接进最终程序。若构建系统设计不清，重复对象可能导致链接错误或符号选择意外。

报告里应使用 nm 或 readelf -Ws 验证最终二进制中公共符号只有预期的一份定义。对于动态库，还应区分普通符号表和动态符号表。能在普通符号表看到，不代表它被动态导出；能被动态导出，则意味着外部加载者可能依赖它。

ABI 稳定性：符号名相同不代表契约相同

最危险的变更是符号名不变，但 ABI 形状变了。比如原来声明是：

```
extern "C" long asm_dot_i32(const int* a, const int* b, long n);
```

后来你把汇编实现改成额外需要 stride，却没有改符号名和 C++ 声明，只是在汇编里从 rcx 读取 stride。调用者并不会自动传入这个参数。结果 rcx 里是调用点上的临时值，程序可能在某些输入下“碰巧能跑”，但 ABI 已经坏了。

正确做法是：如果公共 ABI 参数、返回方式、前置条件或可见副作用发生不兼容变化，就定义新符号，例如 asm_dot_i32_v2，或者在 C++ wrapper 层保持旧入口兼容，把新入口隐藏在内部。库版本管理可以更复杂，但第一原则很朴素：不要让旧声明调用新契约。

常见误区

符号名是链接器看到的名字，不是完整接口。完整接口还包括参数类型、返回方式、寄存器保存、栈对齐、前置条件、全局副作用和异常规则。符号名不变而契约改变，是互操作代码里最隐蔽的破坏。

类型宽度和平台差异不能靠猜

互操作接口里最容易被忽略的细节之一，是 C++ 类型在目标平台上的宽度。比如 `int` 通常是 32 位，`longlong` 通常是 64 位；但 `long` 在不同 ABI 上可能不同。在 Linux x86-64 的 System V ABI 下，`long` 是 64 位；在 Windows x64 下，`long` 仍然是 32 位。若你把 `long` 写进公共汇编接口，就已经让接口带上平台假设。

因此，本书在互操作边界优先使用固定宽度整数：

```
extern "C" std::int64_t asm_add_i64(std::int64_t a, std::int64_t b);
extern "C" std::uint32_t asm_mix_u32(std::uint32_t x, std::uint32_t y);
```

这不会消除所有 ABI 差异，但至少让值宽度明确。汇编实现也应对应宽度选择寄存器视图。32 位返回值放在 `eax`，64 位返回值放在 `rax`。若函数声明返回 `std::uint32_t`，汇编写 `eax` 通常符合预期；若函数声明返回 `std::uint64_t`，只写 `ax` 或 `al` 就很可能留下旧高位，造成错误。

在头文件中静态验证假设

对跨语言结构体或宽度敏感接口，应在 C++ 头文件里写静态断言：

```
#include <cstddef>
#include <cstdint>
#include <type_traits>

struct KernelArgs {
    const std::int32_t* a;
    const std::int32_t* b;
    std::int64_t n;
    std::int64_t stride;
};

static_assert(sizeof(std::int32_t) == 4);
static_assert(sizeof(std::int64_t) == 8);
static_assert(sizeof(KernelArgs) == 32);
static_assert(offsetof(KernelArgs, a) == 0);
static_assert(offsetof(KernelArgs, b) == 8);
static_assert(offsetof(KernelArgs, n) == 16);
static_assert(offsetof(KernelArgs, stride) == 24);
static_assert(std::is_trivially_copyable_v<KernelArgs>);
```

这些断言不是形式主义。若某个字段类型被改了、结构体插入新字段、对齐规则变化、头文件被不同平台编译，断言会尽早失败。没有断言时，汇编可能继续按旧偏移读取字段，程序却只在特定输入下错误。

汇编侧也可以用注释把偏移写清：

```
# KernelArgs layout:
# +0 const int32_t* a
# +8 const int32_t* b
# +16 int64_t n
# +24 int64_t stride
```

注释不能替代 `static_assert`，但它让反汇编和源码审计更容易。高质量互操作代码通常两者都要有：C++ 编译期断言负责阻止布局悄悄变化，汇编注释负责让维护者知道每个偏移的含义。

不要把一个 ABI 的经验搬到另一个 ABI

本书第一册主要使用 Linux x86-64 System V ABI。它和 Windows x64 有明显差异，例如整数参数寄存器、shadow space、callee-saved XMM 寄存器规则、结构体返回细节都不完全一样。macOS 也有自己的对象文件和符号命名细节。AArch64 又是另一套寄存器和调用约定。

因此，汇编互操作代码应在文件名、构建条件和文档中标出目标平台：

```
kernel/
  x86_64_sysv/
    dot_i32.S
  x86_64_windows/
    dot_i32.asm
  aarch64_linux/
    dot_i32.S
```

构建系统也应按平台选择源文件，而不是让一个 .S 文件用大量条件编译硬撑所有 ABI。小范围宏可以接受，跨 ABI 的寄存器分配、栈规则和 unwind 规则差异太大时，分文件通常更清楚。

常见误区

“我的机器上能跑”只证明当前平台、当前编译器、当前构建方式下没有暴露错误。互操作接口若要跨平台，必须逐个平台写 ABI 表、测试和符号审计。

一个最小可运行工程

先做最小例子。目录结构如下：

```
labs/ch14_interop/minimal/
  CMakeLists.txt
  main.cpp
  asm_add.S
```

C++ 调用者

```
// main.cpp
#include <cassert>
#include <cstdio>
#include <cstdlib>

extern "C" std::int64_t asm_add2(std::int64_t a, std::int64_t b);

int main()
{
    const std::int64_t x = 40;
    const std::int64_t y = 2;
    const std::int64_t got = asm_add2(x, y);

    assert(got == 42);
    std::printf("asm_add2(%ld, %ld) = %ld\n", x, y, got);
}
```

这里的 `extern "C"` 不是说这个函数由 C 写成，而是说这个声明使用 C 链接名。没有它，C++ 可能把符号名改写成带类型信息的 mangled name。

汇编实现

```
// asm_add.S
.intel_syntax noprefix
.text

.globl asm_add2
.type asm_add2, @function
asm_add2:
    lea rax, [rdi + rsi]
    ret
.size asm_add2, .-asm_add2

.section .note.GNU-stack,"",@progbits
```

逐行解释：

代码	含义
<code>.intel_syntax noprefix</code>	使用 Intel 语法，寄存器不写 % 前缀。
<code>.text</code>	后续内容进入代码段。
<code>.globl asm_add2</code>	导出符号，让其他目标文件可以引用。
<code>.type asm_add2,@function</code>	标记符号类型为函数，便于工具识别。
<code>asm_add2:</code>	函数入口标签，也是符号地址。
<code>leax, [rdi+rsi]</code>	计算第 1、2 个参数之和，放入返回寄存器。
<code>ret</code>	返回到调用者。
<code>.size asm_add2,.-asm_add2</code>	记录函数大小，从入口到当前位置。
<code>.note.GNU-stack</code>	告诉链接器不需要可执行栈。

常见误区

`.section.note.GNU-stack,"",@progbits` 不会生成运行时代码，但在 Linux ELF 工程中应当保留。省略它可能导致链接器给出 `executable stack` 警告，或者在某些环境里产生不必要的安全风险。

构建脚本

```
# CMakeLists.txt
cmake_minimum_required(VERSION 3.20)
project(ch14_minimal LANGUAGES CXX ASM)

add_executable(ch14_minimal
  main.cpp
  asm_add.S
)

target_compile_features(ch14_minimal PRIVATE cxx_std_20)
target_compile_options(ch14_minimal PRIVATE
  $<$<COMPILE_LANGUAGE:CXX>:-Wall -Wextra -Wpedantic -O2 -g>
)
```

构建：

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build build -j
./build/ch14_minimal
```

审计：

```
nm -C build/ch14_minimal | rg 'asm_add2|main'
objdump -drwC -Mintel build/ch14_minimal > ch14_minimal.objdump.txt
rg -n '<asm_add2>|call.*asm_add2' ch14_minimal.objdump.txt
```

为什么文件后缀用 .S

GNU/Clang 工具链里常见约定：

后缀	含义
<code>.s</code>	汇编源文件，通常不经过 C 预处理器。
<code>.S</code>	汇编源文件，先经过 C 预处理器，再交给汇编器。

使用 .S 后，可以写：

```
#if defined(__linux__) && defined(__x86_64__)
.section .note.GNU-stack,"",@progbits
#endif
```

也可以在以后用宏生成不同 ISA 版本。但宏会降低可读性。本书训练阶段优先写直白的汇编，只有重复结构明显时才引入宏。

CMake 中启用 ASM 的意义

`project(...LANGUAGESCXXASM)` 不是装饰。它告诉 CMake 当前工程包含汇编源文件，需要用合适的汇编器规则处理 `.S`。如果没有启用 ASM，某些构建系统可能不知道如何编译汇编文件，或者把它当成普通文本忽略。大型项目中，汇编文件还可能需要根据目标平台、编译器、操作系统和 ISA 变体选择不同源文件。

训练阶段可以把汇编文件直接加入 `add_executable`。工程扩大后，更常见的做法是把 kernel 放入一个静态库或对象库，再由测试程序、benchmark 和最终应用链接。这样做有三个好处：测试可以复用同一份目标文件，benchmark 不会意外使用另一份实现，符号审计也更清楚。

PIE、PIC 和地址写法

现代 Linux 发行版常默认生成 PIE 可执行文件，动态库则要求位置无关代码。位置无关代码不能假设某个全局对象或函数永远位于固定绝对地址，因为加载器可能把程序或共享库映射到不同虚拟地址。x86-64 下，访问静态数据常使用 RIP-relative addressing；调用外部符号可能经过 PLT/GOT。

这对手写汇编有直接影响。教学中的纯函数只使用参数寄存器，不涉及全局地址，通常不受影响；一旦汇编函数访问全局表、常量池、外部函数或动态库符号，就要使用适合位置无关代码的写法。写死绝对地址在现代用户态程序里通常不是正确做法。

例如取本文件内只读数据地址，应优先写：

```
leaq rax, [rip + table]
```

而不是把某个链接时地址常量硬编码进寄存器。对于跨目标文件或动态库符号，还要看重定位类型和链接模式。若构建失败并出现 relocation 不能用于 PIE 的错误，通常说明你的汇编地址写法不适合当前位置无关要求。

静态链接、动态链接和第一次调用

同一个 C++ 声明，最终调用路径会因链接方式不同而变化。静态链接或同一可执行文件内部符号，可能生成直接 `call`。动态库外部符号，可能经过 PLT；第一次调用可能触发延迟绑定，后续调用使用已解析地址。开启 `-fno-plt` 或符号可见性优化时，路径又可能变化。

因此，互操作报告中不能只写“调用了 `foo`”。应说明你观察的是目标文件、静态链接后的可执行文件，还是动态链接后的运行路径。目标文件里的 `call` 可能带 relocation；最终可执行文件里的 `call` 才有具体相对位移；运行时经过 PLT/GOT 的路径还受加载器影响。

常见误区

如果你的汇编访问全局数据或外部符号，必须考虑 PIE/PIC 和重定位。只在非 PIE 小实验中能链接，不代表它能放进现代共享库或默认 PIE 可执行文件。

把函数签名当成二进制接口

互操作时，最危险的习惯是“先写汇编，能跑就行”。正确顺序应该是：

1. 写 C++ 声明，明确类型和链接名。
2. 按 ABI 推导参数位置、返回位置、保存责任。
3. 写汇编函数框架。
4. 写测试，覆盖普通值、边界值、随机值。
5. 用 `objdump`、`nm`、`readelf` 审计二进制。
6. 再考虑性能。

例如：

```
extern "C" std::int64_t asm_sum8(std::int64_t a0, std::int64_t a1,
                                std::int64_t a2, std::int64_t a3,
                                std::int64_t a4, std::int64_t a5,
                                std::int64_t a6, std::int64_t a7);
```

ABI 推导表：

源代码参数	入口位置	说明
a0	rdi	第 1 个整数类参数。
a1	rsi	第 2 个整数类参数。
a2	rdx	第 3 个整数类参数。
a3	rcx	第 4 个整数类参数。
a4	r8	第 5 个整数类参数。
a5	r9	第 6 个整数类参数。
a6	[rsp+8]	第 7 个参数，返回地址之后。
a7	[rsp+16]	第 8 个参数。
return	rax	64 位整数返回。

汇编：

```
.intel_syntax noprefix
.text

.globl asm_sum8
.type asm_sum8, @function
asm_sum8:
    lea rax, [rdi + rsi]
    add rax, rdx
    add rax, rcx
    add rax, r8
    add rax, r9
    add rax, qword ptr [rsp + 8]
    add rax, qword ptr [rsp + 16]
    ret
.size asm_sum8, .-asm_sum8

.section .note.GNU-stack,"",@progbits
```

注意这里没有函数序言。它是一个 leaf function，不调用其他函数，不需要局部栈空间，不修改 callee-saved 寄存器。这样的函数可以非常短。但这个短小建立在 ABI 推导正确之上，不是随便省略。

深入理解

如果函数入口后执行了 `push rbp`，那么第 7 个参数就不再位于当前 `rsp` 的 `[rsp+8]`。入口原始布局被改变了：

```
on entry:
    [rsp + 0]  return address
    [rsp + 8]  a6
    [rsp +16]  a7

after push rbp:
    [rsp + 0]  saved rbp
    [rsp + 8]  return address
    [rsp +16]  a6
    [rsp +24]  a7
```

所以读写栈参数前必须先确定当前 `rsp` 相对入口移动了多少。很多手写汇编 bug 就来自这里。

测试不是附属品

手写汇编很容易出现“几个样例能跑，但边界条件错”的情况。测试要直接围绕二进制契约设计。

确定性测试

```
#include <array>
#include <cassert>
#include <cstdlib>
#include <limits>
```



```
values[4], values[5],
values[6], values[7]);
assert(got == ref_sum8(values));
}
}
```

常见误区

如果 C++ reference 函数使用 signed overflow，而测试数据可能溢出，那么 reference 本身可能有未定义行为。训练早期可以限制输入范围；后续如果要测试完整 64 位 wraparound，应使用 `std::uint64_t` 或明确使用编译器支持的溢出内建函数。

测试矩阵

互操作测试不能只在一个优化级别、一个输入、一次运行下通过。至少应建立一个小型测试矩阵：

维度	建议覆盖
优化级别	-O0、-O2 或 -O3。
调试信息	带 -g，必要时保留帧指针。
输入规模	0、1、小值、边界值、较大值。
数值分布	零、正数、负数、随机值、极端值。
调用次数	单次调用、多次循环调用、嵌套调用。
寄存器压力	调用点周围保留多个活跃变量，逼出保存问题。
链接方式	至少观察目标文件和最终可执行文件；有条件时观察动态库。

不同维度暴露不同问题。-O0 可能更容易单步，但不能代表优化后调用点；-O3 更容易暴露 inline asm 约束错误和 caller-saved 保存错误；随机值能发现算术边界；多次调用能发现 callee-saved 寄存器污染；动态链接能暴露符号可见性和 PLT/GOT 路径。

测试还要避免 reference 本身不可靠。若汇编语义是 unsigned wraparound，reference 也应该用 unsigned 类型；若汇编要求指针非空，测试非法指针时应测试 wrapper 拒绝输入，而不是直接调用 kernel；若汇编要求对齐，测试应分别覆盖符合对齐和不符合对齐的 wrapper 行为。

错误注入测试

为了确认测试真的能抓到 ABI 错误，可以故意写坏版本。比如删掉 rbx 的保存恢复、把第 7 个参数偏移写成 [rsp]、调用 helper 前不对齐栈、inline asm 漏掉 "cc" 或 "memory"。如果测试完全没有失败，说明测试没有覆盖关键契约。

错误注入不是为了保留坏代码，而是为了验证测试敏感性。高质量报告可以写：“我们曾故意移除 pop rbx 前的恢复，随机测试在 -O3 下失败；恢复后通过。”这比只说“测试通过”更能证明你理解了风险。

核心思想

互操作测试的目标不是证明某个输入能跑，而是让常见 ABI 破坏、参数错位、栈错位和优化器合同错误有机会暴露。

C++ name mangling 与 extern"C"

C++ 支持重载、命名空间、类成员函数、模板实例化。为了让链接器区分这些函数，编译器会把源代码函数名编码成更复杂的链接符号，这叫 **name mangling**（名称修饰）。

```
int add(int a, int b)
{
    return a + b;
}

double add(double a, double b)
{
    return a + b;
}
```

编译并查看符号：

```
clang++ -std=c++20 -O2 -c overload.cpp -o overload.o
nm overload.o
nm -C overload.o
```

可能看到类似：

```
0000000000000000 T _Z3addii
0000000000000010 T _Z3addddd
```

nm-C 会 demangle：

```
0000000000000000 T add(int, int)
0000000000000010 T add(double, double)
```

如果写：

```
extern "C" int asm_add_int(int a, int b);
```

链接符号就是：

```
asm_add_int
```

这比手写汇编更简单。汇编文件只需要导出同名符号：

```
.globl asm_add_int
.type asm_add_int, @function
asm_add_int:
    lea eax, [edi + esi]
    ret
.size asm_add_int, .-asm_add_int
```

深入理解

extern"C" 只改变语言链接规则和符号名，不会把 C++ 类型系统变成 C，也不会禁用 System V ABI。比如 **extern"C"double f(double)** 仍然用 **xmm0** 传参与返回。它也不能用于重载同名函数，因为 C 链接名无法表达重载集合。

目标文件里的符号和重定位

很多人第一次看目标文件反汇编，会困惑为什么 **call** 后面是 0。

看 C++：

```
extern "C" long asm_add2(long, long);

extern "C" long call_add2()
{
    return asm_add2(10, 32);
}
```

编译目标文件：

```
clang++ -std=c++20 -O2 -c caller.cpp -o caller.o
objdump -drwC -Mintel caller.o
readelf -r caller.o
```

你可能看到：

```
0000000000000000 <call_add2>:
0: bf 0a 00 00 00      mov     edi,0xa
5: be 20 00 00 00      mov     esi,0x20
a: e8 00 00 00 00      call    f <call_add2+0xf>
    b: R_X86_64_PLT32    asm_add2-0x4
f: c3                  ret
```

这里的 `e800000000` 不是最终机器码。x86 的 `near call` 编码通常是：

```
e8 rel32
```

`rel32` 是相对下一条指令地址的有符号 32 位位移。目标文件阶段还不知道 `asm_add2` 最终地址，所以汇编器放一个临时值，并生成 relocation 记录：

```
at offset 0xb:
  patch 4 bytes
  relocation type = R_X86_64_PLT32
  symbol = asm_add2
  addend = -4
```

链接后，假设：

```
call instruction address = 0x40112a
next rip after call      = 0x40112f
asm_add2 address         = 0x401150
```

那么：

```
rel32 = target - next_rip
      = 0x401150 - 0x40112f
      = 0x21
```

最终编码会接近：

```
e8 21 00 00 00
```

核心思想

目标文件里的反汇编不是最终地址事实，而是“机器码字节 + 符号 + 重定位请求”的组合。读未链接目标文件时，必须同时看 `objdump-dr` 和 `readelf-r`。

汇编函数调用 C++ 函数

现在反过来：手写汇编调用 C++ helper。

C++ helper

```
#include <stdint>

extern "C" std::int64_t cpp_square(std::int64_t x)
{
    return x * x;
}

extern "C" std::int64_t asm_square_plus(std::int64_t x);
```

目标：`asm_square_plus(x)` 调用 `cpp_square(x)`，再返回 `x*x+x`。
错误版本很容易写成：

```
.intel_syntax noprefix
.text

.globl asm_square_plus_broken
.type asm_square_plus_broken, @function
asm_square_plus_broken:
    mov rbx, rdi
    call cpp_square
    add rax, rbx
    ret
.size asm_square_plus_broken, .-asm_square_plus_broken
```

这段有两个风险：

- 修改了 callee-saved **rbx**，但没有恢复。
- 进入函数时 **rsp** 通常是 $16n+8$ ；直接 **call** 时调用点前栈没有 16 字节对齐。

修复版本

```
.intel_syntax noprefix
.text

.globl asm_square_plus
.type asm_square_plus, @function
asm_square_plus:
    push rbx
    mov  rbx, rdi

    call cpp_square

    add  rax, rbx
    pop  rbx
    ret
.size asm_square_plus, .-asm_square_plus

.section .note.GNU-stack,"",@progbits
```

为什么一个 **push rbx** 同时解决了两个问题？
入口时：

```
rsp = 16n + 8
```

执行 **push rbx** 后：

```
rsp = 16n
```

这时执行 **call cpp_square**，调用点前 **rsp** 满足 16 字节对齐。同时 **rbx** 原值被保存到栈上，返回前 **pop rbx** 恢复。

栈变化：

```
on entry to asm_square_plus:
    [rsp + 0] return address to C++ caller

after push rbx:
    [rsp + 0] saved rbx
    [rsp + 8] return address to C++ caller

inside cpp_square after call:
    [rsp + 0] return address to asm_square_plus
    [rsp + 8] saved rbx
    [rsp +16] return address to C++ caller
```

常见误区

不要把“push 一个寄存器刚好对齐”当成万能模板。如果你还要分配局部栈空间，必须重新计算总共改变了多少 **rsp**。原则是：每个 **call** 执行前，调用者的 **rsp** 应为 16 字节对齐。

leaf 和 non-leaf 汇编模板

手写汇编可以先从两个模板开始。

leaf function 模板

leaf function 不调用其他函数。它可以非常简洁：

```
.intel_syntax noprefix
.text

.globl function_name
.type function_name, @function
```

```
function_name:
    # use caller-saved registers freely:
    # rax, rcx, rdx, rsi, rdi, r8-r11, xmm0-xmm15
    # preserve callee-saved registers if you touch them

    ret
.size function_name, .-function_name

.section .note.GNU-stack,"",@progbits
```

对于短小整数函数，常见策略是只用 caller-saved 寄存器，避免保存恢复开销。

non-leaf function 模板

non-leaf function 会调用其他函数，必须更谨慎：

```
.intel_syntax noprefix
.text

.globl function_name
.type function_name, @function
function_name:
    push rbp
    mov rbp, rsp
    push rbx
    sub rsp, 8

    # now rsp is 16-byte aligned before calls
    # save live caller-saved values yourself before each call if needed

    call some_helper

    add rsp, 8
    pop rbx
    pop rbp
    ret
.size function_name, .-function_name

.section .note.GNU-stack,"",@progbits
```

为什么有 `subrsp, 8`？计算一下：

```
entry:
    rsp = 16n + 8

push rbp:
    rsp = 16n

mov rbp, rsp:
    no change

push rbx:
    rsp = 16n - 8

sub rsp, 8:
    rsp = 16n - 16
```

所以在 `call some_helper` 前，`rsp` 是 16 字节对齐。

深入理解

真实编译器会根据局部变量大小、保存寄存器数量、是否省略帧指针、是否需要栈保护、是否有异常展开信息等因素生成不同序言。模板不是背诵答案，而是帮助你建立栈对齐和保存责任的算术模型。

栈展开、CFI 和异常边界

手写汇编函数不仅要能执行，还要能被工具理解。调试器回溯调用栈、profiler 在函数中间采样、异常展开查找调用者、崩溃报告打印 backtrace，都需要知道当前函数如何改变了 `rsp`、保存了哪些寄存器、返回地址在哪里。这些信息通常通过 unwind metadata 描述，在 GNU 汇编里常见写法是 CFI

directives。

为什么只会运行还不够

一个 leaf function 不修改 **rsp**，不保存 callee-saved 寄存器，通常很好展开。工具可以认为返回地址仍在入口栈顶附近。可是一个 non-leaf function 如果执行了多次 **push**、**sub rsp**，...、保存了 **rbx** 或 **rbp**，工具必须知道这些变化，才能从函数中间恢复调用者状态。

如果缺少展开信息，程序仍然可能计算正确，但调试体验会明显变差。GDB 的 backtrace 可能在手写汇编函数处断掉；采样 profiler 可能把大量时间归到错误调用者；异常从 C++ helper 抛出并试图穿过汇编帧时，运行时可能无法正确清理上层帧；崩溃报告可能只显示一串地址。工程质量要求高时，这些都不能忽略。

第一册不会要求完整掌握 DWARF CFI，但要知道它解决什么问题。CFI 不是给 CPU 执行的指令，而是给调试器、异常运行时和采样工具使用的元数据。它描述“在当前指令位置，调用者的栈顶在哪里，返回地址在哪里，保存寄存器在哪里”。

一个带 CFI 的简单框架

典型框架如下：

```
.globl function_name
.type function_name, @function
function_name:
    .cfi_startproc
    push rbp
    .cfi_def_cfa_offset 16
    .cfi_offset rbp, -16
    mov rbp, rsp
    .cfi_def_cfa_register rbp

    # function body

    pop rbp
    .cfi_def_cfa rsp, 8
    ret
    .cfi_endproc
.size function_name, .-function_name
```

这段元数据的意思可以用人话解释。函数入口时，调用者的返回地址在栈顶；执行 **push rbp** 后，栈顶多了保存的 **rbp**，返回地址相对新的栈顶偏移改变；设置 **rbp** 为帧指针后，工具可以用 **rbp** 作为稳定基准；返回前弹出 **rbp** 后，CFA 又回到基于 **rsp** 的状态。

实际 CFI 写法会随序言不同而变化。若函数省略帧指针，只用 **rsp** 加偏移描述；若保存多个 callee-saved 寄存器，每个保存位置都应有 **.cfi_offset**；若中途调整栈空间，CFA 偏移也要更新。初学阶段可以先保持栈框架简单，让 CFI 容易写对。

异常不要随便穿过裸汇编

C++ 异常展开需要知道每一层栈帧如何恢复。如果手写汇编调用一个可能抛异常的 C++ helper，而汇编帧没有正确展开信息，异常穿过该帧时可能出错。即使有 CFI，还要考虑清理逻辑：汇编函数是否保存了寄存器、是否持有资源、是否需要在异常路径恢复状态。这些工作 C++ 编译器会为 C++ 函数生成，但不会自动为你的手写汇编补齐。

因此，热路径汇编 kernel 的推荐契约通常是：不抛异常，不让异常穿过汇编帧，不在汇编内管理 C++ 对象生命周期。如果需要调用可能失败的 helper，优先让 C++ wrapper 处理异常和错误码，再调用不会抛异常的汇编核心。若确实要写可展开汇编，那已经进入更高级的 ABI 和运行时工程，需要专门测试异常路径。

核心理想

汇编函数的工程契约不止寄存器和返回值，还包括工具能否展开它。CFI、简单栈框架、清晰符号和不让异常随意穿过裸汇编，是可维护互操作代码的重要边界。

C++ 对象、引用和指针在边界上的形状

汇编只看见寄存器、内存和地址。C++ 的引用、指针、小结构体、大结构体在 ABI 层有不同形状。

引用通常就是地址

C++:

```
extern "C" void asm_inc_ref(std::int64_t& x);
```

虽然源代码写的是引用，但汇编入口看到的通常是指针：

```
on entry:
    rdi = address of x
```

汇编实现：

```
.globl asm_inc_ref
.type asm_inc_ref, @function
asm_inc_ref:
    inc qword ptr [rdi]
    ret
.size asm_inc_ref, .-asm_inc_ref
```

这不意味着 C++ 引用“语言上等于指针”。语言层引用有别名、生命周期和绑定规则；ABI 层只是经常用地址传递它。

小结构体可能走寄存器

```
struct Pair64 {
    std::int64_t lo;
    std::int64_t hi;
};

extern "C" Pair64 asm_make_pair(std::int64_t x, std::int64_t y);
```

两个 64 位整数大小的结构体，在常见 System V AMD64 规则下可用 `rax` 和 `rdx` 返回：

```
.globl asm_make_pair
.type asm_make_pair, @function
asm_make_pair:
    mov rax, rdi
    mov rdx, rsi
    ret
.size asm_make_pair, .-asm_make_pair
```

大结构体通常通过隐藏指针返回

```
struct Triple64 {
    std::int64_t a;
    std::int64_t b;
    std::int64_t c;
};

extern "C" Triple64 asm_make_triple(std::int64_t x,
                                   std::int64_t y,
                                   std::int64_t z);
```

常见入口状态：

```
rdi = hidden pointer to return object
rsi = x
rdx = y
rcx = z
```

实现：

```
.globl asm_make_triple
.type asm_make_triple, @function
asm_make_triple:
    mov rax, rdi
    mov qword ptr [rdi + 0], rsi
```

```

mov qword ptr [rdi + 8], rdx
mov qword ptr [rdi +16], rcx
ret
.size asm_make_triple, .-asm_make_triple

```

注意显式参数 `x` 不在 `rdi`，因为 `rdi` 被隐藏返回对象指针占用了。这是读跨语言汇编时很常见的坑。

全局数据和 RIP-relative addressing

在 x86-64 位置无关代码中，访问静态数据常用 RIP-relative addressing。

```

.intel_syntax noprefix
.text

.globl asm_get_message
.type asm_get_message, @function
asm_get_message:
    lea rax, [rip + message]
    ret
.size asm_get_message, .-asm_get_message

.section .rodata
message:
    .asciz "hello from assembly"

.section .note.GNU-stack,"",@progbits

```

C++ 声明：

```
extern "C" const char* asm_get_message();
```

`lea` 不是读取字符串内容，而是计算字符串地址：

```
rax = address of message
```

反汇编目标文件时，仍然可能看到 relocation，因为 `message` 的最终地址要到链接阶段才知道：

```

lea rax, [rip + 0x0]
relocation: R_X86_64_PC32 .rodata-0x4

```

深入理解

RIP-relative addressing 的基准是下一条指令的地址，不是当前指令开头地址。这个规则和 `call rel32` 一样，都是 x86-64 位置无关代码的重要基础。后面讲动态库、PLT/GOT 和 PIC 时会继续展开。

结构体参数：把偏移当成公开契约

有些 kernel 参数太多，全部作为独立参数传递会超过寄存器数量，调用点也难读。此时可以用一个平凡参数块：

```

struct DotConfig {
    const std::int32_t* a;
    const std::int32_t* b;
    std::int64_t n;
    std::int64_t stride_a;
    std::int64_t stride_b;
    std::int64_t bias;
};

extern "C" std::int64_t asm_dot_config(const DotConfig* cfg);

```

ABI 层只有一个显式参数：`rdi` 指向 `DotConfig`。汇编内部按偏移读取字段：

```
# rdi = cfg
```



```

mov r8, qword ptr [rdi + 0] # cfg->a
mov r9, qword ptr [rdi + 8] # cfg->b
mov rcx, qword ptr [rdi +16] # cfg->n
mov r10, qword ptr [rdi +24] # cfg->stride_a
mov r11, qword ptr [rdi +32] # cfg->stride_b
mov rax, qword ptr [rdi +40] # cfg->bias

```

这种写法让函数签名稳定，但把结构体布局变成了汇编契约。只要 C++ 结构体字段顺序、类型、对齐或 padding 变化，汇编偏移就可能失效。为了防止静默错误，头文件必须给出布局断言：

```

static_assert(sizeof(DotConfig) == 48);
static_assert(alignof(DotConfig) == alignof(void*));
static_assert(offsetof(DotConfig, a) == 0);
static_assert(offsetof(DotConfig, b) == 8);
static_assert(offsetof(DotConfig, n) == 16);
static_assert(offsetof(DotConfig, stride_a) == 24);
static_assert(offsetof(DotConfig, stride_b) == 32);
static_assert(offsetof(DotConfig, bias) == 40);
static_assert(std::is_standard_layout_v<DotConfig>);
static_assert(std::is_trivially_copyable_v<DotConfig>);

```

参数块的优点和代价

参数块有几个优点。第一，函数签名短，调用点清楚。第二，未来增加内部字段时，可以通过新增版本结构体或 wrapper 兼容旧接口。第三，汇编函数只占用一个参数寄存器，更多寄存器可用于循环内部状态。第四，调试时可以在 GDB 中一次查看整个参数块。

代价也很明确。第一，汇编入口需要额外 load 字段，不能直接从参数寄存器拿到所有值。第二，参数块内存必须在调用期间有效。第三，布局变更风险更高。第四，如果参数块频繁构造且不在 cache 中，可能增加访存。第五，跨语言或跨编译器 ABI 时，结构体布局约定必须更严格。

因此，参数块适合参数较多、配置较复杂、调用频率相对低于内部循环工作量的 kernel。若函数只有两三个整数参数，直接传寄存器更简单。若参数块用于公共 ABI，建议显式版本化：

```

struct DotConfigV1 {
    std::uint32_t size;
    std::uint32_t version;
    const std::int32_t* a;
    const std::int32_t* b;
    std::int64_t n;
};

```

公共 wrapper 可以检查 `size` 和 `version`，再调用内部汇编 kernel。裸汇编热路径不一定要处理版本逻辑，但公共 API 层应当有办法拒绝不匹配的结构体。

常见误区

结构体参数不是“绕过 ABI 表”的捷径。它只是把多个寄存器/栈参数换成一个指针，再把字段偏移变成新的二进制契约。偏移必须用 `static_assert` 固定，并在汇编注释中写清。

内联汇编为什么危险

很多人一学汇编就想在 C++ 里写 inline asm。工程上要反过来：除非你非常清楚编译器优化器和寄存器分配器会怎样看它，否则优先使用普通 C++、compiler builtin、intrinsics 或单独的 .S 文件。

GNU extended asm 的基本形状：

```

asm volatile (
    "assembly template"
    : output operands
    : input operands
    : clobbers
);

```

它不是“把几行汇编插入 C++”这么简单。它是在告诉优化器：

- 哪些值被这个 asm 读。

- 哪些值被这个 asm 写。
- 哪些寄存器、flags、内存状态被破坏。
- 这个 asm 是否可以删除、移动或合并。

如果这些信息写错，编译器可能生成看似合理但实际错误的代码。

错误示例：没有声明输出

```
std::int64_t broken_add(std::int64_t a, std::int64_t b)
{
    std::int64_t result = a;
    asm("add %1, %0" : : "r"(result), "r"(b));
    return result;
}
```

这段的问题是：模板里修改了 `%0` 对应的寄存器，但约束里没有告诉编译器 `result` 是输出。优化器可以认为 `result` 没有改变。

修复：

```
std::int64_t fixed_add(std::int64_t a, std::int64_t b)
{
    std::int64_t result = a;
    asm("add %1, %0"
        : "+r"(result)
        : "r"(b)
        : "cc");
    return result;
}
```

`"+r"(result)` 表示这个操作数既输入又输出，并且放在某个通用寄存器中。`"cc"` 告诉编译器 condition codes 被修改。

错误示例：忘记 "memory" clobber

考虑一个人为的 store：

```
void broken_store(std::int64_t* p, std::int64_t value)
{
    asm volatile("mov %1, (%0)" : : "r"(p), "r"(value));
}
```

汇编确实写了 `*p`，但 C++ 优化器不知道这个 asm 会改变任意内存。若周围代码读取或写入同一内存，优化器可能重排。

更清晰的写法是把内存作为输出操作数：

```
void better_store(std::int64_t* p, std::int64_t value)
{
    asm volatile("mov %1, %0"
        : "=m"(*p)
        : "r"(value)
        : "memory");
}
```

这里 `"=m"(*p)` 明确描述被写的内存对象，`"memory"` 进一步作为编译器级内存屏障，防止优化器把其他内存访问跨过这个 asm 移动。

常见误区

`"memory"` clobber 是编译器屏障，不是 CPU 内存栅栏。它限制编译器重排，但不会自动生成 `mfence`、`lfence` 或 locked instruction。多线程内存序问题必须用 C++ atomics 或明确的硬件同步指令解决。

volatile 不是万能药

`asm volatile` 的含义接近“这个 asm 有副作用，不要因为输出未使用就删除它，也不要随便合并”。但它不自动说明：

- asm 读写了哪些 C++ 变量。
- asm 修改了哪些寄存器。
- asm 修改了 flags。
- asm 访问了内存。
- asm 可以作为 CPU fence。

所以 inline asm 的正确性来自 operands 和 clobbers，而不是只靠 `volatile`。

常见约束

约束	含义
"r"	任意通用寄存器。
"m"	内存操作数。
"i"	编译期立即数。
"a"	使用 <code>rax/eax/al</code> 类寄存器。
"=r"	只写输出寄存器。
"+r"	读写同一个寄存器操作数。
"=&r"	early-clobber 输出，输出在所有输入读完前就可能被写。
"0"	与第 0 个输出操作数使用同一位置。
"cc"	asm 修改了 flags。
"memory"	asm 可能读写未显式列出的内存，限制编译器重排。

深入理解

inline asm 的约束是你和编译器寄存器分配器之间的合同。你不应该在模板里随便写固定寄存器名，然后忘记告诉编译器。固定寄存器越多，寄存器分配越难，优化器越受限制，错误空间也越大。

读写操作数和 matching constraint

很多指令要求某个输入和输出使用同一位置。最简单写法是 `"+r"`，表示该操作数先作为输入读入，asm 执行后又作为输出写回。前面的 `fixed_add` 就是这种情况：`result` 初值是 `a`，模板里对它执行 `add`，最终 `result` 变成新值。

另一种写法是 `matching constraint`，例如 `"0"` 表示某个输入必须和第 0 个输出放在同一寄存器或同一操作数位置。它适合输出约束和输入约束需要分开表达的场景。初学者不必频繁使用，但看到它时要知道：这不是立即数 0，而是“匹配第 0 个操作数”的约束。

```
std::uint64_t add_with_match(std::uint64_t a, std::uint64_t b)
{
    std::uint64_t out;
    asm("add %2, %0"
        : "=r"(out)
        : "0"(a), "r"(b)
        : "cc");
    return out;
}
```

这里第 0 个输出是 `out`，输入 `"0"(a)` 要求 `a` 放在同一个位置。模板执行前，该位置含有 `a`；执行后，同一位置成为 `out`。如果没有 `matching` 或读写约束，编译器可能把输入和输出放在不同寄存器，模板含义就和你想象不同。

选择 `"+r"` 还是 `matching constraint`，取决于表达哪个更清楚。对简单读写同一 C++ 变量，`"+r"` 通常更直接；对输出变量和输入变量在 C++ 层是不同名字、但机器指令要求同一寄存器的情况，`matching constraint` 更明确。无论哪种，都要记住：约束是在描述数据流，不是在美化语法。

early-clobber：输出过早覆盖输入

`"=&r"` 表示 `early-clobber` 输出。它告诉编译器：这个输出可能在所有输入读取完成之前就被写，所以不能和某些输入分配到同一个寄存器。没有 `early-clobber` 时，编译器在认为安全的情况下可能让输出复用输入寄存器，以减少寄存器压力；但某些多指令 asm 模板中，这会破坏还没来得及读取的

输入。

考虑一个简化的两步模板：先把输出清零，再用输入计算。如果编译器把输出和某个输入放在同一寄存器，第一条清零就会提前毁掉输入值。此时应使用 `early-clobber` 输出，阻止这种重叠。

```
std::uint64_t combine(std::uint64_t a, std::uint64_t b)
{
    std::uint64_t out;
    asm volatile(
        "xor %0, %0\n\t"
        "add %1, %0\n\t"
        "add %2, %0"
        : "=&r"(out)
        : "r"(a), "r"(b)
        : "cc");
    return out;
}
```

这个例子可以用普通 C++ 写得更好，它在教材里只是说明 `early-clobber` 的含义。凡是 `asm` 模板由多条指令组成，并且输出在模板开头就被写，而输入在后面才被读取，都要检查是否需要 `early-clobber`。不要等到某个优化级别、某个寄存器压力场景下才偶发错误。

固定寄存器约束

有些指令天然要求固定寄存器。例如 `rdtsc` 把低 32 位写入 `eax`，高 32 位写入 `edx`；某些乘法使用 `rax/rdx`；系统调用在 Linux x86-64 上有固定寄存器约定。这时可以使用 `"=a"`、`"=d"`、`"a"` 这类约束，让编译器知道固定寄存器需求。

固定寄存器比普通 `"r"` 更限制寄存器分配。它不是坏事，但应该只在指令真的要求时使用。若你只是习惯在模板里写 `rax`、`rcx`，却没有在 `clobber` 或约束里声明，编译器可能同时把某个活跃 C++ 值也放在这些寄存器里，导致值被悄悄破坏。相反，如果你把一大堆寄存器都列为 `clobber`，编译器又会被迫保存和重算更多值。

正确态度是精确表达：固定输出就写固定输出约束；固定输入就写固定输入约束；临时破坏但不作为输出的寄存器才放 `clobber`；能让编译器选择普通寄存器时就用 `"r"`。`inline asm` 的可维护性，很大程度上取决于这种克制。

asmgoto：把 asm 接入 C++ 控制流

GNU 扩展还支持 `asmgoto`，允许 `asm` 根据条件跳到 C++ 标签。它用于内核、低层运行时和某些静态分支机制。它比普通 `inline asm` 更危险，因为它把汇编控制流直接接入 C++ 控制流图，编译器需要知道可能跳到哪些标签。

第一册不要求写 `asmgoto`，但要知道它存在，并且不要用普通 `asm` 偷偷跳到 C++ 标签或函数内部地址。编译器如果不知道 `asm` 可能改变控制流，就会按错误的 CFG 优化周围代码。任何会影响控制流的 `asm`，都必须用编译器支持的机制准确描述目标。

常见误区

`inline asm` 的输出、输入、`early-clobber`、固定寄存器和控制流目标，都必须告诉编译器。模板文本里写了什么，不等于优化器知道了什么。

inline asm 的思维模型

理解 `inline asm` 的关键是：编译器不会解析你的汇编模板并自动推断所有语义。对优化器来说，`asm` 模板大体像一个黑盒节点。这个黑盒有哪些输入、有哪些输出、会破坏哪些寄存器和 `flags`、是否访问未知内存，都要靠 `operands` 和 `clobbers` 描述。描述越准确，编译器越能安全优化；描述缺失，编译器就可能在完全合理的前提下生成错误代码。

例如你在模板里写了 `add`，它会修改 `flags`。如果后面的 C++ 条件判断或另一个 `asm` 依赖 `flags`，而你却没有写 `"cc"`，编译器可能认为 `flags` 没有被破坏。再例如你在模板里写了内存 `store`，却没有把被写内存列为输出，也没有写 `"memory"`，编译器可能把周围 `load/store` 移到 `asm` 前后，导致观察顺序和你想象不同。

反过来，`clobber` 也不是越多越好。滥用 `"memory"` 会让编译器不敢重排许多内存访问，可能损失优化机会；固定太多寄存器会增加寄存器压力；不必要的 `volatile` 会阻止删除和合并。好的 `inline`

asm 应该像好的函数接口一样，精确表达需要的信息，不多不少。

编译器屏障和硬件屏障

“memory” clobber 只告诉编译器：这个 asm 可能读写内存，所以不要把普通内存访问随意跨过它重排。它不一定生成任何硬件 fence，也不一定让其他 CPU 核心看到某种顺序。多线程同步必须使用 C++ 原子操作、系统调用约定、锁指令或明确的硬件屏障。

例如：

```
asm volatile("" ::: "memory");
```

这是一种空的编译器屏障。它没有机器指令，却限制编译器重排。与此不同，mfence、lfence、sfence 或带 lock 前缀的指令是 CPU 层的同步机制，会影响硬件执行和内存可见性。把二者混淆，是并发代码中非常危险的错误。

本书在第一册不深入多线程内存模型，但要强调一条底线：inline asm 不是绕开 C++ 内存模型的捷径。你可以用它表达特定机器指令，但必须同时向编译器和 CPU 两个层面说明顺序需求。只让其中一方知道，另一方仍然可能合法重排。

优先寻找替代方案

很多 inline asm 需求其实有更安全的替代方案。读时间戳可以使用编译器内建或封装好的平台函数；预取可以使用 builtin；SIMD 可以使用 intrinsics；原子操作应使用 std::atomic 或成熟库；特殊系统交互可以放到单独 .S 文件，并用普通函数 ABI 包装。

inline asm 最适合非常局部、没有更好表达方式、且作者能精确描述优化器合同的场景。如果一段 inline asm 变长、需要多处标签、需要复杂寄存器分配、需要和 C++ 控制流交织，通常应该改成单独的汇编函数或 intrinsic 版本。这样边界更清楚，测试和反汇编审计也更容易。

常见误区

inline asm 的危险不在于汇编语法本身，而在于它同时跨越 C++ 优化器、寄存器分配、ABI、内存模型和 CPU 指令语义。任何一个层面描述不完整，都可能只在高优化级或特定调用点暴露。

intrinsics、inline asm 与 .S 的选择

现代高性能代码通常优先选择下面顺序：

方式	优点	风险
普通 C++ compiler builtin	可移植、优化器可理解、测试容易。表达特殊语义，如 prefetch、expect、overflow。	不一定生成你想要的指令。依赖编译器扩展。
intrinsics	直接表达 SIMD 指令，寄存器分配仍由编译器做。	ISA 绑定强，代码可读性下降。
.S 文件	可完全控制函数级汇编和 ABI。	维护成本高，调试和 unwind 要额外注意。
inline asm	可在表达式附近插入特殊指令。	最容易和优化器合同写错。

对于 AVX2/AVX-512/FMA/VNNI 这类后续 AI 算子优化，第二册会大量使用 intrinsics。原因是 intrinsics 让编译器仍然负责寄存器分配、调用约定、栈框架和调度的一部分，同时你可以明确控制核心 SIMD 操作。

单独 .S 文件适合：

- 你需要完全固定函数入口、寄存器分配和指令布局。
- 你正在写非常小的热路径 kernel。
- 你需要验证某个 ABI 或机器码现象。
- 你愿意维护多 ISA 版本和测试矩阵。

inline asm 适合：

- 少数没有合适 intrinsic 的特殊指令。
- 需要非常局部的硬件交互。
- 你已经能准确写出 operands、constraints 和 clobbers。

调试和审计工作流

写互操作代码时，永远保留可审计证据。

编译参数

```
clang++ -std=c++20 -O2 -g -fno-omit-frame-pointer \
main.cpp kernels.S -o app
```

训练阶段建议：

- 用 `-g` 保留调试信息。
- 初期加 `-fno-omit-frame-pointer`，让栈回溯更直观。
- 分别看 `-O0` 和 `-O2/-O3`，理解优化差异。
- 对汇编文件保留注释和清晰符号名。

符号审计

```
nm -C app | rg 'asm_\.cpp_'
readelf -Ws app | rg 'asm_\.cpp_'
```

要检查：

- 汇编函数是否真的导出。
- C++ 是否引用了预期符号名。
- 是否出现未预期的 mangled name。
- 符号类型是否是函数。

反汇编审计

```
objdump -drwC -Mintel app > app.objdump.txt
rg -n '<asm_sum>|<asm_square_plus>|call.*cpp_square' app.objdump.txt
```

要检查：

- 参数寄存器使用是否和 ABI 一致。
- 栈参数偏移是否正确。
- callee-saved 寄存器是否成对保存恢复。
- 调用前 `rsp` 是否对齐。
- 是否出现你没预料到的 PLT 调用或间接跳转。

GDB 单步

```
gdb ./app
(gdb) break asm_square_plus
(gdb) run
(gdb) disassemble /r asm_square_plus
(gdb) info registers rdi rax rbx rsp rip
(gdb) x/6gx $rsp
(gdb) stepi
(gdb) info registers rdi rax rbx rsp rip
```

单步时不要只看源代码行。要看：

- `rip` 是否进入预期符号。
- `rsp` 每次 `push`、`pop`、`call`、`ret` 如何变化。
- 返回地址是否位于预期栈槽。

- 返回值是否写入 `rax` 或 `xmm0`。

CI 中应该保留哪些检查

互操作代码不能只靠本地手工跑一次。进入项目后，至少应在 CI 中保留几类自动检查。第一，构建矩阵：Debug、Release、RelWithDebInfo 至少两种；若项目支持多个编译器，Clang 和 GCC 都应覆盖。第二，测试矩阵：确定性测试、随机测试、边界测试、非法输入 wrapper 测试。第三，工具审计：检查公共符号列表是否符合预期，检查目标文件没有 executable stack 警告，检查关键汇编函数仍然存在而没有链接脚本或构建条件排除。

符号审计可以很轻量。例如把允许导出的 `asm_` 符号写在一个文本清单里，CI 运行 `nm` 或 `readelf-WS` 后比对。若某次改动多导出了 `helper`、`tmp_loop`，或者少了 `asm_dot_i32`，CI 应该失败。这能防止 ABI 面无意扩大，也能防止构建系统漏编某个汇编文件。

反汇编审计也可以自动化一部分。比如检查 `asm_call_helper` 中是否出现 `callcpp_mul_add`，检查 `asm_dot_i32` 是否仍然有预期符号大小，检查某些测试专用坏符号没有链接进 release。不要把 CI 写成依赖每个机器码字节完全不变；编译器、链接器和地址布局会变化。更稳妥的是检查结构性事实：符号存在、重定位类型可解释、没有未定义符号、没有可执行栈、测试通过。

callee-saved 破坏的专门测试

callee-saved 寄存器破坏有时很难被普通结果测试发现。因为调用者不一定在这些寄存器里保存了重要值，或者优化级别不同，寄存器分配不同，错误可能暂时不暴露。为了测试这类问题，可以写一个小的 C++ 或汇编 harness，在调用待测函数前把 `rbx`、`rbp`、`r12` 到 `r15` 设置成哨兵值，调用后检查它们是否保持不变。

这种 harness 本身也需要小心，因为普通 C++ 不能直接保证某个变量放在特定 callee-saved 寄存器。可以使用单独汇编测试包装器，或者在受控 inline asm 中设置和读取寄存器，并准确声明 clobber。第一册不要求你实现完整 harness，但要知道：仅比较函数返回值不能充分证明 ABI 保存责任正确。

另一个实用办法是制造寄存器压力。让调用点周围有多个长期存活变量，编译器更可能把一些值放入 callee-saved 寄存器。若汇编函数破坏这些寄存器，调用后计算就可能失败。后面的汇编调用 C++ helper 实验要求构造这种调用者，就是为了让错误更容易显现。

栈对齐的专门测试

栈不对齐的错误有时不会立刻崩溃。若被调用 helper 只使用普通整数指令，它可能“看起来没问题”；一旦 helper 使用要求对齐的保存策略、调用库函数、编译器生成对齐假设下的 SIMD load/store，问题才暴露。因此，non-leaf 汇编函数必须专门测试栈对齐。

一个实用办法是在 C++ helper 里检查入口附近的栈指针。普通 C++ 不能直接、可移植地读取 `rsp`，但在本章的 x86-64 GNU/Clang 环境中，可以写一个测试专用 helper：

```
extern "C" bool cpp_check_stack_alignment()
{
    std::uintptr_t rsp = 0;
    asm volatile("mov %0, %%rsp; %0 : "=r"(rsp));
    return (rsp % 16) == 8;
}
```

为什么检查 `rsp%16==8`？因为 System V AMD64 约定要求调用者在执行 `call` 前让 `rsp` 16 字节对齐；`call` 会压入 8 字节返回地址，所以被调用函数入口看到的 `rsp` 通常是 `16n+8`。若 helper 入口看到别的余数，就说明调用者没有按约定对齐。

这类 helper 只能用于测试，不应成为生产 API 的一部分。生产代码里更好的做法是通过汇编审计和清晰栈框架保证每个 call site 前对齐正确。测试 helper 的价值在于快速暴露错误版本，尤其适合教学。

保存寄存器的测试包装器

callee-saved 寄存器破坏也可以用汇编测试包装器直接检测。思路是：测试包装器先给 `rbx`、`rbp`、`r12` 到 `r15` 写入哨兵值，调用被测函数，返回后检查这些寄存器是否仍然等于哨兵值。

伪代码如下：

```

set rbx, sentinel0
set r12, sentinel1
set r13, sentinel2
set r14, sentinel3
set r15, sentinel4
call function_under_test
compare saved registers with sentinels
return 0 if preserved, nonzero otherwise

```

这个包装器本身也必须遵守 ABI：它修改 callee-saved 寄存器时，要在返回给 C++ 测试程序前恢复原值。否则测试包装器会污染自己的调用者。实际工程中，可以把这种包装器写在测试专用 .S 文件里，只链接进测试二进制，不进入 release 库。

若暂时不写专用包装器，也应至少使用寄存器压力测试和多次调用测试。比如在调用汇编函数前后保留多组活跃值，调用后参与校验。这样编译器更可能把一部分值放入 callee-saved 寄存器，从而让破坏更容易被发现。但 this 方法是间接的，不如专用包装器可靠。

核心理想

ABI 错误要用 ABI 级测试暴露。只比较函数返回值，往往抓不到 callee-saved 破坏、栈对齐错误和异常展开问题。

保护页和哨兵内存

测试汇编 load/store 越界时，普通数组不一定能暴露问题。越界一两个元素可能仍然落在同一分配块或 padding 中，程序不崩溃，结果也未必立即错。更强的测试方法是使用保护页或哨兵内存。保护页把不可访问页面放在缓冲区旁边，越界访问会触发缺页；哨兵内存存在缓冲区前后填固定字节，调用后检查是否被改写。

保护页测试能发现越界读写，但实现比普通单元测试复杂，需要系统调用分配页、设置页权限，并让数据靠近页边界。哨兵内存更简单，适合 CI 长期运行。两者都不能替代正确性证明，但能提高错误暴露概率。对汇编 kernel 来说，测试设计越接近它可能出错的边界，越有价值。

同时要注意，非法输入测试应该打在 wrapper 层，而不是直接要求热路径 kernel 对所有非法输入安全返回。如果契约写明 `asm_dot_i32` 要求 `a` 和 `b` 指向至少 `n` 个元素，那么直接传空指针给 kernel 导致崩溃，不一定是 kernel bug；wrapper 没有阻止空指针，才是边界 bug。报告要区分这两层。

sanitizer 和手写汇编的边界

AddressSanitizer、UndefinedBehaviorSanitizer、ThreadSanitizer 对 C++ 很有价值，但它们不能自动理解你所有手写汇编内存访问。比如汇编里：

```
mov rax, qword ptr [rdi + rcx*8]
```

如果 `rdi` 指向越界地址，ASan 未必能像检查 C++ 数组访问那样插桩发现。原因是插桩发生在编译器能理解的 IR 层；手写汇编已经绕过了这部分语义。

因此汇编边界测试要更保守：

- C++ wrapper 先检查尺寸、空指针和对齐前置条件。
- 汇编 kernel 文档明确要求输入合法。
- 随机测试使用保护页或额外 padding 检查越界。
- 对 debug 版本提供纯 C++ reference path。
- 必要时在汇编函数外层做 shadow check，而不是假设 sanitizer 能看见内部。

核心理想

优化 kernel 的工程边界通常是“外层 C++ 负责合法性和调度，内层汇编负责极少分支的热路径”。不要把复杂错误处理塞进手写汇编热循环。

工具能看到什么

sanitizer 的能力来自编译器插桩。C++ 源码被编译成中间表示时，编译器知道一次数组访问、一次类型转换、一次原子操作、一次函数调用的语义，于是能在关键位置插入检查。手写汇编已经是目标机器指令，编译器通常不会把 `[rdi+rcx*4]` 还原成“访问第 `rcx` 个元素并检查边界”。这不是 sanitizer 质量差，而是信息层次已经丢失。

因此，sanitizer 仍然可以帮助发现 wrapper 层错误、调用前参数准备错误、C++ reference 错误、越界分配和释放错误，但它不保证覆盖汇编内部每一条 load/store。若汇编写坏了红区、越界写到仍然可访问的 padding、破坏 callee-saved 寄存器、返回错误地址，sanitizer 可能只在后续 C++ 代码中间接报错，甚至完全不报。

这要求互操作项目把检查放在汇编外层。debug wrapper 可以做尺寸检查、对齐检查、重叠检查和范围检查；release wrapper 可以在性能和安全之间取舍；测试程序可以分配带 guard page 的缓冲区，故意把数组放在页边界附近，让越界访问更容易触发崩溃；也可以在调用前后检查哨兵字节和 callee-saved 寄存器间接效果。

纯 C++ reference 的价值

每个汇编 kernel 都应该有纯 C++ reference。reference 不一定快，但必须语义清楚、容易审查、容易被 sanitizer 插桩。汇编测试不应只比较少量手算答案，而应在大量输入上比较汇编结果和 reference 结果。这样即使汇编实现很复杂，正确性基准仍然在编译器可理解的语言层。

reference 还帮助定义边界。如果 `n<0` 是非法输入，reference 可以断言或由 wrapper 拒绝；如果 `n==0` 应返回 0，reference 明确表达；如果 unsigned 溢出是预期 wraparound，reference 使用 unsigned 类型表达。不要让 reference 和汇编在未定义行为上“碰巧一致”。

调试符号和展开信息

手写汇编如果只追求能运行，调试体验会很差。没有清晰符号名，profiler 只显示匿名地址；没有 `.type` 和 `.size`，工具可能难以识别函数边界；没有正确的栈行为，GDB 回溯可能断裂；如果未来涉及异常或采样展开，还需要 CFI 信息描述保存寄存器和栈帧变化。

第一册实验可以先不要求完整 CFI，但应该养成两点习惯：函数符号规范，栈变化简单可推导。等后续写更复杂 kernel，尤其是会调用 helper 或需要在生产 profiler 中出现的函数，就要补充展开信息，让工具能够从函数中间恢复调用者状态。

常见误区

“sanitizer 没报错”和“汇编正确”不是同一件事。手写汇编的正确性必须由接口契约、reference、测试矩阵、反汇编审计和必要的动态观察共同证明。

一个接近算子 kernel 的例子：整数点积

先写标量整数点积，为后续 SIMD 章节做准备。

C++ 声明：

```
extern "C" std::int64_t asm_dot_i32(const std::int32_t* a,
                                   const std::int32_t* b,
                                   std::int64_t n);
```

语义：

```
sum = 0
for i in [0, n):
    sum += int64(a[i]) * int64(b[i])
return sum
```

ABI 入口：

```
rdi = a
rsi = b
rdx = n
rax = return value
```

汇编：

```
.intel_syntax noprefix
.text

.globl asm_dot_i32
.type asm_dot_i32, @function
asm_dot_i32:
    xor rax, rax
    test rdx, rdx
    jle .Ldone

    xor rcx, rcx
.Lloop:
    movsxd r8, dword ptr [rdi + rcx*4]
    movsxd r9, dword ptr [rsi + rcx*4]
    imul r8, r9
    add rax, r8
    inc rcx
    cmp rcx, rdx
    jl .Lloop

.Ldone:
    ret
.size asm_dot_i32, .-asm_dot_i32

.section .note.GNU-stack,"",@progbits
```

地址公式：

```
address of a[i] = rdi + rcx * 4
address of b[i] = rsi + rcx * 4
```

为什么 scale 是 4? 因为 `std::int32_t` 元素大小是 4 字节。这个 scale 是 x86 地址模式支持的合法 scale。若元素结构大小是 12 字节，就不能写 `[base+index*12]`，必须用 `lea` 或其他指令拆解。

这段代码还不是高性能点积。它每次循环只做一个元素，乘法和加载串行依赖较多，没有展开，没有 SIMD，也没有处理内存带宽。但它是非常好的互操作训练材料，因为它包含：

- 指针参数。
- signed load extension。
- 循环控制流。
- 数组地址公式。
- 64 位累加返回值。

汇编边界的文档模板

每个手写汇编函数都应有一段接口注释。示例：

```
# std::int64_t asm_dot_i32(const std::int32_t* a,
#                         const std::int32_t* b,
#                         std::int64_t n)
#
# ABI:
#   rdi = a
#   rsi = b
#   rdx = n
#   rax = return sum
#
# Preconditions:
#   a and b point to at least n int32 elements
#   n >= 0
#   pointers may be unaligned
#
# Clobbers:
#   rax, rcx, r8, r9, flags
#
# Callee-saved:
#   none modified
```


这不是形式主义。几个月后你优化这个 kernel 时，注释会告诉你哪些寄存器本来可以自由使用，哪些语义由外层保证，哪些行为必须保持。

文档要覆盖变更风险

接口注释不只服务当前实现，也服务未来修改。一个汇编 kernel 常会经历多个版本：标量版本、展开版本、SIMD 版本、按 ISA dispatch 的版本、为不同数据规模特化的版本。只要外部 C++ 声明不变，所有版本都必须遵守同一份契约。文档越清楚，后续优化越不容易破坏边界。

建议在接口注释中额外写三类信息。

第一，别名和重叠规则。输入输出是否允许相同指针？是否允许部分重叠？如果 `out==x` 可以，`out` 与 `y` 部分重叠不可以，就要明确写出。许多优化会改变 load/store 顺序，别名规则不清会让优化后的实现错得很隐蔽。

第二，对齐和尾部规则。函数接受任意地址，还是要求 16/32/64 字节对齐？`n` 可以是任意值，还是必须是向量宽度的倍数？尾部由本函数处理，还是由外层 dispatch 到 scalar cleanup？这些规则会直接影响汇编循环结构。

第三，异常和错误处理规则。汇编函数是否可能调用 C++ 函数？是否允许 helper 抛异常？如果前置条件不满足，是返回错误码、断言失败，还是未定义行为？热路径 kernel 通常不抛异常，也不在内部分配内存，这些约束应写清。

第四，符号和可见性规则。这个符号是公共 ABI，还是仅供当前库内部使用？是否允许外部用户直接调用？若将来修改参数或前置条件，是否必须新增版本符号？这些问题看似偏链接层，但它们决定后续维护能否安全演进。

第五，调试和展开规则。函数是否保持简单 leaf 形态？是否有帧指针？是否提供 CFI？是否允许 profiler 从函数中间展开？是否允许异常穿过该帧？如果答案是不允许，也应写清楚，让调用方不要把它放在异常传播路径上。

上线前审计清单

一个准备合入工程的汇编互操作函数，至少应通过下面检查：

1. C++ 头文件声明和汇编导出符号完全一致。
2. 参数位置和返回位置有 ABI 推导表。
3. 每个 callee-saved 寄存器的修改都有保存恢复证据。
4. 每个 `call` 前的 `rsp` 对齐经过计算。
5. 栈参数偏移基于当前 `rsp` 或 `rbp` 状态重新计算。
6. 全局数据访问使用适合 PIE/PIC 的寻址方式。
7. 目标文件 relocation 能解释，最终二进制反汇编已审计。
8. 公共符号和内部符号边界清楚，没有意外导出。
9. non-leaf 或复杂栈帧函数具备可解释的展开策略；需要时提供 CFI。
10. C++ reference 存在，且测试覆盖确定性、随机和边界输入。
11. sanitizer 能覆盖 wrapper 层；汇编内部风险有额外测试。
12. CI 覆盖至少两种构建类型，并包含符号或二进制结构性检查。
13. 性能报告包含输入规模、平台、编译选项和测量方法。

这张清单看起来长，但它的目的不是增加流程负担，而是防止低级 ABI 错误混进后续性能工作。一个边界错误的 kernel，即使跑得很快，也没有工程价值。

核心思想

汇编优化的前提是契约稳定。没有接口文档、测试矩阵和二进制审计，性能数字越漂亮，风险越大。

互操作 API 的设计审查

在写第一行汇编之前，应该先审查 API。很多汇编互操作问题并不是指令写错，而是接口一开始就把不稳定、不清楚、难测试的东西暴露给了二进制边界。一个好 API 会让汇编实现简单、测试容易、后续优化安全；一个坏 API 会让每个版本都在猜调用者到底承诺了什么。

互操作 API 设计可以按五个问题审查。

第一，参数是否表达了完整边界。指针参数通常需要长度；输出缓冲区需要容量；stride 需要单位；矩阵需要行数、列数和 leading dimension；字符串需要说明是否以零结尾，还是显式长度。只传一个裸指针，却让汇编函数在内部猜长度，是危险接口。汇编没有类型系统保护，越要把边界写成参数或前置条件。

第二，类型宽度是否固定。跨汇编边界优先使用 `std::int32_t`、`std::uint64_t`、`std::size_t` 这类语义明确的类型，不要用含糊的 `long` 作为公共接口，除非接口明确只服务某个 ABI。尤其是 Windows x64 和 System V AMD64 上 `long` 宽度不同，把它放进跨平台接口会制造移植陷阱。

第三，错误如何报告。热路径汇编 kernel 常常假设输入满足前置条件，不在内部做复杂错误处理；公共 wrapper 则应检查空指针、长度为负、容量不足、版本不匹配等问题。不要让汇编函数一边承担性能热路径，一边承担用户输入验证、异常转换和日志输出。职责混在一起，既慢又难审计。

第四，内存所有权是否单向清楚。汇编函数一般不应该分配内存再让 C++ 调用者猜如何释放；也不应该保存一个指向调用者临时对象的指针，除非文档明确生命周期。若必须跨边界管理资源，应提供成对的创建/销毁函数，并规定同一个模块负责分配和释放。

第五，版本演进是否有空间。公开 ABI 一旦发布，修改参数顺序、结构体字段、返回方式都会破坏旧调用者。若你预计接口会演进，可以使用参数块加 `size`、`version` 字段，或者保留内部符号只让 C++ wrapper 暴露稳定 API。不要把第一次实验用的裸符号直接变成长期公共 ABI。

心智模型

汇编互操作的 API 不是“让汇编能被调用”就够了。它要同时满足可推导、可测试、可审计、可演进。写汇编之前先审查 API，通常比事后调 ABI bug 更便宜。

wrapper 应该承担哪些责任

C++ wrapper 不是多余包装。它是把语言层复杂性和汇编热路径隔开的缓冲层。一个设计良好的 wrapper 通常承担下面职责：

- 检查公共 API 的输入合法性，例如空指针、长度、容量、版本号。
- 把 C++ 类型转换成汇编 kernel 需要的简单类型，例如裸指针、字节数、元素数。
- 处理尾部、空输入、小输入和错误路径。
- 根据 CPU 特性或构建选项选择不同 kernel。
- 维护 reference 结果或 debug 校验。
- 隐藏内部符号，避免外部用户绕过契约直接调用 kernel。

汇编 kernel 则应尽量只做一件事：在清晰前置条件下执行核心计算。例如 `asm_dot_i32` 可以假设 `a` 和 `b` 指向至少 `n` 个元素，`n >= 0`，返回 64 位累加和。它不应该打印错误，不应该分配内存，不应该解析命令行，不应该在热循环里检查每个指针是否为空。这样写出的 kernel 才容易证明 ABI 正确，也容易测量性能。

wrapper 和 kernel 的边界还可以服务测试。wrapper 测试覆盖公共行为：错误输入、空输入、边界长度、异常路径。kernel 测试覆盖核心计算：寄存器保存、栈对齐、边界 `n`、随机数据、别名规则。两层测试混在一起，会让失败定位困难；分开之后，公共接口错和汇编实现错能更快区分。

哪些接口不适合直接给汇编

并不是所有 C++ 接口都适合直接由汇编实现。下面几类应谨慎。

第一，接受复杂 C++ 对象的接口。对象可能有非平凡构造、析构、虚表、异常语义、allocator、内部不变量。汇编直接操作它们的字段，等于把对象布局和生命周期全部公开。除非这是非常受控的内部实现，否则应通过 wrapper 提取简单数据。

第二，含有模板和内联策略的接口。模板实例化可能生成多个符号，名字修饰复杂，类型组合多。汇编更适合实现少量稳定的 concrete kernel，再由模板 wrapper 做类型分发。

第三，含有异常传播的接口。裸汇编若没有正确 CFI 和异常边界设计，不应让 C++ 异常穿过。公共 wrapper 可以把异常转换成错误码，或保证调用的汇编函数不抛异常、不调用会抛异常的 helper。

第四，跨标准库对象的接口。例如直接传 `std::string`、`std::vector`、`std::span` 到公共汇编 ABI。`std::span` 在 C++ 源码层很轻量，但它的对象布局仍是 C++ 类型约定；公共 ABI 更稳的

做法是把它拆成指针和长度。内部同编译器同版本工程里可以更灵活，但文档要写清范围。

二进制审计应成为开发循环的一部分

互操作开发不能等到最后才看反汇编。推荐每次修改汇编或 wrapper 后都做一次小审计，确认源代码层、目标文件层和链接后层一致。

一个实用开发循环如下：

1. 修改 C++ 声明、wrapper 或 .S。
2. 编译目标文件，不先跑性能。
3. 用 `nm-C` 检查导出符号和未定义符号。
4. 用 `objdump-drwC-Mintel` 检查指令、重定位和调用目标。
5. 用 `readelf-s-r-S` 检查符号表、重定位和 section。
6. 跑正确性测试，包括边界和随机输入。
7. 跑 ABI 专门测试，例如 callee-saved、栈对齐、错误路径。
8. 最后再做 benchmark。

这个顺序有一个重要理由：性能测量只能在正确边界上进行。若符号错、参数错、返回错、栈错，benchmark 结果没有意义。很多“汇编版本快很多”的故事，最后发现只是少算了元素、少处理了尾部、破坏了返回值或被优化器绕开了调用。

符号审计要看正反两面

符号审计不只看你想导出的符号是否存在，还要看不想导出的符号是否泄漏。公共库里意外导出内部 kernel，会让外部用户绕过 wrapper 直接调用，未来你就很难修改前置条件。可以通过可见性属性、版本脚本、命名约定或构建系统控制导出集合。

同时要检查未定义符号是否符合预期。一个 leaf 汇编 kernel 不应该意外引用 `printf`；一个本应完全静态的目标文件不应该留下未解释的外部 helper；一个位置无关对象不应该出现不适合动态库的绝对重定位。未定义符号不是一定错误，但必须能解释。

重定位审计能发现地址层错误

RIP-relative、GOT、PLT、绝对地址、局部标签和外部符号都会为目标文件中留下不同形态。手写汇编若用错地址写法，可能在非 PIE 可执行文件中能链接，在 PIE 或动态库中失败；也可能在链接时通过，但运行时地址被截断。用 `objdump-drwC` 和 `readelf-r` 审计重定位，可以及早发现这类问题。

审计时不要只看指令助记符，要看重定位类型和符号。访问内部只读表、调用外部函数、读取全局变量、返回字符串地址，分别可能需要不同重定位。若你不理解某个重定位项，先不要把代码视为稳定。它可能是合理的，也可能是地址模型不匹配。

互操作测试要故意制造坏情况

只测几个正常输入，不能证明汇编边界安全。互操作测试应故意制造容易暴露 ABI 和边界错误的情况。

第一，长度边界。测试 `n=0`、`n=1`、向量宽度减一、向量宽度、向量宽度加一、大长度，以及可能触发 32 位截断的长度。即使第一册的 kernel 是标量，也应养成这个习惯，因为未来 SIMD 版本最容易在尾部出错。

第二，数据边界。整数 kernel 要测试 0、1、-1、最大值、最小值、随机值和会触发累加溢出的组合。浮点 kernel 要测试 0、负数、极大值、极小值、NaN、Inf、非规格化数是否在接口范围内。若接口声明不支持某些值，也要有测试确认 wrapper 拒绝或文档明确前置条件。

第三，地址边界。使用不同对齐的指针，测试输入和输出相同、部分重叠、完全不重叠，测试靠近保护页末尾的缓冲区以暴露越界读写。保护页测试尤其适合发现“多读几个元素但结果没用”的 SIMD 或展开 bug。

第四，寄存器边界。用测试 wrapper 在调用汇编函数前给 callee-saved 寄存器写特定哨兵值，调用后检查是否保持。也可以在调用前后记录 `rsp`，检查栈是否恢复。普通功能测试很难发现这类错误，因为返回值可能正确，但调用者状态已经被破坏。

第五，链接边界。至少测试 debug/release 两种构建，PIE 与非 PIE，静态库或动态库模式中的一种。如果函数预计作为库内部符号使用，还要测试外部不能直接链接到内部符号。链接方式改变后才出现的问题，通常说明地址模型或符号可见性没有设计清楚。

核心思想

互操作测试的价值在于主动寻找“正常样例看不出来”的错误：尾部、对齐、别名、寄存器保存、栈状态、重定位和符号边界。

从标量 kernel 演进到优化 kernel

本章的点积示例故意保持标量。真实项目里，汇编 kernel 往往会从标量版本演进到展开、SIMD、多 ISA dispatch、预取、分块和多线程版本。演进过程中最重要的是保持外部契约不变，并让每一步都有 reference 对照。

建议采用分阶段策略。

第一阶段，写纯 C++ reference。它应尽量直接、清楚、定义良好，不追求最快。reference 是正确性标准，不是性能目标。若 reference 也写得过度优化，后面发现差异时很难判断谁错。

第二阶段，写标量汇编 kernel。目标是验证 ABI、符号、构建、测试和文档流程。标量汇编未必比 C++ 快，但它能暴露所有互操作基础问题。

第三阶段，做小幅结构优化，例如减少无用 load/store、使用更合适的寄存器分配、展开两到四次。每次优化后都跑完整正确性测试和二进制审计，确保不是靠改变语义获得速度。

第四阶段，引入 ISA 特定实现，例如 SSE、AVX2、AVX-512 或平台专用扩展。这时 wrapper 应负责检测 CPU 特性和选择函数指针，公共 API 不应要求调用者知道内部 ISA。

第五阶段，建立性能矩阵。不同输入规模、对齐、数据分布、尾部长度、cache 状态都要覆盖。一个 SIMD kernel 在大输入上快，不代表小输入也值得 dispatch；一个展开版本在某台 CPU 上快，不代表所有 CPU 上都快。dispatch 策略本身也要测。

不要删除旧的 reference 和标量版本

优化版本合并后，reference 和标量版本仍然有价值。reference 继续作为正确性标准；标量版本继续作为调试退路、无特殊 ISA 机器的 fallback、差异定位工具和教学材料。当 SIMD 版本在某个边界失败时，用标量版本对比可以快速缩小问题范围。

如果担心生产二进制体积，可以通过构建选项控制是否包含调试 kernel，但源码和测试中应保留它们。一个项目只有“最快版本”，没有“最清楚版本”，长期维护会变得困难。

内联汇编的替代路线

很多 inline asm 的需求其实有更安全的替代路线。

第一，普通 C++ 加编译器内建函数。溢出检测可以用 `__builtin_add_overflow`；位操作可以用 `std::countl_zero`、`std::popcount`、`std::rotr`；内存屏障和原子语义可以用 `std::atomic`。这些形式让编译器理解语义，通常比黑盒 asm 更容易优化。

第二，intrinsics。若你需要特定 SIMD 指令，intrinsics 往往比 inline asm 更合适。它们仍然暴露目标 ISA，但编译器知道寄存器、类型和依赖关系，能做调度和寄存器分配。intrinsics 也更容易和 C++ 类型、模板 dispatch、调试信息结合。

第三，单独 .S 文件。若你确实需要完全控制指令序列、标签、对齐和寄存器，单独汇编文件通常比 inline asm 更清晰。它有明确 ABI 边界，可以用 objdump 审计，可以写完整 CFI，也不会把优化器约束和 C++ 表达式混在同一个语句里。

第四，编译器属性或选项。有时你只是想阻止内联、保留帧指针、指定目标 ISA、调整对齐或打开某个优化选项，不需要写 asm。先查编译器是否已有属性或 pragma。用 asm 解决编译器已经支持的问题，通常会降低可移植性和可维护性。

inline asm 不是禁区，但它应该是最后选择。使用它时，必须写清 operands、clobbers、内存语义、flags 语义和测试覆盖。若无法向另一个工程师解释每个约束为什么正确，就说明这段 asm 不应进入核心代码。

互操作代码评审应该问什么

评审汇编互操作代码时，不要只看指令是否“看起来对”。建议 reviewer 按下面问题逐项追问。

1. 公共 C++ 声明在哪里，是否只有一个权威声明。
2. 这个函数是否公共 ABI，还是内部实现细节。
3. 参数和返回值的 ABI 推导是否写在注释或文档里。
4. 前置条件是否覆盖指针、长度、对齐、别名和尾部。
5. 错误输入由 wrapper 处理，还是被声明为未定义前置条件。
6. 所有 callee-saved 寄存器是否在所有路径恢复。
7. 所有 `call` 前栈是否对齐，变参调用是否设置必要元信息。
8. 全局数据访问是否适合目标链接模式。
9. 测试是否包含边界、随机、寄存器保存和栈对齐。
10. benchmark 是否在 correctness 和二进制审计之后进行。

如果评审只能回答“样例能跑”，这段代码还不应合入。底层互操作的风险不在于简单输入失败，而在于某个未来版本、某个链接模式、某个边界长度、某个调用者寄存器状态下失败。

互操作接口的版本化

只要一个汇编符号可能被外部用户、插件、脚本绑定或其他库直接调用，它就需要版本化思维。版本化不是一开始就设计复杂框架，而是承认接口会演进，并给演进留出空间。

最简单的版本化方式是符号名版本化：

```
asm_dot_i32_v1
asm_dot_i32_v2
```

v1 保持旧参数和旧语义，**v2** 可以增加参数、改变前置条件或使用新的结果格式。公共 wrapper 根据调用者需求选择版本。这样旧二进制仍然可以链接旧符号，新代码可以迁移到新符号。缺点是符号数量增加，维护成本上升。

另一种方式是参数块版本化：

```
struct DotConfigV1 {
    std::uint32_t size;
    std::uint32_t version;
    const std::int32_t* a;
    const std::int32_t* b;
    std::int64_t n;
};
```

wrapper 先检查 **size** 和 **version**，再调用内部汇编。未来增加字段时，可以让新结构体尾部扩展，旧字段 offset 不变。内部热路径不一定直接处理所有版本；它可以只接收 wrapper 规范化后的简单参数。

什么时候必须新增版本

下面变化通常需要新增版本或明确破坏 ABI：

- 参数数量、顺序、类型或宽度改变。
- 返回类型或大结构体返回方式改变。
- 参数块字段顺序、对齐、大小或语义改变。
- 允许或禁止输入输出重叠。
- 对齐前置条件从任意地址变成必须 32 字节对齐。
- 错误处理从返回错误码变成未定义行为，或反过来。
- 所有权规则改变，例如返回指针释放方式改变。

下面变化未必需要新增公共 ABI，但需要更新文档和测试：

- 内部寄存器分配改变。
- 标量实现换成展开实现。

- wrapper 内部选择不同 kernel。
- 性能阈值或 dispatch 策略改变。
- 添加新的内部符号但不公开。

判断标准很简单：外部调用者是否需要重新编译或修改调用方式？旧二进制是否仍能正确调用？如果答案不确定，就不要假装兼容。

dispatch：选择实现也是接口的一部分

许多底层库会有多个实现：reference、标量汇编、展开版本、特定 ISA 版本、调试检查版本。选择哪个实现，称为 dispatch。第一册不深入 SIMD 指令，但需要理解 dispatch 的工程边界。

一个安全的 dispatch 层应满足：

- 公共函数签名稳定。
- 每个内部 kernel 共享同一语义合同。
- CPU 特性检测只在 wrapper 或初始化层进行。
- 不支持的 ISA 不会被调用。
- 可以通过环境变量或测试选项强制选择某个实现。
- benchmark 报告记录实际选择的实现。

如果 benchmark 只写“测试了 `dot_i32`”，但没有记录 dispatch 到 `asm_scalar` 还是其他版本，结果不可解释。即使第一册暂时只有标量实现，也应该让报告养成记录 implementation name 的习惯。

函数指针 dispatch 的基本形态

一个常见形态：

```
using DotFn = std::int64_t (*)(const std::int32_t*,
                             const std::int32_t*,
                             std::int64_t);

DotFn select_dot_impl()
{
    if (cpu_supports_fast_impl()) {
        return asm_dot_i32_fast;
    }
    return asm_dot_i32_scalar;
}
```

公共 wrapper：

```
std::int64_t dot_i32(const std::int32_t* a,
                   const std::int32_t* b,
                   std::int64_t n)
{
    static DotFn fn = select_dot_impl();
    return fn(a, b, n);
}
```

这里公共 ABI 是 `dot_i32`，内部 ABI 是多个 `asm_dot_i32_*`。测试应能分别直接测试内部实现，也能测试 dispatch 后的公共函数。性能报告要说明是否包含 dispatch 开销。若 `dot_i32` 在热循环中反复调用，函数指针间接调用开销可能重要；若每次调用处理大量数据，开销可能可忽略。

dispatch 失败的常见模式

第一，CPU 特性检测错误。程序在不支持某 ISA 的机器上调用了需要该 ISA 的 kernel，导致非法指令异常。测试必须包含 fallback 路径，或在 CI 中至少模拟强制 fallback。

第二，多个实现语义不一致。标量版本允许任意对齐，快速版本要求对齐但 wrapper 没检查；标量版本处理尾部，快速版本假设 `n` 是倍数；某个版本对溢出和舍入处理不同。这类错误会表现为“只有某些机器错”。

第三，benchmark 测错实现。环境变量、CPU 检测、构建选项或链接顺序导致实际运行了 reference 或 fallback，而报告以为运行了快速版本。解决办法是让每个实现返回可记录的名称，或在报

告中保存符号/反汇编证据。

互操作发布前的二进制兼容检查

发布一个包含汇编互操作的库前，应做二进制兼容检查。最小检查包括：

1. 导出符号列表是否只包含公共 ABI。
2. 公共符号名是否与上个版本兼容。
3. 参数块 `sizeof/alignof/offsetof` 是否保持。
4. 动态库依赖是否符合预期。
5. 构建是否在 PIE/非 PIE 或共享库模式下都通过。
6. stripped 二进制是否仍保留必要 unwind 信息。
7. 崩溃时能否用 build-id 找到符号和源码。

可以保存一个符号快照：

```
nm -D --defined-only libfast.so | c++filt > abi-symbols.txt
readelf -d libfast.so > dynamic.txt
readelf -Ws libfast.so > symbols-full.txt
```

下一次发布时与旧快照比较。如果公共符号消失、类型从 function 变成 object、可见性改变、版本脚本变化，都要审查。不是所有差异都是错误，但每个公共 ABI 差异都需要解释。

案例：一次看似优化的 ABI 破坏

假设一个点积 kernel 原本声明为：

```
extern "C" std::int64_t asm_dot_i32(const std::int32_t* a,
                                   const std::int32_t* b,
                                   std::int64_t n);
```

后来为了减少参数，开发者改成参数块：

```
struct DotArgs {
    const std::int32_t* a;
    const std::int32_t* b;
    std::int64_t n;
};

extern "C" std::int64_t asm_dot_i32(const DotArgs* args);
```

如果公共符号名仍叫 `asm_dot_i32`，旧调用者会继续按旧 ABI 把 `a` 放在 `rdi`、`b` 放在 `rsi`、`n` 放在 `rdx`。新实现却把 `rdi` 当作 `DotArgs*`，于是会从 `a` 指向的数据开头读取三个字段。返回值可能是垃圾，甚至越界崩溃。

这个错误无法靠“新代码测试通过”发现，因为新测试使用新声明。必须有 ABI 兼容测试或符号版本策略。正确做法是新增 `asm_dot_i32_v2`，保留旧 `asm_dot_i32_v1`，或者明确这是破坏性版本并更改公共库版本。

互操作文档的生命周期

接口文档不是写完一次就结束。每次修改汇编实现或 wrapper，都要检查文档是否仍然真实。常见文档腐烂包括：

- 注释说不修改 callee-saved，但新版本用了 `rbx`。
- 注释说任意对齐，但新版本使用对齐 load。
- 注释说 `n>=0`，wrapper 实际允许负数并转换成大无符号。
- 注释说 leaf function，但新版本调用 helper。
- 注释说不会访问全局状态，但 dispatch 读取环境变量。

可以把文档检查纳入 review checklist。每次改汇编，都问：接口注释、C++ 声明、测试、反汇编审计和 benchmark 报告是否同步更新？底层代码最怕“注释仍然描述旧世界”。错误注释比没有

注释更危险，因为它会给 reviewer 错误信心。

核心理念

互操作代码的维护质量取决于四个同步：声明和实现同步，文档和实现同步，测试和前置条件同步，benchmark 和实际 dispatch 同步。

综合案例：从 C++ wrapper 到汇编 kernel 的完整边界

设计一个内部点积接口：

```
std::int64_t dot_i32(std::span<const std::int32_t> a,
                   std::span<const std::int32_t> b);
```

公共 wrapper 应先检查：

```
if (a.size() != b.size()) {
    throw std::invalid_argument("size mismatch");
}
if (a.empty()) {
    return 0;
}
```

然后调用内部 C ABI kernel：

```
extern "C" std::int64_t asm_dot_i32_kernel(const std::int32_t* a,
                                         const std::int32_t* b,
                                         std::int64_t n);
```

这个设计把职责分开。wrapper 处理 C++ 类型、长度一致、空输入、异常和未来 dispatch；汇编 kernel 只处理已经验证的裸指针和长度。kernel 文档写：

```
Preconditions:
    a and b point to at least n int32 elements
    n > 0
    input ranges do not need special alignment
    function does not allocate, throw, or call back into C++

ABI:
    rdi = a
    rsi = b
    rdx = n
    rax = int64 result
```

测试分三层。第一层测试 wrapper：长度不等、空输入、正常输入、异常路径。第二层测试 kernel：小 n、随机数据、负数、大值、寄存器保存、栈对齐。第三层测试 dispatch：强制 reference、强制 asm、默认自动选择，并记录实际实现名。

benchmark 也分两种。若要测公共 API，就包含 wrapper 和 dispatch 开销；若要测 kernel，就直接调用 `asm_dot_i32_kernel` 或一个薄 C wrapper。报告必须写清测的是哪一层。两者都合理，但不能混淆。

自测清单：互操作工程是否合格

完成本章后，应能回答：

1. 公共 C++ API 和内部汇编 ABI 为什么应分层。
2. `extern "C"` 改变什么，不改变什么。
3. 为什么参数块布局需要 `static_assert`。
4. leaf 和 non-leaf 汇编函数的栈责任有什么不同。
5. inline asm 的 operands、clobbers、volatile 分别解决什么问题。
6. "memory" clobber 为什么不是硬件 fence。
7. sanitizer 能覆盖 wrapper 的哪些问题，覆盖不了裸汇编内部哪些问题。
8. dispatch 选择实际实现时，benchmark 为什么必须记录实现名。

9. 公共 ABI 发生参数变化时为什么需要版本化。
10. 上线前二进制审计至少要保存哪些证据。

如果互操作代码只有一个能跑的样例，没有 ABI 注释、没有边界测试、没有反汇编审计、没有版本策略，那么它还只是实验代码，不是可维护工程代码。

深入讲解：为什么 wrapper 不是性能敌人

很多人写汇编 kernel 时想绕过 wrapper，认为 wrapper 会降低性能。这个判断过于简单。wrapper 的成本要和 kernel 工作量比较，也要看它是否在热循环内。对于处理大量数据的 kernel，一次 wrapper 检查和 dispatch 成本通常可以忽略；对于纳秒级小函数，wrapper 成本可能重要，需要单独测量。不能一概而论。

wrapper 的价值在于把复杂性移出汇编。它可以检查长度、处理空输入、选择实现、转换类型、记录实现名、处理异常和错误码。这样汇编 kernel 可以保持简单、稳定、易审计。若为了省几条 C++ 检查，把所有边界逻辑塞进汇编，通常会让正确性和维护成本恶化。

高质量做法是分层测量。测公共 API，了解真实调用成本；测内部 kernel，了解核心计算成本；测 dispatch，了解选择实现的开销。报告把三者分开，工程上就能决定是否需要 fast path、是否缓存函数指针、是否为小输入使用纯 C++。

深入讲解：互操作失败常常是“边界无人负责”

互操作 bug 的根因常常不是某个人写错一条指令，而是边界责任没有分配。谁检查 $n \geq 0$ ？谁保证指针对齐？谁处理尾部？谁保存 callee-saved？谁设置 `al` 调用变参函数？谁保证参数块 offset？谁记录 dispatch 实现？如果文档没有答案，代码就会靠隐含假设运行。

边界责任应写成明确句子：

```
Wrapper responsibility:
    validate public inputs
    handle n == 0
    select implementation

Kernel responsibility:
    compute result for n > 0
    preserve callee-saved registers
    return int64 sum in rax

Caller responsibility:
    do not call internal symbol directly
```

这类文字看起来简单，却能阻止很多事故。底层工程中，模糊责任比复杂指令更危险。

课堂讲解：互操作代码要把“可替换性”设计进去

一个汇编 kernel 如果只在当前机器、当前编译器、当前输入上能跑，还不能算工程完成。可维护的互操作接口应允许实现被替换：今天是 C++ reference，明天是标量汇编，后天是 SIMD 版本，某些机器回退到 portable 版本。可替换性要求公共 API 稳定、内部 ABI 明确、测试对所有实现使用同一语义、benchmark 记录实际实现名。

最常见的结构是三层。第一层是 public wrapper，面对 C++ 调用者，使用 `std::span`、容器、错误码或异常表达友好接口。第二层是 checked C ABI，使用裸指针、长度和固定宽度类型，但只在 wrapper 检查后调用。第三层是具体 kernel，可以是 C++、汇编或编译器内置版本。dispatch 只在第二层和第三层之间选择实现，不让调用者直接依赖某个内部符号。

这种结构的好处是，性能优化可以在内部演进，而公共语义保持不变。新增 AVX2 版本，只需要加入实现和 dispatch；发现某个汇编版本在某 CPU 上有问题，可以禁用它；benchmark 可以分别测 public wrapper、dispatch 和 kernel。若一开始就把公共 API 直接绑定到一个汇编符号，后续改动会非常痛苦。

参数块：复杂接口的稳定边界

当参数数量增多，或者需要传递多个指针、长度、stride、标志位和输出位置时，直接增加寄存器参数会让 ABI 难读，也容易超过寄存器通道。参数块是常见选择：C++ 端定义一个标准布局结构体，汇编端接收指向参数块的指针，通过固定 offset 读取字段。

参数块必须被当成二进制接口。字段使用固定宽度类型；避免 C++ 容器、引用、虚函数和非平凡对象；提供 size 或 version 字段；用 `static_assert` 检查 `sizeof`、`alignof` 和关键 `offsetof`；汇编文件中的 offset 常量应来自生成头文件或集中维护，不要手写散落的魔法数字。

参数块还要定义所有权。指针字段指向的内存由谁分配、长度是多少、是否允许重叠、是否要求对齐、kernel 是否会写入、调用后是否仍有效，这些都要写清。参数块本身只解决传参形式，不自动解决对象生命周期和别名问题。

上线前二进制审计

互操作代码合入前，应做一次二进制审计。审计不是为了追求形式，而是确认最终产物确实符合接口文档。第一，检查符号。用 `nm-C` 或 `readelf-Ws` 确认导出符号、可见性和名称。第二，检查调用点。确认 wrapper 调用的是预期内部符号，dispatch 路径记录实现名，错误路径不会误入未支持实现。

第三，检查汇编入口。确认参数寄存器和文档一致，若使用参数块，检查 offset；确认 callee-saved 寄存器保存恢复；确认 leaf/non-leaf 栈对齐；确认 `.note.GNU-stack` 和 section 属性。第四，检查返回路径。所有路径都设置返回值，错误路径返回约定值或错误码，尾调用不破坏栈。第五，检查测试覆盖。边界长度、空输入、非对齐地址、随机数据、错误参数和多实现一致性都应覆盖。

性能审计也要分开。先证明正确，再看速度。若汇编版本声称更快，应保存同一输入下 C++ reference 和汇编 kernel 的 raw data、summary、反汇编和环境。若只测了 public wrapper，要说明 wrapper 和 dispatch 开销包含在内；若只测 kernel，要说明真实调用还有额外开销。二进制审计让“我以为测了汇编”变成“证据显示测了这个符号”。

版本化：接口变化要可追踪

互操作接口变化常见于新增参数、改变长度类型、修改返回错误码、增加对齐要求、改变尾部处理策略、引入新的 CPU 特性。只要调用者和实现可能不同步，就需要版本化。版本化可以是符号名版本，例如 `asm_dot_i32_v2`；可以是参数块版本字段；可以是库版本和头文件版本；也可以是 dispatch 表中的能力记录。

不要在不改名的情况下悄悄改变内部 ABI。即使所有代码在同一个仓库里，也可能有旧目标文件、旧动态库、旧测试插件或用户私下调用内部符号。若改动必须破坏 ABI，应在构建和运行时尽早失败，而不是让旧调用者以错误参数继续运行。比如参数块可以包含 size 字段，kernel 或 wrapper 检查不匹配时返回错误。

版本化还服务 benchmark。历史结果必须知道使用的是哪个实现版本。一个 kernel 从 v1 改到 v2 后，性能趋势图若不标注，会把接口变化和优化变化混在一起。报告中记录实现名和版本，是长期性能数据可解释的前提。

互操作审查清单

对 C++ 与汇编互操作改动，review 至少应问：

1. public API、internal ABI 和具体 kernel 是否分层。
2. 输入合法化由 wrapper 负责还是 kernel 负责，是否写清。
3. 长度、stride、字节数和元素数单位是否明确。
4. 空输入、尾部、非对齐地址和重叠区间如何处理。
5. 参数宽度、返回值宽度和 signedness 是否与声明一致。
6. callee-saved、栈对齐、红区、调用 helper 的规则是否满足。
7. 参数块 layout 是否有 `static_assert`。
8. sanitizer、单元测试和随机测试覆盖了哪些层。
9. benchmark 是否区分 wrapper、dispatch 和 kernel。

10. 接口变化是否版本化，历史结果是否能区分实现版本。

这份清单的核心是责任明确。互操作工程失败时，最难修的往往不是某条指令，而是每一层都以为另一层已经检查了条件。

互操作代码的演进路线

一个汇编优化最好按阶段演进。第一阶段写 C++ reference，定义语义和测试。第二阶段写最小汇编 kernel，只覆盖最简单合同。第三阶段加入 wrapper，处理公共输入和错误路径。第四阶段加入 benchmark，区分 public API 和 kernel。第五阶段再考虑多实现 dispatch、CPU 特性和版本化。跳过前面阶段直接写复杂汇编，风险很高。

每个阶段都有退出标准。reference 要正确且覆盖边界；kernel 要通过同一测试并遵守 ABI；wrapper 要拒绝非法输入；benchmark 要证明测到的是目标实现；dispatch 要记录实现名并可强制选择。阶段化能让复杂互操作工程变得可审查。

互操作文档应靠近代码

ABI 注释、参数块布局、前置条件、寄存器使用、clobber 说明、测试命令和 benchmark 命令，应尽量靠近代码或在同一目录文档中。不要只放在聊天记录或远处文档。底层接口一旦修改，远处文档最容易过期。

好的目录可以包含 README.md、abi.md、tests/、bench/、objdump/。README 说明公共 API，abi 文档说明内部合同，tests 验证正确性，bench 验证性能，objdump 保存关键二进制证据。这样互操作代码就不是孤立汇编文件，而是一个可维护单元。

综合项目：从 reference 到汇编 kernel

本章最合适的综合项目，是为一个小函数建立完整互操作链路。以 dot_i32 为例，先写 C++ reference，明确整数溢出策略和返回类型；再写 public wrapper，接收 std::span 并检查长度；再写 internal C ABI 声明，使用裸指针和长度；再写汇编 kernel；最后写测试、反汇编审计和 benchmark。

项目报告要说明每一层职责。reference 定义语义；wrapper 处理公共输入；kernel 假设输入合法并专注计算；测试比较 reference 和 kernel；benchmark 分别测 wrapper 和 kernel。若任何一层职责混乱，例如 kernel 既处理异常又做 dispatch 又计算核心循环，维护成本就会上升。

这个综合项目不需要 SIMD。标量汇编已经足够训练 ABI、寄存器、栈、参数、返回值、整数宽度、正确性和测量。等第二册进入 SIMD 时，读者只需要替换 kernel 实现，而不必重新发明工程边界。

互操作失败后的回滚设计

高性能实现应有回滚路径。若某个汇编版本在特定机器上失败，项目应该能强制使用 reference 或 portable 实现；若 dispatch 判断有 bug，应该能通过环境变量或配置禁用；若 benchmark 发现性能回归，应该能保留旧实现对比。没有回滚路径的底层优化风险很高。

回滚路径本身也要测试。不要只测试最快实现，也要测试 reference、portable、asm 和 dispatch override。报告中记录实际实现名，能让线上问题快速定位到某个实现。可回滚设计不是保守，而是让优化可以安全演进。

互操作中的调试策略

互操作 bug 出现时，要先判断错误发生在边界哪一侧。若 C++ wrapper 已经拒绝非法输入，问题可能在 kernel；若 kernel 单独测试正确，问题可能在 wrapper、dispatch 或声明不一致；若只有动态库环境失败，问题可能在符号和加载路径；若只有优化构建失败，可能是 ABI、clobber、未定义行为或内联改变了边界。

调试时建议保留两个入口：一个直接调用 internal C ABI kernel 的测试入口，一个调用 public wrapper 的测试入口。前者帮助定位 kernel 语义，后者帮助定位真实 API 行为。再加一个强制选择实现的开关，可以排除 dispatch 影响。没有这些入口，互操作问题很容易在多层之间来回推诿。

互操作调试也要保存反汇编。源码声明正确不代表最终调用点正确；汇编看起来正确不代表调用者按同一签名调用。调用点、入口和返回点三处证据合在一起，才能定位边界问题。若只看汇编

文件本身，可能错过 C++ 编译器在调用点做的参数扩展、栈调整、内联和常量传播。

互操作代码的毕业标准

本章的毕业标准不是“C++ 能调用汇编并打印正确结果”。那只是最小演示。合格互操作代码应有稳定 public API、明确 internal ABI、边界输入测试、寄存器和栈审计、参数块布局断言、可选择 reference 实现、可复现 benchmark 和二进制 artifact。缺少任何一项，都说明它还不适合承担长期维护责任。

读者可以把毕业标准写进 PR 模板。新增或修改汇编 kernel 时，作者必须说明 ABI 是否变化、测试是否覆盖边界、反汇编是否审计、benchmark 是否区分 wrapper 和 kernel、是否有回滚路径。这样互操作不再依赖个人记忆，而进入团队流程。

毕业标准还要求能解释失败。若汇编版本算错，报告要能指出是输入合同不满足、寄存器保存错误、栈对齐错误、整数宽度错误，还是 wrapper 和 kernel 声明不一致。若只能说“汇编有问题”，说明边界还没有拆清楚。互操作工程的成熟标志，是能把故障定位到明确责任层。

互操作和团队分工

在团队中，写 public wrapper、写汇编 kernel、写测试、写 benchmark、做二进制审计的人可能不是同一个。接口文档必须让这些角色可以独立工作。wrapper 作者需要知道哪些输入要拒绝；汇编作者需要知道寄存器和内存合同；测试作者需要知道边界值；benchmark 作者需要知道测 public API 还是 kernel；reviewer 需要知道哪些证据必须保存。

若文档只写“调用汇编求和”，团队就会依赖口头约定。口头约定在人员变化、时间推移和版本迭代中最容易丢失。把合同写成文件、测试和 artifact，是为了让项目不依赖某个人一直在场。底层代码越接近硬件，越需要这种显式协作方式。

实验：从零搭建 C++ 调汇编工程

目标：建立一个最小但规范的 .S+C++CMake 工程，并用工具证明符号、调用和返回值正确。

创建目录：

```
mkdir -p labs/ch14_interop/lab14_1_minimal
cd labs/ch14_interop/lab14_1_minimal
```

文件：

```
lab14_1_minimal/
  CMakeLists.txt
  main.cpp
  asm_arith.S
```

任务：

1. 在 C++ 中声明 3 个函数：

```
extern "C" long asm_add2(long a, long b);
extern "C" long asm_sub2(long a, long b);
extern "C" long asm_mix4(long a, long b, long c, long d);
```

2. 在 asm_arith.S 中实现它们。
3. asm_mix4 的语义为 $(a+b)*3-(c-d)$ ，可以使用 lea、add、sub。
4. 写至少 20 个确定性测试，覆盖正数、负数、零和较大值。
5. 构建并运行。
6. 用 nm-C 证明符号存在。
7. 用 objdump-drwC-Mintel 找到 3 个函数的机器码。

推荐命令：

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build build -j
./build/lab14_1_minimal
nm -C build/lab14_1_minimal | rg 'asm_'
objdump -drwC -Mintel build/lab14_1_minimal > lab14_1.objdump.txt
```

交付物：

1. 源码。
2. 运行输出。
3. nm 输出中 3 个汇编符号的行。
4. 每个函数的反汇编。
5. 一张表：源代码参数、ABI 寄存器、汇编使用位置。

实验：符号、mangling 和 relocation 审计

目标：理解 C++ 符号名、extern"C" 和链接器重定位。
创建两个版本：

```
with_c_linkage.cpp
without_c_linkage.cpp
caller.cpp
```

with_c_linkage.cpp:

```
extern "C" int add_c(int a, int b)
{
    return a + b;
}
```

without_c_linkage.cpp:

```
int add_cpp(int a, int b)
{
    return a + b;
}
```

caller.cpp:

```
extern "C" int add_c(int, int);
int add_cpp(int, int);

extern "C" int call_both(int x)
{
    return add_c(x, 1) + add_cpp(x, 2);
}
```

任务：

1. 分别编译成 .o。
2. 用 nm 和 nm-C 比较符号名。
3. 用 objdump-drwC-Mintelcaller.o 查看未链接目标文件。
4. 用 readelf-rcaller.o 查看 relocation。
5. 链接成可执行文件后，再看最终 call 的相对位移。

命令：

```
clang++ -std=c++20 -O2 -c with_c_linkage.cpp -o with_c_linkage.o
clang++ -std=c++20 -O2 -c without_c_linkage.cpp -o without_c_linkage.o
clang++ -std=c++20 -O2 -c caller.cpp -o caller.o
nm caller.o
nm -C caller.o
objdump -drwC -Mintel caller.o > caller.objdump.txt
readelf -r caller.o > caller.reloc.txt
```

交付物：

1. `add_c` 和 `add_cpp` 的原始符号名。
2. `call_both` 中两个 `call` 的 relocation 记录。
3. 解释 `R_X86_64_PLT32` 或你机器上出现的 relocation 类型。
4. 链接后任选一个 `call`，用 `target-next_rip` 计算其 `rel32`。
5. 一段 500 字总结：为什么目标文件反汇编不能只看机器码字节。

实验：ABI 安全的 8 参数汇编函数

目标：实现并严格测试一个包含栈参数的汇编函数。

函数签名：

```
extern "C" long asm_weighted_sum8(long a0, long a1, long a2, long a3,
                                long a4, long a5, long a6, long a7);
```

语义：

```
return a0*1 + a1*2 + a2*3 + a3*4
       + a4*5 + a5*6 + a6*7 + a7*8
```

任务：

1. 写 ABI 参数位置表。
2. 实现汇编函数。第 7、8 个参数必须从栈读取。
3. 写 C++ reference 函数。
4. 写 100000 轮随机测试，限制输入范围避免 signed overflow。
5. 用 GDB 在函数入口断点，记录 `rsp`、`[rsp]`、`[rsp+8]`、`[rsp+16]`。
6. 用 `objdump` 标注每个参数在哪里被使用。

提示：可以用 `lea` 组合小常数乘法，但要保证结果语义清楚。例如 `x*3` 可以写成：

```
lea rax, [rdi + rdi*2]
```

交付物：

1. 参数映射表。
2. 完整汇编。
3. 测试源码和运行结果。
4. GDB 栈截图或文字记录。
5. 解释为什么 `a6` 是 `[rsp+8]` 而不是 `[rsp]`。

实验：汇编调用 C++ helper

目标：写一个 non-leaf 汇编函数，正确处理 callee-saved 寄存器和栈对齐。

C++ helper：

```
extern "C" long cpp_mul_add(long x, long y, long z)
{
    return x * y + z;
}
```

汇编函数签名：

```
extern "C" long asm_call_helper(long x, long y, long z, long bias);
```

语义：

```
return cpp_mul_add(x, y, z) + bias
```

任务：

1. 先故意写一个错误版本：把 `bias` 放进 `rbx`，调用 `helper` 后不恢复。
2. 构造 C++ 调用者，让调用者在调用前后有一个长期存活变量，观察错误是否暴露。
3. 修复 `rbx` 保存恢复。
4. 检查调用 `cpp_mul_add` 前 `rsp` 是否 16 字节对齐。
5. 用 GDB 单步过 `call`，记录 `rsp` 和返回地址。

交付物：

1. 错误版本和修复版本的汇编。
2. 为什么错误版本违反 ABI。
3. 栈对齐计算过程。
4. GDB 记录。
5. 测试结果。

实验：inline asm 约束修复

目标：理解 inline asm 不是文本替换，而是优化器合同。
给定错误代码：

```
#include <stdint>

std::int64_t broken_add(std::int64_t a, std::int64_t b)
{
    std::int64_t result = a;
    asm("add %1, %0" : : "r"(result), "r"(b));
    return result;
}

void broken_store(std::int64_t* p, std::int64_t value)
{
    asm volatile("mov %1, (%0)" : "r"(p), "r"(value));
}
```

任务：

1. 在 `-O0` 和 `-O3` 下分别编译运行，记录现象。
2. 查看反汇编，解释为什么错误版本可能“偶尔看起来能跑”。
3. 修复 `broken_add`，正确使用读写输出和 `"cc"` clobber。
4. 修复 `broken_store`，正确描述内存写和 `"memory"` clobber。
5. 写测试证明修复版本在 `-O3` 下稳定正确。

交付物：

1. 错误版本和修复版本源码。
2. `-O0`、`-O3` 反汇编对比。
3. 每个 constraint 的解释。
4. 说明 `volatile`、`"memory"`、`"cc"` 各自解决什么问题，不能解决什么问题。

实验：标量点积 kernel 的互操作审计

目标：实现一个接近算子 kernel 形态的标量点积，并建立 correctness report。
函数：

```
extern "C" std::int64_t asm_dot_i32(const std::int32_t* a,
```



```
const std::int32_t* b,
std::int64_t n);
```

任务：

1. 写 C++ reference 实现。
2. 写汇编实现。
3. 测试 `n=0,1,2,15,16,17,1024`。
4. 随机生成数组，做 10000 轮测试。
5. 用 `objdump` 标注循环基本块、回边、数组地址公式。
6. 用 `perfstat` 初步测量运行时间、instructions、cycles；如果权限不足，记录错误信息并使用 `/usr/bin/time`。

交付物：

1. 源码和构建脚本。
2. 正确性测试输出。
3. 循环反汇编标注。
4. 地址公式说明。
5. 初步性能表。
6. 说明为什么这个版本还不是高性能实现，列出至少 5 个后续优化方向。

实验：C++ wrapper 与参数块

目标：把复杂 C++ 接口和简单汇编 kernel 分层，并用静态断言固定参数块布局。

任务：

1. 定义一个 `DotConfig` 参数块，包含两个 `const std::int32_t*`、长度、两个 stride 和 bias。
2. 在 C++ 头文件中为 `sizeof`、`alignof` 和每个字段 `offsetof` 写 `static_assert`。
3. 编写公共 C++ wrapper，接收更友好的参数，检查长度、空指针和 stride，再填充 `DotConfig`。
4. 编写 `extern "C"` 汇编 kernel，只接收 `const DotConfig*`。
5. 用确定性测试和随机测试比较 C++ reference 与汇编结果。
6. 用 `objdump` 标出汇编中每个字段偏移对应的 C++ 字段。

交付物：

- C++ 头文件中的布局断言。
- wrapper 代码和汇编 kernel。
- 一张字段偏移表。
- 测试结果和反汇编证据。
- 说明参数块相比直接多参数传递的优点和代价。

实验：栈对齐与 callee-saved 测试

目标：用测试暴露 non-leaf 汇编函数的栈对齐和寄存器保存错误。

任务：

1. 写一个 C++ helper，返回入口处 `rsp` 是否满足 System V AMD64 的入口余数规则。
2. 写一个正确汇编函数，在调用 helper 前保证 `rsp` 16 字节对齐。
3. 写一个错误版本，故意不对齐栈。
4. 写一个测试专用汇编包装器，给 `rbx`、`r12`、`r13`、`r14`、`r15` 设置哨兵，调用被

测函数后检查是否保持。

5. 将正确版本和错误版本都放进测试，确认测试能失败也能通过。

交付物：

- 栈余数检查 helper。
- 保存寄存器测试包装器。
- 正确版本和错误版本的汇编。
- 测试输出。
- 一张栈变化图，说明每个 `call` 前的 `rsp` 对齐状态。

本章作业

习题与作业

作业 1：接口契约报告

选择你在实验中写的任意 3 个汇编函数，为每个函数写一份接口契约报告，必须包含：

- C++ 声明。
- 链接符号名。
- 参数位置表。
- 返回值位置。
- 修改的 caller-saved 寄存器。
- 是否修改 callee-saved 寄存器；如果修改，如何保存恢复。
- 是否调用其他函数；如果调用，如何保证栈对齐。
- 前置条件和未定义行为边界。

作业 2：修复破坏 ABI 的汇编

下面函数被 C++ 调用：

```
.intel_syntax noprefix
.text
.globl broken_kernel
broken_kernel:
    mov r12, rdi
    mov r13, rsi
    call helper
    add rax, r12
    add rax, r13
    ret
```

要求：

1. 找出所有 ABI 问题。
2. 说明 `r12`、`r13` 属于 caller-saved 还是 callee-saved。
3. 修复保存恢复。
4. 修复栈对齐。
5. 给出完整汇编。
6. 用栈布局图证明你的修复正确。

作业 3：重定位计算

给定未链接目标文件片段：

```
0000000000000000 <foo>:
 0: e8 00 00 00 00    call 5 <foo+0x5>
 1: R_X86_64_PLT32 bar-0x4
 5: c3               ret
```

链接后：

```
foo address = 0x401100
bar address = 0x401180
```

要求：

1. 写出 `call` 指令地址。
2. 写出 next `rip`。
3. 计算 `rel32`。
4. 写出小端序 4 字节。
5. 解释为什么 relocation addend 常见为 `-4`。

作业 4: inline asm 审计

阅读下面代码：

```
std::uint64_t rdtsc_bad()
{
    std::uint32_t lo = 0;
    std::uint32_t hi = 0;
    asm volatile("rdtsc");
    return (static_cast<std::uint64_t>(hi) << 32) | lo;
}
```

要求：

1. 解释为什么它错误。
2. 写出正确输出约束，让 `eax` 和 `edx` 分别写入 `lo`、`hi`。
3. 说明 `rdtsc` 的乱序风险。
4. 查阅本机反汇编，确认约束是否生成预期寄存器。
5. 说明为什么 benchmark 不能只靠裸 `rdtsc`。

作业：inline asm 约束深度报告

编写三个小函数，分别演示读写操作数、matching constraint 和 early-clobber 输出。每个函数都要有一个错误版本和一个修复版本。报告不少于 2200 字，必须包含：

- 每个 asm 模板的输入、输出、clobber 表。
- 为什么 `"+r"` 表示同一操作数既读又写。
- `"0"` 这类 matching constraint 如何把输入绑定到输出位置。
- early-clobber 缺失时，输出和输入复用寄存器会怎样破坏值。
- 固定寄存器约束和普通 `"r"` 约束的取舍。
- `-00` 与 `-03` 反汇编对比，以及为什么错误版本可能只在高优化或高寄存器压力下暴露。

作业：符号可见性和 ABI 稳定性审计

构建一个包含静态库或动态库的小工程，库中至少有两个公共汇编入口和两个内部汇编 helper。报告不少于 1800 字，要求：

- 用 `nm` 和 `readelf-WS` 列出普通符号表和动态符号表。
- 区分公共 ABI 符号和内部标签/helper。
- 故意制造一次重复定义，记录链接器错误并解释原因。
- 说明如果某个公共函数参数列表改变，为什么应该新增版本符号或保持 wrapper 兼容。
- 检查是否存在意外导出的内部符号。

作业：展开信息和调试回溯

写两个 non-leaf 汇编函数：一个只有普通 push/pop，没有 CFI；另一个补充基本 CFI。它们都调用一个 C++ helper，再从 helper 内部触发一次可控崩溃或断点。报告不少于

1800 字，要求：

- 用 GDB 比较两者 backtrace 差异。
- 解释 `.cfi_startproc`、`.cfi_def_cfa_offset`、`.cfi_offset` 的作用。
- 说明为什么异常不应随意穿过裸汇编帧。
- 分析 profiler 或崩溃报告为什么需要展开信息。
- 给出你建议的工程规则：哪些汇编函数必须写 CFI，哪些 leaf kernel 可以暂时不写。

作业：wrapper 分层设计

选择一个数组或矩阵小 kernel，设计公共 C++ API、C++ wrapper 和汇编 kernel 三层接口。报告必须包含：

- 公共 API 的类型和错误处理策略。
- wrapper 负责检查哪些前置条件。
- 汇编 kernel 的 `extern"C"` 声明。
- 汇编 kernel 的 ABI 参数表。
- wrapper 测试和 kernel 测试如何分开。
- 为什么不让汇编直接接收复杂 C++ 对象。

作业：参数块布局审计

设计一个至少包含 5 个字段的平凡参数块，并让汇编函数通过指针读取字段。提交：

- C++ 结构体定义。
- `static_assert` 布局检查。
- 汇编偏移注释。
- `objdump` 中对应 load 指令。
- 一段说明：如果中间插入一个新字段，哪些检查会失败，哪些错误可能在没有检查时静默发生。

作业：平台 ABI 对比报告

选择 Linux x86-64 System V 和另一个 ABI，例如 Windows x64 或 AArch64 Linux，写一份对比报告。要求至少覆盖：

- 前几个整数参数寄存器。
- 返回值位置。
- 栈对齐规则。
- callee-saved 寄存器。
- 结构体返回的差异。
- 为什么同一个 `.S` 文件不能不加审计地跨 ABI 复用。

作业 5：互操作小项目

完成一个小项目：

```
interop_project/
  CMakeLists.txt
  include/kernels.hpp
  src/main.cpp
  src/reference.cpp
  src/kernels.S
  tests/test_kernels.cpp
  report.md
```

功能：

- `asm_sum8`：8 参数加权求和。
- `asm_make_pair`：返回两个 64 位字段的小结构体。

- `asm_dot_i32`: 标量点积。
- `asm_call_helper`: 汇编调用 C++ helper。

质量要求:

1. 每个函数都有 reference 实现。
2. 每个函数都有确定性测试和随机测试。
3. 所有汇编函数都有接口注释。
4. 报告中包含 `nm`、`readelf-r`、`objdump` 证据。
5. 报告中至少分析 2 个栈布局。
6. 报告中至少解释 1 个 relocation。
7. 报告中明确列出 sanitizer 能覆盖什么、不能覆盖什么。

评分标准

本章作业按下面标准评价:

1. 能否从 C++ 签名正确推导 ABI 参数和返回值位置。
2. 汇编符号、`.type`、`.size`、section 是否规范。
3. 是否正确处理 caller-saved、callee-saved 和栈对齐。
4. 是否能用 `nm`、`readelf`、`objdump`、GDB 证明结论。
5. inline asm 是否准确描述输入、输出和 clobbers。
6. 测试是否覆盖边界值、随机值和跨优化级别行为。
7. 是否考虑 wrapper 分层、参数块布局断言、符号可见性、ABI 演进、CI 检查和展开信息。
8. 报告是否区分语言层语义、ABI 层契约、目标文件层机制和 CPU 执行行为。

本章小结

本章把 System V AMD64 ABI 放进了真实工程:

```
C++ declaration:
    type, language linkage, call site generation

assembly source:
    symbol definition, sections, prologue/epilogue, instructions

ABI:
    parameter locations, return locations, register ownership, stack alignment

object file:
    symbols, sections, relocations

tools:
    nm, readelf, objdump, gdb, perf

quality:
    deterministic tests, randomized tests, binary-interface report
```

你现在应该能够从一个 C++ 函数声明出发, 写出 ABI 映射表, 再写出手写汇编实现, 并用工具证明它真的按预期链接和运行。你也应该开始对 inline asm 保持谨慎: 它不是简单的“在 C++ 里写汇编”, 而是和优化器签订一份非常精确的合同。

后面的章节会进入测量; 第二册会进入编译器优化、微架构和 SIMD。本章建立的互操作能力会反复使用: 你会写小 kernel、反汇编审计、测量性能、解释瓶颈, 再逐步把标量代码推进到 AVX、FMA、VNNI 和 AI 算子优化。

第零部分

实验原理：可信性能测量

可信性能测量基础

问题与目标

从这一章开始，本书进入性能工程部分。许多性能优化失败，并不是因为改动不够激进，而是因为一开始没有弄清“测到的到底是什么”。如果测量方法不可信，后续结论都可能建立在错误证据上：你以为某个优化快了 30%，实际可能是编译器删掉了循环；你以为某个版本变慢了，实际可能是 CPU 降频、cache 状态、OS 调度、页错误、随机输入或 I/O 噪声在影响。

本章建立性能测量的底层心智模型。你需要从一开始就把 benchmark 看成一次受控实验，而不是把计时器放在代码两端的临时动作。

一个可信 benchmark 至少要回答：

- 被测对象是什么？是一个函数、一个循环、一个完整程序，还是一次系统调用？
- 输入是什么？规模、分布、对齐、cache 状态、随机种子是否明确？
- 计时区域在哪里？是否混入了初始化、分配、I/O、随机数生成、校验、日志输出？
- 编译器是否可能把被测代码删除、合并、常量折叠或移动？
- 结果是否先证明正确，再讨论性能？
- 测量重复了多少次？报告的是最小值、均值、中位数还是分位数？
- 环境是否记录？CPU 型号、频率策略、编译器版本、编译参数、操作系统状态是否可复现？

本章先讲最核心的机制，下一章再系统讨论 benchmark suite 的设计。统计、噪声分析和 PMU 细节会在后续性能专题中继续展开。当前阶段最重要的能力是：看到一个性能数字时，能判断它是否有资格作为工程证据。

核心思想

性能测量不是获得一个时间数字，而是建立一条可复核的证据链：正确性、被测工作量、时间源、控制变量、重复测量、统计口径、二进制审计和环境记录。

测量是一种推断

性能实验容易被误解成“机器给了一个客观数字”。实际上，计时器只告诉你两个读数之间经过了多长时间；你真正想知道的是某个 kernel 在某个 workload、某个编译配置、某个硬件状态下的行为。中间存在一段推断过程：你假设计时区域只包含目标工作，假设结果没有被优化器删除，假设输入代表要研究的场景，假设噪声不会改变结论，假设统计口径能表达真实差异。任何一个假设不成立，数字就会失去解释力。

因此，性能测量更像科学实验，而不是日志打印。实验要有问题陈述、控制变量、可复现步骤、原始数据、误差分析和结论边界。一个成熟工程师看到“版本 A 比版本 B 快 12%”时，不会立刻接受，而会追问：测了哪些输入规模？重复多少次？是 warm cache 还是 cold cache？两个版本是否用同一编译参数？结果是否正确？汇编是否真的不同？差异是否大于测量噪声？是否只在某台机器上成立？

这种审慎不是保守，而是必要。性能工作特别容易出现虚假进步：删掉了计算、改变了输入、污染了计时区域、误用了时间源、比较了不同编译选项、用单次样本代表整体、把系统噪声当成算法差异。只要其中一种发生，优化方向就会被带偏。

心智模型

一次性能结论至少包含三层：

- 观察：计时器、PMU 或日志记录到的原始数据。
- 解释：这些数据和源码、汇编、输入、硬件状态之间的关系。

- 结论：在明确边界内，哪个版本更快、为什么可能更快、还不能证明什么。

没有解释层，观察不能自动变成结论。

测量栈：从源码到机器再到数字

一个 benchmark 的数字不是只由被测函数决定，而是由整条测量栈共同产生。

最上层是问题定义。你究竟要测端到端程序、一个 API 调用、一个 kernel、一个循环，还是一条指令序列？不同层级需要不同实验边界。端到端测量可以包含解析、分配、I/O 和调度；kernel 测量通常要把这些赶出计时区域。

第二层是源码和语义。C++ 代码是否有未定义行为？reference 是否可靠？浮点误差是否允许？输入是否触发异常路径？这些都决定“正确结果”是什么。性能优化不能脱离语义，因为错误结果没有性能意义。

第三层是编译器。优化器可能内联、向量化、展开循环、删除死代码、常量折叠、重排内存访问、替换库调用。测量对象不是源码文本本身，而是编译器生成的机器码。没有二进制审计，就不知道实验对象是否已经变形。

第四层是硬件和操作系统。CPU 频率、缓存状态、分支预测、TLB、NUMA、SMT、调度、中断、页错误、后台进程都会影响时间。某些因素可以控制，某些因素只能记录和用统计方法吸收。

第五层是计时和统计。时间源有开销和分辨率；样本有波动和离群点；不同统计量表达不同问题。最终报告的一个数字，是一系列选择后的摘要，不是原始事实本身。

核心思想

benchmark 的最小闭环是：源码语义正确，二进制对象正确，运行环境记录，计时区域准确，样本统计透明。缺少任一环，结论都要降级。

先定义问题，再选择测量方法

很多性能实验从一开始就错了，因为问题没有定义清楚。程序员打开计时器，跑一段代码，得到一个数字，然后才开始解释这个数字。正确顺序应该反过来：先写出你要回答的问题，再设计 benchmark。问题不同，计时区域、输入、指标、统计口径和环境控制都会不同。

例如“这个 API 调用平均需要多久”和“这个内层循环每秒能处理多少元素”不是同一个问题。前者可能应该包含参数检查、调度、内存分配、错误路径和锁；后者通常要把这些移出计时区域，只测稳定热路径。再例如“用户第一次请求有多慢”和“服务运行十分钟后的稳态吞吐”也不是同一个问题。第一次请求可能包含动态链接、JIT、缓存填充、页错误和初始化；稳态吞吐则更关心持续负载下的 CPU、内存和调度。

开始写 benchmark 前，建议把问题写成一整句完整的话：

```
On machine M, with compiler C and flags F,
for workload W, compare implementation A and B
on metric X, under measurement protocol P.
```

把这句话翻译成中文就是：在哪台机器、哪个编译器、哪些输入、哪两个实现、哪个指标、什么测量流程下比较。只要其中某个位置空着，后面报告就会模糊。比如没有 workload，性能数字不能推广；没有 metric，GB/s、ns/op、p95 延迟可能混在一起；没有 protocol，别人不知道你是否 warmup、是否重复、是否保存原始样本。

延迟、吞吐和容量不是一个指标

延迟关心单次操作需要多久，吞吐关心单位时间能完成多少工作，容量关心系统在达到某个质量目标前能承受多少负载。单线程微基准常报告 ns/op 或 cycles/element；批处理算子常报告 GB/s 或 GFLOP/s；服务端系统常报告 QPS、median latency、p95/p99 latency 和错误率。一个优化可能提高吞吐但增加尾延迟，也可能降低单次延迟但占用更多内存带宽，导致并发容量下降。

因此，指标必须和目标一致。若你优化一个内存 copy kernel，GB/s 是自然指标；若你优化哈希表查询，ns/query 和 cache miss 可能更有意义；若你优化编译器前端，文件级吞吐、诊断质量和内存峰值都可能重要；若你优化 AI 推理，tokens/s、batch latency、首 token 延迟和显存占用是不同问题。不要把容易测的指标误当成真正关心的指标。

微基准、组件基准和端到端基准

微基准只测一个很小的 kernel 或函数，优点是噪声少、定位清楚、容易反汇编审计；缺点是可能脱离真实调用环境。组件基准测一个模块或 API，能包含更多实际开销；缺点是解释更复杂。端到端基准测完整程序或服务路径，最接近用户体验；缺点是变量很多，难以归因。

三种基准没有高低之分。微基准适合回答“这个循环是否因向量化变快”；组件基准适合回答“这个 parser 在真实输入文件上是否更快”；端到端基准适合回答“用户请求是否变快”。一个成熟性能工作流常常三者都用：先用端到端发现问题，再用组件基准缩小范围，最后用微基准验证某个机制，改完后再回到端到端确认收益没有消失。

常见误区

不要用微基准结论直接替代端到端结论。微基准能证明某个机制在受控条件下有效，但真实系统还可能被调度、内存、锁、I/O、数据分布和调用频率重新主导。

为什么计时不等于 benchmark

先看一个常见错误：

```
#include <chrono>
#include <stdint>
#include <stdio>

int main()
{
    auto begin = std::chrono::high_resolution_clock::now();

    std::uint64_t sum = 0;
    for (std::uint64_t i = 0; i < 1000000000ULL; ++i) {
        sum += i;
    }

    auto end = std::chrono::high_resolution_clock::now();
    auto ns = std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin);
    std::printf("%lld\n", static_cast<long long>(ns.count()));
}
```

这个程序表面上在测 10 亿次加法。实际上，它可能没有测到这些加法。优化器可以推导：

```
sum = 0 + 1 + 2 + ... + 999999999
```

如果 `sum` 没有被观察，甚至整个循环都可能被删除。就算打印了 `sum`，编译器也可能用公式替代循环。因为从 C++ 抽象机视角看，只要可观察行为相同，编译器可以自由改写。

另一个错误：

```
auto begin = clock_now();
std::vector<float> x(n);
fill_random(x);
auto result = kernel(x);
std::printf("%f\n", result);
auto end = clock_now();
```

这里计时区域包含了：

```
allocation + initialization + random generation + kernel + printf
```

如果目标是测 `kernel`，这个数字没有表达清楚。它可能主要测到了内存分配器、随机数生成器、首次触页、I/O flush 或动态链接的 lazy binding。

常见误区

“程序运行了多久”和“kernel 花了多久”不是同一个问题。系统性能分析和后续高性能 kernel 通常关心热路径的稳定吞吐；计时区域必须把 setup、timed region、validation 分开。

性能实验的基本结构

一个最小可信实验可以拆成五段：

1. describe environment:
CPU, compiler, flags, OS, governor, command
2. setup:
allocate data, initialize input, pin thread if needed
3. warmup:
run kernel without recording final timing
4. timed region:
measure only the intended workload
5. validation and report:
compare with reference, compute statistics, save raw data

注意顺序：正确性在性能之前。如果一个优化版本结果错了，它再快也没有意义。

心智模型

benchmark 是一个实验系统。kernel 只是其中一个部件；输入、环境、计时器、统计、报告和二进制审计都是实验系统的一部分。训练自己时，不要只问“快不快”，还要问“我凭什么相信这个数字”。

性能结论的三种有效性

评价一个 benchmark，不能只问数字是否漂亮。更重要的是问它是否有效。这里借用实验方法中的三个概念：内部有效性、外部有效性和构造有效性。

内部有效性：差异真由被测因素造成吗

内部有效性关心因果关系。假设你比较版本 A 和版本 B，看到 B 快了 8%。这个差异真的是代码改动造成的吗？还是因为 B 第二个运行，cache 更热？还是 A 编译时没开 `-march=native`？还是运行 A 时机器上有后台任务？还是 A 的结果被校验，B 的结果没校验？如果这些控制变量没有处理，内部有效性就弱。

提高内部有效性的办法包括：同一编译器和参数、同一输入、同一运行命令、随机化运行顺序、重复测量、隔离 setup 和 timed region、保存原始样本、审计两个版本的汇编、确认 correctness 都通过。对微基准来说，还要避免第一个版本总是吃到冷 cache、第二个版本总是吃到热 cache。

外部有效性：结论能推广到哪里

外部有效性关心适用范围。一个版本在 `n=1024`、warm cache、单线程、桌面 CPU 上快，不代表它在 `n=10^8`、内存带宽瓶颈、服务器 CPU、NUMA、并发环境下也快。一个分支优化在随机输入上快，不代表在真实数据高度偏斜时也快。一个 AVX-512 版本在某台机器上快，不代表在降频明显的另一台机器上也快。

提高外部有效性的办法不是无限测所有情况，而是明确报告边界并选择代表性输入。至少要扫描多个规模，记录数据分布，说明 cache 状态，写清硬件和编译参数。结论应写成“在这些条件下，B 的 median 比 A 低 8%”，而不是泛泛写“B 更快”。

构造有效性：指标真的代表你关心的问题吗

构造有效性关心指标是否测到了目标概念。如果你关心用户请求延迟，却只测一个内部函数的 ns/op，指标可能不够。如果你关心内存带宽，却只报告总时间，不报告字节数和 GB/s，指标不完整。如果你关心 AI 推理吞吐，却只测单个算子，不考虑调度、内存复制和 batch 形状，结论可能无法回答真实问题。

选择指标时要问：这个指标对应什么工程目标？优化它会不会改善真实场景？有没有副作用？例如 GFLOP/s 对 GEMM 很有意义，但对分支密集解析器不一定有意义；GB/s 对 copy 很自然，但对随机访问哈希表，cache miss 和查询吞吐可能更重要；median 对稳定吞吐有用，但服务端还要看 p95/p99。

常见误区

一个 benchmark 可以内部有效但外部无效：它在小实验里确实测准了，但不能推广。也可以外部目标明确但构造无效：它想代表真实业务，却选择了错误指标。报告必须同时交代这三类边界。

被测对象：workload、kernel、measurement

先定义几个词。

术语	本书中的含义
workload （工作负载）	数据规模、数据分布、访问模式和运行条件。
kernel （核心计算）	希望重点优化和计时的函数或循环。
timed region （计时区域）	实际包在计时器之间的代码范围。
harness （测试框架）	负责 setup、warmup、重复测量、统计和报告。
reference （参考实现）	语义清晰、容易验证正确性的版本，用来检查优化版本。

例子：

```
float dot_product_ref(std::span<const float> a, std::span<const float> b)
{
    float sum = 0.0f;
    for (std::size_t i = 0; i < a.size(); ++i) {
        sum += a[i] * b[i];
    }
    return sum;
}
```

这里 dot_product_ref 是 reference 或 baseline，不一定是最终 kernel。workload 包括：

- a 和 b 的长度。
- 数据是否随机、全零、全一、递增、含 NaN、含 denormal。
- 指针是否对齐。
- 数据是否已经在 cache 中。
- 是否只测一个长度，还是扫描多个长度。

如果报告只写“dot product 需要 3 ms”，信息不够。至少应写：

```
kernel:
    dot_product_ref(float*, float*, n)

workload:
    n = 1048576
    data = uniform random in [-1, 1]
    arrays allocated once, reused
    warm-cache repeated measurement

compiler:
    clang++ -O3 -march=native -DNDEBUG

machine:
    CPU model, core count, governor, OS kernel

metric:
    median nanoseconds over 31 measured repetitions
```

正确性先于性能

性能优化常见灾难是“优化版本快了，因为少算了”。因此每个 benchmark 都要先有 correctness reference。

以整数求和为例：

```
#include <stdint>
#include <span>

std::uint64_t sum_u64_ref(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
    return sum;
}
```

为什么用 `std::uint64_t`? 因为 unsigned overflow 在 C++ 中定义为模 2^{64} 回绕。若使用 `std::int64_t`, 输入过大时 signed overflow 是未定义行为, reference 也可能被优化器按“不会溢出”的假设改写。

测试思路:

```
void check_sum_kernel(std::span<const std::uint64_t> values)
{
    const std::uint64_t expected = sum_u64_ref(values);
    const std::uint64_t actual = sum_u64_optimized(values);
    if (actual != expected) {
        std::abort();
    }
}
```

如果是浮点, 正确性不能总用完全相等。应根据算法和数值误差选择容忍度:

```
absolute error:
abs(actual - expected) <= epsilon

relative error:
abs(actual - expected) <= epsilon * max(abs(expected), 1)
```

深入理解

浮点优化不仅是性能问题, 也是语义问题。改变求和顺序会改变舍入误差。启用 `-ffast-math` 还会改变 NaN、Inf、signed zero 和结合律相关的语义边界。后续讲 fast-math 和 AI 算子时会专门展开。现在先记住: 浮点 benchmark 必须同时报告数值误差策略。

时间源：你到底在读什么钟

性能测量常见时间源有几类。

时间源	适合用途和主要风险
steady_clock clock_gettime	C++ 通用单调墙钟计时; 分辨率和开销依平台而异。 Linux 下可选择 CLOCK_MONOTONIC 等时间源; 系统调用或 vDSO 路径需要了解。
TSC	低开销 cycle-like 计数; 风险是乱序、迁核、频率解释和序列化问题。
PMU counters	观察硬件事件, 例如 cycles、instructions、cache miss; 风险是权限、事件语义、复用和平台差异。
/usr/bin/time	粗粒度进程级时间和资源统计; 不适合细粒度 kernel。

steady_clock

C++ 中应优先使用 `std::chrono::steady_clock` 作为入门计时器, 因为它承诺单调不倒退。

```
using Clock = std::chrono::steady_clock;

const auto begin = Clock::now();
```

```
run_kernel();
const auto end = Clock::now();

const auto elapsed_ns =
    std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count();
```

不要默认 `high_resolution_clock` 一定更好。标准没有保证它一定是单调钟；在某些实现中它只是其他 clock 的别名。入门阶段用 `steady_clock` 更稳。

计时器也有开销

计时器调用不是免费的。一次 `Clock::now()` 可能通过 vDSO 快路径，也可能更重；一次 `rdtsc` 很轻，但加上 fence 后开销会上升；一次 `perf` 采样或系统级统计也有自己的成本。若被测 kernel 本身只需要几十纳秒，计时器开销就可能和目标工作同量级，甚至超过目标工作。

因此，短 kernel 不能简单测一次。常见方法是 batch：在一次计时区域内重复运行 kernel 多次，然后用总时间除以次数。这样计时器开销被摊薄。但 batch 也会改变实验状态，例如 cache 更热、分支预测更稳定、循环控制开销变成 harness 的一部分。所以 batch 次数要记录，并尽量让每次计时区域持续足够长，比如至少数十微秒或更长，具体取决于时间源分辨率和噪声水平。

另一种方法是测空 harness 开销：

```
time_full = measure(batch with kernel)
time_empty = measure(batch without kernel or with minimal placeholder)
estimated_kernel_time = time_full - time_empty
```

这个方法也有风险，因为空循环和真实循环的优化、分支、cache、寄存器压力可能不同。它适合作为辅助 sanity check，不应机械相信。真正可靠的做法是让被测工作量足够大，并通过汇编审计确认 timed region 中确实执行了目标工作。

分辨率、精度和准确度

计时讨论里常混淆三个词。分辨率是计时器能区分的最小刻度；精度是多次测量的波动有多大；准确度是读数和真实时间接近程度。一个计时器分辨率高，不代表整个 benchmark 准确。系统调度、中断、频率变化和 cache 状态会让样本波动很大。

如果你测到大量样本都是 0 ns 或几个固定值，说明被测工作量太小或计时器分辨率不够。若样本有少量极大值，可能是系统干扰。若样本分布呈现多个簇，可能是频率状态、cache 状态、迁核或不同代码路径造成。不要只看最后一个平均值，要先看原始样本。

预实验：先确认自己测得动

正式比较两个版本前，先做一轮很小的预实验。预实验的目标不是得出性能结论，而是检查测量系统是否工作：计时区域是否足够长，样本是否不是全零，重复测量是否有可解释的波动，结果是否通过 correctness，反汇编是否保留了目标工作。很多失败的 benchmark 如果先做预实验，几分钟内就能发现问题。

预实验可以按下面顺序进行。第一，用一个中等输入规模跑 10 次，观察每次 elapsed 是否大于计时器开销很多倍。第二，把输入规模扩大 10 倍，确认时间大致随工作量增加；如果时间几乎不变，可能被优化器删除、被常量折叠，或者计时区域不正确。第三，把 kernel 结果改错或输入改动一次，确认 correctness 检查确实会失败；如果错误结果仍然通过，校验逻辑本身不可信。第四，生成反汇编，确认目标循环、加载、存储或调用存在。

这一步看似朴素，却非常重要。性能实验最怕“精密地测错东西”：你可以用很漂亮的统计表、很多重复次数、复杂图表，最后仍然只是在测一个空循环或测输入生成。预实验的作用是先排除低级失效，让后面的统计分析有意义。

核心思想

预实验不回答“哪个版本更快”，只回答“这个 benchmark 有没有资格进入正式测量”。如果预实验无法证明目标工作存在、结果正确、计时区域足够长，就不要继续解读性能数字。

batch 校准：把短工作变成长测量

有些 kernel 本来就很短，例如一次整数 hash、一次小对象比较、一次小数组尾部处理。单次调用可能只有几纳秒到几十纳秒，直接包两个计时器会让计时器开销占比过大。这时需要 batch：一次 timed region 内运行多次 kernel，把总耗时拉长。

一个简单校准策略是逐步加倍 batch，直到单个 timed region 超过目标时间：

```
batch = 1
while measured_time(batch) < target_time:
    batch = batch * 2
```

目标时间不是固定真理。入门实验可以选择几十微秒到几毫秒之间的量级，目的是让计时器分辨率、系统调用开销和偶然中断不再主导每个样本。对极短 kernel，batch 可能需要很大；对内存带宽 kernel，输入规模本身已经足够大，batch 反而可能改变 cache 状态。

batch 校准要注意三个边界。第一，batch 后报告的单位必须清楚。总时间是 `elapsed_ns_per_batch`，除以 batch 后才是 `ns_per_call`。第二，batch 会改变状态。连续重复同一个小 kernel 会让 instruction cache、data cache、分支预测和寄存器分配处于非常稳定的热状态，这可能不同于真实调用间隔。第三，batch 循环本身也有成本。若 kernel 很短，循环控制、函数指针调用、分支和防优化工具都可能成为测量对象的一部分。

因此，batch 不是无代价的测量技巧。它解决“太短测不准”的问题，但也把实验问题改成了“重复执行同一 kernel 的稳定吞吐”。如果真实场景是一段代码偶尔执行一次，batch 结果只能说明热路径能力，不能直接代表用户延迟。

空测量和负值陷阱

很多人会尝试测量空 harness，再从总时间中减掉：

```
kernel_time = full_time - empty_time
```

这个思路可以帮助估计计时器和循环框架开销，但不能无条件使用。原因是 empty harness 和 full harness 不一定有相同的优化结果。空循环可能被删除，真实循环可能保留；空循环的寄存器压力、分支预测、cache 行为和流水线占用都和真实 kernel 不同。你减掉的不一定是“共同开销”，可能是另一个实验的时间。

更危险的是，当 full 和 empty 都很小、噪声又较大时，相减可能得到接近零甚至负值。负的 kernel time 当然没有物理意义，它说明你的测量系统不足以分辨这段工作。正确反应不是把负值截断为零，而是扩大工作量、增加 batch、改用更合适的时间源，或者承认当前方法不能测这个对象。

常见误区

空测量只能作为 sanity check。若结论依赖 `full_time-empty_time` 的微小差值，就必须说明误差来源，并用更长 timed region 或反汇编证据支撑。

墙钟时间和 CPU 时间

`steady_clock` 测的是墙钟经过时间，即真实世界过去了多少时间。若线程被操作系统暂停，墙钟仍然前进。进程 CPU time 则更接近进程实际占用 CPU 的时间，不包括部分等待时间。两者回答不同问题。

对于单线程纯计算 kernel，墙钟时间通常就是我们关心的吞吐时间；但如果程序包含 I/O、睡眠、锁等待、系统调用阻塞，墙钟和 CPU time 会明显不同。端到端性能通常关心墙钟，因为用户等待的是实际时间；分析 CPU 密集程度时，CPU time 和 context switch 信息也很有价值。

报告时应写清使用哪种时间。不要把 `/usr/bin/time` 的 user time、sys time、elapsed time 混成一个概念。user time 表示用户态 CPU 时间，sys time 表示内核态 CPU 时间，elapsed 表示墙钟时间。它们差异本身往往就是性能线索。

TSC 与 cycle 的关系

x86 上可以用 `rdtsc` 或 `rdtscp` 读取 time-stamp counter。它返回一个 64 位计数，常见写法是 `edx:eax` 组合：

```
#include <cstdint>
```

```
#include <x86intrin.h>

std::uint64_t read_tsc()
{
    return __rdtsc();
}
```

但是 TSC 不是新手测量的万能工具。风险包括：

- `rdtsc` 不是完全序列化指令，周围指令可能乱序跨过它。
- 如果线程迁移到另一个 core，早期或特殊系统上的 TSC 同步可能成为问题。
- 现代 CPU 的 TSC 常以 invariant rate 增长，不一定等于当前核心实际执行频率。
- Turbo、降频、C-state 会让“每秒多少 TSC tick”和“每条指令需要多少 core cycle”的解释变复杂。

更严谨的 TSC 计时需要 fence：

```
#include <stdint>
#include <x86intrin.h>

std::uint64_t read_tsc_begin()
{
    _mm_lfence();
    const std::uint64_t value = __rdtsc();
    _mm_lfence();
    return value;
}

std::uint64_t read_tsc_end()
{
    _mm_lfence();
    const std::uint64_t value = __rdtsc();
    _mm_lfence();
    return value;
}
```

不同平台和论文中还会看到 `cpuid`、`rdtscp`、`lfence` 的不同组合。本书训练阶段的原则是：

- 初期先用 `steady_clock` 建立正确实验结构。
- 需要 cycle 级分析时，再引入 TSC，并明确 fence 策略。
- 真正分析微架构时，优先结合 PMU counter，而不是只看 TSC。

常见误区

不要把 TSC delta 直接称为“CPU cycles”而不解释。很多现代机器上 TSC 是稳定参考计数，核心实际频率会变化。报告时可以写 `TSCticks` 或说明换算假设。

编译器会删除你的 benchmark

优化器只需要保留 C++ 抽象机的可观察行为。benchmark 里最常见的错误是被测计算没有可观察结果。

dead-code elimination

错误：

```
void benchmark_bad(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
}
```

`sum` 没有被使用。优化器可以删除整个循环。

最简单的防护是把结果返回给调用者：


```
std::uint64_t benchmark_better(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
    return sum;
}
```

但如果调用者也没有使用返回值，仍可能被删除。benchmark harness 需要显式制造可观察使用。

do_not_optimize

GCC/Clang 常用一个空 inline asm 阻止优化器认为值无用：

```
template <class T>
inline void do_not_optimize(const T& value)
{
    asm volatile("" : : "g"(value) : "memory");
}
```

用法：

```
const std::uint64_t result = benchmark_better(values);
do_not_optimize(result);
```

这段 inline asm 没有机器指令，但它告诉编译器：**value** 被某个不可见的 asm 使用，不能随便删除产生它的计算。“memory” 还限制周围内存访问重排。

常见误区

do_not_optimize 是 benchmark 工具，不是业务代码工具。它的目的是约束编译器优化，让实验可测；不要把它放进生产热路径。

clobber_memory

另一个常用工具：

```
inline void clobber_memory()
{
    asm volatile("" : : "memory");
}
```

它告诉编译器：内存状态可能被未知 asm 改变，因此不要把内存读写随便跨过这个点移动。它不会清空 CPU cache，也不是硬件内存栅栏。

constant folding

错误：

```
std::uint64_t x = expensive(123);
```

如果 **expensive(123)** 在编译期可推导，优化器可能直接替换成常量。benchmark 输入应来自运行时数据，或者至少来自编译器无法完全证明的内存。

例子：

```
std::vector<std::uint64_t> make_input(std::size_t size)
{
    constexpr std::uint64_t INPUT_MULTIPLIER = 6364136223846793005ULL;
    constexpr std::uint64_t INPUT_INCREMENT = 1442695040888963407ULL;

    std::vector<std::uint64_t> values(size);
    std::uint64_t state = 1;
    for (std::uint64_t& value : values) {
        state = state * INPUT_MULTIPLIER + INPUT_INCREMENT;
        value = state;
    }
}
```

```
    return values;
}
```

这里用简单线性同余生成输入。它不是高质量随机数生成器，但适合构造可复现、运行时生成的数据。真正 benchmark 时，输入生成仍应放在 setup，不放入 timed region。

内联会改变被测边界

如果你写了一个函数 `kernel(input)`，但编译器把它内联到 harness 里，那么测量仍然可能是合法的，只是被测对象变成了“内联后的热路径”，而不是“独立函数调用”。这会影响解释：函数调用开销消失，参数传递形态可能改变，循环外提和常量传播可能跨过原来的函数边界。

有时你正想测内联后的真实性能，因为生产代码也会内联；有时你想比较两个函数实现本身，希望禁止内联。两种实验都可以，但必须明确。可以用编译器属性、单独翻译单元、链接边界或函数指针调用降低内联，但这些手段也会引入新的成本。没有一种方式天然更正确，关键是和问题陈述一致。

自动向量化和库调用替换

`-O3-march=native` 下，编译器可能把标量循环自动向量化，使用 SSE/AVX 指令一次处理多个元素。它也可能把某些模式识别成库函数，例如内存 `copy`、`fill`、`compare` 等。这样源码中的循环和实际机器码差异很大。

这不是坏事。现代 C++ 性能很大程度依赖编译器优化。但如果你的实验目标是“比较手写标量循环和手写 SIMD”，而 baseline 已经被自动向量化，那么结论就变了。如果你的目标是“普通 C++ 在当前编译器下能有多快”，自动向量化正是应该保留的能力。

报告中至少要说明：

- 核心循环是否被向量化。
- 是否出现 `vmov`、`vpadd`、`vfmadd` 等向量指令。
- 是否调用了 `memcpy`、`memset` 或其它库函数。
- 是否因为 `-march=native` 使用了本机特定 ISA。

LTO 和跨文件优化

链接时优化（LTO）会让编译器在链接阶段继续看到多个翻译单元的 IR，从而跨文件内联、删除未用函数、传播常量和重排代码。没有 LTO 时，目标文件边界可能阻止一些优化；开启 LTO 后，benchmark 代码和被测代码之间的隔离可能变弱。

这对 benchmark 有两面影响。LTO 可能更接近生产构建，也可能让微基准被优化器“看穿”。例如 `reference` 和 `optimized` 两个函数若都在同一可见范围内，编译器可能合并、内联或删除某些路径。使用 LTO 时，更要依靠 `do_not_optimize`、独立输入、二进制审计和正确的可观察输出。

防优化边界也会改变问题

为了防止 benchmark 被删除，常见做法包括返回结果、打印结果、写入 `volatile` 对象、使用 `inline asm` 的 `do_not_optimize`、把函数放到独立翻译单元、用函数指针调用、给函数加 `noinline` 属性。它们都能增加可观察性或降低优化器视野，但没有一种手段是完全中性的。

打印结果最直观，但 I/O 太重，不能放进 timed region。写 `volatile` 可以阻止某些优化，但 `volatile` 内存访问本身有语义，会限制优化并改变机器码。函数指针调用可以降低内联概率，但引入间接调用和分支预测问题。独立翻译单元可以模拟库边界，但若生产构建开启 LTO，这个边界可能并不存在。`noinline` 能保留函数调用形态，但如果真实业务依赖内联后的常量传播，它会低估生产性能。

所以，防优化边界要服务实验问题。若问题是“生产构建下这段 C++ 热路径能跑多快”，就不应随意禁止内联和向量化；应让编译器正常优化，再通过 `do_not_optimize` 消费最终结果，确认目标工作没有消失。若问题是“这个函数调用 ABI 边界的成本是多少”，就应保留调用边界，并明确报告 `noinline` 或跨翻译单元策略。若问题是“手写汇编函数和 C++ 函数的调用成本比较”，就要保证两者在相同 ABI、相同对齐、相同调用次数和相同输入下测量。

手段	作用	主要代价
返回结果并消费	防止结果无用而删除。	调用者仍可能被优化看穿。
<code>do_not_optimize</code>	告诉编译器值被不可见代码使用。	只约束编译器，不代表真实业务。
<code>volatile</code> 写入	制造可观察访问。	改变内存语义和优化空间。
<code>noinline</code> 独立翻译单元	保留函数调用边界。 限制跨函数分析。	可能低估内联生产路径。 与 LTO 或 header-only 真实场景可能不同。
函数指针调用	降低直接内联。	增加间接调用和预测成本。

学习阶段最常用的组合是：输入在运行时生成，目标函数返回结果，结果与 `reference` 校验，校验后用 `do_not_optimize` 消费，最后用反汇编确认目标工作存在。等你明确需要研究调用边界时，再引入 `noinline` 或独立翻译单元。

常见误区

反汇编审计不是附加项。任何性能结论至少要确认：目标工作存在，计时区域没有被优化成别的东西，两个版本的机器码差异与你声称的优化一致。

计时区域必须隔离

错误结构：

```
auto begin = Clock::now();
std::vector<std::uint64_t> values = make_input(size);
const std::uint64_t result = sum_u64(values);
check(result);
auto end = Clock::now();
```

这测的是：

```
allocation + input generation + sum + check
```

正确结构：

```
std::vector<std::uint64_t> values = make_input(size);
const std::uint64_t expected = sum_u64_ref(values);

auto begin = Clock::now();
const std::uint64_t result = sum_u64(values);
auto end = Clock::now();

if (result != expected) {
    std::abort();
}
```

如果要重复测量：

```
setup:
    allocate and initialize input once
    compute expected reference once

warmup:
    run kernel several times

measurement:
    for each repetition:
        begin timer
        run kernel
        end timer
```

```
record elapsed
consume result

validation:
compare result with expected
```

深入理解

有些 workload 本身包含分配或 I/O，比如数据库查询、HTTP 服务、编译器编译一个文件。那不是不能测，而是要诚实命名 benchmark：你测的是 end-to-end workload，不是纯计算 kernel。不同问题需要不同边界。

warmup：第一次运行不代表稳态

第一次运行常常慢，因为它可能包含：

- 代码页和数据页首次触页。
- instruction cache 和 data cache 冷启动。
- branch predictor 尚未学习输入模式。
- 动态库 lazy binding。
- CPU 从低功耗状态升频。
- 内存分配器初始化。

所以 benchmark 通常先 warmup：

```
constexpr int BENCHMARK_WARMUP_REPETITIONS = 5;

for (int iter = 0; iter < BENCHMARK_WARMUP_REPETITIONS; ++iter) {
    const std::uint64_t result = sum_u64(values);
    do_not_optimize(result);
}
```

warmup 不是为了“作弊让 cache 热起来”，而是为了明确你测的是某个状态。你可以测 cold-cache，也可以测 warm-cache，但必须说清楚。

状态	含义
cold-cache	每次计时前尽量让数据不在 cache 中，模拟首次访问或大工作集。
warm-cache	同一数据重复运行，测稳定热路径吞吐。
steady-state	程序已经过初始化、预测器和 cache 已接近稳定状态。

重复测量和统计口径

单次测量几乎没有说服力。需要多次运行并保存原始数据。

常见统计量：

统计量	含义	风险
min	最小观测值，接近“噪声最少时”的能力。	可能过于乐观。
mean	平均值。	对 outlier 敏感。
median	中位数，代表典型运行。	不展示尾部风险。
p95	95 分位，展示较慢尾部。	需要足够样本。
max	最慢观测。	可能只是一次系统干扰。

HPC kernel 优化常看 min 或 median，因为目标是估计稳定热路径上限。服务端延迟常看 p95、p99，因为用户体验受尾延迟影响。AI 算子库通常会同时报告 median、min 和标准差或变异系数。

常见误区

不要只报告一个数字且不说明它是什么。写“用时 1.2 ms”是不完整的；应写“31 次重复, median = 1.2 ms, min = 1.17 ms, p95 = 1.35 ms”。

效应大小和不确定性

重复测量之后, 下一步不是急着宣布胜负, 而是判断差异是否有工程意义。性能比较至少要同时看两个问题: 差异有多大, 差异是否大到超过测量不确定性。

假设版本 A 的 median 是 1000 ns, 版本 B 的 median 是 980 ns。表面看 B 快 2%。如果样本波动只有 2 ns, 这个差异可能很稳定; 如果样本在 930 ns 到 1100 ns 之间跳动, 这个差异就没有说服力。反过来, 若 B 从 1000 ns 降到 600 ns, 即使样本有几十纳秒波动, 也很可能是实质改善。

绝对差异和相对差异

性能报告常写百分比, 但百分比不能替代绝对值。一个函数从 2 ns 到 1 ns 是 2 倍提升, 但绝对只省 1 ns; 如果它在真实请求中只调用一次, 工程价值可能很小。一个任务从 10 s 到 9 s 只有 10% 提升, 但绝对省 1 s, 可能非常有价值。

因此报告建议同时写:

```
absolute_delta_ns = candidate_median_ns - baseline_median_ns
relative_speedup = baseline_median_ns / candidate_median_ns
relative_change = (candidate_median_ns - baseline_median_ns) / baseline_median_ns
```

如果使用吞吐指标, 也要说明方向。时间越小越好, 吞吐越大越好。不要把时间下降 20% 和吞吐提升 20% 混为一谈。时间从 100 到 80 是下降 20%, 吞吐从每秒 10 次到每秒 12.5 次是提升 25%。它们描述同一个变化, 但百分比不同。

差异小于噪声时不要下结论

一个实用规则是: 先看两个版本样本区间是否大量重叠, 再看差异是否稳定出现在多个输入规模和多轮运行中。如果 A 和 B 的 median 差 1%, 而每个版本的 p95 和 min 相差 10%, 这通常不足以支持“B 更快”。可以写“本实验未观察到稳定差异”, 而不是强行挑一个较小数字。

这不是说小差异永远没有意义。在超大规模服务中, 1% CPU 成本可能很重要; 在极稳定的离线 kernel 上, 1% 也可能可测。但小差异需要更强证据: 更多重复、交错运行、固定环境、保存原始样本、反汇编确认差异、必要时跨天复测。证据强度要和结论强度匹配。

中位数之外还要看形状

中位数很好用, 但它只描述排序后中间那个位置。样本分布的形状也重要。若样本集中在一个窄范围内, 说明实验稳定; 若样本有长尾, 说明偶发干扰或系统状态变化; 若样本分成两个明显簇, 说明可能有两种运行状态, 例如迁核、频率档位、cache 状态、分支路径或页错误。

第一册不要求你掌握复杂统计, 但要求你养成看原始样本的习惯。一个简单文本表就足够:

```
sample index:
 0 1 2 3 4 ...

elapsed ns:
980 982 979 981 1450 ...
```

如果只有一两个离群点, 报告可以同时给 median 和 max, 并解释可能原因。如果每隔几次就出现慢样本, 就不能随便把它当作偶然噪声, 应该检查系统状态、输入路径和 harness。性能数据中的“丑陋形状”常常比漂亮平均值更有信息量。

置信区间的最低限度理解

严格统计分析会讨论置信区间、bootstrap、假设检验和效应量。入门阶段不需要把 benchmark 变成统计课, 但要理解一个核心事实: 样本摘要只是对真实运行行为的估计。重复次数越少、噪声越大, 估计越不稳定。

如果需要更严谨地比较 median, 可以用 bootstrap 思路: 从原始样本中有放回地重复抽样, 计算每次抽样的 median 差异, 观察差异分布是否经常跨过零。这里不要求手算, 也不要求本章实现;

你只需要知道它回答的问题是“在当前样本波动下，差异是否稳定”。未来做严肃性能回归检测时，可以用专门工具完成这件事。

无论是否使用统计工具，报告都不要写得过度确定。可以写“在本机、本输入和本测量协议下，B 的 median 稳定低于 A，差异约为 8%”，不要写“B 永远比 A 快”。如果差异很小，可以写“当前证据不足以判断 2% 差异是否真实”。

噪声、偏差和离群点

测量波动不全是同一种东西。随机噪声、系统性偏差和离群点需要不同处理。

随机噪声是不可完全控制的小波动，例如轻微中断、cache 替换、执行资源竞争、分支预测细节。重复测量和统计摘要可以降低它对结论的影响。若两个版本差异远大于随机波动，结论更可信；若差异和波动同量级，就不能轻易说谁更快。

系统性偏差是实验设计造成的固定倾向。例如总是先运行 A 再运行 B，导致 B 总是吃到更热的 cache；总是用不同编译参数；总是让 A 包含校验而 B 不包含；把 setup 放进 A 的计时区域但没放进 B。重复测量不能自动消除系统性偏差，因为每次都在重复同一个错误。

离群点是明显偏离多数样本的观测，可能来自调度、中断、缺页、后台任务、频率变化或热保护。离群点不应随手删除。正确做法是保存原始数据，报告统计口径，并解释处理策略。如果目标是热路径吞吐，可以关注 min/median 并说明 max 受系统干扰；如果目标是服务端延迟，离群点和尾部本身就是重要结果。

运行顺序效应

比较 A/B 两个版本时，运行顺序会影响结果。常见做法包括交替运行：

```
A B A B A B ...
```

或者随机化运行顺序：

```
B A A B B A ...
```

这样可以降低“先运行的版本总是冷，后运行的版本总是热”的偏差。对于很小的 kernel，也可以在同一进程中重复多个 round，每个 round 重新打乱版本顺序。报告中应说明顺序策略。

看原始样本

只看摘要很危险。假设两个版本 median 相同，但一个版本样本非常稳定，另一个版本偶尔慢 10 倍；如果你只报告 median，就漏掉了尾部风险。反过来，一个版本 max 很大，可能只是一次无关中断；若你只报告 max，也会过度悲观。

建议至少保存 CSV 原始样本：

```
kernel,n,round,elapsed_ns
sum_u64,1048576,0,123456
sum_u64,1048576,1,122980
...
```

有了原始样本，后续可以重新计算 median、p95、标准差、变异系数，也可以画散点图或箱线图。没有原始样本，报告只能依赖当时选的摘要，难以复核。

核心理想

重复测量解决随机噪声，不能修复错误实验设计。随机化顺序、保存原始样本、报告统计口径，是防止性能结论自欺的基本手段。

一个最小 C++20 benchmark harness

下面给一个教材用最小 harness。它不是要替代 Google Benchmark，而是让你看清楚每个部件为什么存在。

```
#include <algorithm>
#include <chrono>
#include <cstdint>
```

```

#include <cstdio>
#include <cstdlib>
#include <span>
#include <vector>

namespace {

constexpr std::size_t BENCHMARK_INPUT_SIZE = 1u << 20;
constexpr int BENCHMARK_WARMUP_REPETITIONS = 5;
constexpr int BENCHMARK_MEASURED_REPETITIONS = 31;
constexpr std::uint64_t INPUT_MULTIPLIER = 6364136223846793005ULL;
constexpr std::uint64_t INPUT_INCREMENT = 1442695040888963407ULL;

using Clock = std::chrono::steady_clock;

template <class T>
inline void do_not_optimize(const T& value)
{
    asm volatile("" : : "g"(value) : "memory");
}

std::vector<std::uint64_t> make_input(std::size_t size)
{
    std::vector<std::uint64_t> values(size);
    std::uint64_t state = 1;
    for (std::uint64_t& value : values) {
        state = state * INPUT_MULTIPLIER + INPUT_INCREMENT;
        value = state;
    }
    return values;
}

std::uint64_t sum_u64(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
    return sum;
}

std::uint64_t elapsed_ns_for_one_run(std::span<const std::uint64_t> values,
                                     std::uint64_t* result_out)
{
    const auto begin = Clock::now();
    const std::uint64_t result = sum_u64(values);
    const auto end = Clock::now();

    *result_out = result;
    do_not_optimize(result);

    return static_cast<std::uint64_t>(
        std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count());
}

double median_ns(std::vector<std::uint64_t> samples)
{
    std::sort(samples.begin(), samples.end());
    const std::size_t middle = samples.size() / 2;
    return static_cast<double>(samples[middle]);
}

} // namespace

int main()
{
    const std::vector<std::uint64_t> values = make_input(BENCHMARK_INPUT_SIZE);
    const std::uint64_t expected = sum_u64(values);

    for (int iter = 0; iter < BENCHMARK_WARMUP_REPETITIONS; ++iter) {
        const std::uint64_t result = sum_u64(values);
        do_not_optimize(result);
    }
}

```

```
std::vector<std::uint64_t> samples;
samples.reserve(BENCHMARK_MEASURED_REPETITIONS);

for (int iter = 0; iter < BENCHMARK_MEASURED_REPETITIONS; ++iter) {
    std::uint64_t result = 0;
    const std::uint64_t elapsed_ns = elapsed_ns_for_one_run(values, &result);
    if (result != expected) {
        std::abort();
    }
    samples.push_back(elapsed_ns);
}

const auto [min_it, max_it] = std::minmax_element(samples.begin(), samples.end());
std::printf("n=%zu min_ns=%llu median_ns=%.0f max_ns=%llu\n",
            values.size(),
            static_cast<unsigned long long>(*min_it),
            median_ns(samples),
            static_cast<unsigned long long>(*max_it));
}
```

编译：

```
clang++ -std=c++20 -O3 -march=native -DNDEBUG \
    bench_sum.cpp -o bench_sum
./bench_sum
```

这个 harness 做对了几件事：

- 输入在计时前生成。
- reference 结果在计时前计算。
- warmup 和 measurement 分离。
- 每次计时只包住 `sum_u64`。
- 结果被校验并被 `do_not_optimize` 消费。
- 重复测量并报告 min、median、max。

它还不够完整：

- 没有输出原始 CSV。
- 没有记录环境。
- 没有 CPU affinity。
- 没有 PMU counter。
- 没有多输入规模扫描。
- 没有统计 p95 和变异系数。

这些会在本册后续内容和第二册性能专题中逐步补齐。

环境控制：CPU 不只是在跑你的代码

一次 benchmark 运行时，机器上还有很多因素：

- 操作系统可能调度其他进程。
- 中断可能打断当前 core。
- 线程可能迁移到另一个 core。
- CPU 可能升频或降频。
- SMT sibling 可能共享执行资源。
- 后台服务可能污染 cache 或占用内存带宽。
- 虚拟机或容器可能隐藏真实硬件行为。

入门阶段可以先做低成本控制：

```
# 查看 CPU 型号
lscpu
```

```
# 查看当前频率策略，具体路径依发行版而异
```

```
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
```

```
# 绑定到某个 CPU 运行
taskset -c 2 ./bench_sum

# 粗略查看进程级时间
/usr/bin/time -v ./bench_sum
```

报告中至少记录：

```
CPU:
    model name, cores, SMT on/off if known

OS:
    kernel version, distribution

compiler:
    compiler name and version
    flags

run command:
    exact command line

environment:
    taskset core if used
    governor if known
    whether running on laptop battery, VM, container, remote server
```

深入理解

严格控制频率和隔离 CPU core 可能需要 root 权限和系统配置，例如 performance governor、isolcpus、关闭 turbo、关闭 SMT、固定 NUMA 节点。训练早期不一定要全部做到，但必须知道没有控制意味着结果会有更大噪声。

频率和功耗状态

现代 CPU 会根据温度、功耗、电源策略和负载动态调整频率。短 benchmark 可能在升频过程中运行，长 benchmark 可能触发功耗限制后降频。笔记本电池供电、散热不足、后台负载、不同 governor 都会影响结果。

这意味着“cycles”和“seconds”的关系并不总是简单。TSC 可能以固定参考频率增长，核心实际执行频率可能变化；PMU 的 core cycles 又可能反映另一种事件定义。报告中若只写时间，不记录频率策略，别人很难解释跨机器差异。

训练阶段可以做两件低成本工作。第一，记录 governor 和是否使用电池/虚拟机/容器。第二，运行足够长，让短暂升频过程不主导结果，或者明确测的是短突发性性能。严格实验再考虑固定 governor、关闭 turbo、控制温度和隔离核心。

SMT 和共享资源

SMT 让一个物理核心同时运行多个硬件线程。两个 sibling 线程共享部分前端、执行端口、cache 和其它资源。如果 benchmark 绑定到某个逻辑 CPU，而它的 sibling 正在运行后台任务，结果可能变慢且波动更大。

这不是说必须关闭 SMT。生产环境可能正是开启 SMT 的状态；端到端服务也可能要测共享环境下表现。但如果目标是研究某个 kernel 的单线程上限，就应尽量避免 sibling 干扰，或者至少记录 SMT 状态和绑定 CPU。对于内存带宽型实验，多线程和 SMT 的关系还会更复杂。

NUMA 和内存位置

多路服务器上，内存可能分布在不同 NUMA node。线程运行在一个 socket 上，却访问另一个 socket 分配的内存，会增加延迟并降低带宽。首次触页策略常常决定页面归属：哪个线程先写页面，页面就可能分配在哪个 NUMA node。

第一册不深入 NUMA 调优，但要知道大数组 benchmark 在服务器上可能受 NUMA 影响。报告中如果运行在多 socket 机器，应记录 CPU 绑定和内存绑定策略；若没有控制，就不要把结果轻易推广成“这个算法的内存带宽上限”。

容器、虚拟机和远程机器

很多学习和 CI 环境运行在容器、虚拟机或共享远程机器上。这些环境可以用于功能验证和粗略性能观察，但它们隐藏了很多控制变量。虚拟 CPU 可能不是独占的，PMU 可能不可用或不准确，CPU 型号可能被抽象，频率策略可能不可见，邻居任务可能共享 cache、内存带宽和物理核心。

这不意味着容器里的 benchmark 没有价值。它可以发现数量级问题，例如某个版本慢 10 倍、某个优化被编译器删除、某个算法复杂度不对。它也可以用于固定环境下的相对回归检测。但如果你要发布“某 CPU 上达到多少 GB/s”或“某优化利用了某硬件事件”的结论，就需要在能解释硬件状态的环境中复测。

报告中应诚实写出环境等级。例如：

```
environment:
  container on shared host
  CPU affinity unavailable
  perf cycles unavailable due to permission
  results used only for relative sanity check
```

这样的报告比假装环境可控更专业。性能工程不是必须每次都拥有完美实验室，而是要让读者知道证据能支撑到哪里。

操作系统噪声：线程不是一直在核心上运行

很多入门 benchmark 默认程序一旦开始计时，就连续占用 CPU 执行到结束。这个想象太理想。普通用户态程序运行时，操作系统可以随时因为调度、中断、系统调用、缺页、信号、内核线程或其它进程而改变执行状态。墙钟计时器不会因为你的线程暂停而暂停；所以一次 elapsed time 可能包含“线程真正执行的时间”和“线程等待重新运行的时间”。

调度和迁核

Linux 调度器负责在逻辑 CPU 上安排可运行线程。若系统中有多个可运行任务，调度器可能让你的 benchmark 暂停，让别的任务运行。之后 benchmark 可能回到同一个逻辑 CPU，也可能迁移到另一个逻辑 CPU。迁核会带来几个影响。

第一，cache 热度可能丢失。数据还在原核心附近的私有 L1/L2 cache 中，新核心需要重新加载。第二，分支预测器、TLB 和其它核心本地状态可能不同。第三，若迁移到另一个物理核心或另一个 NUMA node，内存访问和共享资源状态也会变化。第四，若两个核心频率或热状态不同，时间也可能变化。

绑定 CPU 可以减少迁核：

```
taskset -c 2 ./bench_sum
```

但绑定不是万能的。绑定到 CPU 2 只说明进程可在该逻辑 CPU 上运行，不说明系统不会在同一个 CPU 上调度其它任务，也不说明 SMT sibling 空闲。报告中应写“使用 taskset 绑定 CPU 2”，不要写成“完全隔离核心”，除非你确实使用了更严格的系统配置。

中断和内核活动

即使你绑定了 CPU，硬件中断、定时器、网络、存储、内核线程也可能打断执行。短 benchmark 中一次中断就能制造很大的离群点。长 benchmark 中，中断影响会被摊薄，但仍可能增加尾部时间。

这就是为什么 raw samples 中常看到少数慢很多的样本。它们不一定说明 kernel 有 bug，也不一定应该删除。正确做法是保留原始样本，报告 median、p95、max，并结合实验目标解释。如果目标是估计稳定热路径吞吐，median 可能更有代表性；如果目标是用户请求尾延迟，慢样本本身就是重要事实。

page fault 和首次触页

访问一块刚分配的大内存时，第一次写入可能触发 page fault。这里的 page fault 不一定是磁盘换页，很多时候是 demand paging：操作系统在第一次访问时才为虚拟页面分配物理页、清零并建立页表映射。这个成本可能远高于普通内存访问。

例如：


```
std::vector<std::uint64_t> values(n);
auto begin = Clock::now();
for (std::uint64_t& value : values) {
    value = 1;
}
auto end = Clock::now();
```

如果这是第一次触碰 `values` 的页面，计时结果可能包含大量首次触页成本。若你的问题是“初始化新分配数组的端到端成本”，这可以包含；若你的问题是“稳定写带宽”，就应该在 `setup` 中先触页，再测 `steady-state` 写入。

Linux 可以用 `/usr/bin/time -v` 粗略查看 `page fault`：

```
/usr/bin/time -v ./bench_sum
```

输出中的 `minor page faults` 和 `major page faults` 能帮助判断是否混入了页面相关成本。`minor fault` 通常不涉及磁盘 I/O，但仍然需要内核处理；`major fault` 可能涉及更重的 I/O 或换入，微基准中通常应避免。

内存分配器状态

内存分配不是固定成本。第一次调用 `allocator` 可能初始化内部结构；不同大小可能走不同分配路径；释放后的内存可能被复用；多线程程序中 `allocator` 可能有 `per-thread cache`、锁或 `arena`；大块分配可能使用 `mmap`，触页成本又在后续访问时发生。

因此，如果 `benchmark` 的目标不是研究 `allocator`，就不要在 `timed region` 中分配和释放。若目标正是研究分配器，也要把 `benchmark` 命名清楚，例如：

```
bench_vector_reserve_and_fill
bench_malloc_free_4KiB
bench_push_back_with_reallocation
```

这些名字说明被测对象包含分配。不要把包含分配的计时结果解释成纯计算 `kernel` 的性能。

系统调用和 I/O

系统调用会从用户态进入内核态，成本远高于普通函数调用，而且可能受设备、文件系统、锁、调度和权限影响。打印、写文件、读取时间、读取 `/proc`、分配大内存、线程创建、锁等待都可能引入系统路径。若这些发生在 `timed region` 中，你测到的就不再是纯用户态计算。

这不是说系统调用不能测。系统调用性能、文件 I/O、网络 I/O 都是重要主题。但它们需要端到端协议，不能和内层循环微基准混在一起。计时区域必须匹配问题。测 `write` 延迟，就把 `write` 放进 `timed region`；测 `dot product`，就不要把 `printf` 放进去。

核心理想

用户态 `benchmark` 运行在操作系统管理之下。调度、迁核、中断、缺页、分配器和系统调用都可能进入时间样本。可信测量不是假装它们不存在，而是控制、记录或把结论边界写清楚。

实验协议：把隐含条件变成文字

一段 `benchmark` 代码只定义了一部分实验。真正的实验还包括运行命令、机器状态、输入生成、重复策略、计时边界和统计口径。很多错误来自这些条件没有写下来。今天你记得自己用了 `taskset`，下周就可能忘；报告发给别人后，对方更不可能猜到。

最小协议字段

每次正式测量前，建议先写一段协议：

```
question:
    compare scalar and unrolled sum_u64 on warm-cache throughput

machine:
    CPU model and core used
```

```

compiler:
  clang++ version and full flags

binary:
  path, build command, git hash or source state

workload:
  sizes, distribution, seed, alignment if relevant

measurement:
  timer, warmup, repetitions, batch, order strategy

validation:
  reference, checksum, tolerance if floating point

artifacts:
  raw CSV, summary CSV, objdump, environment, command

```

这些字段不需要每次都写成长文，但必须能在结果目录中找到。第 16 章会把它工程化成 bench-mark suite；本章先要求你建立习惯。

运行命令是实验的一部分

性能数据必须能回到具体命令。建议保存：

```

mkdir -p results/run_001
printf '%s\n' './bench_sum --sizes 1024,16384 --repetitions 31' \
> results/run_001/command.txt
./bench_sum --sizes 1024,16384 --repetitions 31 \
> results/run_001/stdout.txt

```

如果命令依赖环境变量，也要记录：

```
env | sort > results/run_001/env.txt
```

不是所有环境变量都影响性能，但记录成本很低。至少编译器路径、动态库路径、线程库变量、OpenMP 变量、BLAS 变量、CPU 绑定变量可能影响行为。

实验记录要能解释失败

好的协议不仅解释成功结果，也帮助解释失败。如果某次结果异常，协议能告诉你：输入 seed 是否变了，batch 是否不同，二进制是否重新编译，运行是否绑定 CPU，是否在容器里，是否开启 PMU 权限，是否保存了 raw samples。没有这些记录，异常只能重新猜。

性能工程中的“可复现”不一定意味着别人能在不同机器上得到完全一样的数字，而是意味着别人能重建实验条件，理解为什么数字可能不同。跨机器结果不同很正常；条件不清楚才是问题。

测量污染：工具也会改变被测对象

工具会观察程序，也会改变程序环境。GDB、sanitizer、strace、perf、日志、断言、调试符号、插桩编译、内存检查器都会影响性能。使用工具时，要先问它适合回答什么问题。

GDB 和断点

GDB 适合观察寄存器、栈、内存和控制流，但不适合直接测性能。断点会让程序陷入调试器；单步执行完全改变时间尺度；调试构建通常关闭优化或保留更多调试信息。用 GDB 证明“这里调用了哪个函数”可以，用 GDB 单步时间证明“这个函数慢”不可以。

Sanitizer

AddressSanitizer、UndefinedBehaviorSanitizer、ThreadSanitizer 非常适合发现内存错误、未定义行为和数据竞争。但 sanitizer 会插入大量检查，改变内存布局、调用路径、分配器和时间成本。性能测试前应该用 sanitizer 验证正确性；正式性能数字通常来自非 sanitizer 的 release 构建。

报告可以写：

```

correctness build:
  clang++ -O1 -g -fsanitize=address,undefined

```

```
performance build:
clang++ -O3 -march=native -DNDEBUG
```

这说明你用工具检查过错误，但性能数字不是 sanitizer 构建的数字。

strace 和系统调用观察

strace 会拦截系统调用，开销很大。它适合回答“程序是否频繁调用系统调用”“是否打开了错误文件”“是否在 timed region 中写日志”，不适合用来获得真实运行时间。若 **strace** 显示 benchmark 每个 sample 都在 **write**，这是强烈信号：计时区域可能污染了 I/O。

perf 的开销和范围

perfstat 通常对程序总时间影响较小，适合收集进程级计数；**perfrecord** 会采样并生成 profile，开销更大，但能定位热点。二者都要注意测量范围。若 benchmark 程序包含 setup、warmup、measurement、validation 和 CSV 写入，**perfstat./bench** 默认统计整个进程，不只统计 timed region。要解释 PMU 数据，就要知道统计范围是否和时间样本一致。

一种办法是让 benchmark 提供只运行目标 kernel 的模式，或者让程序在 measurement 阶段占总时间绝大多数。更细粒度的 PMU 区间控制需要额外工具或代码，第一册先不展开。

常见误区

工具证据要和工具开销一起理解。调试工具适合证明状态，sanitizer 适合证明错误，perf 适合提供硬件线索；它们不自动给出无污染的性能数字。

测量前检查清单

在正式运行 benchmark 前，先用下面清单检查。它比直接跑程序多花几分钟，但能避免大量无效数据。

1. 问题是否写成一句具体话：比较什么、在哪个 workload、用什么指标。
2. 被测对象是否明确：函数、循环、完整程序还是系统调用。
3. 输入是否运行时生成，且 generation 不在 timed region 中。
4. correctness reference 是否存在，失败时是否会终止。
5. 结果是否被观察，能否防止 DCE。
6. setup、warmup、timed region、validation 是否分开。
7. 单个 timed region 是否足够长，是否需要 batch。
8. 编译参数是否记录，是否和 baseline 一致。
9. 是否生成反汇编并确认目标工作存在。
10. 是否记录环境、命令和 raw samples。
11. 是否知道当前环境的主要限制，例如容器、PMU 权限、频率不可控。
12. 结论准备写到什么等级：观察、趋势、解释还是工程建议。

如果其中任何一项答不上来，先修 benchmark，不要先跑大实验。性能数据一旦生成，很容易诱导你开始解释；最好在错误数字出现前就把协议补齐。

核心思想

测量前检查清单的作用，是把“可能测错”的风险提前暴露。真正节省时间的不是少写记录，而是少分析无效数据。

cache 状态会改变结论

同一个内存 kernel，在不同工作集大小下可能完全不同。

例子：

```
sum array of uint64:
n = 1024          likely L1/L2 resident
n = 1048576       likely LLC or memory bandwidth
n = 134217728     definitely memory streaming and paging pressure
```

如果只测一个 n ，你可能误以为某个优化“总是快”。实际上：

- 小数组可能受函数调用、循环开销、前端和分支影响。
- 中等数组可能受 cache bandwidth 和 latency 影响。
- 大数组可能受内存带宽、TLB、预取器和 NUMA 影响。

所以性能报告应把输入规模当作一维变量，而不是固定一个点：

```
n, median_ns, bytes, bandwidth_GBps
1024, ...
4096, ...
16384, ...
65536, ...
1048576, ...
16777216, ...
```

带宽计算：

```
bytes_read = n * sizeof(uint64_t)
bandwidth_GBps = bytes_read / seconds / 1e9
```

注意这个公式只计算源码层显式读取字节数。真实硬件可能按 cache line 传输，可能有 write allocate，可能有预取，可能有额外 page walk。后面内存层次章节会更精确。

冷缓存实验不是简单清空 cache

很多人想测 cold-cache，于是随便访问一个大数组“冲掉 cache”。这个方法可以作为近似，但不精确。现代 CPU 有多级 cache、共享 LLC、预取器、TLB、victim 行、cache replacement policy；一个大数组 sweep 未必按你想象的方式清掉所有相关状态，还可能污染内存带宽和预取器状态。

更严谨的 cold-cache 实验要定义目标：是冷数据 cache，冷 instruction cache，冷 TLB，还是首次触页？不同目标需要不同方法。比如首次触页包含 page fault 和零页映射成本；冷数据 cache 可能需要访问大于 LLC 的 buffer；冷 TLB 可能需要跨很多页面；冷 instruction cache 要关注代码路径。

所以报告中应避免含糊写“清空 cache”。可以写更具体的话：“每次测量前顺序读取一个 256 MiB eviction buffer，用于降低目标数组留在 LLC 中的概率。”这仍然不是完美清空，但描述了实际方法和限制。

TLB 和页面也属于内存状态

大数组性能不只由 cache line 决定。虚拟地址到物理地址的转换需要 TLB 缓存。若工作集跨越很多页面，TLB miss 和 page walk 可能影响性能。使用大页、访问模式、页面大小和数组跨度都会改变结果。

例如顺序扫描一个连续数组，硬件预取器和 TLB 行为通常较友好；随机访问一个大数组，即使访问字节数相同，也可能被 cache miss 和 TLB miss 主导。报告中若只写“读取了 1 GiB 数据”，还不够，还要写访问模式。

常见误区

warm-cache、cold-cache、first-touch、random-access 是不同实验。不要把它们混在一起，也不要用一个输入规模代表所有内存层次。

PMU：从时间走向硬件证据

时间告诉你“慢了多少”，PMU counter 帮你猜“为什么慢”。

Linux 下可以先用：

```
perf stat ./bench_sum
```

更具体：

```
perf stat -r 10 \
-e cycles,instructions,branches,branch-misses,cache-references,cache-misses \
./bench_sum
```

常见指标:

指标	直觉	注意
cycles	花了多少周期。	受频率、停顿和事件定义影响。
instructions	retired instructions 数。	指令少不一定快。
IPC	instructions / cycles。	高 IPC 不一定代表高性能。
branch-misses	分支预测失败。	和输入分布强相关。
cache-misses	cache miss 事件。	不同 CPU 事件语义不同。

如果权限不足，你可能看到:

No permission to enable cycles event.

这不是 benchmark 失败，而是环境限制。报告里应写清楚，并改用可用工具:

```
cat /proc/sys/kernel/perf_event_paranoid
/usr/bin/time -v ./bench_sum
```

常见误区

PMU counter 不是绝对真理。事件可能被复用，事件名称和语义依 CPU 而异，虚拟机里可能不准确。PMU 的正确用法是和源码、汇编、输入规模、时间数据一起形成一致解释。

PMU 不是性能答案机

PMU 事件能提供硬件侧线索，但它不会自动告诉你该怎么优化。看到 IPC 低，不一定说明代码差；内存带宽型 kernel IPC 可能本来就不高。看到 instructions 变少，不一定说明更快；一条复杂指令可能吞吐低或触发降频。看到 cache misses 增加，也要看 miss 代价是否在关键路径上。

正确用法是提出假设并找证据。例如你怀疑某个版本慢是因为分支不可预测，可以同时看输入分布、反汇编中的分支、**branches** 和 **branch-misses**。如果 branch-miss rate 明显上升，且时间也上升，解释更可信。若 branch misses 没变，问题可能不在分支。

再例如你怀疑某个版本是内存带宽瓶颈，可以看输入规模、GB/s、cache miss、cycles、instructions。若 n 增大后 ns/element 稳定在某个平台带宽附近，且算术指令很少，内存解释更合理。若小 n 慢、大 n 快，可能是固定开销和 cache 层级切换。

派生指标要小心

IPC、miss rate、GB/s、cycles/element 都是派生指标。它们有用，但要知道分母是什么。

```
IPC = instructions / cycles
branch_miss_rate = branch_misses / branches
cycles_per_element = cycles / n
GBps = bytes / seconds / 1e9
```

如果 **branches** 很少，branch miss rate 的一点波动可能看起来很夸张。如果 bytes 只按源码显式读写计算，真实硬件流量可能因为 write allocate、cache line 粒度、预取和 page walk 不同而更大。如果 cycles 来自不同频率或不同事件定义，跨机器比较也要谨慎。

因此，派生指标应和原始计数一起保存。报告中可以给摘要表，但原始 **perfstat** 输出也应保留，至少保存在附录或 artifact 中。

事件名称不是跨平台标准语言

cache-misses、**cycles**、**instructions** 这些通用事件在不同 CPU 上可能映射到不同底层事件，语义和精度也可能不同。更专业的分析常使用厂商文档中的具体事件名，例如某级 cache miss、load miss、uops、ports、topdown 指标等。第一册只要求认识这个限制。

如果报告跨机器比较，必须记录 CPU 型号；如果使用非通用事件，必须记录事件名称和工具版本；如果 **perf** 输出提示事件复用、权限不足或 unsupported，不能把结果当作完整硬件证据。

核心思想

PMU 的价值在于验证解释，而不是替代思考。时间、源码、汇编、输入和 counter 必须互相支持，性能结论才稳。

证据等级：从弱结论到强结论

性能报告的质量可以按证据链分级。分级不是为了形式主义，而是为了让结论和证据匹配。

等级	已有证据	允许结论
弱证据	单次计时或少量样本，没有汇编审计。	只能说“观察到一次现象”。
基本证据	重复测量、正确性校验、计时区域清楚。	可以说“在本协议下样本显示差异”。
较强证据	原始样本、统计口径、环境记录、反汇编审计齐全。	可以比较版本并说明适用边界。
强证据	以上全部，加交错顺序、输入规模扫描、PMU 或系统证据、可复现脚本。	可以提出机制解释和工程建议。

多数课堂实验至少应达到“较强证据”：正确性通过，样本保存，反汇编看过，环境记录清楚。若要解释“为什么快”，就要努力达到“强证据”：源码变化、机器码变化、时间变化和硬件事件变化互相支持。

不同问题需要不同证据

如果问题只是“我的计时代码有没有被优化器删除”，反汇编证据比 PMU 更关键。如果问题是“这个内存遍历是否进入带宽瓶颈”，输入规模、GB/s、cache/TLB 相关 counter 和硬件带宽背景更关键。如果问题是“这个服务端改动是否改善 p99 延迟”，单线程微基准就不够，需要端到端负载、并发、尾延迟和错误率。

证据等级不是固定清单，而是围绕问题选择。不要为了显得专业而堆工具，也不要因为在缺少关键证据时给出过强结论。一个只测微基准的报告可以很优秀，只要它诚实地说清微基准能证明什么、不能证明什么。

工具失败时如何降级

真实环境中工具经常失败。perfstat 可能没有权限，taskset 可能不可用，CPU governor 可能看不到，反汇编符号可能被 strip，容器里不能读取硬件信息。工具失败不是报告失败，但必须记录。

降级策略如下。若 PMU 不可用，保留时间样本、环境记录和反汇编，结论只写时间差异，不声称硬件事件原因。若反汇编符号缺失，重新用 -g 或保留符号构建，或者用地址范围和已知函数边界定位；如果仍无法审计，就降低结论强度。若 CPU affinity 不可用，增加重复次数，记录迁核风险，不把小差异写成确定结论。若环境信息不完整，报告中明确“未控制”。

常见误区

工具失败时，最坏的做法是把缺失证据当作不存在。正确做法是记录失败、降低结论强度，并说明还需要什么环境或工具才能确认。

从源码到汇编再到测量

性能测量不能只看源码。至少要审计核心循环是否存在。

命令：

```
clang++ -std=c++20 -O3 -march=native -S -masm=intel bench_sum.cpp -o bench_sum.s
clang++ -std=c++20 -O3 -march=native bench_sum.cpp -o bench_sum
objdump -drwC -Mintel bench_sum > bench_sum.objdump.txt
rg -n "sum_u64|add|vpadd|ret" bench_sum.objdump.txt
```

你要看：

- sum_u64 是否被内联。

- 循环是否还存在。
- 是否被向量化。
- 是否出现预期加载。
- `timed region` 是否包含你以为的函数。

如果函数被内联，不一定是坏事。真实 -03 性能经常依赖内联。但报告必须说明你测的是内联后的热路径，而不是一个独立函数调用。

深入理解

之后讲编译器优化时，你会看到 `scalar replacement`、`loop unrolling`、`vectorization`、`LICM`、`DCE`、`constant propagation` 等优化。`benchmark` 的可怕之处在于：这些优化可能正是你想研究的对象，也可能是把实验悄悄变成另一个实验的干扰因素。

性能数字的单位

不同 `kernel` 应选择不同单位。

单位	适用场景	例子
ns/op	小操作、函数调用、短 <code>kernel</code> 。	每次 <code>hash</code> 、每次分配。
cycles/op	微架构分析。	每元素周期数。
GB/s	内存带宽型 <code>kernel</code> 。	<code>copy</code> 、 <code>fill</code> 、 <code>sum</code> 、 <code>transpose</code> 。
GFLOP/s	浮点计算型 <code>kernel</code> 。	<code>dot</code> 、 <code>GEMM</code> 、卷积。
items/s	解析、压缩、推理 <code>token</code> 。	<code>rows/s</code> 、 <code>tokens/s</code> 。

例子：sum `uint64_t`:

```
elements = n
bytes_read = n * 8
seconds = median_ns / 1e9
GB/s = bytes_read / seconds / 1e9
ns/element = median_ns / n
```

例子：dot product:

```
for each element:
    one multiply
    one add

FLOPs = 2 * n
GFLOP/s = FLOPs / seconds / 1e9
```

要小心 FLOP 计数。`FMA` 指令 `vmadd` 通常按 2 FLOPs 计，因为它完成乘加两个浮点操作。整数算子、量化算子、`softmax`、`layernorm` 的指标要根据业务语义定义。

常见 benchmark 反模式

反模式 1：把打印放进 `timed region`

错误：

```
auto begin = Clock::now();
const auto result = kernel(input);
std::printf("%llu\n", static_cast<unsigned long long>(result));
auto end = Clock::now();
```

打印可能比 `kernel` 慢得多，也可能触发锁、缓冲、系统调用。

反模式 2：每轮重新分配

错误：

```
for (int iter = 0; iter < repetitions; ++iter) {
```

```

auto begin = Clock::now();
std::vector<float> values(size);
kernel(values);
auto end = Clock::now();
}

```

这测了 allocator 和触页。除非目标就是测分配，否则分配应放在 setup。

反模式 3：输入太小

如果 kernel 只运行几十纳秒，计时器开销、函数调用、分支、循环控制都会淹没结果。解决方法：

- 增大输入。
- 每次计时运行多个 batch。
- 单独测计时器 overhead。
- 使用硬件 counter 或专门 benchmark 框架。

反模式 4：只测一次

一次运行可能被中断、调度、缺页、降频影响。至少重复几十次，并保存原始样本。

反模式 5：比较不同编译参数

如果 A 用 `-O2`, B 用 `-O3-march=native`, 你比较的不是代码差异, 而是代码加编译策略的组合。除非这正是实验目标, 否则编译参数必须一致。

反模式 6：没有审计汇编

如果你没有看反汇编, 就不知道测的是手写循环、自动向量化循环、库函数调用, 还是被优化器删掉的空代码。

报告模板

每次实验至少提交：

```

title:
  benchmark name and purpose

machine:
  CPU model
  memory if relevant
  OS and kernel
  governor / taskset / VM status if known

compiler:
  compiler version
  full flags

source:
  commit hash or file version
  kernel signature
  reference implementation

workload:
  input size
  data distribution
  cache state
  repetitions
  warmup count

measurement:
  timer or perf command
  raw data file
  summary statistics

binary audit:
  relevant objdump excerpt
  whether function was inlined/vectorized

correctness:
  validation method

```

```
tolerance for floating point
```

```
conclusion:
  what the data supports
  what remains uncertain
```

如果报告比较两个版本，建议再加一个“证据矩阵”：

```
evidence matrix:
  correctness:
    both versions pass reference check

  timing:
    raw samples saved
    A median / B median / speedup

  binary:
    objdump checked
    inlining/vectorization status recorded

  environment:
    CPU, compiler, flags, taskset, governor recorded

  hardware:
    perf stat available or failure recorded

  conclusion boundary:
    input sizes and cache state covered
    cases not covered
```

这个矩阵的好处是强迫作者把证据链补齐。若某一行空着，结论就应相应降级。例如没有 hardware 证据时，可以比较时间，但不要断言“因为 cache miss 少了”；没有 binary 证据时，可以说“观察到 B 更快”，但不能确信优化机制和源码意图一致。

核心思想

高质量性能报告不是“我的版本更快”。它应该让别人能复现实验、审计二进制、理解误差，并判断结论的适用范围。

样本不是数字列表，而是实验轨迹

很多性能实验把样本看成一串可以直接求平均的数字。这种看法太粗糙。每个样本都是一次实验轨迹：它发生在某个时间点、某个核心、某个频率状态、某个 cache/TLB 状态、某个输入、某个二进制版本和某个系统噪声背景下。样本值只是轨迹的一个投影。若你不记录轨迹条件，后面看到波动时就只能猜。

一个最小 raw sample 不应该只有 `elapsed_ns`。建议至少包含：

```
run_id, kernel, size, batch, iteration, elapsed_ns, checksum, status
```

更完整的样本可以加入：

```
cpu_id, thread_id, order, seed, cache_mode, bytes, ops, notes
```

这些字段不是为了让 CSV 变复杂，而是为了让异常可追溯。若某个样本特别慢，你可以看它是不是第一次运行、是不是某个 size 的第一个 batch、是否 checksum 失败、是否 batch 太小、是否只发生在某个 kernel。没有这些字段，异常样本只能被粗暴删除或无视。

平均值、最小值和中位数回答不同问题

平均值容易受长尾影响。如果系统偶尔发生中断、迁核或 page fault，一个极慢样本会拉高平均值。中位数更稳健，适合表示典型运行。最小值常被用来近似“干扰最少时的潜在性能”，但它不能代表稳定表现。p95 或 p99 则说明长尾，对延迟敏感系统很重要。

因此报告不要只给一个数字。对基础 benchmark，至少给 min、median、p95、max 和样本数。若样本分布很窄，min 和 median 接近，结论更稳；若 min 很好但 p95 很差，说明系统有长尾；若

median 在两个版本间差异很小但 max 差异很大，可能说明某个版本更容易受干扰，或样本数不足。

不同指标服务不同问题。若你要评估热循环的最佳可能吞吐，min 可能有参考价值；若你要比较常规运行，median 更合适；若你要服务实时系统，p95/p99 不能忽略；若你要估计总成本，平均值也有意义。选择统计量前要先定义问题。

样本顺序本身也要保留

把样本排序后只看分位数，会丢掉时间顺序信息。时间顺序能暴露 warmup、降频、热节流、内存分配器状态变化、后台任务周期性干扰。一个版本前半段快后半段慢，和所有样本随机波动，是两种不同问题。

报告中可以保留原始顺序，并在分析时画或检查 `iteration->elapsed_ns`。若你看到样本随时间逐渐变慢，要考虑温度和频率；若每隔固定次数出现慢样本，要考虑周期性系统活动或 harness 中的批处理；若第一个样本总是慢，要考虑首次触页、cache 预热、动态链接或分支预测状态。

误差来源：随机噪声、系统偏差和模型错误

性能测量的不确定性不只来自随机噪声。至少有三类误差。

第一类是随机噪声。它来自调度、中断、cache 状态、分支历史、频率微小波动等因素。重复测量和稳健统计可以降低随机噪声影响，但不能消除所有长尾。

第二类是系统偏差。它来自实验设计固定地测错了东西。例如把 setup 放进 timed region，却把结论写成 kernel 性能；比较两个版本时使用不同编译参数；输入数据对某个版本更友好；计时器开销没有扣除；batch 自动选择策略对两个版本不同。重复一千次也不会修复系统偏差，因为你重复的是同一个错误实验。

第三类是模型错误。它来自解释层面。例如看到时间变慢就归因于 cache miss，但没有硬件证据；看到指令数减少就断言一定更快，却忽略内存瓶颈；把微基准结论推广到端到端应用；把单线程 warm-cache 结果推广到多线程 cold-cache。模型错误会让正确数据支持错误决策。

心智模型

重复测量主要对抗随机噪声；实验协议对抗系统偏差；证据等级和边界说明对抗模型错误。三者缺一不可。

混杂因素要么控制，要么记录，要么承认

混杂因素是同时影响自变量和结果的外部因素。比如你想比较两个实现 A 和 B，但 A 总是在前面运行，B 总是在后面运行；如果 CPU 后面降频，B 会显得慢。运行顺序就是混杂因素。解决方法可以是随机化顺序、交替运行、分块运行，或者至少记录顺序并在报告中承认限制。

另一个例子是输入数据。若 A 处理全零数组，B 处理随机数组，你测到的差异可能来自分支、压缩、cache 或特殊值，而不是实现。输入分布必须作为 workload 参数记录。若实验目标就是比较不同分布，也要把分布放进矩阵，不要混在实现差异里。

硬件状态也是混杂因素。CPU governor、温度、SMT、后台负载、容器限制、NUMA 位置都会影响结果。你不一定能完全控制它们，但至少要记录。无法控制的因素会降低结论等级，而不是让报告无效。专业报告的价值在于诚实说明控制程度。

计时源的选择要匹配问题

`std::chrono::steady_clock`、`clock_gettime`、TSC、性能计数器 `cycle`、benchmark 框架内置计时器，都可以用于测量，但它们回答的问题不完全相同。选择计时源时，要问：你需要墙钟时间、线程 CPU 时间、核心周期、还是硬件事件计数？

墙钟时间包含线程被调度出去的时间。对于用户感知延迟，这是合理的；对于微内核吞吐，它会把 OS 噪声也算进去。线程 CPU 时间排除部分被调度出去的时间，但不一定反映真实延迟，也不直接等于硬件执行周期。TSC 提供低开销时间戳，但需要理解 invariant TSC、频率换算、跨核心同步和序列化问题。PMU cycles 更接近核心执行，但事件语义和权限复杂。

低开销计时不等于高可信

TSC 读取开销低，所以常用于短代码测量。但低开销不自动意味着高可信。若没有足够大的 batch，读 TSC 的开销仍会占比显著；若没有合适的序列化，乱序执行可能让测量边界模糊；若线程迁核，

跨核心 TSC 行为虽在现代机器上通常可用，但仍应记录绑定策略；若 CPU 频率变化，TSC ticks 与 core cycles 的关系也要说明。

反过来，`steady_clock` 开销较高，但对于毫秒级或微秒级 batch 已经足够。初学阶段优先使用简单可靠的墙钟计时，并通过 batch 把被测工作做长，通常比直接写复杂 TSC harness 更好。只有当问题要求纳秒级或 cycle 级分析时，再引入更精细计时源。

空测量只能估计框架成本

空测量常用于估计计时框架成本：

```
begin = now()
do nothing
end = now()
```

或：

```
begin = now()
for batch:
    empty_operation()
end = now()
```

但空测量不是万能修正项。真实 kernel 会改变寄存器压力、分支、cache、流水线和编译器优化边界，不能简单用“测量值减空值”得到真成本。空测量主要用于 sanity check：计时器开销是否远小于被测时间，batch 是否足够大，单位换算是否正常。若空测量与真实测量同量级，应该增加 batch 或换方法，而不是机械相减后相信结果。

PMU 证据要有假设驱动

`perfstat` 很容易让报告看起来专业，但 PMU 事件必须服务假设。先写假设，再选事件。

若假设是“分支不可预测导致变慢”，应关注 `branches`、`branch-misses`、`cycles`、`instructions`，并结合输入分布。若假设是“内存带宽限制”，应关注时间、bytes 口径、`cache misses`、可能的内存控制器事件，并对比 copy 带宽。若假设是“向量化减少指令数”，应关注 `instructions`、`cycles`、反汇编中的向量指令。若假设是“TLB 压力”，应选择 TLB 相关事件，并设计跨页面工作集实验。

不要先收集一大堆事件，再挑看起来符合故事的数字。PMU 事件有平台差异、复用误差、权限限制和名称差异。某些通用事件在不同 CPU 上含义不完全相同，某些事件在虚拟机或容器里不可用。报告中应写清事件列表、命令、是否发生 multiplexing、是否有权限不足、是否重复测量。

instructions 少不一定快，cycles 少才接近时间

`instructions` 下降通常是好信号，但不保证时间下降。一个版本指令更少，却可能引入长延迟 load、除法、分支 miss 或依赖链；一个版本指令更多，却可能有更高并行度或更少 cache miss。`cycles` 更接近执行成本，但也受频率、调度和测量范围影响。最好结合 `instructions`、`cycles`、时间和反汇编一起看。

IPC 也不能孤立解释。高 IPC 可能说明执行单元利用好，也可能只是运行了大量简单指令；低 IPC 可能说明内存等待，也可能说明分支、依赖或前端问题。第一册不深入 top-down 微架构分析，但要先建立一条规则：派生指标只能辅助解释，不能替代对 workload 和二进制的理解。

硬件事件和源码行没有天然一一对应

采样或计数器告诉你某段机器码发生了事件，不会自动告诉你哪行源码负责。优化、内联、循环展开、向量化、尾调用都会改变源码到机器码的映射。若报告把 `perf` 事件直接归因到某行 C++，必须说明调试信息、反汇编和内联上下文。否则应更谨慎地说“事件集中在某个机器循环”。

性能结论的降级规则

为了避免过度表达，可以按证据缺口给结论降级。

如果没有 `correctness`，只能说“计时观察”，不能说“实现更快”。如果没有二进制审计，只能说“该构建产物在该命令下耗时如何”，不能解释为某个源码结构更快。如果没有 raw samples，只能说“summary 显示”，不能判断稳定性和长尾。如果没有环境记录，不能推广到其他机器。如果没有 PMU 或二进制证据，不能断言具体硬件机制。如果样本差异小于噪声，不能给出胜负结论。

可以使用下面模板：

```
Evidence present:
  correctness check passed
  raw samples saved
  objdump audited

Evidence missing:
  no PMU events
  CPU frequency not pinned

Allowed conclusion:
  version B is consistently faster in median time under this harness

Not allowed:
  B is faster because cache misses are lower
  B will be faster on all x86-64 machines
```

这种降级规则能保护报告。它不是保守到不敢下结论，而是让每个结论正好落在证据能支撑的范围内。

可复现实验目录

性能实验最好把产物组织成目录，而不是散落在终端输出里。一个简单目录可以是：

```
results/2026-06-14-sum-u64/
  command.txt
  environment.txt
  build.txt
  raw_samples.csv
  summary.csv
  objdump.txt
  perf_stat.txt
  report.md
  notes.txt
```

command.txt 保存完整运行命令；**environment.txt** 保存 CPU、OS、编译器、governor、taskset、git commit；**build.txt** 保存编译命令和 flags；**raw_samples.csv** 保存每个样本；**summary.csv** 保存汇总；**objdump.txt** 保存关键二进制；**perf_stat.txt** 保存硬件事件；**report.md** 写结论和限制；**notes.txt** 记录异常、失败和人工观察。

这个目录结构的价值在几周后才会显现。你能比较不同日期结果，知道某次数字来自哪个 commit；你能复查某个异常点，知道当时是否 PMU 权限不足；你能发现编译器升级后汇编形态变化，而不是误以为源码优化了。

失败实验也要保存

失败实验同样有价值。checksum 失败说明候选实现有 correctness bug；PMU 权限不足说明硬件证据缺失；样本波动太大说明环境不适合小幅结论；反汇编显示函数被删除说明 harness 错了。若只保存成功结果，团队会重复踩同样坑。

失败记录可以很短，但要写清：

- 失败发生在哪个命令。
- 失败现象是什么。
- 哪些产物仍然可用。
- 当前结论是否作废或降级。
- 下一步如何修复。

这和普通软件测试类似。性能实验也需要失败可诊断。

可信测量案例库

测量原则如果只停留在口号层，很难迁移到真实工程。下面几个案例展示同一条规则如何在不同场景中发挥作用。

案例一：短函数一次调用

你想测一个函数：

```
extern "C" int add2(int a, int b)
{
    return a + b;
}
```

若直接写：

```
auto begin = Clock::now();
int r = add2(1, 2);
auto end = Clock::now();
```

这几乎没有意义。一次整数加法太短，计时器开销、函数调用、编译器内联、常量折叠、进程状态都会主导结果。可信做法应先明确问题：你是测函数调用边界，还是测加法吞吐？如果测函数调用，需要阻止内联、使用运行时输入、重复大量 batch，并确认二进制中仍有调用。如果测加法吞吐，需要设计依赖链或独立加法流，并用 cycle 级方法或足够长 batch。

这个案例的结论是：短函数不能靠单次计时。短函数测量必须先放大工作、控制优化器、审计二进制，再讨论时间。

案例二：顺序内存扫描

你想测：

```
std::uint64_t sum_u64(const std::uint64_t* p, std::size_t n);
```

这里关键变量是工作集大小。若 n 很小，数据在 L1/L2 中，循环开销、展开、向量化和依赖链可能重要；若 n 很大，内存带宽可能主导。单测一个 n 无法说明整体。可信做法是扫描多个规模，计算 ns/element 和 GB/s，记录 cache mode，并保存反汇编确认循环形态。

如果大规模 GB/s 接近内存 copy baseline，继续减少一两条整数指令可能没有收益；如果小规模明显慢，可能是函数调用、batch 或循环开销。这个案例说明，输入规模不是附属参数，而是性能问题本身的一部分。

案例三：数据相关分支

考虑：

```
extern "C" int count_positive(const int* p, std::size_t n)
{
    int c = 0;
    for (std::size_t i = 0; i < n; ++i) {
        if (p[i] > 0) {
            ++c;
        }
    }
    return c;
}
```

如果输入全是正数，分支很容易预测；如果正负随机混合，分支可能难预测；如果数据有长段模式，预测器可能适应。三个输入分布对应三个不同实验。报告如果只写“count_positive 速度是多少”，没有意义。必须写 distribution：全正、全负、交替、随机、块状、真实数据。

若你声称 branchless 版本更快，需要比较多个分布。branchless 可能在随机输入更稳，在全正输入反而不如简单分支。这个案例说明，数据分布是 workload，不是噪声。结论必须谨慎。

案例四：冷启动和稳态混淆

命令行工具的端到端时间包括进程启动、动态链接、配置读取、输入解析、内存分配、核心计算、输出和清理。若你用 `/usr/bin/time./tool` 比较两个核心函数版本，而核心计算只占 5%，总时间变化可能看不出来；或者启动噪声掩盖了真实差异。

可信做法是把问题分成两层。端到端实验测用户感知总时间，保留启动和 I/O；微基准测核心函数，排除启动和 I/O。两个实验都可以有价值，但不能混用。报告应明确写“这是端到端 wall time”还是“这是进程内 kernel time”。

案例五：工具扰动

GDB、sanitizer、**strace**、**perf** 都会改变被观察对象。GDB 断点会改变执行节奏；ASan 会改变内存布局并插入检查；**strace** 会拦截系统调用；**perfstat** 通常扰动较小但仍有开销。工具扰动不是说工具不能用，而是要让工具回答合适问题。

如果目标是找 use-after-free，ASan 很合适；如果目标是发布版吞吐，ASan 构建不能代表最终性能。如果目标是确认程序打开了哪个文件，**strace-eopenat** 很合适；如果目标是纳秒级函数成本，**strace** 完全不合适。工具选择要匹配问题。

异常数字排查流程

性能实验中，异常数字分两类：异常快和异常慢。两者都需要排查。

异常快时，优先检查：

1. 结果是否被使用，循环是否被删除。
2. 输入是否是编译期常量，计算是否被常量折叠。
3. batch 是否为 0 或单位换算是否错误。
4. baseline 是否被不公平削弱。
5. candidate 是否少处理了元素或跳过了校验。

异常慢时，优先检查：

1. 是否首次触页、冷 cache、动态链接或初始化。
2. 是否发生调度、中断、迁核或频率下降。
3. 是否输入规模跨过 cache/TLB 容量边界。
4. 是否走到错误路径、日志路径、分配路径或 fallback。
5. 是否编译参数、ISA dispatch 或库版本改变。

排查时先看 raw samples，不要只看 summary。异常是单个样本、连续一段样本，还是某个 size 全部样本？单个样本可能是系统噪声；连续变慢可能是温度或频率；某个 size 异常可能是工作集边界或代码路径变化。

异常不能偷偷删除

可以使用 median 减少异常值影响，但不能无记录删除异常样本。若决定排除样本，应写清规则，例如“排除 checksum_failed 样本”“排除 status 非 ok 样本”“保留所有 ok 样本，即使很慢”。不要根据结果好不好手工挑选。规则越早固定，结论越可信。

异常样本往往最有价值。它们可能暴露长尾延迟、内存分配抖动、调度干扰、系统调用、页错误或输入边界。高质量报告会解释异常，而不是把异常清理到看不见。

测量设计的审查问题

开始 benchmark 前，先回答下面问题：

1. 我测的是端到端、组件、函数调用还是核心循环。
2. 被测对象的 correctness 如何验证。
3. 输入规模和分布为什么代表问题。
4. 计时区域包含什么，排除了什么。
5. 编译器是否可能删除或改写被测工作。
6. 二进制是否经过审计。
7. 样本数、warmup 和 batch 是否足够。
8. 环境变量、CPU 绑定、governor 和库版本是否记录。
9. raw data 保存在哪里。
10. 结论会限制在哪些条件下。

如果这些问题答不上来，先不要跑大实验。测量设计不清，跑得越久，错误数据越多。

综合案例：一份可信测量小报告

下面是一份简化但合格的报告骨架，展示如何把本章原则落到文字。

Question:	Compare scalar_sum and unrolled_sum for warm-cache sum_u64.
Machine:	Linux x86-64, CPU model recorded in environment.txt.
Build:	clang++ -O3 -march=native, commit recorded, worktree dirty state saved.
Correctness:	Both kernels compared against reference for every input size. checksum stored in raw_samples.csv.
Workload:	sizes = 2 KiB, 32 KiB, 512 KiB, 64 MiB data = deterministic random uint64, seed = 1 cache mode = warm-cache
Measurement:	5 warmup runs 31 measured repetitions batch chosen so each sample is at least 100 us timed region contains only repeated kernel calls and checksum accumulation
Binary:	objdump saved. scalar_sum is a simple scalar loop. unrolled_sum processes four elements per loop iteration.
Data:	raw_samples.csv and summary.csv saved. summary reports min, median, p95, max, ns/element, GB/s.
Conclusion:	unrolled_sum improves median ns/element for 32 KiB and 512 KiB. 64 MiB results are similar, likely because memory bandwidth dominates.
Limits:	single machine only no PMU events yet warm-cache only

这份报告没有使用复杂统计，但已经满足基本可信性。它说明了问题、机器、构建、正确性、workload、计时协议、二进制证据、数据和限制。读者可以复查 raw 数据，也知道哪些结论不能推广。

不合格报告长什么样

不合格报告通常写成：

优化版快 20%，效果很好。

它缺少几乎所有关键信息：哪个优化版、哪个 baseline、哪个输入、怎么计时、是否正确、是否审计二进制、样本是否稳定、机器是什么、是否有异常点、能否复现。这样的文字不能作为工程决策依据。

自测清单：测量是否可信

完成本章后，应能回答：

1. 被测对象是端到端、组件、函数还是核心循环。
2. correctness 如何验证，失败时是否停止性能结论。
3. 计时区域包含什么，排除了什么。
4. 编译器是否可能删除、内联或常量折叠被测工作。
5. batch、warmup、repetitions 为什么足够。

6. raw samples 是否保存, summary 统计量是什么。
7. 输入规模、数据分布和 cache mode 是否记录。
8. 环境、构建命令和 Git 状态是否记录。
9. 二进制证据是否支持你声称的热路径。
10. 结论是否区分事实、推测和限制。

如果一个 benchmark 不能回答这些问题, 它可能仍然是有用的探索, 但还不能成为可信报告。

深入讲解: 测量是一种工程论证

性能测量不是把计时器放在代码两端, 而是构造一个论证。论证的结论可能是“版本 B 在某条件下更快”, 前提包括 correctness、同一输入、同一构建策略、稳定样本、二进制证据和环境记录。任何前提缺失, 结论都要降级。

这和数学证明不同。性能论证很少能做到绝对证明, 因为硬件和系统环境复杂。但它可以做到足够可信: 实验变量清楚、证据链完整、限制明确、他人可复查。工程上, 可信的有限结论比夸大的普遍结论更有价值。

例如“B 快 10%”是弱论证; “在这台机器、这个编译器、这些输入规模、warm-cache 协议下, B 的 median ns/element 比 A 低 8–12%, raw 样本稳定, 反汇编显示 B 减少了循环分支, correctness 通过; 尚未验证 cold-cache 和其他 CPU”是强得多的论证。它没有假装普遍, 却足以支持下一步工程判断。

深入讲解: 为什么正确性失败时不能谈性能

错误实现经常更快。少处理尾部会快, 跳过边界检查会快, 使用未定义行为会快, 不写输出会快, 提前返回会快, 溢出后分配小缓冲区也可能看起来快。若 correctness 不通过, 性能数字只是在测另一个问题。

这条规则在优化竞赛和实际工程中都很重要。任何 speedup 报告都应先给 correctness 证据。对于整数结果, 可以 bit-exact; 对于浮点结果, 要给误差模型; 对于输出 buffer, 要检查边界和全部元素; 对于非确定性算法, 要定义统计正确性或不变量。没有正确性定义, 就没有性能对象。

有时 correctness check 很贵, 不能放在每次 timed region 内。这不等于是可以取消。可以在计时前后检查, 可以抽样检查, 可以 nightly 全量检查, 可以用 checksum 防止删除。但报告必须说明策略和风险。

结论应该怎样写

性能报告的结论要克制。不要把一次实验写成普遍真理, 也不要猜测写成事实。一个合格结论通常包含四个部分: 实验条件、主要观察、解释假设、适用边界。

弱结论:

优化后更快。

较好结论:

在 i7-12700K、clang 18、-O3 -march=native、taskset 绑定 CPU 2、warm-cache、n = 1048576 的 sum_u64 实验中, 优化版本 31 次重复的 median 从 0.42 ms 降到 0.31 ms, min 从 0.40 ms 降到 0.30 ms。
反汇编显示优化版本使用 AVX2 向量加法, 原版本为标量循环。
perf stat 显示 instructions 明显下降, branch 数接近不变。
该结论暂不推广到 cold-cache、大 n 内存带宽瓶颈或非 AVX2 机器。

这样的结论虽然长, 但它说明了数字来自哪里、为什么可能变快、哪些情况还没证明。工程沟通中, 这比一句“快 30%”更有价值。

区分事实、推测和行动

报告中建议显式区分三类语句。

事实来自实验记录: 编译命令、样本数据、反汇编、PMU 输出、正确性校验结果。例如“median 为 0.31 ms”, “核心循环包含 vpaddq”。

推测是对事实的解释。例如“速度提升可能来自向量化减少了循环迭代次数和指令数”。推测应使用“可能”“一致”“支持”这类词，除非有更强证据。

行动是下一步计划。例如“下一步扫描更大 n ，判断是否进入内存带宽瓶颈”，“下一步加入 cold-cache 实验”，“下一步比较 AVX2 和 AVX-512 版本”。行动应直接对应当前不确定性。

什么时候说结论不成立

高质量报告也要敢于说“不能得出结论”。如果样本波动大于版本差异，不能说 A 快于 B；如果两个版本编译参数不同，不能把差异归因于源码；如果正确性校验失败，不能讨论性能；如果反汇编显示被测函数被删除，实验无效；如果 PMU 权限不足，就不能声称硬件事件支持某解释。

承认结论不成立不是失败。它说明测量体系在保护你不做错误决策。性能工程里，排除错误结论和找到优化机会同样重要。

常见误区

不要把 benchmark 写成宣传文案。报告的任务是让结论可审查，而不是让数字看起来最大。

课堂讲解：测量前先写假设

很多 benchmark 的问题不是计时器不够精确，而是根本没有假设。作者只写“测一下 A 和 B 谁快”，然后看到数字再补故事。更可靠的方式是在测量前写下假设：你认为哪个版本会在哪些 workload 上更快，原因是什么，应该在汇编或硬件事件中看到什么证据，哪些结果会推翻你的想法。

例如你认为展开循环会更快，假设可以写成：“对于中等规模数组，展开减少循环分支和归纳变量更新，median ns/element 应下降；对于超大规模 streaming，收益可能消失，因为内存带宽支配；反汇编应显示每次循环处理更多元素；branch 指令数可能下降。”这个假设给实验设计提供方向：必须扫描规模，必须保存反汇编，最好记录分支相关事件。

若结果和假设相反，也不是坏事。小规模变慢可能说明固定开销增加；大规模持平可能说明瓶颈转移；所有规模变慢可能说明寄存器压力或编译器已经做了更好优化。提前写假设能防止事后解释过度灵活。一个可以被推翻的假设，比一个永远能圆回来的故事更有工程价值。

测量边界：端到端、组件和 microbenchmark

性能测量有不同边界。端到端测量回答用户体验或整体吞吐，包含启动、I/O、调度、内存分配、日志、网络和业务逻辑。组件测量回答某个模块在真实上下文中的成本。microbenchmark 测量某个小 kernel 或函数的局部行为。三者都重要，但结论不能互相替代。

microbenchmark 的优势是可控，适合研究指令、数据布局、分支和局部算法。缺点是容易脱离真实 workload，忽略调用开销、cache 状态、输入分布和系统干扰。端到端测量真实，但变量多，难以归因。组件测量介于两者之间，需要小心切分边界。

设计实验前要写清边界。如果目标是优化一个热循环，microbenchmark 可以先探索；如果目标是减少请求延迟，端到端必须验证；如果目标是替换库内部实现，组件 benchmark 和公共 API benchmark 都要有。只用 microbenchmark 证明产品变快，或者只用端到端数字解释某条指令优化，都是层次错位。

异常结果处理流程

异常结果包括快得不合理、慢得不合理、样本波动突然变大、某个 size 出现尖峰、checksum 偶发失败、两次运行趋势相反。遇到异常，不要先挑一个喜欢的解释。按流程处理。

第一，确认数据完整。raw samples 是否包含预期 repetition，单位是否正确，CSV 是否错列，summary 是否读错 schema。第二，确认 correctness。异常快尤其危险，可能是被测工作被优化删除、提前返回、没有写输出或 candidate 算错。第三，确认 timed region。是否把 setup 放进去了，是否把校验放进去了，是否某个版本多做了分配或日志。

第四，确认二进制。关键函数是否被内联、删除、向量化、展开，是否调用了不同实现，是否因为编译参数变化使用了不同 ISA。第五，确认运行环境。CPU 频率、调度、后台任务、热状态、权限、NUMA 和容器限制都可能影响。第六，若前五步不能解释，再提出硬件层假设，如 cache、TLB、分支预测或带宽。

这个顺序很重要。很多人看到尖峰就直接说“cache miss”，但后来发现 CSV 里某个样本包含了输入生成，或者 candidate 在那个 size 上 checksum 失败。先排除实验系统错误，再解释硬件现象。

样本数量和统计量的工程含义

样本数量不是越多越好，而是要足以支持问题。若单次运行很稳定，31 次 repetition 可能足够观察 median 和异常值；若系统噪声大，更多样本也许仍不能解决，需要改环境或实验设计。若被测工作太短，样本多也可能只是在反复测计时器开销，需要 batch 或扩大工作量。

统计量也要服务结论。min 常接近理想无干扰情况，median 表示典型样本，p95 反映尾部，mean 容易受异常值影响但对总成本有意义。报告应说明使用哪个统计量，为什么。若你用 min 声称性能提升，读者会怀疑只挑了最好情况；若你用 mean 而样本有长尾，应解释长尾来源。

比较两个版本时，不要只看单个百分比。应看 raw 分布是否重叠，差异是否大于噪声，异常值是否集中在某个版本，趋势是否在多个 size 上一致。第一册不展开复杂统计检验，但要求读者具备基本克制：小于噪声的差异不应写成确定结论。

测量报告审查清单

一份测量报告提交前，可以按下面清单自查：

1. 问题陈述是否说明比较对象和目标 workload。
2. 是否先给 correctness 证据。
3. 是否明确测量边界：端到端、组件还是 kernel。
4. timed region 是否干净，setup 和校验是否在外部。
5. 是否保存 raw samples，而不是只有截图。
6. summary 的统计量和单位是否说明。
7. 输入规模、分布、seed、cache mode 是否记录。
8. 构建命令、编译器、CPU、系统环境是否记录。
9. 反汇编是否支持你声称的代码路径。
10. 结论是否区分事实、解释、限制和下一步。

这份清单和第 16 章的 suite 设计相连。第 15 章先让一次实验可信，第 16 章再把可信实验组织成可维护系统。若一次实验都不可信，自动化只会更快地产生不可信数字。

测量和优化决策的伦理

性能数字常被用来推动决策，因此报告作者有责任避免误导。误导不一定来自恶意，更多来自省略条件、选择最好一次、隐藏失败样本、只展示有利 workload、把猜测写成事实。底层工程中的“诚实”不是道德口号，而是项目风险控制。错误性能结论会让团队合入复杂代码、破坏可维护性、忽略正确性问题，甚至影响产品容量规划。

诚实报告应主动展示不利信息。若 candidate 在某些 size 变慢，要写出来；若样本波动大，要写出来；若 PMU 权限不足，要写出来；若只测了单机器，要写出来；若 correctness 只做了抽样检查，也要写出来。限制写得清楚，不会削弱报告，反而提高可信度。

优化决策还要尊重成本。一个手写汇编版本可能快百分之三，但需要更多平台测试、更多 ABI 审计和更多维护知识；一个纯 C++ 版本可能略慢，但更容易由编译器适配未来机器。性能报告应把这些成本列为决策输入。工程不是追求单一数字最小，而是在目标、风险和资源之间做选择。

重复实验和独立复核

一次实验最好能被重复，重要结论最好能被独立复核。重复实验是同一作者在相同条件下再次运行，检查结果是否稳定。独立复核是另一个人按报告中的命令和数据重新运行，检查是否能得到相近结论。两者都能发现隐藏前提。

为了支持复核，报告要避免“本机路径”“手工操作”“临时环境”这类不可见条件。命令应尽量脚本化，输入应可下载或可生成，环境差异应记录。若某些条件无法共享，例如真实生产数据或专用机器，也要提供摘要和替代 workload。不能复核的结果可以作为探索，不能作为强决策依据。

复核还会提高写作质量。当你知道别人会按报告执行，就会自然补齐缺失命令、路径和版本。底

层能力的一部分，就是把自己的观察写到别人能检查的程度。

测量失败案例库

建议每个读者建立自己的测量失败案例库。里面不只放成功优化，也放无效实验：被优化器删掉的循环、计时区域包含初始化、输入太小导致测到函数调用开销、样本被后台任务污染、baseline 和 candidate 编译参数不同、checksum 偶发失败、动态链接加载旧库、图表单位写错、summary 脚本读错列。

失败案例库的价值在于形成预警。下次看到“快得离谱”的数字，你会先怀疑是否被删除；看到某个 size 尖峰，你会先查 raw 和环境；看到跨机器差异，你会先比对二进制和库。经验不是只来自成功结论，也来自被证据推翻的错误判断。

这也是本章对“高质量”的定义。高质量不是永远得到漂亮 speedup，而是能识别无效实验、能解释失败、能保护团队不被错误数字带偏。

测量和代码审查的连接

性能改动进入 review 时，测量报告就是代码审查材料的一部分。reviewer 不只看代码是否正确，还要看性能主张是否可信。若 PR 标题写“优化 sum”，报告至少应说明优化对象、输入矩阵、正确性、二进制差异、样本和限制。没有这些，reviewer 只能审查代码风格，无法审查性能主张。

作者也应把测量失败写进 PR。若某些 size 没有提升，说明；若 PMU 数据无法采集，说明；若结果只在本机稳定，说明。这些信息帮助团队决定是否合入、是否需要 feature flag、是否需要更多机器验证。性能 review 的目标不是证明作者厉害，而是保护项目做正确决策。

测量章节的综合训练

综合训练可以选一个很小的优化，例如把单累加器求和改成四个累加器。报告先写假设，再写实验协议，再写 raw 和 summary，再写反汇编，最后写结论限制。若结果不符合预期，也要解释可能原因并设计下一步。

这个训练的评分重点不是 speedup 大小，而是实验是否可信。一个没有提升但证据完整的报告，比一个快很多却无法复现的报告更合格。性能工程首先是可信论证，其次才是优化技巧。

测量结论的生命周期

性能结论不是永久真理。编译器升级、CPU 更换、输入分布变化、库版本变化、代码重构、操作系统策略变化，都可能让旧结论失效。报告应写明结论日期、版本和适用条件。长期项目应定期复跑关键 benchmark，尤其是在工具链升级和硬件迁移时。

结论生命周期还影响文档。不要在代码注释中写“这个版本最快”这种没有条件的话。更好的写法是“在某次报告中，该实现对某 workload 更快，见结果目录”。这样未来如果条件变化，团队知道去哪里复核，而不是被过期注释误导。

把测量失败转成测试

当一次测量失败暴露出 harness 问题，应尽量把它转成测试。发现计时区域包含分配，就加测试或代码结构防止 timed region 调用 allocator；发现 summary 公式错，就加固定输入的 summarizer 测试；发现 checksum 失败还继续汇总，就加状态处理测试；发现编译器删除工作，就加二进制审计或防优化检查。

性能工程也需要回归测试。不是所有性能差异都能在 CI 中精确阻断，但 harness 正确性、数据格式、结果状态和基本 smoke 可以自动保护。把失败转成测试，suite 才会越来越可信。

案例：一个快得离谱的 benchmark

假设某个 PR 声称把求和函数优化了十倍。第一反应不应是庆祝，而应检查无效实验。先看 correctness：结果是否和 reference 一致，是否覆盖非零输入、大输入和边界输入。再看反汇编：循环是否仍存在，结果是否被使用，函数是否被常量折叠。再看计时区域：是否只测了空循环，是否把输入规模传错，是否把 batch 公式除多了。再看 summary：单位是否写错，ns 和 us 是否混淆，ops 是否溢出。

许多“十倍优化”最后来自这些错误。真实的十倍提升并非不可能，但越大的 speedup 越需要更强证据。报告可以把检查过程写清：哪些无效原因已排除，哪些证据支持真实提升，哪些条件还没验证。这样即使结论最终成立，也不会建立在侥幸上。

这个案例还说明，异常好结果和异常坏结果一样需要排查。工程师不能只怀疑坏消息，也要怀疑过分漂亮的好消息。可信测量的目标不是证明自己想要的结论，而是让数据经得起复查。

从一次测量到行动建议

测量报告的最后应给行动建议，而不只是数字。建议可能是合入、拒绝、继续实验、扩大 workload、回滚、加测试、调整阈值或等待更多数据。行动建议要对应证据。例如 correctness 失败，建议应是先修正确性；样本波动大，建议应是改实验环境；只有中等规模提升，建议应是按生产输入决定是否 dispatch；二进制没有变化，建议应是检查构建系统。

行动建议让测量进入工程闭环。没有建议，报告只是记录；有建议，团队才能决策。建议也可以是“不行动”，只要理由清楚。比如差异小于噪声，维护成本高，就可以建议不合入优化。性能工程要服务项目目标，而不是制造无休止实验。

测量结论如何在代码中留下痕迹

性能结论不应只停留在报告里。若某个实现因为特定 workload 被选择，代码注释或文档应链接到对应报告；若某个优化被拒绝，也应保留原因；若某个阈值来自历史噪声，应记录计算依据。这样后续维护者看到复杂实现时，知道复杂性来自哪里，也知道什么时候需要重新验证。

但代码注释不要写成无条件真理。不要写“这个版本最快”，而应写“在某次报告的指定 workload 下，该版本优于 baseline”。一旦编译器、CPU、输入分布或实现改变，旧结论就需要复查。性能知识有生命周期，代码应保存证据入口，而不是把过期结论伪装成永久规则。

测量章节的毕业问题

完成本章后，读者可以用下面问题自测：如果一个 benchmark 显示快百分之五，你能否判断这个差异是否大于噪声；如果显示快十倍，你能否排除工作被删除；如果两个版本结果相反，你能否检查输入、构建和环境；如果 PMU 事件支持某个假设，你能否回到反汇编指出对应代码；如果 correctness 失败，你能否停止性能结论。

这些问题比记住某个计时 API 更重要。计时 API 会变，硬件会变，工具会变；可信论证的方法不会变。第 16 章会把这种一次实验的方法扩展成长期 suite。

实验：修复一个会被优化器删除的 benchmark

目标：亲手观察 dead-code elimination 和防优化工具的作用。

创建 labs/ch15_measurement/lab15_1_dce/bench_bad.cpp：

```
#include <stdint>
#include <vector>

int main()
{
    constexpr std::uint64_t LOOP_LIMIT = 1000000000ULL;
    std::uint64_t sum = 0;
    for (std::uint64_t i = 0; i < LOOP_LIMIT; ++i) {
        sum += i;
    }
}
```

任务：

1. 用 -O0、-O2、-O3 分别编译。
2. 用 objdump-drwC-Mintel 检查循环是否存在。
3. 加入返回值打印，再看汇编是否变成公式或仍保留循环。
4. 把输入上限改成运行时参数，例如从 argv 读取，再观察。
5. 加入 do_not_optimize(sum)，解释汇编变化。

命令：

```
clang++ -std=c++20 -O0 bench_bad.cpp -o bench_00
clang++ -std=c++20 -O3 bench_bad.cpp -o bench_03
objdump -drwC -Mintel bench_03 > bench_03.objdump.txt
rg -n "add|loop|main" bench_03.objdump.txt
```

交付物：

1. 三个优化级别的反汇编关键片段。
2. 哪个版本保留循环，哪个版本删除或改写循环。
3. 用 C++ 抽象机可观察行为解释为什么优化器有权这么做。
4. 修复后的 benchmark。

实验：写一个最小可信 sum benchmark

目标：实现本章最小 harness，并形成第一份性能报告。

要求：

1. 使用 `std::chrono::steady_clock`。
2. setup、warmup、timed region、validation 分开。
3. 输入规模至少包含：

```
1024
16384
262144
1048576
16777216
```

4. 每个规模 warmup 5 次，正式测量 31 次。
5. 输出 CSV：

```
n,min_ns,median_ns,max_ns,bytes,bandwidth_GBps
```

6. 用 `objdump` 审计核心循环。

交付物：

1. 源码和编译命令。
2. CSV 原始输出。
3. 一张结果表。
4. 汇编核心循环标注。
5. 500 字报告：随着 `n` 增大，瓶颈可能如何变化。

实验：比较 `steady_clock` 与 TSC

目标：理解时间源差异，而不是盲目相信某一个计时器。

任务：

1. 对同一个 `sum_u64` kernel，同时记录 `steady_clock` 纳秒和 TSC delta。
2. 用 `rdtsc` 裸读一次。
3. 用 `lfence` 包围 `rdtsc` 再读一次。
4. 每种方式重复 100 次，保存原始数据。
5. 检查线程是否迁核；如果会迁核，用 `taskset` 绑定。

交付物：

1. 计时代码。
2. 原始数据。

3. min、median、max 表。
4. 解释 TSC delta 是否稳定。
5. 说明为什么 TSC delta 不应无条件等同于真实 core cycles。

实验：用 perfstat 建立硬件证据

目标：从单纯时间测量进入硬件 counter 观察。

对前面的最小可信 benchmark 运行：

```
perf stat -r 10 \
-e cycles,instructions,branches,branch-misses,cache-references,cache-misses \
./bench_sum
```

任务：

1. 记录 cycles、instructions、IPC。
2. 记录 cache miss 相关事件。
3. 如果权限不足，记录完整错误信息和 perf_event_paranoid 值。
4. 改变输入规模，观察事件是否随 n 变化。
5. 把 PMU 结果和时间结果放在同一份报告中解释。

交付物：

1. perfstat 输出。
2. 事件含义解释。
3. 输入规模与 counter 变化表。
4. 至少一个你不能确定的点，并说明还需要什么实验。

实验：cache 状态实验

目标：观察 warm-cache 与 cold-ish-cache 对结果的影响。

设计两个版本：

1. warm-cache：同一数组连续重复求和。
2. cold-ish-cache：每次计时前扫描一个远大于 LLC 的 buffer，尽量污染 cache。

示例 cache 污染函数：

```
void touch_buffer(std::span<std::uint64_t> buffer)
{
    constexpr std::uint64_t TOUCH_INCREMENT = 1;
    for (std::uint64_t& value : buffer) {
        value += TOUCH_INCREMENT;
    }
}
```

任务：

1. 选择至少 3 个输入规模。
2. 每个规模分别测 warm-cache 和 cold-ish-cache。
3. 报告 median ns 和 GB/s。
4. 解释为什么 cold-ish-cache 只是近似，不是严格清空所有 cache。
5. 用 perfstat 观察 cache miss 是否变化。

交付物：

1. 源码。
2. 结果表。
3. cache 状态解释。

4. 对后续内存层次章节提出 3 个问题。

本章作业

习题与作业

作业 1: benchmark 批判性阅读

找一个你过去写过的性能测试，或者网上任意一段短 benchmark。写一份审计报告，回答：

1. 被测对象是否明确？
2. setup 和 timed region 是否分离？
3. 是否有 correctness reference？
4. 是否可能被 DCE、constant folding 或 loop invariant hoisting 影响？
5. 是否重复测量？
6. 是否报告统计口径？
7. 是否记录环境？
8. 是否审计汇编？

要求至少指出 5 个问题，并给出修复方案。

作业 2: 最小 harness 代码评审

阅读本章最小 C++20 harness，写一份代码评审：

- 哪些设计是为了防止编译器优化掉被测代码？
- 哪些设计是为了避免把 setup 混进计时区域？
- 哪些常量应该暴露成命令行参数？
- median_ns 当前实现有什么限制？
- 如果要支持多个 kernel，应该如何拆分函数和数据结构？

要求给出不少于 800 字的工程评审，不要只复述代码。

作业 3: 时间源误差分析

对一个空 timed region 测量计时器开销：

```
begin = now()
end = now()
```

再对一个很短 kernel 和一个较长 kernel 分别计时。要求：

1. 报告空计时开销分布。
2. 解释为什么短 kernel 更容易被计时器开销污染。
3. 设计 batch 策略降低相对误差。
4. 说明 batch 策略可能引入什么新问题。

作业 4: 汇编审计报告

选择前面最小可信 benchmark 中的一个输入规模，对可执行文件生成反汇编报告：

```
objdump -drwC -Mintel ./bench_sum > bench_sum.objdump.txt
```

报告必须包含：

1. sum_u64 是否被内联。
2. 核心循环反汇编。
3. 是否出现 SIMD 指令。
4. 每次循环处理多少元素。
5. 循环退出条件。

6. 源码中的 `do_not_optimize` 在汇编层是否生成真实指令。

作业 5：性能实验报告

提交一份完整性能实验报告，主题是 `sum_u64`、`copy_u64` 或 `dot_f32` 三选一。报告必须包含：

- 环境记录。
- 编译命令。
- workload 定义。
- correctness reference。
- 原始测量数据。
- min、median、p95、max。
- 至少一段反汇编。
- 至少一次 `perfstat` 尝试。
- 结论和限制。

作业 6：小差异是否可信

设计两个版本的同一 kernel，让它们的性能差异尽量接近 1% 到 3%。可以通过轻微改变循环结构、增加一个很小的额外操作，或者比较两个近似等价的实现完成。要求：

1. 至少运行 5 轮独立实验，每轮 31 个样本。
2. 每轮交错运行 A 和 B，避免顺序偏差。
3. 报告每轮的 min、median、p95、max。
4. 保存原始样本。
5. 判断差异是否稳定大于噪声。
6. 写出“能得出的结论”和“不能得出的结论”。

这个作业的重点不是制造一个真的快 2% 的优化，而是训练你不要被小百分比欺骗。若最后发现证据不足，也可以得高分，前提是分析清楚。

评分标准

本章作业按下面标准评价：

1. 是否能清楚划分 setup、warmup、timed region 和 validation。
2. 是否能解释编译器删除、移动或常量折叠 benchmark 的原因。
3. 是否能用 reference 证明结果正确。
4. 是否能保存和解释重复测量数据。
5. 是否能选择合适指标，例如 ns/op、GB/s、GFLOP/s。
6. 是否能用 `objdump` 和 `perfstat` 支撑结论。
7. 是否能诚实说明实验限制，而不是过度推断。
8. 是否能区分绝对差异、相对差异和测量不确定性。
9. 是否能根据证据等级控制结论强度。

本章小结

本章建立了性能测量的第一层地基：

```
correctness:
  reference first, performance second

benchmark structure:
  setup -> warmup -> timed region -> validation -> report

compiler safety:
  avoid DCE, constant folding, unintended motion

time source:
```

```
steady_clock for basic timing
TSC and PMU require stronger interpretation

noise:
  OS scheduling, CPU frequency, cache state, migration, background load

evidence:
  raw samples, statistics, objdump, perf stat, environment record
```

从现在开始,你不应该再接受“这段代码跑了几毫秒”这种孤立数字。性能工程的基本素养是:每个数字都必须带着上下文、方法、证据和限制。下一章会继续把这些原则工程化,系统设计 benchmark 输入、热路径、CSV 输出、参数扫描和报告格式。

Benchmark 设计：输入、热路径和报告

问题与目标

上一章解释了性能测量为什么容易出错。本章进一步回答：怎样把测量原则落实为可复现、可审计、可长期维护的 benchmark 工程。

这一步非常关键。以后无论做 HPC kernel、AI 算子、编译器优化还是内存布局优化，都不能长期依赖临时代码计时。你需要的是一套可以扩展、可以审计、可以保存原始数据、可以扫描参数、可以比较版本的实验系统。

本章目标是让你能设计一个小型 benchmark suite，而不是只会写一个 `main()` 计时器。这个 suite 至少要处理：

- 多个 kernel：例如 `sum_u64`、`copy_u64`、`fill_u64`、`dot_f32`。
- 多个输入规模：从 L1 cache 级别到内存带宽级别。
- 多种数据分布：随机、全零、递增、分支友好、分支不友好。
- 多个重复次数：warmup、measured repetitions、batch。
- 正确性校验：每个 kernel 都有 reference 或可验证不变量。
- 原始数据和汇总数据：CSV 输出、环境记录、命令行记录。
- 二进制审计：可以把源码、反汇编和数字联系起来。

从工程角度看，benchmark suite 与普通测试框架有相似之处：它需要清晰的数据流、明确的边界、稳定的输出格式和可维护的扩展点。不同之处在于，benchmark 还必须控制优化器、cache、频率、输入分布和统计口径。

核心思想

高质量 benchmark 不是一段计时代码，而是一套实验协议。它规定被测对象、输入空间、计时边界、重复策略、输出格式、正确性检查、环境记录和审计方法。

从一次计时到实验系统

上一章的最小 harness 解决的是“怎样避免一次测量明显失真”。本章要解决另一个问题：当你需要长期比较多个版本、多个输入规模、多个 kernel、多个机器时，临时代码会迅速失控。每次复制一个 `main()`，改几个常量，手工粘贴结果，最终会出现三个问题。

第一，可复现性下降。你很难知道某张结果表来自哪份源码、哪个命令、哪个输入 seed、哪个编译参数。几天后重新运行，结果不同，也无法判断是代码变了、机器状态变了，还是当时记录不完整。

第二，可比性下降。不同 kernel 用不同 warmup 次数，不同 CSV 列名，不同统计口径，不同 batch 策略，最后数字放在同一张表里却不是同一实验协议下的数字。这样的比较会误导。

第三，可维护性下降。新增一个 kernel 需要复制大量代码，修复一个测量 bug 需要改很多地方，CSV 格式变化会破坏旧脚本。benchmark suite 不是为了炫耀架构，而是为了避免实验本身变成不可控变量。

因此，benchmark suite 的设计目标是把变化点显式化。kernel 可以变，输入规模可以变，数据分布可以变，batch 可以变，统计口径可以变；但配置、样本记录、汇总、环境记录和报告流程应尽量稳定。只有稳定的实验协议，才能支持长期性能演进。

心智模型

把 benchmark suite 看成一个小数据生产系统：

- 配置决定实验矩阵。

- workload factory 生成输入实例。
- runner 产生原始样本。
- summarizer 产生派生指标。
- report 解释数据和边界。

不要让这些职责互相混杂。

Benchmark 设计的主线

把 benchmark 设计成下面的流水线：

```
configuration:
  parse command line
  choose sizes, repetitions, batch, seed, output path

environment record:
  compiler flags, command line, CPU info, OS info, git commit if available

workload construction:
  allocate buffers
  initialize inputs
  compute reference or invariant

warmup:
  run kernel without recording samples

measurement:
  repeat:
    run timed region
    consume result
    validate if cheap enough
    append raw sample

summary:
  compute min, median, p95, max, derived metrics
  write CSV

audit:
  keep binary, objdump excerpt, perf command, raw data
```

这条流水线背后的思想是职责分离：每个阶段只做一类事。输入生成不放进计时区域；统计不混进 kernel；报告不改变运行行为；校验不依赖性能版本本身。

心智模型

设计 benchmark 时，把它看成“实验编译器”：配置是源程序，workload 是输入实例，kernel 是被执行的目标代码，CSV 是输出产物，报告是解释器。任何隐式状态都会降低可复现性。

职责边界：让实验系统可审计

benchmark suite 的代码规模通常不大，但职责边界必须清楚。边界不清的 suite 很快会变成“能运行但不能信任”的脚本：参数解析顺便改全局变量，workload 生成顺便 warmup，runner 顺便写 CSV，summary 顺便过滤异常值，最后没有人知道某个数字到底经过了哪些步骤。

一个可审计的 suite 至少应把下面几类责任分开。

模块	应承担的责任	不应承担的责任
options parser	把命令行变成显式配置。	不运行 kernel，不生成输入。
workload factory runner	分配、初始化、计算 reference。执行 warmup 和 timed region。	不计时，不写结果文件。不解析命令行，不决定统计口径。
checker	判断结果是否正确。	不改变被测输入，除非协议说明。
collector summarizer	保存原始样本。从 raw 计算 summary。	不偷偷删除离群点。不修改 raw，不改变实验事实。
writer recorder	输出 CSV 和 metadata。保存环境、命令、构建信息。	不参与 timed region。不影响 kernel 行为。

这里的“不应承担”比“应承担”更重要。因为 benchmark 的失败往往不是某个模块完全缺失，而是职责串线。例如 runner 在计时循环里顺手写一行日志，数字就混入 I/O；summarizer 看到异常点后直接删除，报告就失去可复核性；workload factory 依赖当前时间生成 seed，实验就不能复现。

显式数据流优先于隐式全局状态

benchmark suite 很容易出现全局变量：

```
global_size
global_seed
global_batch
global_result_file
```

全局状态写起来方便，但会让实验很难审计。一个函数到底使用了哪个 seed？某次 batch 是手工指定还是自动校准得到？写 CSV 的路径来自命令行还是默认值？这些信息如果散落在全局变量和初始化顺序里，后续读报告的人无法重建实验。

更稳的方式是让配置显式流动：

```
BenchmarkOptions -> Workload -> RawSample -> SummaryRow -> CSV
```

每个阶段接收明确输入，产生明确输出。这样做的好处是：测试容易写，日志容易记录，错误容易定位。比如 summary 计算错了，可以直接用 raw samples 重算；workload 出错了，可以只检查 factory；CSV 列缺失，可以回到 RawSample 定义。

metadata 是实验产物，不是附加说明

很多人把环境、命令和构建信息写在报告正文里，甚至只写在聊天记录里。这不够。metadata 应该和 raw samples 一样成为结果目录中的一等产物。原因很简单：性能数据离开上下文就没有意义。没有编译器版本，无法判断优化器行为；没有命令行，无法知道 batch、seed、size；没有 git hash，无法知道源码；没有 CPU 信息，无法解释 cache 和 ISA。

推荐把 metadata 分成三类。第一类是运行前已知的配置，例如 size、seed、batch、warmup、repetitions。第二类是运行时环境，例如 CPU、OS、governor、容器、权限。第三类是构建和源码，例如编译命令、编译器版本、git hash、是否 dirty。三类信息都应该由程序或脚本自动记录，尽量减少手工记忆。

核心理念

benchmark suite 的核心不是“少写几行代码”，而是让每个性能数字都能追溯到配置、输入、机器码、环境和原始样本。

先写清楚问题陈述

动手写代码前，先写一段 problem statement。它不需要长，但必须具体。
差的描述：

measure sum performance

好的描述：

Goal:
Measure warm-cache and streaming performance of unsigned 64-bit sum.

Kernel:
std::uint64_t sum_u64(std::span<const std::uint64_t> values)

Input sizes:
1 KiB, 16 KiB, 256 KiB, 4 MiB, 64 MiB worth of uint64_t elements

Data:
deterministic pseudo-random uint64_t, generated once per size

Measurement:
5 warmup runs, 31 measured repetitions
timed region contains only the kernel call

Output:
raw CSV and summary CSV
fields include n, bytes, sample_ns, median_ns, bandwidth_Gbps

为什么要这么写？因为性能结论总是有适用范围。一个只在小规模输入时更快的版本，不一定在大规模输入时更快；一个 warm-cache 下更快的版本，不一定 cold-cache 下更快；一个随机数据下快的分支版本，不一定排序数据下快。

输入规模：不要只测一个点

输入规模决定瓶颈。对数组 kernel 来说，常见阶段如下：

工作集范围	常见瓶颈	观察重点
很小 L1/L2 内	调用开销、循环开销、前端开销。 指令吞吐、load/store 吞吐、向 量化。	ns/op、cycles/op。 每周期元素数。
LLC 内 超过 LLC	cache 带宽、预取、并行度。 内存带宽、地址转换和 NUMA。	GB/s、cache miss。 GB/s、LLC miss、TLB。

推荐选择不是随便的 1000,10000,100000，而是能跨过 cache 层级的规模。例如对 std::uint64_t：

bytes per element = 8

sizes by bytes:

1 KiB	-> 128 elements
16 KiB	-> 2048 elements
256 KiB	-> 32768 elements
4 MiB	-> 524288 elements
64 MiB	-> 8388608 elements

在命令行或配置中可以写成：

```
--sizes-bytes 1024,16384,262144,4194304,67108864
```

还要测边界规模。比如 SIMD kernel 常见尾部处理问题：

vector width = 8 float elements

important n:

0, 1, 7, 8, 9, 15, 16, 17
1023, 1024, 1025

这些规模不一定用来报告最终吞吐，但必须用于 correctness 和尾部路径性能检查。

常见误区

只测一个大输入很容易掩盖小规模延迟；只测一个小输入又会掩盖内存带宽瓶颈。算子优化通常需要同时报告小规模、中规模、大规模和尾部边界。

输入规模应服务问题

输入规模不是越多越好，而是要覆盖你关心的性能区域。若目标是研究函数调用和短小 kernel 开销，小规模点要更密；若目标是研究内存带宽，大规模点要跨过 LLC；若目标是研究 SIMD 尾部处理，就要专门测向量宽度附近的边界值；若目标是研究算法复杂度，就要覆盖足够大的数量级变化。

可以把规模分成三类。第一类是 correctness 边界规模，例如 `0,1,vector_width-1,vector_width,vector_width+1`。它们主要用于发现尾部 bug，不一定适合吞吐结论。第二类是 cache 层级规模，例如明显落在 L1、L2、LLC 和内存中的工作集。它们用于观察瓶颈迁移。第三类是业务代表规模，例如真实线上请求常见的 tensor shape、图像大小、batch size 或行数。它们用于判断优化是否有实际价值。

报告中最好把这三类规模区分开。一个版本在 correctness 边界规模上慢，可能是固定开销；在业务规模上快，仍然有价值。反过来，一个版本只在人工大规模上快，但真实业务规模很小，工程价值可能有限。

矩阵设计要分阶段

如果一次性组合所有维度：

```
5 kernels * 8 sizes * 5 distributions * 3 cache modes * 3 batch modes
```

就会得到 1800 个配置。每个配置再测 31 次，实验时间、数据量和分析复杂度都会暴涨。更糟的是，结果太多反而让你看不清问题。

更好的方法是分阶段。第一阶段固定 distribution、cache mode 和 batch，只扫描 size，建立性能轮廓。第二阶段在异常 size 上增加 distribution，判断分支或数值路径是否影响。第三阶段对短 kernel 引入 batch 校准，判断计时稳定性。第四阶段对内存 kernel 增加 cold-ish 或 eviction buffer，判断 cache 敏感性。第五阶段才跑完整矩阵，作为最终报告。

这种策略符合工程调试思路：先用小实验定位变量，再用大矩阵确认结论。它也减少无意义数据堆积，避免为了“全面”而失去分析能力。

核心理想

benchmark 矩阵不是越大越专业。专业的矩阵应围绕问题分阶段展开，每增加一个维度都要说明它验证什么假设。

实验变量：一次只让少数东西变化

benchmark 本质上是实验。实验设计的第一条原则是分清变量。一个性能数字背后至少有三类变量。

第一类是自变量，也就是你主动改变的因素。比如输入规模、数据分布、对齐方式、batch 大小、kernel 版本、编译参数、线程数。自变量应该出现在配置和 CSV 中。

第二类是因变量，也就是你观察的结果。比如 elapsed ns、GB/s、ns/element、GFLOP/s、branch-misses、cache-misses。因变量应该有明确单位和计算公式。

第三类是控制变量，也就是你希望保持不变的条件。比如机器、CPU governor、编译器版本、构建类型、输入 seed、内存分配策略、运行用户、后台负载、环境变量。如果控制变量变化了，结果解释就要更谨慎。

很多错误 benchmark 的根源是同时改变太多东西。例如你把算法换了、编译器从 GCC 换成 Clang、输入从全零换成随机、batch 从 1 变成 auto、机器也换了，然后看到版本 B 更快。这个结果不能回答“算法是不是更好”，因为差异可能来自任何一个变化。

专业做法是先固定控制变量，再改变一个主要自变量。若必须改变多个变量，就用矩阵显式记录，并在报告中说明交互关系。例如“版本 B 在随机输入下更快，但在全零输入下没有优势”，说明版本和 distribution 存在交互；“版本 B 在小规模快、大规模持平”，说明版本和 size 存在交互。

变量类型	例子	记录位置
自变量	kernel、size、distribution、alignment、batch	command、CSV、report
因变量	elapsed ns、GB/s、miss rate、speedup	raw、summary、perf output
控制变量	CPU、compiler、flags、governor、seed	environment、metadata

核心理念

benchmark 不是“跑一下看谁快”，而是控制变量后观察自变量如何影响因变量。变量不清，结论就不清。

对照组：baseline 必须稳定

每个优化实验都需要 baseline。baseline 可以是简单 C++ 版本、标准库函数、上一版实现、标量版本、无 SIMD 版本、单线程版本，或者已经验证过的 reference。没有 baseline，数字只是一组孤立时间，无法判断好坏。

baseline 需要满足几个条件。

第一，它必须正确。baseline 不一定最快，但必须语义可信。很多时候 baseline 同时承担 correctness reference 的角色。若 baseline 本身错了，优化版本再快也没有意义。

第二，它必须可复现。baseline 的源码、编译参数、输入和输出都要记录。不要用“我之前那个版本”作为对照，除非能明确指向 commit、文件路径和命令。

第三，它必须和优化版本在同一实验协议下运行。相同机器、相同编译器、相同输入、相同重复次数、相同统计口径。否则比较不公平。

第四，它要尽量保持长期稳定。随着项目发展，baseline 也可能修 bug 或改接口；这时要记录 baseline version。大型项目常常需要多个 baseline：reference baseline 用于正确性，simple baseline 用于教学解释，production baseline 用于工程比较。

reference baseline 和 performance baseline

reference baseline 和 performance baseline 不一定是同一个实现。reference baseline 的目标是语义清楚、容易验证、边界行为可信。它可以慢，可以不用 SIMD，可以使用更高精度或更直接的写法。performance baseline 的目标是代表当前工程里值得比较的版本，例如上一版生产实现、标准库实现、标量优化版或已有手写汇编版。

把二者混在一起会带来两个问题。第一，如果 reference 很慢，候选版本相对 reference 的 speedup 会很好看，但不代表工程上有价值。第二，如果 performance baseline 过于复杂，把它同时当 correctness oracle，就可能把旧 bug 当成标准答案。正确做法是：用 reference 判断结果是否正确，用 performance baseline 判断候选版本是否值得替换。

例如 `dot_f32` 可以有三个版本：

```
reference:
    simple loop, known reduction order, cclear tolerance

baseline:
    current scalar O3 implementation

candidate:
    unrolled or vectorized implementation
```

报告应分别写清：candidate 是否通过 reference 校验；candidate 相对 performance baseline 的时间、吞吐和误差如何变化。这样结论不会被漂亮但无意义的 speedup 误导。

baseline 要能冻结和复跑

长期项目中，baseline 本身会变。如果每次都拿“当前 main 分支”当 baseline，几个月后你很难解释历史曲线。更稳的做法是为重要实验记录 `baseline_version`，可以是 git hash、release tag、源码文件 checksum 或二进制 checksum。

冻结 baseline 不代表永远不更新。它的意思是：每次更新 baseline 都要作为实验协议变化记录下来。比如从标量 baseline 换成自动向量化 baseline，旧 speedup 和新 speedup 不能直接放在同一条曲线里。否则你看到的性能变化可能只是参照物变化。

对教学实验来说，baseline 可以简单写成：

```
baseline_name=scalar_cpp
baseline_source=bench_kernels.cpp
baseline_compile_flags=-O3 -march=native
```

对工程实验，最好记录 commit 和二进制路径：

```
baseline_commit=5364eb0
candidate_commit=working_tree_dirty
binary=build/bench_suite
```

如果工作区有未提交修改，也要记录 dirty 状态。否则别人只能看到 commit，却复现不出你当时运行的源码。

报告 speedup 时，公式要明确：

```
speedup = baseline_median_ns / candidate_median_ns
```

如果使用吞吐，则可能写：

```
throughput_ratio = candidate_GBps / baseline_GBps
```

两者方向相反，不能混用。报告里写“快 2 倍”时，要说明是时间减半还是吞吐翻倍。

常见误区

baseline 不是随便挑一个慢版本。baseline 是实验对照组，必须正确、稳定、可复现，并且和候选版本共享同一实验协议。

运行顺序：不要让时间顺序伪装成性能差异

如果你先跑完 baseline 的所有配置，再跑 optimized 的所有配置，结果可能混入时间顺序效应。机器温度、CPU 频率、后台任务、cache 状态、内存分配状态、系统负载都可能随时间变化。于是 optimized 看起来更快或更慢，未必全是代码差异。

更稳的做法是交错运行。比如对每个 size：

```
for size in sizes:
    run baseline(size)
    run optimized(size)
    run baseline(size)
    run optimized(size)
```

或者把配置顺序打乱，并记录 order index：

```
order_index, kernel, size, variant, elapsed_ns
0, sum_u64, 1048576, baseline, ...
1, sum_u64, 1048576, optimized, ...
2, sum_u64, 32768, optimized, ...
3, sum_u64, 32768, baseline, ...
```

随机化顺序不是为了制造随机，而是为了避免所有 baseline 都处于一种系统状态、所有 optimized 都处于另一种系统状态。对短实验，交错运行通常已经足够；对长实验，记录顺序、时间戳和环境状态更重要。

当然，交错运行也有副作用。一个配置可能预热另一个配置的 cache 或预测器。解决办法不是固定一种做法，而是让运行顺序成为实验协议的一部分。若你测的是 warm-cache 同一输入，交错可能合理；若你测 cold-cache，配置之间要做 eviction 或隔离。

心智模型

性能实验中的时间顺序也是变量。若不记录、不控制，它就会偷偷进入结果。

数据分布：输入不是无关变量

数据分布会改变分支、向量化、数值路径、cache 压缩效果和异常值。
以 `count_positive` 为例：

```
std::uint64_t count_positive(std::span<const std::int32_t> values)
{
    std::uint64_t count = 0;
    for (const std::int32_t value : values) {
        if (value > 0) {
            ++count;
        }
    }
    return count;
}
```

这些输入会导致不同表现：

分布	特征	预测难度
全正数	分支总是 taken。	低
全负数	分支总是 not taken。	低
正负交替	模式简单但频繁变化。	中
随机正负	近似不可预测。	高
长 run	一段正、一段负。	低到中

如果报告只写 `count_positive` 的速度，不写数据分布，结论不完整。

确定性输入生成

benchmark 不应该依赖每次运行都不同的随机输入。使用固定 seed：

```
#include <cstdint>
#include <vector>

namespace {

constexpr std::uint64_t INPUT_MULTIPLIER = 6364136223846793005ULL;
constexpr std::uint64_t INPUT_INCREMENT = 1442695040888963407ULL;
constexpr std::uint64_t INPUT_DEFAULT_SEED = 1ULL;

std::vector<std::uint64_t> make_u64_input(std::size_t size, std::uint64_t seed)
{
    std::vector<std::uint64_t> values(size);
    std::uint64_t state = seed;
    for (std::uint64_t& value : values) {
        state = state * INPUT_MULTIPLIER + INPUT_INCREMENT;
        value = state;
    }
    return values;
}

} // namespace
```

这不是密码学随机数，也不是统计质量最好的生成器。它的作用是：

- 生成运行时数据，避免常量折叠。
- 固定 seed，保证可复现。
- 成本低，但仍然放在 `setup`，不放在 `timed region`。

浮点数据要避开隐藏语义

浮点 benchmark 要明确是否包含：

- NaN。
- Inf。
- signed zero。
- denormal/subnormal。
- 极大或极小值。

例如 dot product 入门实验可以先使用 $[-1, 1]$ 的普通有限值，避免把异常数值路径混进第一个实验。后续研究 denormal、fast-math、flush-to-zero 时，再单独设计对应 workload。

固定 seed 还不够

很多报告写“随机输入，seed = 1”，但这仍然不够。还要写清随机生成算法、取值范围、类型转换方式和是否允许特殊值。不同标准库实现的随机分布细节可能不同；不同平台上浮点生成和舍入也可能带来差异。若你希望跨机器完全复现，最好使用自己明确写出的简单生成器，或者把输入文件保存为 artifact。

例如整数输入可以用固定 LCG，因为它的溢出语义在 unsigned 类型中明确。浮点输入可以先生成整数，再映射到固定范围，避免依赖库分布实现。若实验目标不是随机性质量，而是可复现 workload，这样的简单生成器反而更合适。

数据分布也是实验参数

数据分布必须进 CSV。不要只在报告文字里说“随机”。至少应有字段：

```
distribution,seed,value_range,pattern
```

对分支实验，可以记录 all_positive、all_negative、alternating、random_sign。对浮点实验，可以记录 finite_uniform、contains_nan、subnormal_stress。对内存实验，可以记录 sequential、strided_4、random_index。

这样做的好处是后续分析不会丢上下文。几个月后你看到某个结果异常，可以从 CSV 知道它不是算法突然变慢，而是输入分布换成了随机访问或 subnormal 压力测试。

常见误区

输入不是背景噪声，而是 benchmark 的一部分。没有输入分布，性能数字没有完整语义。

对齐、步长和别名

数组 kernel 的输入不只是长度，还有地址形状。

```
contiguous:
    a[i]

strided:
    a[i * stride]

offset-aligned:
    pointer address is 64-byte aligned

misaligned:
    pointer address = aligned_base + 4
```

这些差异会影响：

- load/store 是否跨 cache line。
- SIMD load 是否需要 unaligned 路径。
- 预取器能否识别访问模式。
- TLB 覆盖范围。
- 编译器能否证明无别名。

如果 kernel 接收两个指针：

```
void copy_u64(std::span<std::uint64_t> dst,
             std::span<const std::uint64_t> src);
```

要明确：

- `dst` 和 `src` 是否允许重叠。
- 若不允许重叠，C++ 实现是否能用 `restrict`-like 信息表达。
- 输入是否按 64 字节对齐。
- 是否测试非对齐偏移。

深入理解

C++ 标准没有标准化的 `restrict` 关键字，但 GCC/Clang 有 `__restrict__` 扩展。后续讲 `alias analysis` 和 `vectorization` 时会看到：编译器是否知道两个指针不别名，可能直接决定是否能向量化。

把地址形状做成可控变量

对齐、偏移、步长和别名不应隐藏在 `allocator` 的偶然结果里。若你想比较 `aligned` 和 `unaligned`，就应显式分配更大的 `buffer`，然后选择不同偏移：

```
base = aligned allocation
aligned_ptr = base
misaligned_ptr = base + 1 element or base + 4 bytes
```

CSV 中记录：

```
alignment_bytes,offset_bytes,stride,alias_mode
```

这样才能解释结果。如果某次 `std::vector` 恰好给了 32 字节对齐，另一次给了 16 字节对齐，而报告没有记录，性能差异就会很难复现。

别名模式要单独测

`copy`、`axpy`、`vector add` 这类 `kernel` 都可能受别名影响。至少区分：

- `no alias`：输入输出完全不同。
- `exact alias`：输出和某个输入相同，例如 `in-place`。
- `partial overlap`：输出与输入部分重叠。

不同语义需要不同实现。普通 `copy` 若允许 `overlap`，就更接近 `memmove`；若不允许 `overlap`，就可以更激进地向量化和重排。`benchmark` 不能把这些混在一起。一个不支持 `overlap` 的高性能 `copy`，在 `no-alias` 条件下很快是合理的；拿它和支持 `overlap` 的通用实现直接比较并不公平。

步长实验要防止容量误判

`strided` 访问中，元素数相同不代表访问的地址范围相同。例如访问 `a[i*16]`，虽然只读取 `n` 个元素，但跨越的内存跨度是连续访问的 16 倍。`cache line` 利用率、`TLB` 压力和预取行为都不同。

因此 `strided benchmark` 应同时记录：

```
logical_elements = n
stride_elements = stride
address_span_bytes = (n - 1) * stride * sizeof(T) + sizeof(T)
logical_bytes = n * sizeof(T)
```

报告 `GB/s` 时也要说明是按 `logical bytes` 还是按 `address span` 解释。通常 `logical bytes` 表示算法实际使用的数据量，`address span` 帮助解释内存系统压力。

核心理想

数组 `benchmark` 的输入不只是 `n`。地址对齐、偏移、步长、别名和地址跨度都会改变机器行为，应该作为实验参数记录。

热路径边界：把 setup 赶出去

一个可维护 benchmark 应有显式阶段：

```
make_workload()
compute_reference()
warmup()
measure()
validate()
write_result()
```

不要在 timed region 里做这些事：

- 分配内存。
- 生成随机数。
- 打印日志。
- 写 CSV。
- 构造 `std::string`。
- 查询 CPU 信息。
- 解析命令行。
- 做复杂校验。

timed region 应尽量像这样：

```
const auto begin = Clock::now();
const std::uint64_t result = sum_u64(workload.values);
const auto end = Clock::now();
do_not_optimize(result);
```

如果 kernel 本身就是“分配一个对象”或“写一个文件”，当然可以把分配或 I/O 放进 timed region。但那时 benchmark 名称要诚实：

```
bench_vector_push_back_with_growth
bench_write_4KiB_sync
```

而不是假装它是纯计算 kernel。

Batch：短 kernel 的必要技巧

如果一次 kernel 调用只需几十纳秒，计时器开销和噪声会占很大比例。解决方法是 batch：

```
one sample:
begin timer
repeat kernel B times
end timer

sample_ns_per_batch = end - begin
ns_per_call = sample_ns_per_batch / B
```

但 batch 不是越大越好。它会改变 cache 状态、分支预测状态和热路径行为。比如同一个小数组重复求和很多次，会变成强 warm-cache 测试；如果你想测 cold-cache，batch 可能破坏实验。

一个简单策略：

- 对短 kernel 使用 batch，目标是让每个 sample 至少达到几十微秒。
- 报告 batch 大小。
- 不同输入规模可以使用不同 batch，但要在 CSV 中记录。
- 不能把 batch 后的数字和非 batch 数字混在一起比较。

```
std::uint64_t run_sum_batch(std::span<const std::uint64_t> values,
                           int batch_count)
{
    std::uint64_t combined = 0;
    for (int batch = 0; batch < batch_count; ++batch) {
        combined += sum_u64(values);
    }
}
```

```

    }
    return combined;
}

```

`combined` 的作用是把每次结果连接起来，避免优化器删除重复调用。实际 harness 还要用 `do_not_optimize(combined)`。

常见误区

batch 会改变 workload。对 cache 敏感、状态ful、随机输入、分支预测敏感的 kernel，batch 可能让测量更稳定，也可能让测量偏离真实场景。报告里必须写明。

哪些 kernel 不适合简单 batch

纯函数式、无状态、输入不变的 kernel 最适合 batch，例如对同一数组重复求和。每次调用语义相同，batch 主要改变 cache 和预测状态。

但有些 kernel 不适合简单重复。随机数生成器会更新内部状态；队列 push/pop 会改变容器大小；排序第一次运行后输入已经有序；fill 和 copy 会改变输出 buffer；解码器可能推进流位置；分配器 benchmark 会改变 heap 状态。对这类 kernel，batch 要么需要在每次调用前恢复状态，要么要把 batch 定义成真实 workload 的一部分。

例如 fill benchmark 若重复填同一个 buffer，第一次可能触发写分配和缺页，后续是热写；如果你想测持续写带宽，这可以接受；如果你想测初始化一个新 buffer 的成本，就不能这样 batch。copy benchmark 若每次源和目标相同，cache 状态会迅速变热；若目标是 streaming copy，应使用足够大的 buffer 或轮换多个 buffer。

checksum 不能污染被测工作

为了防止优化器删除 batch，常把每次结果累加到 `combined`。这对返回标量的 kernel 很方便。但对输出 buffer kernel，若在 timed region 中扫描输出计算 checksum，就会把校验成本混进测量。正确做法通常是：timed region 只执行 kernel；计时后在区域外校验输出，或只对少量 sentinel 做轻量防优化，同时定期做完整校验。

例如 copy kernel 可以在计时后比较 destination 和 source，但不把比较放入 timed region。为了防止编译器删除 copy，必须让输出缓冲区在计时后有可观察使用，例如抽样读取若干个目标位置并送入防优化接口；完整 correctness check 仍然放在计时区域外。

batch 后的单位要一致

使用 batch 后，所有派生指标都要乘以 batch。若一次 sample 运行了 `B` 次，每次处理 `n` 个元素，那么：

```

effective_elements = n * B
effective_bytes = bytes_per_run * B
ns_per_element = elapsed_ns / effective_elements
GB/s = effective_bytes / seconds / 1e9

```

CSV 中如果只有 `n` 没有 batch，后续计算必然出错。不要把 batch 后的 `elapsed_ns` 当成单次调用时间，除非列名明确是 `elapsed_ns_per_batch` 或 `ns_per_call`。

CSV：先保存原始数据

不要只打印最终汇总。原始样本是后续统计和复查的基础。

推荐至少输出两个文件：

```

raw_samples.csv
summary.csv

```

raw samples:

```

kernel,n,bytes,distribution,batch,rep,warmup,elapsed_ns,result_checksum
sum_u64,1048576,8388608,lcg,1,0,5,312000,12345
sum_u64,1048576,8388608,lcg,1,1,5,311400,12345

```

summary:

```
kernel,n,bytes,distribution,batch,repetitions,min_ns,median_ns,p95_ns,max_ns,GBps
sum_u64,1048576,8388608,1cg,1,31,309900,312000,320100,330000,26.88
```

字段设计原则：

- 每一行都能独立解释。
- 不要把单位藏在列名之外。
- 所有影响结果的参数都要进 CSV。
- 原始样本不要覆盖，只追加或写到带时间戳/配置名的文件。
- summary 可以由 raw samples 再生成。

schema 要稳定

CSV schema 一旦被脚本、报告和历史结果依赖，就不要随意改列名。列名是数据接口。比如 `elapsed_ns`、`batch`、`bytes`、`distribution` 这些列，应尽早定好语义。如果后续需要新增列，可以追加；如果必须改变语义，应增加 schema version。

建议在 CSV 中或旁边 metadata 文件里记录：

```
schema_version=1
time_unit=ns
bytes_definition=logical_bytes
batch_semantics=repeated_kernel_calls_per_sample
```

这看起来繁琐，但当你比较几个月前的结果时非常有用。否则你可能不知道旧表里的 `bytes` 是只算读，还是读写合计；`elapsed_ns` 是单次调用，还是 batch 总时间。

raw 和 summary 不要互相替代

summary 方便阅读，raw 方便复查。只保留 summary，会丢掉样本分布、离群点和后续重新统计能力；只保留 raw，每次看报告都要重新处理。两者应该同时存在，并且 summary 能由 raw 再生成。

如果 summary 中有派生指标，例如 GB/s、GFLOP/s、ns/element，应保证公式写在报告或 metadata 中。派生指标不是原始事实，它依赖 bytes、FLOPs、batch 和时间单位的定义。

结果目录应不可变

一次正式实验输出后，尽量不要手工修改结果目录。若发现 bug，重新跑到新目录。旧目录可以标记为 invalid，但不要覆盖。推荐目录名包含日期、实验名和短 git hash：

```
results/2026-06-14_sum_suite_5364eb0/
```

如果没有 git hash，也可以记录源码文件校验和或手工版本号。目标是让报告读者能明确知道数据来自哪一次运行，而不是“我记得是昨天跑的那个版本”。

数据审计：先检查样本是否像样

在计算 speedup 之前，先审计 raw samples。很多错误从 summary 表里看不出来，但 raw samples 一眼就能发现。

最基本的审计包括：

- 每个配置是否有预期数量的 measured repetitions。
- 是否有 0 ns、负数、空行、解析失败或重复行。
- 同一配置的 checksum 是否一致。
- elapsed ns 是否存在明显长尾或异常跳变。
- warmup 是否被误写进 measured samples。
- batch、bytes、n 是否在同一配置内一致。
- 不同配置的 order index 是否符合运行协议。

例如 checksum 不一致，说明某些 sample 的结果不同。原因可能是输入被修改、kernel 有 bug、未初始化数据、随机状态没有固定、batch 改变语义，或者 checksum 设计本身不稳定。在解释速度

前，必须先解释 checksum。

再比如某个配置 31 个样本中有 1 个样本比中位数慢 10 倍。它可能是系统中断，也可能是 page fault，也可能是首次触发某个懒初始化，也可能是温度降频。你可以在 summary 中保留 median，但报告应说明异常点如何处理。直接删除异常点而不记录，是不可审计的。

raw data 审计可以先用简单脚本完成。即使不用脚本，报告中也至少要写：

```
Raw sample audit:
all configurations have 31 samples
checksum is stable within each configuration
no zero elapsed_ns
2 outliers observed in large-size copy; kept in raw, median used in summary
```

核心理想

summary 是解释入口，不是原始事实。正式比较前先审计 raw samples，否则漂亮图表可能只是错误数据的包装。

失败定位：从异常数字回到实验系统

benchmark suite 不是写完就结束。真正使用时，你会经常遇到奇怪结果：某个版本快得不合理，某个输入规模突然慢十倍，checksum 偶尔变化，样本出现 0 ns，某台机器上趋势完全相反。高质量 suite 要帮助你定位这些问题，而不是只输出一个表。

定位异常时，不要先猜微架构。先按实验系统从外到内检查。

第一步：确认数据没有坏

先检查 raw samples 的结构。每个配置是否都有预期样本数？是否有空行、重复行、解析失败、单位错误？elapsed_ns 是否为 0？batch 是否为 0？bytes 是否和 n 匹配？如果数据表本身坏了，后面的性能解释都没有意义。

常见错误包括：CSV 写入中途被打断，只保留了一部分样本；旧结果目录被新实验追加，导致两个协议混在一起；summary 脚本把不同 distribution 合并到同一组；batch 后的总时间被误当成单次调用时间；copy 的 bytes 口径从读字节变成读写合计但 schema 没更新。

这类错误通常不需要 perf 或反汇编，只需要严格的数据审计。一个简单规则是：summary 中每一行都应该能回到 raw 中一组唯一样本。如果回不去，summary 就不可审计。

第二步：确认 correctness 没有漂移

checksum 或 correctness 失败时，不要继续看速度。先判断失败是稳定还是偶发。稳定失败通常说明 candidate 语义错、reference 错、输入边界没处理、浮点 tolerance 不合理，或者 batch 改变了状态。偶发失败更危险，可能说明未初始化内存、越界写、数据竞争、输出 buffer 没重置，或者 kernel 依赖未定义行为。

例如 fill benchmark 若每轮不重置 destination，结果也许仍然正确，但测到的是“把已经填过的 buffer 再填一遍”；copy benchmark 若 source 和 destination 意外重叠，结果可能取决于复制方向；dot benchmark 若 reference 使用 float 累加，candidate 使用 FMA 和不同 reduction tree，误差可能随 n 增大而增长。每种情况都应该先归类为语义问题，而不是性能问题。

第三步：确认 timed region 真的干净

如果某个配置慢得离谱，检查 timed region 是否混入了分配、初始化、日志、CSV 写入、完整校验、异常处理或系统调用。最直接的方法是读代码和反汇编，也可以用临时日志记录阶段耗时：

```
setup_ns
warmup_ns
measurement_ns
validation_ns
write_ns
```

这些阶段耗时不一定进入正式 CSV，但对调试很有用。如果 validation 比 measurement 慢一百倍，这是可以接受的，只要它不在 timed region 中；如果 write CSV 出现在每次 sample 内部，就要修。

第四步：确认机器码符合预期

速度异常常来自编译器改变了被测对象。快得不合理，可能是循环被删除、输入被常量折叠、结果未被观察、两个版本被优化成同一段代码。慢得不合理，可能是内联失败、函数指针导致间接调用、`volatile` 阻止向量化、调试构建未开优化、异常路径或边界检查进入热路径。

反汇编审计要回答几个具体问题：目标循环是否存在？是否出现预期的 `load/store`？是否有意外调用？是否被自动向量化？`candidate` 和 `baseline` 机器码是否真的不同？如果报告说优化来自减少分支，但反汇编里两个版本分支结构相同，这个解释就站不住。

第五步：再考虑硬件和环境

只有在数据、正确性、计时区域和机器码都合理后，才进入硬件解释。此时再看 `cache`、`TLB`、分支、频率、`NUMA`、`SMT`、迁核、后台任务。这样顺序很重要，因为很多“硬件现象”其实是 `harness bug`。比如大输入慢可能是内存带宽，也可能是每轮重新分配触页；随机输入慢可能是分支预测，也可能是随机生成被放进 `timed region`。

可以把异常定位写成一张 `checklist`：

1. raw samples complete?
2. checksum stable?
3. timed region clean?
4. objdump matches expectation?
5. environment recorded?
6. perf or system evidence supports hypothesis?

常见误区

不要从异常数字直接跳到微架构故事。先排除数据、语义、计时边界和编译器问题，再讨论硬件瓶颈。

Schema 版本和结果兼容性

一旦 `benchmark suite` 开始长期使用，`CSV schema` 就会演化。最开始可能只有 `kernel,n,elapsed_ns`，后来加入 `batch`、`distribution`、`alignment`、`variant`、`compiler`。如果不管理 `schema`，旧结果和新结果很容易被错误合并。

建议每个结果目录都包含 `metadata`：

```
schema_version=2
suite_version=ch16-foundation
time_unit=ns
bytes_semantics=logical_bytes
speedup_baseline=baseline_median_ns/candidate_median_ns
```

`CSV` 中也可以加入 `schema_version` 列，或者在同目录放 `metadata.txt`。关键是后续脚本读取旧数据时，能知道列的含义。

`schema` 变化分两类。兼容变化是新增列，不改变旧列语义；旧脚本可以忽略新增列。破坏性变化是改列名、改单位、改 `bytes` 定义、改 `batch` 语义、改 `speedup` 方向。破坏性变化必须升级 `schema version`，并在报告里说明不能直接和旧结果比较。

例如 `bytes` 曾经表示只读字节，后来改成读写合计。如果不升级 `schema`，`copy kernel` 的 `GB/s` 会突然翻倍，看起来像性能提升，实际只是指标定义改变。

常见误区

指标定义改变不是性能提升。`schema` 版本的作用，就是防止数据接口变化伪装成优化成果。

环境记录：报告不是记忆

`benchmark` 输出目录建议包含：

```
results/2026-06-13_sum_suite/
  raw_samples.csv
```



```
summary.csv
environment.txt
command.txt
objdump.txt
perf_stat.txt
report.md
```

`environment.txt` 至少包含：

```
date -Iseconds
uname -a
lscpu
clang++ --version
g++ --version
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
cat /proc/sys/kernel/perf_event_paranoid
```

这些命令不一定每台机器都有同样输出。没关系，记录能记录的，并把失败也记录下来：

```
scaling_governor:
  unavailable: /sys/devices/system/cpu/cpu0/cpufreq does not exist
```

这比假装环境不存在要可靠。

一个小型 benchmark suite 的数据模型

先设计数据结构。不要一开始就把所有逻辑塞进 `main()`。

```
#include <cstdint>
#include <string>
#include <vector>

struct BenchmarkOptions {
    std::vector<std::size_t> sizes;
    int warmup_repetitions = 0;
    int measured_repetitions = 0;
    int batch_count = 0;
    std::uint64_t seed = 0;
    std::string output_prefix;
};

struct RawSample {
    std::string kernel_name;
    std::size_t elements = 0;
    std::size_t bytes = 0;
    int batch_count = 0;
    int repetition = 0;
    std::uint64_t elapsed_ns = 0;
    std::uint64_t checksum = 0;
};

struct SummaryRow {
    std::string kernel_name;
    std::size_t elements = 0;
    std::size_t bytes = 0;
    int batch_count = 0;
    int repetitions = 0;
    std::uint64_t min_ns = 0;
    std::uint64_t median_ns = 0;
    std::uint64_t p95_ns = 0;
    std::uint64_t max_ns = 0;
    double gb_per_second = 0.0;
};
```

这些结构分别对应：

- **BenchmarkOptions**：实验配置。
- **RawSample**：每次测量的原始样本。
- **SummaryRow**：由原始样本计算出的汇总。

这里先用 public data struct，因为它们只是数据载体。如果后续需要不变量检查、解析和格式化，可以再封装成类。

数据模型先于代码组织

写 benchmark suite 时，先设计数据模型比先写 runner 更重要。因为最终你要比较的是数据，而不是函数调用本身。哪些字段进入 `RawSample`，决定了后续能不能复查；哪些字段进入 `BenchmarkOptions`，决定了实验是否可配置；哪些字段进入 `SummaryRow`，决定了报告能不能表达趋势。

例如如果 `RawSample` 忘记记录 `distribution`，后面发现随机输入和全零输入差异很大，就无法从旧数据中区分。若忘记记录 `batch`，就无法正确计算 `ns/element`。若忘记记录 `kernel version`，就无法比较手写汇编和 C++ baseline。数据模型缺字段，后续代码再漂亮也补不回历史信息。

因此，benchmark suite 的核心类型应尽量稳定、朴素、可序列化。不要把大量行为塞进这些结构里；它们更像实验记录表的行。复杂逻辑放在 `parser`、`runner`、`summarizer` 和 `writer` 中。

命令行参数：可复现需要显式配置

硬编码常量适合第一章入门，但 benchmark suite 应该能从命令行配置。

示例：

```
./bench_suite \
  --sizes 1024,16384,262144,1048576 \
  --warmup 5 \
  --repetitions 31 \
  --batch 1 \
  --seed 1 \
  --output results/sum_suite
```

解析器可以先写得简单，但要有清晰错误：

```
#include <charconv>
#include <cstdlib>
#include <cstdlib>
#include <string_view>

namespace {

bool parse_u64(std::string_view text, std::uint64_t* value_out)
{
    const char* first = text.data();
    const char* last = text.data() + text.size();
    std::uint64_t value = 0;
    const auto result = std::from_chars(first, last, value);
    if (result.ec != std::errc{} || result.ptr != last) {
        return false;
    }
    *value_out = value;
    return true;
}

} // namespace
```

不要在解析失败后悄悄用默认值。性能报告里的配置必须可信；输入错了就应当失败并打印错误。

深入理解

成熟项目可以使用 `CLI11`、`cxxopts`、`absl flags` 等库。本书早期用手写解析器，是为了让你看清楚参数如何进入实验系统。工程项目中应根据规模选择库，避免自制复杂解析器。

Workload factory：生成输入但不污染计时

对不同 kernel，可以定义不同 workload。

```
#include <cstdlib>
#include <span>
```

```
#include <vector>

struct SumWorkload {
    std::vector<std::uint64_t> values;
    std::uint64_t expected = 0;
};

struct CopyWorkload {
    std::vector<std::uint64_t> source;
    std::vector<std::uint64_t> destination;
};

struct DotWorkload {
    std::vector<float> a;
    std::vector<float> b;
    float expected = 0.0f;
};
```

factory 负责：

- 分配 buffer。
- 初始化输入。
- 计算 reference。
- 保证数据在 timed region 前准备好。

```
std::uint64_t sum_u64_ref(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
    return sum;
}

SumWorkload make_sum_workload(std::size_t elements, std::uint64_t seed)
{
    SumWorkload workload;
    workload.values = make_u64_input(elements, seed);
    workload.expected = sum_u64_ref(workload.values);
    return workload;
}
```

对 `copy_u64`，reference 可以不是另一个 copy 函数，而是校验不变量：

```
after copy:
destination[i] == source[i] for all i
```

对 `fill_u64`：

```
after fill:
destination[i] == fill_value for all i
```

对 `dot_f32`，reference 需要误差容忍：

```
abs(actual - expected) <= epsilon * max(abs(expected), 1.0)
```

Kernel API：统一但不过度抽象

为了让 suite 能跑多个 kernel，需要统一接口。但不要为了统一而隐藏重要差异。

可以从简单函数开始：

```
std::uint64_t sum_u64_kernel(std::span<const std::uint64_t> values)
{
    std::uint64_t sum = 0;
    for (const std::uint64_t value : values) {
        sum += value;
    }
}
```

```

    return sum;
}

void copy_u64_kernel(std::span<std::uint64_t> destination,
                    std::span<const std::uint64_t> source)
{
    for (std::size_t i = 0; i < source.size(); ++i) {
        destination[i] = source[i];
    }
}

void fill_u64_kernel(std::span<std::uint64_t> destination,
                    std::uint64_t value)
{
    for (std::uint64_t& element : destination) {
        element = value;
    }
}

float dot_f32_kernel(std::span<const float> a, std::span<const float> b)
{
    float sum = 0.0f;
    for (std::size_t i = 0; i < a.size(); ++i) {
        sum += a[i] * b[i];
    }
    return sum;
}

```

这里没有强行把所有 kernel 包成同一个函数指针类型，因为它们返回值和参数不同。更实际的做法是每类 kernel 有自己的 runner，然后输出相同的 `RawSample`。

常见误区

过度抽象会污染 benchmark。虚函数调用、`std::function` type erasure、动态分派、分配和锁都可能进入热路径。抽象应放在 `timed region` 外，或者确认编译器能完全内联。

统一输出，不强行统一调用

多 kernel suite 最需要统一的是输出数据，不一定是调用接口。不同 kernel 的参数和返回值天然不同：sum 返回标量，copy 写输出 buffer，fill 接收常量，dot 返回浮点值。若强行用 `std::function<void()>` 包装所有 kernel，可能把类型信息和内联机会都藏掉。

更实用的做法是每类 kernel 有自己的 runner，但所有 runner 都产出同一种 `RawSample`。这样 `timed region` 可以保持直接、简单、可审计，而 suite 的上层仍然能统一写 CSV 和 summary。

这也是系统设计中的常见取舍：不要为了表面统一牺牲关键路径透明度。benchmark 的关键路径尤其敏感，抽象层应停在 `timed region` 外。

dispatch overhead 要单独命名

如果生产系统确实通过函数指针、虚函数、dispatcher 或 plugin 调用 kernel，那么 dispatch overhead 是真实成本，可以测。但这时 benchmark 名称应明确：

```

dot_f32_direct
dot_f32_dispatch
dot_f32_end_to_end

```

direct 测 kernel 上限；dispatch 测选择后调用成本；end-to-end 测完整 operator 路径。三者都合理，但不能混成一个数字。AI 算子库尤其需要这种区分：microkernel 很快，不代表完整 operator 快；完整 operator 慢，也不一定是 microkernel 慢。

校验策略：不同 kernel 不同方法

正确性校验不一定每次都完整扫描，但必须有明确策略。

kernel	校验方式	注意
<code>sum_u64</code>	比较返回值。	使用 unsigned 避免 signed overflow UB。
<code>copy_u64</code>	比较输出和输入。	校验会扫描内存，不放入 timed region。
<code>fill_u64</code>	检查元素等于常量。	测量前可能要重置 buffer。
<code>dot_f32</code>	误差容忍比较。	说明容忍度和 reference 顺序。

对于会修改输出 buffer 的 kernel，要注意每次测量前的初始状态。比如 `fill_u64`：

```
for each repetition:
    reset destination outside timed region if required
    time fill_u64(destination, value)
    validate destination outside timed region
```

如果 reset 成本很大，不能混进 timed region；如果不 reset，又要确认 kernel 的性能不依赖原始内容。

校验频率也是设计选择

最保守的做法是每个 measured sample 后都完整校验。这最安全，但对大输出 buffer 可能很慢。如果校验不在 timed region，慢一些通常可以接受；但若实验矩阵很大，完整校验会让总运行时间变长。

可以采用分层策略：

- debug 模式：每次 sample 后完整校验。
- release benchmark：每个 size 的 warmup 前后完整校验，measured sample 做轻量 checksum。
- nightly 验证：随机输入和边界输入做完整 correctness sweep。

无论选择哪种策略，都要写进报告。不能因为校验贵就完全不校验，也不能把轻量 checksum 当作完整正确性证明。尤其是 copy/fill 这类输出 buffer kernel，少量 sentinel 正确不代表全数组正确。

浮点校验要记录误差模型

浮点 kernel 的 reference 可能使用不同求和顺序，优化版本可能使用 FMA、向量化、展开或 fast-math。结果有误差是正常的，但误差必须有边界。报告应记录：

- reference 使用什么顺序和精度。
- absolute tolerance 和 relative tolerance。
- 是否允许 NaN/Inf。
- 是否启用 fast-math 或 flush-to-zero。

如果误差超出预期，不能继续只讨论性能。性能优化改变数值语义时，必须把它作为算法取舍，而不是实现细节。

样本汇总：从 raw 到 summary

summary 应由 raw samples 计算。最小实现：

```
#include <algorithm>
#include <cstdint>
#include <vector>

std::uint64_t percentile_sorted(const std::vector<std::uint64_t>& sorted,
                                std::size_t numerator,
                                std::size_t denominator)
{
    if (sorted.empty()) {
        return 0;
    }
    const std::size_t last_index = sorted.size() - 1;
    const std::size_t index = (last_index * numerator + denominator - 1) / denominator;
    return sorted[index];
}
```



```

}

SummaryRow summarize_samples(std::string kernel_name,
                             std::size_t elements,
                             std::size_t bytes,
                             int batch_count,
                             std::vector<std::uint64_t> elapsed_ns)
{
    constexpr std::size_t P95_NUMERATOR = 95;
    constexpr std::size_t PERCENT_DENOMINATOR = 100;
    constexpr double BYTES_PER_GIGABYTE = 1.0e9;
    constexpr double NANOSECONDS_PER_SECOND = 1.0e9;

    std::sort(elapsed_ns.begin(), elapsed_ns.end());

    SummaryRow row;
    row.kernel_name = std::move(kernel_name);
    row.elements = elements;
    row.bytes = bytes;
    row.batch_count = batch_count;
    row.repetitions = static_cast<int>(elapsed_ns.size());
    row.min_ns = elapsed_ns.front();
    row.median_ns = elapsed_ns[elapsed_ns.size() / 2];
    row.p95_ns = percentile_sorted(elapsed_ns, P95_NUMERATOR, PERCENT_DENOMINATOR);
    row.max_ns = elapsed_ns.back();

    const double seconds = static_cast<double>(row.median_ns) / NANOSECONDS_PER_SECOND;
    row.gb_per_second = static_cast<double>(bytes) / seconds / BYTES_PER_GIGABYTE;
    return row;
}

```

这个实现还有局限：

- 空样本只返回 0，生产代码应报告错误。
- p95 使用简单 nearest-rank 近似。
- GB/s 默认每个 sample 只处理一次 **bytes**，如果 batch 大于 1，公式要乘以 batch。

所以更准确：

```

effective_bytes = bytes * batch_count
GB/s = effective_bytes / seconds / 1e9

```

派生指标：不要只给时间

不同 kernel 应输出不同派生指标。

sum_u64

显式读：

```

bytes_read = n * sizeof(uint64_t)
GB/s = bytes_read * batch / seconds / 1e9
ns/element = elapsed_ns / (n * batch)

```

copy_u64

copy 既读又写：

```

bytes_read = n * sizeof(uint64_t)
bytes_written = n * sizeof(uint64_t)
logical_bytes = bytes_read + bytes_written
GB/s = logical_bytes * batch / seconds / 1e9

```

硬件实际流量可能更高，因为写 miss 可能触发 write allocate。后续讲内存系统时会展开。

fill_u64

fill 主要写：

```

bytes_written = n * sizeof(uint64_t)

```

```
GB/s = bytes_written * batch / seconds / 1e9
```

但普通 store 可能也触发 write allocate。非时序 store 是另一个话题。

dot_f32

每个元素通常算 1 次乘法和 1 次加法：

```
FLOPs = 2 * n
GFLOP/s = FLOPs * batch / seconds / 1e9
bytes_read = 2 * n * sizeof(float)
```

如果使用 FMA，仍通常按 2 FLOPs 计。

派生指标的口径陷阱

派生指标很有用，但也最容易误导。GB/s、GFLOP/s、ns/op、speedup 都不是计时器直接测出来的事实，而是根据 elapsed time 和模型计算出来的指标。模型变了，指标就变。

GB/s 的 bytes 要说明口径

对 sum，通常 bytes 是读取的元素总字节数。对 copy，bytes 可以只算读，也可以算读加写。对 fill，bytes 可以只算写，也可以讨论 write allocate 造成的额外硬件流量。不同口径会得到不同 GB/s。

例如 copy_u64 拷贝 n 个 8 字节元素：

```
read_bytes = n * 8
write_bytes = n * 8
logical_bytes = read_bytes + write_bytes
```

如果按 read_bytes 算，GB/s 是一种值；按 logical_bytes 算，数值翻倍。两种都可以，但必须说明。若要和 memcpy 或内存带宽工具比较，也要看对方使用什么口径。

更复杂的是硬件实际流量。普通写入一个未在 cache 中的 line，可能先读入 cache line 再写，这叫 write allocate。此时硬件总流量可能大于 logical bytes。第一册不深入建模，但报告至少要知道：logical GB/s 是算法口径，不一定等于内存总线真实流量。

GFLOP/s 依赖 FLOP 计数约定

dot product 通常每个元素算一次乘法和一次加法，所以记 2 FLOPs。若编译器使用 FMA，一条 FMA 指令也通常按 2 FLOPs 计。这是性能报告的常见约定。

但不是所有浮点表达式都这么简单。比如带条件的计算、近似函数、特殊值处理、向量化尾部、fast-math 改写都会让 FLOP 计数变得模糊。报告 GFLOP/s 时必须写出公式。若公式只是估算，就写成 estimated FLOPs。

不要用 GFLOP/s 掩盖数值语义变化。一个版本启用 fast-math 后 GFLOP/s 更高，但它可能改变 NaN、Inf、signed zero、舍入顺序或异常行为。此时性能提升和语义放宽是同一件事的两个方面，必须一起报告。

Speedup 要说明方向和基准

speedup 的方向必须明确：

```
time_speedup = baseline_time / candidate_time
throughput_ratio = candidate_throughput / baseline_throughput
```

如果 time speedup 大于 1，候选版本更快；如果 throughput ratio 大于 1，候选版本吞吐更高。两者都能表达性能提升，但不要混用。

还要说明 baseline 是谁。比如：

```
baseline = scalar C++ 03
candidate = manually unrolled C++ 03
```

或：

```
baseline = previous commit 5364eb0
candidate = current working tree
```

没有 baseline 名称的 speedup 没有完整语义。

小时间差不要过度解释

如果 baseline median 是 100 ns, candidate median 是 97 ns, 看起来有 3% 提升。但如果样本噪声、计时器开销、频率波动或重复运行差异也在几个百分点范围内, 这个提升可能不稳定。短 kernel 尤其如此。

报告小幅提升时要看：

- raw samples 是否分布分离。
- 多次独立运行是否复现。
- p95 和 max 是否也改善。
- 反汇编是否支持机制解释。
- PMU 或其他工具是否支持瓶颈变化。

本书不要求第一册引入复杂统计检验, 但要求你不把微小差异夸大成确定结论。可以写“观察到约 3% 趋势, 仍需重复实验确认”, 这比硬写“优化成功”更可靠。

输出 CSV 的代码结构

不要在测量循环里频繁 flush。先收集样本, 最后写文件。

```
#include <stdio>
#include <string>
#include <vector>

void write_raw_samples_csv(const std::string& path,
                          const std::vector<RawSample>& samples)
{
    FILE* file = std::fopen(path.c_str(), "w");
    if (file == nullptr) {
        std::perror("fopen raw samples");
        std::abort();
    }

    std::fprintf(file,
                 "kernel,n,bytes,batch,rep,elapsed_ns,checksum\n");

    for (const RawSample& sample : samples) {
        std::fprintf(file,
                     "%s,%zu,%zu,%d,%d,%llu,%llu\n",
                     sample.kernel_name.c_str(),
                     sample.elements,
                     sample.bytes,
                     sample.batch_count,
                     sample.repetition,
                     static_cast<unsigned long long>(sample.elapsed_ns),
                     static_cast<unsigned long long>(sample.checksum));
    }

    std::fclose(file);
}
```

这段代码使用 C stdio 是为了简单直接。生产代码可以用 RAII 封装文件句柄, 也可以使用 C++ 文件流。关键不是 API 选择, 而是: 文件写入不在 timed region 里。

Runner：把一次实验组织起来

下面是一个简化的 sum runner。它展示结构, 不追求覆盖所有错误处理。

```
#include <chrono>
#include <cstdint>
#include <span>
#include <vector>

namespace {
```

```

using Clock = std::chrono::steady_clock;

template <class T>
inline void do_not_optimize(const T& value)
{
    asm volatile("" : : "g"(value) : "memory");
}

std::uint64_t elapsed_ns_since(Clock::time_point begin, Clock::time_point end)
{
    return static_cast<std::uint64_t>(
        std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count());
}

std::uint64_t run_sum_batch(std::span<const std::uint64_t> values,
                           int batch_count)
{
    std::uint64_t combined = 0;
    for (int batch = 0; batch < batch_count; ++batch) {
        combined += sum_u64_kernel(values);
    }
    return combined;
}

void warmup_sum(const SumWorkload& workload,
               int warmup_repetitions,
               int batch_count)
{
    for (int iter = 0; iter < warmup_repetitions; ++iter) {
        const std::uint64_t result = run_sum_batch(workload.values, batch_count);
        do_not_optimize(result);
    }
}

RawSample measure_sum_once(const SumWorkload& workload,
                          std::size_t elements,
                          int batch_count,
                          int repetition)
{
    const auto begin = Clock::now();
    const std::uint64_t result = run_sum_batch(workload.values, batch_count);
    const auto end = Clock::now();

    do_not_optimize(result);

    RawSample sample;
    sample.kernel_name = "sum_u64";
    sample.elements = elements;
    sample.bytes = elements * sizeof(std::uint64_t);
    sample.batch_count = batch_count;
    sample.repetition = repetition;
    sample.elapsed_ns = elapsed_ns_since(begin, end);
    sample.checksum = result;
    return sample;
}

} // namespace

```

注意：

- warmup 和 measurement 分开。
- timed region 只包 batch kernel。
- **RawSample** 记录 batch 和 checksum。
- 校验可以在外层比较 checksum 和 expected batch result。

expected batch result：

```
expected_batch = workload.expected * batch_count
```

因为使用 `std::uint64_t`，乘法和加法都按模 2^{64} 回绕，语义明确。

参数扫描：矩阵比单点更重要

真实 benchmark 通常是多维矩阵：

```
kernel:
    sum_u64, copy_u64, fill_u64, dot_f32

size:
    1 KiB, 16 KiB, 256 KiB, 4 MiB, 64 MiB

distribution:
    zero, lcg, increasing

cache mode:
    warm, cold-ish

batch:
    1, auto
```

不要一开始就跑满所有组合。组合爆炸会浪费时间。训练建议：

1. 先固定 distribution 和 cache mode，扫描 size。
2. 发现异常后，增加 distribution 维度。
3. 对短 kernel 引入 batch。
4. 对内存 kernel 引入 cold-ish 模式。
5. 最后再做完整矩阵。

CSV 里必须记录每个维度，否则后续无法比较。

自动 batch 的思路

自动 batch 的目标是让单个 sample 足够长。例如希望每个 sample 至少 100 微秒：

```
target_sample_ns = 100000
batch = 1
while measured_ns(batch) < target_sample_ns:
    batch *= 2
```

但这个校准本身也有成本和噪声。它应该在正式测量前完成，并记录最终 batch。

```
int choose_batch_count(std::span<const std::uint64_t> values,
                      std::uint64_t target_ns)
{
    constexpr int BENCHMARK_MAX_BATCH = 1 << 20;

    int batch_count = 1;
    while (batch_count < BENCHMARK_MAX_BATCH) {
        const auto begin = Clock::now();
        const std::uint64_t result = run_sum_batch(values, batch_count);
        const auto end = Clock::now();
        do_not_optimize(result);

        if (elapsed_ns_since(begin, end) >= target_ns) {
            break;
        }
        batch_count *= 2;
    }
    return batch_count;
}
```

风险：

- 校准时的数据会变热。
- batch 变大后测的是重复运行场景。
- 不同机器上 batch 不同，比较时要记录。

避免 harness 干扰热路径

benchmark framework 本身也会影响结果。常见干扰：

- 使用 `std::function` 包装 kernel，导致间接调用。
- 在 timed region 里捕获 lambda，编译器无法内联。
- 每次 sample 创建临时 vector。
- 每次 sample 写日志或 CSV。
- 校验代码意外进入 timed region。
- runner 过度泛型，导致编译器生成复杂控制流。

对热路径最稳的办法是：

```
outside timed region:
    choose kernel
    construct workload
    prepare function-specific runner

inside timed region:
    direct call or fully inlined call
```

如果使用函数指针：

```
using SumKernel = std::uint64_t (*)(std::span<const std::uint64_t>);
```

函数指针调用可能不能内联，但可以模拟真实动态 dispatch。如果你的生产系统就是通过函数指针做 ISA dispatch，这可能是合理的。如果目标是测 kernel 本体上限，应尽量测 direct call 或单独测 dispatch overhead。

深入理解

AI 算子库常有 dispatcher：根据 dtype、shape、ISA、线程数选择 kernel。优化时要分清两个 benchmark：一个测 end-to-end operator dispatch，一个测已经选定的 microkernel。二者都有价值，但不能混为一谈。

四个基础 kernel 的设计

sum_u64

目的：

- 训练只读 streaming kernel。
- 观察 cache 层级和内存带宽。
- 学习 DCE 防护和 checksum。

语义：

```
sum = 0
for x in values:
    sum += x
return sum
```

指标：

```
GB/s = n * sizeof(uint64_t) * batch / seconds / 1e9
ns/element = elapsed_ns / (n * batch)
```

copy_u64

目的：

- 训练读加写 memory kernel。
- 观察 write allocate、带宽上限和 memcpy 对比。
- 学习输出 buffer 校验。

语义：

```
for i in [0, n):
    dst[i] = src[i]
```

指标：

```
logical bytes = 2 * n * sizeof(uint64_t)
```

fill_u64

目的：

- 训练写密集 kernel。
- 观察 store 带宽、write allocate 和初始化成本。
- 后续可对比普通 store 与 non-temporal store。

语义：

```
for i in [0, n):
    dst[i] = value
```

dot_f32

目的：

- 训练浮点计算和内存读取混合 kernel。
- 为后续 SIMD、FMA、reduction 打基础。
- 学习 GFLOP/s 和数值误差报告。

语义：

```
sum = 0
for i in [0, n):
    sum += a[i] * b[i]
return sum
```

指标：

```
FLOPs = 2 * n
bytes_read = 2 * n * sizeof(float)
```

报告中的图表和文字

就算没有画图，也要让表格能表达趋势。推荐 summary 表：

```
kernel,n,bytes,median_ns,Gbps,ns_per_element,notes
sum_u64,128,1024,80,12.8,0.625,L1-sized
sum_u64,32768,262144,9000,29.1,0.275,L2-sized
sum_u64,8388608,67108864,3500000,19.2,0.417,memory-sized
```

报告文字不要只写“随着 n 增大性能下降”。要写因果假设：

```
Observation:
    Throughput rises from 1 KiB to 256 KiB, then drops at 64 MiB.

Hypothesis:
    Small sizes are dominated by loop and timer overhead.
    Middle sizes benefit from cache residency.
    Large sizes are limited by memory bandwidth and TLB pressure.

Evidence:
    objdump shows scalar loop, no SIMD.
    perf stat shows cache misses increase at large n.

Limit:
```

```
CPU frequency was not pinned; result needs repeat under performance governor.
```

图表要回答问题

图表不是装饰。每张图都应回答一个问题。若问题是“吞吐如何随规模变化”，横轴用 `n` 或 `bytes`，纵轴用 `GB/s`；若问题是“短 kernel 固定开销多大”，纵轴用 `ns/op` 或 `ns/element`；若问题是“样本是否稳定”，使用散点图或箱线图比单条折线更合适；若问题是“两个版本差异是否稳定”，可以画 `ratio`：

```
speedup = baseline_median_ns / optimized_median_ns
```

但 `ratio` 也要小心。小规模下 `baseline` 很短，计时噪声可能让 `ratio` 看起来很夸张；大规模下两个版本都受内存带宽限制，`ratio` 可能接近 1，但这不代表优化无价值，可能说明瓶颈已经转移。

异常点要解释

如果某个 `size` 的性能突然下降，不要直接删掉。先问：

- 是否跨过某级 `cache` 或 `TLB` 覆盖范围。
- `batch` 是否在该 `size` 改变。
- 是否触发了不同代码路径或尾部处理。
- 输入分布是否不同。
- 是否发生系统干扰或频率变化。
- 反汇编是否显示编译器生成了不同版本。

异常点往往是最有价值的信息。它可能暴露 `bug`、边界条件、`cache` 层级、分支预测变化或 `harness` 问题。高质量报告应标出异常点，并说明已验证和未验证的解释。

报告要连接三类证据

最终文字应连接三类证据：数据证据、二进制证据和硬件证据。数据证据来自 `raw/summary`；二进制证据来自 `objdump` 或编译器输出；硬件证据来自 `perfstat`、系统环境和硬件规格。三者一致时，结论最稳。

例如：

```
Data:
    optimized version is faster for 16 KiB..256 KiB, but equal at 64 MiB.

Binary:
    optimized version uses AVX2 loads and vector reduction.

Hardware:
    large-size GB/s is close to measured memory copy bandwidth.

Interpretation:
    vectorization helps while data is cache-resident; large size becomes memory-bound.
```

这种写法比“AVX2 更快”更准确，因为它说明了在哪些规模更快，为什么大规模不继续提升。

核心思想

benchmark 报告的主线应是“问题、方法、数据、证据、解释、限制”。图表和表格服务这条主线，不是越多越好。

正式报告模板：让别人能复核你的结论

很多 benchmark 失败不是因为计时代码完全错误，而是报告无法复核。读者看到一张表，却不知道输入是什么、版本是什么、编译参数是什么、计时区域在哪里、结果是否正确、异常点如何处理。这样的报告即使数字真实，也很难成为工程证据。

一份正式 benchmark 报告至少应包含下面九部分。

一、问题和结论先行

报告开头先写清楚问题，而不是先堆数据。问题应该具体到版本、workload 和指标。例如：

```
Question:
  Does the unrolled sum_u64 implementation improve warm-cache throughput
  over the scalar baseline on this machine?

Primary metric:
  median ns/element and logical GB/s

Scope:
  single-threaded, Linux x86-64, clang++ -O3 -march=native
```

然后给出一句有边界的结论：

```
Conclusion:
  For 16 KiB to 256 KiB inputs, the unrolled version improves median
  throughput by 8% to 12%. For 64 MiB inputs, both versions are close,
  suggesting memory bandwidth dominates. The result is not a claim about
  cold-cache latency or multi-threaded throughput.
```

这种写法让读者一开始就知道报告回答什么、不回答什么。不要把结论写成“优化版更快”这种无边界句子。

二、被测对象

列出被测 kernel、版本、源码路径和构建方式。若比较多个版本，必须说明 baseline 和 candidate。

```
Kernel:
  sum_u64(std::span<const std::uint64_t>)

Baseline:
  scalar_cpp, source: kernels/sum.cpp

Candidate:
  unrolled4_cpp, source: kernels/sum_unrolled.cpp

Correctness reference:
  sum_u64_ref, simple loop with unsigned wraparound semantics
```

如果 baseline 是旧 commit 或生产版本，要写 commit；如果 candidate 来自 dirty worktree，也要写。dirty 不代表不能测，但要诚实记录，否则别人无法复现。

三、实验环境

环境不是附录里的装饰，而是解释数据的前提。至少写：

- CPU 型号、核心数、SMT 状态。
- cache 信息。
- 内存容量。
- 操作系统和 kernel 版本。
- 编译器版本和编译参数。
- CPU governor、是否固定核心、是否在容器中。
- PMU 权限是否可用。

报告中可以只摘要关键字段，完整输出放进 `environment.txt`。摘要的作用是让读者快速判断这份报告与自己的机器是否可比；完整文件的作用是后续复查。

四、workload 矩阵

workload 矩阵要列出所有自变量：

```
sizes:
  1 KiB, 16 KiB, 256 KiB, 4 MiB, 64 MiB

distribution:
  deterministic LCG, seed = 1
```

```
alignment:
    std::vector allocation, observed pointer alignment recorded separately

cache mode:
    warm-cache repeated measurement

batch:
    auto, target sample time = 100 us
```

这部分最容易漏掉 cache mode 和 batch。一个 warm-cache batch benchmark 不能自动代表 cold-cache 首次访问；一个 auto batch benchmark 不能和 batch 1 的端到端延迟直接比较。

五、计时协议

计时协议说明哪些工作在 timed region 内，哪些不在。

```
Outside timed region:
    allocate vectors
    generate input
    compute reference
    warmup
    validate output
    write CSV

Inside timed region:
    repeat selected kernel batch times
    combine scalar result
```

如果 kernel 本身包含分配或 I/O，也要明确写入 timed region。关键不是一定排除所有开销，而是计时边界必须匹配问题陈述。

六、正确性和防优化策略

正式报告必须说明结果如何被观察，防止优化器删除。对返回标量的 kernel，可以记录 checksum；对输出 buffer 的 kernel，要说明输出如何校验。

```
Correctness:
    Each size is checked against reference before measured repetitions.
    For measured repetitions, checksum is stable across samples.

Optimization barrier:
    Scalar results are passed to do_not_optimize().
    Output buffers are sampled after the timed region and fully checked
    outside the timed region.
```

如果没有 correctness，只能叫计时实验，不能叫性能结论。尤其是快得离谱的结果，第一怀疑对象就是代码被删、结果没被用、输入常量或校验缺失。

七、原始数据和汇总

报告正文可以放 summary 表，但必须给出 raw data 路径。summary 中最好包含 min、median、p95、max，而不是只放一个平均值。

```
Artifacts:
    raw samples: results/ch16/raw_samples.csv
    summary: results/ch16/summary.csv
    objdump: results/ch16/objdump.txt
    perf: results/ch16/perf_stat.txt
```

若有异常点，不要只在 summary 中隐藏。要写明 raw 中保留了异常点，summary 使用哪个统计量，以及异常点是否改变结论。

八、二进制和硬件证据

至少给出一段二进制证据。比如：

```
Binary audit:
    baseline loop uses one load and one add per element.
```



```
candidate loop processes four elements per iteration.
no unexpected calls are present in the hot loop.
```

若使用 `perfstat`，报告应说明事件与假设的关系，而不是简单贴输出。例如 `branch benchmark` 应关心 `branches` 和 `branch-misses`；内存扫描应关心 `cache misses`、`cycles`、`instructions` 和带宽口径；短 `kernel` 应先判断计时器和 `batch` 是否主导。

九、限制和下一步

限制不是削弱报告，而是提高可信度。一个专业结论总有边界：

- 只测单线程，不代表多线程。
- 只测 `warm cache`，不代表首次访问。
- 只测一台机器，不代表所有 `x86-64`。
- 只测一种编译器，不代表其他优化器。
- `PMU` 权限不足，硬件事件证据缺失。
- `CPU` 频率未固定，小幅差异需要复测。

下一步应来自限制，而不是随便列愿望。例如“需要在 `performance governor` 下复测”“需要加入 `cold-cache` 模式”“需要用 `perfstat-ebranches,branch-misses` 验证分支解释”。这样后续工作才有方向。

核心思想

正式 `benchmark` 报告不是把表格贴出来，而是把问题、协议、证据和限制组织成别人能复核的论证。

结论分级：哪些话可以说，哪些话不能说

性能报告最容易过度表达。看到一个候选版本在一次运行中快了，就写“优化成功”；看到 `perf` 里 `cache miss` 变多，就写“瓶颈是 `cache`”；看到 `AVX2` 指令，就写“`SIMD` 带来提升”。这些说法可能对，也可能证据不够。为了避免过度结论，本书建议把性能结论分成四级。

等级	可以说什么	还不能说什么
观察	某次样本或 <code>summary</code> 显示了某个数字。	不能说差异来自算法。
趋势	多次样本或多个规模显示一致方向。	不能说已排除所有混杂因素。
解释	数据、汇编和环境证据支持某个机制。	不能把机制推广到所有机器。
结论	在明确协议下，某版本优于或不优于 <code>baseline</code> 。	不能超出协议边界宣传。

例如：

```
Observation:
Candidate median is 9% lower at n = 32768.

Trend:
Candidate is 8% to 12% lower for 16 KiB, 64 KiB and 256 KiB.

Explanation:
Objdump shows loop unrolling reduced branch overhead, and instruction
count decreases in perf stat. Cache misses are similar.

Conclusion:
Under warm-cache single-threaded measurement on this machine, unrolling
improves mid-size sum_u64 throughput. It does not improve 64 MiB inputs,
where both versions appear memory-bound.
```

这四级能帮你控制语气。很多时候，实验只支持“观察到趋势”，还不支持“机制已证明”。这不是失败，而是诚实。

不要把缺失证据补成故事

人很擅长给数字编故事。一个版本快了，就说“cache locality 更好”；一个版本慢了，就说“分支预测失败”；一个点跳变，就说“跨过 L3”。这些都可能是真的，但必须有证据。没有证据时，应写成假设：

```
Hypothesis:
  The drop at 64 MiB may be caused by memory bandwidth or TLB pressure.

Evidence available:
  Working set exceeds LLC according to lscpu.

Evidence missing:
  No perf stat cache/TLB event was collected yet.
```

把证据缺口写出来，读者就知道如何继续验证。把假设写成结论，只会让报告变得脆弱。

常见反模式：看起来专业但不能信

反模式一：只报告最快一次

有些报告只写“best of N”。最小值确实有用途，它常被用来近似低干扰状态，但不能单独代表整体。若只报告最快一次，读者不知道样本是否稳定、是否有长尾、是否偶然碰到理想调度。更好的做法是同时报告 min、median、p95、max，并说明最终结论主要依据哪个统计量。

反模式二：把 setup 放进 timed region 后又解释 kernel

如果计时区域包含分配、随机数生成、初始化和打印，报告就不能只解释 kernel。也许优化版 kernel 快了，但 setup 主导总时间；也许 kernel 没变，随机数生成器变了。计时区域包含什么，结论就只能关于什么。

反模式三：没有 reference 却比较速度

没有 reference 或 correctness check，快慢没有意义。尤其是浮点、边界条件、尾部处理、手写汇编和 SIMD kernel，错误实现常常比正确实现更快。报告中看到巨大 speedup，第一步不是赞叹，而是找 correctness 证据。

反模式四：比较不同编译参数

baseline 用 `-O2`，candidate 用 `-O3-march=native`，然后说 candidate 更快，这是混淆变量。除非你的问题就是“不同构建配置的端到端影响”，否则比较实现时应保持编译参数一致。若必须改变编译参数，要把它作为自变量写进矩阵。

反模式五：只测一个输入规模

单点结果非常危险。小规模可能被固定开销主导，中规模可能被 cache 主导，大规模可能被内存带宽主导，尾部规模可能被边界处理主导。一个优化可能只在其中一个区域有意义。只测一个点，很容易把局部事实当成全局规律。

反模式六：把微基准当端到端结论

微基准能证明某个小 kernel 在受控条件下的行为，但不能直接代表完整应用。真实系统可能有 dispatch、内存分配、数据转换、线程调度、I/O、锁和其他算子。报告可以写“microkernel 更快”，不能直接写“应用会更快”，除非端到端也验证。

反模式七：图表没有单位和口径

GB/s 的 bytes 是读字节、写字节还是读写合计？GFLOP/s 的 FLOP 公式是什么？ns/op 中的 op 是一次函数调用、一次元素处理还是一次 batch？没有单位和口径，图表就只剩形状，不能复核。

反模式八：只贴 perf 输出不解释

perfstat 输出不是自动结论。不同事件可能受权限、复用、平台支持和采样误差影响。报告要说明选择哪些事件、它们对应哪个假设、结果是否支持假设。如果只是贴一大段输出，读者仍然不知道

该看哪里。

反模式九：把 schema 变化当性能变化

如果某天把 copy 的 bytes 口径从只读改成读写合计，GB/s 会看起来翻倍。若不记录 schema 版本，这种指标变化很容易被误解为优化成功。任何派生指标的公式变化，都必须视为实验协议变化。

反模式十：过度清洗数据

异常值可以分析，可以标注，可以在 summary 中使用稳健统计量降低影响，但不能偷偷删除。若删除样本，必须记录规则和原因。否则报告无法复核，也可能把真实长尾问题清理掉。

常见误区

专业 benchmark 的危险不在于数字不够漂亮，而在于漂亮数字没有证据。反模式的共同点是：隐藏变量、隐藏协议、隐藏原始数据或隐藏失败。

案例：一次快得不合理的优化

假设你把 `sum_u64` 改成一个新版本，summary 显示 candidate 比 baseline 快 100 倍。不要立刻宣布优化成功。按本章 checklist 排查。

第一种可能：循环被删除

如果 candidate 的结果没有被使用，编译器可能删除整个循环。现象是 elapsed time 极小，反汇编里找不到目标 loop，checksum 可能恒定或缺失。处理方法是确保结果进入 `do_not_optimize()`，并且报告中保存 objdump 证据。

第二种可能：输入被常量折叠

如果输入是编译期常量，编译器可能提前计算结果。现象是机器码中没有实际 load，或者只返回常数。处理方法是在运行时生成输入，把生成放在 setup 中，并保证编译器不能把全部输入推导为常量。

第三种可能：计时区域测错了

可能 begin 和 end 放错位置，测到了空区域；也可能 batch 为 0；也可能 elapsed ns 发生整数截断或单位转换错误。处理方法是检查 raw samples 中的 batch、elapsed、n 和 bytes，加入基本 sanity check：

```
assert(batch_count > 0)
assert(elements > 0)
assert(elapsed_ns > 0)
```

第四种可能：baseline 被不公平削弱

baseline 也许使用调试构建、禁用优化、额外校验在 timed region 内、或使用不同输入。处理方法是比较编译命令、计时协议和反汇编。speedup 来自 candidate 变好和 baseline 变坏是两件不同的事。

第五种可能：真实优化但边界很窄

也可能 candidate 真的在某个小规模下快很多，例如把函数调用和循环开销消除了。但这仍然需要说明边界。它是否在大规模仍快？是否结果正确？是否适用于真实 workload？是否只是 warm-cache batch 场景？结论必须跟证据范围一致。

这个案例说明，异常好看的数字和异常糟糕的数字一样，都需要排查。高质量性能工程不是拒绝惊喜，而是要求惊喜通过证据审计。

把 benchmark 当成工程资产

本章的设计比临时代码计时更复杂。原因是 benchmark 一旦进入项目，就不再只是一次实验，而是工程资产。它会被用来比较版本、发现回归、指导优化、说服 reviewer、写文档、做发布决策。工程资产必须可维护。

可维护 benchmark 有几个特征。

第一，新增 kernel 成本低。已有 options、CSV、summary、environment 和 report 机制可以复

用，新增 kernel 只需要定义 workload、runner、checker 和指标公式。

第二，旧结果可读。几个月前的结果目录仍然能说明当时的命令、环境、schema、源码和统计口径。

第三，失败容易定位。checksum 失败、样本缺失、schema 不匹配、二进制异常、环境缺失都能被明确报告，而不是变成一个神秘数字。

第四，结论可降级。证据不足时，可以从“结论”降级为“趋势”或“观察”，而不是强行维持过度说法。

第五，能服务教学和生产。教学中它帮助学生理解变量；生产中它帮助工程师避免性能回归。二者都依赖同一个基础：协议清楚、数据可复核。

核心理想

benchmark suite 的最高价值不是某一次跑出的数字，而是长期积累的可复核性能证据。它应该像测试、文档和构建脚本一样被维护。

性能回归与 CI：不要把噪声当闸门

benchmark suite 很自然会进入 CI：每次改代码都跑一次，看看性能是否退化。这个方向是对的，但直接把“慢 1% 就失败”写成硬规则，通常会制造大量误报。CI 环境常常共享机器、频率不可控、后台负载不可控、PMU 权限受限，性能波动可能比真实改动还大。

更实际的策略是分层。第一层是 correctness 和构建检查，必须稳定通过。第二层是 smoke benchmark，只跑少量规模，用来发现数量级退化，例如慢 2 倍、结果被优化器删除、CSV 写坏。第三层是受控机器上的 nightly 或手工 benchmark，用更多重复、更多输入规模和环境记录判断小幅回归。第四层才是发布前的完整性能报告。

层级	目标	适合阈值
PR smoke	发现严重退化和实验系统损坏。	宽阈值，例如 20% 到 50%。
nightly	跟踪趋势和中等退化。	中等阈值，结合多日趋势。
受控机器	判断小幅性能变化。	窄阈值，但需重复和审计。
发布报告	给出工程结论。	不只看阈值，还要解释机制。

回归阈值要按 kernel 和规模设置

所有 kernel 用同一个阈值通常不合理。短 kernel 噪声大，1 ns 的绝对波动就可能是几个百分点；长时间 streaming kernel 更稳定，但受内存和频率影响；分支敏感 kernel 受输入分布影响；浮点 kernel 还要同时看误差变化。

阈值应至少考虑三件事。第一，历史波动范围：过去一段时间同一配置自然波动多大。第二，工程重要性：这个 kernel 是否在真实场景中频繁调用。第三，证据成本：若阈值很窄，就必须使用更受控环境和更多重复。

例如可以先使用宽松规则：

```
if median regression > 20%:
    fail smoke benchmark
elif median regression > 5%:
    mark as warning and require review
else:
    record trend only
```

这不是最终统计方法，但比小幅波动直接失败更实用。真正要判断 2% 回归，应在稳定机器上复测，并查看 raw samples、反汇编和环境。

CI 报告要保留 artifact

CI benchmark 若只在日志里打印 summary，很快就无法排查。至少应上传：

- raw samples。
- summary。
- command。

- environment。
- binary 或 build id。
- objdump 关键片段。

当某次 CI 显示退化时，reviewer 需要判断它是真退化、环境噪声、schema 变化、baseline 变化，还是 harness bug。没有 artifact，就只能重新跑；重新跑又可能不复现。

性能趋势比单点更有价值

长期看，一次 benchmark 失败不如趋势重要。如果某个 kernel 连续多天缓慢变慢，单次都没超过阈值，最终仍可能形成明显退化。反过来，某次 CI 变慢 10%，下一次恢复，可能只是机器噪声。保存历史 summary 可以让你画出趋势，并把性能变化和 commit、编译器升级、硬件迁移联系起来。

趋势分析仍要警惕协议变化。若某天修改了 bytes 口径、batch 策略、输入分布或 baseline，趋势图必须标注 schema 或 protocol change。否则趋势断点会被误解成性能变化。

核心思想

CI 中的 benchmark 首先是报警系统，不是最终裁判。小幅性能结论需要受控复测；CI 更适合发现大退化、趋势变化和实验系统损坏。

benchmark suite 的架构原则

一个可维护的 benchmark suite 不应该是一个越写越长的 `main.cpp`。它应当有清楚的职责分层：命令行解析负责把实验配置显式化，workload factory 负责生成输入，kernel registry 负责列出可测实现，runner 负责组织重复和计时，checker 负责正确性，recorder 负责写 raw 数据，summarizer 负责汇总，report 层负责解释。每层只做自己的事，实验系统才可审计。

可以用下面结构理解：

```

config
  sizes, repetitions, warmup, seed, kernels, output dir

workload factory
  create input and reference objects

kernel registry
  name -> function pointer or callable

runner
  for each workload, kernel, repetition:
    run warmup if needed
    time batch
    collect checksum

checker
  compare candidate output with reference

recorder
  append raw sample CSV

summarizer
  compute min, median, p95, max, derived metrics

report
  connect summary, objdump, environment, limitations

```

这种架构看起来比临时脚本复杂，但它能解决三个长期问题。第一，新增 kernel 不需要复制整套计时代码。第二，修改统计口径可以集中处理，不会每个实验一套公式。第三，失败时能定位是哪一层坏了：输入生成、正确性、计时、记录、汇总还是报告。

配置要显式，不要藏在代码常量里

实验配置应尽量来自命令行或配置文件，并写入结果目录。输入规模、重复次数、warmup、seed、batch、cache mode、kernel 列表、输出路径，都不应只藏在源码常量里。源码常量可以有默认值，但运行时实际使用的值必须记录。

显式配置有两个好处。第一，可复现。别人可以用同一命令重新跑。第二，可比较。两个结果目

录如果命令不同，就能立即看出差异来自哪里。若配置藏在代码里，比较两个日期的结果时还要查 commit，甚至查不出来当时改了什么。

kernel registry 要有稳定名称

每个被测实现都需要稳定名称，例如 `scalar_ref`、`scalar_opt`、`asm_scalar`、`avx2_unrolled4`。名称会进入 CSV、报告、历史趋势和 CI 阈值。不要使用容易变化的函数地址或临时描述。若实现语义变了，名称也应变，或通过 `version` 字段记录。

registry 还应记录 kernel 属性：是否 reference、是否需要特定 ISA、是否允许某些输入、是否写输出 buffer、是否支持 alias、是否需要对齐。runner 可以根据属性过滤 workload，避免用不支持的输入测试某个 kernel，或至少把失败记录成明确的 `unsupported`，而不是崩溃。

数据模型要先于代码优化

benchmark suite 最核心的资产是数据。若数据模型设计不好，后面图表、CI、回归、报告都会混乱。一个 raw sample 表应尽量采用追加式、长表结构，而不是为每个 kernel 建一列。长表更适合新增 kernel、size 和指标。

推荐 raw schema：

```
schema_version,run_id,timestamp,kernel,workload,size,bytes,ops,
batch,repetition,elapsed_ns,checksum,status,notes
```

推荐 summary schema：

```
schema_version,run_id,kernel,workload,size,bytes,ops,
samples,min_ns,median_ns,p95_ns,max_ns,
ns_per_op,gbps,gops,checksum_status,notes
```

字段名要稳定，单位要写进字段名或 schema 文档。`elapsed` 不如 `elapsed_ns`；`speed` 不如 `gbps` 或 `items_per_s`；`size` 要说明是元素数还是字节数。派生指标公式应写进文档。否则几个月后，没人知道 GB/s 是只算读，还是读写合计。

schema 变化要版本化

一旦 raw 或 summary 字段变化，就应增加 `schema_version`，并在报告中标注。比如把 `bytes` 从读字节改成读写总字节，会直接改变 GB/s。若不版本化，历史趋势会出现假提升。schema 版本化不是大型系统专属，小 benchmark 也需要。

版本化还帮助脚本兼容旧结果。summarizer 可以拒绝未知 schema，或按版本选择解析逻辑。比起默默读错列，显式失败更好。性能数据一旦被错误解析，后续报告和决策都会被污染。

status 字段要表达失败，而不是丢弃失败

raw sample 中应有 `status`。可能值包括 `ok`、`checksum_failed`、`unsupported`、`timeout`、`setup_failed`、`measurement_failed`。不要把失败样本直接不写。失败频率本身就是重要信息。

例如某个 SIMD kernel 只在非 32 倍长度时 checksum 失败，如果你只保存成功样本，summary 会看起来很漂亮；若保留失败 `status`，问题会立刻暴露。CI 也可以基于失败类型做不同处理：`correctness` 失败直接阻断，`unsupported` 可以跳过，`measurement_failed` 需要重跑或检查环境。失败类型要和处理策略绑定，不能只留下汇总数字。

正确性检查要避免污染计时

正确性检查必须存在，但不应混进热路径，除非实验问题就是端到端成本。常见做法是：每个 workload 先用 reference 计算期望结果；每个 candidate 在 warmup 或正式计时前后做校验；计时区域内只执行 kernel 和必要的防优化观察；输出 buffer 的完整比较放在计时外。

对于纯函数式标量返回，可以在每个 batch 后把返回值合并到 checksum，并在计时后检查 checksum 是否与预期模式一致。对于写 buffer 的 kernel，可以在每个样本后抽样检查，或每个 size/repetition 后完整检查，取决于成本。若完整检查太贵，也不能完全取消；可以在 debug 或 nightly 模式全量检查，在 PR smoke 模式抽样检查。

reference 也要定义语义

reference 不是随便写一个“看起来对”的版本。它必须定义边界语义：整数是否允许回绕，浮点误差容忍多少，NaN 如何比较，输入重叠是否允许，输出初值是否参与计算。candidate 与 reference 不一致时，首先要判断 reference 是否真的表达了接口合同。

例如整数点积若使用 64 位累加，reference 应明确每次乘法和累加的宽度。若候选汇编使用 32 位中间结果，会在大值输入下失败。浮点 dot product 则要定义误差模型：逐元素顺序求和、树形求和、FMA 与非 FMA 可能产生不同舍入。若接口允许这些差异，checker 应使用合理容差；若接口要求 bit-exact，就不能随便改求和顺序。

输入生成器要可解释

输入生成不是次要细节。数据分布会影响分支预测、cache 行为、压缩率、特殊值路径、向量化尾部和数值稳定性。一个好 workload factory 应把分布作为显式参数。

常见分布包括：

- 全零，用于测试最佳压缩或特殊值路径。
- 递增，用于测试可预测模式和简单校验。
- 均匀随机，用于减少特殊模式。
- 固定稀疏度，用于测试分支和掩码。
- 对抗分布，用于制造难预测分支或极端数值。
- 真实样本，用于贴近生产 workload。

每种分布都要记录 seed 和生成算法。固定 seed 不是为了让数据“随机”，而是为了让别人复现同一数据。若生成算法变了，即使 seed 相同，输入也变了。报告中应把分布和生成版本当成 workload 的一部分。

真实数据也需要摘要

真实生产数据很有价值，但通常不能直接公开，也可能太大。使用真实数据时，至少记录摘要：样本来源、日期、大小、值域、稀疏度、长度分布、是否脱敏、是否抽样。否则别人无法判断它代表什么。若不能发布数据本身，可以发布生成相似分布的 synthetic workload，并说明与真实数据的差异。

派生指标要绑定 workload

同一个 elapsed time 可以派生出很多指标：ns/op、GB/s、items/s、GFLOP/s、cycles/element。指标不是越多越好。每个指标都要绑定 workload 的逻辑。

对于 `sum_u64`，若每个元素读取 8 字节、不写输出数组，可以定义：

```
bytes_read = n * 8
GB/s = bytes_read / seconds / 1e9
ns/element = elapsed_ns / n
```

对于 `copy_u64`，如果读一个数组写一个数组，可以选择读写合计：

```
bytes_touched = n * 8 * 2
GB/s = bytes_touched / seconds / 1e9
```

但必须写明这是读写合计。若另一个报告只算写字节，两个 GB/s 不能直接比较。对于 `fill_u64`，只有写字节；对于 `dot_f32`，既有 bytes，也有 FLOPs。不同指标回答不同问题：GB/s 说明内存吞吐，GFLOP/s 说明浮点运算吞吐，ns/element 更直观地比较端到端成本。

speedup 要说明方向

speedup 常见公式是：

```
speedup = baseline_time / candidate_time
```

大于 1 表示 candidate 更快。但也有人用 `candidate/baseline` 表示 slowdown。报告必须写公式，不要只写“speedup 1.2”。若使用百分比改善，也要说明：

```
improvement = (baseline_time - candidate_time) / baseline_time
```

时间降低 20% 和吞吐提高 25% 是同一变化的两种表达，不能混用。性能报告里很多沟通错误来自百分比方向不清。

回归判断要结合统计和工程意义

一个版本慢了 1%，统计上可能可测，但工程上不重要；一个版本慢了 15%，样本波动也许很大，但仍值得调查。回归判断需要同时看统计稳定性和工程意义。

可以先用简单规则：

- 若 correctness 失败，性能结论作废。
- 若样本数不足，标记为观察，不做回归判断。
- 若 median 变化小于历史自然波动，记录趋势，不阻断。
- 若 median 变化超过宽阈值，触发警报。
- 若多个相邻规模同向退化，优先级高于单个点。
- 若关键生产规模退化，优先级高于教学规模退化。

后续可以引入更正式的统计检验或置信区间，但不要一开始就用复杂统计掩盖实验设计问题。统计检验无法修复错误计时区域、错误编译参数或错误 correctness。

历史基线要冻结

比较性能时，baseline 必须可追溯。可以使用上一版本 release、某个 commit、某个固定二进制，或当前源码中的 reference kernel。无论哪种，都要记录。如果 baseline 随着 candidate 一起变化，speedup 就失去意义。若 baseline 修 bug 或改协议，应在趋势图中标注，不要让历史比较混在一起。

报告生成可以自动化，解释不能自动化

benchmark suite 可以自动生成 summary、图表、环境摘要和初步警报。但最终解释仍需要工程判断。自动工具可以告诉你 `avx2_unrolled4` 在 `n=32768` 上 median 下降 10%，反汇编包含 `vpaddq`, `perf` instructions 下降；但它不能独立证明这个优化适合生产，也不能知道某个输入分布是否代表真实用户。

因此，自动报告应输出“事实块”，人工报告负责“解释块”。事实块包括命令、环境、raw、summary、二进制审计、PMU。解释块包括问题、结论、机制假设、限制和下一步。把两者分开，可以避免工具生成听起来确定但证据不足的文字。

报告中的否定结果也要保留

优化没有变快，也是一种结果。如果报告只保存成功优化，团队会重复尝试已经证明无效的方向。否定结果应写清：在哪些规模没有提升，二进制是否符合预期，可能瓶颈是什么，下一步是否停止或换方向。

例如：

```
Observation:
  Unrolling by 4 does not improve 64 MiB sum_u64 throughput.

Evidence:
  Both versions reach similar GB/s.
  Objdump confirms unrolling is present.
  Large-size throughput is close to copy bandwidth measured separately.

Conclusion:
  For this workload size, loop overhead is unlikely to be the bottleneck.
  Further work should investigate memory bandwidth or data layout, not
  additional scalar unrolling.
```

这种否定结果能节省后续时间。高质量 benchmark suite 不只是证明哪些优化有效，也帮助停止无效方向。

从教学实验过渡到生产测量

本书第一册的 benchmark 目标是教学和基础可信性，不要求一开始就达到大型性能实验平台的复杂度。但教学实验应保留通向生产的接口：显式配置、raw 数据、summary、环境记录、二进制审计、正确性检查。这样学生后来面对真实项目时，不需要推翻方法，只需要扩大规模。

教学阶段可以简化：

- 只支持单线程。
- 只使用少量 kernel。
- 只记录基础环境。
- 只使用 median、min、max。
- PMU 作为可选证据。

生产阶段再增加：

- 多线程和 NUMA 绑定。
- 更多输入分布和真实数据。
- 历史趋势数据库。
- 自动图表和警报。
- 受控性能机器。
- 更严格统计和发布流程。

关键是两阶段理念一致：先定义问题，再设计 workload，先保证正确，再计时，先保存 raw，再做 summary，先审计二进制，再解释机制。规模可以变，原则不能变。

从零实现一个小型 suite 的步骤

如果你要从空目录开始实现本章的 benchmark suite，可以按七步推进。

第一步，只实现一个 reference kernel 和一个 candidate kernel。不要一开始就写复杂 registry。先让 `sum_u64_ref` 和 `sum_u64_candidate` 在同一输入上返回相同结果，并把结果打印出来。正确性先于性能。

第二步，加入命令行配置。至少支持 `--size`、`--repetitions`、`--warmup`、`--seed`、`--output`。把实际配置写入 `command.txt` 或 `config.json`。从这一步开始，实验就不再依赖记忆。

第三步，加入 raw CSV。每次 repetition 写一行，包含 `kernel`、`size`、`batch`、`elapsed_ns`、`checksum`、`status`。先不要急着画图。原始数据是所有后续分析的基础。

第四步，加入 summary。读取 raw CSV，计算 `min`、`median`、`p95`、`max`、`ns/element`、`GB/s`。summary 可以由同一个程序生成，也可以由脚本生成，但公式必须固定。

第五步，加入环境记录。保存编译器版本、CPU、OS、Git 状态、构建命令。此时你的结果目录已经具备基本复现能力。

第六步，加入二进制审计。自动或手动保存关键函数的 `objdump`。报告中说明函数是否被内联、是否存在目标循环、是否出现意外调用。没有二进制证据的性能解释要降级。

第七步，加入多 kernel registry 和多 size 扫描。只有前六步稳定后，再扩大矩阵。否则，你会同时调试输入生成、计时、CSV、统计和多个 kernel，问题定位很困难。

核心思想

benchmark suite 的实现顺序应服务可诊断性：先 `correctness`，再配置，再 raw，再 summary，再环境，再二进制，最后才扩大矩阵。

结果目录示例

一次完整运行可以生成：

```
results/sum_u64/2026-06-14T120000Z/
  command.txt
  config.json
  environment.txt
  build.txt
  raw_samples.csv
```

```
summary.csv
objdump/
  bench.objdump.txt
  sum_u64.scalar.txt
  sum_u64.unrolled.txt
perf/
  sum_u64_32768.perf.txt
report.md
notes.md
```

command.txt 保存原始命令；**config.json** 保存解析后的配置；**environment.txt** 保存机器和 Git；**raw_samples.csv** 保存每个样本；**summary.csv** 保存汇总；**objdump/** 保存二进制证据；**perf/** 保存可选硬件证据；**report.md** 写结论；**notes.md** 记录异常和失败。

这个目录可以被压缩、上传 CI artifact、附到 review、作为发布报告证据。不要只把结果打印到终端。终端输出是临时观察，不是工程资产。

回归报告案例

假设 nightly benchmark 显示 **sum_u64** 在 **n=32768** 上退化 12%。不要直接把 PR 打回。先写回归报告：

```
Symptom:
  sum_u64 scalar_candidate median ns/element regressed by 12%
  at n = 32768 in nightly run 2026-06-14.

Evidence:
  raw samples show all repetitions shifted, not a single outlier.
  correctness passed.
  environment matches previous nightly except compiler minor version.
  objdump shows loop changed from unrolled-by-4 to scalar loop.

Hypothesis:
  compiler no longer unrolled the loop under current flags, or source change
  introduced an aliasing constraint.

Next checks:
  compare build flags
  compare source diff around kernel
  compile with previous compiler
  inspect aliasing and inlining changes
```

这个报告把现象、证据、假设和下一步分开。它没有直接说“代码变差”，因为证据显示编译器版本也变了。若后续确认编译器升级导致 unroll 决策变化，可以选择固定编译器、调整源码、接受变化，或更新 baseline。不同结论对应不同行动。

回归不是只看一个点

如果只有 **n=32768** 退化，其他规模不变，可能是 cache、batch、unroll 阈值或统计噪声。如果多个相邻规模都退化，可能是真实实现变化。如果所有 kernel 都退化，可能是环境或机器状态。回归报告应包含邻近规模和其他 kernel 的简短对照。

例如：

```
Neighbor sizes:
  n=2048: +1%
  n=32768: +12%
  n=524288: +10%

Other kernels:
  copy_u64: unchanged
  fill_u64: unchanged
```

这说明退化可能与 **sum_u64** 实现有关，而不是整机变慢。若所有 kernel 都慢，优先检查 governor、后台负载、虚拟机迁移或机器温度。

benchmark 代码本身也要测试

benchmark harness 是代码，也会有 bug。常见 harness bug 包括：

- 单位换算错误，把 ns 当 us。
- bytes 口径错误，GB/s 翻倍或减半。
- batch 为 0。
- repetitions 没按配置执行。
- CSV 列顺序和 header 不一致。
- checksum 没有真正依赖结果。
- reference 和 candidate 使用不同输入。
- summary 只读取最后一个 kernel。

因此，应给 harness 写小测试。比如用一个 sleep kernel 验证 elapsed 大致范围；用固定返回值 kernel 验证 checksum；用人工 raw CSV 验证 median 和 p95；用小输入验证 bytes 和 ops 公式。不要以为 benchmark 用来测别人，就不需要自己被测试。

统计脚本要可复查

如果 summary 由脚本生成，脚本也要进入版本控制。报告中应写脚本版本或 commit。统计脚本改变可能导致历史结果不可比。尤其是 p95 的定义、异常过滤、status 处理和分组字段，必须固定。

长期维护中的基线管理

benchmark suite 会随着项目变化。基线管理决定历史数据是否还能解释。

一种简单策略是保留三类基线：

- reference baseline：最清楚、最可靠的实现，用于 correctness。
- performance baseline：当前发布版本或上一个稳定版本，用于 speedup。
- environment baseline：同一机器上基础 kernel 的长期表现，用于判断机器状态。

三类基线不能混用。reference baseline 可能很慢，但语义清楚；performance baseline 代表工程比较对象；environment baseline 代表机器健康。若 copy 带宽 baseline 当天异常下降，所有内存 kernel 的回归都要谨慎解释。

基线更新要写原因

更新 baseline 时，要写原因：

```
baseline update:
old: release-1.2
new: release-1.3
reason:
  release-1.3 is now production baseline
protocol changes:
  none
```

若同时改变了 protocol，例如 bytes 口径、输入分布、编译器版本，应明确标注。否则趋势图断点会被误认为性能变化。

把 benchmark 连接到代码评审

在代码评审中使用 benchmark，应避免两种极端。一种是完全不看性能数据，只凭直觉评审；另一种是只看 speedup 数字，忽略 correctness、二进制和统计。更好的做法是让 PR 提供小型性能证据包：

- 修改说明：性能相关代码改了什么。
- correctness：reference 或测试是否通过。
- benchmark 命令：如何复跑。
- summary：关键规模的 median 和分布。
- raw data 路径：可以复查。

- objdump 摘要：热路径是否符合预期。
- 结论边界：只声称哪些场景。

reviewer 则重点看：比较是否公平、输入是否代表目标、二进制是否符合解释、差异是否大于噪声、是否有退化规模、是否改变了 ABI 或语义。这样 benchmark 就能服务评审，而不是变成数字装饰。

何时停止优化

benchmark suite 不只告诉你哪里变快，也告诉你何时该停。可以停止某个方向的常见信号：

- 多个实现接近同一内存带宽上限。
- 进一步展开不再降低 ns/element。
- 优化只改善不重要的小规模，重要规模无变化。
- 速度提升小于自然波动。
- 代码复杂度和 ABI 风险明显超过收益。
- 端到端实验显示 microkernel 提升被其他成本淹没。

停止不是失败。停止说明证据已经足够支持资源转移。高质量性能工程不是永远优化同一个函数，而是在正确性、维护成本、风险和收益之间做决策。

综合案例：一个小型 suite 的最终形态

到本章结束，一个教学级 suite 可以长这样：

```
bench/
  CMakeLists.txt
  include/
    benchmark_config.hpp
    benchmark_result.hpp
    do_not_optimize.hpp
  src/
    main.cpp
    config_parse.cpp
    environment.cpp
    recorder.cpp
    summary.cpp
    workloads.cpp
    kernels_sum.cpp
    kernels_copy.cpp
    kernels_fill.cpp
    kernels_dot.cpp
  tests/
    test_summary.cpp
    test_workloads.cpp
    test_kernels.cpp
  results/
```

它不需要很大，但职责清晰。config_parse 负责配置；environment 负责机器和构建记录；recorder 写 raw CSV；summary 计算统计；workloads 生成输入；kernels_* 放被测实现；tests 验证 harness 自己。

一次运行：

```
./bench_core --kernels sum_ref,sum_unrolled \
  --sizes 2048,32768,524288,8388608 \
  --repetitions 31 \
  --warmup 5 \
  --seed 1 \
  --output results/sum-unrolled
```

产物：

```
results/sum-unrolled/
  command.txt
  environment.txt
  raw_samples.csv
```

```
summary.csv
report.md
```

如果需要二进制证据，再保存：

```
objdump -drwC -Mintel ./bench_core > results/sum-unrolled/bench.objdump.txt
```

这已经足够支撑第一册级别的可信实验。后续要加入图表、CI、历史趋势、PMU、真实 workload，都可以在这个基础上扩展。

综合案例：从结果到决策

假设 summary 显示：

```
size,baseline_ns_per_element,candidate_ns_per_element
2048,0.45,0.50
32768,0.32,0.28
524288,0.34,0.30
8388608,0.42,0.42
```

不要只写“candidate 更快”。更准确的解释是：

- 小规模 2048 变慢，可能是额外控制开销超过收益。
- 中等规模 32768 和 524288 变快，candidate 的循环结构可能更适合 cache-resident 数据。
- 大规模 8388608 持平，可能进入内存带宽瓶颈。

决策要看目标 workload。如果生产主要处理中等规模，candidate 值得继续；如果生产主要处理小规模请求，可能需要 size-based dispatch；如果生产主要是大规模 streaming，candidate 的收益有限。benchmark suite 的价值不是给出单一速度标签，而是帮助做条件化决策。

自测清单：benchmark suite 是否可维护

完成本章后，应能回答：

1. 新增一个 kernel 需要改哪些文件，是否会复制计时代码。
2. raw schema 和 summary schema 是否有版本。
3. 每个指标的单位和公式是否写清。
4. workload 的 size、distribution、seed、alignment 是否记录。
5. correctness check 是否独立于 timed region。
6. harness 自己是否有测试。
7. CI 中的性能阈值是否按 kernel 和规模设置。
8. 回归报告是否保存 artifact。
9. baseline 更新是否记录原因。
10. 性能结论是否能连接数据、二进制和环境证据。

如果 suite 只能在作者机器上手工运行、只输出一个平均值、没有 raw、没有环境、没有 correctness，那么它只是临时脚本。临时脚本可以用于探索，但不能承担长期性能决策。

深入讲解：suite 的价值在于长期一致性

一次 benchmark 可以回答一个局部问题；benchmark suite 的价值在于长期一致性。它让团队在不同提交、不同机器、不同优化方向之间使用同一套语言比较性能。没有 suite，每个人都会写自己的计时代码，输入不同、单位不同、统计不同、报告不同，最后数字无法合并。

长期一致性来自四件事。第一，schema 稳定。raw 和 summary 字段不随意改变。第二，workload 稳定。输入规模、分布和 seed 有记录。第三，baseline 稳定。比较对象清楚。第四，报告稳定。结论格式区分观察、解释和限制。只要这四点成立，suite 就能积累历史知识。

长期一致性不等于永远不变。schema 可以升级，workload 可以扩展，baseline 可以更新，报告可以改进。但每次变化都要版本化和记录原因。性能数据最怕无声变化：某个字段公式改了，趋势图却继续连线；某个输入分布改了，speedup 却继续比较。suite 的维护重点，就是让变化可见。

深入讲解：性能数据也是项目资产

源代码、测试、文档是项目资产，性能数据也是。它记录了某些实现为什么被选择，某些优化为什么停止，某些退化从哪天开始，某些机器为什么不适合比较。没有这些数据，团队会重复讨论同样问题。

把性能数据当资产，意味着要考虑存储、命名、版本、权限和清理。raw 数据可能很大，但关键 release 和回归数据应保留；临时探索数据可以定期清理；报告和 summary 应随 PR 或 release 保存；重要图表应能追溯到 raw。性能数据如果只存在个人电脑里，就不能支撑团队决策。

深入讲解：benchmark suite 和单元测试的关系

单元测试和 benchmark suite 都是工程反馈系统，但它们回答不同问题。单元测试主要回答“行为是否正确”，通常要求快速、稳定、确定；benchmark 回答“成本如何变化”，通常更慢、更受环境影响、需要统计解释。两者不能互相替代。

正确的关系是：benchmark 依赖单元测试，单元测试保护 benchmark。被测 kernel 应先通过单元测试和 reference 校验，再进入性能测量。benchmark harness 自己也应有单元测试，验证 CSV、summary、median、bytes 公式和配置解析。这样当性能数字异常时，你能先排除 harness bug。

不要让 benchmark 成为唯一 correctness 检查。benchmark 为了减少计时污染，可能不在每次样本内全量校验；单元测试可以更密集地覆盖边界。也不要让单元测试承担性能结论。单元测试运行时间受测试框架、并行执行和环境影响，不适合当作性能指标。

深入讲解：为什么报告模板比图表更重要

图表很直观，但模板决定图表是否可解释。没有问题陈述，图表不知道回答什么；没有 workload，横轴不知道代表什么；没有单位，纵轴无法比较；没有 raw 数据，图表无法复查；没有二进制证据，趋势无法解释；没有限制，读者会过度推广。

报告模板的作用，是强迫作者在图表之前写清上下文。先写问题和结论，再放图表；先写协议，再放 summary；先写证据，再写解释。这样图表服务论证，而不是替代论证。

一个项目可以为不同层级准备不同模板。PR smoke 报告可以短，只列 correctness、关键 summary 和是否超过阈值；nightly 报告可以包含趋势和 artifact；发布报告则要包含完整环境、二进制和限制。模板层级不同，但核心字段一致。

深入讲解：CI 中的 benchmark 要分层

把完整 benchmark suite 直接塞进每个 PR，通常不是好设计。PR 需要快速反馈，完整性能实验需要稳定环境、较多样本和人工解释。把两者混在一起，要么 CI 太慢，要么阈值太粗糙，最后大家开始忽略报警。更合理的做法是分层：PR smoke、nightly trend、release benchmark。

PR smoke 的目标是发现明显问题。它可以运行少量 kernel、少量 size、较少 repetition，重点检查 correctness、harness 是否能跑、性能是否出现巨大退化。它的结论应该保守：通过不代表性能一定好，失败也可能需要复跑确认。PR smoke 适合阻断 checksum 失败、崩溃、CSV 格式错误和非常大的退化，不适合阻断百分之一这类小差异。

nightly trend 的目标是观察趋势。它可以在固定机器或较稳定环境中运行较完整矩阵，保存 raw、summary、环境和图表。nightly 的价值不在于某一天的单点，而在于多天趋势。某个版本后连续几天退化，比单次样本波动更值得调查。nightly 也适合发现机器状态变化，例如内核升级、频率策略改变或后台负载污染。

release benchmark 的目标是支持发布决策。它应该使用明确冻结的源码、构建参数、输入矩阵和机器环境，样本数量更充分，报告更完整。发布报告不仅要说明快慢，还要说明是否存在 correctness 风险、二进制变化、适用 workload、不能推广的范围。release benchmark 不是自动红绿灯，而是工程决策材料。

性能回归从报警到结论

一次 CI 报警只是信号，不是结论。处理性能回归可以按固定流程。第一，确认报警是否可复现。复跑同一提交、同一机器、同一参数，查看 raw samples 是否仍然异常。第二，确认 correctness 和 harness。checksum、样本数、schema、batch、计时边界、环境记录都要先通过。第三，确认比较对

象。baseline 提交是否正确，candidate 是否包含预期改动，构建参数是否相同。

第四，缩小 workload。找出哪些 kernel、size、distribution 退化，哪些不退化。若只有小 size 退化，可能是固定开销；若只有大 size 退化，可能是带宽或工作集；若只有随机分布退化，可能是分支或内存局部性；若所有 workload 都退化，可能是构建、环境或公共路径。第五，查看二进制差异。关键循环是否多了分支、函数调用、溢出检查、别名检查或额外 load/store。

第六，再引入硬件事件。先有 workload 和二进制差异，再用 PMU 支持假设。若 branch misses 增加，要能指出哪条分支变得难预测；若 cache misses 增加，要能指出访问模式或工作集变化；若 instructions 增加，要能指出新增代码路径。第七，形成结论等级。若证据不足，就写“当前只能确认某 workload 退化，原因未定”；不要为了关闭问题强行写故事。

回归报告的最小字段

回归报告至少应包含：报警来源、提交范围、复跑结果、受影响 workload、未受影响 workload、正确性状态、构建参数、关键二进制差异、raw data 路径、summary 表、可能原因、排除原因、下一步。若修复已经存在，还要包含修复前后同一协议下的对比。

这份字段清单看起来比“某 PR 变慢”麻烦，但它能让团队节省讨论成本。性能回归最怕信息碎片化：CI 日志在一个系统，图表在另一个系统，反汇编在个人终端，结论在聊天记录里。报告把证据集中在一起，后面无论是回滚、修复、接受退化还是调整阈值，都有依据。

综合案例：从局部优化到保守合入

设想一个改动把 `sum_u64` 的循环手写展开四倍。作者本机单次运行显示快百分之八。按照本章标准，这还不能直接合入。第一步，确认候选实现和 reference 在空数组、一个元素、非四倍长度、大数组和随机数据上结果一致。第二步，确认编译器没有已经自动展开或向量化出等价代码。若 baseline 本来就很好，手写展开可能只增加代码大小。

第三步，做规模扫描。小规模可能因为额外尾部处理变慢，中等规模可能因为减少分支开销变快，大规模可能完全受内存带宽限制而持平。第四步，交错运行。不要先跑完 baseline 再跑 candidate，因为温度、频率和系统负载会随时间变化。第五步，保存反汇编。若 candidate 多了寄存器压力或溢出到栈，某些输入下变慢就不奇怪。

最后的决策可能不是简单合入或拒绝。若生产 workload 主要是中等规模连续数组，可以保守合入并保留阈值监控；若小规模请求很多，可以做 size-based dispatch；若大规模没有收益且代码变复杂，可以拒绝；若证据不稳定，可以要求在固定机器 nightly 中观察一周。高质量 benchmark suite 的作用，是让这些选择有证据，而不是让最快的一次运行赢。

停止优化也是一种结论

性能工程常见误区是永远追逐更小数字。一个优化是否继续，取决于收益、风险、复杂度和可维护性。若某个改动只在少数不重要 workload 上提升百分之一，却增加大量手写汇编、平台分支和测试成本，它可能不值得。若某个改动带来稳定百分之五收益，但破坏 ABI 或让错误路径难以审查，也需要谨慎。

停止优化不等于失败。它可以是基于证据的结论：当前瓶颈已经转移到内存带宽；当前实现足够接近 baseline；进一步优化需要第二册涉及的 SIMD 或 cache blocking；当前 workload 不支持更复杂分派；当前测量噪声大于预期收益。把这些原因写进报告，后续读者就知道为什么没有继续做。

一个成熟 benchmark suite 应帮助团队回答“是否值得继续”。如果每次报告只说谁快谁慢，它还不够成熟。它应该告诉你收益在哪些条件下出现，风险是什么，复杂度是否合理，下一步需要什么证据，以及停止点在哪里。

数据治理：raw、summary 和图表各司其职

benchmark suite 的输出至少有三层。raw 是原始样本，记录每次运行的事实；summary 是对 raw 的聚合，方便比较；图表是对 summary 或 raw 的可视化，帮助发现趋势。三者不能互相替代。只有图表没有 raw，无法复查；只有 summary 没有 raw，无法检查异常值；只有 raw 没有 summary，团队难以快速判断。

raw 数据应尽量不可变。一次 run 完成后，不要手工修改 raw CSV。如果发现字段错了，应生成新的 run，并在报告中说明旧 run 无效。summary 可以重新生成，但要记录脚本版本和 schema。

图表更是派生产物，应能从 raw 和 summary 重建。这样做可以防止“为了好看改数据”的风险，也方便后续审计。

命名也很重要。结果目录应包含日期、run id、提交号或短哈希、机器名和实验名。文件名应表达内容，而不是 `result1.csv`、`new.csv`。当项目积累几百次实验后，命名规则会决定数据是否还能被使用。

benchmark harness 自己也需要测试

很多团队只测试被测 kernel，不测试 harness。结果是 CSV 写错列、单位换算错、median 算错、batch 除法错、命令行参数没生效、随机 seed 没记录、summary 读到旧文件。harness bug 会污染所有性能结论，因此它也需要测试。

harness 测试可以很小。用固定样本数组测试 median、min、p95 和单位换算；用假 kernel 测试 batch 计数；用故意失败的 checker 测试 status；用临时目录测试 raw 和 summary 文件生成；用固定 seed 测试 workload factory 是否可复现；用参数解析测试默认值和错误输入。测试这些内容不需要长时间 benchmark，却能显著提高可信度。

还可以加入 golden schema 测试。保存一个小 raw CSV 和期望 summary，修改 summarizer 后自动比较。若 schema 升级，测试也跟着升级并记录原因。数据系统一旦自动化，没有 harness 测试就很难发现静默错误。

团队协作中的 benchmark 评审

benchmark 结果进入代码评审时，reviewer 不应只问“快了多少”。更好的问题是：比较对象是否公平；输入矩阵是否覆盖目标 workload；正确性是否通过；二进制是否符合预期；样本是否稳定；结果是否可复现；复杂度是否值得；是否需要更新 baseline；是否影响其他平台。

作者也不应只贴截图。PR 中应链接结果目录或 artifact，列出关键 summary，说明 raw data 路径，给出反汇编证据，写清限制。若结果不稳定，应主动说明，不要只挑最好一次。若某些 workload 变慢，也要列出来。性能评审的目标是做决策，不是证明作者永远正确。

团队还应约定 baseline 更新流程。什么时候允许更新 baseline，谁批准，是否需要发布报告，旧 baseline 是否保留，趋势图如何断点标注。没有流程时，baseline 容易被悄悄移动，性能回归被吸收到历史数据。baseline 是比较标准，不能随意改。

真实 workload 和合成 workload 的关系

合成 workload 容易复现、容易控制变量，适合定位机制。真实 workload 更接近生产，但可能难以公开、难以稳定、包含太多变量。成熟 suite 往往同时包含两者。合成 workload 用来解释原因，真实 workload 用来验证价值。

使用真实 workload 时，要记录摘要。输入来源、日期、规模、分布、脱敏方式、采样策略、是否包含异常值，都影响解释。若不能公开真实数据，可以构造匹配分布的 synthetic 数据，并说明差异。不要把“真实数据”当作自动可信的标签。真实数据也可能偏斜、过期、不代表未来。

合成 workload 也不能太玩具化。若真实系统的长度分布有长尾，合成数据只测固定长度就会误导；若真实数据有大量空输入、错误输入或非对齐缓冲，合成数据全是整齐数组就不够。workload 设计本质上是建模，模型要说明保留了什么，忽略了什么。

最终报告：从数据到工程选择

第一册最后一个能力，是把 benchmark suite 的数据转化为工程选择。工程选择通常不是“最快版本获胜”。还要考虑正确性风险、维护成本、代码大小、平台覆盖、ABI 稳定、调试难度、能耗、发布周期和团队经验。benchmark 提供证据，但决策需要综合权衡。

一个最终报告可以按四段写。第一段给结论：是否建议合入、回滚、继续实验或停止优化。第二段给证据：summary、raw、反汇编、环境和正确性。第三段给风险：哪些 workload 变慢，哪些平台未测，哪些实现更复杂。第四段给后续：需要的测试、监控、baseline 更新或第二册级别优化方向。

这样的报告让性能工程从“数字竞赛”回到项目目标。优化的意义不是让某个 microbenchmark 最好看，而是在明确条件下改善真实目标，同时不破坏正确性和可维护性。

第一册收束：从一次实验到长期体系

第 15 章教你把一次实验做可信，第 16 章教你把可信实验变成长期体系。长期体系的核心不是代码量，而是一致性：一致的输入描述，一致的计时协议，一致的正确性检查，一致的 raw schema，一致的报告格式，一致的 baseline 管理。只有一致，历史数据才有意义。

当读者进入第二册的 SIMD、HPC 和 AI 算子时，benchmark suite 会变得更重要。高级优化往往有多个实现、多个 ISA、多个输入形状、多个精度和多个平台。没有 suite，就无法判断一个优化是否只对某个玩具输入有效；没有 raw 和 artifact，就无法追踪性能回归；没有 correctness，就无法比较近似算法和数值误差。

因此，本章不是附属工具章，而是第一册的收束。它把源码、汇编、ABI、内存和硬件直觉变成可重复实验。读者若能维护一个小而可信的 suite，就具备了继续深入高性能计算的基本工程能力。

benchmark suite 的毕业要求

一个第一册级别的毕业 suite 不需要很大，但要完整。至少包含两个正确性 reference、三个输入规模、两种数据分布、raw 和 summary CSV、环境记录、反汇编 artifact、一个失败样本处理路径、一个 CI smoke 入口和一份报告模板。它应能在另一台同类机器上运行，并能说明哪些结果可比较、哪些不可比较。

毕业要求还包括维护动作：新增 kernel 不复制计时代码，schema 变化有版本，baseline 更新有说明，异常结果有排查流程，报告能链接到 raw data。做到这些，suite 就不再是脚本，而是项目资产。

面向第二册的 suite 扩展方向

第二册会遇到更多变量：SIMD 宽度、ISA dispatch、数据对齐、尾部 mask、cache blocking、线程数、NUMA、真实 AI shape、量化格式和融合算子。第一册的 suite 不需要现在实现这些，但数据模型要能扩展。kernel 名称、实现名、ISA、input shape、dtype、layout、threads、alignment、cache mode 都应有明确字段或扩展位置。

扩展时不要一次性把 suite 做成庞然大物。先保持核心协议稳定，再逐步增加 workload 和指标。每增加一个维度，都要问它服务什么问题，是否能复现，是否有正确性检查，是否会让报告难以解释。高性能 suite 的复杂性应来自真实需求，而不是形式上的完整。

第一册最终交付物

读者完成第一册后，建议提交一个小仓库作为最终交付物。仓库包含一个 C++ reference、一个汇编 kernel、一个 wrapper、一个 ABI 文档、一个 correctness 测试、一个 benchmark runner、raw 和 summary 示例、反汇编 artifact、环境记录和最终报告。这个仓库可以很小，但要闭环。

最终报告回答四个问题：代码做什么，二进制如何证明，结果是否正确，性能结论在什么条件下成立。若能回答这四个问题，读者已经具备进入第二册的基本工程能力。后续无论做 SIMD、GEMM、attention 还是系统 profiling，都可以在这个闭环上继续扩展。

最终综合项目：mini perf lab

可以把第一册最后的成果命名为 mini perf lab。它不追求大而全，只追求闭环。项目中有三个 kernel：求和、拷贝、带条件求和。每个 kernel 有 reference 和 candidate；每个 candidate 有 correctness；每个实验保存 raw、summary、环境和反汇编；每份报告说明结论和限制。这个小项目足以覆盖数组访问、整数范围、控制流、ABI、C++ wrapper 和测量协议。

mini perf lab 的目录可以固定为：

```
src/
tests/
bench/
results/
docs/abi.md
docs/report-template.md
```

目录本身不是重点，重点是责任清楚。src 放实现，tests 放正确性，bench 放测量，results 放不可变结果，docs 放合同和报告模板。这样的结构能让读者把第一册知识真正变成工程资产。

第一册结束时应避免的误解

第一，达到 36 万有效单位不代表质量自动成立；质量来自每段内容是否解释清楚、是否能实验验证、是否有边界。第二，会写一个 benchmark 不代表会做性能工程；还要会维护数据、处理失败、解释限制。第三，会读汇编不代表可以跳过语言语义；汇编解释必须回到 C++ 合同和 ABI。第四，第一册没有深入 SIMD 和 AI，不是因为它们不重要，而是因为没有这些基础，高级优化会变成不可靠技巧。

带着这些边界进入第二册，学习会更稳。读者已经知道如何确认对象、如何读二进制、如何检查 ABI、如何设计实验、如何写报告。后续主题会更难，但方法不变：定义问题，收集证据，建立模型，验证假设，说明限制。

suite 维护者的责任

benchmark suite 一旦成为项目资产，就需要有人维护。维护者不一定每天写新 benchmark，但要保护 schema、baseline、artifact、报告模板和 CI 入口不被随意破坏。新增 kernel 时，维护者要检查 correctness 是否接入；新增指标时，要检查单位和公式；更新 baseline 时，要要求原因；删除旧结果时，要确认是否仍有追溯价值。

维护者还要防止 suite 膨胀。每个 workload、每个指标、每个图表都应服务问题。没有人解释的图表会变成噪声；无人复查的 raw 会变成垃圾；过慢的 CI 会让大家绕流程。好的 suite 是克制而稳定的，不是把所有想法都塞进去。

从第一册到第二册的实验迁移

进入第二册后，实验对象会从标量 kernel 扩展到 SIMD、cache blocking、线程和 AI 算子，但第一册的报告结构仍然适用。问题陈述、正确性、输入矩阵、计时协议、二进制证据、环境和限制，一个都不能少。新增的只是更多字段：ISA、向量宽度、数据布局、线程数、tile size、dtype、误差模型。

如果第一册的 mini perf lab 设计得好，第二册可以自然扩展它，而不是重写实验系统。比如把 `sum_u64` 扩展为 AVX2 版本，把 `dot_f32` 扩展为向量版本，把输入矩阵扩展到对齐和尾部，把报告模板扩展到 ISA dispatch。这样学习路径连续，工程资产也连续。

第一册最终检查表

在进入第二册前，可以用下面检查表确认自己是否准备好。第一，能从零构建并运行一个小项目，保存构建命令和二进制身份。第二，能读一个优化构建的核心循环汇编，区分事实和推断。第三，能解释至少一个结构体布局和数组地址公式。第四，能为一个 C++ 调汇编函数写 ABI 合同和测试。第五，能设计一个不把 setup 混进 timed region 的 benchmark。第六，能保存 raw、summary、环境和反汇编 artifact。第七，能写出结论限制，不把单机结果推广成普遍规律。

若某项做不到，就回到对应章节补实验。第一册不是读完就结束，而是形成一套可反复使用的工作台。第二册的所有高级主题，都应该放在这个工作台上完成。

给未来扩写的边界

后续若继续扩写本书，应保持第一册定位：基础链路、汇编接口、可信测量。更深的 SIMD、HPC、AI 算子、PMU topdown、多线程 NUMA、生产级库设计，可以进入第二册。这样分册边界清楚，第一册不会被高级专题冲散，第二册也能建立在足够扎实的读者能力上。

第一册的每次新增内容，都应问两个问题：它是否帮助读者打通源码到机器执行的基础链路；它是否能通过实验和报告验证。若答案是否定的，就应推迟到第二册或附录。质量来自边界清楚，而不是把所有主题放在同一本书里。

实验：设计输入规模扫描

目标：把单点 benchmark 改成输入规模扫描。

基于上一章的 `sum_u64` benchmark，支持下面命令：

```
./bench_sum_sizes --sizes 128,2048,32768,524288,8388608 \
--warmup 5 \
```

```
--repetitions 31 \
--seed 1 \
--output results/sum_sizes
```

任务：

1. 实现 `--sizes` 解析，逗号分隔。
2. 每个 size 单独构造 workload。
3. 每个 size 单独 warmup 和测量。
4. 输出 `raw_samples.csv` 和 `summary.csv`。
5. `summary` 中加入 GBps 和 `ns_per_element`。

交付物：

1. 源码。
2. 命令行。
3. raw CSV 前 10 行。
4. summary CSV。
5. 一段趋势解释。

实验：加入 `copy_u64` 和 `fill_u64`

目标：把单 kernel benchmark 扩展成多 kernel suite。

新增：

```
void copy_u64_kernel(std::span<std::uint64_t> dst,
                    std::span<const std::uint64_t> src);

void fill_u64_kernel(std::span<std::uint64_t> dst,
                    std::uint64_t value);
```

任务：

1. 为 `copy_u64` 设计 workload。
2. 为 `fill_u64` 设计 workload。
3. 分别定义 correctness check。
4. CSV 中增加 `kernel` 列。
5. 对每个 kernel 正确计算 logical bytes。
6. 对比 `copy_u64` 和 `std::memcpy`，但要分开报告。

交付物：

1. 扩展后的 suite 源码。
2. 三个 kernel 的 summary。
3. 每个 kernel 的指标定义。
4. `copy_u64` 与 `std::memcpy` 的对比报告。

实验：数据分布对分支的影响

目标：观察输入分布如何改变分支 kernel 性能。

实现：

```
std::uint64_t count_positive(std::span<const std::int32_t> values);
```

生成 4 种输入：

- 全正数。
- 全负数。

- 正负交替。
- 固定 seed 的随机正负。

任务：

1. 每种分布使用同样 size、warmup、repetitions。
2. CSV 中记录 **distribution**。
3. 用 **perfstat** 记录 **branches** 和 **branch-misses**。
4. 反汇编核心循环，说明是否使用分支、**cmov** 或 **setcc**。
5. 解释性能差异。

交付物：

1. 四种分布的结果表。
2. **perfstat** 输出。
3. 核心循环反汇编。
4. 800 字报告：输入分布、预测器和性能之间的关系。

实验：batch 校准

目标：为短 kernel 设计自动 batch，并分析它的副作用。

任务：

1. 实现 **--batchauto**。
2. 设定目标单样本时间，例如 100 微秒。
3. 对每个 size 选择 batch。
4. CSV 记录最终 batch。
5. 比较固定 batch 1 和 auto batch 的结果稳定性。

交付物：

1. batch 校准代码。
2. 每个 size 的 batch 表。
3. auto batch 前后的 min、median、max。
4. 说明 batch 是否改变了 cache 状态。

实验：完整 benchmark 报告目录

目标：生成一个可以提交给别人复现的 benchmark 结果目录。

目录：

```
results/ch16_suite/
  command.txt
  environment.txt
  raw_samples.csv
  summary.csv
  objdump.txt
  perf_stat.txt
  report.md
```

任务：

1. 运行 suite，并保存完整命令到 **command.txt**。
2. 保存 **lscpu**、**uname-a**、编译器版本。
3. 保存 **objdump-drwC-Mintel** 输出。
4. 对至少一个 kernel 运行 **perfstat**。
5. 写 **report.md**，包含目标、方法、结果、解释和限制。

交付物：

1. 完整结果目录。
2. 一份不超过 1500 字但信息完整的报告。
3. 至少 3 条你认为需要进一步实验验证的假设。

实验：建立一个性能回归 smoke test

目标：把 benchmark suite 变成一个低成本回归报警工具，同时避免把小噪声当成失败。

任务：

1. 选择两个 kernel，例如 `sum_u64` 和 `copy_u64`。
2. 每个 kernel 只选择 2 到 3 个代表规模，避免 CI 时间过长。
3. 固定 seed、warmup、repetitions 和 batch。
4. 准备一份 baseline summary，可以来自上一轮稳定运行。
5. 写脚本比较当前 summary 和 baseline summary。
6. 若 median 退化超过 20%，返回失败。
7. 若 median 退化在 5% 到 20% 之间，输出 warning 但不失败。
8. 上传或保存 raw、summary、environment 和 command。

比较逻辑示例：

```
time_ratio = current_median_ns / baseline_median_ns

if time_ratio >= 1.20:
    status = fail
elif time_ratio >= 1.05:
    status = warning
else:
    status = pass
```

交付物：

1. smoke benchmark 的配置。
2. baseline summary。
3. 当前 summary。
4. 比较脚本输出。
5. 一段说明：为什么这个阈值只适合报警，不适合证明小幅优化。

本章作业

习题与作业

作业 1：设计一个 benchmark suite 架构

画出你的 benchmark suite 模块图，至少包含：

- options parsing。
- workload factory。
- kernel runner。
- correctness checker。
- sample collector。
- summary generator。
- CSV writer。
- environment recorder。

要求说明每个模块输入、输出和不应承担的责任。

作业 2：输入规模和 cache 假设

选择你的机器，根据 `lscpu` 或系统信息估计 L1/L2/L3 cache 大小。为 `sum_u64` 设计 8 个输入规模：

1. 至少 2 个明显小于 L1。
2. 至少 2 个接近 L2 或 L3。
3. 至少 2 个明显超过 LLC。
4. 至少 2 个尾部边界规模，例如非 8、16、32 的倍数。

写出每个规模的元素数、字节数、预期瓶颈和验证方法。

作业 3：CSV schema 评审

设计 raw 和 summary 两个 CSV schema。要求：

- 每列都有单位或明确语义。
- 每个影响结果的参数都能记录。
- 能支持多 kernel、多 size、多 distribution、多 batch。
- 能从 raw 重新生成 summary。

提交 schema 和 5 行示例数据，并解释为什么这些列足够复现实验。

作业 4：多 kernel suite 实现

实现一个 C++20 benchmark suite，至少包含：

- `sum_u64`。
- `copy_u64`。
- `fill_u64`。
- `dot_f32`。

要求：

1. 命令行可配置 size、warmup、repetitions、batch、seed。
2. 每个 kernel 都有 correctness check。
3. 输出 raw 和 summary CSV。
4. 不把 allocation、random generation、CSV writing 放进 timed region。
5. 提供至少一个 `objdump` 核心循环分析。

作业 5：Benchmark 报告

用你的 suite 做一次完整实验，提交报告：

- 环境。
- 编译命令。
- workload matrix。
- raw data 路径。
- summary 表。
- 至少 2 个 kernel 的趋势解释。
- 至少 1 个 PMU 证据。
- 至少 1 个反汇编证据。
- 实验限制。

作业 6：异常结果排查报告

构造或寻找一次异常 benchmark 结果，例如某个 size 突然慢很多、checksum 不稳定、CSV 样本数不对、某个版本快得不合理。按本章失败定位 checklist 写排查报告：

1. raw samples 是否完整。
2. correctness 是否稳定。

3. timed region 是否干净。
4. 反汇编是否符合预期。
5. 环境是否可能解释异常。
6. 最终判断：这是 harness bug、代码 bug、环境噪声，还是可能的真实性能现象。

要求报告包含证据，不接受只写猜测。

作业 7：性能回归策略设计

为你的 benchmark suite 设计一套 CI smoke benchmark 策略。要求：

- 选择哪些 kernel 进入 PR smoke。
- 选择哪些输入规模。
- 设定 fail 阈值和 warning 阈值。
- 说明哪些 artifact 必须保存。
- 说明哪些结论不能由 CI smoke 得出。

重点是让策略符合现实环境。不要写“任何 1% 退化都失败”，除非你能说明机器、频率、重复次数和历史噪声如何支持这个阈值。

评分标准

本章作业按下面标准评价：

1. benchmark 架构是否职责清晰。
2. 输入规模是否覆盖不同性能区域。
3. 数据分布、batch、cache 状态是否记录。
4. timed region 是否干净。
5. CSV 是否能支持复现和后续统计。
6. correctness check 是否独立可信。
7. 报告是否用数据、汇编和硬件事件支撑结论。
8. 是否能从异常结果反推数据、语义、计时边界、机器码和环境问题。
9. CI 回归策略是否区分报警、趋势和最终性能结论。

本章小结

本章把可信测量原则工程化成 benchmark suite 设计：

```
benchmark suite:
  options
  workload factory
  kernel runner
  correctness checker
  sample collector
  summary generator
  CSV writer
  environment recorder
```

```
input matrix:
  size
  distribution
  alignment
  cache mode
  batch
```

```
outputs:
  raw_samples.csv
  summary.csv
  environment.txt
  command.txt
  objdump.txt
  perf_stat.txt
  report.md
```

```
quality loop:
  raw data audit
  correctness audit
  timed-region audit
  binary audit
  environment audit
  regression smoke test
```

到这里，你应该能把一个临时代码片段升级成可复现的实验系统。统计与噪声分析会在后续性能专题中继续补齐：如何理解样本分布、异常值、均值和中位数，如何比较两个版本是否真的不同，以及如何把实验复现做得更严谨。第一册到这里收束，因为读者已经具备继续进入现代 CPU、SIMD、HPC 和 AI 算子优化所需的基本实验能力。

x86-64 速查表

通用寄存器

寄存器	常见用途
rax/eax	返回值、通用计算
rdi/edi	System V 第 1 个整数/指针参数
rsi/esi	System V 第 2 个整数/指针参数
rdx/edx	System V 第 3 个整数/指针参数
rcx/ecx	System V 第 4 个整数/指针参数
r8/r8d	System V 第 5 个整数/指针参数
r9/r9d	System V 第 6 个整数/指针参数
rsp	栈指针
rbp	栈帧指针或通用 callee-saved 寄存器

常见指令

指令	简化语义	备注
movdst,src	dst=src	复制位模式
leaddst,[addr]	dst=addr	不访问内存
adddst,src	dst+=src	更新 flags
subdst,src	dst-=src	更新 flags
cmpa,b	计算 a-b 更新 flags	不保存结果
testa,b	计算 a&b 更新 flags	常用于判零
jmplabel	无条件跳转	改变 rip
callf	调用函数	压入返回地址
ret	返回	弹出返回地址

附录 B

Linux 与二进制工具命令速查

编译与反汇编

```
g++ -E main.cpp -o main.i
g++ -S -O3 -masm=intel main.cpp -o main.s
g++ -c -O3 main.cpp -o main.o
g++ main.o -o app
objdump -drwC -Mintel main.o
```

ELF 和符号

```
file app
readelf -h app
readelf -S app
readelf -l app
readelf -r app
nm -C app
```

调试

```
g++ -O0 -g main.cpp -o app
gdb -q ./app
```

```
break main
run
stepi
info registers
x/10i $rip
disassemble /m main
```

附录 C

LaTeX 写作规范

章节规则

每章必须从具体问题开始，不能只列概念。所有术语第一次出现时使用术语宏：

```
\term{中文术语}{English explanation}  
\engterm{English term}
```

代码规则

短代码使用：

```
\code{...}
```

多行代码使用 `lstlisting`。

实验和习题

实验使用 `labbox`，习题使用 `exercisebox`，常见误区使用 `warningbox`。

扩写节奏

先扩写 Part 0 和 Part 1，确保刚学完 C/C++ 的读者能进入。之后再扩写汇编、测量、编译器和微架构。

附录 D

参考资料和延伸阅读

公开课程

- CMU 15-213 / 18-213: Intro to Computer Systems。
- MIT 6.172: Performance Engineering of Software Systems。
- UC Berkeley CS61C: Great Ideas in Computer Architecture。
- Stanford CS149: Parallel Computing。
- Georgia Tech CS 6290: High Performance Computer Architecture。

官方和一手资料

- Intel 64 and IA-32 Architectures Software Developer's Manual。
- Intel 64 and IA-32 Architectures Optimization Reference Manual。
- AMD64 Architecture Programmer's Manual。
- LLVM documentation and `llvm-mca` command guide。
- Intel Intrinsics Guide。

性能数据和生产库

- uops.info。
- Agner Fog optimization resources。
- BLIS。
- OpenBLAS。
- oneDNN。

实验和作业评分标准

正确性

所有优化实现必须有 reference。没有 reference 的性能数字没有意义。

复现性

报告必须记录 CPU、OS、编译器版本、编译参数、输入规模、运行次数和统计口径。

证据链

高质量报告必须包含源码、汇编或 IR、性能数据和解释模型。涉及微架构时，应尽量包含 `llvm-mca`、PMU 或替代证据。

失败实验

每个大作业至少记录一个失败实验。失败实验必须解释假设、结果和下一步推断。

Linux 源码阅读案例

为什么第一册要读 Linux 源码

第一册的主线是从 C/C++ 源码追到机器执行，再追到可信测量。Linux 源码阅读不是额外装饰，而是把这条主线接到真实操作系统实现上。前面章节讲进程、虚拟地址空间、ELF 加载、系统调用、`/proc`、调度噪声和 `perf` 时，如果只停留在概念层，读者仍然会把操作系统当成黑盒；若能读几条窄而清楚的 Linux 源码路径，就能知道用户态程序和内核之间的边界究竟在哪里。

不过，源码阅读必须控制范围。第一册不要求读完整 Linux 内核，不要求理解所有锁、RCU、内存回收、文件系统和调度策略。它只要求读者能围绕一个具体问题，找到相关源码入口，顺着少量关键函数读出事实，并把事实和本书前面的工具证据连起来。

核心思想

Linux 源码阅读的目标不是背函数名，而是训练一条证据链：用户态现象、系统调用或文件接口、内核源码入口、关键数据结构、返回路径、可复现实验。

准备源码和记录版本

阅读内核源码前，必须先固定版本。Linux 内核发展很快，同一个机制在不同版本里函数名、文件位置和调用路径都可能变化。报告里如果只写“Linux 源码中这样实现”，通常是不合格的；更准确的写法是“在 Linux 某个 tag 或某个 commit 中，观察到如下路径”。

推荐准备一个单独源码目录：

```
mkdir -p ~/src
cd ~/src
git clone --depth=1 https://github.com/torvalds/linux.git linux-src
cd linux-src
git rev-parse HEAD
git describe --tags --always
```

如果网络或存储受限，也可以使用发行版提供的 kernel source 包，或者只下载某个 release tarball。无论来源是什么，都要记录：

- 源码来源，是上游 Linux、发行版源码包，还是 Android/common kernel。
- commit hash 或 release tag。
- 本机运行内核版本，可用 `uname-a` 记录。
- 阅读工具和搜索命令。
- 你是否真的编译运行了这个内核。多数第一册实验只读源码，不要求运行自编译内核。

常见误区

不要把“正在运行的内核版本”和“你下载的源码版本”混为一谈。除非你确认二者对应，否则源码只能用来理解机制，不能直接证明当前机器内核的精确实现。

源码阅读工具

第一册阶段优先使用简单工具。复杂 IDE 可以提高效率，但不应替代基本搜索能力。

```
rg -n "SYSCALL_DEFINE.*write" .
rg -n "load_elf_binary" fs arch include
rg -n "do_user_addr_fault|handle_mm_fault" arch mm
rg -n "show_map_vma|proc_pid_maps" fs/proc
git grep -n "struct vm_area_struct"
git grep -n "struct task_struct"
```


读源码时建议同时开三类文件：

- 入口文件：例如系统调用、`/proc` 文件接口、ELF loader。
- 数据结构定义：例如 `task_struct`、`mm_struct`、`vm_area_struct`、`file`。
- 实验记录：保存搜索命令、源码片段位置、调用路径、还没理解的问题。

源码报告不应大量复制内核代码。更好的方式是列出文件、函数、关键字段和自己的解释。代码只能作为少量证据，重点是机制路径。

通用阅读模板

每个 Linux 源码案例都按同一个模板完成。

1. 问题：用户态观察到什么现象。
2. 接口：这个现象通过系统调用、`/proc`、ELF loader、调度器还是 PMU 暴露。
3. 入口：在源码中找到第一处稳定入口。
4. 数据结构：列出路径上最关键的结构体。
5. 路径：画出三到七个关键函数组成的调用链。
6. 边界：说明哪些结论来自源码，哪些来自工具观察，哪些只是推测。
7. 实验：用用户态小程序或命令验证一个可观察结果。
8. 报告：记录版本、命令、路径、源码位置、实验输出和限制。

心智模型

读 Linux 源码时不要从“我要看懂这个子系统”开始，而要从“我要解释一个用户态现象”开始。问题越窄，源码路径越可控，报告越容易复查。

案例一：从 `write` 系统调用读到 VFS

起始问题

用户态程序执行：

```
#include <unistd.h>

int main()
{
    write(1, "hello\n", 6);
}
```

这行代码表面上是写标准输出，但底层发生了系统调用。第一册读这个案例的目标不是掌握整个文件系统，而是回答：

用户态参数怎样进入内核？
内核怎样把 `fd = 1` 解释成一个打开文件？
写入路径为什么可能返回短写或错误码？

源码入口

在常见 Linux 6.x 源码中，可以从下面搜索开始：

```
rg -n "SYSCALL_DEFINE.*write" fs include arch
rg -n "ksys_write|vfs_write" fs include
```

常见阅读路径是：

```
SYSCALL_DEFINE3(write, ...)
-> ksys_write(...)
   -> fdget_pos(...)
   -> vfs_write(...)
   -> file specific write operation
```

具体函数名和中间层会随版本变化，报告必须以本地源码搜索结果为准。

要看的结构体

这个案例至少要认识三类对象：

- **structfile**：内核表示一个打开文件对象。
- **structfd**：帮助管理文件描述符引用。
- **file_operations**：文件对象支持哪些操作，例如写入。

读者不需要展开所有字段，但必须知道 **fd=1** 不是文件本身。它是进程文件描述符表里的索引，内核通过它找到对应的打开文件对象，再调用具体文件或设备的写入实现。

实验任务

写一个小程序，分别向标准输出、普通文件和无效 fd 写入数据。用 **strace** 记录系统调用，再用源码报告解释返回值。

```
clang++ -std=c++20 -O2 write_demo.cpp -o write_demo
strace -o write_demo.strace ./write_demo
rg -n "write\\\\" write_demo.strace
```

报告必须包含：用户态代码、**strace** 输出、内核源码版本、源码入口、关键函数链、一个错误码解释。

案例二：ELF 如何被加载到进程

起始问题

第一章已经讲过：**main** 通常不是进程第一条指令，可执行文件也不只是机器码。本案例把这个结论接到 Linux 源码中，回答：

```
Linux 怎样识别 ELF 文件？
程序头表怎样影响虚拟地址映射？
解释器 PT_INTERP 为什么会引入动态链接器？
```

源码入口

从 ELF loader 搜索：

```
rg -n "load_elf_binary" fs
rg -n "PT_INTERP|PT_LOAD" fs/binfmt_elf.c include
```

常见入口是 **fs/binfmt_elf.c** 中的 **load_elf_binary**。读者应重点观察它如何检查 ELF header、遍历 program header、处理 **PT_LOAD** 和 **PT_INTERP**，以及最终准备用户态入口状态。

配套工具证据

源码阅读必须和工具输出对上：

```
readelf -h ./app
readelf -l ./app
readelf -W -l ./app | rg "LOAD|INTERP|Entry"
```

报告中不要只写“内核加载 ELF”。应说明：**readelf-l** 中的 **LOAD** 段怎样对应虚拟地址范围，**INTERP** 指向哪个动态链接器，entry point 与 **main** 有什么区别。

分别编译静态链接和动态链接的小程序，比较 **readelf-l** 输出中是否存在 **INTERP**。阅读 **load_elf_binary** 中处理解释器的代码路径，解释为什么动态链接程序启动时会先进入动态链接器附近，而不是直接进入 **main**。

案例三：/proc/self/maps 从哪里来

起始问题

第四章要求用 **/proc** 观察进程。本案例追问：**/proc/self/maps** 的每一行到底由谁生成？它为什么能显示虚拟地址区间、权限、偏移和文件路径？

源码入口

可以从下面的名字搜索：

```
rg -n "proc_pid_maps|show_map_vma|maps_open" fs/proc
rg -n "struct vm_area_struct" include mm
```

常见路径会进入 `fs/proc/task_mmu.c`。读者重点看两件事：第一，`/proc` 文件并不是磁盘上的普通文件，而是内核根据当前进程状态动态生成的视图；第二，`maps` 行和 `vm_area_struct` 这样的虚拟内存区域结构有关。

实验任务

写一个程序，分别创建栈变量、堆分配、全局变量、`mmap` 匿名映射和映射文件。运行时打印地址，并保存 `/proc/self/maps`。

```
c++ -std=c++20 -O2 maps_demo.cpp -o maps_demo
./maps_demo > maps_report.txt
```

报告要把至少三个地址归入 `maps` 中的区间，并说明这些区间对应栈、堆、可执行映射、共享库或 `mmap` 区域。然后阅读 `fs/proc/task_mmu.c` 中生成 `maps` 的函数，写出源码路径。

案例四：缺页异常和虚拟内存

起始问题

第三章把内存先看成按字节寻址的巨大数组，但真实程序访问虚拟地址时，可能触发缺页异常。第一册不深入页表和内存回收，但应该能回答：

```
为什么第一次访问一大块新分配内存可能更慢？
用户态访问无效地址为什么会变成 SIGSEGV？
Linux 源码中缺页处理大致从哪里进入？
```

源码入口

x86 平台常见搜索路径：

```
rg -n "do_user_addr_fault|handle_page_fault" arch/x86 mm
rg -n "handle_mm_fault" mm include
```

阅读时只要求建立粗路径：

```
CPU page fault
-> arch/x86 fault handler
    -> do_user_addr_fault(...)
        -> handle_mm_fault(...)
            -> allocate/map/fail depending on VMA and permissions
```

不同版本和配置会有差异，报告要注明。

实验任务

写一个程序分配大数组，分别测量只分配不触摸、逐页写入、第二次逐页写入的时间。用 `getrusage` 或 `perfstat` 观察 `page fault` 相关指标；如果权限受限，就记录工具不可用并保留普通计时结果。

报告要说明：第一次触页和第二次触页的差异，哪些现象能由缺页解释，哪些还可能受 `cache`、`TLB`、调度和分配器影响。

案例五：调度噪声从哪里来

起始问题

第十一章讲 benchmark 样本会受 OS 调度、中断、迁核和后台任务影响。源码阅读不要求掌握完整调度器，但应知道用户态线程并不是独占 CPU。问题可以写成：

为什么同一个 kernel 的单次样本偶尔会慢很多？
线程什么时候可能被切走？
Linux 调度器源码入口在哪里？

源码入口

可以从这些名字开始：

```
rg -n "asm linkage.*schedule|void __sched schedule|try_to_wake_up" kernel/sched
rg -n "struct task_struct" include/linux/kernel
```

第一册只要求读者知道 `task_struct` 表示任务，调度器在合适时机选择下一个可运行任务，benchmark 的墙钟时间可能包含线程未在 CPU 上执行的时间。不要在第一册试图完整解释 CFS 或 EEVDF 的所有细节。

实验任务

写一个短 kernel benchmark，分别在空闲系统、后台 CPU 压力、固定 CPU 亲和性三种情况下运行。保存 raw samples，观察 max、p95 和 median 的变化。源码阅读只要求定位 `schedule` 和 `task_struct`，并解释为什么墙钟时间可能包含等待重新运行的时间。

案例六：perfstat 背后的内核子系统

起始问题

`perfstat` 能报告 cycles、instructions、branches、cache-misses 等事件。第一册不要求掌握 PMU 编程，但要知道这些数字不是普通日志，而是内核 perf event 子系统和硬件计数器共同产生的结果。

源码入口

可从下面路径开始：

```
rg -n "perf_event_open" kernel/include/tools
rg -n "SYSCALL_DEFINE.*perf_event_open" kernel
```

常见入口包括 `perf_event_open` 系统调用以及 `kernel/events/` 下的 perf event 核心代码。用户态 `perf` 工具本身通常在 `tools/perf/` 目录中。

实验任务

对同一个求和 kernel 分别运行普通计时和 `perfstat`。如果系统不允许访问硬件事件，要记录错误信息和 `/proc/sys/kernel/perf_event_paranoid` 的值。报告要说明：哪些指标来自硬件事件，哪些只是派生比值，为什么权限限制会影响证据等级。

源码解读报告模板

每份 Linux 源码解读报告至少包含下面字段：

```
Title:
Question:
User-space experiment:
Kernel source version:
Search commands:
Entry functions:
Key structs:
Call path:
Observed output:
Explanation:
Limitations:
Next questions:
```

其中 **Limitations** 很重要。源码阅读最常见的错误，是把一个版本的一条路径写成所有 Linux 版本的绝对结论，或者把源码路径当成当前机器已经实际执行的证据。若没有运行同版本内核，就应明确写出“这是源码机制阅读，不是当前内核行为证明”。

第一册推荐的源码阅读顺序

建议按下面顺序和正文章节配套：

阅读案例	对应章节	学习目标
<code>write</code> 系统调用	第 1、4、13 章	用户态到内核边界，fd 和返回值。
ELF 加载	第 1、4 章	可执行文件、program header、动态链接器。
<code>/proc/self/maps</code>	第 1、3、4 章	虚拟地址空间和 VMA。
缺页处理	第 2、3、15 章	虚拟内存、首次触页、测量噪声。
调度器入口	第 4、15 章	墙钟时间、调度噪声、样本离群点。
<code>perf_event_open</code>	第 15、16 章	PMU 证据、权限、事件解释边界。

验收标准

完成本附录训练后，读者应能做到：

- 固定 Linux 源码版本并记录 commit 或 tag。
- 用 `rg` 或 `gitgrep` 从函数名、系统调用名、结构体名定位源码入口。
- 围绕一个用户态现象画出三到七个函数组成的内核源码路径。
- 把 `strace`、`readelf`、`/proc`、`perfstat` 等工具输出和源码路径对应起来。
- 说明源码阅读结论的版本边界和证据边界。
- 写出一份不依赖口头解释也能复查的源码解读报告。

术语表

中文	英文	简要解释
翻译单元	translation unit	一个源文件经过预处理后的编译输入。
中间表示	intermediate representation	编译器用于优化和 lowering 的内部表示。
架构状态	architectural state	ISA 规定的软件可见状态，如寄存器、内存、flags。
微架构	microarchitecture	CPU 内部实现 ISA 的方式，如 pipeline、cache、uops。
延迟	latency	操作输入 ready 到输出 ready 的时间。
吞吐	throughput	稳态下单位时间能完成的操作数量。
算术强度	arithmetic intensity	FLOPs 与数据搬运字节数的比值。

索引

- ABI, 6, 336
- alignment, 122
- AoS, 135
- AoSoA, 135

- basic block, 284
- big-endian, 108
- bit, 105
- byte, 105

- ELF, 44
- endianness, 108

- harness, 438

- immediate, 207
- induction variable, 301
- instruction
 - adc, 248, 264, 267, 268
 - add, 4, 13, 65, 69, 70, 80, 88, 89, 101, 200, 202, 207, 210, 217, 226, 227, 238, 242–244, 246–249, 260, 263–267, 274, 282, 289, 384, 405, 406, 424
 - add al, 1, 265
 - add dword ptr [mem], 1, 258
 - add dword ptr [mem], edx, 223
 - add dword ptr [rdi], esi, 247
 - add eax, 1, 265, 307
 - add eax, dword ptr [...], 85
 - add eax, edx, 313
 - add eax, esi, 66, 67, 69, 71, 200, 206, 246, 247, 258
 - add rax, imm32, 266
 - add rax, rbx, 5
 - add reg, 1, 249
 - add rsp, ..., 353
 - add rsp, 24, 368
 - add rsp, 8, 351
 - addss, 210
 - and, 207, 254–256, 265, 278, 282, 289
 - and eax, 7, 251
 - call, 10, 13, 16, 18, 67, 75, 93, 98, 102, 165, 167, 170, 173, 176, 182, 192, 204, 213, 215, 216, 218, 220, 225, 226, 228, 229, 233, 237, 239, 283, 285, 305–308, 311, 312, 318, 323, 333–338, 342, 350, 351, 353, 357–359, 362, 364, 374, 383, 385, 392, 396–399, 402, 408, 409, 413, 417, 425–427, 429, 430
 - call 0x0, 213
 - call 0x401180, 306
 - call helper, 367
 - call memcpy, 274
 - PLT, 215
 - plt, 163
 - call rax, 309
 - call rbx, 308
 - call target, 307
 - cdq, 250, 251, 279
 - cmov, 8, 87, 98, 201, 229, 319, 323, 324, 527
 - cmovb, 258, 278, 282
 - cmovcc, 70, 230, 257, 258, 264, 278, 281, 283, 284, 286, 293–295, 310, 312, 317, 323, 330, 334
 - cmovg, 256
 - cmovl, 258, 278, 282, 295
 - cmovle, 328
 - cmp, 13, 18, 69, 70, 85, 89, 93, 97, 98, 217, 219, 224, 226, 242–244, 256–258, 262–265, 273, 278, 281–283, 288–290, 314, 364
 - cmp 10, eax, 257
 - cmp A, B, 314, 323
 - cmp a, b, 248, 256, 265, 289
 - cmp dst, src, 314
 - cmp eax, 0, 258
 - cmp eax, 10, 257
 - cmp eax, esi, 257
 - cmp edi, 0, 288
 - cmp edi, 4, 303, 317
 - cmp edi, esi, 69, 217, 267, 288, 290
 - cmp rdi, rdx, 297
 - cmp reg, 0, 265
 - cpuid, 442
 - cqo, 250, 279
 - cvtsi2sd, 343
 - cvtss2sd, 343
 - dec, 248, 249, 265
 - div, 250, 251, 262, 279, 281, 282
 - div r/m32, 250
 - div r/m64, 250

- idiv, 84, 250–252, 262, 279, 281, 282
- idiv esi, 251
- idiv r/m32, 250
- idiv r/m64, 250
- imul, 80, 217, 243, 249–251, 263, 265, 268, 271, 277, 282
- inc, 248, 249, 265
- ja, 257, 268, 289–291, 303, 314, 317, 323, 334
- ja .Ldefault, 303
- jae, 271, 290, 314
- jb, 244, 248, 257, 264, 267, 268, 271, 282, 289–291, 314
- jbe, 257, 290, 314
- jc, 248, 257, 314
- jcc, 69, 70, 98, 258, 264, 265, 278, 281, 283, 288, 289, 307, 310, 311, 329, 330, 333, 364
- je, 256–258, 289, 314
- jg, 69, 257, 288, 289, 291, 303, 314
- jge, 79, 271, 289, 314
- jge done, 79
- jl, 85, 89, 226, 244, 257, 264, 267, 271, 282, 288, 289, 291, 312, 314
- jl loop, 206
- jle, 13, 217, 232, 234, 257, 287, 289, 314, 327
- jle .Lelse, 285, 291
- jmp, 182, 204, 225, 227, 229, 232, 283, 305–308, 311, 317, 323, 335, 358, 359, 379, 381
- jmp [table + index*8], 303
- jmp callee, 306
- jmp qword ptr [table + index*8], 304
- jmp rax, 309
- jmp target, 307
- jna, 290, 314
- jnae, 290, 314
- jnb, 290, 314
- jnbe, 290, 314
- jnc, 257, 314
- jne, 256–258, 265, 289, 314
- jne .Lloop, 205, 298
- jng, 289, 314
- jnge, 289, 314
- jnl, 289, 314
- jnle, 289, 314
- jnz, 289, 314
- js, 265, 314
- jz, 289, 314
- lea, 21, 22, 69, 70, 89, 94, 101, 117, 120, 121, 127, 132, 136–138, 141, 148, 200, 201, 216, 217, 222–224, 226, 227, 231–233, 235, 240, 241, 244, 247, 249, 250, 261, 263, 264, 266, 271, 275, 277, 282, 288, 289, 298, 299, 402, 412, 424, 426
- lea eax, [rdi + rdi*4 + 1], 219
- lea eax, [rdi + rsi*4], 226
- lea eax, [rdi+rsi], 69
- lea rdi, [rip + ...], 204
- lea rdx, [rdi + rsi*4], 298
- lfence, 404, 407, 442, 473
- lock, 84, 247, 258
- malloc, 274
- mfence, 404, 407
- mov, 2, 65, 70, 136, 148, 200, 202, 207, 210, 219, 223, 225, 226, 231, 232, 238, 244–246, 271, 282, 288, 384
- mov [rip+0x0], 213
- mov al, 1, 259
- mov dword ptr [rdi], eax, 265
- mov eax, 0, 246
- mov eax, [rdi], 23
- mov eax, dword ptr [0x100c], 118
- mov eax, dword ptr [rdi + 4], 219
- mov eax, dword ptr [rdi], 67, 76, 209
- mov eax, edi, 69, 70, 88, 229, 245, 287
- mov rax, qword ptr [rdi], 209
- mov rbx, ..., 336
- mov reg, 0, 246
- movaps, 180, 351, 376, 377
- movsbl, 340
- movsd, 149
- movsd xmm0, qword ptr [rdi], 210
- movss, 120, 149, 210
- movss xmm0, dword ptr [rdi], 210
- movsx, 210, 242, 243, 252, 253, 260, 263, 268, 270, 273, 274, 277, 281, 282, 340
- movsx eax, byte ptr [p], 280
- movsxd, 25, 148, 252, 253, 260, 268, 271, 272, 277, 282, 299
- movsxd rsi, esi, 103, 253
- movups, 377
- movzbl, 340
- movzx, 120, 148, 210, 227, 235, 242, 243, 252, 253, 259, 260, 263, 266, 268, 270, 273, 274, 277, 281, 282, 340
- movzx eax, al, 257, 266, 279
- movzx eax, byte ptr [p], 280
- movzx eax, byte ptr [rdi], 209
- mul, 251

- neg, 242, 261
- neg edi, 261
- nop, 204
- not, 254, 255
- or, 207, 254–256, 265, 278, 282
- pop, 342, 349, 362, 398, 408
- pop rbx, 395
- push, 349, 351, 352, 362, 393, 398, 400, 408
- push rbp, 342, 362, 363, 400
- push rbx, 351, 367
- rdtsc, 406, 430, 440–442, 473
- rdtscp, 441, 442
- ret, 10, 65, 67, 69, 70, 72, 75, 83, 98, 102, 168, 182, 186, 200, 202, 204, 207, 216, 218, 222, 229, 232, 238, 283, 285, 294, 306–311, 324, 326, 333, 335–337, 342, 358, 360, 362, 364, 368, 408
- sal, 253
- sar, 242, 251, 254, 262, 267, 282
- sar 1, 254
- sbb, 242, 261, 264, 267, 268
- sbb eax, eax, 261, 267
- seta, 293, 333
- setb, 244, 248, 256, 257, 262, 267, 268, 278, 281, 282, 290, 293, 333
- setbe, 257
- setcc, 70, 87, 98, 230, 257–259, 264, 278, 281, 283, 284, 292, 293, 295, 329, 527
- sete, 257, 293
- setg, 69, 293, 333
- setg al, 217, 292
- setl, 244, 256, 257, 262, 264, 267, 278, 282, 290, 293, 333
- setl al, 257
- setle, 257
- setne, 217, 256, 257, 266, 293
- setne al, 266, 279
- seto, 248, 263, 267, 279, 281
- sfence, 407
- shl, 120, 127, 141, 148, 253, 271, 299
- shl rax, 4, 136
- shl rsi, 4, 120
- shr, 242, 254, 262, 267, 282
- shr 1, 254
- shr eax, 3, 251
- sub, 69, 70, 207, 217, 246, 247, 249, 264, 265, 282, 289, 314, 424
- sub edi, 10, 317
- sub reg, 1, 249
- sub reg, reg, 246
- sub rsp, ..., 342, 351–353, 400
- sub rsp, 16, 362, 377
- sub rsp, 24, 377
- sub rsp, 8, 337, 351, 362
- sub rsp, N, 362, 363
- syscall, 357, 366
- test, 70, 79, 97, 98, 217, 224, 234, 244, 254, 256–258, 264, 265, 278, 282, 283, 288, 289, 314
- test a, a, 289
- test A, B, 314
- test a, b, 255, 256, 265
- test dil, 2, 256, 257
- test eax, 7, 258
- test eax, eax, 257, 258
- test edi, edi, 79, 288
- test esi, esi; jle .Ldone, 298
- test mask, value, 265
- test rdi, rdi, 258
- test reg, reg, 258, 265
- ud2, 307, 335
- vfmadd, 444, 459
- vmov, 444
- vpadd, 444
- vpaddq, 468, 515
- xor, 246, 254, 255, 265
- xor al, al, 246, 259
- xor eax, eax, 84, 233, 246, 259, 265, 292
- xor edx, edx, 250
- xor reg, reg, 246, 282
- ISA, 5, 66
- kernel, 438
- little-endian, 108
- loop invariant, 299
- name mangling, 355, 395
- object representation, 107
- padding, 123
- red zone, 352
- reference, 438
- register
 - ah, 259
 - al, 68, 69, 208–210, 217, 257, 259, 265, 266, 281, 292, 341, 354, 359, 365, 366, 368, 378, 389, 405, 421
 - ax, 68, 208, 209, 259, 265, 281, 341, 389
 - bh, 259
 - bl, 208

- bp, 208
bpl, 208
bx, 208
ch, 259
cl, 208, 254, 271
cx, 208
dh, 259
di, 208
dil, 208
dl, 208
dx, 208
eax, 64, 66–68, 71, 74–76, 80–82, 84–86, 88, 89, 92, 94, 97, 101, 103, 118, 120, 170, 200, 206–210, 216, 218, 219, 223, 224, 227, 229, 234, 242, 243, 245–247, 250–252, 257–259, 265–268, 273, 275, 279, 281, 286, 292, 295, 296, 299, 313, 314, 317, 318, 339, 341, 344, 352, 362, 364, 365, 367, 378, 389, 405, 406, 430, 441
ebp, 68, 208
ebx, 68, 208, 318
ecx, 68, 208, 243, 254, 268, 296, 299, 339
edi, 64, 68, 69, 72, 74, 79, 88, 97, 100, 200, 206–208, 216, 219, 229, 245, 259, 262, 265, 269, 290, 308, 313, 318, 339–341, 347, 362, 370
edx, 68, 100, 208, 218, 230, 250, 251, 279, 281, 313, 339, 340, 343, 406, 430, 441
esi, 64, 66–69, 71, 72, 74, 97, 100, 103, 192, 200, 206–208, 210, 218, 222, 230, 232, 247, 259, 262, 269, 296, 308, 339, 340, 343, 362
esp, 68, 208
r10, 68, 208, 357, 366
r10b, 208
r10d, 68, 208
r10w, 208
r11, 68, 208, 226, 348, 357, 366
r11b, 208
r11d, 68, 208
r11w, 208
r12, 68, 208, 348, 360, 364, 368, 371, 376, 385, 409, 428, 429
r12b, 208
r12d, 68, 208
r12w, 208
r13, 68, 208, 428, 429
r13b, 208
r13d, 68, 208
r13w, 208
r14, 68, 208, 428
r14b, 208
r14d, 68, 208
r14w, 208
r15, 5, 68, 208, 348, 360, 364, 368, 371, 385, 409, 428
r15b, 208
r15d, 68, 208
r15w, 208
r8, v, 5, 68, 139, 208, 225, 226, 339, 348, 357, 366, 370, 393
r8b, 208
r8d, 68, 208, 328, 339
r8w, 208
r9, v, 68, 208, 225, 339, 357, 366, 370, 393
r9b, 208
r9d, 68, 208, 339
r9w, 208
rax, 5, 13, 15, 65, 67–69, 74, 88, 101, 118, 136, 170, 185, 188, 206, 208–210, 219, 224–227, 242, 246, 250–252, 259, 261, 265, 266, 273, 275, 279, 281, 316, 318, 321, 339, 344–346, 348, 350, 357, 360, 364, 367, 370, 372, 380, 384, 389, 393, 401, 405, 406, 409
rbp, 5, 68, 208, 212, 215, 216, 342, 348, 352, 363, 364, 371, 380, 385, 393, 400, 409, 413
rbx, 5, 21, 23, 65, 68, 208, 227, 336, 348–352, 359–361, 364, 367, 368, 370–372, 376, 380, 385, 395, 398, 400, 409, 419, 427, 428
rcx, v, 5, 65, 68, 85, 137–139, 208, 225, 226, 296, 339, 340, 343, 348, 350, 357, 360, 366, 370, 384, 388, 393, 406
rdi, v, 5, 23, 29, 65, 67, 68, 75, 76, 78, 81, 85, 89, 103, 116, 118–120, 130–132, 136, 168, 173, 176, 192, 200, 208, 210, 212, 218–220, 222, 224–227, 229, 230, 232, 234, 259, 260, 265, 266, 296, 299, 308, 316, 318, 321, 336, 338–340, 345, 348, 350, 356, 357, 360, 361, 363, 364, 366, 370, 372, 393, 402, 410, 419
rdx, v, 5, 65, 68, 208, 218, 225, 226, 232, 250, 251, 279, 281, 298, 299, 339, 344, 345, 348, 357, 364, 366, 370, 393, 401, 406, 419
rip, v, 5, 10, 13, 55, 65–69, 71–75, 78, 79, 84, 87, 88, 94–97, 101, 102, 157,

- 168–170, 173, 178, 181, 182, 185, 188,
189, 194, 197, 200, 201, 206, 212,
213, 221, 283–285, 287, 293, 296,
304–308, 312, 317, 333, 334, 408, 430
- rsi, v, 5, 65, 67, 68, 78, 81, 85, 89, 116,
118, 120, 132, 208, 210, 212, 218,
219, 224–227, 232, 234, 253, 260, 339,
340, 343, 345, 348, 350, 357, 366,
370, 372, 393, 419
- rsp, v, vii, 5, 7, 8, 10, 13, 16, 18, 20, 21,
65, 67, 68, 72, 74, 75, 97, 100–102,
168, 170, 192, 196, 200, 201, 206,
208, 212, 215, 216, 306–308, 318, 333,
334, 336, 338, 341, 342, 348, 350–353,
359–363, 366–371, 380, 384, 393,
398–400, 408, 409, 413, 415, 426–429
- rsp - 8, 342
- si, 208
- sil, 208, 270
- sp, 208
- spl, 208
- xmm, 338, 343, 346, 347, 365, 380
- xmm0, 116, 120, 225, 338, 339, 342–348,
352, 355, 362–364, 370, 396, 409
- xmm1, 343, 345, 362, 364
- xmm15, 348
- xmm2, 343
- xmm3, 343
- xmm7, 342
- ymm, 343
- SoA, 135
- tail call, 305
- timed region, 438
- two's complement, 109
- word, 105
- workload, 438
- 中间表示, 41
- 吞吐, 80
- 延迟, 80
- 微架构, 66
- 抽象语法树, 41
- 时序逻辑, 65
- 架构状态, 66
- 汇编器, 43
- 组合逻辑, 65
- 进程, 46
- 链接器, 44
- 预处理器, 40